

Senior iOS Interview Prep — Muhammed Shahid Siddiqui

Personalized 4-week senior iOS system. **Full chapters are self-contained in Cursor** — you should not need to leave the repo to understand a topic.

Profile anchors: BookMyShow · District · Raw Engineering · FinTrack / GymFlow

Two tracks

Track	Path	Use when
Full study	weeks/week-XX/day-YY/README.md	First time learning a topic
Revision	revision/weeks/	After full study — timed drills only

How to study a day

1. Open `weeks/week-XX/day-YY/README.md`
2. Read `01-foundations` → `02-deep-dive` → `03-production-bridge`
3. Run / read `code/`
4. In `04-questions.md` : cover the full answer → speak from **Answer points** → compare to **Full spoken answer**
5. Do `05-exercises.md`
6. Drill with the revision twin

Answer timing budgets

Format	Budget
Definition	30–45s
Conceptual deep dive	90–120s
Coding approach	2–3 min
Architecture walkthrough	3–5 min
Mobile system design	45 min
Behavioral STAR	2–3 min

Details: [timing/answer-timing-guide.md](#)

Provenance (important)

- **Verified** = resume-backed ([provenance/README.md](#))
- **How I would apply it** = design judgment, not claimed as shipped

Progress tracker

Week 1

- ☐ [01](#) · [02](#) · [03](#) · [04](#) · [05](#) · [06](#) · [07](#)

Week 2

- ☐ [08](#) · [09](#) · [10](#) · [11](#) · [12](#) · [13](#) · [14](#)

Week 3

- [15](#) · [16](#) · [17](#) · [18](#) · [19](#) · [20](#) · [21](#)

Week 4

- [22](#) · [23](#) · [24](#) · [25](#) · [26](#) · [27](#) · [28](#)

Generate PDFs

```
cd roadmap/pdf && ./generate.sh
```

Produces:

- `interview-prep-roadmap.pdf` — full chapters
- `interview-prep-revision.pdf` — revision track

Validate:

```
python3 scripts/validate_roadmap.py
```

Index

Path	Purpose
PHASES.md	Calendar
stories/story-bank.md	STAR stories
provenance/	Verified vs Applied registry
flashcards/	Decks + Anki CSV
coding/dsa-track.md	DSA list
revision/	Short drill notes

4-Week Calendar at a Glance

Target: Senior iOS (ownership, trade-offs, production scale)

Load: ~4–5 hrs weekdays · ~5–6 hrs weekends

System design spine: `ios-system-design/docs/` (linked per day)

Week 1 — Swift mastery, memory, concurrency

Day	File	Focus	Production hook	Parallel SD
1	week-01/day-01	Value/reference, COW, enums, actors intro	Ads type-safety	Social feed scope
2	week-01/day-02	POP, generics, associated types, type erasure	HeroWidget POP+Generics	—
3	week-01/day-03	ARC, retain cycles, Instruments	Crash triage @ BMS	—
4	week-01/day-04	GCD, thread-safe structures	Synchronised dictionaries	—

5	week-01/day-05	async/await, actors, Sendable	Race → actor migration	Feed HLD
6	week-01/day-06	DSA: Arrays/Strings/Two Pointers/Window	—	—
7	week-01/day-07	Revision + Mock #1	Sync-dict story ≤3 min	Feed LLD

Primary SD doc: [social-feed.md](#)

Week 2 — Architecture, networking, SDUI, UI

Day	File	Focus	Production hook	Parallel SD
8	week-02/day-08	MVVM / Clean / MVI / DI	District migration; BMS search	—
9	week-02/day-09	URLSession, interceptors, refresh, cache	Alamofire → URLSession	Networking layer
10	week-02/day-10	SDUI / CMS UI	Backend-driven header; Aces splash	SDUI engine
11	week-02/day-11	UIKit lifecycle, hybrid UI	Grizzlies; LE Bottom Sheet	—
12	week-02/day-12	SwiftUI state, identity, lists	Stories SDK	—
13	week-02/day-13/	DSA: Stack/Queue/LinkedList	—	—
14	week-02/day-14/	Revision + Mock #2	Ads architecture 5 min	Payment + Search

Primary SD docs: [networking-layer.md](#), [sdui-engine.md](#), [payment-checkout.md](#), [search-autocomplete.md](#)

Week 3 — Full architecture, performance, security, platform

Day	File	Focus	Production hook	Parallel SD
15	week-03/day-15/	Modularization, SPM, DI	Stories SDK module	App modularization
16	week-03/day-16/	Image/video pipelines, cache	HeroWidget; Aces audio	Image loading
17	week-03/day-17	Perf: Instruments, p50/p90	Firebase Performance	APM
18	week-03/day-18	Crash reporting, IMOC	99.95% crash-free	Crash SDK
19	week-03/day-19	Security, SSL pinning, Keychain	Pinning + whitelist	Security engine
20	week-03/day-20	Deeplinks, push, CI/CD	Grizzlies; GH Actions	Deeplink/Push/CI
21	week-03/day-21	System-design Mock #3 (45 min)	SDUI or Networking+pinning	Cheatsheet

Also: Persistence decision tree (UserDefaults / Keychain / Files / SQLite / Core Data) — see Day 19 extras and [cheatsheet.md](#).

Week 4 — DSA polish, on-device AI, behavioral, game day

Day	File	Focus	Notes
22	week-04/day-22/	DSA: Trees, BFS/DFS	Mediums; complexity first
23	week-04/day-23/	DSA: HashMap/Heap + mixed	Unknown-pattern simulation
24	week-04/day-24/	On-device AI architecture	FinTrack / GymFlow
25	week-04/day-25/	Machine round (3 hrs)	Paginated list+cache or SDUI renderer
26	week-04/day-26/	Behavioral + leadership	STAR polish; AI tooling judgment
27	week-04/day-27/	Expert Mock #4 full loop	Coding + deep dive + SD
28	week-04/day-28/	Game day	Flashcards + stories only; sleep

Mock schedule

Mock	Day	Format	Focus
#1	7	Self/peer 60–90 min	Concurrency + memory; timed answers
#2	14	Self/peer 60–90 min	Ads or SDUI architecture (5 min talk)
#3	21	Self/expert 45 min	Full mobile system design
#4	27	Expert preferred	Coding 45 + iOS 45 + SD 45

Answer Timing Guide

Practice with a timer every day. Long-winded answers hurt as much as short ones. Senior signal = **agenda + trade-off + number + production example**.

Quick budgets

Format	Time	Structure
Definition / "what is X?"	30–45s	Definition → one property → one BMS/Raw/District example
Conceptual deep dive	90–120s	Claim → mechanism → trade-off → production story
Coding approach (before typing)	2–3 min	Clarify → brute → optimize → complexity → edge cases
Architecture / design pattern	3–5 min	Problem → options → choice → layers → failure modes
Mobile system design	45 min	Clarify 5 → HLD 10 → Data/API 10 → Deep dive 15 → Ops 5
Behavioral STAR	2–3 min	20s S/T → 90s Action → 30s Result → 20s lesson

1. Definition (30–45 seconds)

Template:

"X is The key property is In practice at BookMyShow/Raw, I used it when"

Example (ARC, ~35s):

"ARC is Swift's compile-time reference counting for class instances. Strong refs increment the count; when it hits zero, `deinit` runs. At BMS we hunted retain cycles in ad and navigation closures because leaked VCs showed up as memory

pressure under peak traffic.”

Fail if: You start with history of Objective-C, or never give an example.

2. Conceptual deep dive (90–120 seconds)

Template:

1. Claim (10s)
2. Mechanism (40s)
3. Trade-off / when not (20s)
4. Production proof (30s)

Example (serial queue vs lock, ~110s):

“For shared mutable dictionaries I prefer a GCD serial queue as a mutual exclusion boundary. All writes `async` onto the queue; reads either `sync` for a snapshot or stay on the queue. That eliminates data races without exposing lock ordering bugs. Trade-off: `sync` from the same queue deadlocks, and high read contention may want a reader-writer pattern or an actor. At BMS we introduced synchronised dictionaries this way to stop concurrent-access crashes in `async` shared state.”

3. Coding approach (2–3 minutes before code)

Say this first:

1. Restate constraints + ask 1–2 clarifying Qs (30s)
2. Brute force + complexity (30s)
3. Optimized approach + why (45s)
4. Edge cases + API shape (30s)
5. “I’ll code the optimized version now” (5s)

Do not jump into code silently. Interviewers grade communication.

4. Architecture walkthrough (3–5 minutes)

Agenda opener (10s):

“I’ll cover the problem, options we considered, the architecture we shipped, and failure modes.”

Minute	Content
0:00–0:40	Problem + scale (DAU, revenue, crash)
0:40–1:40	Options (e.g. MVC vs MVVM vs Clean) + why chosen
1:40–3:30	Layers: View → VM → Domain/UseCase → Data/Network; key protocols
3:30–4:30	Failure modes, testing, rollout
4:30–5:00	Stop; invite questions

Use: Ads module, SDUI header, Stories SDK, District Clean migration.

5. Mobile system design (45 minutes)

From [ios-system-design cheatsheet](#):

0–5 Clarify: scope, DAU, offline?, iOS-only?, latency SLO
5–15 HLD: 4-layer client + CDN/API/cache/UI data flow
15–25 Data/API: entities, cursor pagination, payloads, idempotency

25–40 Deep dive: 2–3 hardest subsystems (you choose)

40–45 Ops: failure modes, metrics, rollout, kill switch

Staff signal opener:

"I'll spend 5 minutes on scope, then full architecture, then deep-dive X and Y. Does that work?"

4-layer client (always draw):

1. View (SwiftUI/UIKit — no business logic)
2. ViewModel / Presenter (state, mapping)
3. Domain (use cases, protocols)
4. Data (repositories, network, cache, persistence)

6. Behavioral STAR (2–3 minutes)

Segment	Time	Content
Situation + Task	~20s	Context + your ownership
Action	~90s	3–5 concrete steps (design, trade-off, collaboration)
Result	~30s	Metric if you have one (30% nav reduction, 99.95% CFS, etc.)
Lesson	~20s	What you'd do differently / principle

Never invent metrics. Only resume-backed numbers:

- 30+ lakh DAU
- 99.95%+ crash-free sessions
- 30%+ user flows with fewer full-screen navigations (LE Bottom Sheet)
- Portfolio reuse of Stories SDK
- Zero-regression Xcode 15 migration checklist

Anti-patterns

Anti-pattern	Fix
No agenda on long answers	State structure in first 10s
No trade-off	Always add "we didn't choose X because..."
No number / no production hook	Attach BMS/District/Raw line
Tool worship (Cursor/Claude)	Lead with architecture judgment; AI is accelerator
Apologetic filler ("I'm not sure but...")	State confidence + assumption
Overlong STAR (>4 min)	Cut Action to 3 bullets
Silent coding	Narrate complexity first

Self-scoring rubric (after each recording)

Score	Meaning
5	On time, clear agenda, trade-off, production proof
4	On time, missing one of trade-off / proof

3	Content OK, 30%+ over/under time
2	Rambling or wrong mechanism
1	Blank / incorrect

Target: **≥4** on Normal Qs, **≥3** on Tricky before mocks.

Daily timing drills

- 1. **Sprint (10 min):** 5× definition Qs at 45s each.
- 2. **Deep (15 min):** 3× conceptual at 2 min each.
- 3. **Story (5 min):** 1× STAR at 2:30, then re-cut to 2:00.
- 4. **SD micro (15 min):** Only Clarify + HLD for one system.

STAR Story Bank

Practice each story to **2–3 minutes**. Metrics only from your resume. Pick the story by interview topic tags.

Quick picker

Tag	Story IDs
Architecture / POP / Generics	S1, S9, S10
Concurrency / races	S2
SDUI / search	S3, S12
Security / networking	S4
Performance / observability	S5, S8
Product delivery / UX	S6, S7
Incidents / ownership	S8
Migration / AI tooling	S9
SDK / modularity	S10
Real-time	S11
AV / cold start	S12
Hybrid UI / deeplinks	S13
Platform upgrade	S14
On-device AI	S15, S16

S1 — Ads module refactor + HeroWidget (BookMyShow)

Tags: Architecture, POP, Generics, revenue-critical
Use when: “Tell me about a hard technical project” / design patterns

Situation/Task (~20s):
Highest-revenue Ads module needed a safer, reusable rendering path. Video ads inside a HeroWidget needed correct pause/play with lifecycle.

Action (~90s):

1. Refactored rendering around **protocol-oriented** ad component contracts + **generics** for a type-safe pipeline.
2. Built reusable **HeroWidget** with explicit pause/play tied to visibility / VC lifecycle.
3. Kept revenue paths behind clear protocols so new ad types didn't fork the pipeline.
4. Coordinated with stakeholders on behavior without breaking fill rates.

Result (~30s):

Shipped a maintainable, type-safe ads pipeline; lifecycle-correct video reduced wasted playback and UI glitches on a module that mattered for revenue.

Lesson (~20s):

For revenue-critical UI, prefer POP + generics over inheritance trees; lifecycle is part of the product contract.

Timed opener:

"I'll walk through our highest-revenue Ads refactor — protocols, generics, and video lifecycle."

S2 — Synchronised dictionaries (BookMyShow)

Tags: Concurrency, GCD, crash reduction

Use when: Thread safety / race conditions / GCD vs actors

Situation/Task:

Shared async state was hit from multiple queues → data races / intermittent crashes.

Action:

1. Introduced **synchronised dictionaries** gated by **GCD serial queues** (and read-write locks where read-heavy).
2. Standardized access API so call sites couldn't touch raw storage.
3. Validated under concurrency stress and Crashlytics watch.

Result:

Eliminated concurrent-access crashes in that shared state path; pattern reused where mutable maps were shared across async work.

Lesson:

Serialize mutation at the boundary; don't sprinkle locks ad hoc. Today I'd also evaluate a Swift `actor` for new code.

Timed opener:

"We had races on shared dictionaries — I'll cover the serial-queue design and trade-offs vs actors."

S3 — Backend-driven header & search (BookMyShow)

Tags: SDUI, MVVM, networking UX

Use when: Server-driven UI / search / debounce

Situation/Task:

Main header needed to be CMS/backend-driven; search needed modern UX (debounce, state) aligned to MVVM.

Action:

1. Migrated header to a **generalised, protocol-driven** main-screen implementation.
2. Search: **debouncing**, explicit loading/empty/error state, MVVM binding.
3. Contracted APIs with backend so layout/content could change without app release for many cases.

Result:

Faster content iteration on header; snappier, race-safer search UX.

Lesson:

SDUI needs schema/versioning + client fallbacks — not just "render JSON."

S4 — SSL pinning + Alamofire → URLSession (BookMyShow)

Tags: Security, networking ownership

Use when: MITM / ATS / URLSession design

Situation/Task:

Ads networking stack on Alamofire; need stronger transport security and less dependency surface.

Action (Verified):

1. Migrated Ads pod networking to **URLSession**.
2. Enforced **HTTPS, SSL pinning, domain whitelisting**.

How I would apply it (S4-A1 — design, not claimed as a shipped runbook):

Pin rotation with backup pins, staged rollout, and a controlled break-glass path so cert/key changes do not Sev-1 the app.

Result:

Reduced MITM risk on a sensitive high-traffic module; simpler first-party networking control.

Lesson:

Pinning without a rotation/ops design is incomplete — speak to break-glass as architecture judgment even when the resume only lists pinning + allowlist.

S5 — Firebase Performance traces (BookMyShow)

Tags: Observability, performance culture

Use when: How do you measure performance?

Situation/Task:

Needed data-driven latency visibility across listing, checkout, search.

Action:

1. Instrumented **Firebase Performance** traces for key journeys.
2. Tracked **p50/p90** (not just averages).
3. Used traces to prioritize optimisation across releases.

Result:

Observability layer for journey latency; conversations with PM/backend shifted from anecdotes to percentiles.

Lesson:

p90 matters more than average for user-perceived jank under load.

S6 — LE Bottom Sheet (BookMyShow)

Tags: Product delivery, UX, cross-functional

Use when: Impact / collaboration with PM & Design

Situation/Task:

Users took full-screen navigations for event overview too often.

Action:

1. Led **end-to-end** lightweight event-overview bottom sheet.
2. Aligned PM, Design, Backend on **API contracts** and content.
3. Shipped reusable component integrated into high-traffic flows.

Result:

Reduced full-screen navigations for **30%+ of user flows** (resume metric).

Lesson:

Small UI surfaces with clear contracts beat large rewrites for navigation pain.

S7 — Payment processing-time popup (BookMyShow)

Tags: Checkout UX, state machine

Use when: Payments / handling uncertainty

Situation/Task:

Checkout delays caused drop-off and support load when users saw no status.

Action:

1. Designed lightweight popup for **real-time payment/booking processing status**.
2. Modeled clear states (processing / success / failure / timeout messaging).
3. Coordinated with backend signals for status updates.

Result:

Reduced ambiguity during delays → less drop-off / support friction.

Lesson:

In payments, communication of state is a feature; silent waiting is a bug.

S8 — IMOC + crash-free at scale (BookMyShow)

Tags: Incidents, leadership, reliability

Use when: Conflict / production fire / ownership

Situation/Task:

Consumer app at **30+ lakh DAU**; need **99.95%+ crash-free** and fast P0/P1 response.

Action:

1. Crashlytics triage + structured crash workflows.
2. As **IMOC**, coordinated iOS, backend, QA for P0/P1 during high-traffic events.
3. Drove mitigations (feature guards, hotfix path, comms).

Result:

Sustained high crash-free sessions; minimised downtime in peak events.

Lesson:

Incident leadership is about clarity of owner, blast radius, and rollback — not hero debugging alone.

S9 — Free Parking + Clean/MVVM + AI tooling (District)

Tags: Architecture migration, AI-assisted velocity

Use when: Refactors / how you use AI / Clean Architecture

Situation/Task:

Ship Free Parking billing adjustments; migrate patterns across MVVM and Clean with low regression risk.

Action:

1. Delivered Free Parking feature for billing adjustments; on-call production fixes.
2. Used **Context Engineering** (Cursor/Claude/Copilot) to accelerate migration decisions — not to skip design.
3. AI-assisted reviews + XCTest/XCUITest generation; structured logging for debug.

Result:

Feature shipped; architectural migration progressed with velocity **and** review discipline.

Lesson (critical for interviews):

AI accelerates once boundaries/protocols are clear. You still own architecture, tests, and regressions.

Avoid saying: "AI wrote the app."

Prefer: "I used AI inside a strict architectural envelope and review loop."

S10 — Stories SDK (Raw / Miami Heat)

Tags: SDK design, modularity, reuse

Use when: Modularization / reusable frameworks

Situation/Task:

Need Instagram-style fan Stories across client portfolio, not one-off UI.

Action:

1. Designed **standalone reusable Stories SDK**.
2. Clear public API, isolation from app-specific networking where possible.
3. Adopted across portfolio apps.

Result:

One implementation leveraged by multiple NBA/WNBA apps → faster feature parity.

Lesson:

SDK quality = API surface + versioning + independence from host app shortcuts.

S11 — Live in-arena scoreboard (Raw / Miami Heat)

Tags: Real-time reliability

Use when: Live updates / reliability under flaky networks

Situation/Task:

In-arena live scoreboard must stay trustworthy during games.

Action:

1. Owned live updates path for match statistics.
2. Defensive UI for stale/partial data; reconnect / refresh strategy.
3. Prioritized correctness over flashy transitions.

Result:

High-reliability live scoreboard experience for fans in-venue.

Lesson:

Live UI is a state-machine problem; define stale vs live explicitly.

S12 — Audio streaming + server-driven splash (Raw / Las Vegas Aces)

Tags: AVFoundation, cold start, SDUI

Use when: Media / startup optimisation

Situation/Task:

Live broadcast audio + slow/inflexible splash.

Action:

1. Integrated **audio streaming** for live broadcasts.
2. Revamped splash with **server-driven content** to cut cold-start latency / improve freshness.

Result:

Richer live audio; faster, more flexible cold start content.

Lesson:

Startup path is a product surface — measure time-to-interactive, not just time-to-first-frame vanity.

S13 — SwiftUI↔UIKit + deeplinks + analytics/push (Raw / Memphis Grizzlies)

Tags: Hybrid UI, deeplinks, growth stack

Use when: Interop / navigation / lifecycle

Situation/Task:

Production NBA app needing modern UI with existing UIKit, plus growth tooling.

Action:

1. Architected key UI with **SwiftUI + UIKit interoperability**.
2. Deep linking + navigation/lifecycle handling.
3. Integrated **Mixpanel + Airship** (analytics + push).

Result:

Shipped hybrid UI with production navigation and engagement stack.

Lesson:

Interop costs (identity, lifecycle, hosting) must be designed — not bolted.

S14 — Xcode 15 migration (Raw / Chicago Sky)

Tags: Platform upgrade, risk management

Use when: Messy upgrade / zero regression

Situation/Task:

Xcode 15 + SDK/API breakages across TicketMaster and other SDKs.

Action:

1. Led migration; catalogued breaking changes.
2. QA-driven **zero-regression checklist**.
3. Fixed deprecated APIs methodically; staged validation.

Result:

Migration completed without regression fire drills.

Lesson:

Upgrades are project-managed risk: inventory → fix → checklist → soak.

S15 — FinTrack on-device RAG / Apple Intelligence (Personal)

Tags: On-device AI, privacy

Use when: AI/ML on mobile / privacy-first design

Situation/Task:

Expense tracker must stay local-first; AI coach without cloud LLM dependency.

Action:

1. Flutter app + Hive; biometric lock; no cloud sync of financial data.
2. `flutter_native_ai` bridge to **Apple Foundation Models** / Gemini Nano.
3. **BM25 offline RAG** over local spending index + deterministic rule fallback.

Result:

Privacy-first Savings Coach with fail-soft path on unsupported devices; Play Store release ops (Crashlytics, Remote Config, AdMob).

Lesson:

On-device AI = retrieval + fallback strategy, not just model calls.

S16 — GymFlow TFLite MiniLM recommender (Personal)

Tags: Edge ML, RAG, fail-soft
Use when: Embeddings / TFLite / hybrid systems

Situation/Task:
Workout recommender without cloud LLM cost/latency/privacy issues.

Action:

1. INT8 **MiniLM TFLite** embeddings + WordPiece tokenizer in Dart.
2. Cosine similarity over exercise catalog; trainer routine as RAG context.
3. Fail-soft **TF-IDF** fallback.

Result:
On-device recommendations with graceful degradation.

Lesson:
Always design the “model missing / too slow / thermal” path.

Behavioral quick answers (30–45s variants)

Conflict: Use S8 (IMOC) or S6 (cross-functional contracts) — disagreement resolved via data/API contract, not ego.
Failure: Prefer a real miss (pin rotation risk, race you shipped, SDUI fallback gap) + what process changed.
Leadership without title: S1, S6, S10, S14 — end-to-end ownership.
Why senior: Scale (30L DAU), revenue module ownership, SDK reuse, incident coordination, architecture migration judgment.

Experience registry (resume-backed)

Use these IDs in chapter provenance labels. **Verified** = stated in `resume.tex` . **Applied** = interview design extension (not claimed as shipped).

Verified stories

ID	Company	Resume anchor	Allowed metrics	Topics
S1	BookMyShow	Ads module refactor; HeroWidget pause/play; POP + Generics	revenue-critical module (no invented fill-rate %)	POP, generics, video lifecycle
S2	BookMyShow	Synchronised dictionaries via GCD serial queues and RW locks	race/crash reduction on that path (do not claim sole CFS ownership)	GCD, concurrency
S3	BookMyShow	Backend-driven header; search debounce, state, MVVM	—	SDUI, search, MVVM
S4	BookMyShow	Ads Alamofire → URLSession; HTTPS; SSL pinning; domain whitelisting	—	networking, security
S5	BookMyShow	Firebase Performance traces listing/checkout/search p50/p90	p50/p90 instrumentation	observability
S6	BookMyShow	LE Bottom Sheet	30%+ of user flows fewer full-screen navigations	product, UX
S7	BookMyShow	Payment processing-time popup	intent: reduce drop-off/support (no measured % unless you add real data)	checkout UX, state

S8	BookMyShow	30+ lakh DAU; 99.95%+ CFS; Crashlytics; IMOC P0/P1	30L+ DAU, 99.95%+ CFS	incidents, reliability
S9	District	Free Parking; MVVM/Clean migration; Context Engineering; XCTest/XCUITest assist	—	architecture, AI tooling
S10	Raw / Miami Heat	Stories SDK reused across portfolio; live scoreboard	portfolio reuse	SDK, modularity, realtime
S11	Raw / Miami Heat	Live in-arena scoreboard	—	realtime reliability
S12	Raw / Aces	Audio streaming; server-driven splash	cold-start improvement (no invented ms)	AV, SDUI splash
S13	Raw / Grizzlies	SwiftUI+UIKit; deeplinks; Mixpanel; Airship	—	hybrid UI, push, analytics
S14	Raw / Chicago Sky	Xcode 15 migration; QA zero-regression checklist	—	platform upgrade
S15	Personal	FinTrack on-device RAG / Apple Intelligence / BM25	local-first / no cloud financial sync	on-device AI
S16	Personal	GymFlow TFLite MiniLM + TF-IDF fallback	—	edge ML

Applied extensions (not shipped claims)

ID	Extends	Claim you may say
S2-A1	S2	Greenfield shared maps → Swift actor with same API surface
S4-A1	S4	Pin rotation, backup pins, staged break-glass as design (not “I shipped the runbook”)
S7-A1	S7	Explicit enum state machine for payment UI (design pattern; resume does not name enums)
S3-A1	S3	Schema versioning + unknown-component fallback for SDUI
S5-A1	S5	Journey-level traces vs interceptor path traces

Label format in chapters

```
> Provenance: Verified · S2 · BookMyShow · synchronised dictionaries

> Provenance: How I would apply it · S4-A1 · pin rotation / break-glass design
```

Swift DSA Track — 4 Weeks

Candidate: **Muhammed Shahid** · BMS / District / Raw
Solve in **Swift**. Narrate before typing. Timing: [answer-timing-guide.md](#) § Coding approach (2–3 min).
Resume metrics only (behavioral / SD, not invent LeetCode numbers): 30L+ DAU · 99.95%+ CFS · 30%+ fewer full-screen navs.

Master “say this first” (≈60s)

Use on **every** problem before code:

*"Constraints? Sorted? Duplicates? Mutate in place?
Brute force would be ... $O(?)$.
I'll use ... because Complexity time ... space
Edge cases: empty, single element, all same / null head.
Coding the optimized version now."*

Swift tips: `String` is not random $O(1)$ — convert to `[Character]` when needed; prefer iterative list reverse; note `Array.removeFirst` is $O(n)$.

Week 1 — Arrays / Strings / Two Pointers / Sliding Window

Day: Week 1 Day 06 · Target: **~12 problems** · Goal: pattern ID + communication > clever tricks.

#	LeetCode	Pattern	Complexity	Say this first (60s)
1	Two Sum	Hash map (value→index)	$O(n)$ time / $O(n)$ space	"Need indices — sort loses them. I'll scan once, store complements in a dictionary. Edges: empty, no pair, duplicates."
2	Best Time to Buy and Sell Stock	One-pass min + profit	$O(n)$ / $O(1)$	"One buy/sell. Track min so far and max profit. Brute $O(n^2)$ pairs. Edges: length 1, decreasing prices."
3	Valid Palindrome	Two pointers opposite	$O(n)$ / $O(1)$	"Skip non-alnum, compare ends inward. Edges: empty, all punctuation, unicode — clarify charset."
4	Container With Most Water	Two pointers ends	$O(n)$ / $O(1)$	"Area = min(h)×width. Move the shorter end. Brute $O(n^2)$. Edges: n=2, all equal heights."
5	3Sum	Sort + two pointers	$O(n^2)$ / $O(1)$ – $O(n)$	"Sort, fix i, two-sum on remainder; skip duplicates. Edges: all zero, fewer than 3, many dups."
6	Longest Substring Without Repeating Characters	Variable sliding window + set/map	$O(n)$ / $O(\min(n, \Sigma))$	"Expand right; shrink left when duplicate. Track last index. Edges: empty, all unique, all same."
7	Maximum Subarray	Kadane	$O(n)$ / $O(1)$	"Running sum; reset when negative; track global max. Edges: all negative — return largest element."
8	Product of Array Except Self	Prefix/suffix products	$O(n)$ / $O(1)$ extra (output aside)	"Left products then right pass; no division. Edges: zeros, single zero, negatives."
9	Move Zeroes	Write pointer	$O(n)$ / $O(1)$	"Write non-zeros forward, fill zeros. Edges: no zeros, all zeros, already packed."
10	Longest Consecutive Sequence (optional)	Hash set	$O(n)$ avg / $O(n)$	"Only start chains from numbers without n–1. Edges: empty, dups, negatives."
11	Minimum Size Subarray Sum	Variable window	$O(n)$ / $O(1)$	"Expand until $\geq$ target, shrink left, track min length. Edges: impossible → 0, single element."
12	Group Anagrams (bridge to W4)	Frequency signature map	$O(n \cdot k)$ / $O(n \cdot k)$	"Key by sorted string or count tuple; bucket. Edges: empty strings, single char."

Week 1 exit criteria: 8/12 solved cold in Swift with spoken agenda; Two Sum + window + Kadane fluent.

Week 2 — Stack / Queue / Linked List

Day: Week 2 Day 13 · Target: ~10 problems.

#	LeetCode	Pattern	Complexity	Say this first (60s)
1	Valid Parentheses	Stack matching	$O(n)$ / $O(n)$	"Push opens; on close check top match; empty at end. Edges: empty, only closers, odd length."
2	Min Stack	Aux min stack	$O(1)$ ops / $O(n)$	"Push min alongside; pop both. Edges: pop last, duplicate mins."
3	Daily Temperatures	Monotonic stack	$O(n)$ / $O(n)$	"Next greater: decreasing stack of indices; fill waits. Edges: strictly decreasing → zeros."
4	Evaluate Reverse Polish Notation	Stack eval	$O(n)$ / $O(n)$	"Numbers push; ops pop2 compute. Edges: negatives, single number."
5	Implement Queue using Stacks	Two-stack amortize	Amortized $O(1)$	"In stack + out stack; flip on dequeue. Don't claim worst-case $O(1)$ always."
6	Sliding Window Maximum	Monotonic deque	$O(n)$ / $O(k)$	"Deque of decreasing indices in window; front is max. Edges: $k=1$, $k=n$."
7	Reverse Linked List	3-pointer iterative	$O(n)$ / $O(1)$	"prev/curr/next; rewire. Edges: empty, single node. Prefer iterative for stack depth."
8	Linked List Cycle	Floyd slow/fast	$O(n)$ / $O(1)$	"Meet ⇒ cycle. Hash set is $O(n)$ space alt. Edges: no cycle, cycle at head."
9	Merge Two Sorted Lists	Dummy head	$O(n+m)$ / $O(1)$	"Append smaller; attach remainder. Edges: one empty, equal values."
10	Middle of the Linked List	Slow/fast	$O(n)$ / $O(1)$	"Fast 2x; slow at middle. Clarify even-length convention (second middle)."

Stretch (if ahead): Reverse Nodes in k-Group · Remove Nth Node From End · LRU Cache design sketch.

Week 2 exit: Valid Parentheses + reverse list + Floyd without notes; can explain why `removeFirst` is a bad queue.

Week 3 — Light review + optional trees warm-up

No new heavy DSA day — Week 3 is modularization / images / perf / crashes / security / deeplinks. Keep DSA **alive**:

Block	Time	What
Spaced review	3×20 min during week	Re-solve 2 Week-1 + 2 Week-2 misses cold
Optional warm-up	45–60 min once	Trees preview below (Easy only)

Optional trees warm-up (Easy)

#	LeetCode	Pattern	Complexity	Say this first
1	Maximum Depth of Binary Tree	DFS height / BFS levels	$O(n)$ / $O(h)$ or $O(w)$	"DFS 1+max(L,R) or BFS level count. Edges: null, skewed."
2	Invert Binary Tree	DFS/BFS swap	$O(n)$ / $O(h)$	"Swap children recurse. Edges: null, leaf."

3	Same Tree	Parallel DFS	$O(n) / O(h)$	"Both null OK; values + structure. Edges: shape mismatch."
4	Binary Tree Inorder Traversal	DFS / stack	$O(n) / O(h)$	"Left-root-right. Mention BST sorted property for later."

Week 4 — Trees BFS/DFS, HashMap/Heap, Mixed

A. Trees (~12) — Day 22

#	LeetCode	Pattern	Complexity	Say this first (60s)
1	Binary Tree Level Order Traversal	BFS level-size	$O(n) / O(w)$	"Queue; for count in level append. Edges: null, skewed."
2	Binary Tree Zigzag Level Order Traversal	BFS + reverse odd	$O(n) / O(w)$	"Same as level order; reverse alternate. Edges: single node."
3	Binary Tree Right Side View	BFS last-in-level	$O(n) / O(w)$	"Take last node each level (or DFS priority right). Edges: left-only tree."
4	Maximum Depth of Binary Tree	DFS/BFS	$O(n) / O(h)$	Warm-up refresh.
5	Diameter of Binary Tree	DFS height + global	$O(n) / O(h)$	"At each node leftH+rightH; diameter may not pass root. Edges: line tree."
6	Path Sum / Path Sum II	DFS remaining	$O(n) / O(h)$	"Subtract val; leaf check. II collects paths — copy carefully."
7	Lowest Common Ancestor of a Binary Tree	Postorder both sides	$O(n) / O(h)$	"If both sides found, node is LCA. Trap: don't assume BST."
8	Validate Binary Search Tree	Bounds or inorder	$O(n) / O(h)$	"Pass (lo,hi) or check inorder increasing. Trap: only vs parent."
9	Kth Smallest Element in a BST	Inorder count	$O(h+k) / O(h)$	"Inorder until k. Edges: k=1, k=n."
10	Symmetric Tree	Mirror DFS or BFS pairs	$O(n) / O(h)$	"Compare left/right mirrors. Edges: null children asymmetric."
11	Serialize and Deserialize Binary Tree (<i>stretch</i>)	Preorder + nulls / BFS	$O(n) / O(n)$	"Need explicit nulls. Link to SDUI versioning honesty."
12	Construct Binary Tree from Preorder and Inorder (<i>stretch</i>)	Hash index + recurse	$O(n) / O(n)$	"Root from preorder; split inorder via map."

B. Hash / Heap (~8) — Day 23

#	LeetCode	Pattern	Complexity	Say this first
1	Two Sum	Hash	$O(n) / O(n)$	Refresh; indices.
2	Group Anagrams	Hash signature	$O(n \cdot k)$	Refresh.
3	Subarray Sum Equals K	Prefix + hash	$O(n) / O(n)$	"Count prefix-k. Negatives → not two-pointer."
4	Longest Substring Without Repeating Characters	Window + map	$O(n)$	Refresh.

5	Top K Frequent Elements	Hash + heap or bucket	$O(n \log k)$ or $O(n)$	"Min-heap size k, or freq buckets. Clarify ordered output?"
6	Kth Largest Element in an Array	Min-heap size k / Quickselect	$O(n \log k)$	"Heap of k; peek is answer. Edges: dups, k=1."
7	Merge k Sorted Lists	Heap of heads	$O(N \log k)$ / $O(k)$	"Push heads; pop min; push next. Prod metaphor: merge cursors."
8	K Closest Points to Origin	Max-heap size k	$O(n \log k)$	"Compare distance ² . Edges: ties — clarify."

C. Mixed simulation (~6+ tags)

Cold-open: write **pattern tag in 90s**, then solve ≥ 3 :

Tag	Sample
Tree BFS	Right Side View leftover
Tree DFS	Diameter / LCA
Hash	Clone Graph · Subarray Sum K
Window	Longest substring
Heap	Top K / Merge K
Stack	Daily Temperatures if rusty

Week 4 volume target: Trees 8–12 + Hash/Heap 7–8 + Mixed 3 $\approx$ ~20 problems.

Machine-round briefs (3 hours)

Full day notes: [week-04/day-25.md](#). Timing guide: [answer-timing-guide.md](#).

Shared timebox

Phase	Clock	Do	Don't
Clarify	0:00–0:15	Restate, API shape, UIKit/SwiftUI, offline?, XCTest	Code immediately
Agenda	0:15–0:20	Layers + milestones out loud	Silent architecture
Skeleton	0:20–0:45	Types, protocols, empty UI, fake repo	Pixel UI
Vertical slice	0:45–2:00	One happy path E2E	All edges first
Depth	2:00–2:30	Cache or 2nd SDUI component	Aesthetic refactor
Tests	2:30–2:50	3–6 meaningful unit tests	Coverage chase
Buffer	2:50–3:00	Trade-offs + cut lines README	New features

Proctor line @ 0:15:

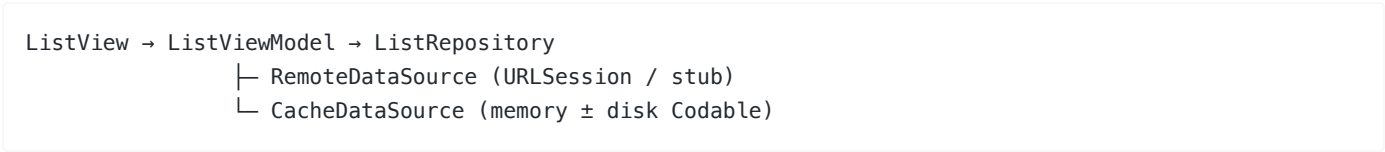
"I'll clarify briefly, then vertical slice: UI → ViewModel → protocol repository — fake first. I'll leave time for tests and document cut lines."

Brief A — Paginated list + cache + unit tests (3 hours)

Prompt:

Build a screen that loads a **paginated** remote list (cursor or page). Show loading / empty / error. Pull-to-refresh + next page on scroll. **Cache** last-good data for fast revisit + refresh. **Unit tests** for pagination + cache policy.

Suggested layers:



Acceptance:

- 1. First page renders (network or stub).
- 2. Next page **appends** (no wipe).
- 3. Page-2 failure keeps page-1 visible.
- 4. Cache: cold open can show stale then refresh (state policy aloud).
- 5. Tests on reducer/repository — not only snapshots.

Cache pick-one: memory LRU · memory+disk · stale-while-revalidate.

Cut lines OK: skeletons, Diffable animations, image pipeline, auth refresh, perfect offline merge.

BMS hook (≤20s): listings/search state + cancel patterns at scale — don’t claim this toy is production BMS.

Rubric A (score 1–5 each)

Dimension	5	3	1
Correctness	Happy path + key failure	Happy path only	Doesn’t run
Architecture	Layers + protocols	Mixed but clear	God VC/VM
Cache depth	Policy explained + coded	Partial	Missing
Tests	3+ fast meaningful	1 weak	None
Communication	Agenda + trade-offs	Occasional notes	Silent
Time honesty	Cut lines written	Overran polish	Unfinished core

Pass: average ≥3.5, correctness ≥4, tests ≥3.

Brief B — SDUI component renderer (3 hours)

Prompt:

JSON document of components (type , props , optional children) → native views. ≥3 types (e.g. text, image, button / vstack). Unknown type → safe fallback. Simple schemaVersion check. Unit-test decode + unknown fallback.

Suggested layers:



Acceptance:

- 1. Decode sample JSON.
- 2. Render ≥3 types.
- 3. Unknown type **does not crash** (placeholder + analytics stub).
- 4. Nested children for one container.
- 5. Tests: decoder + factory fallback.

Cut lines OK: CMS tooling, live reload, expression language, rich actions, Figma parity.

Prod hooks: BMS backend-driven header · Aces splash · Stories — fail-soft, not invent metrics.

Rubric B (score 1–5 each)

Dimension	5	3	1
Correctness	Decode + render + fallback	Partial render	Crash on unknown
Architecture	Document → factory → view	Ad hoc switch in View	JSON parsed in body
Schema/version	Checked + commented	Mentioned only	Ignored
Tests	Decode + fallback covered	One decoder test	None
Communication	Agenda + fail-soft story	Sparse	Silent
Time honesty	Cut lines	Over-scoped	Pretty UI, no fallback

Pass: same bar as A (avg ≥ 3.5 , correctness ≥ 4 , tests ≥ 3).

Timeboxing guidance (daily DSA)

Slot	Budget	Rule
Approach aloud	2–3 min	Mandatory — see timing guide
Code to first green	≤ 25 min Easy/Medium	If stuck 25+ min: brute + note optimize, move on
Edge dry-run	3–5 min	Empty / single / all-same / null
Pattern log	1 min	Tag · complexity said? · miss note
Spaced review	20 min	Yesterday's misses only

Anti-patterns: silent coding · skipping brute · inventing product metrics for LC · chasing Hards in Week 1.

Links

- Timing: [../timing/answer-timing-guide.md](https://github.com/alexeyrasnitsyn/leetcode-timing-guide/blob/master/answer-timing-guide.md)
 - Flashcards: [../flashcards/week-01.md](https://github.com/alexeyrasnitsyn/leetcode-timing-guide/blob/master/flashcards/week-01.md) · [week-04.md](https://github.com/alexeyrasnitsyn/leetcode-timing-guide/blob/master/flashcards/week-04.md)
 - Machine day: [../weeks/week-04/day-25.md](https://github.com/alexeyrasnitsyn/leetcode-timing-guide/blob/master/weeks/week-04/day-25.md)
 - LeetCode: <https://leetcode.com/> (filter Easy/Medium; Swift)
-

Problem count summary

Week	Focus	≈ Problems
1	Arrays / strings / pointers / window	12
2	Stack / queue / linked list	10
3	Review + optional Easy trees	4 optional + reviews
4	Trees + hash/heap + mixed	~20
Machine	Brief A or B	1 timed build
Total DSA solves		~42 + machine

Week 01

Day 01 — Value vs Reference, COW, Enums, Actors Intro

Week 1 · Full study (self-contained) · ~4–5 hrs

Revision twin: [revision/weeks/week-01/day-01.md](#)

Outcomes

By end of day, without notes, you can:

- Choose `struct` vs `class` vs `enum` vs `actor` with a clear decision rule and one production example
- Explain copy-on-write for Array/String/Dictionary: when assignment is cheap and when a real copy happens
- Model UI/network/payment flows as associated-value enums (impossible states eliminated)
- Give a 30s actors intro that bridges GCD serial queues → Swift actors (Day 05 deepens)
- Speak a ≤20s pitch tying value semantics to BMS ads / listing models

How to study (in Cursor only)

1. [01-foundations.md](#) — mental model (intern → mid)
2. [02-deep-dive.md](#) — mechanics, traps, trade-offs
3. [03-production-bridge.md](#) — Verified / Applied hooks
4. [code/](#) — read `LoadState.swift` and `COWDemo.swift`
5. [04-questions.md](#) — speak from **Answer points**; compare to full answer
6. [05-exercises.md](#) — drills + timed practice
7. Revision twin for flashcards / timed drill day-of

Module map

Module	File	Role
Foundations	01-foundations.md	Mental model + glossary + first examples
Deep dive	02-deep-dive.md	COW internals, nested refs, enum machines, actors intro
Production bridge	03-production-bridge.md	S1 / S7 / S2 → interview lines
Questions	04-questions.md	Two-layer Q&A (normal + tricky)
Exercises	05-exercises.md	Coding + speaking drills
Code	code/LoadState.swift , code/COWDemo.swift	Runnable mental models

Time budget (suggested)

Block	Minutes
Foundations	60–75
Deep dive	75–90
Production bridge + code	30–40
Questions (speak 6–8 aloud)	60–75

Exercises + timed drill	45–60
-------------------------	-------

Provenance reminder

Only use Verified IDs from [../../../../provenance/README.md](#) .

Label	Use for today
Verified · S1	Type-safe ads / listing models; prefer value-friendly DTOs
Verified · S7	Payment processing popup intent (states as product feature)
How I would apply it · S7-A1	Explicit enum state machine for payment UI (design; resume does not name enums)
Verified · S2	Soft bridge: serial-queue isolation → actors (intro only)
How I would apply it · S2-A1	Greenfield shared maps as Swift actor

Do **not** invent fill-rate %, drop-off %, or latency numbers.

Agenda openers (pin these)

Topic	5–10s opener
struct vs class	“Semantics → shared mutation → when I pick each.”
COW	“Cheap share until write — then unique copy.”
enum state	“Impossible states out → exhaustiveness in.”
actor intro	“Reference type with isolation; modern serial queue.”

01 — Foundations: Value vs Reference, COW, Enums, Actors

Read this first. Goal: build a crisp mental model an intern can teach back, then layer the senior edges you’ll need in interviews.

0. One-sentence north star

Prefer value types for data; use reference types when you need identity or shared mutable lifetime; use actors when that shared mutability crosses concurrency.

Everything below is unpacking that sentence.

1. Mental model: what “copy” and “share” mean

1.1 Two boxes in your head

Imagine every variable is a **name** taped to either:

Box	Meaning	Swift examples
Value box	The name owns a <i>snapshot</i> of the data. Assigning to another name makes another snapshot (conceptually).	struct, enum, tuple, Int, Bool
Reference box	The name holds a <i>pointer</i> to one shared object on the heap. Two names can point at the same object.	class, actor, closures (by capture), many UIKit types

Value: a → [payload A] b → [payload B] (independent)
 Reference: a → [same object] ← b (shared)

1.2 The intern demo (do this once out loud)

```
struct Point { var x: Int; var y: Int }

var a = Point(x: 1, y: 2)
var b = a           // conceptual copy of the value
b.x = 99
print(a.x)          // 1 – a unchanged
print(b.x)          // 99
```

```
class Box { var value: Int; init(_ v: Int) { value = v } }

var c = Box(1)
var d = c           // both point to the same Box
d.value = 99
print(c.value)      // 99 – shared mutation
```

Say aloud: “Structs copy their data on assignment. Classes share identity; mutating through one name is visible through the other.”

1.3 let vs var is not value vs reference

Binding	What it controls
let	The <i>name</i> cannot be reassigned
var	The <i>name</i> can be reassigned

For a **struct**, `let p = Point(...)` also means you cannot mutate `p`’s properties (because mutation would reassign the whole value under the hood).

For a **class**, `let c = Box(1)` still allows `c.value = 2` — the name still points to the same object; the object’s contents change.

This trip-up shows up in interviews constantly. Separate **binding mutability** from **type semantics**.

2. Decision table: struct / class / enum / actor

Need	Prefer	Why
Model / DTO / UI state snapshot	struct or enum	Independent copies; fewer shared-mutation bugs
Finite set of modes with payloads	enum (associated values)	Impossible combinations become compile errors
Identity (“this exact instance”)	class	===, shared lifetime, UIKit objects
Shared mutable state across tasks	actor	Isolation + await; data-race safety in the type system
ObjC / UIKit inheritance	class	Runtime reality of Cocoa
Inheritance tree for behavior	Prefer protocols + structs; class only if required	POP over deep hierarchies (see S1 ads pipeline)

Senior rule of thumb: Start with `struct` / `enum` . Justify every `class` . Justify every `actor` with a concurrency boundary.

3. Value types in practice (structs)

3.1 Why teams love structs for models

```
struct AdCreative: Equatable {
    let id: String
    let title: String
    let clickURL: URL
}

func decorate(_ ad: AdCreative) -> AdCreative {
    // Returning a new value – caller keeps the old one unless they assign
    var copy = ad
    // mutate copy fields if they were `var`...
    return copy
}
```

If two screens hold an `AdCreative` , neither accidentally mutates the other's copy. For revenue-critical listing/ad UI, that independence is a feature.

Provenance note: At BookMyShow, ads work emphasized a **type-safe** pipeline (protocols + generics). Preferring value-friendly models fits that safety story — **Verified · S1**. You do **not** need to claim “every model was a struct.”

3.2 Structs can contain classes (shallow copy)

```
class ImageCache { var bytes: Data = Data() }

struct CellModel {
    var title: String
    var cache: ImageCache // reference inside a value
}

var m1 = CellModel(title: "A", cache: ImageCache())
var m2 = m1
m2.title = "B" // only m2's title changes
m2.cache.bytes = Data([1]) // m1.cache sees it too – shared reference
```

Intern takeaway: Copying a struct copies its *stored properties*. Reference-typed properties stay shared.

Senior takeaway: “Value type” ≠ deep immutable graph. Audit nested classes when you reason about isolation.

4. Reference types in practice (classes)

4.1 When a class is the right tool

Use a class when you need **identity**:

- The same service instance shared across the app (`APIClient` , session managers)
- UIKit / AppKit objects (`UIViewController` , `UIView`)
- “Is this the same object?” checks with `===`
- Legacy KVO / ObjC runtime requirements

```
final class CheckoutSession {
    let id = UUID()
    var selectedSeats: [String] = []
}

let s1 = CheckoutSession()
let s2 = s1
print(s1 === s2) // true – same identity
```

4.2 == vs ===

Operator	Meaning	Works on
==	Equality of <i>value</i> (you define via Equatable)	Any Equatable type
===	Same <i>object identity</i>	Class instances (reference types)

Structs do not have `===`. Asking “are these the same struct instance?” is a category error — they are values, not identities.

5. Enums: not just “a list of cases”

5.1 Associated values = data that travels with the mode

```
enum LoadState<T> {
    case idle
    case loading
    case loaded(T)
    case failed(Error)
}
```

Compare to boolean soup:

```
// Fragile – illegal combinations are representable
var isLoading = false
var data: T? = nil
var error: Error? = nil
// isLoading && data != nil && error != nil ← all possible
```

With the enum, `loading` cannot also hold `data`. The compiler’s `switch` exhaustiveness forces you to handle every mode.

5.2 Why this matters for UI

Payment / booking flows are state machines in disguise:

```
enum PaymentPopupState {
    case hidden
    case processing(message: String)
    case success(bookingID: String)
    case failure(message: String)
    case timedOut(message: String)
}
```

Each case owns the data the UI needs for that screen. No optional `bookingID` hanging around during `processing`.

Provenance: The payment processing popup's intent is Verified · **S7** (clear status during delays). Modeling it as this enum is **How I would apply it · S7-A1** — a design pattern for interviews, not a claim that production shipped Swift enums by name.

5.3 Recursive enums need `indirect`

Enums have a fixed size known at compile time. A case that contains *another value of the same enum* needs a heap box:

```
indirect enum CommentThread {
    case leaf(String)
    case node(String, replies: [CommentThread])
}
```

`indirect` tells Swift: store that associated value behind a reference so the layout stays finite.

6. Copy-on-write (COW) — intern version

6.1 The problem COW solves

If `Array` truly deep-copied every element on every assignment, large lists would be expensive. If it shared buffers like a class *without* care, mutations would leak across variables (surprising for a “value type”).

COW = share storage until someone writes; then copy if needed.

6.2 The three steps

1. **Assign:** `var b = a` — both may share one buffer. Cheap (pointer + refcount).
2. **Read:** either variable can read freely. Still shared.
3. **Mutate:** before writing, Swift checks “am I the only owner?”
 - Yes → mutate in place
 - No → copy buffer, then mutate the unique copy

```
var a = [1, 2, 3]
var b = a          // share
b.append(4)        // b unique after copy; a still [1,2,3]
print(a)           // [1, 2, 3]
print(b)           // [1, 2, 3, 4]
```

Collections with COW in the standard library (know these names): `Array`, `Dictionary`, `Set`, `String`.

6.3 One-sentence junior explanation

“Arrays act like values, but under the hood they cheaply share memory until someone changes something — then they copy.”

7. Actors — intro only (Day 05 goes deep)

7.1 What problem actors solve

Reference types + concurrency = data races if two tasks mutate the same object without synchronization.

An `actor` is a reference type whose mutable state is **isolated**. You talk to it with `await`; the runtime serializes access.

```
actor Counter {
    private var value = 0
    func increment() { value += 1 }
    func get() -> Int { value }
}
```

```
let counter = Counter()
await counter.increment()
print(await counter.get())
```

7.2 Soft bridge from production

At BookMyShow, shared mutable maps were protected with **GCD serial queues** (Verified · **S2**). Soft interview line:

“Where we serialized dictionary access with a serial queue, a Swift `actor` is the language-native equivalent I’d evaluate for new code.”

That extension is **How I would apply it · S2-A1** — do not claim you rewrote production dictionaries as actors unless you did.

7.3 What *not* to say today

- “Actors replace all classes”
- “Every ViewModel should be an actor”
- Deep hop / reentrancy theory — save for Day 05

Enough for Day 01: **isolated reference type; await to touch state; prevents data races.**

8. Glossary (pin)

Term	Meaning
Value semantics	Independent copies; mutation does not surprise other names
Reference semantics	Shared identity; mutation is visible through all references
COW	Deferred copy until mutation of a non-unique buffer
Associated value	Payload stored in an enum case
Exhaustiveness	switch must cover all cases (or default)
indirect	Heap indirection for recursive enum layout
Isolation	Actor state only accessible in the actor’s executor context
===	Object identity
@MainActor	Isolation domain for main-thread / UI work (related, not identical to a custom actor)

9. Teach-back checklist

Explain each in ≤20s without notes:

1. Struct assignment vs class assignment
2. Why `let` on a class still allows property mutation
3. Why enums beat boolean flags for UI state
4. COW one-liner
5. Actor one-liner + serial-queue bridge

If any fail, re-read that section, then continue to [02-deep-dive.md](#) .

10. Mini examples to type once

Struct vs class mutation

```

struct Ticket { var seat: String }
class Hold { var seat: String; init(_ s: String) { seat = s } }

var t1 = Ticket(seat: "A1")
var t2 = t1
t2.seat = "B2"
// t1.seat == "A1"

let h1 = Hold("A1")
let h2 = h1
h2.seat = "B2"
// h1.seat == "B2"

```

Enum switch exhaustiveness

```

func title(for state: LoadState<String>) -> String {
    switch state {
    case .idle: return "Start"
    case .loading: return "Loading..."
    case .loaded(let value): return value
    case .failed: return "Something went wrong"
    }
}

```

Adding a new case forces every switch to update — that pressure is the feature.

Next

Go to [02-deep-dive.md](#) for COW uniqueness checks, large-struct costs, nested reference traps, payment state machines, and actor vs `@MainActor` boundaries.

02 — Deep Dive: Mechanics, Traps, Trade-offs

Senior depth. Assumes [01-foundations.md](#). Still self-contained — no external reading required.

1. Value semantics under pressure

1.1 What the compiler actually does (mental model)

For small structs stored in registers / stack, assignment often *is* a bitwise copy of stored properties. For larger values, Swift may still copy stored properties eagerly — **unless** the type implements COW internally (std collections do).

Your own `struct Huge` with ten large non-COW properties pays copy cost on each assignment / pass-by-value.

```

struct ListingRow {
    var id: String
    var title: String
    var subtitle: String
    var imageURL: URL?
    var badge: String?
    // ...many fields
}

```



```
func decorate(_ row: ListingRow) -> ListingRow {
    var copy = row // copies stored properties
    copy.badge = "Ad"
    return copy
}
```

Interview nuance: “Structs are always cheaper than classes” is false. Structs win on *semantics* and often on small data; large graphs may want COW wrappers, `inout`, or a reference boundary.

1.2 `inout` and exclusivity

```
func bumpX(_ p: inout Point) {
    p.x += 1
}

var point = Point(x: 0, y: 0)
bumpX(&point) // mutate in place – no conceptual “return a copy”
```

Swift’s exclusivity rules prevent overlapping access to the same value. That’s part of why value semantics stay predictable.

1.3 Mutation through `mutating` methods

```
struct Counter {
    private(set) var value = 0
    mutating func increment() { value += 1 }
}

var c = Counter()
c.increment()

let frozen = Counter()
// frozen.increment() // compile error – mutating on let binding
```

A `mutating` method is allowed to replace `self`. That only works on `var` bindings.

2. Copy-on-write — senior mechanics

2.1 Uniqueness

Stdlib collections store a reference to a buffer object. Before mutating, they ask roughly: “is this buffer uniquely referenced?”

Swift exposes a related primitive for **your** COW types:

```
final class Storage {
    var items: [Int]
    init(_ items: [Int]) { self.items = items }
}

struct COWList {
    private var storage: Storage

    init(_ items: [Int]) {
        storage = Storage(items)
    }

    private mutating func ensureUnique() {
```

```

        if !isKnownUniquelyReferenced(&storage) {
            storage = Storage(storage.items) // copy buffer
        }
    }

    mutating func append(_ value: Int) {
        ensureUnique()
        storage.items.append(value)
    }

    var items: [Int] { storage.items }
}

```

Teaching point: `isKnownUniquelyReferenced` only works with **class** instances. COW is “value type façade over a reference-counted buffer.”

2.2 When does A change after mutating B?

```

var a = [1, 2, 3]
var b = a
b.append(4)
// a is still [1,2,3] – COW copied for b

```

Trap answer: “Always shared” or “Always copied on assign.”

Correct: **shared until a write forces uniqueness.**

Edge case people forget:

```

var a = [1, 2, 3]
var b = a
a.append(4) // a may copy; b keeps old buffer

```

Same rule — whoever mutates while sharing pays for the copy (if not unique).

2.3 Wrapping arrays in a class kills COW across that wrapper

```

final class ArrayBox {
    var values: [Int]
    init(_ values: [Int]) { self.values = values }
}

let box1 = ArrayBox([1, 2])
let box2 = box1
box2.values.append(3)
// box1.values also [1,2,3] – the *box* is shared

```

The inner `Array` still has COW *relative to other Array values*, but two names pointing at one `ArrayBox` share the same array *property storage path*. Interviewers use this to test whether you confuse collection COW with object identity.

2.4 Performance interview answer (90s shape)

1. **Claim:** COW makes value-typed collections cheap to pass until mutation.
2. **Mechanism:** shared buffer + uniqueness check on write.
3. **Trade-off:** unexpected copies if you mutate while aliases exist; profiling matters for huge buffers.
4. **Prod:** listing/search arrays — avoid defensive deep copies in hot paths; let COW work; don’t wrap in classes “for safety” without thinking.

3. Nested references and “fake” value types

3.1 Struct + class property

Already covered in foundations — deepen the failure mode:

```
struct UserSession {
    var token: String
    var profiler: Profiler // class
}

final class Profiler {
    var events: [String] = []
    func log(_ e: String) { events.append(e) }
}
```

Copying `UserSession` duplicates `token` but shares `profiler`. Two “sessions” pollute one event log. Fixes:

- Make `Profiler` a struct with COW / value semantics
- Deep-copy explicitly when needed
- Don't put shared services inside DTOs — inject them at the boundary

3.2 Closures capture references

```
final class Loader {
    var label = "x"
    func makePrinter() -> () -> Void {
        { print(self.label) } // captures self strongly by default
    }
}
```

Closures are reference-ish for captures. Pair with ARC / retain-cycle day — but for *this* day, know that “I used a struct” does not eliminate shared mutable state if you capture classes.

4. Enums as state machines (production-grade)

4.1 Impossible states are bugs you don't ship

Boolean flags explode combinatorially:

isLoading	hasData	hasError	Meaning?
T	F	F	loading
F	T	F	success
F	F	T	failure
T	T	T	???

An enum with four cases encodes only the legal rows.

4.2 Mapping network `Result` → UI state

Keep `Result` at the networking edge; map to a UI-facing enum:

```
enum LoadState<T> {
    case idle
    case loading
    case loaded(T)
    case failed(Error)
}

extension LoadState {
    static func from(_ result: Result<T, Error>) -> LoadState {
        switch result {
            case .success(let value): return .loaded(value)
            case .failure(let error): return .failed(error)
        }
    }
}
```

Why not use `Result` alone in the ViewModel?

`Result` has no `idle` / `loading`. UI needs those modes. Domain enums communicate screen semantics; `Result` communicates one-shot outcomes.

4.3 Payment popup machine (Applied · S7-A1)

```
enum PaymentPopupState: Equatable {
    case hidden
    case processing(message: String)
    case success(bookingID: String)
    case failure(code: String, message: String)
    case timedOut(message: String)

    mutating func apply(_ event: PaymentEvent) {
        switch (self, event) {
            case (.hidden, .userStartedCheckout):
                self = .processing(message: "Confirming payment...")
            case (.processing, .backendSuccess(let id)):
                self = .success(bookingID: id)
            case (.processing, .backendFailure(let code, let message)):
                self = .failure(code: code, message: message)
            case (.processing, .timeout):
                self = .timedOut(message: "Still working – check your bookings.")
            case (.success, .dismiss), (.failure, .dismiss), (.timedOut, .dismiss):
                self = .hidden
            default:
                break // ignore illegal transitions; or assert in DEBUG
        }
    }
}

enum PaymentEvent {
    case userStartedCheckout
    case backendSuccess(bookingID: String)
    case backendFailure(code: String, message: String)
    case timeout
    case dismiss
}
```

Interview framing:

- Verified · **S7**: designed a processing-time popup so delays weren't silent (intent: less drop-off / support friction — **no invented** %).
- Applied · **S7-A1**: I'd model transitions as an enum state machine so illegal UI modes can't exist.

4.4 Versioning new cases (SDUI / backend-driven adjacent)

When backend can add modes:

```
enum HeaderComponent {
    case banner(BannerPayload)
    case spacer(height: CGFloat)
    case unknown(type: String, raw: [String: String])
}
```

Unknown cases keep the app resilient. Tie to S3 search/header themes later; today just know: **enums + fallback beats crashing on new raw values**.

4.5 Recursive / nested domain trees

```
indirect enum FeedNode {
    case ad(AdCreative)
    case event(EventSummary)
    case section(title: String, children: [FeedNode])
}
```

Useful for nested listings / comment threads. Mention `indirect` when asked about recursive enums.

5. Classes: identity, sets, and mutation hazards

5.1 Hashing classes

If you put class instances in a `Set` / `Dictionary` key:

- Prefer stable ID-based `Hashable` or identity via `ObjectIdentifier`
- **Never** mutate fields that participate in `hash(into:)` while the object is in the collection — undefined behavior / lost entries

```
final class Seat: Hashable {
    let id: String
    var isSelected: Bool
    static func == (l: Seat, r: Seat) -> Bool { l.id == r.id }
    func hash(into hasher: inout Hasher) { hasher.combine(id) }
    init(id: String) { self.id = id; isSelected = false }
}
```

5.2 When a model *should* be a class

Rare but real:

- Shared mutable cache entry with intentional identity
- ObjC interop / UIKit subclass
- “One session object” everyone must see updates on

Otherwise prefer values + a single owner (ViewModel / store).

6. Actors intro — boundaries you'll be asked about

6.1 Actor vs class

	Class	Actor
Semantics	Reference	Reference + isolation
Cross-task mutation	You synchronize	Compiler / runtime enforce
Call site	Direct	Often await
Inheritance	Possible	Limited (actors don't freely inherit like classes)

6.2 Actor vs serial queue (S2 bridge)

Approach	Pros	Cons
GCD serial queue around a dictionary	Works on older OS; familiar	Easy to misuse sync (deadlock); no compile-time isolation
Swift actor	Type-system isolation; clearer API	Requires concurrency adoption; hop latency; learning curve

Soft pitch (Verified · S2 + Applied · S2-A1):

"We fixed races by serializing access with a GCD queue. For greenfield code I'd expose the same API behind an `actor` so isolation is checked by the compiler."

6.3 Why not every ViewModel is an actor

UI must update on the main actor. Common pattern:

```
@MainActor
final class SearchViewModel: ObservableObject {
    @Published private(set) var state: LoadState<[String]> = .idle
}
```

`@MainActor` is an isolation *domain* for UI affinity. A custom `actor` for every VM adds await noise at every bind site. Isolate the **shared mutable store** (caches, in-flight maps), keep UI models on `@MainActor`.

6.4 Actor reentrancy (awareness only)

While an actor `await`s, other work may run on that actor before your method continues. Don't assume "I set flag X, awaited, flag X still means what I think" without care. Deep dive = Day 05.

7. Trade-off tables (memorize shape)

7.1 Type choice

Choice	When	Cost
Small struct / enum	Models, state, DTOs	Negligible copy
Large struct, many copies	Hot paths with fat values	Copy churn — measure; consider COW / inout / reference boundary
Class for models	True shared identity	Shared mutation bugs, retain cycles
Actor for shared maps	Concurrent mutation	Await hops; API friction
<code>@MainActor</code> VM	UI state	Don't block main with heavy work

7.2 State modeling

Choice	When	Cost
Boolean flags	Prototypes only	Impossible states
Result only	One-shot async outcome	No idle/loading
Domain LoadState	Screens	Slightly more mapping code
Event + enum reduce	Complex flows (payments)	Need discipline on transitions

8. Failure modes checklist

Failure	Symptom	Fix mindset
Class used for DTO	Flaky UI after “copy”	Prefer struct
Nested class in struct	Surprising shared side effects	Audit graph
Assume array assign deep-copies	Unexpected perf or wrong sharing mental model	Teach COW
Boolean UI state	Impossible screens / crashy force unwraps	Enum machine
Actor on every type	Contorted call sites	Isolate the real shared mutable core
Mutating hashed fields in Set	Lost objects / weird lookups	Hash stable identity only

9. Spoken “agenda → answer” templates

Struct vs class (45s)

Agenda: semantics → mutation → example.

Structs have value semantics — copies are independent. Classes share identity — mutation is visible everywhere. I default to structs for ad/listing models; classes for UIKit and true shared services.

COW (45s)

Agenda: share → uniqueness → mutate.

Assignment of Array is cheap because buffers are shared. On mutation, if the buffer isn't uniquely referenced, Swift copies first. So value semantics stay intact without paying full copy on every assign.

Enum state (45s)

Agenda: impossible states → exhaustiveness → payload.

Booleans allow illegal combinations. An associated-value enum makes each mode carry only the data it needs, and switches stay exhaustive. That's how I'd model payment processing UI.

Actor intro (45s)

Agenda: race → isolation → bridge.

An actor is a reference type that serializes access to its state. Call sites await. It's the modern equivalent of the serial-queue boundary we used around shared dictionaries.

10. Diagram: decision flow

Do you need identity or ObjC/UIKit inheritance?

|

```

└─ yes → class (or @MainActor class for UI)
    │
    └─ shared mutable across tasks?
        │
        └─ yes → actor (or isolate behind actor)
            │
            └─ no → plain class / main-actor type
└─ no → is it a finite mode + payload?
    │
    └─ yes → enum
    └─ no → struct
        │
        └─ large / shared buffer needs?
            │
            └─ rely on std COW or implement COW

```

11. Connect to code in this chapter

File	What to notice
code/LoadState.swift	Generic UI state + Result mapping + payment enum sketch
code/COWDemo.swift	Array sharing vs mutation; handmade COW list

Read both before questions.

Next

[03-production-bridge.md](#) — turn S1 / S7 / S2 into tight interview lines without inventing metrics.

03 — Production Bridge

Turn Day 01 concepts into resume-honest interview lines.

Labels: **Verified** = resume-backed · **How I would apply it** = design extension (not a shipped claim).

1. Story map for today

Concept	Story	Label
Value-friendly / type-safe models	Ads module refactor + HeroWidget	Verified · S1
Explicit UI states during uncertainty	Payment processing-time popup	Verified · S7
Enum state machine for payment UI	Design pattern on top of S7	How I would apply it · S7-A1
Isolation of shared mutable maps	Synchronised dictionaries (GCD)	Verified · S2
Actors as modern equivalent	Greenfield redesign of S2	How I would apply it · S2-A1

Full STAR writeups: [../../stories/story-bank.md](#) (S1, S2, S7).

2. Verified · S1 — Ads / type-safe models

What you can say (safe)

- Highest-revenue Ads module; safer reusable rendering path
- Protocol-oriented contracts + generics for a **type-safe** pipeline
- HeroWidget with explicit pause/play tied to visibility / lifecycle

- Prefer models that don't invite accidental shared mutation across UI surfaces

What you must not invent

- Fill-rate percentages, revenue deltas, exact crash rates for ads
- "Every ad model was a struct" (unless you personally know that)

Interview lines

≤20s pitch (value semantics):

"I default to value-friendly DTOs for ad and listing models so accidental shared mutation can't corrupt revenue UI; classes or actors only at identity or concurrency boundaries."

45s conceptual with S1:

"On the Ads refactor we pushed type safety with protocols and generics so new creatives plugged into one pipeline. That mindset extends to model choice: structs and enums for render data keep copies independent across cells and widgets, while classes stay at UIKit and shared services. For video, identity and lifecycle mattered — HeroWidget owned pause/play against visibility, which is a reference-type concern."

Agenda opener:

"I'll cover why value semantics matter for revenue UI, then how POP/generics fit the Ads pipeline."

Provenance: Verified · S1 · BookMyShow · Ads type-safe pipeline / HeroWidget

3. Verified · S7 — Payment processing popup

What you can say (safe)

- Checkout delays caused drop-off / support load when status was unclear
- Designed a lightweight popup for **real-time processing status**
- Clear states: processing / success / failure / timeout messaging
- Coordinated with backend signals
- Intent: reduce ambiguity → less drop-off / support friction

What you must not invent

- Measured drop-off %, conversion lift, support ticket deltas
- Claiming the shipped code used Swift `enum` by name

Interview lines

≤20s:

"For payment delays we shipped a processing popup with explicit status so users weren't staring at a silent spinner."

STAR Action slice (~45–60s) focused on state:

"The product problem was uncertainty during booking/payment confirmation. I designed a lightweight popup driven by backend status signals, with distinct processing, success, failure, and timeout messaging. The key was treating communication of state as part of the feature — silent waiting was the bug."

Agenda opener:

"I'll walk through the payment processing popup — states, signals, and why ambiguity is a product defect."

Provenance: Verified · S7 · BookMyShow · payment processing-time popup

4. Applied · S7-A1 — Enum state machine (design)

Use this when the interviewer asks how you'd model the states in Swift.

Say explicitly that this is the design you'd use / recommend:

“Resume-level work was the popup and the state intent. How I’d model it in Swift is an associated-value enum — `processing`, `success(bookingID)`, `failure`, `timedOut` — so illegal combinations can’t exist and the UI switches exhaustively.”

Optional sketch (also in [code/LoadState.swift](#)):

```
enum PaymentPopupState {
    case hidden
    case processing(message: String)
    case success(bookingID: String)
    case failure(message: String)
    case timedOut(message: String)
}
```

Do not say: “We shipped it as a Swift enum state machine” unless that is personally true.

Provenance: *How I would apply it · S7-A1 · enum state machine for payment UI*

5. Verified · S2 + Applied · S2-A1 — Actors bridge

Verified · S2 (keep short today)

- Shared async state hit from multiple queues → races / intermittent crashes
- Synchronised dictionaries behind GCD serial queues (RW locks where read-heavy)
- Standardized access API so call sites couldn’t touch raw storage

Soft Day-01 actor line

“Where we serialized dictionary access with a GCD serial queue, a Swift actor is the language-native equivalent I’d evaluate for new code — same API surface, compile-time isolation.”

Provenance: *Verified · S2 · BookMyShow · synchronised dictionaries*

Provenance: *How I would apply it · S2-A1 · greenfield shared maps as `actor`*

Day 05 expands concurrency; today only the bridge.

6. Combining stories without metric inflation

Question flavor	Lead with	Support with
struct vs class	S1 value-friendly models	HeroWidget as identity/lifecycle class concern
enums vs booleans	S7-A1 design	S7 product intent
COW / arrays	Listing / search scale context (no fake numbers)	“avoid defensive copies; trust COW”
actors	S2-A1	S2 serial queue as prior art

Scale context you may use when relevant (from S8, not Day 01 core): 30+ lakh DAU, 99.95%+ crash-free — only if the question is about production risk, not as decoration on every answer.

7. Anti-patterns in interviews

Anti-pattern	Fix
“Structs are always faster”	“Prefer for semantics; measure large copies”
“We used actors in the ads module”	Don’t invent — use S2-A1 as design only

"Payment enum cut drop-off 40%"	No invented metrics — intent only for S7
Diving into Apple docs mid-answer	Stay in your mental model + BMS example
Skipping agenda	Always: claim → mechanism → trade-off → prod

8. Flash “map to your work” card

Company / feature: BookMyShow — Ads / listing models; Payment processing popup

What you did: Kept render models in a type-safe ads pipeline; modeled processing UI as explicit states rather than silent waiting.

Interview line (≤20s):

"I default to structs for ad and listing models so accidental shared mutation can't corrupt revenue UI; classes/actors only at identity or concurrency boundaries."

→ STAR: [S1](#), [S7](#)

Next

Drill spoken answers in [04-questions.md](#) . Speak from **Answer points** first; then compare to **Full spoken answer**.

04 — Questions (two-layer Q&A)

How to practice: Cover **Full spoken answer**. Speak from **Answer points** only. Uncover and compare.

Timing: Normal ≈ 30–45s (some 45–60s) · Tricky ≈ 90–120s.

Every question uses the same shape.

Normal questions

Q1. Struct vs class — when do you choose each? (30–45s)

Answer points (frame first):

- Value semantics (struct/enum) vs reference semantics (class)
- Shared mutation only with references
- Prefer struct for models/DTOs; class for identity, UIKit, ObjC
- One production hook: ads/listing models value-friendly

Agenda opener:

"Semantics → shared mutation → when I pick each."

Full spoken answer:

"Structs and enums have value semantics — assignment gives you an independent copy, so mutating one variable doesn't surprise another. Classes have reference semantics — multiple names can point at the same instance, so mutation is shared. I default to structs for models and DTOs, and reach for classes when I need identity, UIKit objects, or Objective-C interop. On the BookMyShow ads side, keeping render models value-friendly fits a type-safe pipeline so accidental shared mutation doesn't corrupt revenue UI."

Common wrong answer:

"Structs are always faster / always stack-allocated, so always use structs." (Semantics first; large structs can be costly; classes aren't "wrong.")

Follow-up ladder:

- **L1:** When is a class mandatory?
- **L2:** What if a struct contains a class property?
- **L3:** Would you make a ViewModel a struct or a class/ `@MainActor` type — why?

Provenance: Verified · S1 · BookMyShow · type-safe ads / listing models

Q2. What is copy-on-write? (30–45s)

Answer points (frame first):

- Collections share a buffer on assign
- Mutation checks uniqueness
- Unique → in place; shared → copy then write
- Array / Dictionary / Set / String

Agenda opener:

"Cheap share until write — then unique copy."

Full spoken answer:

"Copy-on-write means value-typed collections like Array can share their storage after assignment so the assign is cheap. When you mutate, the runtime checks whether that buffer is uniquely referenced. If it is, it mutates in place; if not, it copies the buffer first and then mutates. That preserves value semantics without paying a full element copy on every assignment."

Common wrong answer:

"Assigning an array always deep-copies every element" or "arrays are reference types so A always sees B's append."

Follow-up ladder:

- **L1:** Does `let a = b` on arrays copy elements immediately?
- **L2:** How would you implement COW on your own type?
- **L3:** What does Instruments look like when COW thrash happens on a hot path?

Provenance: How I would apply it · S1 · listing arrays — avoid defensive copies in hot paths

Q3. Why prefer enums over booleans for UI state? (30–45s)

Answer points (frame first):

- Booleans allow impossible combinations
- Associated values carry per-state payloads
- Switch exhaustiveness forces handling
- Payment popup as state machine example

Agenda opener:

"Impossible states out → exhaustiveness in."

Full spoken answer:

"Boolean flags like `isLoading` plus optional `data` and `error` can represent illegal combinations — loading and failed at once, for example. An associated-value enum makes each mode explicit and attaches only the data that mode needs. Switches stay exhaustive when you add cases. For the payment processing popup, I'd model processing, success, failure, and timeout as cases so the UI can't enter a nonsense state."

Common wrong answer:

"Enums are just nicer syntax; booleans are fine if you're careful." (Carefulness doesn't scale; types do.)

Follow-up ladder:

- **L1:** How do you map `Result` into UI state?
- **L2:** How do you version new cases if the backend adds modes?

- **L3:** Show a transition function vs scattering `if` updates.

Provenance: How I would apply it · S7-A1 · enum state machine for payment UI (Verified intent · S7)

Q4. What is an actor at a high level? (30–45s)

Answer points (frame first):

- Reference type with isolated mutable state
- Cross-isolation access uses `await`
- Prevents data races at the boundary
- Bridge: serial queue around dictionaries → actor for new code

Agenda opener:

"Isolated reference type — modern serial queue."

Full spoken answer:

"An actor is a reference type whose mutable state is isolated. You interact with it asynchronously — typically with `await` — and the system serializes access so you don't get data races on that state. At BookMyShow we used GCD serial queues around shared dictionaries for the same kind of problem; for greenfield code I'd evaluate a Swift actor as the language-native equivalent."

Common wrong answer:

"Actors are value types" or "actors replace DispatchQueue for everything including UI timers."

Follow-up ladder:

- **L1:** Are actors value or reference types?
- **L2:** Actor vs `@MainActor` ViewModel?
- **L3:** What is actor reentrancy at a high level?

Provenance: Verified · S2 · synchronised dictionaries; How I would apply it · S2-A1

Q5. Value type containing a class — what happens on copy? (45s)

Answer points (frame first):

- Struct stored properties are copied
- Class properties copy the *reference*
- Nested object still shared
- Deep copy only if you build it

Agenda opener:

"Shallow value copy — nested references still shared."

Full spoken answer:

"When you assign a struct, Swift copies its stored properties. Value-typed fields become independent; class-typed fields copy the reference, so both structs point at the same object. Mutating that nested object through either copy is visible to both. If you need a deep copy, you have to implement it explicitly — value semantics don't recursively deep-copy the graph."

Common wrong answer:

"Structs always deep-copy everything, including nested classes."

Follow-up ladder:

- **L1:** Give a UIKit cell-model example of this bug.
- **L2:** How do you redesign to avoid it?
- **L3:** Interaction with COW arrays stored inside a class property?

Q6. `===` vs `==` ? (30s)

Answer points (frame first):

- `==` is equality (`Equatable`)
- `===` is object identity (classes)
- Structs: equality only, no `===`
- Use identity for “same instance,” equality for “same data”

Agenda opener:

“Identity versus equality.”

Full spoken answer:

“Double equals asks whether two values are equal according to `Equatable`. Triple equals asks whether two class instances are the exact same object. Structs don’t have triple equals because they aren’t identities — you compare their values. I use identity for things like the same view controller instance, and equality for model data.”

Common wrong answer:

Using `===` on structs, or assuming `==` on classes compares identity by default without implementing `Equatable` carefully.

Follow-up ladder:

- **L1:** Should ViewModels use identity or equality in tests?
- **L2:** Hashing a class by identity vs by fields?
- **L3:** `Equatable` enums with associated `Error` — pitfalls?

Q7. When would you use a class for a model? (45s)

Answer points (frame first):

- Intentional shared mutable identity
- ObjC / UIKit / KVO constraints
- Shared session-like object everyone must observe
- Still ask: could a store + values replace it?

Agenda opener:

“When identity is the requirement, not convenience.”

Full spoken answer:

“I’d use a class for a model when shared identity is intentional — one session object many screens must see mutate — or when UIKit/ObjC interop forces it. For ordinary DTOs I still prefer structs. If the real need is safe concurrent mutation of shared storage, I’d consider an actor rather than an unsynchronized class.”

Common wrong answer:

“I use classes for all models because that’s what I’m used to from Java/UIKit tutorials.”

Follow-up ladder:

- **L1:** Could an actor replace that class?
- **L2:** How do you test shared mutable models safely?
- **L3:** Reference cycles between model and observers?

Q8. Recursive enums — why `indirect` ? (45s)

Answer points (frame first):

- Enums need fixed-size layout
- Recursive associated values need a heap box
- `indirect` inserts that indirection
- Examples: AST, nested comments, nested feed sections

Agenda opener:

"Fixed layout → heap box for recursion."

Full spoken answer:

"Swift enums have a size the compiler needs to know. If a case contains another value of the same enum, that would be infinite size unless you introduce indirection. Marking the enum or case `indirect` stores the associated value behind a reference so the layout stays finite. You see this with nested comment threads or tree-shaped feed nodes."

Common wrong answer:

"`indirect` is only for performance" or forgetting it and being confused by the compiler error.

Follow-up ladder:

- **L1:** `indirect` on whole enum vs one case?
- **L2:** How does this compare to a class-based tree?
- **L3:** Equatable/Hashable synthesis with recursive enums?

Provenance: How I would apply it · S1 · nested domain trees in listing/feed-like UI

Q9. Are actors value or reference types? (30s)

Answer points (frame first):

- Reference types
- Plus isolation
- Assignment shares the actor instance
- Mutation only through isolated interface

Agenda opener:

"Reference type with isolation."

Full spoken answer:

"Actors are reference types — assigning an actor shares the same instance — but unlike a plain class, their mutable state is isolated. You don't freely mutate from arbitrary threads; you go through the actor's interface, usually with `await`."

Common wrong answer:

"Actors are value types like structs, so they copy."

Follow-up ladder:

- **L1:** What happens if two tasks call the same actor method?
- **L2:** Can actors inherit from classes freely?
- **L3:** Actor vs lock vs serial queue trade-offs?

Provenance: How I would apply it · S2-A1

Q10. Explain COW in one sentence to a junior. (30s)

Answer points (frame first):

- Share until write
- Then copy if needed
- Keeps value fresh with cheap assigns

Agenda opener:

"One sentence, then a tiny example."

Full spoken answer:

"Arrays act like values, but they cheaply share memory until somebody writes — then they copy if they're not the only owner. So `var b = a` is cheap, and `b.append` won't change `a`."

Common wrong answer:

A long digression into retain counts without the share-until-write punchline.

Follow-up ladder:

- **L1:** Which stdlib types use COW?
- **L2:** Why doesn't every struct get COW automatically?
- **L3:** Implement `ensureUnique` with `isKnownUniquelyReferenced`.

Provenance: How I would apply it · S1 · teaching listing-array behavior on the team

Q11. `let` on a class vs `let` on a struct? (30–45s)

Answer points (frame first):

- `let` binds the name
- Struct: no property mutation through that binding
- Class: reference fixed, object properties may still mutate
- Separate binding mutability from type semantics

Agenda opener:

"Binding versus object mutability."

Full spoken answer:

"`let` means the name can't be reassigned. For a struct, that also blocks mutating properties because mutation is really replacing the value. For a class, `let` only fixes which instance you point at — you can still change the instance's properties. That's why `let` doesn't mean 'immutable object' for reference types."

Common wrong answer:

"`let` always means deep immutability."

Follow-up ladder:

- **L1:** How do you make a class's properties immutable?
- **L2:** `private(set)` on structs vs classes?
- **L3:** Actors and mutation from outside?

Provenance: How I would apply it · S1 · clarifying model API for junior reviewers

Q12. How do you explain preferring values for ads/listing models? (45s)

Answer points (frame first):

- Independent copies across cells/widgets
- Fewer cross-screen mutation bugs
- Type-safe pipeline mindset from ads refactor
- Classes reserved for lifecycle/identity (`HeroWidget`)

Agenda opener:

"Revenue UI → independent data → identity only where needed."

Full spoken answer:

"For ad and listing render data I prefer value semantics so one surface's mutation doesn't leak into another. That sits well with the type-safe ads pipeline we built with protocols and generics. Where identity and lifecycle matter — like HeroWidget pause/play tied to visibility — that's a reference-type concern. So values for data, classes for identity boundaries."

Common wrong answer:

Inventing fill-rate or revenue percentage claims.

Follow-up ladder:

- **L1:** How do generics fit that pipeline?
- **L2:** What breaks if an ad DTO secretly holds a shared cache object?
- **L3:** Testing strategy for value models vs singleton services?

Provenance: Verified · S1 · BookMyShow · Ads type-safe models / HeroWidget

Tricky questions

T1. You mutate `arrayB` after `let arrayA = arrayB` — does A change? (90s)

Answer points (frame first):

- Trap: always-shared vs always-copied
- COW: share until write
- Mutating B copies if not unique → A unchanged
- If both wrapped in same class box, different story
- Mental model: uniqueness at mutation time

Agenda opener:

"I'll separate assignment sharing from mutation uniqueness — then the class-wrapper trap."

Full spoken answer:

"With Array, assignment can share a buffer. When you mutate B, if that buffer isn't uniquely referenced, Swift copies first, then mutates B's copy — so A stays the same. People wrongly say either 'A always changes' or 'assignment always deep-copied.' The truth depends on uniqueness at mutation time. If A and B are two variables of a class that holds an array, mutating through the shared class instance will show up on both names — that's identity sharing, not Array COW across independent values."

Common wrong answer:

Absolute "yes A changes" or absolute "no, assign always copies elements."

Follow-up ladder:

- **L1:** What if you mutate A instead of B?
- **L2:** How does `isKnownUniquelyReferenced` relate?
- **L3:** How would you demo this in a playground without relying on print myths?

Provenance: How I would apply it · S1 · listing arrays mental model

T2. Large struct passed through many functions — performance? (90–120s)

Answer points (frame first):

- Trap: "structs always faster"
- Small structs win; large stored props may copy often
- Mitigations: COW wrapper, `inout`, reference/actor boundary

- Measure with Allocations — don't guess
- Semantics still often worth it until proven hot

Agenda opener:

"Semantics first, then copy cost, then mitigations."

Full spoken answer:

"I wouldn't claim structs are always faster. Small models are cheap and safer. A large struct with many non-COW stored properties can get expensive if you pass and assign it everywhere. Mitigations include keeping bulky buffers behind COW, using inout for in-place updates, or placing a reference or actor at the boundary for truly shared heavy state. I'd confirm with Instruments Allocations before rewriting architecture. For listing rows I still bias to values unless profiling says otherwise."

Common wrong answer:

Prematurely converting all models to classes "for performance" without evidence.

Follow-up ladder:

- **L1:** Where do you look in Instruments?
- **L2:** When is a class the right perf boundary?
- **L3:** How does COW change the analysis for arrays of large structs?

Provenance: How I would apply it · S1 / S5 mindset · measure before rewriting (no invented ms)

T3. Enum with associated `Error` vs `Result` for ViewModel state (90s)

Answer points (frame first):

- Trap: one true way
- `Result` = one-shot success/failure
- UI needs idle/loading too → domain `LoadState`
- Map at the boundary
- Payment popup: domain enum communicates screen semantics

Agenda opener:

"Boundary types versus screen types."

Full spoken answer:

"There's no single winner. `Result` is excellent for a finished async attempt. A ViewModel usually also needs idle and loading, so I prefer a domain enum like `LoadState` with `idle`, `loading`, `loaded`, and `failed`. Internally I may still use `Result` from the client, then map at the boundary. For payment UI I'd go further with explicit processing, success, failure, and timeout cases — that's screen language, not just `Result`."

Common wrong answer:

"Always use `Result` in the ViewModel" or "never use `Result`."

Follow-up ladder:

- **L1:** How do you unit-test transitions?
- **L2:** Equatable with `Error` associated values?
- **L3:** Cancellation mid-loading — new case or event?

Provenance: How I would apply it · S7-A1; Verified intent · S7

T4. Why not make every ViewModel an actor? (90s)

Answer points (frame first):

- Trap: actors solve all threading
- UI needs main-actor affinity

- Extra await hops and API friction
- Isolate shared mutable stores, not every VM
- Bridge from serial-queue discipline

Agenda opener:

"Isolate the shared mutable core — don't actor-wrap the world."

Full spoken answer:

"Actors are for protecting shared mutable state across concurrency. ViewModels that drive UI usually belong on the main actor so updates are UI-safe without sprinkling await on every property bind. Making every ViewModel a custom actor adds hop latency and awkward call sites without buying much if the VM isn't the race boundary. I'd keep `@MainActor` view models and put an actor — or a serial queue, as we did historically — around the shared dictionaries or caches that multiple tasks touch."

Common wrong answer:

"Actors are the new best practice, so everything is an actor."

Follow-up ladder:

- **L1:** `@MainActor` class vs `actor` type differences?
- **L2:** How do you call an actor from a `MainActor` VM?
- **L3:** Reentrancy bug example after await?

Provenance: Verified · S2; How I would apply it · S2-A1

T5. Class instances in a `Set` — what's required? (90s)

Answer points (frame first):

- Need `Hashable` / `Equatable`
- Prefer stable ID or `ObjectIdentifier`
- Don't mutate hashed fields while inserted
- Identity vs field equality choice is product-driven

Agenda opener:

"Hashable contract first — then mutation hazards."

Full spoken answer:

"Class instances can live in a `Set` if they conform to `Hashable`. You typically hash a stable ID or use object identity. The critical trap is mutating fields that participate in the hash while the object is in the set — you can lose the object for lookups. So either hash identity, or treat hashed fields as immutable for the membership lifetime."

Common wrong answer:

"Classes can't go in `Sets`" or hashing mutable `isSelected` flags.

Follow-up ladder:

- **L1:** Dictionary keys with classes — same rules?
- **L2:** `NSObject` subclasses hashing pitfalls?
- **L3:** Value-type keys vs reference-type keys in API design?

Provenance: How I would apply it · S1 · seat/selection style models (design)

T6. How would you model the payment processing popup as a state machine? (90–120s)

Answer points (frame first):

- Product intent: no silent waiting (S7)
- Cases: hidden / processing / success / failure / timeout
- Associated payloads per case

- Events drive pure transitions
- Design label S7-A1 — don't overclaim shipped enums

Agenda opener:

"Product states first, then Swift enum + events."

Full spoken answer:

"At BookMyShow the problem was checkout delays with unclear status — we designed a processing popup with clear processing, success, failure, and timeout messaging so users weren't abandoned. In Swift I'd model that as an associated-value enum and apply payment events through a transition function: start checkout enters processing, backend success carries a booking ID, failures and timeouts carry messages, dismiss returns to hidden. That design keeps illegal UI states unrepresentable. I present the enum machine as how I'd apply it — the resume-backed piece is the popup and the state intent."

Common wrong answer:

Inventing conversion metrics, or claiming "we shipped Swift enums" as a verified fact.

Follow-up ladder:

- **L1:** Where does timeout fire — client timer vs backend?
- **L2:** How do you test illegal transitions?
- **L3:** Multi-step payment (3DS) — extend enum or nested state?

Provenance: Verified · S7; How I would apply it · S7-A1

T7. Struct with two large arrays — when do copies get expensive? (90s)

Answer points (frame first):

- Arrays themselves COW
- Assigning the struct shares array buffers until write
- Mutating either array may copy that buffer
- Multiple mutations with aliases → copy churn
- Still usually better than careless class sharing

Agenda opener:

"Struct copy versus buffer copy — COW still helps."

Full spoken answer:

"A struct that stores two arrays will copy its storage references on struct assignment, but the array buffers can still be shared thanks to COW. Cost shows up when you mutate while aliases exist — each mutated array may uniquely copy its buffer. If you repeatedly fork and mutate large collections, you can churn memory. I'd avoid unnecessary defensive copies, keep mutations localized, and profile before introducing a class wrapper that might accidentally share identity across UI."

Common wrong answer:

"Any struct with arrays is expensive on every assign because arrays deep-copy immediately."

Follow-up ladder:

- **L1:** inout mutation of one array field?
- **L2:** Hand-rolled COW for a custom buffer type?
- **L3:** Threading: is Array COW enough for concurrent mutation? (No — need isolation.)

Provenance: How I would apply it · S1 · listing payloads; concurrency caution → S2

T8. Compare GCD serial queue isolation vs a Swift actor for a shared map. (90–120s)

Answer points (frame first):

- Same problem: shared mutable dictionary

- Queue: proven, older OS, easy deadlock with sync
- Actor: compile-time isolation, await API
- S2 verified queues; S2-A1 actors as design for new code
- Choose based on codebase concurrency maturity

Agenda opener:

"Same mutual exclusion goal — different enforcement."

Full spoken answer:

"Both protect a shared map. With a GCD serial queue you funnel reads and writes onto one executor; it works well and is what we used to stop races on synchronised dictionaries, but sync-from-same-queue deadlocks and isolation isn't compiler-checked. A Swift actor makes that isolation part of the type system and call sites use await. For new modules I'd lean actor if the codebase has adopted concurrency; for existing queue-based APIs I wouldn't rewrite blindly. The lesson from production was serialize at the boundary — the tool can evolve."

Common wrong answer:

"Actors make GCD obsolete overnight" or "queues are outdated so our old fix was wrong."

Follow-up ladder:

- **L1:** Reader-writer locks vs actor for read-heavy maps?
- **L2:** How do you keep the same API surface during migration?
- **L3:** Testing concurrency regressions?

Provenance: Verified · S2; How I would apply it · S2-A1

Drill set (pick per session)

Session	Speak aloud
A (definitions)	Q1, Q2, Q3, Q4, Q10
B (edges)	Q5, Q6, Q8, Q11, Q12
C (tricky)	T1, T2, T4, T6
D (concurrency bridge)	Q4, Q9, T4, T8

Score with [../../timing/answer-timing-guide.md](#) . Log misses in gotchas.

Optional citations (appendix only — not required to study)

- Swift book: Classes and Structures; Enumerations; Concurrency (Actors)
- Chapter code: [code/LoadState.swift](#) , [code/COWDemo.swift](#)

05 — Exercises

Do these after reading foundations + deep dive + production bridge. Prefer speaking out loud even for coding prompts.

Exercise 1 — LoadState fluency (20–25 min)

Prompt: Using [code/LoadState.swift](#) as a base (or rewrite from memory):

1. Implement `LoadState<T>` with `idle` | `loading` | `loaded(T)` | `failed(Error)` .
2. Add `static func from(result: Result<T, Error>) -> LoadState<T>` .
3. Add `map` that transforms `loaded` values and preserves other cases.

4. Write a tiny ViewModel-shaped function:

```
func reduce(state: LoadState<[String]>, event: SearchEvent) -> LoadState<[String]>
```

Define `SearchEvent` as `startSearch` | `succeed([String])` | `fail(Error)` | `reset` .

Agenda to say before coding (30s):

"I'll define the enum, map Result at the boundary, then a pure reduce for events."

Done when: You can explain why `Result` alone is insufficient for UI.

Stretch: Make `failed` hold a domain error enum instead of `Error` for Equatable tests.

Exercise 2 — Payment state machine (20–30 min)

Prompt: Implement `PaymentPopupState` + `PaymentEvent` + `apply` from the Applied · S7-A1 sketch.

Required transitions:

From	Event	To
hidden	userStartedCheckout	processing
processing	backendSuccess	success(bookingID)
processing	backendFailure	failure
processing	timeout	timedOut
success/failure/timedOut/processing	dismiss	hidden

Speak (≤90s) after coding:

Product intent (Verified · S7) → why enum beats booleans → one illegal state you eliminated.

Honesty check: End with "enum machine is how I'd apply it (S7-A1)."

Exercise 3 — COW playground (15–20 min)

Prompt: Run or mentally simulate [code/COWDemo.swift](#) .

- Two `Array` vars — assign, mutate one, predict prints.
- `ArrayBox` class wrapper — predict shared mutation.
- Implement or read `COWList` with `isKnownUniquelyReferenced` .

Speak (30s):

"Share until write; class boxes share identity; handmade COW is a value façade over a buffer class."

Trap to avoid: Claiming assignment always deep-copies elements.

Exercise 4 — Nested reference audit (15 min)

Prompt: Find the bug and fix the design:

```
final class Metrics {
    var taps: [String] = []
}

struct AdCellModel {
```

```

    var title: String
    var metrics: Metrics
}

var a = AdCellModel(title: "A", metrics: Metrics())
var b = a
b.title = "B"
b.metrics.taps.append("click")
// What is a.title? a.metrics.taps?

```

Write:

- Observed behavior
- Root cause (one sentence)
- Two redesigns (prefer values / don't store shared services in DTOs)

Story hook: Why value-friendly ads models matter (Verified · S1) without inventing metrics.

Exercise 5 — Actor intro sketch (15–20 min)

Prompt: Given a queue-style API:

```

final class SyncedMap {
    private var storage: [String: Int] = [:]
    private let queue = DispatchQueue(label: "synced.map")

    func set(_ key: String, _ value: Int) { /* serialise */ }
    func get(_ key: String) -> Int? { /* serialise */ }
}

```

1. Fill in serial-queue implementations (sync/async choices — speak trade-offs).
2. Rewrite as `actor SyncedMap` with the same method names.
3. Speak the S2 → S2-A1 bridge in ≤45s.

Do not claim you rewrote production dictionaries as actors.

Exercise 6 — Timed speaking drill (25–35 min)

Record (phone voice memo is fine):

1. Q1 struct vs class
2. Q2 COW
3. Q3 enums vs booleans
4. T1 array mutate after assign
5. T6 payment state machine

Score with [../../..../timing/answer-timing-guide.md](https://github.com/robertadams/actor-timing-guide/blob/master/timing/answer-timing-guide.md) :

Check	Pass?
Agenda in first 10s	
Trade-off mentioned	
Production hook without invented metrics	
Finished inside budget	

Log misses → gotchas file / notes.

Exercise 7 — Social Feed clarifying questions (20–30 min)

SD spine warm-up (requirements only mindset):

Write **5 clarifying questions** you would ask before designing a BMS-scale listing/feed (use scale context carefully: 30L+ DAU is Verified · S8 if you mention it).

Examples of good question shapes (write your own):

- Offline / stale content expectations?
- Ad slots vs organic mixing rules?
- Pagination vs realtime invalidation?
- Video autoplay lifecycle (HeroWidget-like)?
- Failure modes for SDUI-ish components?

Keep this as questions only — full SD is later weeks.

Solutions pointers (don't spoil yourself)

Exercise	Look at
1–2	code/LoadState.swift
3	code/COWDemo.swift
4	Deep dive §3 nested references
5	Deep dive §6; production bridge §5
6	04-questions.md full answers
7	Your notes; Week 1 SD spine later

Exit criteria

You may mark Day 01 complete when you can, **without notes**:

- ☐ Decision rule for struct / class / enum / actor
- ☐ COW one-liner + uniqueness nuance
- ☐ Why enums beat boolean UI flags
- ☐ ≤20s S1 value-semantics pitch
- ☐ ≤90s S7 + S7-A1 payment state answer with honest provenance
- ☐ ≤45s S2 → actor bridge without overclaiming

Then use the revision twin for spaced drills:

[../../revision/weeks/week-01/day-01.md](#)

Day 02 — Protocols, POP, Generics, Associated Types, Type Erasure

Week 1 · Full study (self-contained) · ~4–5 hrs

Revision twin: [revision/weeks/week-01/day-02.md](#)

Outcomes

By end of day, without notes, you can:

- Explain protocol-oriented design vs inheritance for UI / ad component pipelines

- Use generics + associated types + `where` clauses and say when each belongs
- Contrast `some` vs `any`, and when PATs force generics or type erasure
- Explain type erasure (AnyPublisher-style boxing) and its cost
- Deliver a ≤5 min architecture talk on BMS Ads / HeroWidget (Verified · S1) without inventing fill-rate metrics
- Soft-bridge to Stories SDK (Verified · S10) for reusable protocol APIs at a boundary

How to study (in Cursor only)

1. [01-foundations.md](#) — mental model (intern → mid)
2. [02-deep-dive.md](#) — PATs, dispatch, erasure, trade-offs
3. [03-production-bridge.md](#) — Verified · S1 / S10 hooks
4. [code/](#) — mini ads pipeline; walk through aloud
5. [04-questions.md](#) — speak from **Answer points**; compare to full answer
6. [05-exercises.md](#) — drills + timed practice
7. Revision twin for flashcards / timed drill day-of

Module map

Module	File	Role
Foundations	01-foundations.md	POP mental model + first generics
Deep dive	02-deep-dive.md	associatedtype, some/any, erasure, dispatch
Production bridge	03-production-bridge.md	S1 Ads + HeroWidget + S10 soft bridge
Questions	04-questions.md	Two-layer Q&A (normal + tricky)
Exercises	05-exercises.md	Coding + speaking drills
Code	code/AdsPipeline.swift , code/TypeErasureDemo.swift	Runnable mental models

Time budget (suggested)

Block	Minutes
Foundations	60–75
Deep dive	75–90
Production bridge + code	35–45
Questions (speak 7–9 aloud)	60–75
Exercises + timed drill	45–60

Provenance reminder

Only use Verified IDs from [../.../provenance/README.md](#) .

Label	Use for today
Verified · S1	Ads module refactor; POP + Generics type-safe pipeline; HeroWidget pause/play lifecycle
Verified · S10	Soft bridge: Stories SDK — reusable protocol APIs across portfolio
Verified · S3	Soft mention only: protocol-driven header / SDUI-ish shape (deepen later)
How I would apply it · S3-A1	Schema versioning + unknown-component fallback (design; not Day 02 core claim)
Learning-lab	AdsPipeline.swift / type-erasure demos — not claimed as shipped BMS source

Do **not** invent fill-rate %, revenue deltas, or crash rates for ads.

Agenda openers (pin these)

Topic	5–10s opener
POP	"Compose capabilities with protocols — prefer that over deep inheritance."
associatedtype vs generic	"Associated type is chosen by the conformer; generic param by the caller."
some vs any	"Opaque concrete vs existential box."
Type erasure	"Box a PAT into one type — pay allocation and lose specialization."
S1 Ads	"Highest-revenue Ads path — protocols, generics, video lifecycle."

01 — Foundations: POP, Generics, Associated Types

Read this first. Goal: build a crisp mental model an intern can teach back, then layer the senior edges you'll need in interviews.

0. One-sentence north star

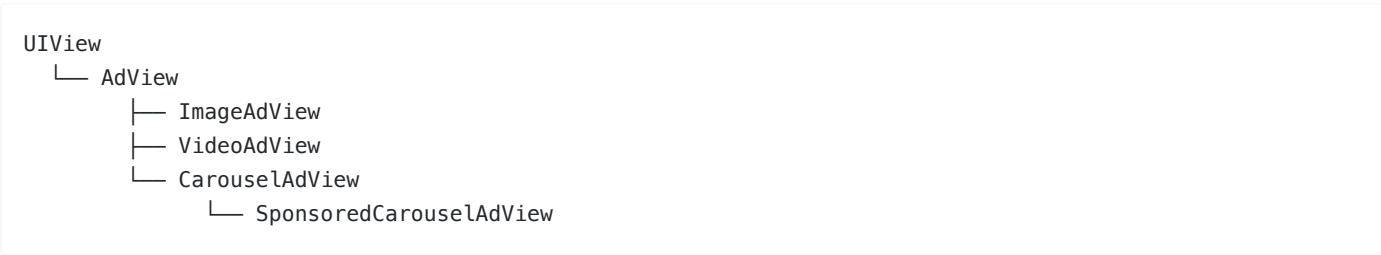
Design by composing capabilities (protocols + generics), not by growing inheritance trees — especially on revenue-critical UI pipelines.

Everything below unpacks that sentence with Ads / HeroWidget as the production hook (Verified · S1).

1. Why POP exists (intern story)

1.1 The inheritance trap

Classic UIKit tutorials teach:



Problems at scale:

Pain	Why it hurts
Fragile base class	Shared configure grows god-methods
Forced shared identity	Everything is a class even when data isn't
Hard to test	Must subclass or mock deep trees
New creative type	Fork or fight the hierarchy
Multiple "is-a" needs	Swift has single inheritance for classes

Say aloud: "Inheritance models *what you are*. Protocols model *what you can do*."

1.2 Capability thinking

An ad unit might need several **capabilities**:

Capability	Protocol sketch	Who cares
Render into a view	AdRenderable	Listing / hero surface
Impression / click tracking	AdTrackable	Analytics / revenue
Video lifecycle	PlaybackControllable	HeroWidget pause/play
Prefetch assets	Prefetchable	Scroll performance

A video hero ad composes `AdRenderable` & `AdTrackable` & `PlaybackControllable` .

An image tile might only need `AdRenderable` & `AdTrackable` .

That composition is **protocol-oriented programming (POP)**.

1.3 The intern demo (do this once out loud)

```
protocol Describable {
    var summary: String { get }
}

struct ImageCreative: Describable {
    let title: String
    var summary: String { "Image: \(title)" }
}

struct VideoCreative: Describable {
    let title: String
    let duration: TimeInterval
    var summary: String { "Video: \(title) \(Int(duration))s" }
}

func logAll(_ items: [any Describable]) {
    items.forEach { print($0.summary) }
}
```

Say aloud: “Different types, same capability. No shared base class required.”

When `Describable` gains an `associatedtype` , the story changes — see §4 and deep dive. Keep this simple demo for the first 10 minutes.

2. Protocols as contracts

2.1 Requirements vs extensions

```
protocol AdTrackable {
    var creativeID: String { get }
    func trackImpression()
    func trackClick()
}

extension AdTrackable {
    // Default – convenient, but see dispatch traps in deep dive
    func trackClick() {
        print("default click \(creativeID)")
    }
}
```

```
}  
}
```

Piece	Role
Protocol body	Requirements — part of the contract; usually dynamic dispatch via witness table
Extension defaults	Convenience; may not override polymorphically if not declared as requirements

Intern takeaway: Put methods you need to override/customize in the protocol requirement list.

Senior takeaway: Extension-only defaults can surprise you at call sites typed as the protocol — Day 02 deep dive covers this trap in full.

2.2 Protocol composition

```
typealias HeroAd = AdRenderable & AdTrackable & PlaybackControllable  
  
func installHero<A: HeroAd>(_ ad: A) {  
    // Compiler knows A has all three capability sets  
}
```

Composition is **and**, not multiple inheritance myths. You’re requiring a type that satisfies multiple contracts.

2.3 Class-bound protocols (`AnyObject`)

```
protocol AdDelegate: AnyObject {  
    func adDidFail(_ error: Error)  
}  
  
final class AdLoader {  
    weak var delegate: AdDelegate?  
}
```

Need	Protocol style
weak delegate / identity	: AnyObject (class-bound)
Struct-friendly models	Avoid AnyObject; prefer value conformers
UIKit subclassing	Class conformers are fine; still prefer protocols for <i>capabilities</i>

3. Generics — caller chooses the type

3.1 Mental model

A **generic parameter** is a blank the **caller** fills in:

```
struct Renderer<Creative> {  
    let creative: Creative  
}  
  
let imageRenderer = Renderer(creative: ImageCreative(title: "Show"))  
let videoRenderer = Renderer(creative: VideoCreative(title: "Trailer", duration: 15))
```

Add constraints so you can use the type:

```
struct Renderer<Creative: AdTrackable> {
    let creative: Creative

    func bind() {
        creative.trackImpression()
    }
}
```

3.2 Why generics beat `Any` in revenue paths

```
// Fragile
func render(_ creative: Any) -> UIView {
    if let image = creative as? ImageCreative { ... }
    if let video = creative as? VideoCreative { ... }
    fatalError("unknown")
}

// Type-safe pipeline
func render<C: AdRenderable>(_ creative: C) -> C.ContentView {
    creative.makeView()
}
```

Casts hide bugs until a new creative ships on Friday night. Generics push mismatches to compile time.

Provenance: *Verified · S1 · BookMyShow · POP + Generics for a type-safe ads pipeline.*
*You may say the pipeline was type-safe. You may **not** invent fill-rate percentages.*

3.3 `where` clauses (first pass)

```
func merge<C>(_ a: C, _ b: C) -> C where C: AdTrackable & Equatable {
    // Only types that are trackable AND equatable
    return a == b ? a : b
}
```

`where` keeps the generic signature readable when constraints grow (especially with associated types — deep dive).

4. Associated types — conformer chooses the type

4.1 Mental model

An **associated type** is a blank the **conforming type** fills in:

```
protocol AdRenderable {
    associatedtype ContentView: UIView
    func makeView() -> ContentView
}

struct ImageAd: AdRenderable {
    // Conformer picks ContentView
    func makeView() -> UIImageView { UIImageView() }
}
```

```
struct VideoAd: AdRenderable {
    func makeView() -> UIView { PlayerContainerView() }
}
```

Concept	Who chooses the concrete type?
func f<T: P>(_ x: T)	Caller (generic parameter)
associatedtype inside P	Conformer

Say aloud: “Generics: I tell the function what T is. Associated types: the type that adopts the protocol decides.”

4.2 Why PATs feel “hard”

`AdRenderable` has an associated type → historically you could not freely write:

```
let ads: [AdRenderable] = ... // ❌ often illegal / constrained
```

Because each conformer may have a *different* `ContentView`, so the existential can’t present one uniform shape without erasure or newer constrained `any` features.

Workarounds you’ll learn today:

1. Keep the pipeline **generic**: `func install<R: AdRenderable>(_ r: R)`
2. **Type-erase** to a common box when you truly need heterogeneous arrays
3. Use carefully constrained `any AdRenderable` where the language allows (know the limits)

4.3 Mini example tying both together

```
protocol Creative {
    associatedtype Body
    var body: Body { get }
}

struct Pipeline<C: Creative> {
    let creative: C
    func materialize() -> C.Body { creative.body }
}
```

Generic `Pipeline` is parameterized by a `Creative` that itself has an associated `Body`. This is the “POP + generics” shape used in component pipelines.

5. `some` vs `any` (foundations cut)

Keyword	Meaning	Intern line
<code>some P</code>	Opaque type — <i>one</i> concrete type, hidden from the caller	“There’s a real type; you just can’t name it.”
<code>any P</code>	Existential — box that can hold different conformers (with limits)	“Could be different types behind the curtain.”

```
func makeBadge() -> some Describable {
    ImageCreative(title: "Badge") // always ImageCreative
}

func makeItems() -> [any Describable] {
```

```
[ImageCreative(title: "A"), VideoCreative(title: "B", duration: 10)]
}
```

Performance intuition (enough for today):

- Generics / `some` → compiler can specialize (static-ish)
- `any` → witness table / existential container (dynamic)

Deep dive expands PAT + existential interactions.

6. Protocol extensions as shared behavior

```
protocol Prefetchable {
    var assetURLs: [URL] { get }
    func prefetch()
}

extension Prefetchable {
    func prefetch() {
        assetURLs.forEach { URLSession.shared.dataTask(with: $0).resume() }
    }
}
```

Use extensions for:

- Shared default behavior across many conformers
- Conditional helpers (`where` on extension)
- Keeping conformers thin

Do **not** use extensions as a place to hide requirements you actually need to customize polymorphically — declare those as requirements.

7. Conditional conformance (first pass)

```
struct Pair<A, B> {
    var a: A
    var b: B
}

extension Pair: Equatable where A: Equatable, B: Equatable {
    static func == (lhs: Pair<A, B>, rhs: Pair<A, B>) -> Bool {
        lhs.a == rhs.a && lhs.b == rhs.b
    }
}
```

Same idea as `Array: Equatable where Element: Equatable`. Model layers love this.

8. When inheritance is still OK

POP is not a religion.

Still use class inheritance when	Example
UIKit requires a subclass	UIView, UIViewController

Shared identity + ObjC runtime	Delegates, responders
Framework base types you don't control	UICollectionViewCell

Senior pattern: Subclass UIKit for the *view*, put *capabilities* on protocols the cell/view model conforms to. HeroWidget can be a real view object with lifecycle *and* sit behind playback protocols.

9. Glossary (pin)

Term	One-liner
POP	Design by composing protocol capabilities + extensions
PAT	Protocol with associated type(s)
Generic specialization	Compiler generates concrete code for used types
Witness table	Runtime dispatch table for protocol requirements
Existential	any P — value that can hold different conformers
Opaque type	some P — one hidden concrete type
Type erasure	Manual/API box turning PAT into a single usable type
Protocol composition	A & B — must satisfy both
Conditional conformance	Type conforms only when constraints hold
Class-bound	AnyObject — only classes; enables weak

10. First production bridge (keep short)

At BookMyShow, the highest-revenue **Ads** module was refactored around **protocol-oriented** contracts and **generics** so rendering stayed type-safe as creatives grew. **HeroWidget** owned video pause/play against visibility / lifecycle — a reference-type lifecycle concern sitting on top of a POP pipeline.

≤20s pitch:

"We made ad rendering a generic protocol pipeline so new creatives plugged in without forking the revenue path — and HeroWidget tied video playback to visibility."

Provenance: Verified · S1 · BookMyShow · Ads POP + Generics / HeroWidget

Full STAR and anti-claims live in [03-production-bridge.md](#) .

11. Self-check before deep dive

Without notes, can you:

- ☐ Explain POP in one sentence with a capability example?
- ☐ Contrast "caller picks T" vs "conformer picks associatedtype"?
- ☐ Say why [AdRenderable] is awkward when AdRenderable has associated types?
- ☐ Give one reason generics beat Any casts in an ads pipeline?
- ☐ Name when you'd still subclass UIView ?

If yes → [02-deep-dive.md](#) .

If no → re-speak §§1–4 aloud once.

02 — Deep Dive: PATs, Dispatch, some/any, Type Erasure

Mechanics and traps. Intern foundations assumed. This is the interview depth layer.

1. Associated types under pressure

1.1 Self and associated types constrain existentials

```
protocol AdRenderable {
    associatedtype ContentView: UIView
    associatedtype Model
    func render(_ model: Model) -> ContentView
}
```

Problems you must be able to explain:

1. **Heterogeneous collections** — different `Model` / `ContentView` per conformer.
2. **Returning `AdRenderable` from a function** — which associated types?
3. **Storing as a property** typed only as the protocol — same issue.

Historical teaching (still valid as *mental model*): PATs were not usable as simple existentials the way `Error` or `Equatable` -free protocols were.

Modern Swift (`any` , primary associated types, constraints on existentials) improved some cases — you still need a strategy:

Strategy	When
Stay generic end-to-end	Hot pipeline, max safety + specialization
Type erase at the boundary	Heterogeneous list / plugin registry
Constrained any P	Language allows the operations you need
Enum of known creatives	Closed set, rare additions

1.2 Primary associated types (awareness)

```
protocol AdRenderable<Model> {
    associatedtype Model
    associatedtype ContentView: UIView
    func render(_ model: Model) -> ContentView
}

// Call sites can constrain the existential more usefully in newer Swift:
func handle(_ ad: any AdRenderable<HeroModel>) { ... }
```

Interview line: "Primary associated types make some existentials usable by fixing part of the type shape — they don't make every PAT free as `[any P]` for arbitrary operations."

1.3 Same idea with `IteratorProtocol` / `Sequence`

Swift's own standard library uses type erasure (`AnyIterator` , `AnySequence`) precisely because associated types block easy existentials. `AnyPublisher` in Combine is the same idea for reactive pipelines.

If you can explain *why* `AnyPublisher` exists, you can explain ads type erasure.

2. Generics + PATs together (the ads pipeline shape)

2.1 Preferred shape

```
protocol Creative {
    associatedtype View: UIView
    func makeView() -> View
}

protocol PlaybackControllable {
    func pause()
    func play()
}

struct HeroPipeline<C: Creative> {
    let creative: C

    func install(into parent: UIView) -> C.View {
        let view = creative.makeView()
        parent.addSubview(view)
        return view
    }
}

// Video-only overlay of capabilities
func bindPlayback<C: Creative & PlaybackControllable>(_ c: C) {
    c.pause()
}
```

Teaching points:

- Outer API is generic → compiler sees concrete `C`.
- Video-only behavior is composed, not forced onto image creatives.
- No `as? VideoCreative` in the revenue path.

2.2 `where` clauses for associated types

```
func dump<C: Creative>(_ c: C) where C.View: UIImageView {
    // Only creatives whose View is UIImageView
}

extension Array {
    func renderAll<V: UIView>() -> [V] where Element: Creative, Element.View == V {
        map { $0.makeView() }
    }
}
```

This is how you keep specialization without casting.

2.3 Generic typealiases for readability

```
typealias TrackedCreative = AdRenderable & AdTrackable
typealias HeroCapable = TrackedCreative & PlaybackControllable
```

Use in signatures so interview whiteboards stay readable.

3. Static vs dynamic dispatch (senior must-know)

3.1 Three dispatch flavors you'll mention

Mechanism	Typical trigger	Intuition
Static (direct)	Concrete type, <code>final</code> , generics specialized	Compiler knows exact method
Witness table	Protocol requirement called via existential / generic unconstrained path	Dynamic but protocol-scoped
ObjC / dynamic	@objc / NSObject message send	Runtime messaging

You do **not** need assembly. You need this sentence:

"Generics can specialize; protocol existentials go through witness tables; extension defaults that aren't requirements may bind to the static type."

3.2 The extension default trap (classic interview)

```
protocol Greeter {
    func greet()
}

extension Greeter {
    func greet() { print("protocol default") }
    func wave() { print("extension-only wave") } // NOT a requirement
}

struct Person: Greeter {
    func greet() { print("person greet") }
    func wave() { print("person wave") }
}

let p: any Greeter = Person()
p.greet() // "person greet" - requirement, dynamic
p.wave()  // "extension-only wave" - static via Greeter, surprise!
```

Trap: Expecting `wave` to call `Person.wave` through the existential.

Fix: Declare `wave()` as a protocol requirement if polymorphic behavior matters.

3.3 Why this matters for ads

Shared `track()` defaults in an extension are fine for identical behavior. Custom per-creative tracking that must override must be a **requirement** (or a generic call on the concrete type).

4. `some` vs `any` (deep)

4.1 Opaque (`some`)

```
func heroWidget() -> some PlaybackControllable {
    VideoHero() // one concrete type forever for this function
}
```

Rules of thumb:

- Caller cannot change which concrete type is returned.
- Good for hiding implementation (SwiftUI `some View` energy).

- Still one type → specialization-friendly.

4.2 Existential (any)

```
var tile: any AdTrackable
tile = ImageAd()
tile = VideoAd() // OK if both conform and associated types don't block the operations you use
```

Costs:

- Existential container / witness table
- Less specialization
- Operations involving `Self` or associated types may be unavailable

4.3 Interview contrast table

Need	Prefer
Hide one concrete return type	some P
Heterogeneous stored values	any P or type eraser
Max performance in hot bind	Generics on concrete / some
API flexibility for plugins	Erasure or registry of factories

5. Type erasure — full mental model

5.1 When you need it

You need a single type `AnyAd` such that:

```
let feed: [AnyAd] = [AnyAd(ImageAd()), AnyAd(VideoAd())]
feed.forEach { $0.trackImpression() }
```

But `AdRenderable` has associated types, so you cannot store raw PATs.

5.2 Hand-rolled eraser (learning-lab pattern)

```
protocol AdTrackable {
    var creativeID: String { get }
    func trackImpression()
}

struct AnyAd: AdTrackable {
    private let _id: String
    private let _track: () -> Void

    var creativeID: String { _id }

    init<A: AdTrackable>(_ base: A) {
        _id = base.creativeID
        _track = { base.trackImpression() }
    }

    func trackImpression() { _track() }
}
```

For richer APIs (render returning different view types), erase to a **common** output (`UIView`) inside the box:

```
struct AnyRenderable {
  private let _render: () -> UIView
  init<R: AdRenderable>(_ base: R) {
    _render = { base.makeView() } // views upcast to UIView
  }
  func makeView() -> UIView { _render() }
}
```

You intentionally collapse `ContentView` to `UIView` at the boundary.

5.3 Costs you must say aloud

Cost	Detail
Allocation	Closures / box class often heap-allocate
Indirection	Extra call through stored function
Lost specialization	Compiler sees AnyAd, not VideoAd
API narrowing	Associated richness reduced to common denominator

Senior rule: Keep generics **inside** the hot pipeline; erase **at** module boundaries or heterogeneous lists only.

5.4 Combine mental transfer

`AnyPublisher<Output, Failure>` exists so you can return/store publishers without leaking nested generic operator types. Same motive as `AnyAd` .

6. Open vs closed component sets (SDUI bridge)

6.1 Closed enum

```
enum AdKind {
  case image(ImageModel)
  case video(VideoModel)
}
```

- Pros: exhaustive switches, simple.
- Cons: every new creative is an app release + enum edit.

6.2 Open protocol registry

```
protocol AdComponentFactory {
  func supports(_ type: String) -> Bool
  func make(from json: [String: Any]) -> AnyRenderable?
}

final class AdRegistry {
  private var factories: [AdComponentFactory] = []
  func register(_ f: AdComponentFactory) { factories.append(f) }
  func make(type: String, json: [String: Any]) -> AnyRenderable? {
    factories.first { $0.supports(type) }?.make(from: json)
  }
}
```

```
}  
}
```

Pros: CMS / backend can introduce types your factories understand.

Cons: unknown types need fallback; versioning matters.

*Soft bridge to BookMyShow backend-driven header (Verified · S3) and Applied · S3-A1 unknown-component fallback — deepen in SDUI weeks. Today only note the **shape** matches POP.*

7. Mixing POP with UIKit identity (HeroWidget)

HeroWidget needs:

1. A real view / VC lifecycle (class identity)
2. Pause/play when visibility changes
3. Clean contracts so the ads pipeline doesn't care about player internals

Sketch:

```
protocol PlaybackControllable: AnyObject {  
    func play()  
    func pause()  
}  
  
final class HeroWidget: UIView, PlaybackControllable {  
    private let player: AVPlayer  
  
    func play() { player.play() }  
    func pause() { player.pause() }  
  
    func didEnterVisibleViewPort() { play() }  
    func didLeaveVisibleViewPort() { pause() }  
}
```

Interview line:

"POP gave us the pipeline; HeroWidget was still a class because lifecycle and player identity are reference concerns — pause/play tied to visibility."

Provenance: Verified · S1 · HeroWidget pause/play lifecycle

8. Trade-off tables (memorize)

8.1 Architecture choices

Choice	When	Cost
POP + generics pipeline	Many variants; test seams; revenue safety	Learning curve; PAT friction
Inheritance tree	Rare shared UIKit identity	Fragile base; hard reuse
Type erasure	Heterogeneous arrays / boundaries	Allocation, indirection
any Protocol	Flexibility	Existential overhead; PAT limits
Closed enum	Stable small set	App release for every new case

8.2 YAGNI vs revenue scale

Situation	Advice
Two similar ad types forever	Maybe don't abstract yet
Highest-revenue module, growing creatives	Abstraction pays for itself (S1 justification)
SDK public surface (Stories)	Protocols at boundary — Verified · S10 soft bridge

Trap answer: “Always POP everything.”

Senior answer: “Introduce POP when variants or test seams demand it; at BMS ads scale, variants justified the pipeline.”

9. Conditional conformance & protocol inheritance (extra)

9.1 Protocol inheritance

```
protocol AdRenderable { ... }
protocol VideoAdRenderable: AdRenderable, PlaybackControllable {
    var duration: TimeInterval { get }
}
```

Refine capabilities without inventing a class hierarchy.

9.2 Conditional helpers

```
extension AdRenderable where Self: PlaybackControllable {
    func renderAndPrime() -> ContentView {
        let v = makeView()
        pause() // safe — Self is playback-capable
        return v
    }
}
```

10. SDK boundary lessons (S10 soft)

Stories SDK reused across a portfolio → public API should expose **protocols** (and carefully chosen value models), not a forest of concrete types clients must subclass.

Say:

“Same instinct as ads: contracts at the boundary, concretes inside. On Stories we leaned on reusable SDK surfaces across brands.”

Provenance: Verified · S10 · Stories SDK portfolio reuse
Do not invent client counts or latency numbers.

11. Common interview whiteboard flow (5 min)

Use this for Day 14 architecture dry-run / today's timed drill:

- Agenda:** “Revenue ads path — problems with inheritance → POP contracts → generics pipeline → HeroWidget lifecycle → trade-offs.”
- Problem:** New creatives forking render code; video lifecycle bugs.
- Design:** Creative / AdTrackable / PlaybackControllable ; Pipeline<C: Creative> .
- Lifecycle:** HeroWidget pause/play on visibility.
- Trade-off:** Generics inside; erasure only if heterogeneous feed requires it.

6. **Honesty:** No invented fill-rate %; maintainable type-safe pipeline is the claim.

12. Anti-patterns checklist

Anti-pattern	Fix
Any + cast ladder in bind	Generic constraint or typed factory
Deep AdView subclass tree	Capability protocols
Erasing every generic	Erase at boundary only
Extension-only overrides	Promote to requirements
Claiming any == free	Know existential + PAT limits
"Structs can't do POP"	Structs are first-class conformers
Forgetting UIKit subclass needs	Class for view identity; POP for capabilities

13. Code tour (do before questions)

1. Open [code/AdsPipeline.swift](#) — explain each protocol + generic pipeline aloud.
2. Open [code/TypeErasureDemo.swift](#) — explain why `AnyTrackable` exists.
3. Speak one trap: extension dispatch **or** PAT-in-array.

Then continue to [03-production-bridge.md](#) .

14. Deep self-check

- ☐ Can you draw POP vs inheritance for three ad types?
- ☐ Can you explain associatedtype vs generic parameter in 20s?
- ☐ Can you justify type erasure cost in one breath?
- ☐ Can you demo the extension-default dispatch surprise?
- ☐ Can you deliver S1 architecture agenda in 15s?

If yes → production bridge. If no → re-read §§3–5 and re-speak.

03 — Production Bridge

Turn Day 02 concepts into resume-honest interview lines.

*Labels: **Verified** = resume-backed · **How I would apply it** = design extension · **Learning-lab** = demo code.*

1. Story map for today

Concept	Story	Label
POP + generics type-safe ads pipeline	Ads module refactor	Verified · S1
Video pause/play tied to visibility	HeroWidget	Verified · S1
Reusable protocol APIs at SDK boundary	Stories SDK	Verified · S10
Protocol-driven header / open components	Backend-driven header	Verified · S3 (soft)
Unknown component fallback	SDUI versioning design	How I would apply it · S3-A1 (soft)

Mini pipeline / eraser demos	Local snippets	Learning-lab
------------------------------	----------------	--------------

Full STAR: [.././././stories/story-bank.md](#) (S1, S10; S3 optional).

2. Verified · S1 — Ads / HeroWidget (core)

What you can say (safe)

- Highest-revenue Ads module needed a safer, reusable rendering path
- Refactored around **protocol-oriented** ad component contracts + **generics** for a **type-safe** pipeline
- New ad types plug into the pipeline without forking the revenue path
- Built reusable **HeroWidget** with explicit pause/play tied to visibility / VC lifecycle
- Coordinated behavior with stakeholders without breaking fill behavior (qualitative — no invented %)
- Lesson: for revenue-critical UI, prefer POP + generics over inheritance trees; lifecycle is part of the product contract

What you must not invent

- Fill-rate percentages, revenue deltas, CTR, exact crash rates for ads
- “We type-erased every renderer” (unless personally true — prefer Learning-lab for erasure demos)
- “Every creative was a struct”
- Fake team size, sprint counts, or App Store rankings

Interview lines

≤20s pitch:

“We made ad rendering a generic protocol pipeline so new creatives plugged in without forking the revenue path — and HeroWidget tied video playback to visibility.”

Architecture talk opener (5–10s):

“I’ll walk through our highest-revenue Ads refactor — protocols, generics, and video lifecycle.”

≈90s Action slice (STAR):

“The Ads module was highest-revenue and needed a safer reusable rendering path. I refactored rendering around protocol-oriented contracts and generics so the pipeline stayed type-safe as creatives grew — new types conformed and plugged in instead of forking bind code. Separately, video inside HeroWidget needed correct pause and play against visibility and view-controller lifecycle, so we made that lifecycle explicit on the widget. Stakeholder coordination mattered because behavior changes on a revenue surface aren’t casual. The result was a maintainable type-safe pipeline and fewer playback glitches on video creatives.”

Result (honest):

“Shipped a maintainable, type-safe ads pipeline; lifecycle-correct video reduced wasted playback and UI glitches on a module that mattered for revenue.”

Trade-off to volunteer:

“Generics inside the pipeline for safety and specialization; I’d only type-erase at a heterogeneous list or module boundary — erasure isn’t free.”

Provenance: Verified · S1 · BookMyShow · Ads POP + Generics / HeroWidget

3. Mapping concepts → S1 lines

Question flavor	Lead with	Support with
What is POP?	Capability composition	S1 pipeline contracts
Why generics?	Compile-time safety vs Any casts	Revenue path correctness

associatedtype pain	Keep generic / erase at edge	Learning-lab eraser if asked “how”
Inheritance vs POP	Fragile base on ad variants	S1 lesson line
Video lifecycle	HeroWidget pause/play	Class identity + protocol capability
some vs any	Opaque vs existential	Prefer generics in hot bind
Extension dispatch	Requirement vs default	Shared track defaults

4. Verified · S10 — Stories SDK soft bridge

What you can say (safe)

- Built a Stories SDK reused across a portfolio of apps
- Reusable surfaces / modularity mattered
- Same instinct: **contracts at the boundary**, concretes inside

What you must not invent

- Number of client apps as a precise metric unless you know it
- Latency / engagement % for stories

≤20s bridge:

“Same POP instinct showed up later in a Stories SDK — reusable protocol-oriented surfaces across brands rather than copy-pasted concretes.”

Provenance: Verified · S10 · Raw / Miami Heat · Stories SDK portfolio reuse

Use S10 when asked “Have you designed reusable module APIs?” — not as a replacement for S1 on ads-specific questions.

5. Soft · S3 / S3-A1 — open registries

Only if interviewer pivots to SDUI / CMS components:

Verified · S3: backend-driven header; generalised protocol-driven main-screen implementation.

Applied · S3-A1: how you’d handle unknown component types (fallback + versioning) — say it is design, not a shipped claim.

One liner:

“An open protocol registry matches CMS growth better than an forever-closed enum — with an explicit unknown fallback. That’s the design I’d apply for versioning (S3-A1); the shipped header work was protocol-driven (S3).”

Do not let this hijack Day 02 — keep S1 as the spine.

6. Learning-lab — what the code files are

File	Claim level
code/AdsPipeline.swift	Teaching sketch of S1 shape — not shipped BMS source
code/TypeErasureDemo.swift	Erasure mechanics demo

If asked “Did you write it like this?”:

“This is the teaching shape of the contracts we used — protocol + generic pipeline. I’m not claiming this file is production source.”

7. Anti-patterns in interviews

Anti-pattern	Fix
Inventing fill-rate %	Qualitative: revenue-critical, maintainable, type-safe
"POP means never use classes"	HeroWidget is a class; capabilities are protocols
"Type erasure is free abstraction"	Name allocation + lost specialization
Diving into Generics Manifesto trivia	Stay: safety → pipeline → lifecycle → trade-off
Skipping agenda on architecture Q	Always agenda in 10s
Claiming actors/SwiftUI for S1	Don't invent stack; stick to POP + generics + lifecycle

8. Flash "map to your work" card

Company / feature: BookMyShow — Ads module / HeroWidget

What you did: Refactored highest-revenue module with POP + Generics for type-safe rendering; video pause/play lifecycle on HeroWidget.

Interview line (≤20s):

"We made ad rendering a generic protocol pipeline so new creatives plugged in without forking the revenue path."

→ STAR: [S1](#) · optional [S10](#)

9. Timed story drills

Drill	Budget	Pass bar
S1 ≤20s pitch	20s	Protocols + generics + HeroWidget
S1 Action only	90s	No invented metrics; trade-off optional
Full S1 STAR	≤3 min	Agenda + result + lesson
Architecture dry-run	≤5 min	Whiteboard flow from deep dive §11
S10 bridge	20s	Boundary protocols; no fake numbers

Next

Drill spoken answers in [04-questions.md](#) . Speak from **Answer points** first; then compare to **Full spoken answer**.

04 — Questions (two-layer Q&A)

How to practice: Cover **Full spoken answer**. Speak from **Answer points** only. Uncover and compare.

Timing: Normal ≈ 30–60s · Tricky ≈ 90–120s.

Every question uses the same shape.

Normal questions

Q1. What is protocol-oriented programming? (30–45s)

Answer points (frame first):

- Design by composing capabilities (protocols + extensions)
- Prefer over deep class inheritance for variants
- Structs and classes can both conform
- Ads pipeline hook: render / track / playback as capabilities

Agenda opener:

"Compose capabilities with protocols — prefer that over deep inheritance."

Full spoken answer:

"Protocol-oriented programming means designing around capabilities — protocols and extensions — instead of growing a deep inheritance tree. Types opt into what they can do: renderable, trackable, playback-controllable. Structs and classes can both conform, so models stay value-friendly while UIKit views stay classes. On the BookMyShow Ads refactor we used protocol contracts so new creatives plugged into one pipeline instead of forking an AdView subclass hierarchy."

Common wrong answer:

"POP means never use classes or inheritance." (UIKit + HeroWidget still need class identity.)

Follow-up ladder:

- **L1:** How is composition different from multiple inheritance?
- **L2:** When is a class hierarchy still justified?
- **L3:** How do you unit-test a protocol-based pipeline?

Provenance: Verified · S1 · BookMyShow · Ads POP pipeline

Q2. associatedtype vs generic parameter? (45–60s)

Answer points (frame first):

- Generic parameter: caller chooses
- associatedtype: conforming type chooses
- PATs complicate simple existentials / arrays
- Pipeline stays generic to avoid that pain

Agenda opener:

"Associated type is chosen by the conformer; generic param by the caller."

Full spoken answer:

"A generic parameter is filled in by the caller — `Renderer<ImageCreative>` . An associated type is filled in by the type that conforms to the protocol — `ImageAd` decides what its `ContentView` is. That's powerful for flexible contracts, but protocols with associated types are awkward as plain existentials, so you often keep APIs generic — `install<R: AdRenderable>` — or erase at a boundary if you need a heterogeneous list."

Common wrong answer:

"They're the same thing" or "just write `var x: [AdRenderable]` with associated types and move on."

Follow-up ladder:

- **L1:** Can you put a PAT in an array?
- **L2:** What are primary associated types helping with?
- **L3:** Show `where` constraining an associated type.

Provenance: Learning-lab · PAT mental model; Verified · S1 pipeline shape

Q3. some vs any ? (45s)

Answer points (frame first):

- `some` = opaque — one concrete hidden type
- `any` = existential — may hold different conformers
- Performance: opaque/generics specialize better
- Use `any` when you need heterogeneity

Agenda opener:

"Opaque concrete versus existential box."

Full spoken answer:

"`some Protocol` means one concrete type that conforms, but the caller can't name it — like SwiftUI's `some View`. `any Protocol` is an existential box that can hold different conforming types, with more dynamic dispatch and limits around `Self` and associated types. I use opaque or generics when the concrete type is stable and I care about specialization; I use `any` or type erasure when I truly need heterogeneous storage."

Common wrong answer:

"They're synonyms" or "`any` is always better because it's flexible."

Follow-up ladder:

- **L1:** Why does SwiftUI return `some View` ?
- **L2:** What's the performance intuition?
- **L3:** When does `any` refuse an associated-type operation?

Provenance: Learning-lab · language mechanics

Q4. Why generics in an ads pipeline? (45s)

Answer points (frame first):

- Compile-time type safety
- Avoid `Any` cast ladders on revenue path
- New creatives conform and plug in
- Specialization in hot bind paths

Agenda opener:

"Type safety on the revenue path — no cast ladders."

Full spoken answer:

"Generics let the ads pipeline stay type-safe as creatives grow. Instead of `Any` and downcasting in bind code — which breaks when a new creative ships — each creative conforms to protocol contracts and flows through a generic installer. That kept the highest-revenue module maintainable: new types plug in without forking the revenue path, and the compiler catches mismatches."

Common wrong answer:

"Generics make everything faster automatically" (semantics/safety first; measure perf).

Follow-up ladder:

- **L1:** What did HeroWidget add on top?
- **L2:** Would you expose the generic pipeline across modules?
- **L3:** When would you introduce type erasure?

Provenance: Verified · S1 · BookMyShow · type-safe ads pipeline

Q5. Protocol extension default methods — dispatch catch? (60s)

Answer points (frame first):

- Requirements dispatch via witness table
- Extension-only methods bind to static/existential type
- Customization that must override → declare as requirement
- Shared identical defaults can live in extensions

Agenda opener:

"Defaults are handy — until you expect polymorphic override."

Full spoken answer:

"If a method is a protocol requirement, calling it through an existential uses the conformer's implementation via the witness table. If it's only defined in a protocol extension and not a requirement, the call can bind to the extension default based on the static type — even if the concrete type defines the same method name. So for behavior that must customize polymorphically, declare it as a requirement. Use extension defaults for truly shared behavior."

Common wrong answer:

"Everything in a protocol extension is dynamically dispatched like Java default interfaces."

Follow-up ladder:

- **L1:** Show a minimal surprise example.
- **L2:** How do you force dynamic dispatch?
- **L3:** Implications for a shared `track()` helper?

Provenance: Learning-lab · dispatch trap

Q6. What is type erasure? (45–60s)

Answer points (frame first):

- Wrap disparate conformers in one concrete box
- Needed when PATs block easy existentials / heterogeneous arrays
- AnyPublisher / AnySequence same idea
- Cost: allocation, indirection, lost specialization

Agenda opener:

"Box a PAT into one type — pay for the convenience."

Full spoken answer:

"Type erasure means wrapping types that conform to a protocol — especially one with associated types — in a single concrete type that forwards calls, so you can store them in arrays or return one type from APIs. Combine's AnyPublisher is the standard example. The cost is extra allocation and indirection, and you lose generic specialization. I keep generics inside the hot ads pipeline and erase only at boundaries that need heterogeneity."

Common wrong answer:

"Type erasure is free abstraction" or "always erase for clean APIs."

Follow-up ladder:

- **L1:** Sketch AnyTrackable in 30s.
- **L2:** What do you collapse associated views to?
- **L3:** Erasure vs constrained any today?

Provenance: Learning-lab · TypeErasureDemo.swift ; pattern applies to S1 boundaries

Q7. Protocol composition (A & B)? (30s)

Answer points (frame first):

- Require multiple capabilities at once

- Not multiple inheritance of stored state
- `Renderable` & `Trackable` ads example
- Typealiases for readability

Agenda opener:

"And-together capabilities — one type must satisfy both."

Full spoken answer:

"Protocol composition means a type must satisfy multiple protocols — `AdRenderable` & `AdTrackable`. It's capability intersection, not C++-style multiple inheritance of implementations and stored properties. For hero video ads I'd compose render, track, and playback protocols; image tiles might only need render and track."

Common wrong answer:

"Swift supports multiple inheritance via protocol composition."

Follow-up ladder:

- **L1:** Composition vs protocol inheritance?
- **L2:** How does this interact with generics?
- **L3:** Class-bound + composition for delegates?

Provenance: Verified · S1 · capability composition on ads

Q8. When is inheritance still OK? (45s)

Answer points (frame first):

- UIKit / AppKit required subclasses
- Shared identity / ObjC runtime
- Mix: class for view, protocols for capabilities
- HeroWidget as lifecycle class

Agenda opener:

"Inheritance where the platform demands identity — POP for capabilities."

Full spoken answer:

"I still use inheritance when UIKit requires a subclass — views, cells, view controllers — or when I need reference identity and Objective-C interop. The senior move is to subclass for the platform object and attach capabilities via protocols. HeroWidget is a class because player lifecycle and visibility hooks are identity concerns, while the ads pipeline around it stays protocol- and generic-oriented."

Common wrong answer:

"POP means rewrite UIKit without subclasses."

Follow-up ladder:

- **L1:** Can a UIView subclass conform to your ad protocols?
- **L2:** Fragile base class symptoms?
- **L3:** Testing UIKit subclasses vs protocol fakes?

Provenance: Verified · S1 · HeroWidget lifecycle class

Q9. Conditional conformance? (45s)

Answer points (frame first):

- Type conforms only when constraints hold
- `Array: Equatable` where `Element: Equatable`
- Keeps generics honest in model layers

- Useful for wrapper types in pipelines

Agenda opener:

"Conform only when the pieces allow it."

Full spoken answer:

"Conditional conformance means a generic type adopts a protocol only when its parameters meet constraints — Array is Equatable when its Element is. That keeps model layers honest: a Pair of Equatables can be Equatable without forcing Pair to always be Equatable. In pipelines I use the same idea for wrappers that should only be Hashable or Codable when their creative payload is."

Common wrong answer:

"Conditional conformance is automatic for every generic."

Follow-up ladder:

- **L1:** Write `Pair` Equatable where clause.
- **L2:** Codable conditional pitfalls?
- **L3:** Difference from `where` on a function?

Provenance: Learning-lab · model-layer Swift

Q10. Generics for repository / boundary interfaces? (45s)

Answer points (frame first):

- Associated Model or generic methods at boundaries
- Keeps domain types explicit
- Prefer protocols at SDK edges
- Stories SDK soft bridge

Agenda opener:

"Boundaries expose contracts; concretes stay inside."

Full spoken answer:

"At module boundaries I like protocol interfaces with an associated model or generic methods so call sites stay typed — Repository returning a concrete domain model rather than [String: Any] . Inside, implementations can vary. That same instinct showed up on a Stories SDK reused across apps: stable protocol-oriented surfaces rather than leaking concretes."

Common wrong answer:

"Always expose concrete view models across packages."

Follow-up ladder:

- **L1:** associatedtype Model vs generic method?
- **L2:** How do you mock in tests?
- **L3:** Versioning broken contracts?

Provenance: Verified · S10 soft bridge; Clean/MVVM instinct · S9 optional if asked District

Q11. protocol P: AnyObject vs struct-friendly protocols? (45s)

Answer points (frame first):

- AnyObject = class-bound
- Enables weak delegates
- Structs cannot conform to class-bound protocols
- Choose based on identity / weak needs

Agenda opener:

"Class-bound when you need weak identity — otherwise keep it value-friendly."

Full spoken answer:

"Marking a protocol `AnyObject` restricts conformance to classes, which lets you hold `weak` references — classic delegates. Struct-friendly protocols omit that bound so value types can conform. For ads tracking on value creatives I keep protocols struct-friendly; for HeroWidget playback delegates or UIKit-style delegates I use class-bound protocols."

Common wrong answer:

"All protocols should be `AnyObject` because `UIKit`."

Follow-up ladder:

- **L1:** Why can't weak point at a struct?
- **L2:** Closure callbacks vs weak delegates?
- **L3:** Actor-bound protocols awareness?

Provenance: Learning-lab · weak delegate rules; S1 HeroWidget class-bound playback

Tricky questions

T1. Why can't you freely write `var x: [AdRenderable]` with associated types? (120s)

Answer points (frame first):

- Each conformer may pick different associated types
- Existential can't present one uniform associated shape
- Options: generics, type erasure, constrained `any`, closed enum
- Prefer generics in pipeline; erase at heterogeneous edge

Trap:

"Just use the protocol name like Java interfaces."

Full spoken answer:

"If `AdRenderable` has associated types like `Model` and `ContentView`, each conformer can choose different concrete types. A homogeneous array existential would need one shared shape for those associated types, which you don't have. Historically that blocked easy `[AdRenderable]`. Today I'd keep the installer generic, or erase to something like `AnyRenderable` that collapses views to `UIView`, or use constrained `any` when the language lets me fix part of the shape. On a revenue pipeline I prefer generics inside and erasure only if the feed must be heterogeneous."

Follow-up ladder:

- **L1:** How does `AnyPublisher` relate?
- **L2:** Primary associated types — what improves?
- **L3:** Would an enum of creatives be better?

Provenance: Learning-lab · PAT + erasure; applies to S1 feed boundaries

T2. Default implementation in extension vs requirement — override surprise (90s)

Answer points (frame first):

- Non-requirement extension methods: static binding risk
- Requirements: witness-table dispatch
- Promote customized APIs to requirements
- Demo mental model with greet/wave

Trap:

Expect polymorphic call to concrete override via existential for extension-only methods.

Full spoken answer:

"If `wave` lives only in a protocol extension and isn't a requirement, calling `wave` on an `any Greeter` can hit the extension default even when the concrete type defines `wave`. Requirements like `greet` dispatch to the conformer. The surprise bites teams who put 'overridable' behavior only in extensions. Fix: declare polymorphic methods as requirements; keep extensions for true shared defaults."

Follow-up ladder:

- **L1:** Does generics calling a concrete type avoid the surprise?
- **L2:** Interaction with inherited protocol defaults?
- **L3:** How would you unit-test dispatch expectations?

Provenance: Learning-lab · dispatch trap

T3. Type-erasing a generic renderer — when is it worth it? (90s)

Answer points (frame first):

- Worth it: heterogeneous lists, plugin registries, API boundaries
- Not worth it: inside a monomorphic hot pipeline
- Cost: allocation, indirection, lost specialization
- S1 instinct: generics first

Trap:

Erase everything for 'cleaner' types.

Full spoken answer:

"I'd type-erase a renderer when I must store different creatives in one collection or cross a module boundary that can't be generic end-to-end. Inside the hot ads bind path I'd keep `Pipeline<C: Creative>` so the compiler specializes and types stay precise. Erasure pays heap and witness-style indirection and collapses associated richness — fine at the edge, expensive if you wrap every call. At BMS ads scale the justified abstraction was POP + generics; erasure is a boundary tool, not the architecture."

Follow-up ladder:

- **L1:** What common denominator do you erase views to?
- **L2:** Class-based eraser vs struct-of-closures?
- **L3:** Measuring the cost in Instruments?

Provenance: How I would apply it · S1 boundary design; Learning-lab eraser

T4. POP for SDUI component registry vs enums (120s)

Answer points (frame first):

- Enum = closed world, exhaustive, app release to extend
- Protocol registry = open world for CMS growth
- Need unknown fallback + versioning
- Soft S3 / S3-A1 honesty

Trap:

Enum of all components forever as the only design.

Full spoken answer:

"An enum of every component is a closed set — great exhaustiveness, painful when CMS invents types. A protocol-oriented registry lets factories register for type strings and construct components, which matches backend-driven UI growth. The trade-off is handling unknowns: you need a fallback component and schema versioning so old apps don't crash on new types. At

BookMyShow we did protocol-driven header work; the explicit unknown-fallback design is how I'd apply versioning — I'm not claiming a particular fallback implementation shipped."

Follow-up ladder:

- **L1:** What does the fallback render?
- **L2:** How do you version JSON schemas?
- **L3:** Mixing ads creatives into SDUI slots?

Provenance: Verified · S3 soft; How I would apply it · S3-A1

T5. Generics overkill for two ad types? (90s)

Answer points (frame first):

- YAGNI for tiny stable sets
- Revenue + growing variants justified abstraction at BMS
- Introduce when third variant or test seams demand
- Don't abstract for fashion

Trap:

Always abstract on day one / never abstract because YAGNI.

Full spoken answer:

"If you truly have two stable ad types forever, a small enum or shared function might be enough — YAGNI applies. On the highest-revenue Ads module, creatives and lifecycle concerns were growing, and forking bind code was the risk. That's when POP and generics paid off: type-safe plug-in points and testable contracts. I'd introduce the abstraction when a third variant or a painful test seam appears — or earlier on a revenue path where a cast bug is expensive. It's judgment, not dogma."

Follow-up ladder:

- **L1:** What signals say 'time to abstract'?
- **L2:** How do you migrate an inheritance tree incrementally?
- **L3:** Metrics you'd watch after refactor (honest: qualitative + crash/stability, not fake fill-rate)?

Provenance: Verified · S1 · justified abstraction on revenue module

T6. rethrows + generic higher-order functions (90s)

Answer points (frame first):

- Function only throws if closure throws
- Clean mapper APIs
- Avoid forcing try on non-throwing call sites
- Useful in generic transform pipelines

Trap:

Mark everything `throws` or ignore error propagation.

Full spoken answer:

"`throws` lets a higher-order function declare that it throws only when its function argument throws. So a generic `map` over creatives can take a throwing transform without forcing every non-throwing caller to write `try`. That keeps pipeline APIs clean: errors propagate when real, stay silent when not. I'd use it on generic mappers at module boundaries; I wouldn't sprinkle `throws` on APIs that never fail."

Follow-up ladder:

- **L1:** Difference between `throws` and `throws ?`
- **L2:** async + throws interaction at a high level?
- **L3:** Typed throws awareness (newer Swift) — optional?

Timed set recommendations

Set	Questions	Focus
A	Q1, Q4, Q6, T1, T4	POP + erasure + SDUI
B	Q2, Q3, Q5, T2, T3	Types + dispatch
C	Q7, Q8, Q11, T5, S1 5-min	Composition + story

Score with [../../timing/answer-timing-guide.md](#) .

Exit criteria for questions module

- ☐ 8+ normals spoken from Answer points without reading Full
- ☐ 3+ trickies with trap named explicitly
- ☐ S1 ≤20s + ≤5 min architecture dry-run once
- ☐ Zero invented fill-rate metrics in any recording

05 — Exercises

Do these after foundations + deep dive + production bridge. Prefer speaking out loud even for coding prompts.

Exercise 1 — Mini ads pipeline (25–35 min)

Prompt: Using [code/AdsPipeline.swift](#) as a base (or rewrite from memory):

- Define `Creative` with associatedtype `Body` + `AdTrackable` .
- Implement `ImageCreative` (struct) and `VideoCreative` (class + `PlaybackControllable`).
- Implement `AdPipeline<C: Creative & AdTrackable>` with `install() -> C.Body` .
- Add `bindPlayback` generic constrained to `PlaybackControllable` .

Agenda to say before coding (30s):

“Capabilities first — Creative and Trackable — then a generic pipeline, then video-only playback composition.”

Done when: You can explain why `VideoCreative` is a class while `ImageCreative` is a struct.

Stretch: Add `CarouselCreative` without touching `AdPipeline` ’s body — only new conformance.

Exercise 2 — Type eraser (20–30 min)

Prompt: Implement `AnyTrackable` as in [code/TypeErasureDemo.swift](#) .

- Store `id` + `track` closure.
- Build `[AnyTrackable]` with two concrete types.
- Speak costs in ≤30s.

Speak (≤60s) after coding:

When you’d erase vs keep generics on an ads feed.

Honesty check: Label as Learning-lab — not shipped BMS source.

Exercise 3 — Extension dispatch surprise (15–20 min)

Prompt: Reproduce the greet/wave trap from deep dive §3.2 in a playground or scratch file.

Write predicted output **before** running.
Then fix by promoting `wave` to a protocol requirement.

Speak (30s):

"Requirements witness-dispatch; extension-only methods may bind statically."

Exercise 4 — HeroWidget lifecycle sketch (20–25 min)

Prompt:

```
final class HeroWidget {
  // own a VideoCreative or player façade
  // didEnterVisibleViewport / didLeaveVisibleViewport
}
```

- 1. Implement pause/play wiring.
- 2. Conform to `PlaybackControllable`.
- 3. Speak Verified · S1 Action slice in 90s tying widget to pipeline.

Honesty check: No fill-rate %. Lifecycle-correct video is the claim.

Exercise 5 — Open vs closed registry (20–25 min)

Prompt: Sketch both:

- 1. `enum AdKind { case image; case video }` switch renderer.
- 2. `AdRegistry` with `AdComponentFactory` protocol and unknown fallback returning a placeholder.

Speak (90s):

Trade-offs; mention S3 soft + S3-A1 as design for unknown fallback.

Exercise 6 — Timed speaking drill (30–40 min)

Record:

- 1. Q1 POP
- 2. Q4 generics in ads
- 3. Q6 type erasure
- 4. T1 PAT array
- 5. T5 YAGNI vs revenue

Optional: 5 min S1 architecture dry-run.

Score with [../..../timing/answer-timing-guide.md](#) :

Check	Pass?
Agenda in first 10s	
Trade-off mentioned	
Production hook without invented metrics	
Finished inside budget	

Exercise 7 — Stories SDK boundary (15 min)

Prompt: Write 5 bullet API rules for a reusable Stories SDK surface (protocols vs concretes, versioning, what not to expose).

Speak ≤20s S10 bridge after writing.

Provenance: *Verified · S10 · portfolio reuse — no invented client counts.*

Solutions pointers

Exercise	Look at
1, 4	code/AdsPipeline.swift
2	code/TypeErasureDemo.swift
3	Deep dive §3.2
5	Deep dive §6
6	04-questions.md
7	Production bridge §4

Exit criteria

You may mark Day 02 complete when you can, **without notes**:

- ☐ POP one-liner + capability example
- ☐ associatedtype vs generic parameter in 20s
- ☐ Type erasure cost in one breath
- ☐ Extension dispatch trap named
- ☐ ≤20s S1 pitch + ≤5 min architecture talk once
- ☐ Honest provenance (no fill-rate invention)

Then use the revision twin:

[../../../../revision/weeks/week-01/day-02.md](#)

Day 03 — ARC, Retain Cycles, weak/unowned, Instruments

Week 1 · Full study (self-contained) · ~4–5 hrs

Revision twin: [revision/weeks/week-01/day-03.md](#)

Outcomes

By end of day, without notes, you can:

- Explain how ARC inserts retains/releases, when `deinit` runs, and what strong / weak / unowned mean
- Name classic retain-cycle patterns (escaping closures, strong delegates, Timer target-selector, NotificationCenter block observers, parent↔child) and the fix for each
- Distinguish **true leaks** (unreachable) from **abandoned memory / retain cycles** (reachable but unused) — and which tool finds which
- Walk an interview triage: Memory Graph → Allocations → Leaks, without claiming the wrong instrument

- Tie reliability culture to Verified **S8** (30L+ DAU, 99.95%+ CFS, Crashlytics, IMOC) without inventing “I used Memory Graph at BMS” unless labeled Applied

How to study (in Cursor only)

1. 01-foundations.md — mental model + glossary (intern-first)
2. 02-deep-dive.md — full mechanics, tables, failure modes
3. 03-production-bridge.md — S8 reliability culture + Applied triage playbook
4. code/RetainCycleDemo.swift — read the cycle, then the fixes
5. 04-questions.md — cover full answers; speak from **Answer points**; compare
6. 05-exercises.md — drills with in-repo solutions
7. Revision twin for timed drill under pressure

Module map

Module	File
Foundations	01-foundations.md
Deep dive	02-deep-dive.md
Production bridge	03-production-bridge.md
Questions	04-questions.md
Exercises	05-exercises.md
Code	code/RetainCycleDemo.swift

Critical correctness (memorize)

Claim	Truth
Retain cycle	Objects keep each other alive → reachable abandoned memory
Instruments Leaks	Finds unreachable allocated memory (no live pointers)
Cycles in Leaks?	No — cycles stay reachable; use Memory Graph / Allocations
Timer scheduledTimer(timeInterval:target:selector:...)	Timer strongly retains the target until invalidate
NotificationCenter block API	Returns an observer token ; store it and remove on teardown

Provenance reminder

Only use Verified IDs from [../..../provenance/README.md](#) .

- **Verified · S8** — reliability culture, Crashlytics, IMOC, 99.95%+ CFS at 30L+ DAU
- **How I would apply it** — Memory Graph / Allocations triage playbook at BMS-scale apps (not claimed as shipped personal workflow unless you add real evidence later)
- **Learning-lab** — `RetainCycleDemo.swift` and playground drills

Time budget (suggested)

Block	Minutes
Foundations + deep dive	90–120
Production bridge + code	45
Questions (half bank aloud)	60–75

Exercises	45–60
Timed drill (revision twin)	30

01 — Foundations: ARC mental model

Intern-first. Read this before the deep dive. No external docs required.

1. The one-sentence model

ARC (Automatic Reference Counting) is how Swift/ObjC keep track of how many *strong* owners a class instance has. When the strong count hits zero, the object is destroyed and `deinit` runs. There is no garbage-collector pause world like Java/Go — the compiler inserts retain/release calls at compile time.

Structs, enums, and tuples are **value types**. They are not ARC-managed as heap objects in the same way. Nested *classes* inside a struct still use ARC.

2. Three kinds of references

Kind	Optional?	What happens when owner goes away	Use when
strong (default)	No	Keeps the object alive	Normal ownership
weak	Yes (T?)	Reference becomes <code>nil</code>	Break cycles; owner may outlive you
unowned	No (T)	Becomes a dangling pointer — crash if you use it	You are <i>sure</i> the other object outlives you

Intern picture

Imagine a sticky note on every object: “how many strong owners?”

- Create an object → count = 1 (the variable that owns it).
- Assign to another strong property → count += 1.
- That property is set to `nil` or goes out of scope → count -= 1.
- Count hits 0 → memory freed, `deinit` prints if you log it.

`weak` does **not** bump the sticky-note count. When the object dies, the weak pointer is zeroed to `nil` so you can safely write `guard let self else { return }`.

`unowned` also does **not** bump the count, but it does **not** zero. If you touch it after the object died, you crash. That is why seniors treat `unowned` as a sharp tool, not a default.

3. What is a retain cycle?

A **retain cycle** is a loop of strong references:



Nothing outside the loop needs those objects anymore, but their counts never hit zero — so they never deallocate. They are still **reachable** from each other.

That is **abandoned memory** (still referenced, but unused by the live app graph you care about). It is *not* the same as an Instruments **Leak**, which is memory with **no** live references at all.

Why this matters in interviews

Interviewers love the trap: “retain cycles show up in the Leaks instrument.”

Wrong. Leaks looks for *unreachable* allocations. A cycle is *reachable*. You find cycles with:

- 1. **Xcode Debug Memory Graph** (visual ownership edges, cycle highlighting)
- 2. **Allocations** instrument (persistent growth / objects that should have died)

Use **Leaks** for true unreachable leaks (malloc without owners, some C/ObjC edge cases, bugs that drop all pointers).

4. Glossary (speak these cleanly)

Term	Meaning
Retain count	Number of strong references to a class instance
dealloc	Method called when count hits zero (last chance to invalidate timers, etc.)
Escaping closure	Closure that outlives the function that created it; can capture self strongly
Capture list	[weak self], [unowned self] — how the closure holds outer values
Abandoned memory	Still referenced, but no longer useful (classic: retain cycle, forever cache)
True leak	Allocated memory with no remaining references
High watermark	Peak memory during a session; not the same as a leak
Jetsam	iOS kills your process under memory pressure
Observer token	Object returned by NotificationCenter’s block API; keep it to unregister

5. Cycle hotspots you’ll see every week

Pattern	Why it cycles	Fix (one line)
Escaping closure on self	Object owns closure; closure captures self strong	[weak self] + guard let self
delegate stored strong	VC ↔ delegate both strong	weak var delegate (class-bound)
Timer target-selector	Timer retains its target	invalidate() in dealloc; prefer block timer + weak
NotificationCenter block observer	Center holds block; block may hold self	Store token ; remove; [weak self] in block
Parent → child → parent	Two-way strong ownership	One side weak / unowned

6. weak vs unowned — intern decision tree

```
Does this reference participate in a possible cycle
OR might the other object die first?
├─ YES, other might die first → weak (optional, safe nil)
└─ Lifetimes are mathematically tied
    (child owned by parent; child never outlives parent)
    → unowned is *allowed*, still prefer weak if unsure
```

Default interview stance: prefer `[weak self]` for escaping async work (network, ads, timers, notifications). Reach for `unowned` only when you can prove lifetime in one sentence.

7. What “deinit not called” means

If you expected an object to die and `deinit` never runs, the object still has ≥ 1 strong owner. Checklist:

1. Escaping closure / Combine / Task still holding `self`
2. Timer not invalidated
3. NotificationCenter observer / token still registered with a strong capture
4. Delegate / parent strong back-reference
5. Still in a navigation stack, presented, or cached by a singleton
6. Instruments: Memory Graph — who points at me?

Do **not** say “ARC is broken.” ARC is doing exactly what the strong graph asks.

8. Mini demo mental walkthrough

```
final class Box {
    var onDone: (() -> Void)?
    deinit { print("Box deinit") }
}

do {
    let box = Box()
    box.onDone = {
        // strong capture of box → cycle: box → closure → box
        print(box)
    }
}
// Box deinit does NOT print – abandoned, still reachable via the cycle
```

Fix:

```
box.onDone = { [weak box] in
    guard let box else { return }
    print(box)
}
```

Full annotated examples: [code/RetainCycleDemo.swift](#).

9. Tools at a glance (foundations)

Tool	Finds	Does <i>not</i> find well
Memory Graph	Ownership edges, retain cycles, who retains a VC	Historical allocation timeline
Allocations	Growth over time, persistent objects, abandoned heaps	Pretty cycle diagrams
Leaks	Unreachable malloc/ObjC objects	Retain cycles (they're reachable!)

10. Bridge to senior thinking

After this file you should be able to say:

“ARC frees class instances when strong count hits zero. Cycles keep counts above zero even when the UI is gone — that’s abandoned memory, found with Memory Graph or Allocations, not Leaks. I break cycles with weak captures, weak delegates,

timer invalidation, and NotificationCenter tokens.”

Next: [02-deep-dive.md](#) for mechanisms, edge cases, and trade-off tables.

02 — Deep dive: ARC mechanics, cycles, Instruments

Senior depth. Assumes [01-foundations.md](#). Still self-contained — no “go read Apple docs first.” Optional citations at the end.

1. What the compiler actually does

For class instances, the Swift/ObjC runtime maintains a retain count (plus side tables for weak references). The compiler emits:

- `retain` / `release` (or equivalent ARC runtime calls) at ownership transfer points
- `deinit` invocation when the last strong reference is released
- Zeroing weak bookkeeping so `weak` becomes `nil` after destroy

Interview-ready contrast with GC:

	ARC	Tracing GC (Java, Go, etc.)
When work happens	Mostly at assign / scope exit	Periodic heap scan / pause or concurrent mark
Cycles	Must break manually (weak/unowned)	Collector can reclaim cyclic garbage
Determinism	<code>deinit</code> timing is predictable relative to last release	Finalizers are less deterministic
Cost model	Predictable per-operation	Throughput vs pause trade-offs

ARC does **not** mean “no memory bugs.” It means ownership bugs show up as cycles, early dealloc crashes (`unowned`), or abandoned caches — not as “forgot to free” in everyday Swift class code.

2. Strong, weak, unowned — semantics table

	strong	weak	unowned	unowned(unsafe)
Increments retain count	Yes	No	No	No
Type	T	T?	T	T
After owner dies	N/A (you <i>are</i> an owner)	Becomes <code>nil</code>	Dangling — use → crash	Dangling — undefined / crash risk
Runtime cost	Baseline	Side table / zeroing	Cheaper than weak	Cheapest, least safe
App code default?	Yes	Yes for cycles / async	Rare, proven lifetime	Almost never

When `unowned` is justified

Example: a `Person` strongly owns a `CreditCard`, and the card’s `owner` must always exist for the card’s entire life:

```
final class Person {
    var card: CreditCard!
    init() { card = CreditCard(owner: self) }
}

final class CreditCard {
```

```

    unowned let owner: Person // card cannot outlive person by construction
    init(owner: Person) { self.owner = owner }
}

```

If a network callback might outlive a dismissed VC, `unowned self` is a bug waiting to happen. Prefer `weak` .

3. Closures: how captures create cycles

3.1 Escaping vs non-escaping

- **Non-escaping** closures (default for function parameters that don't escape) usually cannot outlive `self` , so a strong capture of `self` often cannot form a long-lived cycle.
- **Escaping** closures stored in properties, dispatched async, or passed to APIs that keep them → **can** outlive the call site → cycle risk if they capture `self` and `self` owns them.

3.2 Capture lists

```

// Cycle: self → onComplete → self
onComplete = { self.refresh() }

// Breaks the cycle edge from closure → self
onComplete = { [weak self] in
    guard let self else { return }
    self.refresh()
}

```

Notes seniors get right:

- `[weak self]` alone is not enough if you then create *another* escaping closure that strongly captures the unwrapped `self` without care.
- Nested closures each need their own discipline.
- Capturing a specific property (`[weak self] in self?.foo`) vs capturing many locals — prefer narrow captures when readability allows.

3.3 lazy var closures

```

lazy var formatter: DateFormatter = {
    // This closure runs later and can capture self strongly while
    // self is initializing / retaining the lazy property.
    let f = DateFormatter()
    f.timeZone = self.timeZone // careful: strong self capture
    return f
}()

```

Safer patterns: don't touch `self` inside lazy init if avoidable; or compute without storing a self-capturing escaping closure; or use explicit weak if the design truly needs `self` .

4. Classic UIKit / AppKit cycle patterns

4.1 Delegates

```

protocol FeedDelegate: AnyObject {
    func feedDidRefresh()
}

final class FeedController {

```

```

    weak var delegate: FeedDelegate?
}

```

Class-bound (AnyObject) delegates are `weak` so `UIViewController` ↔ child controller / view model does not form a permanent loop. Value-type “delegates” (structs) don’t use ARC the same way — usually you pass closures or use different patterns.

4.2 Timer — target-selector retains the target

```

final class Ticker {
    private var timer: Timer?

    func start() {
        // IMPORTANT: scheduledTimer(target:selector:) RETAINS target (self)
        timer = Timer.scheduledTimer(
            interval: 1,
            target: self,
            selector: #selector(tick),
            userInfo: nil,
            repeats: true
        )
    }

    @objc private func tick() { /* ... */ }

    deinit {
        timer?.invalidate() // release the target retain
        timer = nil
    }
}

```

Rules:

1. Target-selector repeating timers keep the target alive until `invalidate()` .
2. Always `invalidate()` in `deinit` (and when leaving the screen if you intend teardown earlier).
3. Prefer block-based timers with `[weak self]` when you control the API surface:

```

timer = Timer.scheduledTimer(withTimeInterval: 1, repeats: true) { [weak self] _ in
    self?.tick()
}
// Still invalidate on teardown – RunLoop ownership is separate from the capture list.

```

Block timers still need invalidation so the RunLoop drops them; the capture list prevents the *block* → *self* cycle, but you still manage timer lifetime.

4.3 NotificationCenter — block API uses tokens

Modern block-based API:

```

private var token: NSObjectProtocol?

func observe() {
    token = NotificationCenter.default.addObserver(
        forName: .NSSystemTimeZoneDidChange,
        object: nil,
        queue: .main
    )
}

```

```

    ) { [weak self] _ in
        self?.reload()
    }
}

deinit {
    if let token {
        NotificationCenter.default.removeObserver(token)
    }
}

```

Key facts:

- The block API returns an **observer token**. Store it.
- Remove with `removeObserver(_:)` using that token (or remove in `deinit` / `viewWillDisappear` as your lifecycle policy dictates).
- Still use `[weak self]` inside the block — the center owns the block; the block must not own `self` if `self` also keeps the observation alive indefinitely.
- Older selector-based `addObserver(self, selector:...)` does **not** retain the observer the same way; you still must remove in `deinit` on older patterns. Prefer the token API in new code for clarity.

4.4 Parent ↔ child

```

final class Parent {
    var child: Child?
}

final class Child {
    weak var parent: Parent? // break the cycle
}

```

Use `owned` only if the child is guaranteed never to outlive the parent *and* you want non-optional access.

4.5 Task / unstructured concurrency

```

final class SearchVC {
    private var searchTask: Task<Void, Never>?

    func query(_ text: String) {
        searchTask?.cancel()
        searchTask = Task { [weak self] in
            guard let self else { return }
            // ... await network ...
            await self.apply(results)
        }
    }

    deinit {
        searchTask?.cancel()
    }
}

```

A running `Task` retains objects it strongly captures. Cancel on disappear / `deinit`, and prefer `[weak self]` when the task may outlive the screen.

5. Leak vs abandoned vs high watermark

Concept	Definition	Typical cause	Primary tools
True leak	Memory allocated, no live references	Rare in pure Swift ARC; more common with unsafe / CF / missed CFRelease / some ObjC	Leaks instrument
Abandoned memory	Still referenced, but unused by current product logic	Retain cycles, singleton caches that never prune, forgotten observers	Memory Graph, Allocations
High watermark	Peak bytes during a session	Large images, peak concurrency, one-time spikes	Allocations / Memory report — may be OK if it drops

The sentence that wins interviews

"Retain cycles are abandoned but still reachable, so they don't show as Leaks. I use Memory Graph to see the cycle and Allocations to see persistent growth. Leaks is for unreachable memory."

Never say: "I found the retain cycle in the Leaks instrument."

6. Instruments workflow (senior playbook)

6.1 Debug Memory Graph (Xcode debugger)

1. Reproduce the bug (push VC, play ad, start timer, pop VC).
2. Debug → Memory Graph (or the gauge button).
3. Filter by your class name / module.
4. Inspect instances that should be gone.
5. Walk incoming references until you see the cycle (closure ↔ object, timer → target, etc.).

Strength: visual "who retains me?"

Weakness: snapshot in time; less about historical growth curves.

6.2 Allocations

1. Record while navigating hot paths (feed, checkout, ads).
2. Look for persistent growth of your types across generations / marks.
3. Use generation analysis / "persistent" bytes after you expect teardown.
4. Compare before/after a fix.

Strength: proves growth and abandoned heaps over time.

Weakness: doesn't draw the cycle as clearly as Memory Graph.

6.3 Leaks

1. Record the same session.
2. Investigate *true* leaks (unreachable).
3. Do **not** conclude "no cycles" just because Leaks is clean.

Strength: unreachable memory.

Weakness: silent on retain cycles.

6.4 Suggested order when a VC "won't die"

1. Add a temporary `deinit { print("...") }` (learning lab) or breakpoint.
2. Memory Graph — find unexpected owners.
3. Allocations — confirm the type persists across navigation generations.
4. Leaks — only if you suspect unsafe / CF / true unreachable issues.
5. Fix ownership; re-verify `deinit` + Graph.

7. Trade-off table

Choice	When	Cost / risk
Always [weak self] on escaping work	Uncertain lifetime (network, UI, ads)	Optional unwrapping noise; early nil skips work
unowned self	Nested object with proven shorter life	Crash if assumption wrong
unowned(unsafe)	Extreme low-level / proven hot path	Never casual app code
Invalidate timer in deinit only	Simple owned timer	If VC stays alive, timer keeps firing — also invalidate on disappear if needed
Store NC token + weak capture	Block observers	Must not forget removal
Strong delegate	Almost never for UIKit class delegates	Instant cycle risk
Cache with no eviction	“Performance”	Abandoned memory → jetsam under peak DAU

8. Failure modes (production-shaped)

Symptom	Likely ownership bug	First check
Memory climbs as user navigates	Abandoned VCs / cycles / caches	Memory Graph on popped VC type
Crash on callback after dismiss	unowned or force-unwrap after weak nil	Change to weak + guard
Timer fires after leave screen	Not invalidated; target retained	invalidate + nil
Observer fires after teardown	Token not removed / strong capture	Remove token; weak self
CFS dips during big events	Jetsam + crashes from pressure / races	Crashlytics + memory triage culture (S8)

9. Autorelease pools (short, accurate)

In tight loops that create many temporary ObjC objects (bridging, image processing helpers), drain with:

```
for item in hugeList {
    autoreleasepool {
        // temporaries released each iteration
        process(item)
    }
}
```

This is **not** a retain-cycle fix. It reduces peak temporary memory. Mention it only when the question is about high watermark in ObjC-heavy loops.

10. Value types vs ARC (edge)

- Structs themselves are not reference-counted.
- A struct storing a `class` instance shares that reference; mutating the struct may copy the struct shell but still share/retain the class unless you implement COW yourself.
- Closures are reference types under the hood — capturing large value trees can still keep class graphs alive.

11. Interview answer skeleton (90s deep dive)

- 1. **Claim:** ARC frees at retain count zero; cycles prevent that.
- 2. **Mechanism:** strong vs weak/unowned; escaping closures; timer retain; NC tokens.
- 3. **Tools:** Graph/Allocations for abandoned; Leaks for unreachable — don't confuse them.
- 4. **Production:** reliability culture at scale (Verified S8 metrics); triage playbook as Applied judgment.

Optional citations (appendix only)

- Swift Language Guide — Automatic Reference Counting
- Xcode Help — Debug memory graph / Instruments Allocations & Leaks

Studying this chapter alone is sufficient; links are optional deepeners.

Next: [03-production-bridge.md](#).

03 — Production bridge: reliability culture vs memory triage

Separate what you **verified shipped** from what you would **apply as design judgment**. Never blur them in an interview.

1. Provenance map for this chapter

Label	ID	What you may claim
Verified	S8	BookMyShow consumer app at 30+ lakh DAU ; sustained 99.95%+ crash-free sessions ; Crashlytics triage workflows; IMOC coordination for P0/P1 during high-traffic events
How I would apply it	S8 extension (not a separate registry ID)	Memory Graph + Allocations + Leaks triage playbook for abandoned VCs / retain cycles under peak traffic — as <i>how a reliability-minded iOS engineer would hunt memory pressure</i> , not as “I personally shipped Memory Graph runbooks at BMS”
Learning-lab	—	RetainCycleDemo.swift, playground cycles, Instruments practice on a sample app

Forbidden phrasing (unless you later add real evidence)

- “At BMS I opened Memory Graph and found the ad retain cycle.”
- “We measured X% memory reduction from fixing cycles.” (not in resume)
- “Leaks showed our retain cycles.” (technically false anyway)

Preferred phrasing

“At BMS scale we treated crash-free and incident ownership as first-class — Crashlytics + IMOC under 30L+ DAU and a 99.95%+ CFS bar (Verified S8). For memory specifically, my triage approach would be Memory Graph for cycles and Allocations for growth — not Leaks for retain cycles (How I would apply it).”

2. Verified · S8 — what reliability culture actually means

Provenance: Verified · S8 · BookMyShow · 30L+ DAU · 99.95%+ CFS · Crashlytics · IMOC

Resume-backed facts you can own:

- 1. **Scale:** tens of lakhs of daily users — small per-user memory bugs become fleet-wide pressure.
- 2. **Bar:** 99.95%+ crash-free sessions is a reliability KPI, not a vanity metric.
- 3. **Workflow:** Crashlytics triage with structured crash workflows.
- 4. **Ownership:** IMOC role coordinating iOS, backend, QA on P0/P1 during peak events.

How this connects to *today's topic* without overclaiming:

- Memory pressure (jetsam) and bad lifecycle ownership show up as **crashes / kills / instability** in the same reliability conversation as races and nil crashes.

- Senior signal is **process**: reproduce → classify (crash vs abandoned memory vs true leak) → fix ownership → verify → communicate blast radius.
- S8 proves you operated under a high CFS bar; it does **not** by itself prove a specific Instruments screenshot from BMS.

20-second interview line (Verified only)

"At BookMyShow we held a 99.95%+ crash-free bar at 30L+ DAU — Crashlytics workflows and IMOC ownership for P0/P1. Memory and lifecycle bugs were reliability issues, not 'just perf.'"

45-second bridge into ARC (Verified + Applied clearly labeled)

"Verified: that reliability culture is S8. Applied: if we saw climbing memory or VCs that never deinit'd after navigation, I wouldn't start in Leaks for a suspected retain cycle — I'd use Memory Graph to see who retains the object and Allocations to confirm persistent growth, then fix weak captures / timer invalidation / observer tokens."

3. How I would apply it — triage playbook at BMS-shaped apps

Provenance: How I would apply it · memory triage for retain cycles / abandoned VCs

Trigger signals

Signal	Hypothesis
Memory climbs while user browses listings → detail → back	Abandoned VCs / image caches / cycles
Ad module / video surface left on screen stack	Closure or player retain
Live ticking UI after leave	Timer target retain
Weird callbacks after pop	Notification / Task / network completion holding self
Jetsam clusters in Crashlytics during big events	Peak memory + abandoned heaps

Step-by-step (say this in interviews)

1. **Reproduce** on a debug build with the suspect flow (push/pop N times).
2. **Expect death**: temporary `deinit` log on the VC / controller under test (lab).
3. **Memory Graph**: filter type; inspect unexpected remaining instances; walk retain edges.
4. **Classify**: cycle / singleton cache / still-on-window — not "ARC bug."
5. **Allocations**: mark generations before/after navigation; confirm persistent bytes of that type.
6. **Leaks**: only if Graph suggests unreachable weirdness or CF/unsafe involvement — do not use Leaks as the cycle detector.
7. **Fix**: weak capture, weak delegate, `timer.invalidate()` , NC token removal, Task cancel.
8. **Verify**: `deinit` runs; Graph clear; Allocations flat across generations.
9. **Reliability wrap**: if it shipped, watch Crashlytics / memory metrics; communicate blast radius like any P1.

Ads / navigation flavor (Applied, not a fake war story)

Ads and media surfaces are classic cycle magnets: completion handlers, players, timers, notification hooks. At revenue-critical modules (Verified S1 exists for ads architecture — separate story), a senior instinct is: **escaping work must not strongly own the screen**.

You may say:

"How I would apply it in an ads or checkout flow: every escaping completion gets a capture-list review, timers invalidate on teardown, and I confirm the VC disappears in Memory Graph before calling the memory bug fixed."

Do **not** invent a specific BMS ticket ID or metric.

4. District / Raw hooks (light touch)

Context	Provenance	Memory angle
---------	------------	--------------

District Free Parking / MVVM-Clean (S9)	Verified S9 for architecture; Applied for memory	Migrations often leave orphaned observers or long-lived VMs — review teardown during refactors
Raw live scoreboard (S10/S11)	Verified realtime ownership	Timers / live updates: invalidate and weak captures or you keep scoreboard controllers forever
Personal apps	Learning-lab	Safe place to practice Graph without claiming production

Keep Day 03 centered on **S8 + Applied triage**. Don't force every company into a fake memory anecdote.

5. Mapping tools to production language

Engineer says	Interview-correct meaning	Tool
"We leaked the VC" (casual)	Usually abandoned via cycle / cache	Graph / Allocations
"True leak"	Unreachable allocation	Leaks
"Memory pressure / jetsam"	OS killed process	Crashlytics + memory reports
"High watermark"	Peak usage, may be OK	Allocations

Teach PMs/QA the difference only if useful; teach interviewers by using precise words.

6. STAR sketch (S8) — keep ARC as supporting color

When the behavioral question is incident ownership, lead with S8 STAR (see story bank). Memory enters as *one class of reliability defect*, not the hero claim unless the interviewer asks about leaks.

Situation/Task: 30L+ DAU, 99.95%+ CFS, fast P0/P1.

Action: Crashlytics workflows; IMOC coordination; mitigations and comms.

Result: Sustained crash-free; reduced downtime in peaks.

If asked about memory: pivot to Applied triage playbook above — labeled honestly.

Full STAR: [../..../stories/story-bank.md](https://github.com/GoogleChrome/arc/blob/master/stories/story-bank.md) #s8--imoc--crash-free-at-scale-bookmyshow

7. Checklist before you speak in an interview

- ☐ Did I label Verified vs Applied?
- ☐ Did I avoid "cycles show in Leaks"?
- ☐ Did I mention Timer retain + invalidate?
- ☐ Did I mention NC **tokens** for the block API?
- ☐ Did I invent a BMS Memory Graph war story? → rewrite as Applied

Next: [code/RetainCycleDemo.swift](#), then [04-questions.md](#).

04 — Questions: ARC, cycles, Instruments

Study flow: cover **Full spoken answer** → speak from **Answer points** only → uncover and compare → hit the time budget.

Legend for provenance:

- **Verified · S8** — reliability culture / CFS / Crashlytics / IMOC
- **How I would apply it** — Memory Graph / Allocations triage (not claimed as personal BMS Memory Graph usage)
- **Learning-lab** — demo code / playground

Normal questions

Q1. What is ARC? (30–45s)

Answer points (frame first):

- Compile-time retains/releases for class instances (not a Java-style GC pause model)
- Strong count → 0 runs `deinit`
- One reliability hook without inventing unproven memory metrics (S8 culture OK)

Agenda opener: "ARC is how Swift manages class instance lifetimes with reference counts..."

Full spoken answer:

"ARC is Automatic Reference Counting — the compiler inserts `retain` and `release` calls for class instances. Each strong reference bumps the count; when it hits zero, the object is destroyed and `deinit` runs. It's not a tracing garbage collector, so we don't get GC pause semantics, but we do have to break retain cycles ourselves with `weak` or `unowned`. At BookMyShow, reliability meant holding a 99.95%+ crash-free bar at 30L+ DAU — lifecycle and memory bugs sit in that same reliability conversation, not as an afterthought."

Common wrong answer: "Swift has a garbage collector like Java." / "ARC pauses the app to collect."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Do structs use ARC?	Value types aren't refcounted as objects; nested classes still are.
L2	When does <code>deinit</code> run?	When the last strong reference is released — deterministic relative to that release.
L3	ARC vs MRC?	Manual retain/release era vs compiler-inserted; same runtime idea, less human error.

Provenance: Verified · S8 reliability culture

Q2. weak vs unowned? (45s)

Answer points (frame first):

- `weak` = optional, zeroing; safe if other side dies first
- `unowned` = non-optional, non-zeroing; crash if dangling
- Prefer `weak` for escaping async / UI callbacks

Agenda opener: "Both avoid incrementing the retain count, but they differ when the object dies..."

Full spoken answer:

"`weak` is an optional reference that doesn't keep the object alive, and it becomes `nil` when the object deinits — so you guard `let self`. `unowned` also doesn't keep it alive, but it isn't optional and isn't zeroed; if you touch it after `deinit`, you crash. I use `weak` for network, ads, and UI callbacks where lifetime is uncertain. I only use `unowned` when lifetimes are provably nested — like a child object that cannot outlive its parent by construction."

Common wrong answer: "They're the same; `unowned` is just faster so I always use it."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Why is <code>weak</code> optional?	Because it must represent "already gone."
L2	When is <code>unowned</code> justified?	Child owned by parent; child never escapes parent's lifetime.
L3	What about <code>unowned(unsafe)</code> ?	Skips safety checks — not casual app code.

Provenance: Learning-lab · capture-list discipline

Q3. What's a common retain cycle in Swift? (45s)

Answer points (frame first):

- Object stores escaping closure; closure captures `self` strongly
- Counts never hit zero → abandoned but reachable
- Fix with capture list `[weak self]`

Agenda opener: "The classic one is an object that owns a closure that captures itself..."

Full spoken answer:

"The textbook cycle is a view controller or service that stores an escaping completion handler, and that handler strongly captures `self`. Now `self` owns the closure and the closure owns `self`, so retain counts never reach zero — the object is abandoned but still reachable. The fix is a capture list: `[weak self]` plus a guard. Same shape shows up with delegates stored strongly, timers, and notification blocks."

Common wrong answer: "Any closure that mentions self is a leak." (non-escaping often fine)

Follow-up ladder:

Level	Ask	Point + short answer
L1	Escaping vs non-escaping?	Escaping can outlive the call — cycle risk.
L2	How do you detect it?	Memory Graph / missing deinit — not Leaks.
L3	Nested closures?	Each escaping edge needs capture discipline.

Provenance: Learning-lab · RetainCycleDemo.swift

Q4. Should delegates be weak? (30–45s)

Answer points (frame first):

- Class-bound delegates: yes, `weak var` to avoid VC ↔ delegate cycles
- Need `AnyObject` (or class-only) for weak
- Value-type "delegate" patterns differ

Agenda opener: "For class delegates in UIKit-style APIs, yes — make them weak..."

Full spoken answer:

"If the delegate is a class — typically a view controller — storing it `strong` creates an easy cycle when the child also owns its parent logically. So we declare `weak var delegate` and constrain the protocol to `AnyObject`. That way the child doesn't keep the parent alive. If you're using struct-based callbacks, you usually pass closures with explicit capture lists instead of a weak delegate property."

Common wrong answer: "Delegates are always strong so they don't go nil."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Why <code>AnyObject</code> ?	Weak references require reference types.
L2	Closure vs delegate?	Closures need capture lists; same ownership problem.
L3	multicast delegates?	Often separate bag with weak boxes — still avoid strong loops.

Provenance: Learning-lab · UIKit patterns

Q5. Memory leak vs abandoned memory? (60s)

Answer points (frame first):

- True leak = unreachable allocated memory
- Abandoned = still referenced but unused (retain cycles, immortal caches)
- Tools: Leaks vs Memory Graph / Allocations — **do not swap them**

Agenda opener: "I separate unreachable leaks from reachable abandoned memory..."

Full spoken answer:

"A true leak is memory that's allocated but has no live references — nothing can touch it, so it can never be freed. Abandoned memory still has references — classic retain cycles or caches that never evict — but the app no longer needs those objects. Retain cycles are abandoned and still reachable, so they typically do **not** show in the Leaks instrument. I use Memory Graph to see ownership edges and Allocations to see persistent growth. Leaks is for unreachable cases. High watermark is different again — peak usage that may drop later."

Common wrong answer: "Retain cycles show up in Leaks."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Why can't Leaks see cycles?	Because pointers still exist — objects are reachable.
L2	What's a high watermark?	Peak bytes; not necessarily a leak.
L3	Jetsam relation?	Abandoned heaps raise pressure → OS may kill the app.

Provenance: How I would apply it · Instruments triage

Q6. Does [weak self] always fix cycles? (45–60s)

Answer points (frame first):

- Only breaks that specific closure → self edge
- Other owners (timer, NC, nested closures, singleton) may remain
- Still need invalidate / remove / cancel

Agenda opener: "It fixes the capture edge — not every ownership bug..."

Full spoken answer:

"[weak self] only weakens that closure's reference to self. If a Timer is still retaining you via target-selector, or a NotificationCenter block is registered without teardown, or another nested escaping closure captures self strongly, you can still fail to deinit. So weak is necessary for escaping closures, but the checklist is broader: timers invalidate, observer tokens removed, tasks cancelled, delegates weak, caches bounded."

Common wrong answer: "I put weak self everywhere, so I can't have leaks."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Example that weak doesn't fix?	Timer target-selector without invalidate.
L2	Nested escaping?	Inner closure can re-introduce strong self.
L3	When strong self is OK?	Non-escaping work; or intentional ownership with clear teardown.

Provenance: Learning-lab

Q7. deinit not called — checklist? (60s)

Answer points (frame first):

- Still has ≥ 1 strong owner — not “ARC broken”
- Walk: closures, timer, observers, delegate, nav/presented, singleton cache
- Confirm with Memory Graph

Agenda opener: “If deinit doesn’t run, something still strongly owns the object...”

Full spoken answer:

“First I assume the object is still retained — ARC isn’t randomly broken. I check escaping closures and tasks, repeating timers that weren’t invalidated, NotificationCenter tokens still registered, strong delegates, and whether the controller is still in a navigation stack, presented, or held by a singleton cache. Then I’d open Memory Graph and inspect who points at the instance. Once I break the unexpected strong edge, deinit should fire. I’d verify with Allocations that the type doesn’t persist across navigation generations.”

Common wrong answer: “Swift bug / need to call deinit manually.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	Can you call deinit?	No — runtime calls it when count hits zero.
L2	Print in deinit OK?	Fine for lab; remove before ship or guard with debug flags.
L3	Graph vs Allocations?	Graph = who; Allocations = growth over time.

Provenance: How I would apply it · Memory Graph triage

Q8. Value types and ARC? (30–45s)

Answer points (frame first):

- Structs/enums aren’t ARC’d as heap objects the same way
- Stored class references inside values still refcount
- Closures can keep class graphs alive

Agenda opener: “Value types themselves aren’t reference-counted, but they can hold references...”

Full spoken answer:

“Structs and enums are value types — copying them doesn’t use ARC the way class instances do. But if a struct stores a class reference, that reference is still strong and retains the class. Closures are reference types under the hood, so a struct that somehow participates in escaping closure graphs can still keep objects alive. So ‘we only use structs’ doesn’t automatically mean ‘no retain cycles’ if classes and closures are in the mix.”

Common wrong answer: “If I use structs I can’t have memory issues.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	COW connection?	Collections share buffers until mutation — different topic, still refs under the hood.
L2	Actor / class boxes?	Reference semantics return when you wrap.
L3	Why classes for UIKit VCs?	Identity + inheritance / UIKit object graph.

Provenance: Learning-lab · language model

Q9. Autorelease pools — when? (45s)

Answer points (frame first):

- Drain temporary ObjC / bridged objects in tight loops
- Lowers peak watermark — not a cycle fix
- Mention only when relevant

Agenda opener: "Autorelease pools are about temporary peak memory, not retain cycles..."

Full spoken answer:

"Autorelease pools let you drain temporary autoreleased objects sooner — useful in tight loops that create lots of bridged ObjC temporaries, like some image or string processing paths. That reduces high watermark. It does not break retain cycles. If the interviewer asks about climbing memory from a cycle, I talk weak captures and Instruments Graph — not autorelease pools."

Common wrong answer: "Wrap everything in autoreleasepool to fix leaks."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Default pool?	RunLoop drains pools; tight loops may need explicit ones.
L2	Pure Swift structs loop?	Often unnecessary.
L3	Relate to jetsam?	Lower peak can help pressure; still fix abandonment separately.

Provenance: Learning-lab

Q10. How do you prevent leaks with async Task ? (60s)

Answer points (frame first):

- Task retains strongly captured locals / self
- Cancel on disappear / deinit
- Prefer `[weak self]` when task may outlive the screen

Agenda opener: "Treat unstructured Tasks like escaping closures with cancellation..."

Full spoken answer:

"A `Task` keeps alive anything it strongly captures for as long as it runs. If a view controller fires a task and the user pops the screen, a strong `self` capture can keep the controller around until the task finishes. I store the task, cancel it in `deinit` or `onDisappear`, and use `[weak self]` inside when the work is optional after teardown. Structured concurrency children cancel with their parent, which is preferable when the API allows it."

Common wrong answer: "async/await means ARC doesn't matter."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Task vs Task.detached?	Detached is unstructured and easier to forget — be stricter.
L2	Cancellation cooperative?	Check <code>Task.isCancelled</code> / try <code>Task.checkCancellation()</code> .
L3	Combine cancellables?	Same idea — store and cancel on teardown.

Provenance: Learning-lab · concurrency ownership

Q11. How does a Timer create a retain cycle / keep a target alive? (45–60s)

Answer points (frame first):

- Target-selector API **retains** the target until `invalidate`
- Repeating timer → object may never deinit
- Invalidate in `deinit` (and on leave-screen); prefer block + weak

Agenda opener: "The important detail is that `scheduledTimer` with `target:selector` retains the target..."

Full spoken answer:

"When you use `Timer.scheduledTimer(interval:target:selector:...)`, the timer strongly retains the target until you call `invalidate`. For a repeating timer on a view controller, that means the controller can live forever even after you think you dismissed it. The fix is to invalidate in `deinit` and usually when leaving the screen, and set the timer property to `nil`. Block-based timers with `[weak self]` avoid the selector retain semantics for `self`, but you still invalidate so the `RunLoop` drops the timer."

Common wrong answer: "Timers don't retain anything; only closures do."

Follow-up ladder:

Level	Ask	Point + short answer
L1	One-shot timer?	Still retains until it fires / invalidate — know the API.
L2	<code>RunLoop</code> role?	Timer scheduled on a run loop; invalidation removes it.
L3	Display link?	Similar ownership discipline — invalidate on teardown.

Provenance: Learning-lab · `TickerFixed` in demo

Q12. NotificationCenter block observers — what must you remember? (45–60s)

Answer points (frame first):

- Block API returns a **token** — store it
- Remove observer using the token on teardown
- `[weak self]` inside the block

Agenda opener: "With the block API, ownership is token plus capture list..."

Full spoken answer:

"`addObserver(forName:object:queue:using:)` returns an observer token. I store that token on the object and remove it in `deinit` or when stopping observation. Inside the block I capture `[weak self]` so the center's ownership of the block doesn't keep my controller alive forever. Forgetting either the token removal or the weak capture is a common abandoned-memory bug. I don't rely on 'Leaks will tell me' — I'd expect Memory Graph to show the observer graph."

Common wrong answer: "NotificationCenter never retains; you never need tokens."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Selector-based API?	Different retain rules historically; still remove in <code>deinit</code> on older patterns.
L2	Queue parameter?	Delivers on that queue — main for UI.
L3	Combine publishers?	<code>NotificationCenter.default.publisher</code> — store <code>AnyCancellable</code> .

Provenance: Learning-lab · `TimeZoneObserver`

Tricky questions

T1. `unowned self` in an async callback after the VC is dismissed (90s)

Answer points (frame first):

- Trap: `unowned` for “cleaner” syntax
- Callback can outlive VC → dangling → crash
- Prefer `weak` + guard; tie to network/ad SDK realities

Agenda opener: “I’d treat that as a lifetime bug waiting to happen...”

Full spoken answer:

“If a view controller dismisses while a network or ad callback is still outstanding, `unowned self` becomes a dangling reference the moment `deinit` runs. The next callback touch crashes. That’s why I default to `[weak self]` for escaping async work. `unowned` is for nested ownership you can prove in one sentence — not for `URLSession` or third-party SDK completions. In a reliability culture like BMS — Verified S8 with Crashlytics and a high CFS bar — this class of crash is exactly what triage workflows exist to catch. How I would apply Instruments here: reproduce dismiss-during-flight, confirm with Graph that the VC is gone, and ensure the callback path is weak-safe.”

Common wrong answer / trap: “`unowned` is fine if I’m careful about timing.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	Why crash not nil?	<code>unowned</code> doesn’t zero.
L2	<code>weak</code> + continue work?	Often cancel work on disappear instead of updating UI.
L3	MainActor implication?	Still need lifetime safety; actor isolation ≠ ownership.

Provenance: Verified · S8 crash culture + How I would apply it · weak callbacks

T2. NotificationCenter + closure observer cycle (90s)

Answer points (frame first):

- Center holds block; block holds self; self holds token/lifetime
- Store token; remove; weak capture
- Don’t say “Leaks will catch it”

Agenda opener: “The bug is usually forgotten token removal or a strong self capture...”

Full spoken answer:

“With the block API, NotificationCenter keeps the block. If that block strongly captures self, and self stays alive because of other reasons — or you effectively keep the observation for the app lifetime — you can abandon controllers or create hard-to-see ownership. The correct pattern is: store the returned token, remove it on teardown, and capture `[weak self]` in the block. If `deinit` isn’t firing, Memory Graph is the right tool; Allocations confirms persistence. Leaks may stay quiet because everything is still reachable.”

Common wrong answer / trap: “I used weak self so I can ignore the token.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	What does <code>removeObserver(token)</code> do?	Unregisters that block observation.
L2	Observe on background queue?	Possible; hop to main for UI.
L3	System notifications flood?	Coalesce / debounce UI updates.

T3. `lazy var` closure capturing self (90s)

Answer points (frame first):

- Lazy initializer closure can strongly capture self
- During first access, ownership can surprise you
- Prefer no self in lazy init; or redesign

Agenda opener: "Lazy property initializers are closures — they can capture self..."

Full spoken answer:

"A `lazy var` 's initializing closure can capture `self` strongly while `self` is also retaining the lazy property. Depending on what's stored, that can create a cycle or at least confusing lifetime. The senior move is to avoid referencing `self` inside lazy initializers when possible — pass needed values explicitly — or don't store an escaping self-capturing closure as a property. If you must touch self, be deliberate about capture lists and whether the stored value permanently closes over self."

Common wrong answer / trap: "lazy is just syntax sugar; no ownership impact."

Follow-up ladder:

Level	Ask	Point + short answer
L1	When does lazy run?	First access — on whatever thread accessed it (watch thread safety).
L2	Computed vs lazy?	Computed doesn't store; lazy stores once.
L3	Thread-safe lazy?	Not automatically — synchronize if needed.

T4. Collection of closures storing self (90–120s)

Answer points (frame first):

- Each escaping closure is an edge
- Weakening only one isn't enough
- Prefer structured teardown / cancellation tokens

Agenda opener: "I'd audit every escaping closure in the collection as its own retain edge..."

Full spoken answer:

"If an object stores an array of handlers and each handler captures self strongly, you have many edges forming the same cycle shape. Marking one closure `[weak self]` while others stay strong doesn't save you. I either weaken all escaping handlers, or better redesign: use a delegate, a single pipeline, or structured concurrency with cancellation so the collection doesn't immortalize the owner. At scale, abandoned controllers from listener bags show up as memory growth in Allocations and as leftover instances in Memory Graph — again, not as Leaks."

Common wrong answer / trap: "I weak'd the outer closure; inner ones are fine."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Observer pattern fix?	Weak observer boxes / token unregister.
L2	Combine?	Store <code>AnyCancellables</code> ; cancel on deinit.
L3	Production risk?	Navigation-heavy apps accumulate abandoned VCs → jetsam risk.

T5. Why Memory Graph shows a cycle but Leaks doesn't (90s)

Answer points (frame first):

- This is the money question — get the definitions crisp
- Cycle $\Rightarrow$ reachable $\Rightarrow$ not a Leaks hit
- Graph visualizes edges; Allocations shows growth

Agenda opener: "Because a retain cycle is still reachable memory..."

Full spoken answer:

"The Leaks instrument looks for allocated memory with no remaining references — truly unreachable objects. A retain cycle has plenty of references; the objects point at each other, so they're reachable from inside the cycle. Memory Graph shows those edges and can highlight the cycle. Allocations shows that those objects persist over time. So seeing a cycle in Graph with a clean Leaks recording is expected, not contradictory. If an interviewer says cycles 'show in Leaks,' I'd politely correct that — it's a common mix-up."

Common wrong answer / trap: "Leaks and Memory Graph are interchangeable."

Follow-up ladder:

Level	Ask	Point + short answer
L1	When <i>does</i> Leaks help?	Unreachable malloc/CF/ObjC edge cases.
L2	Order of tools?	Graph for who; Allocations for growth; Leaks last for unreachable.
L3	Production narrative?	Applied triage under S8-style reliability bar — labeled honestly.

T6. UIKit view retain via layer / ownership edges (90s)

Answer points (frame first):

- Not only Swift closures — UIKit/Core Animation ownership quirks exist
- Know to inspect Graph beyond your closure
- Don't claim obscure API myths as personal BMS fact

Agenda opener: "I'd widen the search beyond Swift capture lists..."

Full spoken answer:

"Seniors know retain issues aren't only [weak self]. Views, layers, gesture targets, and delegates can participate in the object graph. Historically people have been bitten by strong relationships around layer delegates and similar UIKit edges. In practice I don't memorize every historical quirk as a war story — I use Memory Graph to see the actual incoming references when a view or controller won't die. That's more reliable than guessing. Learning-lab: reproduce, Graph, fix the edge you see."

Common wrong answer / trap: "Only escaping Swift closures cause cycles in iOS."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Gesture targets?	Target-action can retain — check teardown.
L2	CADisplayLink?	Invalidate like timers.
L3	SwiftUI difference?	Different lifetime model; still watch tasks/closures in observable objects.

T7. Singleton cache forever holding VCs or models (90–120s)

Answer points (frame first):

- Not always a “cycle” — sometimes one-way immortal ownership
- Still abandoned memory; Graph shows singleton → objects
- Fix with eviction, weak maps, or lifecycle scopes

Agenda opener: “Abandoned memory isn’t always a two-node cycle...”

Full spoken answer:

“A singleton image cache or ‘debug retain forever’ dictionary can hold view controllers or heavy models with a single strong edge. That’s not a cycle, but it’s still abandoned from a product perspective — those objects never die. Memory Graph shows the singleton as the root owner; Allocations shows monotonic growth. The fix is bounded caches, weak wrappers where appropriate, or scoping the cache to a flow. At 30L+ DAU, immortal caches are a jetsam risk. I’d talk about this as Applied judgment under a Verified S8 reliability bar — not as a fake ‘we fixed cache X at BMS’ claim without evidence.”

Common wrong answer / trap: “If there’s no cycle, memory must be fine.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	NSMutableDictionary weak keys?	One tool for weak caches — know trade-offs.
L2	NSCache?	Evicts under pressure; still not magic for your strong wrappers.
L3	Measuring?	Allocations generations while repeating the flow.

Provenance: Verified · S8 scale context + How I would apply it · cache policy

T8. “We have no Leaks in Instruments — ship it?” (90s)

Answer points (frame first):

- Clean Leaks ≠ no abandoned memory
- Require Graph/Allocations for cycle suspects
- Reliability bar: memory pressure still matters for CFS/jetsam

Agenda opener: “I would not treat a clean Leaks run as proof the memory graph is healthy...”

Full spoken answer:

“A clean Leaks instrument means you didn’t find unreachable allocations in that recording. It does **not** mean view controllers aren’t stuck in retain cycles or singleton caches. Before shipping a memory fix, I want Memory Graph to show the suspect type gone after teardown, and Allocations to show flat persistent growth across navigation generations. Under a high crash-free bar — Verified S8 at BMS scale — jetsam and memory pressure are reliability issues. How I would apply it: Leaks is necessary for unreachable bugs, never sufficient for cycle hunting.”

Common wrong answer / trap: “Leaks green = no memory bugs.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	What’s your release checklist?	deinit proof or Graph clear + Allocations flat.
L2	MetricKit / mem graphs in prod?	Fleet signals differ from Instruments — use both worlds.
L3	Communicate to QA?	Give a navigation soak script, not just ‘run Leaks’.

Timed drill (subset)

Record aloud: **Q1, Q2, Q5, Q11, T1, T5, T8.**

Score against [../../.. / timing/answer-timing-guide.md](https://github.com/ash2k8/arc42/blob/master/timing/answer-timing-guide.md) .

Log misses (especially Leaks-vs-cycle confusion) in your gotchas list.

Revision twin: [../../.. / revision/weeks/week-01/day-03.md](https://github.com/ash2k8/arc42/blob/master/revision/weeks/week-01/day-03.md) .

05 — Exercises: ARC & Instruments

Solutions are in-repo below each exercise. Try first with notes closed.

Exercise A — Predict `deinit` (Learning-lab)

Task: Without running code, for each snippet say whether `deinit` prints when `outer` ends.

A1

```
final class A {
    deinit { print("A") }
}
do {
    let a = A()
    _ = a
}
```

A2

```
final class B {
    var hold: (() -> Void)?
    deinit { print("B") }
    func arm() { hold = { print(self) } }
}
do {
    let b = B()
    b.arm()
}
```

A3

```
final class C {
    var hold: (() -> Void)?
    deinit { print("C") }
    func arm() { hold = { [weak self] in print(self as Any) } }
}
do {
    let c = C()
    c.arm()
}
```

Solution A

	deinit prints?	Why
A1	Yes	No extra owners; count → 0 at end of do.
A2	No	Cycle: B → hold → self. Abandoned, reachable.
A3	Yes	Weak capture breaks cycle; property may remain but doesn't keep c alive.

Interview line: A2 is abandoned memory — Memory Graph would show the cycle; Leaks may not.

Exercise B — Fix the timer (Learning-lab)

Broken:

```
final class Pulse {
    private var timer: Timer?
    func start() {
        timer = Timer.scheduledTimer(timeInterval: 1, target: self,
                                     selector: #selector(tick),
                                     userInfo: nil, repeats: true)
    }
    @objc private func tick() {}
    // no deinit
}
```

Tasks:

1. Why can `Pulse` never deallocate after `start()` ?
2. Write a fixed version (invalidate + optional block-based form).

Solution B

1. Target-selector timers **retain the target** until `invalidate()` . Repeating timer ⇒ immortal `Pulse` while scheduled.
2. Fixed:

```
final class PulseFixed {
    private var timer: Timer?

    func start() {
        timer = Timer.scheduledTimer(withTimeInterval: 1, repeats: true) { [weak self] _ in
            self?.tick()
        }
    }

    private func tick() {}

    func stop() {
        timer?.invalidate()
        timer = nil
    }

    deinit { stop() }
}
```

Also call `stop()` from `viewWillDisappear` / flow teardown if the object outlives the need to tick.

See also: [code/RetainCycleDemo.swift](#) `TickerFixed` .

Exercise C — NotificationCenter token (Learning-lab)

Broken sketch:

```
final class Listener {
    func start() {
        NotificationCenter.default.addObserver(forName: .init("ping"),
                                                object: nil, queue: .main) { _ in
            self.handle() // strong self
        }
    }
    func handle() {}
}
```

Tasks:

1. List two ownership bugs.
2. Rewrite with token storage, removal, and `[weak self]` .

Solution C

1. (a) Block strongly captures `self` . (b) Return token discarded — cannot cleanly remove; observation may keep the block graph alive.
2. Fixed pattern:

```
final class ListenerFixed {
    private var token: NSObjectProtocol?

    func start() {
        token = NotificationCenter.default.addObserver(
            forName: Notification.Name("ping"),
            object: nil,
            queue: .main
        ) { [weak self] _ in
            self?.handle()
        }
    }

    func handle() {}

    func stop() {
        if let token {
            NotificationCenter.default.removeObserver(token)
            self.token = nil
        }
    }

    deinit { stop() }
}
```

Exercise D — Tool selection (interview drill)

For each symptom, pick **primary** tool: Memory Graph / Allocations / Leaks. One sentence why.

#	Symptom	Your pick
D1	Popped VC's deinit never runs; you suspect a closure cycle	
D2	Memory bytes climb every time user opens checkout 20x	
D3	Suspected CTypeRef / unreachable malloc after a C bridge	
D4	Leaks instrument is clean; interviewer asks if cycles are impossible	

Solution D

#	Primary	Why
D1	Memory Graph	See who still retains the VC; visualize cycle.
D2	Allocations	Prove persistent growth across generations / repeated flow.
D3	Leaks	Hunting unreachable allocations.
D4	Answer: No — cycles are reachable, so clean Leaks proves little about cycles. Use Graph/Allocations.	

Exercise E — Speak for 60s (Verified + Applied labels)

Prompt: "Tell me how you'd hunt a retain cycle in a large consumer app."

Constraints:

- Must mention Graph vs Leaks correctly
- Must label Verified S8 vs Applied triage
- Must not invent "I used Memory Graph at BMS" as fact

Solution E (model)

"Verified: at BookMyShow we operated under a 99.95%+ crash-free bar at 30L+ DAU with Crashlytics workflows and IMOC ownership — memory pressure and lifecycle bugs sit in that reliability culture. Applied: if a screen won't die, I wouldn't start with Leaks for a suspected retain cycle. I'd reproduce push/pop, check Memory Graph for leftover instances and retain edges, use Allocations to confirm persistent growth, then fix weak captures, timer invalidation, or NotificationCenter tokens. Leaks is for unreachable memory — cycles stay reachable, so a green Leaks run wouldn't convince me."

Exercise F — Code reading (from demo file)

Open [code/RetainCycleDemo.swift](#).

1. Why does `BoxBroken.armCycle` prevent `deinit` ?
2. Why does `TickerBroken` mention `RunLoop` and retain?
3. What two responsibilities does `TimeZoneObserver.deinit` fulfill?
4. Why is `scheduleUnownedCrashRisk` dangerous after releasing the last strong ref to `Ephemeral` ?

Solution F

1. `onDone` is stored on `self` and captures `self` strongly → cycle.
2. Timer is scheduled on a `RunLoop` (keeps the timer alive) **and** target-selector retains `self` until `invalidate` — double ownership discipline.
3. Remove the NC token **and** (via `stop`) clear local state so observation ends; also guarantees teardown if caller forgot `stop()` .
4. After `Ephemeral` deinit's, `unowned self` dangles; the delayed block crashes when it runs.

Exercise G — Parent/child design choice (45s speak)

When would you use `weak` var parent vs `unowned let` parent on a child?

Solution G

- **weak:** parent might nil out while child briefly remains (or you're unsure) — safer optional.
 - **unowned:** child's lifetime is strictly nested under parent by API construction; you want non-optional access and accept crash if the invariant breaks.
 - Interview default under uncertainty: **weak**.
-

Exercise H — Mini Memory Graph script (device / simulator)

Provenance: *Learning-lab (and How I would apply it if you narrate it for BMS-shaped apps). Do not claim this script as verified BMS production history.*

1. Create a blank iOS app; paste `LeakyViewController` / `SafeViewController` from the commented section in `RetainCycleDemo.swift`.
2. Present leaky VC, assign `onFinish`, dismiss.
3. Open **Debug Memory Graph**; filter the VC type — confirm instance remains.
4. Repeat with safe VC — confirm absence.
5. Optional: run **Allocations** with generation marks; optional: run **Leaks** and note it may stay clean for the leaky cycle.

Pass criteria: You can explain *why* Leaks stayed clean while Graph showed the object.

Exercise I — Write the trap card

Write one flashcard front/back for each:

1. Leaks vs retain cycle
2. Timer target retain
3. NC block token

Solution I (sample backs)

1. **Front:** Do retain cycles show in Leaks? **Back:** No — reachable abandoned memory → Graph/Allocations. Leaks = unreachable. Trap: treating tools as synonyms. Prod: Applied triage under S8 reliability culture.
 2. **Front:** Why won't my VC die after starting a repeating Timer? **Back:** `scheduledTimer(target:selector:)` retains target until `invalidate`. Invalidate in `deinit` / disappear.
 3. **Front:** NotificationCenter block API must-dos? **Back:** Store token; remove on teardown; `[weak self]` in block.
-

Done checklist

- ☐ Explained ARC + weak/unowned aloud in <45s
- ☐ Corrected someone (or yourself) on Leaks vs cycles
- ☐ Fixed a timer + NC example without peeking
- ☐ Spoke Exercise E with Verified/Applied labels
- ☐ Recorded timed subset from `04-questions.md`

Next: revision twin timed drill — [../../../revision/weeks/week-01/day-03.md](https://github.com/robertmoores/revision/weeks/week-01/day-03.md).

Day 04 — GCD Queues, sync/async, Barriers, Thread-Safe Dictionary

Outcomes

By end of day, without notes, you can:

- Contrast serial vs concurrent queues; main vs global QoS
- Explain `sync` vs `async` and why sync-to-current-queue deadlocks (not only main)
- Implement a thread-safe dictionary behind a **private** serial queue API
- Explain reader-writer with barriers on a concurrent queue
- State the **async write / sync read visibility** rule correctly
- Deliver Verified · S2 (synchronised dictionaries) in ≤3 min with an actor-migration coda (S2-A1)
- Use `final class` (correct Swift keyword casing) in code and speech

How to study (in Cursor only)

1. [01-foundations.md](#) — queue mental model (intern → mid)
2. [02-deep-dive.md](#) — deadlocks, barriers, SafeDict correctness
3. [03-production-bridge.md](#) — Verified · S2 / S2-A1
4. [code/SafeDict.swift](#) — read, predict, then explain aloud
5. [04-questions.md](#) — speak from **Answer points**; compare
6. [05-exercises.md](#) — race demo + speaking drills
7. Revision twin for flashcards / timed drill day-of

Module map

Module	File	Role
Foundations	01-foundations.md	Queues, sync/async, QoS
Deep dive	02-deep-dive.md	Deadlock, RW barriers, visibility
Production bridge	03-production-bridge.md	S2 STAR + actor coda
Questions	04-questions.md	Two-layer Q&A
Exercises	05-exercises.md	Coding + speaking
Code	code/SafeDict.swift , code/BarrierDict.swift	Learning-lab

Time budget (suggested)

Block	Minutes
Foundations	55–70
Deep dive	75–90
Production bridge + code	35–45
Questions (speak 7–9 aloud)	60–75
Exercises + timed drill	45–60

Provenance reminder

Label	Use for today
-------	---------------

Verified · S2	Synchronised dictionaries via GCD serial queues / RW locks; race/crash reduction on that path
How I would apply it · S2-A1	Greenfield shared maps as Swift actor
Verified · S8	Soft: reliability culture / Crashlytics watch (do not claim S2 alone produced 99.95% CFS)
Learning-lab	SafeDict / barrier demos — not shipped BMS source

Do **not** invent crash counts or claim sole ownership of app-wide crash-free rate.

Critical correctness notes (pin)

1. **Async write then sync read may not see the write until the write runs.** Prefer **sync write** when the caller needs read-after-write on the same API, or document ordering clearly.
2. Spell it **final class**, never `Final class`.
3. **Hide the queue** — expose safe methods only (S2 lesson).

Agenda openers (pin these)

Topic	5–10s opener
Serial vs concurrent	"One-at-a-time versus overlapping execution."
sync vs async	"Wait for completion versus schedule and return."
Safe dict	"Private serial queue — sync get, careful set."
Barrier	"Exclusive writer on a concurrent queue."
S2	"Races on shared dictionaries — serial-queue API and trade-offs."

01 — Foundations: GCD Queues, sync/async, Thread Safety

Read this first. Goal: intern-clear queue mental model, then senior edges for S2 interviews.

0. One-sentence north star

Serialize access to shared mutable state at a clear boundary — don't sprinkle locks ad hoc — and never block a queue waiting for itself.

At BookMyShow that boundary was synchronised dictionaries behind GCD (Verified · S2). Day 05 modernizes the same idea with actors (S2-A1).

1. Why threads race (intern story)

1.1 Shared mutation without rules

```
var cache: [String: Data] = [:]

// Thread A
cache["a"] = Data()

// Thread B (same time)
_ = cache["a"]
```

Dictionaries are **not** thread-safe. Concurrent mutation → data races → intermittent crashes / corruption. Hard to reproduce; shows up under load.

Say aloud: “If two threads touch shared mutable memory without synchronization, that’s a race — undefined behavior territory.”

1.2 What we want instead

Property	Meaning
Mutual exclusion for writes	No two writers overlap unsafely
Defined read visibility	Readers see a coherent state
Safe API	Callers cannot touch raw storage
No self-deadlock	Avoid sync onto the queue you’re on

2. Dispatch queues — the boxes

2.1 Serial vs concurrent

Queue	Behavior	Picture
Serial	One task at a time; FIFO start order	Single checkout lane
Concurrent	Tasks may run overlapping	Many lanes; start order FIFO, finish order varies

```
let serial = DispatchQueue(label: "app.safe.dict")
let concurrent = DispatchQueue(label: "app.images", attributes: .concurrent)
```

2.2 Main queue

- Serial queue tied to the main thread
- **UIKit / SwiftUI UI updates belong here**
- Never do heavy JSON parse / image decode synchronously on main

```
DispatchQueue.main.async {
    self.label.text = value
}
```

2.3 Global queues + QoS

```
DispatchQueue.global(qos: .userInitiated).async { ... }
```

QoS (high → low intuition)	Use
.userInteractive	Tiny, UI-critical work
.userInitiated	User waiting (fetch for screen)
.default	General
.utility	Progress-visible long work
.background	Cleanup, prefetch that can wait

Senior line: Don’t mark everything `.userInteractive` — you steal energy and starve work.

3. async vs sync

3.1 Definitions

Call	Meaning
async	Schedule block; return immediately
sync	Schedule block; wait until it finishes ; return results

```
queue.async {  
    // runs later (soon) on queue  
}  
  
let value = queue.sync { () -> Int in  
    return storage.count  
}
```

3.2 Deadlock rule (learn early)

Never `sync` onto the same serial queue you are already executing on.

Classic: main thread does `DispatchQueue.main.sync { ... }` → main waits for main → deadlock.

Also true for **any** private serial queue:

```
serial.async {  
    serial.sync { // DEADLOCK – already on serial  
        print("never")  
    }  
}
```

Say aloud: "Sync waits. If the waiter is the only worker that can run the work, you freeze."

3.3 When sync is useful

- Need a **return value** from the queue (read)
- Need **happens-before** with prior tasks on that queue
- Keep critical sections short

When async is better:

- Fire-and-forget updates
- Avoid blocking callers (especially main)

4. Thread-safe dictionary — Pattern A (serial)

4.1 Shape used in interviews (and S2)

```
final class SafeDict<Key: Hashable, Value> {  
    private var storage: [Key: Value] = [:]  
    private let queue = DispatchQueue(label: "safe.dict")  
  
    func get(_ key: Key) -> Value? {  
        queue.sync { storage[key] }  
    }  
}
```

```
func set(_ key: Key, value: Value) {
    queue.async { self.storage[key] = value }
}
}
```

Notes:

- Spell **final class** (Swift keyword `final`, lowercase).
- Queue is **private** — callers never see it.
- `get` uses `sync` so it can return a value and wait for prior scheduled work.
- `set` often uses `async` so writers don't block — **but see visibility caveat below**.

4.2 CRITICAL: async write then sync read

```
dict.set("k", value: 1) // async – schedules write
let v = dict.get("k")   // sync – may run BEFORE the write if write hasn't started!
```

Fact: On a serial queue, tasks run in submission order *once enqueued*. But:

- If `set` uses `async`, the write is only ordered **after it has been enqueued**.
- A `get` that is `sync` 'd **immediately** after `set` returns will wait for work **already on the queue**.
- In practice, the `async` block is usually enqueued before `get` 's `sync` runs — so you often see the write.
- Relying on wall-clock assumptions across threads without documenting API guarantees is fragile.
- **If the caller needs read-after-write certainty from the same call site, use `sync` write (or a single `sync` transaction).**

Preferred when read-after-write matters:

```
func set(_ key: Key, value: Value) {
    queue.sync { self.storage[key] = value }
}
```

Or expose:

```
func mutate(_ body: (inout [Key: Value]) -> Void) {
    queue.sync { body(&storage) }
}
```

Interview line:

"Async write plus sync read is a common pattern, but if I need the write visible to the next read on that API, I sync the write — async write may not have run yet when the caller's next line executes."

4.3 Never return a mutable interior reference

```
// BAD
func unsafeStorage() -> [Key: Value] { storage } // races + escapes protection

// GOOD
func snapshot() -> [Key: Value] {
    queue.sync { storage } // value copy (Dictionary COW helps)
}
```

5. Pattern B — concurrent queue + barrier (reader-writer)

```
final class BarrierDict<Key: Hashable, Value> {
    private var storage: [Key: Value] = [:]
    private let queue = DispatchQueue(label: "rw.dict", attributes: .concurrent)

    func get(_ key: Key) -> Value? {
        queue.sync { storage[key] } // concurrent readers OK
    }

    func set(_ key: Key, value: Value) {
        queue.async(flags: .barrier) { // exclusive writer
            self.storage[key] = value
        }
    }
}
```

Operation	Flag	Effect
Read	plain sync/async	May overlap with other reads
Write	.barrier	Waits for prior reads; excludes others until done

When: Read-heavy maps.

Cost: Harder to reason about; writer starvation possible under constant reads; still hide the queue.

S2 used serial queues generally; RW locks / barriers where read-heavy — say that honestly.

6. DispatchGroup (foundations)

```
let group = DispatchGroup()

for url in urls {
    group.enter()
    download(url) { _ in group.leave() }
}

group.notify(queue: .main) {
    // all done
}
```

Use: fan-out prefetch, then merge on main.

Trap: forget `leave` → notify never fires.

7. Semaphores (awareness only)

```
let sem = DispatchSemaphore(value: 4) // max 4 in flight
sem.wait()
// work
sem.signal()
```

Limit concurrency (image decodes). Easy to deadlock if `wait` on wrong queue. Prefer structured concurrency / TaskGroup in modern code (Day 05).

8. Race vs deadlock vs priority inversion

Term	One-liner
Race	Unsynchronized shared mutable access
Deadlock	Wait circle — including sync to current serial queue
Priority inversion	Low-priority holder blocks high-priority waiter

9. Actors preview (one paragraph)

Swift `actor` gives language-enforced isolation for the same problem SafeDict solves with a serial queue. For **new** code you'd evaluate an actor (How I would apply it · S2-A1). Today's depth is GCD — the production story you shipped.

10. Glossary

Term	One-liner
Serial queue	Tasks one-at-a-time
Concurrent queue	Tasks may overlap
Barrier	Exclusive section on concurrent queue
QoS	Priority / energy class for work
sync	Wait for block completion
async	Schedule and return
Safe API	Hide storage + queue; expose methods
Snapshot	Value copy returned under sync

11. First production bridge (short)

Shared async state at BookMyShow was hit from multiple queues → races / intermittent crashes. Fix: synchronised dictionaries gated by GCD serial queues (RW where read-heavy), with a standardized access API so call sites couldn't touch raw storage.

≤20s:

"We gated shared dictionaries behind a serial queue API so call sites couldn't race the storage — crashes went away on that path."

Provenance: Verified · S2 · BookMyShow · synchronised dictionaries

12. Self-check before deep dive

- ☐ Serial vs concurrent in one sentence each?
- ☐ Why main.sync from main deadlocks?
- ☐ Sketch SafeDict get/set?
- ☐ State the async-write / sync-read visibility caveat?
- ☐ Spell `final class` correctly?

→ [02-deep-dive.md](#)

02 — Deep Dive: Deadlocks, Visibility, Barriers, SafeDict

Correctness

Interview depth. Foundations assumed.

1. Queue targeting and thread hops

1.1 async always hops (eventually)

```
networkSession.dataTask(...) { data, _, _ in
    // URLSession callback queue – NOT main by default (config-dependent)
    DispatchQueue.main.async {
        self.viewModel.apply(data)
    }
}
```

Rule: UI on main. Parse off main when heavy.

1.2 sync from background to main

```
// Background thread:
DispatchQueue.main.sync {
    // UI touch – works but BLOCKS background until main runs the block
}
```

Prefer `main.async` for UI unless you have a rare need to wait. Blocking a thread pool worker on main can cause stalls under load.

1.3 Target queue inversion bugs

Passing your private queue into an API that syncs back onto the caller → re-entrancy surprises. **Hide queues; expose methods.**

2. Deadlock patterns (memorize)

2.1 Main sync-to-main

```
// On main:
DispatchQueue.main.sync { print("dead") }
```

2.2 Private serial re-entry

```
final class Service {
    private let q = DispatchQueue(label: "service")

    func a() {
        q.sync {
            self.b() // b also syncs to q → deadlock
        }
    }

    func b() {
        q.sync { print("hi") }
    }
}
```

```
}  
}
```

Fix patterns:

- Use `async` for nested work
- Factor “already on queue” unsafe internals (`_bUnlocked`) called only from queue
- Assert queue with `dispatchPrecondition` in debug

```
private func mutateUnlocked() {  
    dispatchPrecondition(condition: .onQueue(queue))  
    storage["x"] = 1  
}
```

2.3 Cross-queue ABBA deadlock

```
Thread1: lock A → wait B  
Thread2: lock B → wait A
```

With queues: Queue1 sync waits on Queue2 while Queue2 sync waits on Queue1. Keep a **lock/queue hierarchy** — always acquire in one order.

3. Ordering and visibility on serial queues

3.1 FIFO start order

Serial queues start tasks in submission order. That gives a happens-before between task N and task N+1 **on that queue**.

3.2 Async write / sync read — precise statement

```
final class SafeDict<Key: Hashable, Value> {  
    private var storage: [Key: Value] = [:]  
    private let queue = DispatchQueue(label: "safe.dict")  
  
    func setAsync(_ key: Key, value: Value) {  
        queue.async { self.storage[key] = value }  
    }  
  
    func setSync(_ key: Key, value: Value) {  
        queue.sync { self.storage[key] = value }  
    }  
  
    func get(_ key: Key) -> Value? {  
        queue.sync { storage[key] }  
    }  
}
```

Scenario A — sync write then sync read (same thread):

```
dict.setSync("k", value: 1)  
let v = dict.get("k") // ✅ sees 1 — write completed before setSync returned
```

Scenario B — async write then sync read (same thread):

```
dict.setAsync("k", value: 1) // enqueues write; returns immediately
let v = dict.get("k")        // sync: runs after previously enqueued work
```

Usually `v == 1` because the async block was enqueued before `get`'s sync block.

But: documenting "set is fire-and-forget" means callers must not assume immediate visibility across *other* threads without joining, and you should not treat async set as a completed write.

Scenario C — async write; read from another thread without sync API:

If someone races on a leaked `storage` reference — undefined. Hence private storage.

Interview-grade sentence:

"If I need read-after-write on the call site, I use a synchronous set — or one sync transaction — because an async write only guarantees ordering once it's on the queue, and the caller's next line isn't a completion handler."

3.3 Returning values vs mutating in place

Prefer returning **values** (structs, copied Dictionary snapshots). Never hand out `:inout` storage or class wrappers that mutate outside the queue.

4. Concurrent + barrier deep dive

4.1 Mental model

```
Readers:  R R R R      (may overlap)
Barrier:   |W|         (exclusive)
Readers:   R R
```

4.2 Incorrect "concurrent dict"

```
// ❌ DATA RACE
concurrent.async { storage[k] = v } // write without barrier
concurrent.async { _ = storage[k] }
```

4.3 Writer starvation

If readers constantly enter, a barrier writer may wait a long time. Mitigations: QoS, batching writes, or fall back to serial if simplicity > read parallelism.

4.4 S2 honesty

"We used serial queues for synchronised dictionaries, and read-write locks where access was read-heavy."

Don't claim you always used barriers everywhere.

5. Designing the SafeDict API (senior)

5.1 Checklist

Requirement	Implementation
Hide storage	<code>private var storage</code>
Hide queue	<code>private let queue</code>
Thread-safe get	<code>queue.sync</code>

Thread-safe set	queue.sync or carefully documented async
Snapshot	sync copy
No queue escape	never return DispatchQueue
final class	avoid subclassing surprises

5.2 Full teaching implementation

See [code/SafeDict.swift](#) — prefer **sync set** for correctness teaching; comment the async alternative.

5.3 Stress testing mindset

- Parallel writers + readers via `DispatchQueue.concurrentPerform`
- Assert invariants
- Crashlytics watch in prod (S2)

6. DispatchGroup pitfalls

Pitfall	Fix
Unbalanced enter/leave	defer { leave() } after enter
notify on wrong queue	Choose main for UI merge
Forever wait	wait with timeout in tools; avoid blocking main

7. Locks vs queues vs actors

Tool	When	Cost
Serial queue wrapper	Simple shared maps (S2)	Sync read latency under load
Concurrent + barrier	Read-heavy	Complexity, starvation
NSLock / os_unfair_lock	Tiny critical sections	Easy to forget unlock; priority inversion
actor	New Swift code (S2-A1)	async API surface; reentrancy across await

Migration pitch (not a shipped claim):

“Same safe API — get/set/snapshot — implemented by an actor; callers await. Isolation moves from convention to the type system.”

8. Mixing GCD with async/await (trap preview)

```
func bridge() async -> Int {
    await withCheckedContinuation { cont in
        queue.async {
            cont.resume(returning: self.storage.count)
        }
    }
}
```

Traps:

- Calling `queue.sync` from an async context can block a thread in the cooperative pool — bad.

- Prefer actors or async wrappers that don't sync-block.

Day 05 deepens this; today just know: **don't casually sync from async world.**

9. QoS and priority inversion

If a `.background` task holds a lock needed by `.userInteractive`, the UI waits. GCD QoS inheritance helps in some queue designs; locks are easier to get wrong. Another reason "serialize at one queue boundary" is nicer than ad-hoc locks.

10. Trade-off table (memorize)

Choice	When	Cost
Serial queue <code>SafeDict</code>	Default shared maps	Readers queue behind writers
RW barrier	Read-heavy large maps	Complexity
Sync set	Read-after-write needed	Caller blocks on write
Async set	Throughput; eventual visibility OK	Visibility surprises
Expose queue	Never for S2-style safety	Call sites reintroduce races
Actor rewrite	Greenfield / migrate module	API becomes async

11. Whiteboard: 2-minute `SafeDict` talk

1. Problem: shared dict raced across queues → crashes.
 2. Design: `final class` wrapper; private storage; private serial queue.
 3. API: sync get; sync set (explain why not async if asked visibility).
 4. Alternative: barrier RW for read-heavy.
 5. Trade-off: actor today for new code.
 6. Result: races on that path eliminated; pattern reused.
-

12. Anti-patterns

Anti-pattern	Fix
Final class spelling	<code>final class</code>
Public var storage	private + snapshot
sync to self	unlocked internals / async
Concurrent writes w/o barrier	barrier or serial
Claiming S2 fixed all app crashes	Path-specific race fix
Returning mutable reference	return values

13. Code tour

1. [code/SafeDict.swift](#) — line-by-line aloud.
 2. [code/BarrierDict.swift](#) — contrast.
 3. Predict: async set + immediate get on another thread using only public API.
-

14. Interview micro-scripts (pin)

SafeDict in 45s:

"final class, private dictionary, private serial queue. Sync get and sync set so read-after-write is defined. Snapshot returns a copy. Call sites never see storage or the queue — that was the BookMyShow fix for raced shared maps."

Visibility in 20s:

"Async write then sync read may not see the write until the write runs — for call-site certainty I sync the write."

Actor coda in 20s:

"Same API as an actor for new modules — Applied S2-A1, not a claim we rewrote production."

Spelling check: say and type `final class` — never `Final class` .

→ [03-production-bridge.md](#)

03 — Production Bridge

Resume-honest lines for GCD / synchronised dictionaries.

Verified · **S2** core · **How I would apply it** · **S2-A1** actor coda · **Learning-lab** code.

1. Story map

Concept	Label
Serial-queue synchronised dictionaries	Verified · S2
RW locks where read-heavy	Verified · S2
Standardized access API (no raw storage)	Verified · S2
Crash/race reduction on that path	Verified · S2
Greenfield actor migration	How I would apply it · S2-A1
Crashlytics validation culture	Verified · S8 soft (don't attribute CFS solely to S2)
SafeDict.swift	Learning-lab

2. Verified · S2 — Synchronised dictionaries

What you can say (safe)

- Shared async state hit from multiple queues → data races / intermittent crashes
- Introduced synchronised dictionaries gated by **GCD serial queues**
- Used **read-write locks** where access was read-heavy
- Standardized access API so call sites couldn't touch raw storage
- Validated under concurrency stress and Crashlytics watch
- Eliminated concurrent-access crashes **in that shared state path**
- Pattern reused where mutable maps were shared across async work
- Lesson: serialize mutation at the boundary; don't sprinkle locks ad hoc
- Today you'd also evaluate a Swift `actor` for new code

What you must not invent

- Exact crash counts, percentages fixed by this work alone
- "I alone brought the app to 99.95% CFS" (CFS is S8 culture metric — don't steal it for S2)
- Claiming every dictionary in the app was converted
- Claiming production used your Learning-lab file literally

Interview lines

≤20s:

"We gated shared dictionaries behind a serial queue API so call sites couldn't race the storage — crashes went away on that path."

Timed opener:

"We had races on shared dictionaries — I'll cover the serial-queue design and trade-offs vs actors."

STAR Action (~90s):

"Shared mutable dictionaries were accessed from multiple queues, which produced data races and intermittent crashes. I introduced synchronised dictionary wrappers gated by GCD serial queues — and read-write locking where the access pattern was read-heavy — and standardized the access API so call sites could not touch raw storage. We validated under concurrency stress and watched Crashlytics. The races on that path went away, and we reused the pattern wherever shared async maps showed up."

Result:

"Eliminated concurrent-access crashes in that shared state path; pattern reused for similar shared maps."

Lesson + coda:

"Serialize at the boundary. For greenfield code today I'd evaluate a Swift actor with the same safe API surface."

Provenance: Verified · S2 · BookMyShow · synchronised dictionaries

3. How I would apply it · S2-A1

Say explicitly this is design direction, not a claim you rewrote production:

"Production used GCD. How I'd apply it in a new module is an actor SafeDict with get/set/snapshot — callers await, isolation is in the type system, same boundary idea."

Point at Day 05 [../day-05/code/SafeDictActor.swift](#) as Learning-lab.

Provenance: How I would apply it · S2-A1

4. Mapping concepts → lines

Question	Lead	Support
Serial vs concurrent	Definitions	SafeDict uses serial
Thread-safe dict design	Private queue API	S2
async vs sync write	Visibility caveat	Prefer sync for read-after-write
Barrier	RW pattern	S2 read-heavy mention
Deadlock	sync re-entry	Any serial queue
Actors	S2-A1	Don't claim shipped actors for S2
Why hide queue	Call sites re-race	S2 standardized API

5. Anti-patterns

Anti-pattern	Fix
--------------	-----

"Fixed all crashes at BMS"	Path-specific race elimination
Inventing crash %	Qualitative intermittent races → eliminated on path
Final class in code samples	final class
"Async set always visible on next line"	Teach sync set / visibility
Skipping actor coda when asked modern approach	S2-A1 honest label

6. Flash map card

Company / feature: BookMyShow — synchronised dictionaries

What you did: GCD serial queues / RW locks; safe access API; stress + Crashlytics validation.

Interview line (≤20s):

"We gated shared dictionaries behind a serial queue API so call sites couldn't race the storage — crashes went away on that path."

→ [S2](#)

7. Timed drills

Drill	Budget
≤20s pitch	20s
Full S2 STAR	≤3 min
SafeDict whiteboard	2 min
S2 → S2-A1 migration	90s

Next

[04-questions.md](#) — Answer points first, then Full spoken answer.

04 — Questions (two-layer Q&A)

Cover **Full spoken answer**. Speak from **Answer points**. Uncover and compare.

Normal ≈ 30–60s · Design Q4 ≈ 90–120s · Tricky ≈ 90–120s.

Normal questions

Q1. Serial vs concurrent queue? (30–45s)

Answer points (frame first):

- Serial: one task at a time
- Concurrent: tasks may overlap
- SafeDict uses private serial for mutex-by-queue
- Main is serial (UI)

Agenda opener:

"One-at-a-time versus overlapping execution."

Full spoken answer:

"A serial queue runs one block at a time in FIFO start order — it's mutual exclusion via queue. A concurrent queue can run multiple blocks overlapping. The main queue is serial and owns UI work. For synchronised dictionaries at BookMyShow we used a private serial queue as the exclusion boundary so shared map access couldn't race."

Common wrong answer:

"Concurrent queues are always faster so always use them for shared mutable state."

Follow-up ladder:

- **L1:** What is the main queue?
- **L2:** When prefer concurrent + barrier?
- **L3:** How does QoS interact?

Provenance: Verified · S2 · serial-queue dictionaries

Q2. async vs sync? (30–45s)

Answer points (frame first):

- `async`: schedule, return immediately
- `sync`: wait for completion (can return value)
- sync-to-current-serial-queue deadlocks
- UI hops usually `async` to main

Agenda opener:

"Wait for completion versus schedule and return."

Full spoken answer:

"`async` enqueues work and returns immediately. `sync` enqueues and waits until the block finishes — useful when you need a return value. The danger is syncing onto the serial queue you're already on, including `main.sync` from main — that deadlocks. For UI updates after networking I hop with `main.async`, not `sync`, so I don't block the callback thread waiting on the main run loop unnecessarily."

Common wrong answer:

"`sync` is safer because you always know it's done — so `sync` everywhere."

Follow-up ladder:

- **L1:** Why does `main.sync` from main deadlock?
- **L2:** Can private serial queues deadlock the same way?
- **L3:** `sync` from `async/await` contexts — concern?

Provenance: Learning-lab · GCD mechanics

Q3. Why not sync to main from main? (45s)

Answer points (frame first):

- Main waits for a block that only main can run
- Classic deadlock
- Same pattern for any serial queue re-entry
- Use `async` for UI hops

Agenda opener:

"You're waiting for yourself."

Full spoken answer:

"If you're already on the main queue and call `DispatchQueue.main.sync`, the main thread blocks waiting for that block to run, but that block can't run until main is free — deadlock. The same self-wait happens on any serial queue if you sync re-enter it. So UI work from main should just run directly or via async for deferred work — never `main.sync` from main."

Common wrong answer:

"Only the main queue can deadlock; private queues are safe."

Follow-up ladder:

- **L1:** Show private serial re-entry deadlock.
- **L2:** How do you structure unlocked internals?
- **L3:** `dispatchPrecondition` usage?

Provenance: Learning-lab · deadlock patterns

Q4. How do you implement a thread-safe dictionary? (90–120s)

Answer points (frame first):

- `final class` wrapper; private storage; private serial queue
- sync get; sync set when read-after-write matters
- Never expose storage/queue
- Optional: concurrent + barrier if read-heavy
- S2 story hook + actor coda

Agenda opener:

"Private serial queue — safe API, no raw storage."

Full spoken answer:

"I'd wrap the dictionary in a `final class`, keep storage private, and keep a private serial `DispatchQueue`. Reads go through `queue.sync` so I can return a value and respect prior writes already on the queue. Writes I'd make `queue.sync` when callers need read-after-write — async write is fine for fire-and-forget but the next line's sync read isn't a completion handler. Snapshots return a copied dictionary under sync. The critical product lesson from BookMyShow was standardizing that API so call sites couldn't touch raw storage — that removed races on that path. For read-heavy maps we also used RW locking. Today for new modules I'd evaluate a Swift actor with the same surface."

Common wrong answer:

"Just mark methods synchronized" / expose the queue / use concurrent writes without barriers / `Final class`.

Follow-up ladder:

- **L1:** Why not return `storage` directly?
- **L2:** Async set then immediate get — visibility?
- **L3:** Migrate to actor without big-bang?

Provenance: Verified · S2; How I would apply it · S2-A1; Learning-lab · SafeDict.swift

Q5. QoS levels — why care? (45s)

Answer points (frame first):

- Priority and energy
- Don't mark everything `userInteractive`
- Impacts scheduling under load
- Background work shouldn't starve UI carelessly (and vice versa with locks)

Agenda opener:

"Priority and energy — not all work is interactive."

Full spoken answer:

"Quality of Service tells GCD how urgent work is — from userInteractive down to background. It matters for responsiveness and battery. If I classify analytics batching as userInteractive, I compete with real UI work. I pick QoS to match user expectation: fetches the user is waiting on higher; prefetch and cleanup lower."

Common wrong answer:

"Always use the highest QoS so things finish faster."

Follow-up ladder:

- **L1:** List QoS levels roughly.
- **L2:** Priority inversion with locks?
- **L3:** QoS vs Task priority in Swift concurrency?

Provenance: Learning-lab · scheduling judgment

Q6. DispatchGroup use case? (45s)

Answer points (frame first):

- Fan-out parallel work; notify when all done
- enter/leave balance
- Prefetch then merge on main
- Trap: forgotten leave

Agenda opener:

"Join parallel work — then one notify."

Full spoken answer:

"DispatchGroup is for fan-out/fan-in: enter before each async child, leave when it finishes, notify when the count hits zero. I use it for parallel prefetches then merge results on the main queue. The classic bug is forgetting leave, so notify never fires — I prefer `defer { leave() }` right after enter when the structure allows."

Common wrong answer:

"Group replaces the need for thread-safe storage."

Follow-up ladder:

- **L1:** notify queue choice?
- **L2:** wait vs notify?
- **L3:** TaskGroup equivalent?

Provenance: Learning-lab · fan-out patterns

Q7. Barrier flag purpose? (45s)

Answer points (frame first):

- Exclusive work on concurrent queue
- Reader-writer: concurrent reads, barrier writes
- Without barrier, concurrent writes race
- Still hide the queue

Agenda opener:

"Exclusive writer on a concurrent queue."

Full spoken answer:

"A barrier block on a concurrent queue waits for previously started work and runs exclusively — no other blocks overlap it. That's the GCD reader-writer pattern: normal sync reads may overlap; writes use `.barrier`. If you write without a barrier on a concurrent queue guarding a dictionary, you still have a data race. At BookMyShow we used RW approaches where maps were read-heavy; otherwise serial queues kept the mental model simpler."

Common wrong answer:

"Concurrent queue makes dictionary access safe by itself."

Follow-up ladder:

- **L1:** Writer starvation?
- **L2:** async barrier vs sync barrier visibility?
- **L3:** Serial vs RW trade-off?

Provenance: Verified · S2 · RW where read-heavy; Learning-lab · BarrierDict

Q8. Main thread rule for UI? (30s)

Answer points (frame first):

- UIKit/SwiftUI updates on main
- Heavy work off main
- Hop with async after network
- Don't parse huge JSON on main

Agenda opener:

"UI on main — heavy work elsewhere."

Full spoken answer:

"UIKit and SwiftUI UI updates must happen on the main queue. Networking callbacks often aren't on main, so I hop with `DispatchQueue.main.async` before binding views. I keep JSON parsing and image decode off the main thread so scrolling stays smooth."

Common wrong answer:

"SwiftUI removes the main-thread rule."

Follow-up ladder:

- **L1:** `@MainActor` relationship?
- **L2:** What breaks if you update UI off main?
- **L3:** Instruments for main-thread hangs?

Provenance: Learning-lab · UI rule

Q9. Race vs deadlock? (45s)

Answer points (frame first):

- Race: unsynchronized shared mutation
- Deadlock: wait circle / sync to self
- Races intermittent; deadlocks freeze
- S2 was race-class bugs

Agenda opener:

"Corruption versus freeze."

Full spoken answer:

"A race is unsynchronized access to shared mutable state — intermittent corruption or crashes. A deadlock is when threads or queues wait on each other forever — including syncing to the serial queue you're on. At BookMyShow the synchronised-

dictionary work targeted races on shared maps; deadlock avoidance is the companion discipline when you introduce queues and locks.”

Common wrong answer:

Using the words interchangeably.

Follow-up ladder:

- **L1:** How do you detect each in tools?
- **L2:** Example of ABBA deadlock?
- **L3:** Actor eliminates which class of bug?

Provenance: Verified · S2 · race reduction on path

Q10. When semaphore over group? (45s)

Answer points (frame first):

- Semaphore limits in-flight concurrency
- Group joins a known set of tasks
- Semaphore easy to misuse / deadlock
- Prefer TaskGroup later for structured limits

Agenda opener:

“Limit in-flight work versus join a batch.”

Full spoken answer:

“Use a group when you have a batch of tasks and need one continuation when they’re all done. Use a semaphore when you need a concurrency limit — at most N image decodes at once. Semaphores are easy to deadlock if you wait on the wrong queue, so I’m cautious; in modern Swift I’d rather reach for task grouping and explicit limits.”

Common wrong answer:

“Semaphore is the general replacement for serial queues.”

Follow-up ladder:

- **L1:** value of semaphore = 1 means?
- **L2:** Deadly embrace with sync?
- **L3:** OperationQueue maxConcurrent?

Provenance: Learning-lab · concurrency limiting

Q11. DispatchQueue.main.async after network? (30s)

Answer points (frame first):

- Callback often background
- Hop to main before UI bind
- Keep model apply thread-safe
- Don’t block with main.sync casually

Agenda opener:

“Network off main — bind on main.”

Full spoken answer:

“URLSession completions typically aren’t on the main queue, so after I parse I dispatch async to main before touching UIKit. That keeps UI work legal without deadlocking via sync. If the view model is @MainActor, the hop may be structured differently, but the rule remains: UI affinity on main.”

Common wrong answer:

Always main.sync after network for 'safety'.

Follow-up ladder:

- **L1:** Where should parsing happen?
- **L2:** Cancellation if VC gone?
- **L3:** Combine/async alternatives?

Provenance: Learning-lab · UI hop

Tricky questions

T1. Read with sync + write with async on same serial queue — ordering? (90s)

Answer points (frame first):

- Serial FIFO for enqueued work
- Async write returns before write runs
- Sync read waits for work already enqueued
- For call-site read-after-write certainty → sync write

Trap:

Assume async write is always visible to the next statement as if it completed.

Full spoken answer:

"On a serial queue, tasks run one-by-one in enqueue order. An async write schedules the mutation and returns immediately; a following sync read will wait for previously enqueued tasks — so if the write was enqueued first, the read usually sees it. But async write is not a completion handshake for the caller. If the API promises read-after-write on the next line, I implement set with sync — or expose a single sync mutate transaction. At BMS we cared about a race-free boundary more than micro-optimizing async writes."

Follow-up ladder:

- **L1:** What if another thread calls get without going through the API?
- **L2:** Would you async get?
- **L3:** Actor equivalent of sync set?

Provenance: Learning-lab · visibility caveat; applies to S2 API design

T2. Concurrent queue + non-barrier write (90s)

Answer points (frame first):

- Data race / UB on Swift Dictionary
- Concurrent ≠ thread-safe storage
- Need barrier writes or serial queue
- Intermittent crashes match S2 symptom class

Trap:

"Concurrent is fine for dict because reads are parallel."

Full spoken answer:

"A concurrent queue only allows blocks to overlap — it does not make Dictionary thread-safe. If two blocks mutate storage without a barrier, that's a data race. The fix is barrier writes with concurrent reads, or a serial queue for simplicity. The intermittent crashes we saw on shared maps are exactly this failure class — which is why we put a real exclusion boundary and a safe API in front of the storage."

Follow-up ladder:

- **L1:** Can two barrier writes overlap?
- **L2:** Instruments Thread Sanitizer?
- **L3:** Serial vs RW choice criteria?

Provenance: Verified · S2 symptom class; Learning-lab

T3. sync from serial queue to itself via nested call (90s)

Answer points (frame first):

- Not only a main-queue problem
- Nested sync on same serial queue deadlocks
- Use unlocked internals / async / assert queue
- Design APIs to avoid re-entry sync

Trap:

"Only main deadlocks on sync."

Full spoken answer:

"Any serial queue can deadlock on sync re-entry. If `a()` does `queue.sync` and inside that calls `b()` which also `queue.sync`s, you're waiting for yourself. I fix it by splitting locked wrappers from unlocked internals that assume they're already on the queue, or by making the nested path async. Debug asserts with `dispatchPrecondition` catch accidental off-queue use."

Follow-up ladder:

- **L1:** Sketch `_setUnlocked` .
- **L2:** Recursion through delegates while holding queue?
- **L3:** Actor reentrancy contrast (Day 05)?

Provenance: Learning-lab · deadlock patterns

T4. RW lock vs serial queue — when? (120s)

Answer points (frame first):

- Serial: simple, correct mental model
- RW: read-heavy benefit
- Costs: complexity, writer starvation
- S2: serial default; RW where read-heavy

Trap:

Always RW because 'optimized'.

Full spoken answer:

"I default to a serial queue wrapper — it's easy to reason about and enough for many shared maps. If profiling shows read-heavy access and reader queuing hurts, a reader-writer design — GCD barriers or RW locks — can let reads overlap. The costs are complexity and possible writer starvation under constant reads. At BookMyShow we used serial queues for synchronised dictionaries and RW locks where access was read-heavy. I wouldn't start with RW for fashion."

Follow-up ladder:

- **L1:** How do you know it's read-heavy?
- **L2:** Actor instead?
- **L3:** Returning snapshots vs holding locks across IO?

Provenance: Verified · S2 · serial + RW where read-heavy

T5. Priority inversion with locks (90s)

Answer points (frame first):

- Low priority holds resource high priority needs
- UI can stall
- Prefer short critical sections
- Queue QoS / system inheritance helps some designs

Trap:

Ignore QoS and lock duration.

Full spoken answer:

“Priority inversion is when a low-priority task holds a lock or resource a high-priority task needs — the high-priority work waits behind low-priority execution. On iOS that can feel like UI jank. I keep critical sections tiny, avoid doing IO under lock, and I’m careful with QoS. Another reason I like a single serial-queue boundary for maps: the coordination story stays clearer than ad-hoc locks everywhere.”

Follow-up ladder:

- **L1:** os_unfair_lock vs NSLock?
- **L2:** Holding lock across await — why bad?
- **L3:** How would TSan vs Time Profiler show symptoms?

Provenance: Learning-lab · concurrency hazards

T6. Exposing internal queue to callers (90s)

Answer points (frame first):

- Callers async onto your queue incorrectly
- Sync re-entry / ordering bugs
- Bypass safe API → races return
- S2 lesson: standardize access API

Trap:

Let callers `async` onto your queue ‘for flexibility’.

Full spoken answer:

“If you expose the queue, callers will schedule arbitrary work on it — including sync re-entry patterns you didn’t anticipate — or they might touch storage again and reintroduce races. The BookMyShow lesson was to standardize a safe access API so call sites couldn’t touch raw storage or the coordination primitive. Flexibility belongs in higher-level methods like `mutate` and `snapshot`, not in leaking `DispatchQueue`.”

Follow-up ladder:

- **L1:** Is a read-only queue accessor ever OK?
- **L2:** How do you unit-test without exposing queue?
- **L3:** Actor makes this leak impossible how?

Provenance: Verified · S2 · standardized access API

Timed sets

Set	Items
A	Q4, Q2, T3, T4
B	Q1, Q7, T1, T2
C	Full S2 STAR + T6 + S2-A1 90s

Score with [../../../../../timing/answer-timing-guide.md](https://github.com/robertatkinson/swift-timing/answer-timing-guide.md) .

05 — Exercises

After foundations + deep dive + production bridge. Speak while coding.

Exercise 1 — SafeDict (25–35 min)

Implement [code/SafeDict.swift](#) from memory:

1. `final class SafeDict<Key: Hashable, Value>`
2. private storage + private serial queue
3. `get / set (sync) / snapshot / count`
4. Optional `setAsync` with a comment on visibility

Agenda (20s):

"final class, hide storage and queue, sync get/set for read-after-write."

Done when: You can explain why sync set is the teaching default.

Exercise 2 — Race contrast (20–30 min)

Prompt: Write a tiny harness (playground or unit test ideas):

1. Unsynchronized `[String: Int]` mutated from many queues — expect chaos conceptually.
2. Same operations through `SafeDict` — expect coherent counts.

Use `DispatchQueue.concurrentPerform` or many `global().async` + group notify.

Speak (45s): What S2 fixed at the API boundary.

Exercise 3 — Deadlock lab (15–20 min)

Prompt: Deliberately write (but don't leave in prod):

```
queue.async {
    queue.sync { print("?") }
}
```

Predict hang. Then refactor to unlocked internal method pattern.

Speak (30s): "Not only main — any serial re-entry sync."

Exercise 4 — BarrierDict (20–25 min)

Implement [code/BarrierDict.swift](#) .

Speak trade-off vs serial SafeDict in 60s. Mention S2 RW-where-read-heavy.

Exercise 5 — Visibility experiment (15 min)

Compare:

```
dict.setAsync("k", 1)
```

```
print(dict.get("k"))
```

vs `sync` `set` . Write one paragraph in your notes stating the interview-safe rule.

Exercise 6 — Timed speaking (30–40 min)

Record:

1. Q4 thread-safe dictionary (2 min)
2. Q2 `async` vs `sync`
3. T1 visibility
4. T3 re-entry deadlock
5. Full S2 STAR ≤ 3 min

Score with timing guide. Zero invented crash %.

Exercise 7 — Actor coda (10–15 min)

Read Day 05 [../day-05/code/SafeDictActor.swift](https://github.com/uber/swift/blob/master/Day05/code/SafeDictActor.swift) .

Speak S2 $\rightarrow$ S2-A1 in ≤ 90 s with honest provenance labels.

Exercise 8 — Visibility assertion note (10 min)

Write in your gotchas file (exact wording):

"Async write then sync read may not see the write until the write runs; use sync write for read-after-write."

Speak it once. Check `SafeDict.set` vs `setAsync` comments.

Exercise 9 — Spelling drill (5 min)

Type five times correctly:

```
final class SafeDict<Key: Hashable, Value> {
```

Never `Final` class .

Exit criteria

- ☐ Sketch `SafeDict` without notes (`final class` spelled right)
- ☐ State `async`-write / `sync`-read caveat correctly
- ☐ Explain barrier RW in 45s
- ☐ S2 STAR ≤ 3 min $\geq$ score 4 target
- ☐ S2-A1 coda without claiming shipped actors
- ☐ Visibility sentence memorized

Revision twin: [../../revision/weeks/week-01/day-04.md](https://github.com/uber/swift/blob/master/revision/weeks/week-01/day-04.md)

Day 05 — `async/await`, Structured Concurrency, Actors, Sendable

Week 1 · Full study (self-contained) · ~4–5 hrs

Revision twin: [revision/weeks/week-01/day-05.md](https://github.com/uber/swift/blob/master/revision/weeks/week-01/day-05.md)

Outcomes

By end of day, without notes, you can:

- Explain `async / await` as cooperative suspension (not “background thread magic”)
- Contrast structured concurrency (`async let` , `TaskGroup`) with unstructured `Task` / `Task.detached`
- Describe cooperative Task cancellation and why search debounce must cancel in-flight work
- Explain actor isolation, **reentrancy** (state may change across `await`), and `@MainActor`
- State `Sendable` rules correctly — including that value types are `Sendable` **only if** all stored properties are
- Pitch the BMS synchronised-dictionary story (Verified · S2) and a greenfield actor migration (How I would apply it · S2-A1)
- Speak carefully about Swift 6.2 / Approachable Concurrency / default `MainActor` isolation as **settings when enabled**, not universal language law

How to study (in Cursor only)

1. `01-foundations.md` — mental model + glossary (intern → competent)
2. `02-deep-dive.md` — full mechanics, reentrancy, `Sendable`, settings caveats
3. `03-production-bridge.md` — S2 / S2-A1 / S3 hooks and interview lines
4. `code/SafeDictActor.swift` — read, then explain aloud line by line
5. `04-questions.md` — cover Full spoken answers; speak from **Answer points**; compare
6. `05-exercises.md` — coding + speaking drills
7. Revision twin for timed drill after the full read

Module map

Module	File
Foundations	01-foundations.md
Deep dive	02-deep-dive.md
Production bridge	03-production-bridge.md
Questions	04-questions.md
Exercises	05-exercises.md
Code	code/SafeDictActor.swift

Provenance reminder

Only use Verified IDs from [../..../provenance/README.md](#) .

Label	What you may say
Verified · S2	GCD serial queues / RW locks around shared dictionaries at BookMyShow; race/crash reduction on that path (do not claim sole ownership of app-wide crash-free %)
How I would apply it · S2-A1	Greenfield shared maps → Swift actor with the same safe API surface
Verified · S3	Search debounce + explicit loading/empty/error state + MVVM at BMS
Learning-lab	Illustrative snippets (<code>SafeDictActor</code> , reentrancy demos) not claimed as shipped

Source / version note

Teaching targets **Swift concurrency as available in modern Xcode** (`async/await`, actors, `Sendable`). Behaviors that depend on **Swift 6 language mode**, **Approachable Concurrency**, or **default actor isolation** settings are called out explicitly as *when*

enabled — they are not presented as “how every Swift app works.”

Timed drill (after full study)

1. Speak Q4, Q5, T1, T3 from 04-questions.md on a timer.
2. 90s pitch: “How would you migrate synchronised dictionaries to actors?” (S2 → S2-A1).
3. Score against [answer-timing-guide.md](#); log misses.

Day 05 — Foundations: async/await, Tasks, Actors, Sendable

Read this first. Intern-friendly mental models, then a short bridge to senior vocabulary.
Deep mechanics live in [02-deep-dive.md](#).

1. Why this day exists

Yesterday (Day 04) you learned **GCD**: queues, `sync / async`, and how a serial queue can protect a shared dictionary. That pattern shipped at BookMyShow as **synchronised dictionaries** (Verified · S2).

Today is the **modern language-native** half of the same problem:

- How do you write asynchronous work without callback pyramids?
- How do you keep parent/child work structured so cancellation and errors make sense?
- How do you protect shared mutable state with an `actor` instead of (or after) a GCD serial queue?
- How does the compiler help you avoid data races with `Sendable`?

You do **not** throw GCD away. Large apps mix both. The senior skill is knowing which tool for which boundary — and how to migrate safely.

2. Plain-English mental model

2.1 “Async” means “I might pause here”

An `async` function can **suspend**. Suspension is not the same as blocking a thread.

Idea	What it means
Blocking	A thread sits idle waiting (e.g. <code>DispatchQueue.sync</code> waiting, or a lock held). That thread can’t do other useful work.
Suspending	The <code>async</code> function yields. The runtime may reuse the thread for other work. Later, the function resumes where it left off.

When you write:

```
let (data, response) = try await URLSession.shared.data(from: url)
```

you are saying: “Fetch this URL. While waiting for the network, don’t hold a thread hostage. When the response is ready, continue.”

Intern takeaway: `await` marks a **possible pause point**. Code after `await` may run later — possibly on a different executor/thread — and other concurrent work may have progressed in between.

2.2 Tasks are units of asynchronous work

A **Task** is “a piece of `async` work the system can schedule.”

You will see three common shapes:

Shape	Plain meaning	When to reach for it
async function + <code>await</code>	Call async work from another async context	Inside structured async code
Task { ... }	Bridge from synchronous code (e.g. a button handler) into async	UI entry points, “start this work”
Task.detached { ... }	Start work that does not inherit the parent’s actor/priority/task-local context the same way	Rare — truly independent background work

Intern takeaway: Prefer staying inside structured async calls. Use `Task { }` when you must leave the sync world. Treat `Task.detached` as a sharp tool, not a default.

2.3 Structured concurrency = parent owns children

Structured concurrency means asynchronous work forms a **tree**:

- A parent starts children.
- The parent does not finish until children finish (or are cancelled).
- Cancellation and error handling can propagate along that tree.

Analogy: a function that calls two helpers waits for both before returning. Structured concurrency is that idea for async work.

API	Role
<code>async let</code>	Fixed, small number of parallel children
<code>withTaskGroup</code> / <code>withThrowingTaskGroup</code>	Dynamic fan-out (N items)
Unstructured Task { }	Escapes the tree — you own lifetime and cancellation yourself

Intern takeaway: If you start work with plain `Task { }` and forget to cancel it, that work can outlive the screen that started it. Search typing is the classic interview example (Verified · S3 debounce).

2.4 Actors = a room with one conversation at a time

An **actor** is a reference type that **isolates** its mutable state.

- Only one task at a time executes on the actor (for its isolated methods/properties).
- From **outside**, calling into an actor usually requires `await` (you may need to wait your turn).
- From **inside**, you can touch the actor’s state without locks.

Analogy: a private office. People knock (`await`). One person talks at a time. When that person steps out to wait for a phone call (`await` something else), **someone else can enter the office** before the first person returns. That last sentence is **reentrancy** — the single most-missed senior interview point. Deep dive covers it fully.

Intern takeaway: Actors prevent *data races on the actor’s isolated state*. They do **not** freeze time across `await` points inside actor methods.

2.5 @MainActor = “this belongs on the UI thread”

UIKit and much of SwiftUI expect UI updates on the **main** actor/thread.

`@MainActor` marks types or functions as isolated to the main actor. Calling them from elsewhere typically requires `await` (a hop to main).

Intern takeaway: Put UI-facing ViewModel state on `@MainActor`. Do **not** put heavy CPU work on main “because it’s an actor.” Actors serialize access; `@MainActor` specifically serializes onto the UI executor.

2.6 Sendable = “safe to hand across concurrency domains”

`Sendable` means a value can safely cross isolation boundaries (e.g. into an actor, or between tasks) without creating data races.

Rough beginner rule (refined in the deep dive):

- Immutable value types whose **stored properties are all Sendable** can be Sendable.
 - Classes are harder: mutable shared classes are usually **not** Sendable unless carefully synchronized or made into actors.
 - `@unchecked Sendable` means “trust me, I synchronized this” — the compiler stops checking.
- Intern takeaway:** Sendable is the compiler’s way of asking, “If two tasks hold this, can they race?” Prefer sending **values** (copies) or isolating **references** inside actors.

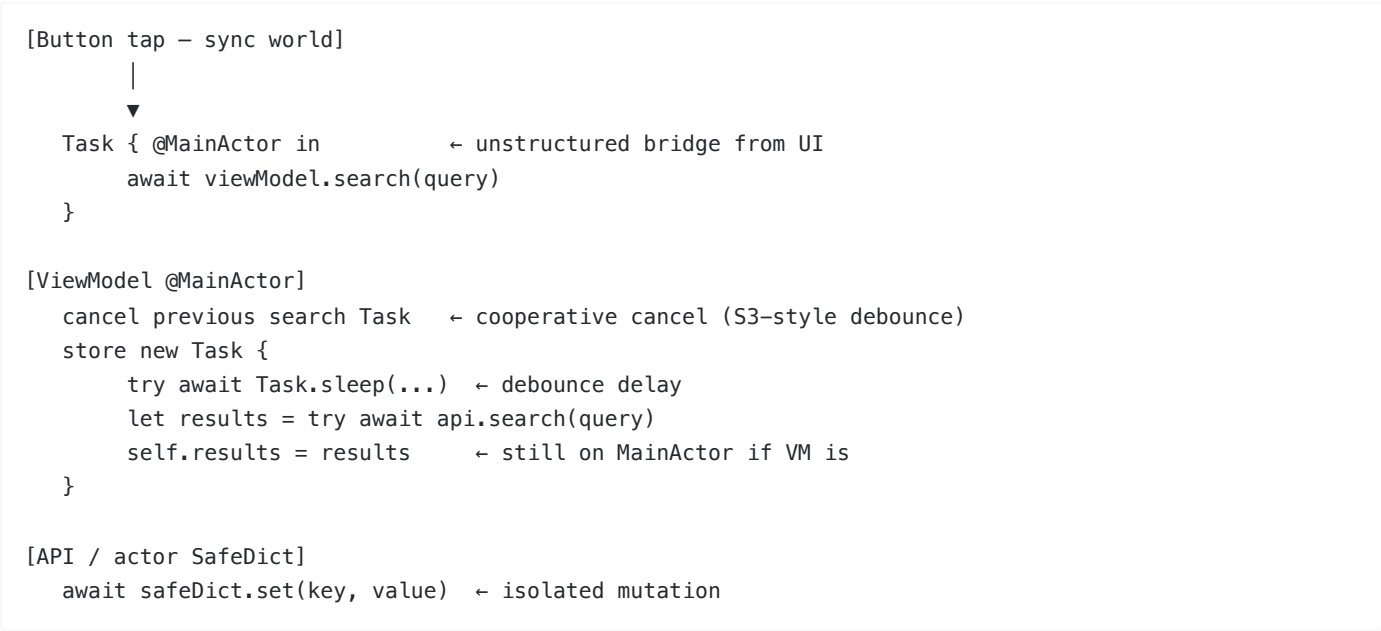
3. async/await vs GCD callbacks (first contrast)

Concern	GCD / callbacks	async/await
Control flow	Nested closures, “pyramid of doom”	Linear <code>try await</code>
Errors	Error params / Result in each callback	<code>throws / try</code>
Cancellation	Manual (<code>URLSessionTask.cancel</code> , flags)	Cooperative Task cancellation (plus API participation)
Shared state	Queues, locks, barriers	Actors + Sendable (and still GCD where legacy)
Mental model	“Which queue am I on?”	“Which isolation domain am I in?”

Both schedule work. Async/await is not automatically faster. It is usually **clearer** and **composable** with structured concurrency.

Day 04 serial-queue dictionaries remain valid. Day 05 asks: for **new** shared mutable maps, would an `actor` be a cleaner API? (How I would apply it · S2-A1.)

4. A tiny end-to-end picture



You do not need to memorize this diagram. You need to **narrate** it: UI entry → Task bridge → cancel previous work → await network → hop back to main for UI state → isolate shared caches in actors if needed.

5. Glossary (study until these feel boring)

Term	Definition
Suspension	Async function pauses at <code>await</code> ; may resume later without blocking a thread the whole time

Executor	Runtime component that runs jobs (e.g. main actor executor, actor's serial executor)
Isolation	Compiler/runtime rule: which code may touch which mutable state
Actor	Reference type with isolated mutable state; serialized access
Reentrancy	After <code>await</code> inside an actor method, other work may have run on that actor; state may have changed
Structured concurrency	Parent/child task hierarchy with scoped lifetimes
Unstructured Task	<code>Task { }</code> / detached work you must manage yourself
Cooperative cancellation	Cancelled tasks keep running until they check cancellation or hit a cancellable <code>await</code>
Sendable	Type safe to share across concurrency domains
@MainActor	Isolation to the main actor (UI affinity)
Approachable Concurrency	Optional/compiler-settings direction that can change default isolation behavior — treat as settings , not always-on language law
Data race	Concurrent unsynchronized read/write (or write/write) of shared mutable state — undefined/bad behavior
Race condition	Broader logic bug from ordering; may exist even without a data race

6. What “good” sounds like in an interview (preview)

30s definition:

“`async/await` lets a function suspend at `await` points instead of nesting callbacks. Structured concurrency keeps child tasks under a parent so cancellation propagates. Actors isolate shared mutable state; after an `await` inside an actor, I re-check state because of reentrancy.”

90s production bridge:

“At BookMyShow we fixed shared-dictionary races with GCD serial queues and a closed API — Verified S2. For greenfield modules I’d expose the same `get/set` surface on a Swift actor — S2-A1. For search, debounce isn’t just a timer; you cancel the previous `Task` so stale responses can’t win — S3.”

Full spoken answers are in [04-questions.md](#).

7. Self-check before deep dive

Answer these aloud in one sentence each:

1. Does `await` always mean “jump to a background thread”?
2. What happens to child work when a structured parent is cancelled?
3. Can another call interleave on an actor while one method is suspended at `await` ?
4. Are all structs automatically `Sendable` ?
5. Is “Swift 6 defaults everything to `MainActor`” a safe universal claim?

Expected answers (short):

1. No — it means a suspension point; resume executor depends on context.
2. Cancellation propagates; children should stop cooperatively.
3. Yes — that is reentrancy; state may change.
4. No — only if all stored properties are `Sendable` (and other `Sendable` rules apply).

5. No — default `MainActor` isolation / Approachable Concurrency are **settings when enabled**.

If any answer felt fuzzy, keep that question in mind while reading [02-deep-dive.md](#).

Day 05 — Deep Dive: Mechanics, Reentrancy, Sendable, Settings

Senior-depth, still self-contained. Assumes [01-foundations.md](#).

1. Suspension, resume, and “which thread?”

1.1 What `async` actually changes

An `async` function may contain **suspension points**. At each `await`, the function can:

1. Suspend (yield).
2. Allow the runtime to schedule other work.
3. Resume later with the awaited result (or throw).

Important clarifications:

- **Not every `await` suspends.** If the callee can complete synchronously (already satisfied, cached, same actor already available), resume may be immediate. Still treat `await` as a **logical** boundary: state you care about may be stale after it.
- **`await` does not mean “background thread.”** It means “possible suspension.” Where you resume depends on actor isolation and the callee’s executor.
- **Threads ≠ tasks.** Many tasks can multiplex onto a thread pool. Blocking inside async code (e.g. heavy `sync` GCD, long locks) can starve the pool.

1.2 Writing async APIs

```
func loadShowtimes(for venueID: String) async throws -> [Showtime] {
    let url = makeURL(venueID: venueID)
    let (data, response) = try await URLSession.shared.data(from: url)
    try validate(response)
    return try JSONDecoder().decode([Showtime].self, from: data)
}
```

Linear control flow. Errors use `throws`. Callers write `try await loadShowtimes(for:)`.

Compared to callback style, you gain readability and composition. You still must design **cancellation**, **timeouts**, and **main-actor hops** explicitly for UI.

2. Structured concurrency in depth

2.1 Why structure matters

Unstructured fire-and-forget tasks are easy to start and hard to reason about:

- Who cancels them when the user leaves the screen?
- If one of three fetches fails, do the others keep running?
- If results arrive out of order, which one wins?

Structured APIs answer: **the scope that started the work owns it**.

2.2 `async let` — fixed parallel children

```
func loadVenuePage(id: String) async throws -> VenuePage {
    async let details = fetchDetails(id)
```

```

    async let shows = fetchShowtimes(id)
    async let offers = fetchOffers(id)

    return try await VenuePage(
        details: details,
        showtimes: shows,
        offers: offers
    )
}

```

- Starts children concurrently.
- The enclosing function awaits them (explicitly or at scope exit).
- If the parent is cancelled, children are cancelled.
- Great for a **known small fan-out**.

2.3 Task groups — dynamic fan-out

```

func prefetchPosters(_ urls: [URL]) async -> [URL: Data] {
    await withTaskGroup(of: (URL, Data?).self) { group in
        for url in urls {
            group.addTask {
                (url, try? await download(url))
            }
        }
        var out: [URL: Data] = [:]
        for await (url, data) in group {
            if let data { out[url] = data }
        }
        return out
    }
}

```

Use when **N is dynamic**. Prefer bounded concurrency patterns if N can be huge (don't stampede the network). Semaphores are one approach; careful TaskGroup design and chunking is another — interviewers care that you **name the risk**.

2.4 Unstructured Task and Task.detached

```

@MainActor
final class SearchViewModel: ObservableObject {
    @Published var results: [Hit] = []
    private var searchTask: Task<Void, Never>?

    func queryChanged(_ text: String) {
        searchTask?.cancel()
        searchTask = Task {
            do {
                try await Task.sleep(for: .milliseconds(300))
                try Task.checkCancellation()
                let hits = try await api.search(text)
                self.results = hits
            } catch is CancellationError {
                // expected under debounce
            } catch {
                // map to UI error state
            }
        }
    }
}

```

```
}  
}
```

Notes:

- `Task { }` from a `@MainActor` context commonly inherits `MainActor` isolation for the closure body (**behavior can interact with language mode / settings** — see §7). Prefer being explicit about where UI mutations happen.
- Store the `Task` handle so you can **cancel**.
- `Task.detached` does **not** inherit actor isolation the same way; use rarely, and pass `Sendable` values explicitly.

Interview line: Prefer structured concurrency inside async APIs; use unstructured `Tasks` only at sync boundaries (UI actions), and always plan cancellation.

3. Task cancellation (cooperative)

3.1 Model

Cancellation is **cooperative**, not preemptive thread killing:

1. Something calls `task.cancel()` (or a parent cancels).
2. The task is marked cancelled.
3. Work stops when code:
 - hits a cancellable suspension that throws `CancellationError`, or
 - checks `Task.isCancelled` / try `Task.checkCancellation()`, or
 - otherwise finishes early by convention.

If you run a tight CPU loop with no checks and no cancellable awaits, cancellation may appear “ignored.”

3.2 What to say about `URLSession`

- **Async** `URLSession` APIs (`data(for:)` , `bytes(for:)` , etc.) participate in Swift `Task` cancellation — cancelling the `Task` typically cancels the underlying request.
- **Callback** `dataTask` APIs need **explicit** `URLSessionTask.cancel()` plus stale-response guards if you keep that style.

Don’t overclaim that “all networking magically cancels.” State the API family.

3.3 Search debounce (S3 hook)

Debounce without cancellation is a bug:

1. User types `a` , then `av` , then `ave` .
2. Three requests fly.
3. Slow `a` response can overwrite fresh `ave` results.

Fix pattern:

- Cancel the previous `Task` on each keystroke (after scheduling a new one).
- Optionally track a generation token as belt-and-suspenders.
- Ignore `CancellationError` as success-path noise.

Provenance: *Verified · S3 · BookMyShow · search debounce, state, MVVM*

4. Actors — isolation and reentrancy

4.1 Isolation basics

```
actor Counter {  
    private var value = 0  
  
    func increment() -> Int {
```

```

        value += 1
        return value
    }
}

```

From outside:

```

let c = Counter()
let v = await c.increment()

```

The actor's serial executor ensures isolated state isn't accessed concurrently. You do not write locks for `value`.

4.2 Reentrancy — the critical senior fact

Rule: When an actor method suspends at `await`, **other tasks may enter the same actor** before the suspended method resumes. Therefore, **after `await`, actor state may have changed.**

This is intentional. Actors are reentrant at suspension points so the system doesn't deadlock waiting for the actor while the actor waits for the world.

Broken intuition

```

actor Wallet {
    private var balance: Int
    private var isRefreshing = false

    init(balance: Int) { self.balance = balance }

    func spend(_ amount: Int) async throws {
        if balance < amount {
            // BUG intuition: "I'm alone in the actor, so balance stays put
            // while I refresh."
            await refreshFromServer() // ← suspension: others can interleave
            // balance / flags may have changed here
        }
        guard balance >= amount else { throw WalletError.insufficient }
        balance -= amount
    }

    func refreshFromServer() async {
        // pretend network
    }
}

```

Across `await refreshFromServer()`, another `spend` (or another mutator) may run. Assuming `balance` is unchanged is a **logic race** even though there is no data race on the storage.

Hardening patterns

1. **Snapshot into locals** before `await` when you only need the old value.
2. **Re-validate invariants** after `await` (re-check balance, generation, flags).
3. **Avoid long awaits while holding “logical locks”** expressed only as boolean flags — flags can be wrong after reentry unless carefully designed.
4. **Perform non-isolated async work outside**, then apply a short, synchronous state update on the actor.
5. Prefer **idempotent** transitions and version tokens for refresh/single-flight designs.

```

actor Wallet {
    private var balance: Int
    private var generation = 0

    init(balance: Int) { self.balance = balance }

    func spend(_ amount: Int) async throws {
        if balance < amount {
            let gen = generation
            let remote = await fetchRemoteBalance() // non-isolated async work
            // Re-check: did someone else refresh or spend while we were away?
            if gen == generation {
                balance = remote
                generation += 1
            }
        }
        guard balance >= amount else { throw WalletError.insufficientFunds }
        balance -= amount
    }

    private func fetchRemoteBalance() async -> Int { /* network */ 0 }
}

enum WalletError: Error { case insufficientFunds }

```

Interview phrase to memorize:

“Actors prevent data races on isolated state, but they are reentrant at await points — I always re-validate state after await.”

4.3 Actor vs class + lock vs GCD serial queue

Tool	Enforcement	Reentrancy / pitfalls	Best fit
GCD serial queue	Manual API discipline	sync to same queue deadlocks; easy to expose raw storage	Legacy shared maps (S2)
Lock (NSLock, etc.)	Manual	Priority inversion, forgotten unlock, lock ordering	Tiny critical sections
actor	Compiler isolation	State may change across await	New shared mutable state (S2-A1)

Actors don’t magically solve every concurrency bug. They move the bug class from “data race on the dictionary” to “logic across suspension points” — still your job.

4.4 @MainActor vs custom actors

	@MainActor	Custom actor
Purpose	UI-affined state and UIKit/SwiftUI updates	General shared mutable state
Executor	Main actor	Actor’s own serial executor
Heavy work	Avoid — jank / watchdog risk	OK if work is appropriately chunked / offloaded
Typical types	ViewModels, UI coordinators	Caches, session stores, sync engines

Don’t make “every ViewModel an actor” as a fashion. Prefer @MainActor for UI state; use custom actors for **non-UI** shared mutability.

4.5 Deadlock myths

GCD: `queue.sync` on the **current** serial queue deadlocks.

Actors: awaiting another actor (or `MainActor`) from an actor method **suspends** and allows reentrancy — different model. You can still create **logical waits**, priority inversions via bridges, or block thread pools by calling into synchronous locking APIs from async code. Prefer async end-to-end; avoid “sync hop to main” from a context that must stay responsive.

5. Sendable — precise rules

5.1 What Sendable means

A `Sendable` type can be transferred across concurrency domains without introducing data races.

5.2 Value types are **not** automatically Sendable forever

Correct rule:

*A struct/enum is Sendable when the compiler can prove it — typically when **all stored properties are Sendable** (and nesting obeys the same rule). Generic structs are Sendable when their `Sendable`-constrained parameters are.*

```
struct Showtime: Sendable {
    let id: String
    let startsAt: Date
}

struct Box<T> {
    var value: T
}
// Box<T>: Sendable where T: Sendable (when conforming / inferred under checking)
```

Counterexample:

```
final class MutableLabel {
    var text: String
    init(_ text: String) { self.text = text }
}

struct Wrapper {
    var label: MutableLabel // class reference inside a struct
}
```

`Wrapper` is a value type, but copying the struct **shares** the class reference. Concurrent mutation of `label.text` is a race. `Wrapper` is **not** safely Sendable just because it is a struct.

Trap to kill in interviews: “All value types are Sendable.”

Replace with: “Value types are Sendable when their stored properties are.”

5.3 Classes and `@unchecked Sendable`

Reference types need careful treatment:

- Immutable classes with `let` Sendable stored properties can sometimes be Sendable.
- Mutable classes generally need isolation (actor) or explicit synchronization.
- `@unchecked Sendable` disables compiler proof — use only when you **document and enforce** the synchronization invariant (and prefer actors for new code).

5.4 Crossing boundaries

Prefer:

- 1. Send **values** across tasks/actors.
- 2. Keep **mutable references** inside an actor.
- 3. Use `nonisolated` carefully and sparingly for truly immutable or computed pieces.

6. Mixing GCD and Swift concurrency

Large codebases (BMS-scale) will have both.

Rules of thumb:

- Don't call `DispatchQueue.main.sync` from async code that may already be on main or from thread-pool threads in ways that deadlock or block.
- Prefer `await MainActor.run { ... } / @MainActor` functions over sync main hops.
- Blocking the cooperative thread pool with long `sync` work reduces concurrency throughput.
- When wrapping legacy queue-based APIs, expose `async` facades with continuations — and ensure continuation resume-once correctness.

Migration is usually **strangler-fig**: leave stable GCD modules alone; adopt actors/ `async` at new boundaries (S2-A1 mindset).

7. Swift 6.2 / Approachable Concurrency / default MainActor — settings, not universal law

Apple has been evolving **default actor isolation** and **Approachable Concurrency** features so teams can adopt stricter checking with less boilerplate.

Hard correctness rule for this handbook:

*Treat Swift 6 language mode, Approachable Concurrency, and “default MainActor isolation” as **project/toolchain settings**. Say “**when enabled**” or “**under Swift 6 checking**”. Do **not** claim they are how every Swift program behaves worldwide.*

What you can say safely in interviews:

- 1. Under **complete concurrency checking** / Swift 6 mode (**when enabled**), crossing isolation without Sendable/isolation annotations becomes errors instead of warnings.
- 2. Some Xcode / Swift settings can default UI-facing code toward MainActor isolation (**when enabled**) — know that your project's build settings matter.
- 3. Regardless of settings, the **concepts** remain: isolation domains, Sendable, actor reentrancy, structured cancellation.

If asked “Does Swift 6 put everything on MainActor?”:

“Not universally. Default isolation behavior depends on language mode and concurrency settings. I verify the target's settings and write isolation explicitly for shared mutable state and UI boundaries.”

8. Failure modes checklist (production thinking)

Failure	Symptom	Mitigation
Fire-and-forget Task	Work after VC gone; stale UI updates	Store + cancel; <code>[weak self]</code> / MainActor VM patterns
Debounce without cancel	Out-of-order search results	Cancel previous Task (S3)
Reentrancy assumption	Corrupt balances/flags after await	Re-validate; short critical updates
<code>@unchecked Sendable</code> lie	Intermittent crashes under load	Prefer actor; prove sync
Heavy work on <code>@MainActor</code>	Scroll jank, hangs	Offload CPU; hop back for UI
Blocking GCD sync in async	Pool starvation, ANRs-like hangs	Async facades

9. Decision rules (speak these)

1. **New shared mutable state** → prefer `actor` (or `@MainActor` if UI-only).
2. **Legacy stable GCD isolation that works** → don't rewrite for fashion; wrap at edges.
3. **UI state** → `@MainActor` ; heavy work elsewhere.
4. **Parallel known children** → `async let` ; **dynamic N** → `TaskGroup` (bound if needed).
5. **Any user-driven repeated request** → cancel previous `Task`.
6. **After every `await` in an actor** → assume state may have changed.
7. **Sendable** → values with `Sendable` stored props; isolate mutable classes.
8. **Settings claims** → qualify Swift 6 / Approachable Concurrency / default `MainActor` with "when enabled."

10. Optional citations (not required to study)

- Swift Book — Concurrency
- WWDC: Meet `async/await` in Swift; Protect mutable state with Swift actors; Swift concurrency: Behind the scenes
- Apple docs: `Sendable` , `Task` , `actor`

Studying this chapter + `code/SafeDictActor.swift` is sufficient for Day 05 interview prep.

Day 05 — Production Bridge: S2, S2-A1, S3

Map concurrency concepts to verified BookMyShow work and honest applied extensions.

Registry: [../..../provenance/README.md](https://github.com/BookMyShow/provenance/README.md)

1. Verified · S2 — Synchronised dictionaries (BookMyShow)

Provenance: Verified · S2 · BookMyShow · synchronised dictionaries via GCD serial queues and RW locks

Situation / Task

Shared `async` state was hit from multiple queues. Unsynchronized dictionary mutation produced data races and intermittent crashes on that path.

Action (what you can claim)

1. Introduced **synchronised dictionaries** gated by **GCD serial queues** (and read-write locks where read-heavy).
2. Standardized a **closed access API** so call sites could not touch raw storage.
3. Validated under concurrency stress and watched Crashlytics for that failure mode.

Result (honest scope)

Eliminated concurrent-access crashes **on that shared-state path**. Pattern reused where mutable maps were shared across `async` work.

Do not say: "This alone produced 99.95% crash-free." Crash-free is an app-wide reliability culture metric (S8 adjacency). S2 owns the race fix on the dictionary path.

Lesson

Serialize mutation at the boundary. Don't sprinkle locks ad hoc across call sites.

Timed opener (≤15s)

"We had races on shared dictionaries — I'll cover the serial-queue design and trade-offs vs actors."

90–120s STAR spine (practice aloud)

"Shared maps were read and written from multiple async contexts, which caused intermittent crashes. We hid the storage behind a synchronised-dictionary API: writes scheduled on a serial queue, reads synchronized for a safe snapshot, and where reads dominated we used a reader-writer approach. Call sites never touched the raw dictionary. That removed the race class on that path. Trade-off: `sync` reads can deadlock if misused on the same queue, and under extreme read contention you revisit the locking strategy. Today, for greenfield modules, I'd consider a Swift actor with the same API surface."

Full story text: [story-bank S2](#)

2. How I would apply it · S2-A1 — Actor migration for greenfield

Provenance: How I would apply it · S2-A1 · greenfield shared maps → Swift actor

This is **design judgment**, not a claim that BMS rewrote all dictionaries to actors.

Migration pitch (interview-ready)

Goal: Keep the **same safe API ideas** (hide storage, serialize mutation), change the **implementation** to language-native isolation.

Legacy (S2)	Greenfield (S2-A1)
final class SafeDict + serial queue	actor SafeDict
sync get / async set on queue	await get / await set
Manual discipline not to expose storage	Compiler-enforced isolation
Deadlock risk with queue.sync	Reentrancy across await instead

Strangler strategy (say this if asked “how do you migrate?”)

1. **Don't big-bang** rewrite every call site in a revenue app.
2. Introduce actor SafeDict for **new modules** / new shared maps.
3. Optionally wrap the actor behind an async façade that matches domain needs.
4. Leave stable GCD dictionaries alone until a feature touch requires change.
5. Teach the team the **reentrancy** rule so “we moved to actors” doesn't create logic bugs.

API shape to defend

See [code/SafeDictActor.swift](#):

- Generic Key: Hashable & Sendable , Value: Sendable
- get / set / remove / snapshot
- Never return a mutable interior reference that callers can race

45–60s applied answer

"Production fix was GCD serial-queue dictionaries — Verified S2. For greenfield shared maps I'd expose an actor with the same get/set surface so isolation is compiler-checked. I wouldn't rewrite a stable module just for fashion; I'd strangler-migrate at boundaries and train on actor reentrancy so state is re-validated after await."

3. Verified · S3 — Search debounce and Task cancellation

Provenance: Verified · S3 · BookMyShow · backend-driven header; search debounce, state, MVVM

S3 is broader (SDUI header + search UX). For **this day**, the concurrency slice is:

- Debounced search input
- Explicit loading / empty / error state
- MVVM binding

- Cancellation so in-flight requests don't clobber newer queries

Concurrency lesson from S3

Debounce is not only `sleep(300ms)`. The senior piece is **cancel the previous Task** (and treat `CancellationError` as normal).

Interview line (≤20s)

"For search we debounced and cancelled the previous in-flight Task so slower older responses couldn't overwrite fresher results."

Tie-back to Day 05 vocabulary

Concept	S3 application
Unstructured Task	Started from UI/ViewModel boundary
Cooperative cancel	New keystroke cancels prior Task
@MainActor VM	UI state updates on main
Structured vs unstructured	Prefer cancel handles at this boundary

Full story: [story-bank S3](#)

4. What not to invent

Claim	Status
GCD synchronised dictionaries at BMS	✅ Verified · S2
Actor migration shipped org-wide at BMS	❌ Not verified — use S2-A1 "how I would apply it"
Search debounce / MVVM at BMS	✅ Verified · S3
Specific ms debounce constant from production	❌ Don't invent numbers
S2 caused app-wide 99.95% CFS alone	❌ Overclaim — CFS is S8 culture/metric

5. Ready-to-speak bridges (keep short)

Bridge A — GCD → actors (S2 → S2-A1)

"We serialized dictionary access with GCD. Actors are the language-native equivalent for new code: same boundary idea, compiler isolation, but revalidate after await."

Bridge B — Cancellation (S3)

"Unstructured Tasks need ownership. In search, I store the Task and cancel on each query change so cancellation is part of the product behavior, not an afterthought."

Bridge C — Settings humility (Swift 6)

"Under Swift 6 checking when enabled, Sendable and isolation violations become errors. Default MainActor isolation is a setting, not something I assert as universal."

6. Social feed / SD adjacent note (light)

If Week 1 social-feed HLD asks where concurrency fits:

- Image/poster prefetch → TaskGroup with a bound
- Feed pagination → cancel in-flight page on pull-to-refresh
- In-memory metadata maps → actor or GCD-safe store (S2 / S2-A1)

Keep this as architecture judgment; don't claim a specific BMS feed implementation detail not in the registry.

Day 05 — Questions (two-layer Q&A)

Study flow: cover **Full spoken answer** → speak from **Answer points** only → uncover and compare timing.
Budgets follow [answer-timing-guide.md](#).

Bank size: 12 normal + 8 tricky = **20 questions**.

Normal questions

Q1. async/await vs GCD callbacks? (45–60s)

Answer points (frame first):

- Both schedule work; async/await is suspension + linear control flow
- Errors via `throws` ; cancellation composes with Tasks
- GCD still valid for queues/legacy; not “delete Dispatch”
- One production bridge (mixed codebase / S2 still GCD)

Agenda opener: “Both schedule asynchronous work — the difference is composition and suspension versus queue callbacks...”

Full spoken answer:

“GCD gives you queues and explicit scheduling with closures. *async/await* lets a function suspend at await points so control flow stays linear, errors use `throws`, and work composes with structured concurrency and Task cancellation. GCD isn't obsolete — at BookMyShow our synchronised dictionaries were serial-queue based — but for new asynchronous APIs I prefer *async/await* for readability and cancellation, and I keep GCD where a stable queue boundary already works.”

Common wrong answer: “*async/await* replaces GCD and is always faster.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	Does await mean background thread?	No — suspension point; resume depends on isolation/executor
L2	When keep GCD?	Stable serial isolation, ObjC-heavy modules, incremental migration
L3	Mixing risks?	Blocking sync from async code; prefer async façades

Provenance: Verified · S2 (GCD path) · Learning-lab for async rewrite judgment

Q2. What is structured concurrency? (45–60s)

Answer points (frame first):

- Parent/child task tree; scoped lifetimes
- Cancellation/errors can propagate
- Prefer `async let` / TaskGroup over detached fire-and-forget
- Contrast with unstructured `Task { }`

Agenda opener: “Structured concurrency means async work forms a parent-owned tree...”

Full spoken answer:

"Structured concurrency organizes asynchronous work into a parent/child hierarchy. The parent owns child lifetimes, so cancellation and completion compose naturally. APIs like `async let` and `TaskGroup` keep fan-out inside a scope. Unstructured `Task` braces are still needed to bridge from synchronous UI code, but if everything is detached fire-and-forget you lose that ownership story — which shows up as stale search results or work after a screen disappears."

Common wrong answer: "Structured concurrency just means using `async/await` syntax."

Follow-up ladder:

Level	Ask	Point + short answer
L1	<code>async let</code> vs <code>TaskGroup</code> ?	Fixed vs dynamic child count
L2	What must you manage with <code>Task {}</code> ?	Store handle, cancel, error surfacing
L3	Detached default?	No — rare independent lifetime/priority

Provenance: Learning-lab · concepts; product hook via S3 Task ownership

Q3. How does Task cancellation work? (60s)

Answer points (frame first):

- Cooperative, not kill-thread
- `cancel()` marks cancelled; work checks or hits cancellable `await`
- `Task.checkCancellation()` / `isCancelled`
- Search debounce needs cancel (S3)

Agenda opener: "Cancellation in Swift is cooperative..."

Full spoken answer:

"Calling `cancel` marks a `Task` cancelled; it doesn't forcibly kill a thread. The task stops when it hits a cancellable suspension that throws `CancellationError`, or when code checks `Task.isCancelled` or `checkCancellation`. Tight CPU loops that never check can ignore `cancel`. In search debounce, I store the `Task` and cancel the previous one on each keystroke so older in-flight work doesn't overwrite newer results — that's the BookMyShow search UX pattern."

Common wrong answer: "Cancel instantly stops all code like killing a thread."

Follow-up ladder:

Level	Ask	Point + short answer
L1	<code>URLSession</code> <code>async cancel</code> ?	Async APIs participate with <code>Task cancel</code> ; callback tasks need explicit <code>cancel</code>
L2	Swallow <code>CancellationError</code> ?	Usually yes at debounce boundaries; don't present as app failure
L3	Parent cancel children?	In structured concurrency, cancellation propagates down

Provenance: Verified · S3 · search debounce

Q4. Actor vs class + lock / GCD serial queue? (60–90s)

Answer points (frame first):

- Actor: compiler-enforced isolation
- GCD serial: proven S2 pattern; manual API discipline
- Locks: small sections; ordering/priority risks
- Migration judgment: S2 verified, S2-A1 applied for greenfield

Agenda opener: "All three serialize access — the difference is enforcement and failure modes..."

Full spoken answer:

"A GCD serial queue can protect a dictionary if every access goes through it — that's what we shipped for synchronised dictionaries at BookMyShow. Locks work for tiny critical sections but are easy to misuse under complexity. An actor gives compiler-checked isolation and an async API from the outside. For greenfield shared maps I'd prefer an actor with the same safe get/set surface; for a stable GCD module I wouldn't big-bang rewrite. The actor trade-off is reentrancy: after await, state may change, so I re-validate."

Common wrong answer: "Actors are just locks; no new bugs possible."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Why hide storage?	Call sites racing raw dict defeat the wrapper (S2 lesson)
L2	RW locks when?	Read-heavy maps; more complexity than serial
L3	Migrate how?	Strangler: new modules actors, leave stable queues (S2-A1)

Provenance: Verified · S2 · How I would apply it · S2-A1

Q5. What is actor reentrancy? (60–90s)

Answer points (frame first):

- At `await` inside actor, other calls may run on same actor
- Isolated state may change across await — no data race, possible logic race
- Re-check invariants / use locals / short updates
- One concrete example (balance/refresh or token)

Agenda opener: "Actor reentrancy is the rule that suspension points allow other work on the same actor..."

Full spoken answer:

"Actors serialize execution on their isolated state, but they are reentrant at await points. If an actor method awaits, another task may enter the actor before the first resumes, so fields may have changed even though you never had a data race. Seniors don't assume continuity across await — they snapshot what they need, re-check invariants after resume, and keep mutations short. That's the main trap when people say actors make concurrency bugs impossible."

Common wrong answer: "Inside an actor method, nothing else can run until the method returns."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Why designed this way?	Avoid deadlocks while actor waits on external work
L2	Fix pattern?	Re-validate after await; version tokens; do I/O off critical assumptions
L3	Diff vs GCD sync queue?	Sync queue won't interleave that way but can deadlock on sync reentry

Provenance: Learning-lab · core concurrency correctness

Q6. What is `@MainActor` for? (30–45s)

Answer points (frame first):

- Isolate UI-affined work/state to main
- Hop with await from other contexts
- Don't put heavy CPU on main
- Contrast with custom actor for non-UI stores

Agenda opener: "MainActor is isolation for UI-affined state..."

Full spoken answer:

"@MainActor marks code that must run on the main actor — where UIKit and much of SwiftUI expect updates. ViewModels that own UI state often sit on MainActor so published updates are safe. I still offload heavy decoding or image work elsewhere and await back. A custom actor is better for non-UI shared mutable caches."

Common wrong answer: "Put all app logic on MainActor because it's safer."

Follow-up ladder:

Level	Ask	Point + short answer
L1	How hop to main?	await MainActor.run or call @MainActor method
L2	Sync DispatchQueue.main.sync?	Avoid from async — deadlock/blocking risk
L3	Default MainActor isolation?	Settings when enabled — not universal (see Q12/T8)

Provenance: Learning-lab · UI isolation practice

Q7. Sendable in one clear explanation? (45–60s)

Answer points (frame first):

- Safe to share across concurrency domains
- Value types Sendable **only if** stored props are Sendable
- Classes need care; @unchecked is a smell unless proven
- Prefer values across boundaries; isolate mutable refs

Agenda opener: "Sendable means a type can cross isolation domains without data races..."

Full spoken answer:

"Sendable is the compiler's contract that a value can be shared across concurrency domains safely. Structs aren't magically Sendable — they are when all stored properties are Sendable. A struct holding a mutable class reference can still race. Classes usually need immutability, synchronization, or an actor. @unchecked Sendable opts out of checking; I only use it when I've proven the invariant, and for new shared mutability I prefer an actor."

Common wrong answer: "All value types are Sendable; all classes aren't."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Generic struct?	Sendable when parameters are constrained/proven Sendable
L2	Why Swift 6 cares?	When enabled, crossings become errors under checking
L3	UIKit types?	Often not Sendable — hop to MainActor instead of sending freely

Provenance: Learning-lab · correctness emphasis

Q8. async let vs withTaskGroup ? (45s)

Answer points (frame first):

- async let: fixed small parallel set
- TaskGroup: dynamic N
- Both structured; cancel with parent
- Bound fan-out when N large

Agenda opener: "Both run children concurrently — the difference is fixed versus dynamic fan-out..."

Full spoken answer:

"async let is ideal when I know a small fixed set of parallel children — details, showtimes, offers. TaskGroup is for dynamic N, like prefetching N poster URLs. Both stay structured under the parent. If N can be huge I talk about bounding concurrency so we don't stampede the network."

Common wrong answer: "Always use TaskGroup; async let is legacy."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Error handling?	Throwing group vs try await each async let
L2	Unordered results?	Iterate group as children complete
L3	vs DispatchGroup?	TaskGroup integrates cancellation/async; GCD group is callback-era

Provenance: Learning-lab

Q9. When is `Task.detached` appropriate? (45s)

Answer points (frame first):

- Rare
- Independent priority/lifetime; doesn't inherit actor the same way
- Must pass Sendable state carefully
- Default should be structured or inheriting Task

Agenda opener: "Detached tasks are a sharp tool, not a default..."

Full spoken answer:

"I use Task.detached rarely — when work must be independent of the caller's actor isolation and lifetime, and I'm prepared to pass Sendable inputs explicitly. For UI-driven work I prefer Task that inherits context, with an explicit cancel path. Detached everywhere loses structure and makes MainActor hops easy to get wrong."

Common wrong answer: "Detached is how you always do background work."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Priority?	Can set priority; still don't starve UI carelessly
L2	Inherit task-locals?	Detached doesn't inherit the same way — know implications
L3	Alternative?	nonisolated async functions + actor/MainActor hops

Provenance: Learning-lab

Q10. How do you replace callback URLSession with async? (45–60s)

Answer points (frame first):

- `try await session.data(for:) / bytes APIs`
- throws for failures
- Cancellation via Task with async APIs
- Still validate HTTP status / decode errors

Agenda opener: "Modern URLSession exposes async methods that suspend until the transfer finishes..."

Full spoken answer:

"I'd use URLSession's async data APIs with try await, validate the HTTP response, then decode. Errors surface through throws instead of optional error parameters. If this runs inside a Task, cancelling that Task participates in cancelling the async transfer. I still keep stale-response guards for higher-level flows like search where multiple generations exist."

Common wrong answer: "Async URLSession means I never think about cancellation again."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Callback dataTask still?	Need explicit URLSessionTask.cancel + generation tokens
L2	Progress?	bytes streams / delegates depending on need
L3	Auth challenges?	Separate topic — don't hand-wave; Week 2 networking day

Provenance: Learning-lab · (S4 is URLSession migration but pinning day owns depth)

Q11. Task priority / QoS — what do you say? (45s)

Answer points (frame first):

- Tasks have priorities; often inherit from parent
- Don't run huge low-level work at user-interactive without need
- Avoid starving UI
- Bridge mindset from GCD QoS, not identical API

Agenda opener: "Priority influences scheduling — I inherit where possible and avoid elevating bulk work..."

Full spoken answer:

"Tasks carry priority and often inherit from their parent, similar in spirit to caring about GCD QoS. Interactive UI work should stay responsive; bulk prefetch shouldn't casually run at the highest priority. I set priority deliberately for true background work and avoid assuming the runtime will save me from doing heavy decoding on MainActor."

Common wrong answer: "Priority doesn't matter in async/await."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Inherit always?	Common case yes; detached/explicit overrides exist
L2	Priority inversion?	Possible with locks/bridges — prefer async isolation
L3	Energy?	Lower priority for deferrable work on mobile

Provenance: Learning-lab

Q12. Swift 6 / Approachable Concurrency / default MainActor — how do you phrase it? (45–60s)

Answer points (frame first):

- These are **settings / language modes when enabled**
- Not universal behavior of all Swift apps
- Concepts (isolation, Sendable) still matter regardless
- Verify target settings in real projects

Agenda opener: "I treat Swift 6 concurrency hardening as settings-dependent..."

Full spoken answer:

"Under Swift 6 language mode and stricter concurrency checking when enabled, isolation and Sendable mistakes become harder to ignore. Approachable Concurrency and default MainActor isolation are also settings that can change defaults for a target — I don't claim they apply to every codebase worldwide. In interviews I speak in concepts: isolation domains, Sendable, actors, cancellation — and I qualify defaults with 'when enabled' after checking build settings."

Common wrong answer: "In Swift 6 everything is MainActor automatically everywhere."

Follow-up ladder:

Level	Ask	Point + short answer
L1	What still true without those settings?	Reentrancy, cancel, actor isolation concepts
L2	Migration tip?	Turn checking on module-by-module; fix boundaries
L3	UIKit legacy?	Expect annotations/hops; don't force-fit Sendable

Provenance: Learning-lab · toolchain humility

Tricky questions

T1. "There's an await inside my actor method — can I still race?" (90–120s)

Answer points (frame first):

- No data race on isolated storage from outside concurrently
- Yes logic race: state may change across await (reentrancy)
- Example + mitigation
- Don't claim actors eliminate all concurrency bugs

Agenda opener: "There's a subtle distinction between data races and logic across suspension..."

Full spoken answer:

"You won't get a classic data race on the actor's isolated properties from two tasks mutating them unsynchronized — the actor serializes that. But if your method awaits, another task can run on the actor first, so the assumptions you made before the await can be false afterward. That's reentrancy. I treat every await inside an actor as a point where I must re-read flags, balances, generations, or whatever invariant I depend on. Actors move bugs from memory corruption toward logical races — still real bugs."

Common wrong answer / trap: "Actor means no races ever inside a method."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Show a fix	Version token; re-check after await; minimize awaited critical sections
L2	Compare GCD sync critical section	Sync section won't interleave but can deadlock
L3	Single-flight refresh?	Store in-flight Task/continuation on actor; await same flight; careful resume-once

Provenance: Learning-lab · must-nail Day 05

T2. @unchecked Sendable on a class with mutable state (90s)

Answer points (frame first):

- Silences compiler; doesn't create safety
- Only if manually synchronized and proven

- Prefer actor for new code
- Interview smell if used casually

Agenda opener: "Unchecked Sendable is an assertion of safety, not a grant of safety..."

Full spoken answer:

"If I mark a mutable class @unchecked Sendable, I'm telling the compiler not to prove thread safety. That's only honest if I've synchronized every access — for example with an internal lock — and I can explain the invariant. Otherwise I've just hidden races. For greenfield shared mutability I'd use an actor instead of teaching the team to sprinkle unchecked. Unchecked belongs in narrow interop/legacy cases with documentation, not as a default escape hatch."

Common wrong answer / trap: "Add @unchecked Sendable to silence Swift 6."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Immutable class?	let Sendable stored props can be Sendable without unchecked
L2	Struct with class prop?	Still not safe — shared reference
L3	Module boundaries?	Public APIs need careful annotation; send DTOs

Provenance: Learning-lab

T3. Task in ViewModel without cancellation (search) (90–120s)

Answer points (frame first):

- Stale responses / out-of-order UI
- Store Task; cancel on new query / deinit / onDisappear
- Debounce sleep + checkCancellation
- S3 verified hook

Agenda opener: "The bug isn't starting a Task — it's abandoning ownership..."

Full spoken answer:

"If every keystroke starts a Task and none are cancelled, responses can complete out of order and older results win. In a Search ViewModel I keep a searchTask property, cancel it when the query changes, wait a debounce interval with a cancellable sleep, check cancellation, then fetch and publish on MainActor. CancellationError is expected. That's aligned with BookMyShow search debounce work — cancellation is part of correct UX, not an optimization."

Common wrong answer / trap: "Fire-and-forget is fine if the network layer is fast."

Follow-up ladder:

Level	Ask	Point + short answer
L1	weak self?	Avoid retaining cycles; MainActor VM still should cancel
L2	Generation token too?	Yes as belt-and-suspenders with cancel
L3	Structured alternative?	Harder at sync UI boundary; unstructured + cancel is normal

Provenance: Verified · S3

T4. Calling MainActor from an actor — will you deadlock like GCD? (90s)

Answer points (frame first):

- Different model from queue.sync deadlock

- await MainActor suspends; actor can reenter
- Still avoid sync main bridges / blocking
- Logical wait / priority issues possible

Agenda opener: “People map GCD deadlock instincts onto actors incorrectly...”

Full spoken answer:

“With GCD, syncing onto the queue you’re already on deadlocks. Awaiting MainActor from a custom actor doesn’t work like that — your actor method suspends, which allows reentrancy, and MainActor work can proceed. You can still create poor designs: blocking with DispatchQueue.main.sync from thread-pool async code, or long circular waits that hurt responsiveness. I prefer async hops end-to-end and keep actor mutations short after returning from MainActor work.”

Common wrong answer / trap: “Never call MainActor from an actor or you’ll deadlock always.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	When hop needed?	Publishing UI state owned on MainActor
L2	Reverse hop?	MainActor awaiting custom actor for cache lookup — fine if not blocking
L3	Reentrancy angle?	While awaiting MainActor, other tasks may enter your actor

Provenance: Learning-lab

T5. GCD sync inside async contexts (90s)

Answer points (frame first):

- Can block cooperative pool threads
- Deadlock risk with main / same serial queue
- Prefer async primitives / continuations
- Mixed codebases need façades

Agenda opener: “The danger is blocking a thread the runtime expected to stay cooperative...”

Full spoken answer:

“Swift concurrency multiplexes many tasks onto a thread pool. If async code calls DispatchQueue.sync and blocks, you can starve that pool and create hangs. Sync to main is especially sharp. In mixed codebases I wrap legacy queue APIs with async façades using continuations, resume exactly once, and avoid sync bridges from async paths. At BMS we still have GCD dictionary isolation — the lesson isn’t ‘sync everywhere,’ it’s ‘serialize at the boundary without blocking the world.’”

Common wrong answer / trap: “Feel free to mix sync GCD and await freely.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	Continuation pitfalls?	Resume twice / never resume — hang or crash
L2	When sync OK?	Tiny known contexts; not as async architecture spine
L3	Actor instead?	Greenfield shared state — S2-A1

Provenance: Verified · S2 context · Learning-lab mixing guidance

T6. Sendable across modules with public classes (90–120s)

Answer points (frame first):

- Public classes may not be Sendable
- Don't force unchecked to push them through
- Send value DTOs; isolate references in actors
- Annotate APIs intentionally under checking when enabled

Agenda opener: "Module boundaries make Sendable a design problem, not a checkbox..."

Full spoken answer:

"If a public class from another module isn't Sendable, I shouldn't paper over it with unchecked just to call an actor. Prefer converting to Sendable value DTOs at the boundary, or keep the reference inside an isolated domain and only expose async methods that return values. Under Swift 6 checking when enabled, these crossings surface as errors — which is good pressure to design boundaries. UIKit types are a classic example: hop to MainActor rather than pretending UIView is a free Sendable token."

Common wrong answer / trap: "Ignore library types; only my models need Sendable."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Generic APIs?	Constrain to Sendable where crossing tasks
L2	@MainActor class public?	Callers must await; document isolation
L3	Binary frameworks?	May lack annotations — isolate usage

Provenance: Learning-lab

T7. Migrate SafeDict GCD → actor without big-bang (90–120s)

Answer points (frame first):

- S2 verified current; S2-A1 applied future
- Strangler: new maps on actor; stable leave alone
- Same API philosophy: hide storage, Sendable values
- Teach reentrancy during migration

Agenda opener: "I'd migrate by strangling boundaries, not boiling the ocean..."

Full spoken answer:

"Production synchronised dictionaries at BookMyShow used GCD serial queues and a closed API — that fixed races on that path. For greenfield modules I'd introduce an actor SafeDict with get/set/snapshot and Sendable values, matching the safety philosophy. I wouldn't rewrite every call site in one PR in a large consumer app. Feature teams adopt the actor at new touch points; legacy queues remain until modified. Training matters: engineers must learn that await inside the actor requires re-validation, or we trade data races for logic races."

Common wrong answer / trap: "Actors are better, so rewrite the whole app's dictionaries next sprint."

Follow-up ladder:

Level	Ask	Point + short answer
L1	API async impact?	Call sites become async — plan propagation
L2	Testing?	Concurrency tests; stress under parallel tasks
L3	Metrics?	Watch crash signatures for that path — don't claim CFS ownership

Provenance: Verified · S2 · How I would apply it · S2-A1

T8. "Swift 6.2 Approachable Concurrency means I can stop thinking about MainActor." (90s)

Answer points (frame first):

- Reject absolute claim
- Settings when enabled may ease defaults — still understand isolation
- Shared mutable non-UI state still needs actors/Sendable thought
- Humility about toolchain

Agenda opener: "Approachable Concurrency doesn't delete isolation as a concept..."

Full spoken answer:

"I wouldn't stop thinking about MainActor. Approachable Concurrency and related defaults can change how much boilerplate a target needs when those settings are enabled, but they don't remove the need to know where UI state lives, where shared mutable caches live, and how Sendable crossings work. I verify the project's concurrency settings, write isolation explicitly at important boundaries, and still teach reentrancy and cancellation. Tooling can help adoption; it doesn't replace concurrent design."

Common wrong answer / trap: Treat Approachable Concurrency as universal automatic safety.

Follow-up ladder:

Level	Ask	Point + short answer
L1	What do you check in Xcode?	Language mode / concurrency checking / default isolation settings for the target
L2	Cross-module?	Settings may differ; don't assume
L3	Interview phrasing?	Always say 'when enabled' for defaults

Provenance: Learning-lab · settings correctness

Timed set suggestions

Drill	Questions	Focus
Core 10 min	Q4, Q5, T1, T3	Actor + cancel
Migration 8 min	Q1, T7, Q12	S2 → S2-A1 + settings humility
Sendable 8 min	Q7, T2, T6	Exact Sendable rules

Day 05 — Exercises

Solutions are in-repo. Do not leave Cursor to "look up" the drills.

A. Conceptual drills (write answers in 5–8 bullets each)

A1. Reentrancy storyboard

Draw or list steps for:

1. Task A enters actor Wallet.spend
2. Sees insufficient balance, await refresh()
3. Task B enters spend or applyRemoteBalance before A resumes
4. Task A resumes

Write: What can go wrong if A assumes pre-await balance? What local variables / generation checks fix it?

Solution points:

- After await, balance/flags may differ — A must re-read.
- Snapshotting the old balance into a `let` doesn't update the actor field — good for comparison, not for assuming the field is unchanged.
- Generation / version token: only apply remote if generation matches; bump on successful apply.
- Prefer: fetch remote as value, then short synchronous apply on actor after re-check.

A2. Sendable true/false

Mark true/false and fix false ones:

1. "Every struct is Sendable."
2. "A struct whose only stored property is `let name: String` can be Sendable."
3. "A struct storing `var cache: NSMutableDictionary` is Sendable because it's a value type."
4. " `@unchecked Sendable` proves thread safety."
5. "Default MainActor isolation is always on in all Swift 6.2 apps."

Solution:

1. False — stored props must be Sendable.
2. True (String is Sendable).
3. False — shared mutable reference inside struct.
4. False — disables checking; you must prove safety.
5. False — settings **when enabled**, not universal.

A3. Cancellation checklist for search

List 6 concrete code/actions for debounce search correctness.

Solution sketch:

1. Store `searchTask`
2. Cancel previous on each query change
3. Cancellable debounce sleep
4. `try Task.checkCancellation()` before/after network
5. Ignore `CancellationError` for UX
6. Publish results on `@MainActor` ; optional generation token

B. Coding drills

B1. Implement / extend `SafeDictActor`

Open [code/SafeDictActor.swift](#).

Tasks:

1. Explain aloud why `Key` and `Value` require `Sendable` .
2. Add `func merge(_ other: [Key: Value])` that replaces keys from `other` .
3. Add a **broken** commented method that `await` s a simulated network fetch mid-update and documents the reentrancy hazard in comments.
4. Write a brief comment contrasting this API with Day 04 GCD `sync` `get` / `async` `set`.

Acceptance: You can narrate isolation, Sendable bounds, and reentrancy without reading notes.

B2. Debounced search Task (Learning-lab)

In a Playground or scratch file, write a `@MainActor` class:

```
@MainActor
final class SearchLab {
    private var task: Task<Void, Never>?
```

```

private(set) var latest: String = ""

func onQuery(_ q: String) {
    // cancel + debounce + assign latest
}
}

```

Simulate slow searches with `Task.sleep`. Prove that without cancel, older finishes can overwrite newer; with cancel, they cannot.

Solution idea: Use an actor or MainActor-isolated counter of completions; print query id on finish; show overwrite vs cancel.

B3. async let vs TaskGroup

Implement two functions that fetch three fake resources:

- `loadFixed()` using `async let`
- `loadDynamic(_ ids: [String])` using `withThrowingTaskGroup`

Print ordering differences. Cancel the parent early and observe child cancellation.

C. Debugging drills

C1. Symptom: intermittent wrong search results

Given: Debounce sleep exists; Tasks are not stored.

Find: Root cause + fix in two sentences.

Solution: Multiple in-flight Tasks; cancel/store Task (S3 pattern).

C2. Symptom: “Impossible” actor invariant break

Given: Actor method sets `isRefreshing = true`, awaits network, sets result, clears flag. Sometimes two refreshes interleave oddly.

Find: Name the concurrency concept + one fix.

Solution: Reentrancy across await; single-flight in-flight task/continuation or re-check flags/generation after await.

C3. Symptom: Swift 6 errors on crossing a class into an actor

Find: What to change architecturally (not just `@unchecked`).

Solution: Send Sendable DTO values; keep class inside MainActor/actor isolation; avoid unchecked unless proven.

D. Speaking drills (record)

#	Prompt	Budget	Provenance
D1	async/await vs GCD	45–60s	S2 bridge
D2	Actor reentrancy	60–90s	Learning-lab
D3	Migrate sync dict → actor	90s	S2 → S2-A1
D4	Search cancellation	60–90s	S3
D5	Sendable + value types caveat	45–60s	Learning-lab
D6	Swift 6 defaults humility	45s	Settings when enabled

Score each on: agenda opener, mechanism, trade-off, provenance honesty, timing.

E. Flash recall (cover / uncover)

Front	Back
Structured concurrency	Parent owns children; cancel propagates · Trap: detached everywhere · Prod: search Task ownership
Actor	Isolated mutable state · Trap: ignore reentrancy · Prod: S2-A1 greenfield
Reentrancy	State may change after await · Trap: assume continuity · Prod: refresh/single-flight
Sendable	Cross-domain safe · Trap: "all structs" · Need: all stored props Sendable
@MainActor	UI isolation · Trap: heavy work on main · Prod: VM state
Task cancel	Cooperative · Trap: kill thread · Prod: S3 debounce
async let	Fixed parallel · Trap: unbounded N · Prod: dual/triple fetch
TaskGroup	Dynamic parallel · Trap: forget await children · Prod: prefetch
Detached	Independent lifetime · Trap: default choice · Prod: rare
S2	GCD sync dictionaries verified · Trap: claim app-wide CFS · Path-scoped races
S2-A1	Actor for greenfield · Trap: big-bang rewrite · Strangler
Settings	Swift 6 / Approachable / default MainActor · Trap: universal claim · Say when enabled

F. Day close-out checklist

- ☐ Explained reentrancy without notes
 - ☐ Stated Sendable stored-property rule correctly
 - ☐ Delivered S2 STAR ≤ 3 min with honest metrics scope
 - ☐ Delivered S2-A1 migration as **applied**, not shipped rewrite
 - ☐ Tied debounce cancellation to S3
 - ☐ Qualified Swift 6 / Approachable Concurrency / default MainActor as settings when enabled
 - ☐ Read `SafeDictActor.swift` aloud line-by-line once
 - ☐ Completed timed set from `04-questions.md`
-

Day 06 — DSA: Arrays, Strings, Two Pointers, Sliding Window

Week 1 · Full study (self-contained) · ~5–6 hrs

Revision twin: [revision/weeks/week-01/day-06.md](#)

Track index (optional): [../../coding/dsa-track.md](#) — patterns are **embedded below**; you do not need to leave this chapter.

Outcomes

By end of day, without notes, you can:

- Open every problem with a **60–90s approach script** (clarify → brute → optimize → edges)
- Identify two-pointer vs sliding-window vs prefix/hash patterns from problem signals
- Solve the Week 1 core set in Swift while narrating time/space
- Handle Swift String indexing honestly (`[Character]` when random access needed)
- Catch up one weak Week 1 topic (30–45 min) without opening new rabbit holes

How to study

1. [01-foundations.md](#) — pattern signals + Swift tips + say-this-first
2. [02-deep-dive.md](#) — worked approaches for each core problem
3. [03-production-bridge.md](#) — how to talk DSA as a senior (BMS hooks light)
4. [code/](#) — full Swift solutions; cover, re-implement from approach
5. [04-questions.md](#) — two-layer pattern Q&A + approach scripts
6. [05-exercises.md](#) — timed solves + catch-up
7. Revision twin for spaced drills

Module map

Module	File
Foundations	01-foundations.md
Deep dive	02-deep-dive.md
Production bridge	03-production-bridge.md
Questions	04-questions.md
Exercises	05-exercises.md
Code	code/ Solutions.swift + per-problem files

Time budget

Block	Minutes
Foundations	45–60
Deep dive (approaches)	60–75
Code: re-solve 6+ cold	150–180
Questions / approach scripts	45–60
Catch-up weak day	45–60

Provenance

DSA rarely maps 1:1 to resume metrics. Use:

Label	Use
Learning-lab	All code/ solutions
Communication skill	Approach narration (interview signal)
Soft product metaphors	Debounce $\approx$ cancel in-flight (S3); listing windows $\approx$ pagination (SD later) — not "I used Kadane in production ads"

Critical notes (pin)

1. **Patterns are embedded in this chapter** — optional link to dsa-track is for spaced review across weeks only.
2. Speak **time and space** after every approach.
3. Swift `String` is not Int-indexed $O(1)$ — convert to `[Character]` when needed and say the cost.
4. Communication grade > clever silent solve.

Agenda openers

Topic	Opener
Any LC problem	"Constraints → brute → optimal reason → edges → code."
Two pointers	"Two indices, invariant, move the one that restores it."
Sliding window	"Expand right; shrink left when invariant breaks."
Swift String	"Not O(1) random index — I'll use [Character] if needed."
Kadane	"Running max of extend vs restart; handle all-negative."

Files in `code/`

File	Problem
TwoSum.swift	Two Sum
MaxProfit.swift	Best Time to Buy/Sell Stock
ValidPalindrome.swift	Valid Palindrome
ContainerWater.swift	Container With Most Water
LengthOfLongestSubstring.swift	Longest Substring Without Repeating
MaxSubArray.swift	Maximum Subarray (Kadane)
ProductExceptSelf.swift	Product of Array Except Self
MoveZeroes.swift	Move Zeroes
ThreeSum.swift	3Sum (stretch)
MinSubArrayLen.swift	Min Size Subarray Sum (stretch)
GroupAnagrams.swift	Group Anagrams (bridge)
Solutions.swift	Smoke asserts

01 — Foundations: Patterns, Swift Tips, Say-This-First

Intern → competent. By the end you can classify a problem before coding.

0. North star

Interviews grade process under uncertainty. Pattern ID + clear narration beats a silent clever trick.

1. Master "say this first" (60–90s) — memorize

Use on **every** problem before typing:

*"Constraints? Sorted? Duplicates? Mutate in place?
Brute force would be ... O(?).
I'll use ... because Complexity time ... space
Edge cases: empty, single element, all same / all duplicates.
Coding the optimized version now."*

Pin it. If you skip it, restart the problem in practice.

2. Pattern cheatsheet (embedded — no external read required)

Pattern	Signals in the prompt	Target complexity	Classic problems
Hash map	Need indices / complements / counts	$O(n)$ time / $O(n)$ space	Two Sum, Group Anagrams
One-pass running state	Track min/max so far	$O(n)$ / $O(1)$	Buy/Sell Stock, Kadane
Two pointers (opposite)	Sorted array, palindrome, max area from ends	$O(n)$	Valid Palindrome, Container Water, 3Sum after sort
Write pointer	In-place compact / remove	$O(n)$ / $O(1)$	Move Zeroes, remove dups
Sliding window fixed	Exactly k length	$O(n)$	Fixed max sum of k
Sliding window variable	Longest/shortest substring/subarray with constraint	$O(n)$	Longest Substring Without Repeating, Min Size Subarray Sum
Prefix / suffix products	Range aggregates; no division	$O(n)$	Product Except Self
Prefix + hash	Subarray sum counts (esp. with negatives)	$O(n)$	Subarray Sum Equals K (later)

2.1 Two pointers vs sliding window (intern distinction)

	Two pointers	Sliding window
Idea	Indices move to satisfy a relation	Maintain a contiguous range invariant
Motion	Often both ends, or fast/slow	Right expands; left shrinks
Example	Container With Most Water	Longest substring without repeat

They overlap in vocabulary — say **invariant** either way.

2.2 Fast/slow (awareness)

Cycle detection / middle of list — Week 2. Today: know the name.

3. Complexity fluency

Say	Meaning
$O(n)$ time	One or few passes
$O(n^2)$	Nested loops / each pair
$O(1)$ space	Few scalars (output array may be excluded by convention — clarify)
$O(n)$ space	Hash map / copy

Always state time and space after the approach.

4. Swift-specific tips (interview landmines)

4.1 String indexing is not array indexing

```
let s = "hello"
// s[2] // ❌ not Int subscript
let chars = Array(s) // O(n) copy; then O(1) index
let arr = Array(s.lowercased())
```

Say aloud:

"Swift String is not random-access $O(1)$. I'll convert to `[Character]` when I need repeated indexing."

4.2 Prefer `Array` indices carefully

```
for i in 0..O(n) each  $\rightarrow O(n^2)$ 
```

4.3 Integer overflow / Int

LeetCode Swift usually uses `Int`. Mention overflow only if constraints suggest it.

4.4 Sorting costs

Sorting is $O(n \log n)$. If you sort to enable two pointers (3Sum), say that cost up front.

5. Core problem set (Week 1 Day 06)

Minimum today (solve cold after reading approaches once):

#	Problem	Pattern	Code file
1	Two Sum	Hash map	TwoSum.swift
2	Best Time to Buy/Sell Stock	Running min	MaxProfit.swift
3	Valid Palindrome	Two pointers	ValidPalindrome.swift
4	Container With Most Water	Two pointers ends	ContainerWater.swift
5	Longest Substring Without Repeating	Variable window	LengthOfLongestSubstring.swift
6	Maximum Subarray (Kadane)	Running reset	MaxSubArray.swift
7	Product of Array Except Self	Prefix/suffix	ProductExceptSelf.swift
8	Move Zeroes	Write pointer	MoveZeroes.swift

If time: **3Sum, Min Size Subarray Sum, Group Anagrams** — see deep dive + `Solutions.swift`.

6. Approach templates (fill-in)

6.1 Hash map

"I need ... so sorting would lose I'll scan once storing ... in a dictionary. Brute is Edges:"

6.2 Opposite pointers

"Sorted/ends problem. Left and right; move the side that ... because $O(n)$ after any sort cost."

6.3 Variable window

"Expand right to include. While invariant broken, shrink left. Track best. Each index moves $\leq$ once $\rightarrow O(n)$."

6.4 Kadane

"Running sum; if negative, reset. Track global max. All-negative $\rightarrow$ largest element."

7. Communication > clever

Senior signal phrases:

- "I'd clarify whether we can mutate the input."
- "Brute is clear; here's why we can do better."
- "I'll dry-run on [...] before coding further."
- "If you need $O(1)$ space, we can discuss trade-offs."

Never silently rewrite for 10 minutes after a constraint change — **restate the new plan**.

8. Catch-up rule (weekend volume)

If Day 03–05 is shaky: **45–60 min on the weakest topic beats a 4th Hard LC**.

Mark the catch-up topic in exercises; DSA still gets ≥ 3 timed problems.

9. Complexity cheat sheet (say out loud)

Pattern family	Typical time	Typical extra space
Hash one-pass	$O(n)$	$O(n)$
Opposite two pointers (unsorted $\rightarrow$ sort first)	$O(n)$ after $O(n \log n)$	$O(1)$ – $O(n)$
Variable sliding window	$O(n)$	$O(\Sigma)$ alphabet map
Kadane	$O(n)$	$O(1)$
Prefix/suffix products	$O(n)$	$O(1)$ aside from output
Write pointer in-place	$O(n)$	$O(1)$

Interview habit: After naming the pattern, immediately pair it with time **and** space.

10. Decision tree (30s ID)

Need indices / complements?	$\rightarrow$ hash map
Contiguous + constraint?	$\rightarrow$ sliding window
Sorted / ends / palindrome?	$\rightarrow$ opposite pointers
In-place compact?	$\rightarrow$ write pointer
Max subarray sum?	$\rightarrow$ Kadane
Range product w/o division?	$\rightarrow$ prefix/suffix
Many range sums / count sums?	$\rightarrow$ prefix (+ hash if negatives)

If two patterns fit, pick the one matching the **output** (indices vs length vs subarray).

11. Practice cadence today

Block	Goal
First pass	Read approaches; don't memorize code
Second pass	Cover code; re-implement 3 Easy
Third pass	Timed Medium window + pointers
End	Catch-up weak Week 1 topic

12. Self-check

- ☐ Recite say-this-first without looking
- ☐ Name 4 patterns with one signal each
- ☐ Explain Swift String indexing cost
- ☐ Know which 8 problems are "minimum today"
- ☐ Walk the decision tree on a random prompt

→ [02-deep-dive.md](#)

02 — Deep Dive: Worked Approaches (then code/)

Read the approach script → hide it → re-speak → implement from [code/](#).

Full solutions live in code; this file is the **thinking** layer.

1. Two Sum

Prompt: Given `nums` and `target`, return indices `i, j` with `nums[i] + nums[j] == target`.

Say this first (60s):

"Constraints? Duplicates? Guaranteed one answer?

Brute: try all pairs $O(n^2)$.

Sorting loses original indices unless I store pairs.

I'll scan once: for each value look up `target - value` in a dictionary of value→index, else store current.

Time $O(n)$, space $O(n)$. Edges: empty, no pair, negatives, duplicates."

Invariant: Map holds values seen so far with an index.

Dry-run: `nums = [2,7,11,15]`, `target = 9` → at 7 find 2 → `[0,1]`.

Trap: Returning values instead of indices; using the same element twice.

Code: [code/TwoSum.swift](#)

2. Best Time to Buy and Sell Stock

Prompt: Max profit from one buy + one later sell; else 0.

Say this first:

"One transaction. Brute all pairs $O(n^2)$.

Track minimum price so far and max profit = price - min.

$O(n)/O(1)$. Edges: length 1, strictly decreasing → 0."

Trap: Selling before buying; resetting min after computing profit wrong order.

Code: [code/MaxProfit.swift](#)

3. Valid Palindrome

Prompt: Alphanumeric only, case-insensitive palindrome?

Say this first:

*"Two pointers from ends. Skip non-alnum; compare lowercased.
O(n)/O(1) extra if we index the string carefully — in Swift I'll use `Array(s)` for clarity O(n) space, or walk `String.Index`.
Edges: empty, all punctuation, unicode — I'll clarify charset; LeetCode is ASCII alnum."*

Trap: Forgetting to skip; comparing uppercase vs lower.

Code: [code/ValidPalindrome.swift](#)

4. Container With Most Water

Prompt: Heights at indices; max `min(h[l], h[r]) * (r - l)`.

Say this first:

*"Brute all pairs O(n²).
Start at ends — max width. Move the shorter side inward; only a taller short-side can improve.
O(n)/O(1). Edges: n=2, all equal."*

Why move shorter: Width shrinks by 1 always; height limited by min. Moving taller can't increase min; moving shorter might.

Code: [code/ContainerWater.swift](#)

5. Longest Substring Without Repeating Characters

Prompt: Length of longest substring with all unique chars.

Say this first:

*"Variable window. Expand right; if duplicate, shrink left until unique — typically with last-seen index map.
Each index moves at most once → O(n). Space O(min(n, alphabet)).
Edges: empty, all unique, all same."*

Invariant: `s[left...right]` has unique characters.

Trap: Off-by-one when updating left to `lastIndex + 1`; not taking max with current left.

Code: [code/LengthOfLongestSubstring.swift](#)

6. Maximum Subarray (Kadane)

Prompt: Contiguous subarray with largest sum.

Say this first:

*"Brute O(n²) sums. Kadane: running sum; if running < 0 reset to 0 before adding next — actually classic: `running = max(x, running+x)`; track global max.
All-negative: answer is largest element — the `max(x, running+x)` form handles it.
O(n)/O(1)."*

Trap: Resetting to 0 when all negative (wrong if you force non-empty and reset poorly).

Code: [code/MaxSubArray.swift](#)

7. Product of Array Except Self

Prompt: `out[i] = product of all except i ; no division; O(n).`

Say this first:

*"Left products pass into output; right products multiply on the way back.
O(n) time, O(1) extra if output doesn't count.
Edges: zeros (one zero, two zeros), negatives."*

Trap: Using division; O(n) extra left/right arrays without mentioning you can optimize.

Code: [code/ProductExceptSelf.swift](#)

8. Move Zeroes

Prompt: Move zeroes to end in place; keep relative order of non-zeroes.

Say this first:

*"Write pointer: write non-zeroes forward, then fill zeroes.
O(n)/O(1). Edges: no zeroes, all zeroes, already packed."*

Code: [code/MoveZeroes.swift](#)

9. 3Sum (stretch)

Say this first:

*"Sort O(n log n). Fix i; two-sum on remainder with left/right; skip duplicates.
O(n^2). Edges: all zero, <3 elements, many dups."*

Code: [code/ThreeSum.swift](#)

10. Minimum Size Subarray Sum (stretch)

Say this first:

*"Positive nums assumed for two-pointer window. Expand until sum ≥ target; shrink left; track min length.
Impossible → 0. O(n)."*

Code: [code/MinSubArrayLen.swift](#)

11. Group Anagrams (bridge)

Say this first:

*"Key by sorted string or 26-count signature; bucket in dictionary.
O(n·k log k) sorted keys or O(n·k) counts."*

Code: [code/GroupAnagrams.swift](#)

12. Pattern ID drills (no code)

Prompt fragment	Reach for
-----------------	-----------

"indices of two numbers"	Hash map
"longest substring with at most K"	Variable window
"sorted array, pairs"	Opposite pointers
"maximum subarray sum"	Kadane
"in-place remove / compact"	Write pointer
"product except self / no division"	Prefix-suffix
"anagrams"	Frequency signature

13. When interviewer changes constraints mid-flight

Change	Response
Need O(1) space after map solution	Restate trade-off; ask if sort/mutate OK; propose alt
Want brute coded too	Sketch brute verbally; code optimal; note $n \leq 20$ brute OK
Streaming input	Discuss what breaks (need full array?)

Trap: Silent panic rewrite.

14. Off-by-one window checklist

Before submitting mentally:

1. Is `right` inclusive in length `right - left + 1` ?
2. When duplicate at `right`, is `left = last[c] + 1` ?
3. Did you `max(left, ...)` so left never moves backward?
4. Dry-run `"abba"` for longest unique substring.

15. Full dry-runs (speak these once)

15.1 Two Sum — `[2,7,11,15]`, target `9`

i	value	need	seen before step	action
0	2	7	{}	store 2→0
1	7	2	{2:0}	found → [0,1]

15.2 Longest substring — `"abba"`

right	ch	last	left before	left after	best
0	a	—	0	0	1
1	b	—	0	0	2
2	b	1	0	$\max(0,2)=2$	2
3	a	0	2	$\max(2,1)=2$	2

Answer length `2` (`"bb"` or `"ba"` depending on window — length 2).

15.3 Kadane — [-2,1,-3,4,-1,2,1,-5,4]

Track running / best through each element; best ends at 6 ([4,-1,2,1]).

15.4 Container water — [1,8,6,2,5,4,8,3,7]

Start l=0, r=8 → min(1,7)*8=8; move left (shorter). Continue until best 49.

15.5 Product except self — [1,2,3,4]

Left pass out= [1,1,2,6] ; right multiply → [24,12,8,6] .

16. Brute → optimal comparison table

Problem	Brute	Optimal	What you say
Two Sum	O(n²) pairs	O(n) map	"Indices → map beats sort+scan unless O(1) space forced"
Stock	O(n²) pairs	O(n) min	"Running min is enough for one transaction"
Palindrome	Build cleaned string + reverse	Two pointers	"Same O(n); pointers save an allocation"
Water	O(n²)	O(n) ends	"Move shorter; width shrinks anyway"
Unique substring	O(n²) check sets	O(n) window	"Each index moves ≤ once"
Max subarray	O(n²)/O(n³)	O(n) Kadane	"Reset when extending loses"
Product	Divide by nums[i]	Prefix/suffix	"Division fails on zeros; prompt forbids it"
Move zeroes	Extra array	Write pointer	"Stable compact in place"
3Sum	O(n³)	Sort + O(n²)	"Pay n log n to enable two-sum"
Min subarray	O(n²)	O(n) window	"Positives → shrink safely"

17. Edge-case bank (quiz yourself)

Problem	Edge	Expected intuition
Two Sum	Duplicate values, distinct indices	Map stores latest or first — be consistent with "one answer"
Stock	Strictly decreasing	Profit 0
Palindrome	","."	true
Water	n=2	Only one container
Unique substring	"" / " "	0 / 1
Kadane	All negative	Largest (least negative) element
Product	[0,0,1]	Careful zeros
Move zeroes	[0,0,0]	Still all zeroes
3Sum	[-1,-1,2]	One triplet; skip dups

Min window sum	Target larger than total	0
----------------	--------------------------	---

18. Narration anti-patterns

Anti-pattern	Fix
Coding while silent	Forced 60s plan
"I've seen this" then blank	Still state pattern + invariant
Ignoring interviewer hints	Rephrase hint into plan change
Optimizing microconstants	Big-O + clarity first
Swift <code>s[i]</code> with <code>Int</code>	[Character] honesty

19. Link to dsa-track (optional)

Canonical list also lives in [../..../coding/dsa-track.md](#) .

This chapter embeds Week 1 patterns and solutions — use the track for spaced review across weeks, not as a blocker today.

Next

Implement from memory using [code/](#) , then [03-production-bridge.md](#) .

Exit checklist (deep dive)

- ☐ Spoke Two Sum + window + Kadane approaches without peeking
- ☐ Dry-ran "abba" and one Kadane array
- ☐ Named brute vs optimal for each core problem once
- ☐ Opened at least 6 files under `code/` and explained them aloud
- ☐ Recited say-this-first from memory once more before coding block

03 — Production Bridge

DSA as senior communication. Light BMS metaphors only — no fake "I shipped Kadane in ads."

1. How seniors talk about DSA

Weak signal	Strong signal
Silent coding for 20 min	60–90s plan first
Only optimal, can't explain brute	Brute → why optimize
Pretend String is $O(1)$ index	Honest Swift costs
Panic on constraint change	Restate plan
"I memorized this problem"	Pattern + invariant

Interview line:

"I treat DSA as structured communication under uncertainty — clarify, brute, optimize, then code."

2. Soft product metaphors (optional color — not claims)

DSA idea	Soft metaphor	Honest label
Cancel / restart window logic	Search debounce cancels in-flight	Verified · S3 is debounce product work — not “sliding window in prod”
Window over a list	Feed pagination cursors	SD later — clarifying question skill
Race-free shared maps	Day 04 SafeDict	Verified · S2 if interviewer pivots to concurrency

Do **not** say you used two pointers to optimize BookMyShow listing FPS.

3. When DSA interviews pivot to iOS

Be ready to hop:

- Swift String indexing
- Copy costs / COW on arrays you mutate
- Main-thread: don't solve LC on main in an app loop
- Cancellation if a “search” flavored problem appears → S3 debounce instinct

Keep hop ≤30s then return to the problem.

4. Catch-up bridge to Week 1 stories

If energy remains after 6 solves:

Weak topic	20s refresh
POP	S1 pitch
ARC	weak vs unowned
GCD	SafeDict sync set + visibility caveat
Actors	reentrancy one-liner
Value types	struct vs class

5. Mapping “communication grade” to timing guide

From [../.../timing/answer-timing-guide.md](https://leetcode.com/timings/answer-timing-guide.md) coding section:

Segment	Budget
Clarify + approach	2–3 min
Code	bulk of remaining
Test edges aloud	continuous
Complexity recap	15–20s at end

Fail the communication grade if you skip approach even with a correct solution.

6. Sample pivot scripts (honest)

Interviewer: “Have you used this at work?”

"Not this exact LeetCode. The skill I reuse is clarifying constraints and stating trade-offs under time pressure — same muscle as production debugging. For concurrency maps we used GCD SafeDict patterns (S2); for search UX we canceled in-flight work (S3). I won't pretend Kadane shipped in ads."

Interviewer: "Why Swift arrays not linked lists?"

"Random access and contiguous layout fit these problems; I'd reach for lists when insert/delete at head dominates — Week 2."

7. Anti-patterns

Anti-pattern	Fix
Inventing LC → production metrics	Don't
Skipping approach under time pressure	Always 60s plan
Copying solution without edges	Dry-run one example
Only Easy forever	Mix Medium windows/pointers
Claiming sliding window = S3	S3 is product debounce; pattern is Learning-lab

8. Flash map card

What you practice today: Pattern ID + narration + Swift solutions in `code/` .

Interview line:

"Clarify, brute, optimize, edges — then code."

Next

[04-questions.md](#) — pattern Q&A + full approach scripts as Full spoken answers.

04 — Questions (two-layer Q&A)

Pattern / meta questions use the standard two-layer shape.

*Problem rows use **Answer points** = approach outline; **Full spoken answer** = full 60–90s approach script (then you'd code).*

Normal — pattern & meta

Q1. Explain two pointers (30–45s)

Answer points (frame first):

- Two indices moving through a sequence
- Opposite ends or same-direction / write pointer
- Maintain an invariant; move the side that restores it
- Sorted pairs, palindrome, container water

Agenda opener:

"Two indices, an invariant, and a rule for which one moves."

Full spoken answer:

"Two pointers means I maintain two indices into an array or string and move them according to an invariant. Classic opposite-ends: palindrome checks or container-with-most-water, where I move the pointer that can still improve the answer. Same-

direction patterns include a write pointer that compacts non-zeroes in place. It's usually $O(n)$ time and $O(1)$ extra space when the input is already suitable — if I must sort first, I say that cost up front."

Common wrong answer:

"Two pointers always means left and right at the ends."

Follow-up ladder:

- **L1:** Fast/slow vs opposite ends?
- **L2:** When does sorting + two pointers beat a hash map?
- **L3:** Write pointer vs allocate a new array?

Provenance: Learning-lab · pattern fluency

Q2. Sliding window vs two pointers? (45s)

Answer points (frame first):

- Window = contiguous range with an invariant
- Expand right; shrink left when broken
- Two pointers is broader family
- Variable vs fixed window

Agenda opener:

"Window maintains a contiguous invariant; pointers are the mechanism."

Full spoken answer:

"A sliding window is a contiguous subarray or substring where I expand the right edge to include elements and shrink the left when a constraint breaks — like longest substring without repeating characters. Two pointers is the broader family that includes windows, opposite ends, and write pointers. I'd call it a window when the answer is about a contiguous range with a running condition."

Common wrong answer:

Treating them as unrelated jargon with no invariant talk.

Follow-up ladder:

- **L1:** Fixed vs variable window?
- **L2:** Why negatives break some window sum tricks?
- **L3:** Off-by-one checklist?

Provenance: Learning-lab

Q3. When prefix sums? (30–45s)

Answer points (frame first):

- Many range-sum queries
- $O(n)$ prep $\rightarrow O(1)$ range
- Prefix/suffix products variant (product except self)
- Negatives $\rightarrow$ often prefix + hash, not two pointers

Agenda opener:

"Pay $O(n)$ once for fast range aggregates."

Full spoken answer:

"Prefix sums help when I need many range sums: build a prefix array so a range is a subtraction. Product-of-array-except-self is the same idea with prefix and suffix products without division. If the problem has negatives and asks for subarray sums equaling k , I shift to prefix sums plus a hash map — pure two-pointer windows usually need monotonic positivity."

Common wrong answer:

"Prefix sums replace Kadane for maximum subarray."

Follow-up ladder:

- **L1:** Space trade-off of storing prefix?
- **L2:** Difference from Kadane?
- **L3:** Mutable vs immutable prefix?

Provenance: Learning-lab

Q4. Why narrate before coding? (30s)

Answer points (frame first):

- Interviewer grades process
- Catch wrong approach early
- Align on constraints
- Shows senior communication

Agenda opener:

"Process under uncertainty is the product."

Full spoken answer:

"Narrating the approach first lets the interviewer correct constraints early and shows how I think — brute force, why a pattern fits, complexity, and edges. Silent coding hides mistakes until the end. As a senior signal I'd rather burn sixty seconds aligning than ten minutes implementing the wrong plan."

Common wrong answer:

"Talking wastes time; coding proves skill."

Follow-up ladder:

- **L1:** What if they say 'just code'?
- **L2:** How do you handle mid-problem constraint change?
- **L3:** Time split for 45-minute interview?

Provenance: Communication skill

Normal — problem approach scripts

Q5. Two Sum — approach (60–90s)

Answer points (frame first):

- Need indices → careful with sorting
- Brute $O(n^2)$ pairs
- One pass map value→index
- Edges: duplicates, no pair, negatives

Full spoken answer:

"Constraints — is there exactly one answer? Duplicates allowed? Brute force tries all pairs in $O(n^2)$. Sorting would lose indices unless I store pairs, so I'll scan once with a dictionary from value to index: for each number I look up target minus that number; if found I return both indices; otherwise I store the current value. Time $O(n)$, space $O(n)$. Edges: empty input, no valid pair, negatives, and not using the same index twice. Coding that now."

Trap: Same index twice; returning values not indices.

Follow-up ladder:

- **L1:** Return all pairs?
- **L2:** $O(1)$ space if allowed to sort?
- **L3:** Streaming numbers?

Provenance: Learning-lab · `TwoSum.swift`

Q6. Buy/Sell Stock — approach (60–90s)

Answer points (frame first):

- One buy one sell
- Track min so far + max profit
- Brute pairs $O(n^2)$
- Edges: length 1, decreasing $\rightarrow 0$

Full spoken answer:

“One transaction — buy once, sell later. Brute force is all buy/sell pairs $O(n^2)$. I’ll track the minimum price so far and at each day compute profit as price minus that minimum, keeping the max profit. Time $O(n)$, space $O(1)$. Edges: single price returns zero; strictly decreasing returns zero. Coding the one-pass now.”

Trap: Sell before buy; wrong min update order.

Follow-up ladder:

- **L1:** k transactions?
- **L2:** With cooldown?
- **L3:** Fee per transaction?

Provenance: Learning-lab · `MaxProfit.swift`

Q7. Valid Palindrome — approach (60–90s)

Answer points (frame first):

- Opposite pointers
- Skip non-alnum; casefold
- Swift: `[Character]` for indexing
- Edges: empty, all punctuation

Full spoken answer:

“I’ll use two pointers from both ends. Skip non-alphanumeric characters, compare lowercased equals, move inward. In Swift String isn’t $O(1)$ random access, so I’ll convert to an array of characters for clear indexing — $O(n)$ space — unless you prefer String.Index walking. Time $O(n)$. Edges: empty string is true; all punctuation is true; I’ll assume ASCII alnum per typical LeetCode unless you want unicode rules.”

Trap: Ignoring case; not skipping; pretending `s[i]` works with Int.

Follow-up ladder:

- **L1:** Unicode normalization?
- **L2:** $O(1)$ extra space with indices?
- **L3:** Almost-palindrome (one delete)?

Provenance: Learning-lab · `ValidPalindrome.swift`

Q8. Container With Most Water — approach (60–90s)

Answer points (frame first):

- Area = min height × width
- Start at ends; move shorter

- Brute $O(n^2)$
- Edges: $n=2$, equal heights

Full spoken answer:

"Area is min of the two heights times the distance between indices. Brute checks all pairs $O(n^2)$. I'll start at both ends for maximum width and move the shorter pointer inward, because width always shrinks and only a taller short side can improve the min height. Time $O(n)$, space $O(1)$. Edges: exactly two lines; all heights equal. Coding that."

Trap: Moving the taller pointer; off-by-one width.

Follow-up ladder:

- **L1:** Prove why move shorter?
- **L2:** Histogram largest rectangle contrast?
- **L3:** 3D variant joke → stay focused?

Provenance: Learning-lab · `ContainerWater.swift`

Q9. Longest Substring Without Repeating — approach (60–90s)

Answer points (frame first):

- Variable window + last index map
- Expand right; lift left past duplicate
- $O(n)$; each index moves $\leq$ once
- Edges: empty, all unique, all same

Full spoken answer:

"I need the longest contiguous substring with unique characters. I'll keep a sliding window and a map of character to last index. Expand right; if the character was seen inside the window, move left to lastIndex + 1 — taking max so left never jumps backward. Track max window length. Each index moves at most once so $O(n)$ time, space $O(\text{alphabet})$. Edges: empty string, all unique, all identical characters. I'll dry-run 'abba' mentally for the left jump. Coding now."

Trap: Forgetting `max(left, prev+1)` ; wrong length formula.

Follow-up ladder:

- **L1:** At most k distinct characters?
- **L2:** Return the substring not length?
- **L3:** Byte vs Character in Swift?

Provenance: Learning-lab · `LengthOfLongestSubstring.swift`

Q10. Maximum Subarray — approach (60–90s)

Answer points (frame first):

- Kadane running max
- `running = max(x, running+x)`
- All-negative → largest element
- $O(n)/O(1)$

Full spoken answer:

"Maximum contiguous sum — brute is $O(n^2)$. Kadane keeps a running sum: at each value, running becomes max of starting fresh at x versus extending `running + x`, and I track the global max. That correctly handles all-negative arrays by returning the largest element. Time $O(n)$, space $O(1)$. Edges: single element; all negative; mix of signs. Coding Kadane."

Trap: Resetting to zero and returning 0 on all-negative when empty subarray isn't allowed.

Follow-up ladder:

- **L1:** Also return bounds?
- **L2:** Circular maximum subarray?
- **L3:** 2D kadane?

Provenance: Learning-lab · `MaxSubArray.swift`

Q11. Product Except Self — approach (60–90s)

Answer points (frame first):

- No division
- Left products into output; right pass multiply
- $O(n)$; $O(1)$ extra aside from output
- Edges: zeros, negatives

Full spoken answer:

"Each output index is the product of all other elements without division. I'll put prefix products into the output array on a left-to-right pass, then multiply by a running right product on the way back. Time $O(n)$. Extra space $O(1)$ if the output array doesn't count. Edges: one zero, two zeros, negatives. I won't use division because zeros break it and the prompt forbids it."

Trap: Division; not handling zeros; $O(n)$ left/right arrays without noting optimization.

Follow-up ladder:

- **L1:** Follow-up $O(1)$ extra?
- **L2:** Modular product?
- **L3:** Streaming?

Provenance: Learning-lab · `ProductExceptSelf.swift`

Q12. Move Zeroes / write pointer — approach (60s)

Answer points (frame first):

- Write non-zeroes forward
- Fill remaining zeroes
- Stable order
- $O(n)/O(1)$

Full spoken answer:

"I'll keep a write index. Scan left to right, and whenever I see a non-zero I write it at the write index and advance. Then fill the tail with zeroes. That keeps relative order and uses $O(1)$ extra space. Edges: no zeroes, all zeroes, already compacted."

Trap: Removing during for-in; swapping incorrectly and scrambling order if unstable swap used carelessly.

Follow-up ladder:

- **L1:** Move zeroes to front?
- **L2:** `removeElement` general pattern?
- **L3:** Why not `removeAll` repeatedly?

Provenance: Learning-lab · `MoveZeroes.swift`

Tricky

T1. Interviewer asks $O(1)$ space after your map solution (90s)

Answer points (frame first):

- Don't panic rewrite silently
- Restate trade-off

- Ask if sort/mutate allowed
- Propose alternative or say map is justified for indices

Trap: Silent erase of working solution.

Full spoken answer:

"If we need $O(1)$ extra space, I'd clarify whether I may sort or mutate the input. For Two Sum, sorting with paired indices can work then two-pointer, but it's $O(n \log n)$ time. If indices matter and I can't use a map, I need that clarity. I'm happy to recode after we lock the constraint — I won't silently thrash the current approach."

Follow-up ladder:

- **L1:** When is $O(n)$ space acceptable?
- **L2:** External memory constraints?
- **L3:** Parallelism?

Provenance: Communication skill

T2. Swift String indexing feels $O(n)$ (90s)

Answer points (frame first):

- String not Int-subscript random access
- Convert to `[Character]` when needed
- Mention cost honestly
- Alternative: `String.Index` walking

Trap: Pretend `s[i]` with Int works / is $O(1)$.

Full spoken answer:

"Swift's String isn't a random-access array of characters with Int subscripts. Indexing with String.Index can be linear if I advance from start repeatedly. In an interview I'll convert to `Array(s)` for $O(1)$ indexing and call out the $O(n)$ copy, or carefully walk indices once. I'd rather be honest about the cost than write incorrect Int subscripts."

Follow-up ladder:

- **L1:** Character vs Unicode scalar?
- **L2:** UTF-8 ContiguousBytes?
- **L3:** Performance in production parsing?

Provenance: Learning-lab · Swift realism

T3. Off-by-one in window (90s)

Answer points (frame first):

- State invariant out loud
- Dry-run a known tricky string
- $\text{Length} = \text{right} - \text{left} + 1$
- $\text{left} = \text{last} + 1$ with max

Trap: Expand/shrink without invariant.

Full spoken answer:

"When a window is wrong, I restate the invariant: characters from left through right are unique. Length is right minus left plus one. When I see a duplicate, left becomes $\max(\text{left}, \text{lastIndex} + 1)$ so I never move backward. I'll dry-run a case like 'abba' before adjusting code. That catches most off-by-ones faster than random tweaks."

Follow-up ladder:

- **L1:** Inclusive vs exclusive right?

- **L2:** Fixed window off-by-ones?
- **L3:** Empty window handling?

Provenance: Learning-lab

T4. Mutating array while iterating (90s)

Answer points (frame first):

- Don't remove during for-in casually
- Write pointer pattern
- Or iterate indices carefully
- Prefer new array if space OK and clarity wins

Trap: `for x in arr { arr.remove }` chaos.

Full spoken answer:

"Mutating an array's length while using for-in is a footgun. For in-place compaction I use a write pointer over indices: read with `i`, write non-removed elements forward, then truncate or zero-fill. If mutation rules allow allocating, a new array can be clearer — I'll ask. For Move Zeroes the write-pointer pattern is the intended $O(1)$ -space approach."

Follow-up ladder:

- **L1:** `removeFirst` in a loop cost?
- **L2:** Copy-on-write surprises when sharing arrays?
- **L3:** Concurrent mutation — Day 04 link?

Provenance: Learning-lab · write pointer

T5. They want brute and optimal (120s)

Answer points (frame first):

- Sketch brute verbally with complexity
- Code optimal
- Note when brute OK (tiny n)
- Keep calm timeboxing

Trap: Only coding optimal with no brute discussion — or coding both fully and running out of time.

Full spoken answer:

"I'll state the brute force clearly — for example all pairs $O(n^2)$ — so you know I see the naive baseline, then I'll implement the optimal $O(n)$ approach. If n is tiny, say ≤ 20 , brute might be acceptable in production; here with interview constraints I'll code the linear pass. If you want both fully typed, I'll timebox brute as a short stub and focus on the optimal with tests."

Follow-up ladder:

- **L1:** How long for brute stub?
- **L2:** Property tests vs examples?
- **L3:** Complexity proof depth expected?

Provenance: Communication skill

Stretch approach scripts

Q13. 3Sum — approach (90s)

Answer points (frame first):

- Sort first $O(n \log n)$
- Fix i ; two-sum with left/right

- Skip duplicates on i/left/right
- $O(n^2)$; edges all-zero, <3 elems

Full spoken answer:

"I'll sort, then for each index i run two pointers on the remainder looking for sum zero, skipping duplicate values so we don't emit the same triplet. Sorting costs $n \log n$; the nested scan is $O(n^2)$. Edges: fewer than three numbers, all zeros, heavy duplicates. Coding sort plus two-sum now."

Trap: Forgetting duplicate skips; using hash without sorting discussion.

Provenance: Learning-lab · `ThreeSum.swift`

Q14. Min Size Subarray Sum — approach (90s)

Answer points (frame first):

- Positive nums $\rightarrow$ variable window
- Expand until sum $\geq$ target; shrink; track min length
- Impossible $\rightarrow$ 0
- $O(n)/O(1)$

Full spoken answer:

"Assuming positive numbers, I'll expand a right pointer adding to a running sum; while the sum is at least target I'll shrink from the left and track the minimum window length. If I never reach the target, return zero. Each pointer moves at most n times so $O(n)$. If negatives were allowed, this shrink logic would break and I'd switch strategies."

Trap: Using window with negatives; off-by-one length.

Provenance: Learning-lab · `MinSubArrayLen.swift`

Q15. Group Anagrams — approach (60–90s)

Answer points (frame first):

- Key by sorted string or count signature
- Bucket in dictionary
- $O(n \cdot k \log k)$ or $O(n \cdot k)$
- Edges: empty strings

Full spoken answer:

"Anagrams share a sorted character key or a 26-length count signature. I'll map key to a list of strings and return the buckets. Sorted keys are $O(n \cdot k \log k)$; count signatures are $O(n \cdot k)$. Edges: empty strings, single characters, already identical inputs."

Provenance: Learning-lab · `GroupAnagrams.swift`

Timed sets

Set	Focus
A	Q1, Q2, Q9, T2, T3
B	Q5–Q8 approach scripts only (no code)
C	Two timed problems: 3 min approach + 25 min code each
D	Q13–Q15 stretch scripts

Score approach quality with [../../../timing/answer-timing-guide.md](https://github.com/learning-lab/timing/answer-timing-guide.md) coding section.

05 — Exercises

Exercise 1 — Recite then solve (per problem 30–40 min)

For each of: Two Sum, Max Profit, Valid Palindrome, Container Water, Longest Substring, Kadane:

1. Cover `code/` and deep dive.
2. Speak 60–90s approach into a recorder.
3. Implement cold in Swift.
4. Compare to [code/](#).

Minimum 6 cold solves today.

Exercise 2 — Stretch pair (40–60 min)

3Sum + Min Size Subarray Sum or Product Except Self + Group Anagrams.

Exercise 3 — Pattern ID flash (15 min)

Close notes. For 12 prompt fragments in deep dive §12, name the pattern aloud in <5s each.

Exercise 4 — Timed contest simulation (60–70 min)

1. Problem A: 3 min approach + 25 min code.
 2. Problem B: same.
 3. Retro: pattern mis-IDs → gotchas.
-

Exercise 5 — Catch-up (45–60 min)

Pick weakest of Days 01–05. Redo 5 flashcards + one spoken Q from that day's `04-questions.md`.

Weak topic today: _____

Exercise 6 — SD light (15 min)

Sketch Social Feed cursor pagination request/response JSON (fields only). Clarifying questions > perfect schema.

Exercise 7 — Smoke (10 min)

Read [code/Solutions.swift](#) assertions; mentally execute or run in playground.

Exercise 8 — Approach-only marathon (25–35 min)

Without coding, speak full 60–90s scripts for all of:

Two Sum · Stock · Palindrome · Water · Unique substring · Kadane · Product except self · Move zeroes

Record; flag any script missing edges or complexity.

Exercise 9 — Wrong-pattern recovery (20 min)

Pick a problem you mis-IDed this week (or pretend):

1. State the wrong pattern you almost used.
2. Why the signal misled you.
3. Correct pattern + one-sentence invariant.
4. Re-code in ≤ 20 min.

Exercise 10 — Peer/self rubric (10 min)

For one timed problem, score yourself:

Check	Pass?
Clarified constraints	
Named brute + cost	
Named optimal + why	
Edges listed	
Time + space spoken	
Swift String caveat if relevant	
Finished or mid-point check-in	

Exit criteria

- ☐ say-this-first fluent
- ☐ 6+ problems solved cold in Swift
- ☐ Two timed full problems with spoken approach
- ☐ Swift String indexing explained honestly once
- ☐ Approach-only marathon completed once
- ☐ Catch-up topic touched if needed

Revision twin: [../../revision/weeks/week-01/day-06.md](https://github.com/alexeyrasnitsyn/revision/weeks/week-01/day-06.md)

Day 07 — Week 1 Revision + Mock Interview #1

Week 1 · Mock day · Full study (self-contained) · ~5–6 hrs

Revision twin: [revision/weeks/week-01/day-07.md](https://github.com/alexeyrasnitsyn/revision/weeks/week-01/day-07.md)

Outcomes

By end of day:

- Active-recall Week 1 core ($\geq 80\%$ flashcard pass target)
- Deliver **S2** synchronised dictionaries in ≤ 3 min at score ≥ 4
- Complete **Mock #1** with full script, scoring rubric, and recorded answers
- Speak warm-up pool answers from **Answer points** (full two-layer answers live in `04-questions.md` — not skeletons)
- Close Social Feed HLD clarifying + cache/scroll bullets (light)

How to study

1. [01-foundations.md](https://github.com/alexeyrasnitsyn/revision/weeks/week-01/day-01.md) — revision map + mock format
2. [02-deep-dive.md](https://github.com/alexeyrasnitsyn/revision/weeks/week-01/day-02.md) — full Mock #1 interviewer script
3. [03-production-bridge.md](https://github.com/alexeyrasnitsyn/revision/weeks/week-01/day-03.md) — S2 centerpiece + reliability spine
4. [04-questions.md](https://github.com/alexeyrasnitsyn/revision/weeks/week-01/day-04.md) — **full two-layer** warm-up + deep pool

5. [05-exercises.md](#) — run the mock + scorecard
6. Optional: skim prior day folders only if a card fails — chapter stays self-contained via Q&A

Module map

Module	File
Foundations	01-foundations.md
Deep dive / mock script	02-deep-dive.md
Production bridge	03-production-bridge.md
Questions	04-questions.md
Exercises / scorecard	05-exercises.md
Code	code/MockScorecard.md

Provenance spine today

Label	Role
Verified · S2	Mock centerpiece story
How I would apply it · S2-A1	Actor migration follow-up
Verified · S8	Reliability / CFS culture when relevant
Verified · S1	Optional POP warm-up / preview Mock #2
Days 01–06 concepts	Warm-up + deep dive pool

Pass criteria (Mock #1)

- Average ≥ 3.5 on deep-dive answers
- S2 ≥ 4
- No answer $> 2\times$ budget without self-correction
- At least one explicit trade-off in concurrency discussion
- Zero invented fill-rate / crash-% ownership claims

Agenda opener for the mock

"Defs → concurrency + memory deep dive → S2 story → feed HLD → retro."

01 — Foundations: Week 1 Revision Map + Mock Format

No new topics. Compress Week 1 into speakable atoms, then run Mock #1.

0. North star

Prove you can retrieve and trade-off Week 1 under a timer — with resume-honest provenance.

1. Revision map (must-nail)

Topic	Day	Must-nail line
-------	-----	----------------

struct/class/COW/enum	01	Value semantics; payment states as enums (S7-A1 design)
POP/generics/type erasure	02	Ads pipeline rationale (S1); erasure costs
ARC/cycles/Instruments	03	weak vs unowned; Graph vs Leaks
GCD SafeDict	04	Serial API; deadlock; async-write visibility; <code>final class</code>
async/await/actors/Sendable	05	Reentrancy; cancel; S2-A1
DSA patterns	06	Window + two pointers + say-this-first

2. Flashcard strategy (morning)

1. Full Week 1 deck: [../../../flashcards/week-01.md](#)
2. Mark misses.
3. Re-speak misses as 20s answers (not silent reread).
4. Target $\geq 80\%$ correct on second pass.

Meta cards for today:

Front	Back
Mock agenda	Defs → concurrency/memory → S2 → feed HLD → retro
S2 time box	≤ 3 min STAR
Score 5	On time + trade-off + prod proof + honest provenance
Weak topic rule	Re-drill before Week 2
SD clarify first	DAU, offline, pagination, ads mix
No new topics	Revision only

3. Mock #1 format (60–90 min)

Block	Time	Content
Warm-up defs	10 min	5× ~45s from warm-up pool
Deep dive	25 min	Concurrency + memory follow-ups
Story	10 min	S2 full STAR + actor migration follow-up
Mini SD	20 min	Social feed: clarify + HLD only
Retro	10–15 min	Score with rubric

Partner or self: interviewer reads from [02-deep-dive.md](#) . Candidate uses **no notes**.

4. Scoring rubric (1–5)

Score	Meaning
1	Blank / wrong concept
2	Partial; major hole or invented claim
3	Correct core; weak structure or overtime

4	On time; agenda; trade-off or prod hook
5	4 + crisp provenance + follow-up ready

Use `../../../../timing/answer-timing-guide.md` for budgets.

5. Warm-up pool (IDs — full answers in 04)

1. struct vs class
 2. COW
 3. weak vs unowned
 4. serial vs concurrent
 5. async/await vs GCD
 6. thread-safe dictionary design
 7. actor isolation
 8. Sendable
 9. retain cycle examples
 10. Main-queue deadlock
 11. Task cancellation
 12. POP in ads pipeline
-

6. Deep pool (IDs)

1. Actor reentrancy after await
 2. sync to serial queue re-entry deadlock
 3. Memory Graph cycle vs Leaks
 4. `@unchecked Sendable` ethics
 5. Type erasure cost in renderer
 6. COW uniqueness traps
 7. Mixing GCD sync inside async contexts
 8. Migrating SafeDict GCD → actor without big-bang
-

7. Self-check before mock

- ☐ S2 ≤20s pitch cold
- ☐ Async write / sync read caveat one sentence
- ☐ weak vs unowned one sentence
- ☐ Actor reentrancy one sentence
- ☐ S1 ≤20s available as optional encore

→ [02-deep-dive.md](#) for the live script.

02 — Deep Dive: Full Mock #1 Interviewer Script

*Print or split-screen this. **Interviewer** reads bold prompts. Timing visible.*

Candidate answers without notes. After each answer, optional follow-up from the ladder.

0. Setup (2 min)

Interviewer says:

"This is Mock #1 — Week 1 concurrency and memory. Agenda: short definitions, a deep dive, your synchronised-dictionaries story, then a social-feed HLD sketch. I'll cut you off if you're 2× over time — self-correct and continue. Ready?"

Start timer for the whole mock.

1. Warm-up definitions (10 min)

Ask **any 5** from the list. Budget ~45–60s each.

Full model answers: [04-questions.md](#) W1–W12.

Scripted set A (default)

#	Prompt	Budget
1	"Struct versus class — when do you choose each?"	45s
2	"What is copy-on-write?"	45s
3	"weak versus unowned?"	45s
4	"Serial versus concurrent queue?"	45s
5	"In one minute: how would you design a thread-safe dictionary?"	60–90s

Alternate set B

POP in ads · Main-queue deadlock · Actor isolation · Sendable · Task cancellation

Interviewer notes: Mark agenda? trade-off? provenance honesty?

2. Deep dive (25 min)

Ask **4–5** items. Allow follow-ups. Budgets 90–120s unless noted.

D1 — Actor reentrancy (120s)

"You `await` inside an actor method. Can another task mutate the actor's state before you resume? What breaks if you assume continuity?"

Follow-ups:

- How do you harden load-if-missing?
- Contrast with GCD serial queue (no await suspension in the same way)

D2 — Serial sync re-entry (90s)

"You're on a private serial queue and call `queue.sync` again from nested code. What happens? Is this only a main-queue issue?"

Follow-ups:

- Unlocked internal pattern?
- `dispatchPrecondition` ?

D3 — Memory Graph vs Leaks (90s)

"Memory Graph shows a retain cycle but Leaks shows nothing. Why aren't those tools synonyms?"

Follow-ups:

- Abandoned memory vs leak?
- What do you do next in Xcode?

D4 — GCD → actor migration (120s)

"You have a GCD `SafeDict` in production. How would you migrate a module to an actor without a big-bang rewrite?"

Follow-ups:

- API surface stay sync somehow? (adapters / async façade)
- Label Verified S2 vs Applied S2-A1

D5 — Pick one stretch

Choose based on misses:

- `@unchecked Sendable` ethics (90s)
- Type erasure cost in ads renderer (90s)
- Async write then sync read visibility (90s)
- Mixing `queue.sync` inside `async` functions (90s)

Model answers: [04-questions.md](#) deep section.

3. Story — S2 (10 min)

Prompt

"Tell me about a concurrency bug you fixed in production. You have three minutes."

Listen for STAR. Score with story rubric:

Beat	Present?
Situation: shared dicts raced	
Action: serial queues / RW / safe API	
Result: path-specific crash reduction	
Lesson + actor coda	
No CFS ownership theft / no fake crash %	

Required follow-up (90s)

"How would you design this today with Swift concurrency?"

Expect **How I would apply it · S2-A1** — actor, same safe API, await, not "we rewrote everything as actors."

Optional follow-up

"Why hide the queue instead of letting callers dispatch onto it?"

4. Mini system design — Social Feed (20 min)

Prompt

"Design the client side of a social/listing feed for a large consumer app — think BookMyShow-scale traffic. Clarify first, then high-level design only. No need for full LLD today."

Good clarifying questions (candidate should ask ~4–6)

- Organic vs ads mixing rules?
- Pagination model (cursor vs offset)?
- Offline / stale content expectations?
- Image / video autoplay policy?
- Realtime invalidation vs pull-to-refresh?

- Approximate DAU / latency targets? (candidate may cite 30L+ DAU as **Verified · S8** scale context — not invent new numbers)

HLD bullets interviewer wants to hear

Area	Expect
API	Cursor pagination request/response
Caching	Memory + disk tiers; TTL/invalidation sketch
Images	Prefetch; cancel on scroll away
Concurrency	Don't block main; cancel in-flight
Failure	Empty/error/retry states
Ads	Slot injection without forking feed pipeline (S1 instinct OK)

Interviewer: Cut at 20 min even if incomplete — note what was missing.

5. Retro (10–15 min)

1. Fill [code/MockScorecard.md](#) .
2. Average deep-dive scores.
3. List top 5 weak cards → pin to Week 2 warm-ups.
4. Optional encore: S1 Ads POP talk ≤5 min (preview Mock #2).

6. Interlocutor “cut” lines (use sparingly)

“You’re at 2× — give me the trade-off in one sentence.”

“Don’t invent a metric — what’s the honest result?”

“Agenda first.”

“Is that Verified or how you’d apply it?”

7. Self-mock mode

If solo: record voice memo; play back against 04-questions Full spoken answers; still fill scorecard honestly.

03 — Production Bridge — Mock Narrative Spine

Today’s story spine: **reliability under concurrency at BMS scale**.

1. Centerpiece — Verified · S2

≤20s:

“We gated shared dictionaries behind a serial queue API so call sites couldn’t race the storage — crashes went away on that path.”

≤3 min **STAR:** Use story-bank S2 beats — races → serial queues / RW → safe API → stress + Crashlytics → path fixed → actor coda.

Must not: Claim sole ownership of 99.95% CFS from S2 alone.

Provenance: Verified · S2 · BookMyShow · synchronised dictionaries

2. Follow-up — S2-A1

"For a new module I'd expose an actor with the same get/set/snapshot surface — callers await; isolation moves into the type system. That's how I'd apply it — production was GCD."

Provenance: *How I would apply it · S2-A1*

3. Adjacent reliability — S8 (soft)

Use when asked about production risk / why races matter:

- 30L+ DAU
- 99.95%+ crash-free
- Crashlytics / IMOC culture

Do not paste CFS onto every answer.

Provenance: *Verified · S8*

4. Optional encore — S1

If energy allows after mock:

"We made ad rendering a generic protocol pipeline so new creatives plugged in without forking the revenue path."

Provenance: *Verified · S1*

5. Social feed talk — honesty

Feed HLD is design skill. Tie ads slots to S1 *instinct* only. Scale context from S8 when asked "how big."

6. Anti-patterns today

Anti-pattern	Fix
New topics mid-mock	Redirect to Week 1
Metric invention	Qualitative path fix
Skipping provenance labels on actor migration	Say Applied
Overlong defs	Agenda + cut

Next

Drill warm-up pool in [04-questions.md](#) **before** the timed mock if any card is shaky.

04 — Questions (two-layer Q&A) — Mock Warm-up + Deep Pool

Not skeletons. Speak from **Answer points**; compare to **Full spoken answer**.

This file is self-contained for Mock #1 practice.

Warm-up pool

W1. Struct vs class? (45s)

Answer points (frame first):

- Value vs reference semantics
- Shared mutation only with classes
- Prefer struct for models; class for identity/UIKit
- Ads models value-friendly hook

Agenda opener: "Semantics → shared mutation → when I pick each."

Full spoken answer:

"Structs and enums have value semantics — assignment gives an independent copy, so mutating one variable doesn't surprise another. Classes have reference semantics — multiple names can share one instance. I default to structs for models and DTOs, and use classes when I need identity, UIKit objects, or Objective-C interop. On BookMyShow ads work, keeping render models value-friendly fits a type-safe pipeline so accidental shared mutation doesn't corrupt revenue UI."

Common wrong answer: "Structs are always faster / always stack-allocated."

Follow-up ladder:

- **L1:** Struct containing a class?
- **L2:** ViewModel struct or class/ @MainActor ?
- **L3:** Actors vs classes for shared state?

Provenance: Verified · S1 · type-safe ads / listing models

W2. What is COW? (45s)

Answer points (frame first):

- Collections share buffer on assign
- Mutation checks uniqueness
- Unique → in place; else copy then write
- Array/Dictionary/String/Set

Agenda opener: "Cheap share until write — then unique copy."

Full spoken answer:

"Copy-on-write means value-typed collections like Array can share storage after assignment so the assign is cheap. On mutation, the runtime checks whether the buffer is uniquely referenced. If it is, it mutates in place; if not, it copies first. That preserves value semantics without paying a full element copy on every assignment. Trap: don't assume two variables always see each other's mutations."

Common wrong answer: "Assignment always deep-copies every element."

Follow-up ladder:

- **L1:** isKnownUniquelyReferenced ?
- **L2:** Handmade COW wrapper?
- **L3:** COW thrash on hot path?

Provenance: Learning-lab · value semantics

W3. weak vs unowned? (45s)

Answer points (frame first):

- weak: optional, zeroing
- unowned: non-optional, non-zeroing; crash if dangling
- Prefer weak for escaping async/UI callbacks
- unowned only when lifetimes provably nested

Agenda opener: "Both skip retain — they differ when the object dies."

Full spoken answer:

" weak is an optional reference that doesn't keep the object alive and becomes nil on deinit — so you guard let self. unowned also doesn't keep it alive, but it isn't optional and isn't zeroed; use after free crashes. I use weak for network and UI callbacks where lifetime is uncertain. I only use unowned when a child cannot outlive its parent by construction."

Common wrong answer: "Always unowned — it's faster."

Follow-up ladder:

- **L1:** Why is weak optional?
- **L2:** Closure cycle fix?
- **L3:** unowned(unsafe) ?

Provenance: Learning-lab · ARC discipline

W4. Serial vs concurrent queue? (45s)

Answer points (frame first):

- Serial: one at a time
- Concurrent: may overlap
- SafeDict uses private serial
- Main is serial for UI

Agenda opener: "One-at-a-time versus overlapping execution."

Full spoken answer:

"A serial queue runs one block at a time — mutual exclusion via queue. A concurrent queue may run overlapping blocks. Main is serial and owns UI. For synchronised dictionaries at BookMyShow we used a private serial queue as the exclusion boundary so shared map access couldn't race."

Common wrong answer: "Concurrent queues make dictionaries safe."

Follow-up ladder:

- **L1:** Barrier purpose?
- **L2:** QoS?
- **L3:** Actor equivalent?

Provenance: Verified · S2

W5. async/await vs GCD? (60s)

Answer points (frame first):

- async/await: suspension points; structured concurrency
- GCD: queues schedule blocks; isolation by convention
- Actors language-isolate state
- S2 was GCD; S2-A1 actors for new code

Agenda opener: "Suspension and structure versus queue scheduling."

Full spoken answer:

"GCD schedules work on queues — serial or concurrent — and you build thread safety by convention, like a serial queue around a dictionary. async/await models suspension points and pairs with structured concurrency so parent tasks own children and cancellation propagates. Actors add compiler-enforced isolation for mutable state. At BookMyShow we shipped synchronised dictionaries with GCD; for greenfield modules I'd evaluate an actor with the same safe API."

Common wrong answer: "async/await always runs on a background thread" / "actors replace all GCD including timers UI."

Follow-up ladder:

- **L1:** Task vs DispatchQueue.async?
- **L2:** Don't sync-block from async contexts?
- **L3:** MainActor vs main queue?

Provenance: Verified · S2; How I would apply it · S2-A1

W6. Thread-safe dictionary design (90s)

Answer points (frame first):

- `final class` ; private storage; private serial queue
- sync get; sync set for read-after-write
- Hide queue; snapshot returns copy
- RW/barrier if read-heavy; actor coda

Agenda opener: "Private serial queue — safe API, no raw storage."

Full spoken answer:

"I'd wrap the map in a `final class` with private storage and a private serial queue. Gets use sync so they can return values and respect prior work. Sets I'd sync when callers need read-after-write — async set is fire-and-forget and isn't a completion handshake for the next line. Snapshots return a copied dictionary under sync. Never expose storage or the queue. At BookMyShow that standardized API removed races on the shared-state path; we used RW locks where read-heavy. Today I'd evaluate an actor for new modules."

Common wrong answer: `Final class` / expose queue / concurrent writes without barrier / invent crash %.

Follow-up ladder:

- **L1:** Async write then sync read visibility?
- **L2:** Deadlock on sync re-entry?
- **L3:** Migrate to actor?

Provenance: Verified · S2; Learning-lab · Day 04 SafeDict

W7. Actor isolation (60s)

Answer points (frame first):

- Reference type with isolated mutable state
- Cross-isolation uses await
- Prevents data races at boundary
- Reentrancy: state may change across await

Agenda opener: "Isolated reference type — modern serial queue."

Full spoken answer:

"An actor is a reference type whose mutable state is isolated. External calls hop with await and the system serializes access so you don't data-race that state. It's the language-native version of what we did with a GCD serial queue around dictionaries. Important: after an await inside an actor method, other tasks may have run — reentrancy means you re-validate assumptions."

Common wrong answer: "Actors are value types" / "no reentrancy issues."

Follow-up ladder:

- **L1:** `@MainActor` ?
- **L2:** Sendable interaction?
- **L3:** Actor vs class + lock?

Provenance: How I would apply it · S2-A1; Day 05 concepts

W8. Sendable? (45s)

Answer points (frame first):

- Marker for safe sharing across concurrency domains
- Value types Sendable if stored properties are
- Classes need care / unchecked is a smell
- Compiler checks in strict modes

Agenda opener: "Safe to cross isolation boundaries."

Full spoken answer:

"Sendable means a type can be safely shared across concurrency domains. Many value types are Sendable when all stored properties are. Classes aren't automatically Sendable because of shared mutation. @unchecked Sendable opts out of checking — I'd treat it as a last resort with a documented invariant, not a convenience annotation."

Common wrong answer: "All structs are always Sendable" (nested non-Sendable breaks it).

Follow-up ladder:

- **L1:** Why actors are Sendable?
- **L2:** Closures @Sendable ?
- **L3:** Ethics of unchecked?

Provenance: Learning-lab · Swift concurrency

W9. Retain cycle examples (60s)

Answer points (frame first):

- Object owns escaping closure capturing self strongly
- Strong delegates; Timer targets; nested captures
- Fix: [weak self] , weak delegates, invalidate
- Detect: Memory Graph / missing deinit — not only Leaks

Agenda opener: "Reachable cycle — counts never hit zero."

Full spoken answer:

"Classic cycle: an object stores an escaping completion that strongly captures self — each owns the other. Same shape with strong delegates, repeating timers targeting self, and nested escaping closures. Fix with capture lists, weak delegates, and invalidation. Leaks instrument finds unreachable leaks; retain cycles are still reachable, so Memory Graph / Allocations matter."

Common wrong answer: "Any mention of self in a closure leaks."

Follow-up ladder:

- **L1:** Escaping vs non-escaping?
- **L2:** Task retaining self?
- **L3:** NotificationCenter tokens?

Provenance: Learning-lab · Day 03; reliability culture soft · S8

W10. Main-queue deadlock (45s)

Answer points (frame first):

- main.sync from main waits for itself
- Same for any serial re-entry sync
- UI hops: main.async
- Don't sync-block casually

Agenda opener: "You're waiting for yourself."

Full spoken answer:

"If you're on the main queue and call `DispatchQueue.main.sync`, main blocks waiting for a block only main can run — deadlock. Any serial queue can deadlock the same way on sync re-entry. After networking I hop to main with async for UI updates rather than syncing onto main from main."

Common wrong answer: "Only main can deadlock."

Follow-up ladder:

- **L1:** Private serial nested sync?
- **L2:** Unlocked internals pattern?
- **L3:** async world + queue.sync danger?

Provenance: Learning-lab · Day 04

W11. Task cancellation (60s)

Answer points (frame first):

- Cooperative: check `Task.isCancelled` / try `Task.checkCancellation`
- Structured parents cancel children
- Search debounce must cancel in-flight
- Unstructured Task needs manual lifecycle

Agenda opener: "Cooperative cancel — check or inherit structure."

Full spoken answer:

"Swift task cancellation is cooperative — work should check cancellation or use APIs that throw on cancel. With structured concurrency, canceling a parent cancels children. For search-as-you-type, debouncing means cancel the in-flight request when a new keystroke arrives so late responses don't overwrite fresh results. Fire-and-forget unstructured tasks need explicit cancel on disappear."

Common wrong answer: "Cancel instantly aborts all CPU work without checks."

Follow-up ladder:

- **L1:** `Task.detached` cancel behavior?
- **L2:** URLSession cancel?
- **L3:** S3 search debounce link?

Provenance: Verified · S3 soft for debounce product; concurrency mechanics Learning-lab

W12. POP in ads pipeline (60s)

Answer points (frame first):

- Capability protocols over inheritance trees
- Generics for type-safe install/bind
- HeroWidget lifecycle for video
- No invented fill-rate %

Agenda opener: "Protocols and generics on the revenue path."

Full spoken answer:

"On BookMyShow's highest-revenue Ads module we refactored rendering around protocol-oriented contracts and generics so new creatives plugged into a type-safe pipeline instead of forking an inheritance tree. HeroWidget owned video pause and play against visibility and lifecycle. The claim is maintainability and type safety on a revenue path — not an invented fill-rate percentage."

Common wrong answer: Invented metrics; "never use classes."

Follow-up ladder:

- **L1:** Type erasure when?
- **L2:** associatedtype array problem?
- **L3:** Stories SDK boundary (S10)?

Provenance: Verified · S1

Deep pool

D1. Actor reentrancy after await (120s)

Answer points (frame first):

- Await suspends; other tasks may enter actor
- No data race on isolated storage, but logic bugs
- Re-check state after await
- load-if-missing hardening

Trap: “Actors prevent all concurrency bugs.”

Full spoken answer:

“Actors prevent data races on their isolated state, but they are reentrant across await. If I read a key, await a loader, then write, another task may have entered and changed that key meanwhile. The fix is to re-validate after await — check again, use a generation token, or load outside and apply a short synchronous set. I’d mention that explicitly so we don’t confuse race-freedom with logic-correctness.”

Follow-up ladder:

- **L1:** Contrast GCD serial queue without await?
- **L2:** Single-flight patterns?
- **L3:** Nonisolated notes?

Provenance: Learning-lab · Day 05 SafeDictActor

D2. sync serial re-entry deadlock (90s)

Answer points (frame first):

- Nested sync on same serial queue deadlocks
- Not only main
- Unlocked internals / async nested work
- Design API to avoid re-entry

Trap: “Only main.sync deadlocks.”

Full spoken answer:

“Any serial queue can deadlock if you’re already executing on it and call sync again — you’re waiting for yourself. Main is the famous case, but private queues fail the same way when APIs nest. I split public sync wrappers from unlocked internals that assume they’re already on the queue, or I schedule nested work async. Debug asserts with dispatchPrecondition help.”

Follow-up ladder:

- **L1:** Sketch `_setUnlocked` .
- **L2:** Delegate callback while holding queue?
- **L3:** Actor contrast?

Provenance: Learning-lab · Day 04

D3. Memory Graph cycle vs Leaks (90s)

Answer points (frame first):

- Leaks: unreachable leaked objects
- Cycles: still reachable → may not show as Leaks
- Graph visualizes ownership
- Abandoned memory still referenced

Trap: Treating tools as synonyms.

Full spoken answer:

"Leaks finds objects with no pointers — true leaks. A retain cycle keeps objects reachable from each other, so they may never appear as classic leaks even though they won't deallocate. Memory Graph shows those cycles visually; Allocations helps with abandoned growth. At scale we treat that as reliability work alongside Crashlytics culture — without inventing a memory-only metric."

Follow-up ladder:

- **L1:** Abandoned vs leaked?
- **L2:** deinit not called checklist?
- **L3:** Autorelease pools?

Provenance: Learning-lab · Day 03; soft S8 reliability

D4. @unchecked Sendable ethics (90s)

Answer points (frame first):

- Opts out of compiler checks
- Requires documented invariant
- Prefer redesign (actor, immutable, value types)
- Code smell in reviews

Trap: Sprinkle unchecked to silence errors.

Full spoken answer:

"@unchecked Sendable tells the compiler to trust you without verifying. That's sometimes needed at boundaries with legacy classes, but it's an ethics and review issue — you must document the invariant that makes crossing threads safe, or you reintroduce races under a green build. I'd rather wrap mutable legacy state in an actor or expose immutable snapshots than normalize unchecked."

Follow-up ladder:

- **L1:** Example justified case?
- **L2:** How to test the invariant?
- **L3:** Swift 6 strictness?

Provenance: Learning-lab · concurrency hygiene

D5. Type erasure cost in renderer (90s)

Answer points (frame first):

- Allocation + indirection + lost specialization
- Use at heterogeneous boundaries
- Keep generics in hot pipeline
- S1 instinct: POP + generics first

Trap: Erase everything for cleaner types.

Full spoken answer:

"Type erasure boxes disparate conformers into one type — useful for heterogeneous ad lists — but you pay allocation, indirection, and lost generic specialization, and you often collapse associated types to a common denominator like UIView. On a

revenue pipeline I'd keep generics inside and erase only at the boundary that needs heterogeneity. That matches the Ads POP + generics approach without claiming we erased every renderer."

Follow-up ladder:

- **L1:** AnyPublisher analogy?
- **L2:** any vs hand eraser?
- **L3:** Measure in Instruments?

Provenance: Verified · S1 shape; Learning-lab erasure

D6. COW uniqueness traps (90s)

Answer points (frame first):

- Two vars may share until mutation
- Mutation may or may not copy depending on uniqueness
- Nested class references still shared
- Don't rely on reference-like sharing for value collections

Trap: Always shared or always copied.

Full spoken answer:

"After `var b = a` on an Array, they may share a buffer. Mutating `b` copies if the buffer isn't unique — `a` stays old. If something else holds a reference that breaks uniqueness, you pay a copy. Also a struct containing a class still shares that class on 'copy.' I explain COW as cheap share until write, then uniqueness decides."

Follow-up ladder:

- **L1:** Handmade COW?
- **L2:** Instruments?
- **L3:** Defensive copy anti-pattern?

Provenance: Learning-lab · Day 01

D7. GCD sync inside async contexts (90s)

Answer points (frame first):

- sync blocks a thread
- Can stall cooperative thread pool
- Prefer async bridges / actors
- Don't casually mix

Trap: "sync is fine everywhere for correctness."

Full spoken answer:

"Calling `queue.sync` from an async function blocks a thread until the queue runs the block. In the cooperative concurrency model that can starve other work. Prefer awaiting an actor or using continuation-based async wrappers that schedule with async, not sync. Correctness without blocking the async world is the goal."

Follow-up ladder:

- **L1:** `withCheckedContinuation` sketch?
- **L2:** When is sync still OK?
- **L3:** `MainActor.assumeIsolated` caveats?

Provenance: Learning-lab · Day 04/05 bridge

D8. Migrating SafeDict GCD → actor (120s)

Answer points (frame first):

- Same API: get/set/snapshot
- Facade or module-by-module
- Call sites gain await
- Dual-run / feature flag optional
- Label Applied · S2-A1

Trap: “Big-bang rewrite next sprint” or claiming already done.

Full spoken answer:

“Production synchronised dictionaries used GCD — that’s Verified. How I’d migrate without big-bang: introduce an actor with the same get/set/snapshot semantics behind a protocol, move one module at a time, and let call sites await. You can keep a GCD façade temporarily that bridges to the actor for stragglers. Validate under concurrency stress like we did originally. I wouldn’t claim the migration already shipped.”

Follow-up ladder:

- **L1:** Sync API over actor? (generally avoid)
- **L2:** Sendable values across boundary?
- **L3:** Reentrancy in load-if-missing?

Provenance: Verified · S2 → How I would apply it · S2-A1

Timed mock warm-up sets

Set	Questions
A	W1 W2 W3 W4 W6
B	W5 W7 W8 W10 W12
C	W6 W9 W11 + D1 D2

Deep set: D1 D2 D3 D8 + one stretch.

05 — Exercises — Run Mock #1

Exercise 1 — Flashcards morning (45–60 min)

Full [../../../flashcards/week-01.md](https://github.com/robertkowalski/flashcards/week-01.md) .

Second pass on misses only. Target ≥80%.

Exercise 2 — Warm-up speaking (30–40 min)

Record set A from 04-questions (W1 W2 W3 W4 W6).

Cover Full answers; speak from Answer points; compare.

Exercise 3 — Full Mock #1 (70–90 min)

Follow [02-deep-dive.md](https://github.com/robertkowalski/02-deep-dive.md) exactly.

Fill [code/MockScorecard.md](https://github.com/robertkowalski/code/MockScorecard.md) .

Pass bar: avg deep ≥3.5; S2 ≥4; provenance clean.

Exercise 4 — Weak-five pin (15 min)

Write top 5 misses → Week 2 daily warm-up list.

Exercise 5 — DSA light (30–40 min)

Re-solve 2 misses from Day 06 cold with say-this-first.

Exercise 6 — Optional S1 encore (10 min)

≤5 min Ads POP architecture talk. Preview Mock #2.

Exit criteria

- ☐ Mock #1 completed with scorecard
- ☐ S2 ≥4
- ☐ Warm-up pool answers practiced as full two-layer (not skeletons)
- ☐ Weak-five pinned
- ☐ No new topics introduced

Revision twin: [../../../revision/weeks/week-01/day-07.md](#)

Week 02

Day 08 — MVVM / Clean / MVI · Layering · DI

Week 2 · Full study (self-contained) · ~4–5 hrs
Revision twin: [revision/weeks/week-02/day-08.md](#)

Outcomes

By end of day, without notes, you can:

- Choose **MVVM vs Clean vs MVI** with a clear decision rule and one production example each
- Draw the **layer stack** (View → VM → UseCase → Repository → DataSource) and say what each layer must never own
- Defend **constructor + protocol DI** without DI-framework religion; name when factories / Environment are OK
- Deliver **S9** honestly: Free Parking + MVVM↔Clean migration; AI as **accelerator inside an architectural envelope**, not author of record
- Walk **S3** BMS search as MVVM: debounce, cancel, explicit idle/loading/results/empty/error states

How to study (in Cursor only)

- 01-foundations.md — mental model + glossary (intern → mid)
- 02-deep-dive.md — layering, DI graphs, migration strangler, MVI, search VM shape
- 03-production-bridge.md — Verified S9 + S3 + interview scripts (AI wording)
- code/ — SearchViewModel.swift , FreeParkingUseCase.swift , FeatureAssembler.swift
- 04-questions.md — cover full answers; speak from **Answer points**; compare
- 05-exercises.md — whiteboard + coding + speaking drills
- Revision twin for timed recall after the full read

Module map

Module	File
Foundations	01-foundations.md

Deep dive	02-deep-dive.md
Production bridge	03-production-bridge.md
Questions	04-questions.md
Exercises	05-exercises.md
Code	code/SearchViewModel.swift , code/FreeParkingUseCase.swift , code/FeatureAssembler.swift

Provenance reminder

Label	Meaning for today
Verified · S9	District Free Parking billing adjustments; MVVM↔Clean migration; Context Engineering; XCTest/XCUI Test assist
Verified · S3	BMS search debounce / MVVM state / cancel; backend-driven header is Day 10's SDUI beat
Learning-lab	Illustrative Search VM, UseCase, assembler snippets in this chapter

Do **not** claim “AI wrote the architecture,” invented velocity %, or that every screen was Clean-ified.

Agenda opener (say this first in an architecture HLD)

“I’ll default to MVVM for feature UI, introduce Clean UseCases where domain or migration risk demands it, and keep DI as protocol + constructor injection — then map to District Free Parking and BMS search.”

Time budget

Block	Minutes
Foundations	45–60
Deep dive + code	90–120
Production bridge + S9 STAR	30–40
Questions (record 5)	45–60
Exercises	30–40
Revision twin skim	15

Source / version note

Teaching targets **modern Swift** with `async/await` ViewModels and optional `@Observable` (iOS 17+) as the SwiftUI binding path. Clean Architecture here means **UseCase/Entity boundaries**, not a cult of folder names. Optional citations only at the end of [02-deep-dive.md](#) — this chapter is self-contained.

01 — Foundations: What Architecture Layers Actually Are

Read this before the deep dive. Goal: build a mental model an intern can repeat, then raise it to senior vocabulary.

1. Plain-English mental model

UI frameworks render pixels. **Architecture** decides *where truth lives* and *who is allowed to change it*.

Without layers:

- A button handler talks to the network, parses JSON, applies billing rules, pushes a screen, and fires analytics.
- Tests require the whole app.
- Migrations become archaeology.

With layers, each piece has **one job**:

Piece	Job in one sentence
View	Draw state; forward user intents.
ViewModel / Presenter	Hold UI state; map domain → display; orchestrate async for <i>this screen</i> .
UseCase / Interactor	Enforce product/business rules independent of UIKit/SwiftUI.
Repository	Decide cache vs network; hide DTOs.
DataSource	Talk to URLSession, DB, CMS, Keychain.

Think of a restaurant:

Kitchen metaphor	Architecture piece
Dining room	View
Waiter taking orders + describing dishes	ViewModel
Chef following recipes	UseCase
Pantry + supplier contracts	Repository
Delivery truck / fridge sensors	DataSource

If the waiter starts cooking *and* rewriting supplier contracts, service collapses under load — the mobile equivalent of a fat ViewController / fat ViewModel.

2. Glossary (learn these cold)

Term	Meaning
MVC	Model–View–Controller. In UIKit practice, Controllers often absorb networking + state → “Massive View Controller.”
MVVM	Model–View–ViewModel. View observes state; VM owns presentation logic and screen async.
Clean Architecture	Concentric dependency rule: inner domain doesn’t import UI/frameworks. UseCases + Entities at the center.
Entity	Core domain type (e.g. <code>ParkingAdjustment</code>) with rules that aren’t UI strings.
UseCase / Interactor	One application action: inputs → policy → outputs. Unit-testable without views.
Repository	Abstraction over data sources; owns merge/cache policy, not product billing rules.
DTO	Data Transfer Object — wire/decode shape. Map to domain before UseCase/VM if shapes diverge.
MVI / Unidirectional	Intent → reduce/process → immutable State → View. Side effects explicit.
Intent	User or system event (“ <code>queryChanged</code> ”, “ <code>retryTapped</code> ”).
Reducer / Processor	Pure(ish) function: (State, Intent) → State (+ effects).

DI (Dependency Injection)	Pass collaborators in (usually constructors) instead of constructing hidden globals.
Protocol boundary	Abstract dependency at a seam (SearchRepository, BillingAdjusting).
Constructor injection	init(repo: SearchRepository) — default senior choice.
Factory / Assembler	Builds the object graph for a feature/module.
Service locator	Global “give me X” registry — convenient, hides deps, hard tests. Avoid for domain.
Strangler migration	Incrementally replace old paths behind stable contracts instead of big-bang rewrite.
Context Engineering	Feeding AI tools boundaries, examples, acceptance criteria so output stays inside your architecture.
Fat ViewModel	VM that owns networking strings, routing, analytics dumps, <i>and</i> business rules.
Presentation state	What the screen shows: idle / loading / content / empty / error (often an enum).
Coordinator / Router	Owns navigation side effects so VMs stay testable.

3. Why interviewers care

A senior architecture answer proves you can:

- **Scope ceremony** — not Clean-everywhere, not MVC-forever.
- Draw **ownership** (who cancels Tasks? who owns billing rules?).
- Make features **testable** without launching the app.
- Migrate **live apps** without hero rewrites.
- Talk about **AI tooling** without sounding like you outsourced judgment.

You do **not** need TCA, RIBs, or a DI container to sound senior. Clear boundaries + honest trade-offs beat framework name-dropping.

4. Pattern map (45-second HLD)

Pattern	Core idea	UI binding	Best fit
MVC	VC mediates Model ↔ View	Imperative outlets	Tiny screens; legacy UIKit islands
MVVM	View observes VM state; VM holds presentation + screen async	Closures / Combine / Observation	Feature screens, search, forms
Clean	UseCases + entities independent of UI	VM/Presenter as adapter at the edge	Multi-team modules, migrations, heavy domain
MVI / unidirectional	Intent → State → View; effects explicit	Single state stream	Checkout, live scoreboard, multi-source fights

Interview line (memorize):

"I default to MVVM for feature UI. I introduce Clean boundaries when domain rules, reuse, or migration risk demand an explicit UseCase layer. I use unidirectional state when many async sources fight over one screen."

5. Layer shape (60-second HLD)

View (SwiftUI / UIKit)
→ ViewModel / Presenter (UI state, mapping, user intents, cancel Tasks)

- UseCase / Interactor (optional; business rules, orchestration)
- Repository (cache + remote policy, DTO ↔ domain)
- DataSource (URLSession, DB, CMS)

Dependency rule: arrows point **inward** toward policies you want stable. UI frameworks sit at the outside. Domain must not import UIKit/SwiftUI.

Speak this aloud once. Then add: "Views never see URLSession. UseCases never push view controllers. Repositories don't encode Free Parking billing policy."

6. Intern path: one happy search

1. User types in `SearchView` .
2. View calls `viewModel.onQueryChange("aveng")` .
3. ViewModel debounces ~300ms, cancels the previous `Task` , sets state to `.loading` .
4. ViewModel asks `SearchRepository.search(query:)` .
5. Repository hits remote (and maybe cache), returns domain `Movie` models.
6. ViewModel maps to `.results([...])` or `.empty` .
7. View renders.

Failure branches you must name:

What happened	Typical mapping
User keeps typing	Cancel prior Task; don't apply stale results
Offline / timeout	<code>.error</code> with retry intent
Empty hit list	<code>.empty</code> (not an error)
Task cancelled	Ignore — not a user-facing error
Decode / contract break	Error + metric; don't crash

7. Intern path: Free Parking with a UseCase

1. User confirms a billing adjustment on a Free Parking screen.
2. ViewModel forwards a typed input (`ParkingAdjustmentRequest`) — not raw form strings forever.
3. `AdjustFreeParkingBilling` UseCase validates policy (eligibility, amounts, idempotency key if needed).
4. UseCase calls `BillingRepository` to persist / sync.
5. ViewModel maps success/failure → presentation state.
6. Analytics / navigation happen via injected clients or emitted events — not buried inside the UseCase as UIKit pushes.

Why extract the UseCase? Billing rules are product truth. They should be unit-tested without spinning UI. At District, Free Parking lived next to an MVVM↔Clean migration — extract where rules hurt, leave simple chrome as MVVM.

8. DI without dogma (intern picture)

Bad:

```
final class SearchViewModel {
    func search() {
        URLSession.shared.dataTask(...) // hidden dependency
    }
}
```

Better:

```
protocol SearchRepository: Sendable {
    func search(query: String) async throws -> [Movie]
}

final class SearchViewModel {
    private let repository: SearchRepository
    init(repository: SearchRepository) { self.repository = repository }
}
```

Tests inject a fake. Production assembler wires the real repository. That is DI — no container required.

Technique	When	Cost
Constructor injection of protocols	Default for VMs / UseCases	Init boilerplate
Feature factory / assembler	Module boundaries, SPM	Need clear graph ownership
SwiftUI .environment	UI-tree plumbing (theme, shallow deps)	Easy to hide networking
Service locator / app singleton	Avoid for domain	Untestable coupling

9. AI tooling — the seniority filter

Wrong interview framing: “Cursor wrote our Clean Architecture.”

Right framing (S9):

“I used Context Engineering — boundaries, existing patterns, acceptance tests — so AI accelerated scaffolding and test drafts inside protocols I owned. Review and regressions stayed human.”

AI is a **force multiplier for typing**, not a substitute for:

- Choosing MVVM vs Clean for a screen
- Placing debounce/cancel
- Deciding what is a UseCase vs Repository
- Owning on-call fixes when billing breaks

10. Where things go wrong (preview of deep dive)

Smell	Symptom	Fix direction
Fat VM	Networking URLs + routing + rules	Extract UseCase + Router
Anemic UseCase	Pass-through to repo with no policy	Maybe you didn't need it
Business rules in Repository	“If promo then half price” next to JSON decode	Move to UseCase
Navigation in UseCase	nav.push(...)	Emit event / Coordinator
Debounce in URLSession	Timing policy leak	Debounce in VM/UseCase
AI-generated empty layers	Folders without ownership	Delete ceremony; keep seams that tests need

11. Two production anchors (preview)

District · S9 — Free Parking + migration

Shipped Free Parking billing adjustments while migrating patterns across MVVM and Clean. Used AI inside a strict envelope (reviews + XCTest/XCUITest). You own architecture.

BookMyShow · S3 — Search MVVM

Debounce search with explicit states and cancellation — race-safer UX on a high-traffic surface. Header SDUI is Day 10; today's beat is the **search VM shape**.

12. Self-check before deep dive

You are ready for 02-deep-dive.md when you can say aloud:

- 1. MVVM vs Clean in one sentence each.
- 2. What a Repository owns vs a UseCase.
- 3. Why debounce lives in the VM (or UseCase), not the transport layer.
- 4. Why “AI wrote it” is the wrong S9 line.
- 5. The five presentation states for search.

13. Flash preview (foundations only)

Front	Back
View duty	Render + forward intents
VM duty	UI state + screen async + mapping
UseCase duty	Product/business policy
Repository duty	Cache/remote + DTO map
DI default	Protocol + constructor
S9 AI line	Accelerator inside envelope
S3 search keys	Debounce + cancel + states

02 — Deep Dive: Layering, DI, Migration, MVI, Search

Senior depth. Assumes 01-foundations.md. Still self-contained — no “go read a blog first.” Optional citations at the end.

1. Ownership matrix (interview gold)

Concern	View	ViewModel	UseCase	Repository	DataSource
Layout / a11y	✓	✗	✗	✗	✗
loading/empty/error UI state	✓ renders	✓ owns	✗	✗	✗
Debounce keystrokes	✗	✓ (usual)	✓ if product rule	✗	✗
Cancel in-flight search	maybe onDisappear signal	✓	maybe	may cancel tasks	✓ task cancel
Billing eligibility rules	✗	✗	✓	✗	✗
Cache TTL / stale-while-revalidate	✗	✗	policy hint OK	✓	implements

JSON decode	✗	✗	✗	maps	often performs
UINavigationController.push	maybe UIKit	emit event	✗	✗	✗
Analytics "search_performed"	✗	often	sometimes domain events	✗	✗
URLSession config	✗	✗	✗	✗	✓

Senior one-liner: Put policy next to the reason it exists. Timing of keystrokes is presentation. Eligibility of Free Parking adjustment is domain. HTTP caching is data.

2. MVVM in practice (UIKit + SwiftUI)

2.1 State enum beats boolean soup

```
enum SearchUIState: Equatable {
    case idle
    case loading(query: String)
    case results([MovieRow])
    case empty(query: String)
    case error(message: String, query: String)
}
```

Booleans (`isLoading` , `hasError` , `isEmpty`) create impossible combinations (`isLoading && hasError`). Enums make illegal states unrepresentable — same lesson as Day 01.

2.2 UIKit binding styles

Style	Shape	Watch-outs
Closures / callbacks	<code>vm.onStateChange = { [weak self] in ... }</code>	Retain cycles; clear on deinit
Combine	<code>@Published</code> / <code>PassthroughSubject</code>	Cancel <code>AnyCancellable</code> store
Delegation	rare for continuous state	OK for one-shot events

2.3 SwiftUI binding styles

Style	When
<code>@Observable</code> VM (iOS 17+)	New feature screens — prefer
<code>@Bindable</code>	Two-way fields into observable model
<code>ObservableObject</code> + <code>@Published</code>	Legacy / pre-Observation codebases
Pure <code>@State</code>	View-local chrome only

Trap: Duplicating every toggle into the VM. Hoist `async/domain`; keep pure UI local (Day 12 expands).

2.4 Cancellation is part of MVVM

```
onQueryChange(q)
    → debounce wait
    → cancel previous Task
    → state = loading
```

```
→ await repo
→ if Task.isCancelled: return
→ apply state
```

Without cancel, slow responses overwrite newer queries → “racey search.” BMS search (S3) is the production proof: debounce + explicit states + race-safer UX.

See learning-lab: [code/SearchViewModel.swift](#).

3. Clean boundaries without folder theater

3.1 What “Clean” means here

Not “I have `Domain/Data/Presentation` folders.” It means:

1. **Entities / domain types** don’t import UIKit/SwiftUI.
2. **UseCases** express application actions and can be tested with fakes.
3. **Adapters** (VM, Repository implementations) sit at the edges.
4. Dependencies point **inward**.

3.2 When to introduce a UseCase

Introduce when **at least one** is true:

- Rules are non-trivial (eligibility, pricing, multi-step orchestration).
- Same action is shared across screens / entry points.
- You’re migrating and need a stable seam under a fat VC/VM.
- You want the cheapest regression net (unit tests on policy).

Skip when:

- Screen is a settings toggle with no domain.
- UseCase would be a pure pass-through to one repository method with no policy.
- Ceremony would exceed risk.

3.3 Free Parking shape (illustrative)

```
AdjustFreeParkingBilling
input:  ParkingAdjustmentRequest
output: ParkingAdjustmentResult
deps:  BillingRepository, Clock, (optional) IdempotencyStore
```

Steps:

1. Validate request (amounts, user eligibility)
2. Apply domain policy
3. Persist via repository
4. Return typed result

VM maps `Result` → alert / success / error UI. VM does **not** re-implement eligibility.

See [code/FreeParkingUseCase.swift](#).

3.4 Repository vs UseCase — the confusion interviewers love

	UseCase	Repository
Question it answers	“What should the product do?”	“Where do bytes/models come from/go?”
Knows UIKit?	No	No
Knows URLSession?	No	Via DataSource / client

Contains billing rules?	Yes	No
Contains cache TTL?	Rarely (may pass policy)	Yes
Orchestrates multiple repos?	Yes	Usually no

Wrong: Put “if free parking promo then waive fee” next to JSON decoding.

Wrong: Put URL building inside the UseCase.

4. MVI / unidirectional data flow

4.1 Loop

```
View --Intent--> Store/Processor --State--> View
                |
                └─ Effect (network, analytics) → Intent (result)
```

4.2 When MVI wins

- Multiple async sources mutate one screen (live scoreboard, checkout with polling + user edits).
- You need a replayable log of intents for debugging.
- Team already standardized on unidirectional patterns.

4.3 When MVI is overkill

- CRUD forms with one repository call.
- Junior-heavy team with no reducer literacy yet.
- You only needed an enum state machine inside an MVVM VM (often enough — S7 payment states are cousins).

4.4 Relation to TCA / Redux-ish frameworks

Same family. Frameworks buy structure and tooling; cost is boilerplate and onboarding. Hand-rolled unidirectional for one complex screen is valid. Don't claim “we use TCA” unless you do.

Interview line: “Unidirectional when async contention is high; MVVM+enum state when it's not.”

5. Dependency injection graphs

5.1 Feature assembler

```
FeatureAssembler
  builds: URLSessionClient → RemoteSearchDataSource → SearchRepository
         → SearchViewModel → SearchView / SearchViewController
```

Production composition root (app or feature module) owns wiring. Tests skip the assembler and inject fakes at the VM/UseCase boundary.

See [code/FeatureAssembler.swift](#).

5.2 Protocol granularity

Good protocol	Bad protocol
SearchRepository	DoesEverythingService
BillingAdjusting	IBillingManagerProtocol for a concrete with one impl forever
AnalyticsClient	Protocol per single function with no test value

Prefer protocols at **boundaries** (network, persistence, analytics, clock). Don't protocol every internal struct.

5.3 Circular dependencies

Smell: `UseCase` needs `Router` , `Router` needs `UseCase` to decide destination.

Fixes:

- Emit `NavigationEvent` from VM; Coordinator observes.
- Split protocols (`BillingAdjusting` vs `BillingPresenting`).
- Move shared policy to a third type both depend on.

5.4 SwiftUI Environment as DI

OK in Environment	Risky
Theme, locale, layout direction	Sole <code>NetworkClient</code> for all features
Feature-scoped observable store passed deliberately	Hidden service locator via custom <code>EnvironmentKey</code> soup

SDKs (Day 12 / S10) prefer **explicit init injection** so hosts see dependencies.

5.5 DI frameworks on iOS

Optional. Manual constructor DI + factories until graph pain is real. Frameworks help large graphs; cost is magic and opacity. Protocols first — always.

6. Fat ViewModel — smells and surgery

6.1 Smells

- Stringly URLs / endpoint paths in the VM
- Switch on HTTP status codes
- Business eligibility checks
- Direct `navigationController?.push`
- 800-line file “just for this screen”
- Hard to unit test without `URLProtocol` gymnastics at the VM layer

6.2 Extraction order (District-style strangler)

1. **Freeze contracts** — API shapes, analytics names, nav entry points.
2. **Characterization tests** — lock current behavior where risk is high.
3. **Extract UseCase** under the VM — VM becomes adapter; behavior unchanged.
4. **Unit test UseCase** with fakes (cheapest confidence).
5. **Extract Repository** if VM still speaks DTO/HTTP.
6. **Router/Coordinator** for cross-feature nav.
7. **Delete dead paths**; keep feature flags for rollback if needed.

Do not rewrite the whole module in one PR while shipping Free Parking. Ship value *while* migrating.

7. MVVM ↔ Clean migration playbook (S9)

Legacy fat layer

|

|— new Free Parking flow on UseCase + thin VM ✅ ship value

|— strangler: old screens call new UseCase

|— AI scaffolds tests/migrations inside envelope

|— human review: boundary checklist + on-call ownership

7.1 Context Engineering envelope (what to feed AI)

- Layering rules doc ("no UIKit in UseCase", "no URLSession in VM")
- One exemplar PR of a good extraction
- Acceptance tests / failing tests first when possible
- Naming conventions already in the module

7.2 Review checklist (human gate)

Check	Fail if
UseCase imports UIKit/SwiftUI	Boundary violation
VM builds URLRequest	Wrong layer
Missing cancellation on search-like async	Race risk
AI-generated empty protocol with one unused type	Ceremony
XCTest doesn't fail when policy inverted	Theater tests
Analytics PII logged	Privacy bug

7.3 What AI commonly gets wrong

- Puts debounce in the repository
- Creates UseCase + Interactor + Service that all pass through
- Forgets `Task` cancel on new query
- Writes UITests that sleep for timing
- "Cleans" a screen that should stay MVVM

Interview readiness: Have one concrete miss ready ("it placed billing rules in the repository; I moved them to the UseCase and added a unit test").

8. BMS search — full MVVM walkthrough (S3)

8.1 Responsibilities

Piece	Owns
Search bar View	Text, forward <code>onQueryChange</code> , render state
<code>SearchViewModel</code>	Debounce, cancel, state enum, map rows
<code>SearchRepository</code>	Remote/cached results, <code>DTO</code> → <code>Movie</code>
Ranking	Usually server ; client filters only if product requires

8.2 Timing policy

- Debounce ~300ms is a **UX/presentation** choice — lives in VM (or UseCase if product-standardized across surfaces).
- Transport may still cancel tasks; it should not know about keystroke cadence.

8.3 Out-of-order responses

Without cancel or generation tokens:

Query "a" → slow response returns later
 Query "av" → fast response returns first
 UI shows "a" results after user already typed "av" ← bug

Fix: cancel prior work **or** ignore stale generations (monotonic request ID). Prefer cancel + structured concurrency where possible.

8.4 Agenda opener for search design

"I'd put debounce and cancellation in the search ViewModel, keep ranking server-side unless product needs local filter, and model idle/loading/results/empty/error explicitly — that's how we made BMS search race-safer."

9. Navigation ownership

Scenario	Owner
Simple push after tap	VM emits <code>NavigationEvent</code> ; thin router or view handles
Cross-feature, auth gate, deeplink	Coordinator / app-edge router (S13)
Sheet vs push product choice	Product + VM intent; LE sheet (S6) is presentation choice
UseCase decides screen stack	❌ smell

Test VMs by asserting events, not by pushing real `UINavigationController` s.

10. Testing pyramid for architecture

Layer	Test type	Cost	Catch
UseCase	XCTest unit	Low	Policy regressions
VM	Unit with fake repo	Low-med	State transitions, cancel
Repository	Unit with fake client / <code>URLProtocol</code>	Med	Mapping, cache policy
UI	XCUITest / snapshots sparingly	High	Critical flows only

District (S9): AI-assisted XCTest/XCUITest generation **inside review** — tests are a gate, not a badge.

Prefer fakes over heavy mocks. Don't mock value types needlessly.

11. Architecture vs crash-free at scale

Clear ownership + fail-soft states + fewer god objects → fewer nil races and lifecycle bugs. Architecture **reduces blast radius**; it does not replace Crashlytics / IMOC (S8).

Examples of fail-soft:

- Search `.error` with retry instead of crash
- SDUI unknown component skip (Day 10 / S3-A1 design)
- Payment explicit states (S7)

At **30L+ DAU**, silent force-unwraps in fat VCs are incident fuel.

12. Trade-off tables (memorize)

12.1 Pattern choice

Choice	When	Cost
Fat MVVM only	Single team, simple screens	God VMs over time
Clean everywhere day 1	Rarely	Ceremony slows small features
MVI for every list	Overkill	Boilerplate; onboarding tax
MVVM + selective UseCases	Most production apps	Need discipline on "when"
Unidirectional for live/checkout	Multi-source contention	More moving parts

12.2 DI choice

Choice	When	Cost
Protocol + constructor	Default	Explicit wiring
Feature assembler	Modules / SPM	Ownership of graph
DI framework	Large painful graphs	Magic / opacity
Service locator	Almost never for domain	Hidden deps
Environment-only networking	Convenient demos	Untestable hosts

12.3 Migration choice

Choice	When	Cost
Big-bang rewrite	Almost never in live apps	Regression cliffs
Strangler + flags	Default	Temporary dual paths
New features on new boundaries only	Low-risk start	Legacy rot remains
Extract UseCase first	Policy risk high	Needs tests

13. Whiteboard scripts (5 minutes each)

Script A — Search MVVM

1. Draw View ↔ VM ↔ Repository.
2. Place debounce + cancel on VM.
3. Draw state enum.
4. Mention stale response race.
5. Tie to S3.

Script B — Free Parking Clean edge

1. Draw VM → UseCase → Repository.
2. Put eligibility in UseCase.
3. Show XCTest on UseCase with fake repo.
4. Tie to S9 migration + AI envelope.

Script C — “Is Clean overengineering?”

1. Scope: not every screen.
2. Criteria for UseCase.
3. Counterexample: settings toggle stays MVVM.
4. District: extract where billing rules lived.

14. Failure modes

Failure	Cause	Mitigation
Stale search results	No cancel / generation	Cancel Task; ignore stale IDs
Untestable VM	Hidden URLSession.shared	Protocol + init injection
Ghost navigation	Dual routers	Single nav owner
Empty Clean layers	AI/ceremony	Delete pass-through types
Billing bug in prod	Rules in UI	UseCase tests + on-call ownership

"AI broke prod" narrative	No review gate	Envelope + human review + tests
---------------------------	----------------	---------------------------------

15. Decision rule card (pin this)

1. Is UI mostly render + simple async? → MVVM
 2. Are there non-trivial / shared domain rules? → Add UseCase (Clean edge)
 3. Do many async sources fight one state? → Consider MVI / unidirectional
 4. Dependencies: protocol + constructor; assembler at module edge
 5. Migrate with strangler; AI accelerates inside envelope – you own design

16. Optional citations (appendix only)

You do **not** need these to study. If interviewers ask sources:

- Clean Architecture (Uncle Bob) — dependency rule concept
- Apple: Testing with XCTest; SwiftUI Data Essentials (state ownership)
- In-repo system design: `ios-system-design/docs/search-autocomplete.md` , `app-modularization.md`

17. Bridge to production chapter

Next: [03-production-bridge.md](#) — how to speak **S9** and **S3** with honest provenance, including the AI tooling trap.

03 — Production Bridge: District Free Parking & BMS Search

Convert theory into interview stories without overclaiming.

1. Provenance map for today

ID	Label	Exact claim you may make
S9	Verified	District Free Parking billing adjustments; MVVM↔Clean migration; Context Engineering (Cursor/Claude/Copilot); AI-assisted reviews + XCTest/XCUITest ; structured logging; on-call fixes
S3	Verified	BMS search debounce , explicit loading/empty/error state, MVVM binding; race-safer search UX
Learning-lab	Illustrative	<code>SearchViewModel</code> , <code>FreeParkingUseCase</code> , <code>FeatureAssembler</code> in this chapter

Forbidden overclaims (do not say as fact)

- "AI wrote our architecture / owned the migration."
- "We Clean Architecture'd the entire District app."
- Invented velocity %, regression %, or billing savings %.
- "I never needed to review AI output."
- Claiming S3 header SDUI deep mechanics as today's only story (header is Day 10; search MVVM is today).

2. Verified S9 — STAR you can deliver (2–3 min)

Timed opener (~10s)

"I'll walk through shipping Free Parking at District while migrating MVVM and Clean incrementally — with AI as an accelerator inside boundaries I owned."

Situation / Task (~20s)

We needed to ship Free Parking billing adjustments and move patterns across MVVM and Clean without a regression-heavy rewrite. Velocity mattered; so did design ownership.

Action (~90s)

- 1. Delivered the **Free Parking** feature for billing adjustments and handled **on-call production fixes** when issues surfaced.
- 2. Migrated architecture **incrementally** — extracted UseCase-style boundaries where domain rules lived; left simpler UI as MVVM rather than ceremony-everywhere.
- 3. Used **Context Engineering** with Cursor/Claude/Copilot: fed existing patterns, layering constraints, and acceptance intent so generations stayed inside an envelope.
- 4. Used AI to accelerate **reviews and XCTest/XCUITest drafts**; I still reviewed for boundary violations, races, and naming — tests were a gate, not a substitute for judgment.
- 5. Relied on **structured logging** to debug production issues quickly during on-call.

Result (~20–30s)

Feature shipped; migration progressed with velocity **and** review discipline — not “AI shipped it.”

Lesson (~15–20s)

AI accelerates once boundaries and protocols are clear. You still own architecture, tests, and regressions. Never say “AI wrote the app.”

Provenance tag

Provenance: Verified · S9 · District · Free Parking; MVVM/Clean migration; Context Engineering; XCTest/XCUITest assist

3. AI tooling — speak carefully (critical)

Phrase	Verdict
"AI wrote the Clean Architecture."	✗ Forbidden
"I used AI inside a strict architectural envelope and review loop."	✓
"Context Engineering was the control plane."	✓
"Tests generated by AI shipped without reading."	✗
"AI drafted migrations/tests; I owned boundaries and regressions."	✓

Script fragment:

"At District, AI was an accelerator — scaffolding and test drafts inside protocols and layering rules I defined. Review caught misplaced rules and missing cancellation. I’m the author of record for architecture; the tool is not."

4. Verified S3 — Search MVVM (secondary STAR / deep-dive beat)

Timed opener (~10s)

"On BookMyShow search I used MVVM with debounce, cancellation, and explicit UI states so results stayed race-safe on a high-traffic surface."

Situation / Task

Search needed modern UX — typing is bursty; naive request-per-keystroke races and wastes network.

Action

- 1. ViewModel owned **debounce** and **in-flight cancellation**.

- 2. Explicit states: loading / results / empty / error (not boolean soup).
- 3. Repository hid networking; View bound state only.
- 4. (Header SDUI was a parallel track — Day 10.)

Result

Snappier, race-safer search UX; clearer test surface for state transitions.

Lesson

Presentation policy (debounce) lives with presentation; transport cancels but doesn't know keystroke timing.

Provenance: *Verified · S3 · BookMyShow · search debounce / MVVM state*

5. Interview line (≤20s)

S9 primary:

"At District I shipped Free Parking while migrating MVVM↔Clean incrementally — AI accelerated scaffolding inside protocols I owned, with tests and review as the gate."

S3 secondary:

"BMS search sits in MVVM: debounce and cancel in the ViewModel, explicit idle/loading/results/empty/error — so out-of-order responses can't win."

6. Mapping chapter topics → verified vs learning-lab

Topic	Verified?	How to speak
Free Parking billing adjustments	Yes · S9	"I shipped Free Parking for billing adjustments."
MVVM↔Clean migration	Yes · S9	"Incremental migration across patterns."
Context Engineering / AI assist	Yes · S9	Accelerator + review — not author of record
XCTest/XCUITest assist	Yes · S9	AI-assisted generation inside review loop
On-call fixes	Yes · S9	Owned production fixes
Search debounce + MVVM states	Yes · S3	BMS search
Exact UseCase class names / folder layout	Learning-lab	Illustrative shape in chapter code
MVI/TCA adoption at District	No	Discuss as pattern judgment only
Invented % faster with AI	No	Do not invent

7. Common interviewer pushes & honest replies

Push	Weak reply	Strong reply
"Isn't Clean overengineering?"	"Always use Clean."	"Scope it — UseCases where rules/migration risk live; MVVM for simple UI. District extracted where billing lived."
"Did AI do the design?"	"Basically yes."	"I set the envelope; AI drafted; I reviewed boundaries and owned regressions."
"Where does debounce live?"	"In URLSession."	"VM/UseCase presentation policy; repo/transport cancel tasks."

"Show me DI container."	Scramble for Swinject religion	"Constructor + protocols + feature assembler; container optional until graph pain is real."
"Whole app Clean?"	"Yes."	"No — strangler; selective UseCases."
"How do you test?"	"UITests only."	"UseCase unit tests cheapest; VM with fakes; UITests for critical paths."

8. Adjacent hooks (keep them honest)

Hook	Use when	Provenance
S1 ads protocols	Testable pipelines / POP	Verified · S1
S10 Stories SDK DI	Injectable host deps	Verified · S10
S13 deeplink router	Nav ownership outside UseCase	Verified · S13
S7 payment states	Enum state machines / MVI cousins	Verified · S7
S8 crash-free	Architecture reduces blast radius, doesn't replace IMOC	Verified · S8

9. Practice: 60s vs 3 min (S9)

60s: Free Parking ship → incremental MVVM/Clean → AI inside envelope + tests → owned regressions.

3 min: Add extraction order, what AI got wrong once, why not Clean-everywhere, on-call logging, tie search only if asked (S3).

10. Link back to study artifacts

- Code: [code/SearchViewModel.swift](#), [code/FreeParkingUseCase.swift](#), [code/FeatureAssembler.swift](#)
- Questions: [04-questions.md](#)
- Revision twin: [../../revision/weeks/week-02/day-08.md](#)
- Story bank: [../../stories/story-bank.md#s9--free-parking--cleanmvvm--ai-tooling-district](#) · [S3#s3--backend-driven-header--search-bookmyshow](#)

04 — Questions (two-layer answers)

*Study flow: cover the full spoken answer → speak from **Answer points** only → uncover and compare timing.*

*Totals: **12 normal + 8 tricky = 20.***

Normal questions

Q1. Explain MVVM vs MVC. (30–45s)

Answer points (frame first):

- MVC VCs accumulate networking + state
- MVVM: View renders; VM presentation + async
- Model/domain separate
- UIKit bindings vs SwiftUI Observation

Agenda opener: "I'd contrast ownership, not acronyms..."

Full spoken answer:

"In UIKit MVC, view controllers often accumulate networking, state, and layout — Massive View Controller. MVVM keeps the View focused on rendering and forwarding intents, while the ViewModel owns presentation state and screen-level async. The

model or domain stays free of UIKit. In UIKit you bind with closures or Combine; in SwiftUI you typically observe an @Observable model. I used that split on BookMyShow search and when thinning layers during District's migration."

Common wrong answer: "MVVM means no ViewControllers."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Where does networking live?	Repository behind a protocol — not in the View.
L2	UIKit without Combine?	Closures/onStateChange with [weak self].
L3	SwiftUI kill VM?	No — hoist async/domain; keep pure UI local.

Provenance: Verified · S3 · search VM; Verified · S9 · migration

Q2. When do you introduce Clean Architecture? (45–60s)

Answer points (frame first):

- Non-trivial or shared domain rules
- Migration seam under fat VC/VM
- Entities/UseCases independent of UI
- Don't Clean-ify a settings toggle

Agenda opener: "I'd scope Clean to risk and domain..."

Full spoken answer:

"I introduce Clean boundaries when domain rules are non-trivial, shared across screens, or I'm migrating safely under a fat layer. UseCases and entities stay independent of UIKit and SwiftUI; the ViewModel becomes a thin adapter that maps results to view state and still cancels tasks. I don't Clean-ify a settings toggle — ceremony has to earn its keep. At District, Free Parking billing rules were a natural UseCase extraction while simpler chrome stayed MVVM."

Common wrong answer: "Every screen should be Clean Architecture."

Follow-up ladder:

Level	Ask	Point + short answer
L1	How thin is the VM?	Maps UseCase output → UI state; cancels work.
L2	Folders required?	No — dependency rule matters, not folder cosplay.
L3	Pass-through UseCase?	Delete it; call repository from VM.

Provenance: Verified · S9 · Free Parking / Clean edge

Q3. What is MVI / unidirectional data flow? (45s)

Answer points (frame first):

- Intent → reduce/process → State → View
- Side effects explicit
- Great for multi-source UI
- Heavier than MVVM for CRUD

Agenda opener: "I'd describe the loop and when I'd pay for it..."

Full spoken answer:

"MVI or unidirectional flow means the View sends Intents into a processor that produces a new immutable State, and the View renders that state. Side effects like networking are explicit and usually feed results back as Intents. It's excellent when checkout or a live scoreboard has many async inputs fighting one screen. For simple CRUD, MVVM with an enum state machine is usually enough — I don't pay reducer boilerplate by default."

Common wrong answer: "MVI replaces MVVM everywhere."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Relation to TCA?	Same family; discuss framework vs hand-rolled trade-off.
L2	Payment UI?	Enum states are cousins — S7 mindset.
L3	Testing?	Replay intents; assert states.

Provenance: Learning-lab · pattern judgment; cousins S7/S11

Q4. How do you do DI on iOS without a container? (45–60s)

Answer points (frame first):

- Protocol boundaries
- Constructor injection
- Feature factory/assembler
- Avoid service locator for domain
- Environment carefully

Agenda opener: "I'd start with protocols and constructors..."

Full spoken answer:

"My default is protocol boundaries with constructor injection — ViewModels and UseCases receive repositories and clients through init. A feature assembler builds the graph at the module edge. Tests inject fakes. I avoid service locators for domain dependencies. SwiftUI Environment is fine for theme and shallow UI deps, but I don't hide the NetworkClient only in Environment for an SDK-style feature."

Common wrong answer: "You must use Swinject/Needle to be senior."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Circular deps?	Split protocols; emit nav events instead of UseCase→Router cycles.
L2	SDK hosts?	Explicit injectable deps — S10 mindset.
L3	When a container?	When manual graph pain is real and team agrees.

Provenance: Verified · S9 · testable boundaries; Learning-lab · assembler

Q5. How would you structure BMS search in MVVM? (60–90s)

Answer points (frame first):

- SearchViewModel owns debounce + cancel
- State enum idle/loading/results/empty/error
- Repository hides networking
- Ranking usually server-side
- Ignore cancellation errors

Agenda opener: "I'd place debounce and cancel in the ViewModel..."

Full spoken answer:

"I'd give SearchViewModel a query handler that debounces around 300ms, cancels the previous Task, and moves through an explicit state enum — idle, loading, results, empty, error. The View only renders state and forwards text changes. A SearchRepository returns domain models; ranking stays server-side unless product needs local filter. Cancellation must not surface as a scary error. That's aligned with how we made BookMyShow search race-safer under MVVM."

Common wrong answer: "Fire a request on every keystroke and hope."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Why debounce + cancel?	Avoid out-of-order apply; save network/battery.
L2	Stale response without cancel?	Generation token / monotonic ID ignore.
L3	Where debounce must not live?	URLSession/transport-only.

Provenance: Verified · S3 · BMS search MVVM

Q6. Fat ViewModel — smells and fix? (45s)

Answer points (frame first):

- Smells: URLs, routing, business rules, analytics dumps
- Fix: UseCase + Router + AnalyticsClient
- Extract under test first

Agenda opener: "I'd name smells, then extraction order..."

Full spoken answer:

"A fat ViewModel mixes networking strings, routing, analytics, and business rules. The fix is extraction: UseCase for policy, Router/Coordinator for navigation, AnalyticsClient protocol for events. I extract the UseCase under characterization or unit tests first so behavior doesn't drift — that was the spirit of District's incremental migration."

Common wrong answer: "Just split the file into extensions."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Migration order?	UseCase first, then repo/router.
L2	Keep VM?	Yes — as adapter for UI state.
L3	AI risk?	Generates empty layers — review checklist.

Provenance: Verified · S9 · extract under migration

Q7. How do View and ViewModel communicate in UIKit vs SwiftUI? (45s)

Answer points (frame first):

- UIKit: closures/Combine/delegates
- SwiftUI: @Observable/@Bindable
- State down, events up
- Watch retain cycles in UIKit

Agenda opener: "I'd separate binding mechanism from ownership..."

Full spoken answer:

"In UIKit, ViewModels typically push state through closures, Combine publishers, or occasionally delegates for one-off events — always mind `[weak self]`. In SwiftUI, I prefer an `@Observable` model with `@Bindable` for two-way fields. Pattern is state down, events up. Observation reduces some Combine cycle footguns, but long-lived Tasks still need cancellation on disappear."

Common wrong answer: "SwiftUI doesn't use ViewModels."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Hybrid hosting?	Same VM; hosting lifecycle Day 11/S13.
L2	Legacy ObservableObject?	Know <code>@Published</code> for interviews.
L3	Cell closures?	Clear in <code>prepareForReuse</code> .

Provenance: Learning-lab · binding; hybrid S13 adjacent

Q8. How did you use AI in the District migration without losing seniority? (60–90s)

Answer points (frame first):

- Context Engineering envelope
- AI drafts migrations/tests
- Human reviews boundaries
- Never "AI wrote the app"
- Own on-call regressions

Agenda opener: "I'd separate accelerator from author of record..."

Full spoken answer:

"At District I used Context Engineering — feeding architecture constraints, existing module patterns, and acceptance intent into Cursor/Claude/Copilot. AI accelerated scaffolding, review assistance, and XCTest/XCUITest drafts. I reviewed every boundary-sensitive diff for misplaced rules, missing cancellation, and naming. Success wasn't lines generated; it was shipping Free Parking and migrating patterns without skipping design ownership. I never say AI wrote the app."

Common wrong answer: "We let Copilot own architecture."

Follow-up ladder:

Level	Ask	Point + short answer
L1	What did AI get wrong?	Be ready: rules in repository / missing cancel.
L2	Metric of success?	Regression rate + review quality, not LOC.
L3	Vs product AI?	S9 tooling ≠ FinTrack on-device AI (S15).

Provenance: Verified · S9 · Context Engineering — critical story

Q9. Repository vs UseCase — difference? (30–45s)

Answer points (frame first):

- UseCase = application policy
- Repository = data policy
- VM shouldn't know `URLSession`
- UseCase shouldn't know `UIKit`
- UseCase may orchestrate multiple repos

Agenda opener: "I'd split product policy from data policy..."

Full spoken answer:

"A UseCase answers what the product should do — adjust Free Parking billing under eligibility rules. A Repository answers where data comes from and how cache versus remote is applied, including DTO mapping. The ViewModel shouldn't know URLSession; the UseCase shouldn't know UIKit. UseCases often orchestrate multiple repositories; repositories usually shouldn't encode billing policy."

Common wrong answer: "They're the same layer with different names."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Cache TTL where?	Repository.
L2	Eligibility where?	UseCase.
L3	Can VM call repo directly?	Yes for simple screens — no fake UseCase.

Provenance: Verified · S9 · Free Parking policy placement

Q10. How do you keep feature modules testable? (45–60s)

Answer points (frame first):

- Protocols at edges
- Pure UseCases
- Deterministic fakes
- UITests sparingly
- CI gates

Agenda opener: "I'd optimize for cheap tests at policy boundaries..."

Full spoken answer:

"Protocols at the edges, pure UseCases, and deterministic fakes give a cheap unit-test core. I add snapshot or UI tests sparingly for critical flows. CI gates matter more than clever mocks. At District, AI-assisted XCTest/XCUI Test drafts were useful only inside a review loop — theater tests that never fail when policy flips are worse than no tests."

Common wrong answer: "Mock every type including value models."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Fake vs mock?	Prefer fakes with real behavior.
L2	Ads parallel?	S1 protocol pipelines aid tests.
L3	Flaky UITest?	Remove sleeps; wait on conditions.

Provenance: Verified · S9 · XCTest/XCUI Test assist

Q11. Coordinator vs Router vs NavigationPath? (45s)

Answer points (frame first):

- Coordinator/Router owns cross-feature nav
- VM emits events
- SwiftUI NavigationPath at app edge
- No UINavigationController in UseCases

Agenda opener: "I'd keep navigation out of domain..."

Full spoken answer:

"For simple pushes, the ViewModel can emit a NavigationEvent. Cross-feature flows, auth gates, and deeplinks belong in a Coordinator or Router so VMs stay testable. In SwiftUI, NavigationPath and deeplink handlers live at the app edge — Grizzlies taught deliberate ownership. UseCases must not push UIKit controllers."

Common wrong answer: "Put nav in the UseCase for Clean."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Deep links?	Parse at app boundary → intent → router (S13).
L2	Sheet vs push?	Product choice — S6 LE sheet.
L3	Test?	Assert events, don't push VCs.

Provenance: Verified · S13 adjacent; S6 presentation choice

Q12. How does architecture relate to crash-free at 30L+ DAU? (45–60s)

Answer points (frame first):

- Clear ownership reduces nil races
- Fail-soft states
- Fewer god objects
- Doesn't replace Crashlytics/IMOC
- Blast radius thinking

Agenda opener: "I'd connect structure to blast radius..."

Full spoken answer:

"Architecture doesn't magically create 99.95% crash-free sessions — observability and incident process still matter. What layers buy you is smaller blast radius: fail-soft UI states, fewer god objects, and clearer cancellation so you don't force-unwrap your way through races. At BookMyShow scale, search error states and disciplined ownership are reliability features."

Common wrong answer: "If we use Clean we won't crash."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Fail-soft example?	Search error + retry; SDUI skip unknown.
L2	S8 link?	IMOC/Crashlytics complement architecture.
L3	Fat VC risk?	Lifecycle + nil races under load.

Provenance: Verified · S8 culture + S3 states; scale context

Tricky questions

T1. Isn't Clean Architecture overengineering for mobile? (90–120s)

Answer points (frame first):

- Trap: binary yes/no
- Scope to domain + multi-team + migration risk
- District: UseCases where billing lived
- Counterexample screen you'd skip

- Measure ceremony vs regression cost

Agenda opener: "I'd refuse the binary and scope it..."

Full spoken answer:

"It can be overengineering — if you wrap every toggle in entities and four empty protocols. It's worth it when domain rules are real, multiple teams touch the module, or you're strangling a risky legacy path. At District we extracted UseCases where Free Parking billing rules lived and left simple UI as MVVM. The senior move is measuring ceremony against regression cost, not defending a folder religion."

Common wrong answer: Always Clean / never Clean.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Show a screen you wouldn't Clean-ify.	Settings toggle / static about page.
L2	How know extraction worked?	UseCase tests fail when policy inverted.
L3	Android parity?	Shared policy concepts; separate UI adapters.

Provenance: Verified · S9 · scoped Clean

T2. Where should debounce live — View, VM, or Repository? (90s)

Answer points (frame first):

- Presentation policy → VM or UseCase
- Repository shouldn't know keystrokes
- Network may still cancel
- BMS: debounce in search VM

Agenda opener: "I'd put timing with presentation policy..."

Full spoken answer:

"Debounce is a presentation policy about human typing cadence, so it belongs in the ViewModel — or in a UseCase if the product standardizes that policy across surfaces. The Repository shouldn't know keystroke timing. The networking layer may still cancel tasks, but it doesn't own the 300ms decision. On BookMyShow search, debounce plus cancel lived with the MVVM presentation layer so out-of-order responses couldn't win."

Common wrong answer: Only in UIControl or only in URLSession.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Different debounce for analytics?	Separate pipelines.
L2	View-level debounce?	Possible but harder to test; prefer VM.
L3	Server-side debounce?	Doesn't stop client races.

Provenance: Verified · S3 · search debounce placement

T3. MVVM doesn't work with SwiftUI — discuss. (90–120s)

Answer points (frame first):

- SwiftUI wants clear state ownership
- VM/@Observable still useful for async + tests

- Don't duplicate every @State
- Hoist async/domain; local for pure UI

Agenda opener: "I'd reframe as ownership, not tribalism..."

Full spoken answer:

"SwiftUI doesn't kill MVVM; it demands clearer state ownership. An @Observable ViewModel is still useful for async work, testability, and sharing screen state across child views. What fails is cloning every toggle into a VM. Prefer view-local @State for pure UI chrome and hoist async or domain state. That's compatible with Day 12's Observation guidance and with how I'd structure search or Free Parking screens."

Common wrong answer: "Delete all ViewModels in SwiftUI."

Follow-up ladder:

Level	Ask	Point + short answer
L1	@Observable vs ObservableObject?	Prefer Observation in new iOS 17+ code; know legacy.
L2	EnvironmentObject god bag?	Avoid — split models.
L3	Test?	Test observable models in XCTest.

Provenance: Learning-lab · SwiftUI MVVM; Day 12 bridge

T4. How do you migrate a live app from MVC to Clean without a rewrite? (120s)

Answer points (frame first):

- Strangler pattern
- New features on new boundaries
- Extract UseCases behind old VC
- Characterization tests + flags
- Ship value while migrating — Free Parking

Agenda opener: "I'd describe strangler, not big-bang..."

Full spoken answer:

"You don't rewrite. You strangle: put new features on new boundaries, extract UseCases behind old view controllers, add characterization tests around risky behavior, and use feature flags for rollback. At District we shipped Free Parking value while migrating MVVM and Clean incrementally. Big-bang rewrites on live consumer apps are how you buy regressions without shipping product."

Common wrong answer: Freeze the app for a quarter and rewrite.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Rollback plan?	Keep old path behind flag.
L2	What to extract first?	Highest policy risk / shared rules.
L3	AI role?	Scaffold inside envelope; human owns gate.

Provenance: Verified · S9 · incremental migration

T5. Dependency injection frameworks on iOS — yes/no? (90s)

Answer points (frame first):

- Optional
- Manual constructor DI until graph pain
- Frameworks help large graphs
- Cost: magic/opacity
- Protocols first

Agenda opener: "I'd treat frameworks as optional tooling..."

Full spoken answer:

"Yes or no is the wrong frame. Prefer manual constructor injection and feature factories until the object graph's pain is real. DI frameworks can help large multi-team graphs, but they add magic and compile/runtime opacity. Protocols at boundaries come first either way. Stories SDK-style modules should expose injectable dependencies clearly to hosts rather than hiding a container."

Common wrong answer: Framework advocacy without trade-offs.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Swinject in interviews?	Discuss trade-offs; don't require it.
L2	Testing without framework?	Init fakes directly.
L3	Circular registration?	Smell — split protocols.

Provenance: Learning-lab · DI judgment; S10 adjacent

T6. Who owns navigation — VM or Coordinator? (90s)

Answer points (frame first):

- Simple: VM emits NavigationEvent
- Cross-feature/deeplink: Coordinator
- Test via events
- No absolute rule

Agenda opener: "I'd split by complexity and testability..."

Full spoken answer:

"There's no absolute rule. For a simple push after a row tap, the ViewModel can emit a NavigationEvent and the view or a thin router performs it. Cross-feature flows, auth gates, and deeplinks need a Coordinator or app-edge router — otherwise you get dual stacks and untestable VMs. I test ViewModels by asserting events, not by pushing real controllers. Product choices like LE Bottom Sheet versus push stay presentation decisions with clear contracts."

Common wrong answer: Always VM / always Coordinator.

Follow-up ladder:

Level	Ask	Point + short answer
L1	LE sheet?	Sheet for shallow overview — S6.
L2	UseCase push?	Never.
L3	SwiftUI path?	NavigationPath owned at edge.

Provenance: Verified · S13/S6 adjacent nav judgment

T7. How do you prevent AI-generated Clean Architecture from becoming nonsense layers? (90–120s)

Answer points (frame first):

- Documented layering envelope
- Example PRs + checklist
- Human review boundaries
- Tests must fail for wrong layer
- Context Engineering = control plane

Agenda opener: "I'd describe the control plane..."

Full spoken answer:

"You prevent nonsense with an envelope: documented layering, exemplar PRs, and a checklist — no UIKit in UseCases, no URLSession in ViewModels, no empty pass-through types. AI proposes; humans review boundaries; tests must fail when policy is inverted. Context Engineering is the control plane: if you don't feed constraints, you get ceremony. At District that discipline let AI accelerate Free Parking and migration work without becoming author of record."

Common wrong answer: Ban AI / praise AI blindly.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Success metric?	Escaped defects + review cycle, not LOC.
L2	Common AI miss?	Debounce in repo; empty protocols.
L3	Junior team?	Same envelope; pair on first extractions.

Provenance: Verified · S9 · AI envelope

T8. Should ranking live in the search ViewModel? (90s)

Answer points (frame first):

- Usually server-side
- Client filter only if product requires
- VM maps display models
- Don't reimplement relevance in UI

Agenda opener: "I'd separate relevance from presentation..."

Full spoken answer:

"Relevance ranking usually belongs on the server where you have signals and can iterate without app releases. The ViewModel maps results into display models and owns UX state. Local filtering is fine for constrained product needs — like filtering an already-fetched list — but reimplementing relevance in the VM duplicates logic and drifts from Android. On a high-traffic BMS-style search, keep ranking server-side unless product explicitly needs on-device filter."

Common wrong answer: Always rank on device for "senior" control.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Offline search?	Cached index is a product decision + repo.
L2	Personalization?	Server + privacy-safe client context.
L3	A/B rankers?	Server experiments; client renders.

Provenance: Verified · S3 · search responsibilities

05 — Exercises

Solutions are in-repo. Prefer speaking aloud before peeking at solutions.

A. Conceptual

A1. Layer recall

From memory, write what each layer owns in one line: View, VM, UseCase, Repository, DataSource.

Solution:

- View: layout + forward intents; no networking/rules.
- VM: UI state, mapping, screen async, cancel.
- UseCase: product/business policy; no UIKit.
- Repository: cache/remote policy + DTO map.
- DataSource: URLSession/DB/CMS bytes.

A2. Debounce placement (two sentences)

Where does debounce live and why not in URLSession?

Solution:

Debounce is presentation policy about typing cadence, so it lives in the ViewModel (or a shared UseCase policy). URLSession may cancel tasks but must not own keystroke timing.

A3. True/False — fix the false ones

1. Every screen at District was Clean Architecture.
2. AI was author of record for Free Parking.
3. Search empty results should use the error state.
4. UseCases may orchestrate multiple repositories.
5. Service locators are the default senior DI choice.

Solution:

- 1 F — selective UseCases; MVVM remained.
- 2 F — accelerator inside envelope; you owned design.
- 3 F — empty ≠ error.
- 4 T.
- 5 F — constructor + protocols default.

A4. Provenance hygiene

Rewrite:

"Copilot designed our Clean Architecture and shipped Free Parking with 40% fewer bugs."

Solution:

"I shipped Free Parking billing adjustments at District while migrating MVVM↔Clean incrementally. I used AI inside a Context Engineering envelope — scaffolding and test drafts I reviewed — and I owned architecture and regressions. I'm not claiming a measured bug-reduction percentage."

B. Coding

B1. Read and critique

Open [code/SearchViewModel.swift](#).

1. What happens if you omit `inFlight?.cancel()` on new queries?
2. Why ignore `CancellationError` in the UI state mapping?

- Optional: write an XCTest-shaped async test that fires `onQueryChange("a")` then quickly `onQueryChange("avengers")` against a slow fake and asserts final state is for `"avengers"`.

Solution sketch:

- Stale responses can overwrite newer queries.
- Cancellation is expected control flow — not a user-facing failure.
- Fake with delay; assert `.results` or `.empty` tied to final query; ensure no `.loading("a")` terminal.

B2. Free Parking policy

Open [code/FreeParkingUseCase.swift](#).

- Why is the amount cap in the UseCase, not the ViewModel?
- Add a failing unit-test idea: eligible user + amount 0 → `invalidAmount`.
- What would make this UseCase “anemic” / deletable?

Solution:

- Cap is domain policy — must hold for any UI entry point.
- `execute` throws before repository persistence.
- If it only forwarded to `persistAdjustment` with no validation/policy.

B3. Assembler

Using [code/FeatureAssembler.swift](#), sketch how a test skips the assembler.

Solution:

`SearchViewModel(repository: FakeSearchRepository(...))` — no `NetworkClient` required.

C. Speaking drills

C1. S9 opener (20s)

Record once. Must include: Free Parking, MVVM↔Clean, AI accelerator (not author).

C2. S3 search (60–90s)

Debounce + cancel + state enum. No SDUI deep dive unless asked.

C3. T1 Clean overengineering (90–120s)

Binary refused; District scope; counterexample screen.

C4. Agenda opener for architecture HLD

Deliver the README agenda opener from memory.

D. Whiteboard (5 min)

Draw either:

- D1. Search MVVM sequence with race annotation, or
- D2. Free Parking VM → UseCase → Repository with test seam marked.

Agenda first:

“I’ll define layers, where debounce/cancel live, then map to BMS search and District migration trade-offs.”

E. Timed drill

- Pick 3 Normal + 2 Tricky (recommended: Q5, Q8, Q2 + T1, T7). Record.

- 2. Score against [answer-timing-guide.md](#).
- 3. Log misses in gotchas (AI ownership wording + debounce placement).
- 4. Deliver S9 STAR in **2–3 min** once; opener only in **20s** once.

F. Flashcard self-check

Front	Back
MVVM one-liner	View renders; VM presentation + async · Trap: net in View · Prod: S3
Clean one-liner	UseCases/entities independent of UI · Trap: everywhere · Prod: S9
MVI one-liner	Intent → State → View · Trap: CRUD boilerplate · Prod: judgment
Debounce home	VM/UseCase · Trap: URLSession · Prod: S3
DI default	Protocol + constructor · Trap: locator · Prod: tests
AI line	Accelerator + review · Trap: "AI wrote it" · Prod: S9
Fat VM fix	UseCase + Router · Trap: extensions only · Prod: S9
Repo vs UseCase	Data vs product policy · Trap: same layer · Prod: S9

Day 09 — URLSession Networking Layer

Week 2 · Full study (self-contained) · ~4–5 hrs
Revision twin: [revision/weeks/week-02/day-09.md](#)

Outcomes

By end of day, without notes, you can:

- Sketch a **protocol-oriented networking layer** on `URLSession` (endpoint → build → intercept → execute → decode → map errors)
- Explain **single-flight token refresh** under concurrent 401s (actor / critical section, one retry, failure fan-out)
- Distinguish **HTTP/ URLCache vs app cache**, and when “no cache” is required
- Cancel correctly: **async URLSession participates in Task cancellation**; callback `dataTask` needs **explicit cancel + stale guards**
- Explain **SSL pinning basics**, including that **SPKI = SHA-256 of Subject Public Key Info DER** (not raw `SecKeyCopyExternalRepresentation` bytes), and that **ATS ≠ pinning**
- Deliver the **S4** story honestly: Ads **Alamofire** → **URLSession** with **HTTPS, SSL pinning, domain whitelist** — and treat pin rotation / backup pins / break-glass as **S4-A1 design**, not a shipped runbook

How to study (in Cursor only)

- 1. `01-foundations.md` — mental model + glossary (intern → senior ramp)
- 2. `02-deep-dive.md` — layer shape, interceptors, refresh, cache, cancel, pinning internals
- 3. `03-production-bridge.md` — Verified S4 + Applied S4-A1 + interview scripts
- 4. `code/SingleFlightRefresh.swift` — actor-safe single-flight (no racy inner `Task` mutating actor state)
- 5. `04-questions.md` — cover full answers; speak from **Answer points**; compare
- 6. `05-exercises.md` — whiteboard + coding + speaking drills
- 7. Revision twin for timed recall after the full read

Module map

Module	File
--------	------

Foundations	01-foundations.md
Deep dive	02-deep-dive.md
Production bridge	03-production-bridge.md
Questions	04-questions.md
Exercises	05-exercises.md
Code	code/SingleFlightRefresh.swift

Provenance reminder

Label	Meaning for today
Verified · S4	Ads networking migrated Alamofire → URLSession; enforced HTTPS, SSL pinning, domain whitelist
How I would apply it · S4-A1	Pin rotation, backup pins, staged break-glass — design judgment , not “I shipped the ops runbook”
Verified · S3	Search debounce / cancel (cancellation UX hook)
Verified · S5	Firebase Performance p50/p90 (observability on the same stack)
Learning-lab	Illustrative client / refresh / pinning snippets in this chapter

Do **not** claim shadow-traffic rollout, remote pin kill-switches, or a production pin-rotation runbook as shipped work.

Agenda opener (say this first in a networking HLD)

“I’ll scope a first-party `URLSession` client — endpoints, interceptors, decode/errors — then single-flight refresh, cancellation, cache boundaries, and the Ads security controls: HTTPS, pinning, and host whitelist.”

Time budget

Block	Minutes
Foundations	45–60
Deep dive + code	90–120
Production bridge + S4 STAR	30–40
Questions (record 5)	45–60
Exercises	30–40
Revision twin skim	15

Source / version note

Teaching targets **modern Swift concurrency** (`async` / `await`) with `URLSession` . Pinning APIs live on `URLSessionDelegate` authentication challenges. ATS is an Info.plist / system policy layer; pinning is an **app-level** identity check on top of TLS. Optional citations only at the end of `02-deep-dive.md` — this chapter is self-contained.

01 — Foundations: What a Networking Layer Actually Is

Read this before the deep dive. Goal: build a mental model an intern can repeat, then raise it to senior vocabulary.

1. Plain-English mental model

Your app almost never “talks to the internet” directly from a button handler. It goes through a **networking layer**: a small SDK-shaped boundary that:

1. Knows **where** to call (path, method, query, body).
2. Knows **how** to authenticate (headers, tokens).
3. Sends bytes with **URLSession** (Apple’s HTTP stack).
4. Turns the response into **typed Swift models** or a **typed error**.
5. Handles messy cross-cutting concerns: **retries, cancellation, logging, token refresh**.

Think of it as a post office with rules:

Post-office piece	Networking piece
Address + postage form	APIEndpoint / request builder
Security screening	Auth interceptor, HTTPS, pinning, host whitelist
Delivery truck	URLSession task
Sorting on arrival	Status validation + JSONDecoder
“Package cancelled”	Task / URLSessionTask cancellation
“Many packages, one stamp renewal”	Single-flight token refresh

If you dump raw `URLSession.shared.dataTask` calls into every ViewModel, you get duplicated auth bugs, inconsistent errors, and no single place to pin or whitelist hosts.

2. Glossary (learn these cold)

Term	Meaning
URLSession	Apple’s networking API. Creates tasks (data/upload/download) against a configuration (timeouts, cache, cellular, TLS).
URLRequest	The outbound message: URL, method, headers, body, cache policy.
URLSessionTask	One in-flight unit of work (dataTask, uploadTask, ...).
APIEndpoint	Your app’s description of one API call (path, method, auth need, response type).
Interceptor	A pipeline step that mutates request/response (auth inject, logging, tracing) without owning business rules.
Bearer token	Access token usually sent as <code>Authorization: Bearer <token></code> .
401 Unauthorized	Server rejected credentials (expired/invalid token is the common app case).
Single-flight refresh	Many concurrent 401s share one refresh call; waiters share the result.
Idempotent	Repeating the request doesn’t create extra side effects (typical GETs; not typical payment POSTs).
Backoff + jitter	Retry delays grow (1s, 2s, 4s...) with randomness so clients don’t sync-stampede.
URLCache / HTTP cache	Transport-level cache driven by <code>Cache-Control</code> / validators.
App cache	Your memory/disk store of decoded models or offline payloads (often a Repository / Day 10 concern).

ATS (App Transport Security)	System policy preferring secure HTTPS; blocks cleartext by default. Not the same as pinning.
TLS	Encrypts the connection and authenticates the server via certificates in the trust chain.
SSL / cert pinning	Extra check: server identity must match pins you embedded (or configured). Mitigates some MITM cases TLS alone doesn't stop (compromised CA / rogue cert on device trust store).
SPKI	Subject Public Key Info — an ASN.1/DER structure describing the public key. Pinning hashes this DER , not "whatever bytes SecKeyCopyExternalRepresentation returned."
Domain whitelist	Client only allows requests to approved hosts.
Cancellation	Stop work the UI no longer needs; must not surface as a scary user error.

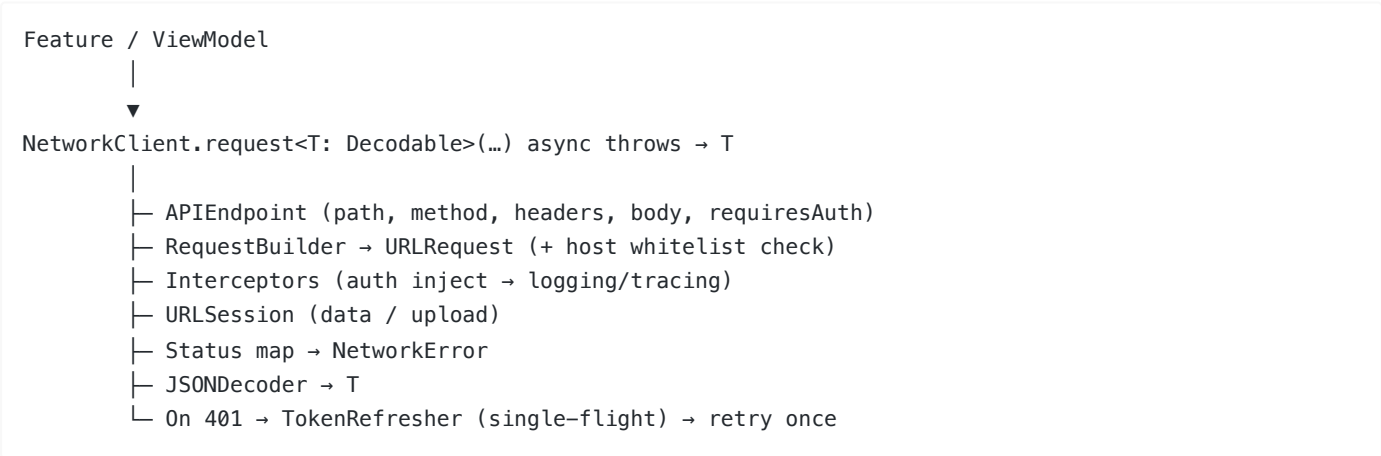
3. Why interviewers care

A senior networking answer proves you can:

- Draw a **clean boundary** (SDK vs feature vs cache vs UI).
- Reason about **concurrency** (401 storms, actor/critical section).
- Respect **security** (HTTPS baseline, pinning threat model, whitelist).
- Avoid **payment disasters** (blind POST retry).
- Cancel without **rages** (stale search results applying out of order).

You do **not** need Alamofire to sound senior. First-party `URLSession` control is often the point — especially for pinning hooks and dependency surface on a revenue module.

4. Layer shape (60-second HLD)



Speak this aloud once. Then add: "Session is injectable for tests; tokens live in Keychain; UI never sees raw JSON error strings."

5. Intern path: one happy request

1. ViewModel calls `client.request(GetProfile())` .
2. Builder turns endpoint into `URLRequest` for `https://api.example.com/v1/me` .
3. Auth interceptor reads access token from a token store, sets `Authorization` .
4. `try await session.data(for: request)` runs.
5. Status is 200; decode `Profile` with `JSONDecoder` .
6. Return `Profile` to the ViewModel; UI renders.

Failure branches you must name:

What happened	Typical mapping
---------------	-----------------

Offline / DNS / timeout	Transport error → retryable UI
401	Refresh path (or logout if refresh fails)
4xx (other)	Client error → usually don't blind-retry
5xx	Server error → maybe retry if idempotent
Decode fail	Decoding error → treat as bug / contract break
Task cancelled	Silent / ignore in UI

6. Interceptors — what they are (and aren't)

An interceptor is a **pipe fitting**, not a product feature.

Interceptor	Does	Must not
Auth	Attach Bearer / device headers	Block the main thread; log secrets
Logging	Path, method, status, latency	Log tokens, PII, full bodies in prod
Tracing	Start/stop latency spans	Become business logic
Retry	Backoff for safe methods	Blindly retry payment POSTs

Composition: ordered pipeline. Hard failures short-circuit. Keep order memorable: **auth** → **observe (log/trace)** → **send** → **map**.

7. Token refresh — intern picture

Access tokens expire. Many screens fire requests at once. If each 401 starts its own refresh:

- Auth server gets a **thundering herd**.
- Some OAuth setups **revoke** older refresh grants → cascading logout.
- Retries fight each other.

Single-flight means: first waiter starts refresh; others **await the same result**; on success, retry originals **once**; on failure, **fail everyone** and force re-auth.

Say aloud: “Refresh is a critical section; retries are bounded.”

8. Cancellation — two APIs, two stories

8.1 Async URLSession (`data(for:)` , `bytes(for:)` , ...)

These APIs **participate in Swift Task cancellation**. If the surrounding `Task` is cancelled (new search query, `onDisappear`), the await throws `CancellationError` (or fails as cancelled). Prefer this style in modern clients.

8.2 Callback `dataTask(with:completionHandler:)`

Cancellation is **not** automatic just because a ViewModel went away. You must:

1. Keep the `URLSessionTask` (or wrap it).
2. Call `task.cancel()` when the UI no longer wants the result.
3. Use a **generation / request-id stale guard** so a late completion cannot apply outdated data.

Trap: “I used async/await so I’m done thinking about cancel” — true for participation, false if you still have callback tasks, Combine wrappers, or fire-and-forget `Task { }` without a handle.

9. Caching — split the brain

Layer	Mechanism	Good for
HTTP / URLCache	Cache-Control, ETag, URLSession cache	Public, static-ish GETs
App memory (NSCache, dictionaries)	Hot decoded models	Session UX snappiness
Disk / DB	Files, SQLite, SDUI fallback	Offline / Day 10
No cache	Authed personalized, payments	Correctness

Trap: caching authenticated personalized JSON **without keying by user** → user B sees user A's data after account switch.

Networking SDK **may** expose cache policy on the request; **domain offline cache** usually belongs in Repository / feature layers — don't turn the HTTP client into a kitchen-sink disk database.

10. Security ladder (do not collapse these)

1. HTTPS only (no cleartext API traffic)
 2. ATS baseline (system policy)
 3. Correct certificate validation (default trust evaluation)
 4. Optional pinning (extra identity check) for high-threat / high-value traffic
 5. Domain whitelist (client only calls approved hosts)

ATS ≠ pinning. ATS pushes you toward TLS and blocks insecure cleartext. Pinning says “even among TLS servers, only **these** public keys / certs are acceptable for our hosts.”

You can have ATS without pinning (common). Pinning without HTTPS is nonsense. Interviewers love catching people who treat the words as synonyms.

11. SPKI in one honest paragraph

When people say “public key pinning,” they usually mean **SPKI pinning**:

1. During TLS, you obtain the server certificate.
2. You extract the certificate’s **Subject Public Key Info** as **DER-encoded bytes** (the SPKI structure).
3. You compute **SHA-256** over those DER bytes (often store as Base64).
4. You compare against your embedded pin set (primary + ideally backups).

Wrong teaching (common bug in sample code): hash the raw bytes from `SecKeyCopyExternalRepresentation` and call that “SPKI.” That API exports key material in a key-specific format; it is **not** the SPKI DER blob. Mis-labeling it will fail interoperability with standard pin generators (`openssl dgst` over SPKI) and confuse rotation docs.

Cert pinning vs SPKI/public-key pinning:

Style	Pins	Renewal behavior
Leaf certificate pinning	Exact cert	Breaks when cert is reissued (even with same key)
SPKI / public-key pinning	Hash of SPKI	Survives cert reissue only if the same key pair is reused

Pinning always needs an **ops story** (rotation, backups). That ops story for you is **S4-A1 design**, not a verified shipped runbook — see production bridge.

12. Alamofire vs URLSession (foundation take)

	Alamofire	URLSession first-party
Speed to CRUD	Excellent	More boilerplate

Async/await	Supported	Native
Pinning / challenges	Possible via session/delegate wiring	Direct URLSessionDelegate ownership
Dependency / binary surface	Extra	Smaller
Interview signal	Fine if you understand underneath	Often preferred for security-sensitive modules

At BookMyShow Ads (**Verified · S4**), the migration motivation was **ownership**: HTTPS enforcement, SSL pinning, domain whitelist on a high-traffic revenue module — not “libraries are evil.”

13. Error model you should standardize

```
enum NetworkError: Error {
    case invalidURL
    case transport(URLError)    // offline, timedOut, cancelled, ...
    case httpStatus(Int)       // or split 4xx/5xx
    case unauthorized          // 401 after refresh path exhausted / or before
    case decoding(Error)
    case cancelled             // optional explicit case
    case hostNotAllowed        // whitelist
    case pinningFailed         // TLS challenge rejected
}
```

UI maps: **retryable** vs **fatal** vs **silent** (cancel). Never dump decoder dumps into alerts.

14. Decision rules (early)

Situation	Prefer
Security-sensitive / pinning / whitelist	First-party URLSession client
Rapid internal tooling CRUD	Alamofire acceptable if team owns risks
Search-as-you-type	Cancel previous Task; ignore cancel errors
Payment POST	Idempotency keys; no generic retry helper
Public CMS config GET	HTTP cache OK with validators
Authed “my tickets”	Cache only if user-keyed; often no URLCache
Pin rotation questions	Speak design (S4-A1); don’t invent shipped ops

15. Mini self-check

Answer without scrolling up:

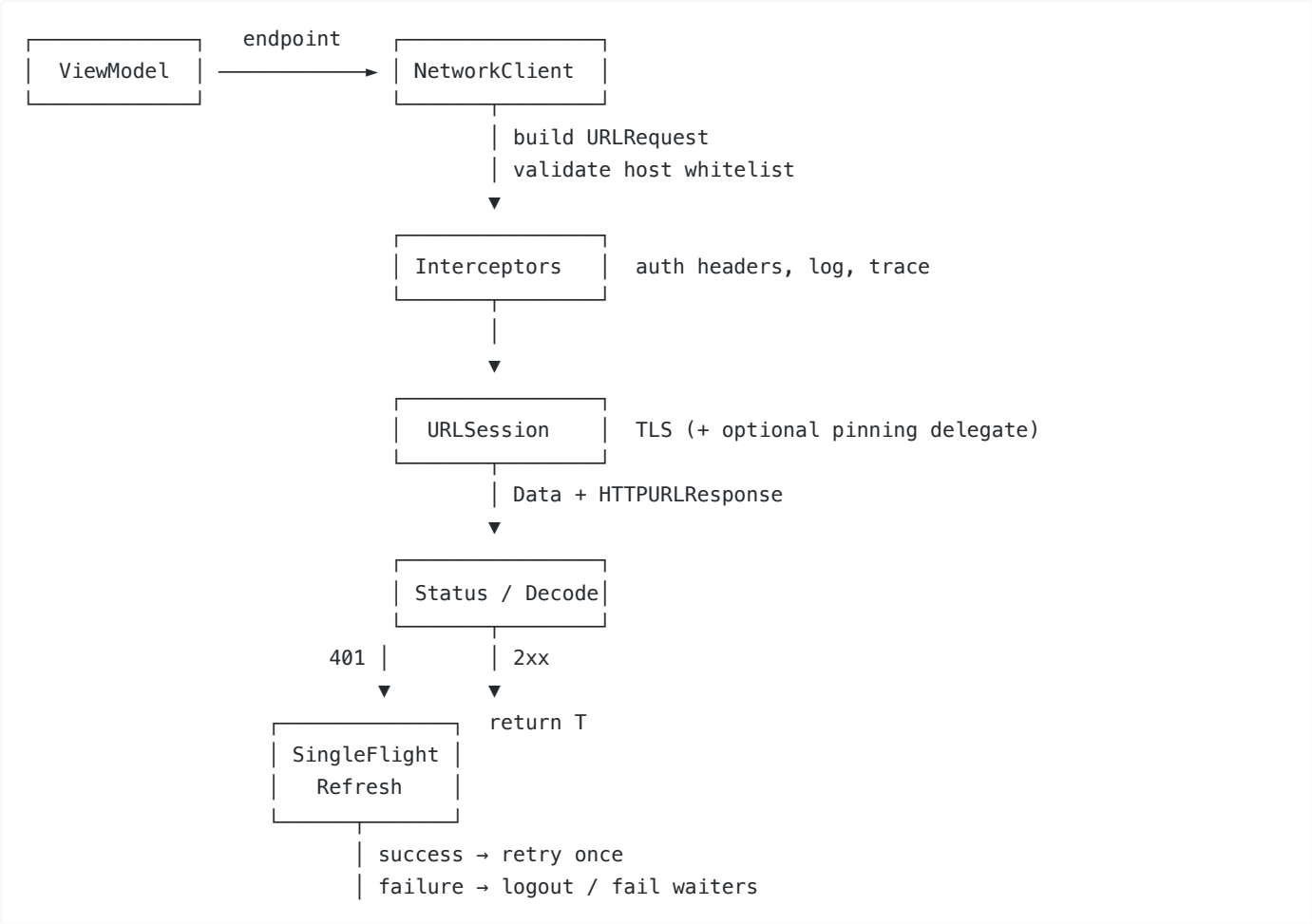
1. Name the pipeline from endpoint to decoded model.
2. What goes wrong if every 401 refreshes independently?
3. ATS vs pinning — one sentence each.
4. What is SPKI hashing, and what must you **not** hash?
5. Async `data(for:)` vs callback `dataTask` — cancellation difference?
6. What exactly is Verified in S4 — three controls?

If any answer is fuzzy, re-read that section once, then continue to `02-deep-dive.md` .

02 — Deep Dive: Mechanics, Internals, Failure Modes

Senior depth. Still self-contained — you should not need to leave Cursor to understand these mechanics.

1. End-to-end data flow



Responsibilities checklist

Piece	Owns	Does not own
APIEndpoint	Contract for one call	UI state
RequestBuilder	URL, method, body, whitelist gate	Token refresh
Interceptors	Cross-cutting mutate/observe	Feature branching
URLSession	Transport, TLS, connection pool	Domain offline DB
TokenRefresher	Single-flight refresh critical section	Payment idempotency keys
Repository	App cache / merge policy	Raw socket config

2. Protocol-oriented client (compilable shape)

```
import Foundation

enum HTTPMethod: String {
```

```

    case get = "GET"
    case post = "POST"
    case put = "PUT"
    case delete = "DELETE"
}

protocol APIEndpoint {
    associatedtype Response: Decodable
    var path: String { get }
    var method: HTTPMethod { get }
    var headers: [String: String] { get }
    var queryItems: [URLQueryItem] { get }
    var body: Data? { get }
    var requiresAuth: Bool { get }
    var cachePolicy: URLRequest.CachePolicy { get }
}

extension APIEndpoint {
    var headers: [String: String] { [:] }
    var queryItems: [URLQueryItem] { [] }
    var body: Data? { nil }
    var requiresAuth: Bool { true }
    var cachePolicy: URLRequest.CachePolicy { .useProtocolCachePolicy }
}

protocol NetworkSession: Sendable {
    func data(for request: URLRequest) async throws -> (Data, URLResponse)
}

extension URLSession: NetworkSession {}

enum NetworkError: Error {
    case invalidURL
    case hostNotAllowed(String)
    case httpStatus(Int)
    case unauthorized
    case decoding(Error)
    case transport(Error)
}

struct APIConfig: Sendable {
    var baseURL: URL
    var allowedHosts: Set<String>
}

final class NetworkClient: Sendable {
    private let session: any NetworkSession
    private let config: APIConfig
    private let tokens: TokenStore
    private let refresher: SingleFlightRefresh

    init(
        session: any NetworkSession,
        config: APIConfig,
        tokens: TokenStore,
        refresher: SingleFlightRefresh
    ) {
        self.session = session
    }
}

```

```

        self.config = config
        self.tokens = tokens
        self.refresher = refresher
    }

    func request<E: APIEndpoint>(_ endpoint: E) async throws -> E.Response {
        let built = try makeURLRequest(endpoint)
        return try await execute(endpoint, request: built, didRefresh: false)
    }

    private func execute<E: APIEndpoint>(_ endpoint: E,
        request: URLRequest,
        didRefresh: Bool
    ) async throws -> E.Response {
        var req = request
        if endpoint.requiresAuth, let token = await tokens.accessToken() {
            req.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")
        }

        let data: Data
        let response: URLResponse
        do {
            (data, response) = try await session.data(for: req)
        } catch {
            throw mapTransport(error)
        }

        guard let http = response as? HTTPURLResponse else {
            throw NetworkError.transport(URLError(.badServerResponse))
        }

        if http.statusCode == 401, endpoint.requiresAuth, !didRefresh {
            try await refresher.refresh { [tokens] in
                try await tokens.performRefresh()
            }
            return try await execute(endpoint, request: request, didRefresh: true)
        }

        guard (200..<300).contains(http.statusCode) else {
            if http.statusCode == 401 { throw NetworkError.unauthorized }
            throw NetworkError.httpStatus(http.statusCode)
        }

        do {
            return try JSONDecoder().decode(E.Response.self, from: data)
        } catch {
            throw NetworkError.decoding(error)
        }
    }

    private func makeURLRequest<E: APIEndpoint>(_ endpoint: E) throws -> URLRequest {
        guard var components = URLComponents(
            url: config.baseURL.appendingPathComponent(endpoint.path),
            resolvingAgainstBaseURL: false
        ) else { throw NetworkError.invalidURL }

        components.queryItems = endpoint.queryItems.isEmpty ? nil : endpoint.queryItems
    }

```

```

guard let url = components.url else { throw NetworkError.invalidURL }

guard let host = url.host, config.allowedHosts.contains(host) else {
    throw NetworkError.hostNotAllowed(url.host ?? "<nil>")
}

var request = URLRequest(url: url)
request.httpMethod = endpoint.method.rawValue
request.httpBody = endpoint.body
request.cachePolicy = endpoint.cachePolicy
endpoint.headers.forEach { request.setValue($1, forHTTPHeaderField: $0) }
return request
}

private func mapTransport(_ error: Error) -> NetworkError {
    if error is CancellationError { return NetworkError.transport(error) }
    return NetworkError.transport(error)
}
}

/// Minimal token store seam – Keychain in production.
actor TokenStore {
    private var access: String?
    private var refresh: String?

    func accessToken() -> String? { access }

    func install(access: String, refresh: String) {
        self.access = access
        self.refresh = refresh
    }

    func performRefresh() async throws -> String {
        // Learning-lab: pretend network refresh; production hits /oauth/token.
        guard refresh != nil else { throw NetworkError.unauthorized }
        let newAccess = "access-\(UUID().uuidString)"
        access = newAccess
        return newAccess
    }

    func clear() {
        access = nil
        refresh = nil
    }
}

```

Line-by-line intent:

- `NetworkSession` lets tests inject fakes without subclassing everything.
- Whitelist runs **before** the session sees the request.
- Auth header is applied at execute time so a post-refresh retry picks up the new token.
- `didRefresh` ensures **one** automatic retry, not an infinite 401 loop.
- Decoding failures stay typed — contract bugs shouldn't look like "no internet."

3. Interceptor pipeline (explicit)

Some teams keep interceptors as types; others inline auth. Either is fine if you can explain the order.


```

protocol RequestInterceptor: Sendable {
    func prepare(_ request: URLRequest) async throws -> URLRequest
}

protocol ResponseInterceptor: Sendable {
    func deliver(data: Data, response: HTTPURLResponse) async throws
}

struct AuthInterceptor: RequestInterceptor {
    let tokens: TokenStore
    let requiresAuth: Bool

    func prepare(_ request: URLRequest) async throws -> URLRequest {
        guard requiresAuth else { return request }
        var copy = request
        if let token = await tokens.accessToken() {
            copy.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")
        }
        return copy
    }
}

struct LoggingInterceptor: ResponseInterceptor {
    func deliver(data: Data, response: HTTPURLResponse) async throws {
        // Debug only: method/path/status/latency – never tokens or PII bodies in prod logs.
        _ = (data, response)
    }
}

```

Firebase Performance (preview): baseline path latency can live in an interceptor; product journeys (listing → checkout) often stay at call sites. Avoid double-counting. (**Verified** · **S5** for p50/p90 culture; placement is judgment — **S5-A1** if you discuss journey vs interceptor spans.)

4. Single-flight token refresh (the concurrency exam)

4.1 Failure mode without single-flight

```

t0 Request A, B, C all in flight with expired access token
t1 All three receive 401
t2 A starts refresh1, B starts refresh2, C starts refresh3
t3 refresh1 succeeds; server rotates refresh token
t4 refresh2 / refresh3 fail → mass logout / thrash

```

4.2 Desired sequence

```

t0 A, B, C get 401
t1 A enters refresher; B and C await the same in-flight work
t2 One refresh network call
t3 Success → update Keychain-backed store → A/B/C retry originals once
t4 Failure → resume all waiters with error → clear tokens → login

```

4.3 Actor safety — what went wrong in many samples

Broken pattern (do **not** ship this):

```
actor BrokenAuthManager {
    private var accessToken: String?
    private var refreshToken: Task<String, Error>?

    func refreshToken() async throws -> String {
        if let refreshToken { return try await refreshToken.value }

        let task = Task { () -> String in
            let newToken = try await executeRefreshAPI()
            // ✗ Unstructured Task is NOT actor-isolated.
            // Mutating accessToken here races the actor's isolation model.
            self.accessToken = newToken
            return newToken
        }
        refreshToken = task
        defer { refreshToken = nil }
        return try await task.value
    }

    private func executeRefreshAPI() async throws -> String { "x" }
}
```

Correct rules:

1. Actor state mutations happen **on the actor** (in actor methods), not inside a detached/unstructured `Task` body that captures `self`.
2. Prefer **continuations** or: run network work in a `Task`, then assign tokens **after** `await task.value` back on the actor.
3. Fan out success/failure to all waiters; clear in-flight state carefully around reentrancy.

See [code/SingleFlightRefresh.swift](#) for a continuation-based implementation that follows these rules.

4.4 Retry policy after refresh

Method	Auto-retry after refresh?
GET / HEAD (idempotent reads)	Usually yes, once
PUT with idempotent semantics	Maybe, if your API contract says so
POST create / payment charge	Only with server idempotency key ; otherwise don't blind-retry

Networking SDK should let endpoints **opt into** retry. Payments belong with explicit status polling (checkout UX), not a generic "retry all 401s including charge."

5. Cancellation internals

5.1 Async API participation

```
func loadResults(query: String) async throws -> [Hit] {
    let request = try makeSearchRequest(query)
    // If this function's Task is cancelled, URLSession cancels the underlying work
    // and the await fails with cancellation.
    let (data, response) = try await session.data(for: request)
    _ = response
    return try decoder.decode([Hit].self, from: data)
}

// ViewModel
```

```

private var searchTask: Task<Void, Never>?

func onQueryChange(_ q: String) {
    searchTask?.cancel()
    searchTask = Task { [weak self] in
        do {
            let hits = try await self?.loadResults(query: q)
            guard !Task.isCancelled else { return }
            self?.apply(hits ?? [])
        } catch is CancellationError {
            // Silent – expected when typing fast.
        } catch {
            self?.show(error)
        }
    }
}

```

Ties to search UX (Verified · S3): debounce in the VM; cancel stale in-flight; never show cancel as a red banner.

5.2 Callback dataTask — explicit cancel + stale guards

```

final class LegacySearchService {
    private var task: URLSessionDataTask?
    private var generation = 0

    func search(query: String, completion: @escaping (Result<[Hit], Error>) -> Void) {
        task?.cancel()
        generation += 1
        let myGeneration = generation

        let request = makeSearchRequest(query)
        let newTask = session.dataTask(with: request) { data, response, error in
            // Stale guard: a newer search started after we were cancelled/replaced.
            guard myGeneration == self.generation else { return }

            if let error = error as? URLError, error.code == .cancelled {
                return // not a user-facing failure
            }
            // ... map data/response ...
            _ = (data, response, completion)
        }
        task = newTask
        newTask.resume()
    }

    func cancel() {
        task?.cancel()
        task = nil
        generation += 1
    }
}

```

Without the generation check, a slow response can overwrite newer results (**out-of-order apply**).

5.3 Checklist

API style	Auto with Task.cancel?	You must also
-----------	------------------------	---------------

session.data(for:) etc.	Yes — participates	Hold Task handle; ignore CancellationError in UI
dataTask completion	No	task.cancel() + stale/generation guard
Fire-and-forget Task { }	Only if you cancel that Task	Don't orphan work on disappear

6. Retries and backoff

```
struct RetryPolicy: Sendable {
    var maxAttempts: Int = 3
    var baseDelayNanoseconds: UInt64 = 250_000_000 // 0.25s
    var retryableStatusCodes: Set<Int> = [408, 429, 500, 502, 503, 504]
    var retryableMethods: Set<HTTPMethod> = [.get]

    func delay(forAttempt attempt: Int) -> UInt64 {
        let exp = baseDelayNanoseconds << attempt
        let jitter = UInt64.random(in: 0..  
(baseDelayNanoseconds / 2))
        return exp + jitter
    }
}
```

Rules to say aloud:

- Retry **transient** failures on **idempotent** methods.
- Honor `Retry-After` on 429 when present.
- Cap attempts; log the final failure with path + status (no secrets).
- Never install “retry everything” as a global interceptor.

7. Caching responsibilities

7.1 URLSession / URLCache

- Driven by HTTP headers and `URLRequest.cachePolicy`.
- Good default for public content.
- Use `.reloadIgnoringLocalCacheData` for force refresh.
- Authenticated responses: prefer `Cache-Control: private` / no-store from server; client should not invent aggressive caching.

7.2 App-level cache

Store	Use
Memory	Hot screens, decoded models
Disk	Offline / SDUI fallback (Day 10)
None	Payments, highly personalized live data

Invalidation levers: TTL, pull-to-refresh, version keys, logout clears user-keyed entries.

SDK boundary: optional transport cache configuration in the client; offline domain cache in repositories — keeps the networking layer focused.

8. SSL pinning mechanics (basics)

8.1 Where it hooks

Implement `URLSessionDelegate`:

`urlSession(_:didReceive:completionHandler:)` for server trust challenges (`NSURLAuthenticationMethodServerTrust`).

Flow:

1. System presents server trust.
2. You evaluate / inspect certificates in the trust chain (often the leaf).
3. Compute pin(s); compare to allow-list.
4. `completionHandler(.useCredential, credential)` or `.cancelAuthenticationChallenge` .

8.2 SPKI correctly

SPKI = Subject Public Key Info structure inside the X.509 certificate, encoded as **DER**.

Pin value = `Base64(SHA256(spkiDER))` (common mobile convention; some docs use hex — be consistent with your pin generator).

```
Certificate (X.509)
└─ TBSCertificate
    └─ subjectPublicKeyInfo ← hash THESE DER bytes (SPKI)
        ├── algorithm
        └─ subjectPublicKey
```

Not SPKI: raw output of `SecKeyCopyExternalRepresentation` (key bytes in a format that depends on key type). Hashing that and calling it “SPKI pinning” is a common sample-code defect.

8.3 ATS vs pinning (again, with ops)

	ATS	Pinning
Layer	System / Info.plist policy	App delegate / trust evaluation
Goal	Prefer HTTPS; block cleartext	Extra identity constraint
Breaks when	Misconfigured exceptions	Pins outdated after key/cert change
Default	On for apps	Off unless you add it

Apple’s posture: pinning is **optional** and operationally costly. Use when threat model justifies it (high-value traffic, elevated MITM risk). Always design **backup pins** and rotation — as **design (S4-A1)**, not as a claim you shipped a full ops runbook unless verified.

8.4 Domain whitelist

Even with pinning, a CMS typo or malicious config could point the client at unexpected hosts. Central builder validation:

```
guard let host = url.host, allowedHosts.contains(host) else {
    throw NetworkError.hostNotAllowed(url.host ?? "")
}
```

Ads modules especially benefit: third-party creative URLs are a separate concern from **API** hosts — don’t conflate “load an image from CDN” with “POST credentials to random host.”

9. Alamofire → URLSession migration mechanics

Migration shape (teachable checklist):

1. Introduce `NetworkClient` / `NetworkSession` protocols matching call sites.
2. Implement URLSession backend behind the protocol.
3. Parity tests: status mapping, decode, error cases (fixtures / `URLProtocol`).
4. Enforce **HTTPS, SSL pinning, domain whitelist** on the Ads path.
5. Design pin rotation / backup / break-glass (**S4-A1**) so pinning isn’t a silent outage generator.

6. Roll out carefully on the Ads module — watch crash + fill / revenue-adjacent metrics you already have access to.

Do not claim “we shipped shadow traffic” unless verified. Staged rollout language is fine as **engineering judgment**; shadow traffic is not in the Verified S4 resume bullet.

10. Testing without flaky CI

Technique	Use
Protocol-wrap <code>NetworkSession</code>	Return fixture Data + <code>HTTPURLResponse</code>
<code>URLProtocol</code> subclass	Integration-ish tests through real <code>URLSession</code> config
Fake <code>TokenStore</code> / refresher	Drive 401 → refresh → retry state machine
Few smoke hits to staging	Optional; keep secrets out of fixtures

Assert: single refresh call when N parallel 401s; cancel doesn’t call UI error path; whitelist rejects bad hosts.

11. Observability

- Interceptor: method, path template, status, duration.
- Correlate with backend via `X-Request-ID` if available.
- Prefer **p50/p90** over averages (**S5**).
- Client timeouts should be coherent with backend SLOs — a 10s client timeout against a 30s server p99 creates self-inflicted errors.

12. Failure modes table

Failure	Detection	Response
Offline	<code>URLError.notConnectedToInternet</code>	UI offline; don’t spin forever
Timeout	<code>URLError.timedOut</code>	Retry if idempotent policy allows
401 storm	Many unauthorized	Single-flight refresh
Refresh 401/invalid_grant	Refresh endpoint fails	Clear tokens; force login
Pin mismatch	Challenge cancelled / custom error	Fail closed; security metric
Stale pin after rotate	Mass TLS failures	Backup pin / break-glass design
Decode break	<code>DecodingError</code>	Treat as release bug; don’t infinite retry
Cancel	<code>CancellationError</code> / <code>.cancelled</code>	Silent
Host not allowed	Whitelist	Fail before send; log config bug

13. Trade-off matrix

Choice	When	Cost
<code>URLSession</code> first-party	Security-sensitive modules (Ads)	More boilerplate
Alamofire	Fast CRUD, team fluency	Dependency + less direct control
Aggressive HTTP cache	Public content	Stale / wrong-user risk if misused
Retry all methods	Never	Duplicate charges / side effects
Pinning	High-threat / high-value	Rotation + outage risk if mis-ops

Continuations single-flight	Clear actor isolation	Slightly more code than naïve Task
Kitchen-sink network SDK	Avoid	Disk cache + image + GraphQL sprawl

14. Debugging workflow

1. Reproduce with Charles/Proxyman **only on builds that disable pinning** or use a debug trust path — never weaken prod pinning casually.
2. Log safe metadata: path, status, latency, request id.
3. For refresh bugs: count refresh network calls under parallel 401s (must be 1).
4. For cancel bugs: prove generation guard with slow stubbed responses.
5. For pin failures: confirm pin generator hashes **SPKI DER**, not raw key export bytes.

15. Ready-to-speak 90s networking HLD

"I'd build a protocol-oriented client on URLSession: endpoints describe calls, a builder produces URLRequest with a host whitelist, interceptors inject auth and safe logging, then we execute with async URLSession APIs. Responses map to typed models or typed errors. On 401, an actor runs single-flight refresh so concurrent callers share one refresh, then we retry once. Cancellation uses Task cancellation with async APIs; callback tasks get explicit cancel and stale guards. Caching splits HTTP cache from app/offline cache. For Ads at BMS we moved off Alamofire to own HTTPS, pinning, and whitelist — and I'd design backup pins and break-glass rather than treating pinning as set-and-forget."

16. Optional citations (not required to study)

- Apple: URLSession , URLSessionDelegate , App Transport Security
- Swift: structured concurrency cancellation
- OWASP MASVS / MASTG: network security, pinning guidance (threat-model based)

Continue to [03-production-bridge.md](#).

03 — Production Bridge: BMS Ads Networking & Honest Provenance

Convert theory into interview stories without overclaiming.

1. Provenance map for today

ID	Label	Exact claim you may make
S4	Verified	Ads networking migrated Alamofire → URLSession ; enforced HTTPS , SSL pinning , domain whitelisting
S4-A1	How I would apply it	Pin rotation , backup pins , staged break-glass as design — not "I shipped the ops runbook"
S3	Verified	Search debounce / state / cancel behavior (cancellation UX sibling)
S5	Verified	Firebase Performance traces; talk p50/p90 culture
S5-A1	How I would apply it	Journey-level traces vs per-request interceptor spans
S7	Verified	Payment processing popup intent — use when discussing no blind POST retry
Learning-lab	Illustrative	SingleFlightRefresh, sample client shapes in this chapter

Forbidden overclaims (do not say as fact)

- "I shipped a pin-rotation runbook / break-glass flag to production."
- "We rolled out with shadow traffic." (not in Verified S4)
- "Pinning alone caused / fixed app-wide crash-free rate."
- "ATS is our pinning."
- "We hash `SecKeyCopyExternalRepresentation` as SPKI."

2. Verified S4 — STAR you can deliver (2–3 min)

Timed opener (~10s)

"I'll walk through migrating BookMyShow Ads networking from Alamofire to URLSession so we owned transport security on a revenue-critical module."

Situation / Task (~20s)

Ads networking sat on Alamofire. For a high-traffic, revenue-adjacent module we needed stronger first-party control over transport security and a smaller dependency surface — specifically HTTPS enforcement, SSL pinning, and domain whitelisting.

Action (~90s)

1. Introduced a first-party networking path on `URLSession` behind clear call-site boundaries so Ads code didn't care about Alamofire types.
2. Enforced **HTTPS** for API traffic on that stack.
3. Added **SSL pinning** via session trust evaluation so unexpected certificates fail closed for pinned hosts.
4. Added a **domain whitelist** so the client only talks to approved hosts — reducing misconfig / unexpected host risk.
5. Treated pinning as incomplete without an ops mindset: rotation and failure behavior must be designed (see S4-A1 — design, not a claim I shipped the runbook).

Result (~20–30s)

Reduced MITM exposure on a sensitive high-traffic Ads module and simplified first-party networking ownership. Dependency surface shrank on that pod path.

Lesson (~15–20s)

Pinning without a rotation / backup / break-glass **design** is how teams create self-inflicted outages when certs or keys change. Security controls need operational paired thinking.

Provenance tag

Provenance: Verified · S4 · BookMyShow · Ads Alamofire → URLSession; HTTPS; SSL pinning; domain whitelist

3. S4-A1 — Pin ops as design (speak carefully)

When interviewers ask "what happens when the cert rotates?", answer as **design judgment**:

Design element	Intent
Backup pins	Ship ≥2 acceptable SPKI hashes so a planned key transition doesn't hard-brick the app
Pin set ownership	Know who generates pins from SPKI DER (not raw key export bytes)
Staged exposure	Prefer canary / phased release of pin changes; watch TLS failure metrics
Break-glass	A carefully scoped emergency path (build flag / remote config) — monitored, time-boxed, never "silent forever off"
Fail closed by default	MITM resistance means mismatch cancels the challenge

Script fragment:

"On Ads we shipped pinning and whitelist on URLSession. Separately, as design, I'd require backup pins and a monitored break-glass plan — pinning without rotation thinking is an outage generator. I'm not claiming I shipped that full ops runbook; I'm claiming the control set we owned and the design I'd insist on."

Provenance: *How I would apply it · S4-A1 · pin rotation / backup pins / break-glass as design*

4. Interview line (≤20s)

"I moved Ads off Alamofire onto URLSession so we owned HTTPS, SSL pinning, and a domain whitelist on a high-traffic revenue module — and I'd pair pinning with backup pins and a break-glass design so rotation doesn't brick the app."

5. Adjacent production hooks (keep them honest)

Search cancel — S3

Use when asked about cancellation UX: debounce + cancel in-flight search; ignore cancellation errors; prevent out-of-order apply.

Provenance: *Verified · S3 · search debounce / MVVM state*

Performance — S5

Use when asked how networking relates to latency culture: instrument journeys; optimize with p50/p90, not averages.

Provenance: *Verified · S5 · Firebase Performance p50/p90*

If discussing interceptor vs journey placement:

Provenance: *How I would apply it · S5-A1 · journey traces vs interceptor spans*

Payments — S7

Use when asked about retries: networking SDK must not blindly retry charge POSTs; checkout needs idempotency / status polling discipline.

Provenance: *Verified · S7 · payment processing UX intent (no invented drop-off %)*

6. Mapping chapter topics → what you built vs what you design

Topic	Verified?	How to speak
Alamofire → URLSession	Yes · S4	"We migrated Ads networking."
HTTPS enforcement	Yes · S4	"Enforced HTTPS on that stack."
SSL pinning	Yes · S4	"Added pinning for Ads hosts."
Domain whitelist	Yes · S4	"Only approved hosts."
Pin rotation runbook	No · S4-A1	"Design I'd apply / insist on."
Backup pins	No · S4-A1	Design
Break-glass disable	No · S4-A1	Design; monitored
Shadow traffic rollout	No	Do not claim
Single-flight refresh actor	Learning-lab / general architecture	"How I'd implement refresh concurrency."
SPKI DER hashing	Correctness teaching	Technical accuracy, not a BMS metric

7. Threat model for Ads (short)

Why Ads deserved first-party controls:

- High volume / revenue-adjacent traffic.
- Attractive target for injection / MITM on hostile networks.
- Third-party creative ecosystems increase “unexpected URL” risk — **API whitelist** is still valuable even if images load from CDNs under separate rules.
- Dependency ownership: fewer moving parts when security reviews ask “who controls the session delegate?”

8. Migration narrative beats (whiteboard)

1. Protocol boundary at call sites
2. URLSession implementation + parity tests
3. HTTPS only
4. Pinning delegate (SPKI hashes)
5. Host whitelist in builder
6. Design: backup pins + rotation + break-glass (S4-A1)
7. Observe TLS failures + Ads health metrics

Do not invent a sixth verified bullet like “shadow traffic.” If asked how you’d roll out: answer as judgment — phased release, metrics, canary — labeled as design.

9. Common interviewer pushes & honest replies

Push	Weak reply	Strong reply
“Why not keep Alamofire?”	“Libraries are bad.”	“Ads needed first-party pinning/whitelist ownership and less dependency surface; cost justified on that module.”
“Did pinning break you?”	“Never.”	“Pinning’s failure mode is outage; that’s why backup pins and break-glass are part of the design.”
“ATS enough?”	“Yes.”	“ATS is baseline HTTPS policy; pinning is an extra identity check for our threat model.”
“Whole app on URLSession?”	“Obviously.”	“Risk-based: start where threat/value highest — we did Ads first.”
“Show SPKI code”	Hash raw SecKey bytes	Explain SPKI DER → SHA-256; call out the common mistake

10. Practice: 60s vs 3 min versions

60s: problem (Alamofire on Ads) → action (URLSession + HTTPS + pin + whitelist) → result (first-party control / MITM ↓) → lesson (design rotation).

3 min: add parity testing, why whitelist, ATS≠pinning, SPKI one-liner, S4-A1 backup/break-glass as design, cancel/refresh only if asked.

11. Link back to study artifacts

- Code: [code/SingleFlightRefresh.swift](#)
- Questions: [04-questions.md](#)
- Revision twin: [.././revision/weeks/week-02/day-09.md](#)
- Story bank: [.././stories/story-bank.md#s4--ssl-pinning--alamofire--urlsession-bookmyshow](#)

04 — Questions (two-layer answers)

Study flow: cover the full spoken answer → speak from **Answer points** only → uncover and compare timing.

Totals: **12 normal + 8 tricky = 20.**

Normal questions

Q1. Why prefer URLSession over Alamofire in a senior design? (30–45s)

Answer points (frame first):

- First-party control of session / trust challenges
- Smaller dependency surface
- Native async/await
- Alamofire fine for speed; Ads needed security ownership
- Hook S4 without overclaiming rollout theater

Agenda opener: "I'd choose based on control and threat model, not ideology..."

Full spoken answer:

"Alamofire is fine for rapid CRUD, but on a security-sensitive module I prefer a first-party URLSession client. You own the session delegate for pinning, you reduce dependency surface, and async/await is native. At BookMyShow we moved Ads networking off Alamofire onto URLSession so we could enforce HTTPS, SSL pinning, and a domain whitelist on a high-traffic revenue module — that was about ownership, not hating libraries."

Common wrong answer: "Never use third-party networking; Alamofire is obsolete."

Follow-up ladder:

Level	Ask	Point + short answer
L1	What did you lose?	Convenience adapters/plugins — reimplement only what you need.
L2	Whole app migrate?	Risk-based; start where threat/value highest (Ads).
L3	How test parity?	Protocol session + fixtures; status/decode/error parity tests.

Provenance: Verified · S4 · Ads Alamofire → URLSession

Q2. Sketch a generic request API. (45–60s)

Answer points:

- `APIEndpoint` + `request<T: Decodable>` / associated `Response`
- Inject `NetworkSession`
- Map HTTP → domain `NetworkError`
- Auth flag on endpoint
- No UI types in the client

Agenda opener: "I'd start with an endpoint protocol and a generic async client..."

Full spoken answer:

"I define an `APIEndpoint` with path, method, headers, body, and whether auth is required, usually with an associated `Response: Decodable`. The client exposes something like `request(_ endpoint:) async throws -> Response`. A builder turns that into a `URLRequest`, interceptors attach auth, `URLSession` executes, then I validate status and decode. The session is behind a protocol so tests inject fixtures. Errors become a typed `NetworkError` — transport, HTTP, decoding, unauthorized — not raw strings for the UI."

Common wrong answer: "One giant enum of all URLs with stringly-typed callbacks."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Uploads?	Separate upload task APIs; progress as AsyncSequence or delegate.
L2	Multiple base URLs?	Config per service; endpoint or client holds base URL.
L3	Priority?	URLSessionTask.priority for UI vs analytics.

Provenance: Learning-lab · chapter client shape

Q3. How do auth interceptors work? (45s)

Answer points:

- Pre-send: read token store, attach Authorization
- On 401: refresh path
- Never log secrets
- `requiresAuth` gate for guest endpoints

Agenda opener: "Auth is a pipeline step, not ViewModel copy-paste..."

Full spoken answer:

"Before send, an auth interceptor reads the access token from a Keychain-backed store and attaches a Bearer header when the endpoint requires auth. It must not block the main thread or log tokens. On 401, control moves to the refresh coordinator rather than each call site inventing retry logic. Guest endpoints simply flip `requiresAuth` off so we don't attach or refresh unnecessarily."

Common wrong answer: "Put the token in UserDefaults and append it in every ViewModel."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Where tokens live?	Keychain; not UserDefaults.
L2	Order of interceptors?	Auth before logging/tracing send.
L3	Device attestation headers?	Same interceptor seam; keep PII-safe.

Provenance: Learning-lab · auth pipeline / Verified · S4 stack ownership mindset

Q4. Explain safe token refresh under concurrency. (60-90s)

Answer points:

- N parallel 401s → one refresh (single-flight)
- Actor or serial critical section
- Waiters share result
- Retry once; failure fan-out → logout
- No actor-state mutation inside unstructured Task

Agenda opener: "Refresh is a critical section..."

Full spoken answer:

"If ten requests get 401 at once, you must not start ten refreshes — that thundering herd can invalidate grants and log everyone out. I keep refresh behind an actor: the first caller starts the refresh network call; others await the same in-flight result via continuations or by joining one task. On success I update the token store and retry the original requests once. On

failure I fail all waiters and force re-auth. Importantly, I don't mutate actor-isolated token state inside an unstructured Task body — that races isolation. Mutations happen on the actor after the await, or I use continuations only."

Common wrong answer: "Each request refreshes then retries in a loop until success."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Refresh token reuse detection?	Server revoke → clear local → login.
L2	Non-idempotent POST after refresh?	Don't auto-retry without idempotency key.
L3	Show code smell?	Task { self.token = ... } inside actor method.

Provenance: Learning-lab · SingleFlightRefresh / architecture answer

Q5. How do you cancel in-flight search requests? (45–60s)

Answer points:

- New query cancels previous Task
- Async URLSession participates in cancellation
- Ignore CancellationError in UI
- Debounce in VM (S3)
- Callback tasks need explicit cancel + stale guards

Agenda opener: "Search cancel is about correctness, not just saving bytes..."

Full spoken answer:

"On each query change I cancel the previous Task and start a new one, usually after debounce. With async URLSession APIs, cancelling the Task cancels the underlying transfer — they participate in Swift cancellation. I treat CancellationError as silent so typing fast doesn't flash errors. If I still had callback dataTask s, I'd call cancel() myself and use a generation counter so a late response can't overwrite newer results. That pairs with the search MVVM work we did for debounce and state."

Common wrong answer: "Cancellation is automatic for every URLSession API."

Follow-up ladder:

Level	Ask	Point + short answer
L1	dataTask difference?	Needs explicit cancel + stale guard.
L2	Shared request across screens?	Repository coalescing / refcount.
L3	onDisappear?	Cancel Task handle tied to lifecycle.

Provenance: Verified · S3 · search debounce/cancel

Q6. Retry policy — what do you retry? (45–60s)

Answer points:

- Transient 408/429/5xx
- Idempotent GETs
- Exponential backoff + jitter
- Cap attempts; honor Retry-After
- No blind payment POST retry

Agenda opener: "Retry is a policy per endpoint, not a global hammer..."

Full spoken answer:

"I retry transient failures — timeouts, 408, 429, some 5xx — on idempotent methods like GET, with exponential backoff and jitter and a hard attempt cap. On 429 I honor Retry-After when present. I do not blindly retry POSTs that create charges; payments need idempotency keys and explicit status handling. The networking SDK should let endpoints opt into retry so a convenience helper can't double-charge."

Common wrong answer: "Retry all failed requests three times."

Follow-up ladder:

Level	Ask	Point + short answer
L1	400?	Usually no — client bug.
L2	Payment case?	Idempotency + poll status (S7 caution).
L3	Offline?	Fail fast to UI; don't backoff forever.

Provenance: How I would apply it · retry policy / Verified · S7 payment caution

Q7. HTTP cache vs app cache? (45–60s)**Answer points:**

- URLLCache honors headers
- App cache = decoded models / offline
- Key private data by user
- SDK vs repository boundary
- SDUI disk/TTL often Day 10

Agenda opener: "I split transport cache from domain cache..."

Full spoken answer:

"HTTP caching via URLLCache follows Cache-Control and validators — great for public, static-ish GETs. App cache holds decoded models in memory or disk for UX and offline. Authenticated personalized data must be keyed by user or not cached. I keep the networking SDK focused on transport policy; offline/domain caches usually live in repositories — for example SDUI fallback engines — so the client doesn't become a kitchen-sink database."

Common wrong answer: "Cache everything in the networking singleton for speed."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Account switch?	Clear user-keyed caches on logout.
L2	Force refresh?	reloadIgnoringLocalCacheData / TTL bust.
L3	Should SDK own disk?	Usually no — transport optional only.

Provenance: Learning-lab · cache boundaries

Q8. What is SSL pinning and what goes wrong? (60–75s)**Answer points:**

- Extra identity check beyond default TLS
- Prefer SPKI hash of Subject Public Key Info DER
- Not raw SecKeyCopyExternalRepresentation bytes
- Failure mode: outdated pins → outage

- Backup pins / rotation as design (S4-A1)
- ATS ≠ pinning

Agenda opener: "Pinning is an app-level identity constraint on top of TLS..."

Full spoken answer:

"SSL pinning means we reject TLS connections whose server identity doesn't match pins we embedded — mitigating MITM cases where trust stores or CAs aren't enough. I pin the SHA-256 of the certificate's Subject Public Key Info DER — true SPKI — not the raw bytes from SecKeyCopyExternalRepresentation, which is a common sample-code mistake. The main failure mode is operational: if keys or certs change and pins don't, the app bricks networking. ATS is only the baseline HTTPS policy; it is not pinning. On Ads we shipped pinning with URLSession; backup pins and break-glass I'd treat as required design, not set-and-forget."

Common wrong answer: "ATS pins our certificates automatically."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Cert vs SPKI pin?	Leaf cert breaks on reissue; SPKI survives if key reused.
L2	Where implemented?	URLSessionDelegate server-trust challenge.
L3	Debug HTTPS proxy?	Debug-only trust path; never weaken prod casually.

Provenance: Verified · S4 · pinning / How I would apply it · S4-A1 rotation design

Q9. Domain whitelisting — why? (30–45s)

Answer points:

- Only approved hosts
- Reduces misconfig / unexpected third-party API calls
- Enforce in central request builder
- Distinct from image CDN rules if needed

Agenda opener: "Whitelist is a client hard gate before resume..."

Full spoken answer:

"A domain whitelist means the Ads or API client refuses to send requests to hosts outside an allow-list. That cuts down SSRF-ish misconfiguration and surprising third-party calls if CMS or config goes wrong. I enforce it centrally in the request builder before the session runs. Image CDNs can be a separate policy from authenticated API hosts — don't conflate them."

Common wrong answer: "HTTPS already means any host is fine."

Follow-up ladder:

Level	Ask	Point + short answer
L1	How enforced?	<code>allowedHosts.contains(url.host).</code>
L2	Dynamic hosts?	Signed config / build-time set; careful updates.
L3	Tie to S4?	Verified control on Ads migration.

Provenance: Verified · S4 · domain whitelist

Q10. How do you test networking without flaky CI? (45–60s)

Answer points:

- Protocol-wrap session / fixtures
- URLProtocol optional
- Fake 401 refresh state machine
- Keep secrets out of fixtures
- Few integration smokes only

Agenda opener: "Unit-test the state machine; don't hit prod from CI..."

Full spoken answer:

"I wrap URLSession behind a protocol and return fixture Data and status codes in unit tests. That lets me test decoding, error mapping, whitelist rejections, and the 401 → single-flight refresh → retry-once path without networks. URLProtocol is useful for slightly higher fidelity. I keep tokens and PII out of fixtures. A tiny staging smoke suite can exist, but CI stay deterministic."

Common wrong answer: "Our tests call production APIs with retries until green."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Assert single-flight?	N parallel 401s → one refresh invocation.
L2	Cancel tests?	Expect silent cancel; no error UI path.
L3	Migration parity?	Same fixtures against old/new client.

Provenance: Learning-lab · test strategy / S4 migration discipline

Q11. Error mapping you use in production? (45s)

Answer points:

- Transport / HTTP / decoding / cancel / auth
- UI: retryable vs fatal vs silent
- No raw JSON in alerts
- Pinning / whitelist as explicit cases when useful

Agenda opener: "Typed errors at the boundary, friendly mapping in UI..."

Full spoken answer:

"I map to transport failures like offline and timeout, HTTP status errors, decoding failures, cancellation, and auth failures after refresh is exhausted. Pinning and whitelist failures should be explicit for metrics even if the UI shows a generic failure. The UI decides retryable versus fatal and never shows raw server JSON. Cancellation stays silent."

Common wrong answer: "Throw NSError and `localizedDescription` everywhere."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Partial success?	Domain-specific; don't hide in generic client.
L2	401 vs 403?	401 → refresh path; 403 → permissions UI.
L3	Telemetry?	Status + path template; no bodies.

Provenance: Learning-lab · error model

Q12. How does networking relate to p50/p90? (45s)

Answer points:

- Trace spans / journey latency
- Averages lie — watch p90
- Timeouts align with backend SLOs
- Client fixes: cache, prefetch, cancel waste

Agenda opener: "Latency culture beats anecdotal 'it feels slow!'"

Full spoken answer:

"Networking shows up in user journeys, so we instrument traces and look at p50 and p90, not just averages. Slow endpoints and fat tails drive perceived jank. Client timeouts should match backend reality. Client-side wins include caching public data, prefetching, and cancelling wasteful in-flight work. At BookMyShow we used Firebase Performance with p50/p90 on listing, checkout, and search."

Common wrong answer: "If mean latency is fine, we're done."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Interceptor vs journey?	Baseline path vs product trace (S5-A1).
L2	Double counting?	Don't nest identical spans carelessly.
L3	What not to log?	PII, tokens, full bodies.

Provenance: Verified · S5 · Firebase Performance p50/p90

Tricky questions

T1. "N requests get 401 at once — design refresh." (90–120s)

Answer points:

- Trap: refresh per request
- Single-flight actor
- One retry
- Failure → global logout
- Draw sequence
- Idempotency caveat on POST

Agenda opener: "I'll treat refresh as a single-flight critical section..."

Full spoken answer:

"The trap is starting one refresh per 401. I'd detect unauthorized once per request, then enter an actor-backed refresher: if refresh is already in flight, await the same result; otherwise perform exactly one refresh call. On success, update Keychain-backed tokens and retry each original request once with the new access token. On failure, resume every waiter with the error, clear credentials, and force login. I'd sketch the sequence for three parallel 401s. For non-idempotent POSTs, I still won't blindly retry without an idempotency key even after a successful refresh."

Common wrong answer: "Mutex around each request's full execute including decode."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Why actor?	Serialize refresh state safely with async.
L2	Task mutation bug?	Don't assign tokens inside unstructured Task.
L3	Refresh returns 401?	Fail closed → logout all waiters.

T2. "Should the networking SDK own disk cache?" (90s)

Answer points:

- Trap: kitchen-sink SDK
- Transport cache optional
- Domain/offline in Repository / SDUI
- Focus: build, auth, execute, decode, cancel

Agenda opener: "I'd keep the HTTP client narrow..."

Full spoken answer:

"Transport-level HTTP caching can live near the client via URLCache and request cache policy. Disk offline caches for domain models or SDUI documents usually belong in a repository or fallback engine. If the networking SDK owns everything — images, disk DB, GraphQL, offline sync — it becomes untestable and violates boundaries. I keep the SDK to build, auth, execute, decode, cancel, and policy hooks."

Common wrong answer: "Yes, one NetworkManager singleton with NSCache and files."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Image loading?	Separate pipeline (Kingfisher-style).
L2	Who invalidates?	Feature/repository policies + logout.
L3	Interview HLD tip?	Explicitly list out-of-scope.

Provenance: Learning-lab · boundary design

T3. "Pinning broke production after cert rotate — your fault?" (90–120s)

Answer points:

- Trap: blame-only or "pinning bad"
- Pinning without rotation design is incomplete
- Backup pins, monitoring, break-glass (S4-A1 design)
- S4 shipped pinning; ops path = design honesty
- Detect via TLS failure metrics / canary

Agenda opener: "I'd separate the control from the operational pairing..."

Full spoken answer:

"If pinning is in prod without a rotation story, that's an incomplete design — the failure mode is a total networking outage. I wouldn't conclude 'pinning is bad'; I'd conclude we needed backup pins, a planned pin update path, TLS failure monitoring, and a carefully scoped break-glass. On Ads we did migrate to URLSession with pinning and whitelist. I'm careful not to claim I shipped a full pin-rotation runbook — but in interview design I'd insist on those operational pieces before calling the security work done. Early detection is canary releases plus spiking TLS failure metrics."

Common wrong answer: "Pinning always causes outages so we removed it."

Follow-up ladder:

Level	Ask	Point + short answer
L1	SPKI vs leaf?	SPKI survives reissue if key reused.
L2	Break-glass risk?	Time-box; monitor; don't leave off.

L3	Who generates pins?	From SPKI DER via known tooling.
----	---------------------	----------------------------------

Provenance: Verified · S4 · How I would apply it · S4-A1

T4. "Alamofire already supports interceptors — why migrate?" (90s)

Answer points:

- Trap: NIH
- First-party pinning/whitelist control
- Dependency reduction
- Pod ownership consistency
- Justified on revenue module — not ideology

Agenda opener: "Migration cost needs a security/control ROI..."

Full spoken answer:

"Alamofire's interceptors weren't the blocker — ownership was. For Ads we wanted first-party URLSession control for pinning and host whitelisting, plus less third-party surface on a revenue-critical pod. We paid migration cost for that control, with parity testing around status and decode. I wouldn't migrate an entire app out of ideology; I'd start where threat and value are highest."

Common wrong answer: "We rewrote it because Alamofire can't do HTTPS."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Keep Alamofire elsewhere?	Yes, risk-based.
L2	What parity tests?	Fixtures for success/401/decode fail.
L3	Shadow traffic?	Don't claim shipped; phased rollout as design.

Provenance: Verified · S4

T5. "How do you cancel URLSession work with async/await correctly?" (90s)

Answer points:

- Trap: fire-and-forget Task
- Hold Task handle; cancel on lifecycle / new query
- Async APIs participate in Task cancellation
- Callback APIs need explicit cancel + stale guards
- Map cancel to silent UI

Agenda opener: "Cancellation starts with owning the Task handle..."

Full spoken answer:

"I keep an explicit Task handle on the ViewModel and cancel it when the query changes or the screen disappears. Async URLSession methods participate in that cancellation, so the await fails with cancellation and I ignore it in the UI. The trap is fire-and-forget tasks you can never cancel. If any code path still uses callback dataTasks, participation doesn't save you — you must cancel the URLSessionTask and guard against stale completions with a generation token."

Common wrong answer: "Async/await means URLSession always auto-cancels everything."

Follow-up ladder:

Level	Ask	Point + short answer
-------	-----	----------------------

L1	withTaskCancellationHandler?	Useful to bridge legacy cancel hooks.
L2	Multiple consumers?	Coalesce in repository.
L3	Detached Task?	Avoid for UI-bound work.

Provenance: Verified · S3 · cancel discipline / Learning-lab mechanics

T6. “Retry + payment checkout.” (90–120s)

Answer points:

- Trap: generic retry on all routes
- Idempotency keys
- Status polling / explicit states
- Endpoint opt-in retry policy
- Tie to checkout UX caution (S7)

Agenda opener: “Payments break the generic retry story...”

Full spoken answer:

“A global retry interceptor on POST charges is how you double-bill. For checkout I’d require server idempotency keys, make retry policy opt-in per endpoint, and prefer explicit payment status polling with clear UI states over silent client replays. Networking can refresh auth and retry safe GETs, but charge creation is a different contract. That mindset matches why payment UX needs careful processing-state handling rather than ‘just hit the API again.’”

Common wrong answer: “Our client retries every 5xx including /charge.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	What is idempotency key?	Client-generated key server dedupes.
L2	Refresh then POST?	Still need idempotency.
L3	User double-tap?	UI disable + same key.

Provenance: Verified · S7 · payment caution / How I would apply it · retry policy

T7. “Man-in-the-middle on public Wi-Fi — what layers help?” (90s)

Answer points:

- Trap: only “use HTTPS”
- ATS/HTTPS baseline
- Default trust evaluation
- Pinning for high-value
- Whitelist
- No cleartext fallback; careful WebViews
- User education ≠ engineering control

Agenda opener: “Defense in layers, starting with TLS baseline...”

Full spoken answer:

“HTTPS with ATS is the baseline — no cleartext APIs. Default certificate validation still matters. For high-value Ads or auth traffic, pinning adds an identity check against embedded SPKI pins, and a host whitelist prevents talking to surprise endpoints. I’d avoid cleartext exceptions and be careful with WebViews that can bypass app networking controls. Telling users ‘don’t use public Wi-Fi’ is not an engineering control. On Ads we owned HTTPS, pinning, and whitelist on the URLSession stack for that reason.”

Common wrong answer: "TLS means MITM is impossible."

Follow-up ladder:

Level	Ask	Point + short answer
L1	ATS vs pin?	Policy vs app identity pin.
L2	Compromised CA?	Pinning helps; still need ops.
L3	Charles users?	Debug builds only.

Provenance: Verified · S4 · Ads threat model

T8. "Where do you put Firebase Performance tracing — interceptor or call site?" (90s)

Answer points:

- Trap: only one right answer
- Interceptor for baseline path latency
- Call sites for product journeys
- Avoid double-counting; PII-safe names
- p50/p90 culture (S5)

Agenda opener: "Both — for different questions..."

Full spoken answer:

"I'd put lightweight per-path latency in an interceptor for baseline networking health, and keep product journey traces — listing to checkout, search submit to results — at the call site or coordinator so they match user-visible flows. Avoid double-counting the same interval under two names, and keep trace names free of PII. We instrumented Firebase Performance with p50/p90 on those journeys; exact interceptor-versus-journey split is a design choice I'd make explicit."

Common wrong answer: "Only interceptors; journeys are unnecessary."

Follow-up ladder:

Level	Ask	Point + short answer
L1	What is p90?	90% of samples faster than this.
L2	Client-only wins?	Cancel waste, cache, prefetch.
L3	Cardinality?	Template paths, not raw query strings.

Provenance: Verified · S5 · How I would apply it · S5-A1

Timed drill set (recommended)

Record: **Q4, Q8, Q5, T1, T3** — then deliver **S4 STAR** in 2–3 minutes.

Score against your answer-timing guide; log misses on refresh concurrency and SPKI/ATS distinctions.

Revision twin for skeleton recall: [../../revision/weeks/week-02/day-09.md](https://revision/weeks/week-02/day-09.md)

05 — Exercises

Solutions are in-repo. Prefer speaking aloud before peeking at solutions.

A. Conceptual

A1. Pipeline recall

Draw from memory: endpoint → ... → decoded model, including the 401 branch.

Solution:

```
APIEndpoint → RequestBuilder (+ whitelist) → Interceptors (auth/log)
→ URLSession → status/decode
→ on 401: SingleFlightRefresh → retry once → decode
→ on refresh fail: fail waiters → logout
```

A2. ATS vs pinning (two sentences)

Write two sentences that an intern couldn't confuse.

Solution:

ATS is system policy that prefers HTTPS and blocks cleartext by default. Pinning is an additional app-level check that the server's identity matches embedded pins (typically SPKI hashes) during trust evaluation.

A3. SPKI true/false

Mark each statement T/F and fix the false ones.

1. SPKI pinning hashes the Subject Public Key Info DER.
2. `SecKeyCopyExternalRepresentation` bytes are the same as SPKI DER.
3. SPKI pins always survive certificate renewal.
4. Domain whitelist replaces the need for HTTPS.

Solution:

- 1 T.
- 2 F — raw key export ≠ SPKI DER; don't call that SPKI.
- 3 F — survives renewal **only if the same key pair is reused**.
- 4 F — whitelist constrains hosts; HTTPS/TLS still required.

A4. Provenance hygiene

Rewrite this bad interview sentence so it's honest:

"I shipped Alamofire to URLSession with pinning, whitelist, shadow traffic, and a production break-glass runbook."

Solution:

"I migrated Ads networking from Alamofire to URLSession and enforced HTTPS, SSL pinning, and a domain whitelist. Pin rotation, backup pins, and break-glass I'd treat as required design alongside pinning — I'm not claiming I shipped a shadow-traffic rollout or the full ops runbook."

B. Coding

B1. Read and critique

Open [code/SingleFlightRefresh.swift](#).

1. Explain why the continuation design avoids the racy `Task { self.token = ... }` bug.
2. What goes wrong if you set `isRefreshing = false` **after** resuming waiters?
3. Optional: implement a unit-test-shaped async function that starts three concurrent `refresh` calls and asserts a single leader execution using an `OSAllocatedUnfairLock / actor` counter inside `performRefresh`.

Solution sketch:

1. Waiters never mutate actor state from a non-isolated Task; only the actor method writes `accessToken` / waiter lists.
2. A resumed waiter that immediately calls `refresh` again may see `isRefreshing == true` and append to an already-draining waiter list — hanging or lost wakeups. Clear flag before resume.

3. Counter inside `performRefresh` should end at 1 across three concurrent callers (leader-only execution).

B2. Broken refresh — fix it

Fix this (conceptually or in a playground):

```
actor Broken {
  var token: String?
  var task: Task<String, Error>?
  func refresh() async throws -> String {
    if let task { return try await task.value }
    let task = Task {
      let t = try await network()
      self.token = t // bug
      return t
    }
    self.task = task
    defer { self.task = nil }
    return try await task.value
  }
  func network() async throws -> String { "ok" }
}
```

Solution direction: Move `token = t` to after `await task.value` on the actor, or switch to the continuation-based `SingleFlightRefresh`. Prefer not mutating actor state inside the unstructured Task.

B3. Callback cancel + stale guard

Write a minimal class that:

- Starts a `URLSessionDataTask` for search
- Cancels previous task on new query
- Ignores late completions via a generation `Int`

Solution pattern: see Deep Dive §5.2 — `task?.cancel(); generation += 1; capture myGeneration; guard myGeneration == generation`.

B4. Whitelist gate

Given `allowedHosts = ["api.bms.example"]`, show the guard that throws before `session.data`.

Solution:

```
guard let host = url.host, allowedHosts.contains(host) else {
  throw NetworkError.hostNotAllowed(url.host ?? "")
}
```

C. Debugging scenarios

C1. Users see old search results after typing fast

Likely causes: no cancel; callback task without stale guard; applying results without `Task.isCancelled` check.

Fix: cancel previous Task / dataTask; generation guard; ignore cancel errors.

C2. Mass logout at 8am traffic peak

Likely causes: refresh storm on expired access tokens; refresh token rotation invalidating siblings.

Fix: single-flight refresh; one retry; verify only one refresh call under load tests.

C3. "Pinning mismatch" after routine cert renew

Likely causes: leaf pin instead of SPKI; SPKI pin but **new key**; pins generated from wrong bytes (SecKeyCopyExternalRepresentation).

Fix: confirm pin pipeline hashes SPKI DER; use backup pins; design rotation (S4-A1).

C4. CI networking tests flake

Likely causes: live network; shared mutable URLRequest state across parallel tests.

Fix: inject NetworkSession fakes; isolate protocol mocks per test.

D. Speaking drills

D1. 60s Ads migration

Record once. Must include: Alamofire → URLSession, HTTPS, pinning, whitelist. Must **not** invent shadow traffic or shipped runbook.

D2. 90s refresh design

Whiteboard three parallel 401s; say "critical section" and "retry once."

D3. 45s cancellation distinction

One breath on async participation vs callback explicit cancel + stale guards.

D4. Full agenda opener

Deliver the README networking HLD opener cold, then expand 3 minutes covering refresh, cache split, S4, S4-A1.

Opener:

"I'll scope a first-party URLSession client — endpoints, interceptors, decode/errors — then single-flight refresh, cancellation, cache boundaries, and the Ads security controls: HTTPS, pinning, and host whitelist."

E. Whiteboard prompts

- 1. Sequence diagram: three 401s → one refresh → three retries → one failure path to logout.
- 2. Table: ATS | TLS defaults | Pinning | Whitelist — layer and failure mode each.
- 3. List Ads migration steps (protocol → URLSession → HTTPS → pin → whitelist → **design** rotation).
- 4. Mark each step Verified S4 vs S4-A1.

F. Self-score rubric

Skill	0	1	2
Layer HLD	Missing pieces	Correct happy path	Happy + 401 + cancel + cache boundary
Refresh	Per-request refresh	Single-flight named	Actor-safe + retry once + fan-out
Cancellation	Vague	Async cancel only	Async + callback stale guards
Pinning	ATS confusion	Pinning named	SPKI DER correct + outage mode
Provenance	Overclaims	S4 accurate	S4 + explicit S4-A1 design language

Target: ≥ 8/10 before marking Day 09 complete.

G. After exercises

- 1. Skim revision twin: [.././revision/weeks/week-02/day-09.md](#)

- 2. Log gotchas: refresh concurrency, SPKI vs raw key bytes, ATS ≠ pinning, no shadow-traffic claim.
- 3. Next day builds on persistence / offline — keep cache boundary language ready for Day 10.

Day 10 — SDUI / CMS · Schema Versioning · Fallbacks

Week 2 · Full study (self-contained) · ~4–5 hrs
Revision twin: [revision/weeks/week-02/day-10.md](#)

Outcomes

By end of day, without notes, you can:

- Explain **server-driven UI**: schema → version gate → registry → native render + allowlisted actions — **not** “eval JSON as code”
- Defend **schema versioning**, **unknown-component skip**, and **fallback/cache** as crash-free insurance
- Deliver **S3** BMS backend-driven header honestly; treat **S3-A1** versioning/unknown fallback as **design**
- Deliver **S12** Aces server-driven splash (cold-start flexibility / freshness — no invented ms)
- Contrast SDUI vs **native Ads** media lifecycle (S1) without overclaiming

How to study (in Cursor only)

1. 01-foundations.md — mental model + glossary
2. 02-deep-dive.md — schema, registry, actions, cache, cold start, parity
3. 03-production-bridge.md — Verified S3 + S12; Applied S3-A1
4. code/ — SchemaVersionGate.swift , ComponentRegistry.swift , FallbackEngine.swift
5. 04-questions.md — cover full answers; speak from **Answer points**
6. 05-exercises.md — drills
7. Revision twin for timed recall

Module map

Module	File
Foundations	01-foundations.md
Deep dive	02-deep-dive.md
Production bridge	03-production-bridge.md
Questions	04-questions.md
Exercises	05-exercises.md
Code	code/SchemaVersionGate.swift , code/ComponentRegistry.swift , code/FallbackEngine.swift

Provenance reminder

Label	Meaning for today
Verified · S3	Backend-driven / CMS header; protocol-driven main-screen; API contracts for layout/content without release when possible; search is Day 08’s MVVM beat
How I would apply it · S3-A1	Schema versioning + unknown-component fallback — design judgment , not “I shipped the versioning ops runbook as a named system”
Verified · S12	Aces server-driven splash (+ audio streaming context); cold-start flexibility/freshness — no invented latency ms

Learning-lab	Gate/registry/fallback snippets in this chapter
--------------	---

Do **not** claim JS eval SDUI, invented cold-start milliseconds, or that S3-A1 was a shipped named framework.

Agenda opener

"I'll define SDUI as schema plus native registry with version gates and fail-soft unknowns, then map to the BMS header and Aces splash — and call out schema versioning as design I'd insist on."

Time budget

Block	Minutes
Foundations	45–60
Deep dive + code	90–120
Production bridge + STAR	30–40
Questions (record 5)	45–60
Exercises	30–40
Revision twin skim	15

Source / version note

SDUI here means **typed JSON** → **native components**. WebViews may appear as an island, not the architecture. Optional citations only at end of 02-deep-dive.md .

01 — Foundations: What SDUI Actually Is

Intern → mid mental model. Read before the deep dive.

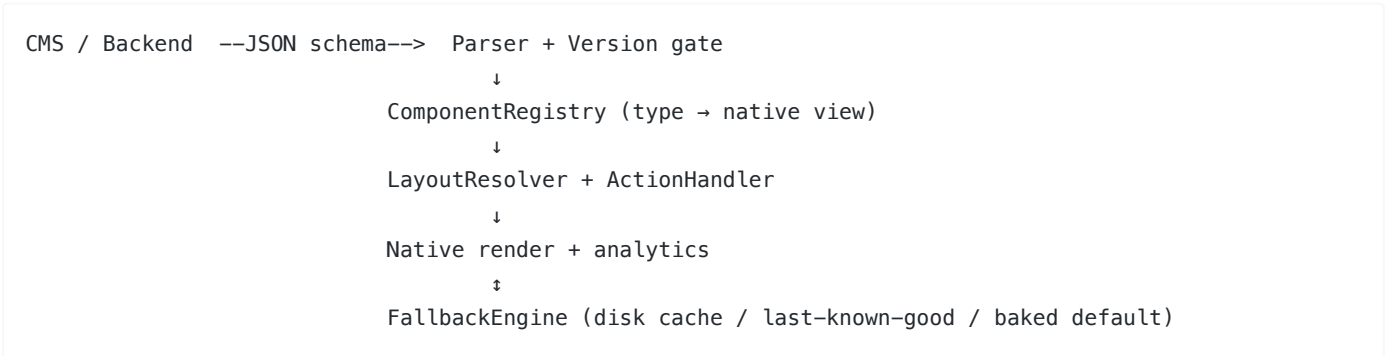
1. Plain-English mental model

Server-driven UI (SDUI) means the backend (or CMS) sends a **structured description** of what to show — types, props, children, actions — and the app maps those types to **native** SwiftUI/UIKit components.

It is **not**:

- Evaluating JavaScript from the CMS
- "Just a WebView for the whole app"
- Shipping arbitrary code in JSON

Think of a set of LEGO instructions (schema) and a box of official bricks (native registry). Unknown brick types are **skipped**, not force-fit into explosions (crashes).



2. Glossary

Term	Meaning
Schema	Contract describing nodes: type, props, children, actions, optional schemaVersion.
schemaVersion	Compatibility number; major mismatch → reject or hard fallback.
ComponentRegistry	Map from server type string → native factory/view.
Unknown component	Type not in registry — skip + metric , never crash.
Capability flags	Client advertises supported types / features to server.
Additive change	New optional fields; old clients ignore unknowns.
Breaking change	Requires major bump + dual-publish period.
Action allowlist	Only known action.type values execute (deeplink, refresh, openURL...).
FallbackEngine	Last-known-good cache, baked default, or partial tree render.
Last-known-good	Previously validated payload on disk for offline / fail.
Hybrid SDUI	CMS slots inside native chrome (most real apps).
Cold start splash	First paint / branded startup surface — product-critical (S12).
Time-to-interactive	When user can act — more honest than first-frame vanity.
Canary payload	% rollout of new schema to watch metrics before 100%.
Contract test	Fixture schemas asserted against registry per version.

3. Why interviewers care

SDUI answers prove you can:

- Ship content velocity **without** sacrificing crash-free
- Design **compatibility** as a product feature
- Secure **actions** (no arbitrary code)
- Reason about **cold start** and cache freshness
- Stay honest about **Verified vs Applied** (S3 vs S3-A1)

At **30L+ DAU** and **99.95% crash-free** culture, SDUI without fallbacks is an incident generator.

4. Wins and costs

Wins: experiment without App Store; personalize surfaces; share contracts across iOS/Android; marketers/CMS iterate layout/content.

Costs: schema discipline; capability matrix; QA of combinations; offline/fallback; action security; new component types still need an app release.

5. Intern path: happy header render

1. App launches; ViewModel asks repository for header payload.
2. Repository returns network JSON or cached last-known-good.
3. Version gate checks `schemaVersion` against client max-supported.
4. Parser builds a tree of nodes.
5. Registry maps `logo`, `promoBanner`, `searchEntry` → native views.
6. Unknown `type: "sparkle_v9"` → skip + log; siblings still render.

7. User taps CTA → allowlisted `open_deepLink` action → router.

6. Intern path: sad paths you must name

Failure	Client behavior
Offline	Show last-known-good if fresh enough; else baked default
Major version too new	Hard fallback layout; metric <code>schema_reject</code>
Unknown node mid-tree	Skip node; continue
Empty root after skips	Hard fallback (never blank chrome)
Action type unknown	No-op + metric; no crash
Parse error	Fallback + metric

7. Schema versioning — 60s picture

Strategy	Behavior
Major / minor	Major mismatch → reject or safe fallback
Capability flags	Server avoids sending unsupported types when possible
Unknown fields	Ignore (forward compatible)
Unknown types	Skip + metric
Breaking change	New major + dual-publish old+new

Interview line: “Compatibility is a product feature. Unknown nodes fail soft; known nodes validate strictly.”

Provenance note: Full versioning + unknown fallback as a disciplined system is **S3-A1 design** on top of verified S3 header work. Speak carefully in production bridge.

8. Actions & security (intern)

Server may send `action: { type, payload }`. Client **allowlists** types. Never execute scripts. Validate URLs against domain policy (Day 09 whitelist mindset). Auth-sensitive actions re-check client-side.

9. Two production anchors

BMS header · S3

Generalised, protocol-driven main header from CMS/backend so many layout/content changes ship without an app release. Paired historically with MVVM search (Day 08).

Aces splash · S12

Server-driven splash improved cold-start flexibility/freshness alongside audio streaming work. Startup is a product surface — cache + timeout to default; don’t block forever on network. **No invented ms** on resume.

10. SDUI vs WebView vs native Ads

Approach	Use
SDUI native registry	Dynamic layout/content with a11y/perf budgets
WebView	Rare complex docs/legal islands — not primary chrome

Native Ads/HeroWidget (S1)	Revenue media lifecycle often stays native; SDUI may configure placement
----------------------------	--

11. Self-check

Ready for deep dive when you can say:

- 1. SDUI ≠ JS eval ≠ whole-app WebView
- 2. Unknown component policy
- 3. Why empty root needs hard fallback
- 4. S3 verified vs S3-A1 design
- 5. Splash: cache + timeout default (S12)

12. Flash preview

Front	Back
SDUI def	Schema → native registry
Unknown	Skip + metric
Fallback	Last-known-good / baked
Action	Allowlist only
S3	BMS header CMS
S3-A1	Versioning design
S12	Aces splash

13. Schema evolution worked examples

Additive (safe for old clients)

Old client knows `promoBanner` . New payload adds optional `props.subtitle` . Old client ignores unknown field; banner still shows title.

New type (needs app release + fail-soft)

Server sends `type: "countdownTimer"` . Old clients **skip** + metric. New clients register the type after App Store ships. CMS enables countdown only for builds advertising capability.

Breaking (dual-publish)

`children` array renamed to `nodes` without major bump → parsers break. Correct approach: `schemaVersion: 4` with dual-publish of v3 and v4 until old clients fall below threshold — or keep additive shape.

14. Header component inventory (BMS-style thinking)

Type	Native responsibility	Skip impact
logo	Brand chrome	High — hard fallback if missing at root
citySelector	Location UX	Medium — may hide
promoBanner	CMS campaign	Low — skip OK
searchEntry	Opens search	Medium — provide native default entry
navIcons	Profile/tickets	Medium

15. Splash inventory (Aces-style thinking)

Concern	Policy
Brand/logo	Required minimum
Seasonal art	Cacheable; skippable if unknown
Legal/age gate	Product-required; prefer native if critical
Enter CTA	Required affordance
Network freshness	Soft refresh with timeout

16. Metric dictionary (speak these names)

Metric	Meaning
sdui_schema_reject	Version gate refused payload
sdui_unknown_component	Skipped type
sdui_unknown_action	Allowlist miss
sdui_fallback_used	LKG or baked engaged
sdui_splash_source	cache network default

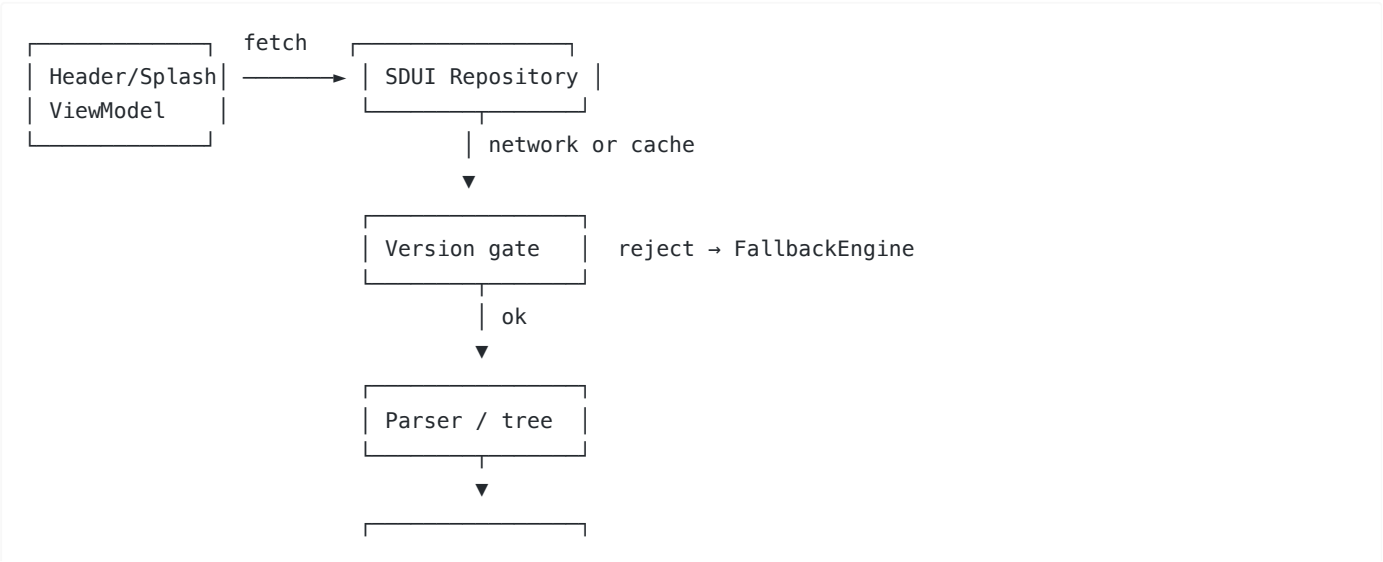
17. 90-second teaching script (record once)

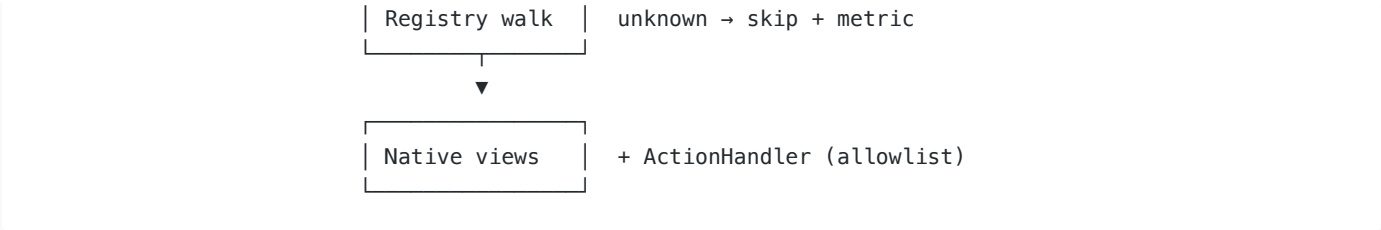
"SDUI maps a versioned schema to native components through a registry. Unknown types skip with metrics; incompatible majors fall back. Actions are allowlisted. Cached last-known-good protects offline and bad payloads. On BMS I worked a backend-driven header; on Aces a server-driven splash. Schema versioning and unknown fallbacks I'd insist on as design so CMS velocity doesn't become crash velocity."

02 — Deep Dive: Schema, Registry, Fallbacks, Cold Start, Parity

Senior depth. Assumes [01-foundations.md](#). Self-contained.

1. End-to-end data flow





Responsibilities

Piece	Owns	Does not own
ViewModel	Fetch/cache UI state; loading/error	Component drawing details
Repository	Network + disk cache policy	Business checkout rules
Version gate	Compatibility decision	Visual design
Registry	type → native factory	Networking
ActionHandler	Allowlisted side effects	Arbitrary script exec
FallbackEngine	Defaults / last-known-good	Inventing new component types

2. Schema shape (illustrative)

```
{
  "schemaVersion": 3,
  "id": "home_header",
  "root": {
    "type": "vstack",
    "children": [
      { "type": "logo", "props": { "style": "primary" } },
      { "type": "promoBanner", "props": { "title": "Weekend deals" },
        "action": { "type": "open_deeplink", "payload": { "url": "bms://offers/weekend" } } },
      { "type": "sparkle_v9", "props": {} }
    ]
  }
}
```

Client with registry {vstack, logo, promoBanner} :

- Renders logo + promo
- Skips sparkle_v9
- Emits unknown_component metric with type name + schemaVersion

3. Version gate mechanics

See [code/SchemaVersionGate.swift](#).

Case	Gate result	UI
payload.version <= clientMax and >= clientMin	accept	Render
payload.version > clientMax	reject_too_new	Fallback
payload.version < clientMin	reject_too_old	Fallback or force-upgrade messaging (product)
Missing version on legacy payload	treat as v1 policy	Documented default

Dual-publish: Backend serves old and new majors during transition; clients advertise capability. Don't force-upgrade 30L users for a banner experiment.

Additive fields: Old clients ignore unknown keys. Prefer additive evolution; reserve majors for structural breaks.

4. Component registry

See [code/ComponentRegistry.swift](#).

Design rules

- 1. **Fail soft on unknown** — never `fatalError` on CMS strings.
- 2. **Validate known props strictly** — bad props on a *known* type may skip that node or use safe defaults (pick a policy; be consistent).
- 3. **Stable identity** — each node should carry a server `id` for SwiftUI identity / analytics (Day 12 link).
- 4. **Keep leaves dumb** — registry returns views; VM owns payload lifecycle.
- 5. **App vs SDK ownership** — app-specific header components in app module; shared primitives can live in a UI kit. Stories SDK lesson (S10): clear public API if shared.

Partial failure vs empty root

Skipping children is fine. If the **root** resolves to nothing meaningful (no logo on splash, no header chrome), engage **hard fallback**. Skipping forever without metrics hides broken contracts.

5. FallbackEngine

See [code/FallbackEngine.swift](#).

Layer	When	Risk
Memory cache	Fast revisit	Process death loses it
Disk last-known-good	Offline / 5xx / parse fail	Stale content — TTL + version stamp
Baked default	No cache	Less fresh; always safe
Feature flag kill	Bad schema in wild	Disable SDUI surface → native default

TTL + freshness: Optional “updated earlier” affordance if product needs honesty. Splash often prioritizes **fast safe paint** over perfect freshness — with timeout to default.

Cache keying / privacy: Personalized headers must key by user/session; clear on logout. Don't leak user A's promo into user B's disk slot.

6. Actions & security

```
ActionHandler.handle(action)
  → type in allowlist?
    no → metric + return
    yes → validate payload (URL host, deeplink scheme)
          → dispatch to Router / Safari / refresh
```

Action type (examples)	Notes
open_deeplink	App schemes only; router owns nav (S13)
open_url	HTTPS + domain policy; prefer in-app browser
refresh_module	Re-fetch payload

custom_js	Never
-----------	-------

Tie to Day 09: domain whitelist mindset for anything that leaves the app.

7. Analytics in SDUI

Server may attach `analytics: { event, params }`. Client should:

- Validate event names against allowlist or prefix rules
- Inject common context (screen, app version, user bucket)
- Refuse to log unchecked PII from CMS

Don't trust CMS as a raw logging pipe.

8. BMS header (S3) — architecture reading

Verified claims:

- Protocol-driven generalised main-screen header from backend/CMS
- Contracts so many layout/content changes don't need App Store
- Search MVVM sibling (Day 08)

Senior elaboration (honest):

- Registry + fail-soft are how you keep CMS velocity from becoming crash velocity
- **S3-A1**: schema versioning + unknown-component fallback as the design you'd insist on — even if resume doesn't name a "VersionGate" type

Limits: New component *types* still need a client release. SDUI is not infinite flexibility.

9. Aces splash (S12) — cold start reading

Verified: server-driven splash; cold-start flexibility/freshness; audio streaming companion work. **No invented milliseconds.**

Design checklist for splash SDUI:

1. Prewarm / read disk cache on launch path
2. Network fetch with **short timeout** → default if slow
3. Minimum viable tree (logo / brand) if partial skip
4. Don't serialize audio init *and* splash network on one blocking main-thread chain
5. Measure time-to-interactive, not only first frame

Interview line: "Startup is a product surface — cached splash with timeout-to-default beats blocking on perfection."

10. iOS / Android parity

Practice	Why
Shared schema spec	One contract
Capability negotiation	Temporary gaps OK
Contract tests per platform registry	Catch drift in CI
Server targeting	Don't send iOS-only types to old Android blindly
Dual-publish rules	Breaking changes need runway

Parity is **negotiated**, not hoped.

11. Testing SDUI

Test	Catches
Fixture per schema version	Decode + gate
Registry unit tests	Known types render/factory
Chaos payload (unknown types)	Skip path / no crash
Empty root fixture	Hard fallback
Action allowlist tests	Unknown action no-op
Snapshot critical layouts	Visual chrome
Few XCUITest golden paths	Launch → header visible

Don't try to UITest every CMS combination — combinatorial explosion.

12. SDUI vs feature flags vs A/B

Tool	Changes
Feature flag	Code path on/off
A/B assignment	Variant bucketing
SDUI	Layout/content via payload

Often combined: flag enables SDUI surface; CMS/A/B supplies variant layout.

13. When SDUI is a bad idea

- Highly interactive unique UI with heavy custom animation
- Strong one-off a11y requirements hard to schema
- Rarely changing surfaces
- Team without schema QA / contract tests
- Revenue video lifecycle that needs native pause/play guarantees (S1 HeroWidget) — use SDUI for config/placement, keep media native

14. Performance notes

- Parse **off main** when payloads grow
- Budget interaction frames; prewarm splash cache
- Incremental render if tree is large
- Image props still go through a bounded image pipeline (Week 3)

Parser-on-main dropping frames is an NFR bug, not "JSON's fault."

15. Trade-off tables

Choice	When	Cost
Full SDUI home	Super-app experiment velocity	QA matrix explosion
Hybrid slots + native chrome	Most consumer apps	Slot contract discipline
Native-only	Rare change / regulated	Slow iteration
Skip unknown	Always for resilience	Visual gaps — measure
Disk cache splash/header	Cold start + offline	Stale risk — TTL + version

WebView island	Docs/legal	Perf/a11y/brand trade-offs
----------------	------------	----------------------------

16. Whiteboard script (5 min) — Mock #2 style

1. Draw schema → gate → registry → actions → fallback
2. State unknown-component policy
3. Map BMS header + Aces splash
4. Call out S3-A1 versioning as design
5. Contrast native Ads media if asked

Agenda:

"I'll define schema + registry + versioning + fail-soft, then map to BMS header and Aces splash, then trade-offs vs native Ads."

17. Failure modes

Failure	Mitigation
Breaking schema to 100% users	Version gate + canary + fallback + kill switch
Blank splash	Minimum tree + hard fallback
Personalized cache leak	User-keyed cache; clear on logout
Action open any URL	Allowlist + host policy
Silent skip spike	Alert on unknown_component rate
Main-thread parse jank	Background parse + budget

18. Decision rule card

1. Dynamic content/layout need + native quality? → SDUI hybrid
 2. Always: version gate + unknown skip + fallback
 3. Actions allowlisted; no script exec
 4. Splash: cache + timeout default
 5. New types need app release; CMS isn't infinite
 6. Speak S3 verified; S3-A1 as design

19. Optional citations (appendix)

- In-repo: `ios-system-design/docs/sdui-engine.md` , `feature-flag-system.md` , `ab-testing-experimentation-sdk.md`
- Apple: native view performance / accessibility guidance (not required to study)

20. Bridge

Next: [03-production-bridge.md](#) — S3, S12, S3-A1 honesty.

03 — Production Bridge: BMS Header, Aces Splash, S3-A1 Design

Convert theory into interview stories without overclaiming.

1. Provenance map for today

ID	Label	Exact claim you may make
S3	Verified	BMS backend-driven / CMS header ; generalised protocol-driven main-screen; API contracts for layout/content changes without release when possible; search debounce/MVVM is sibling (Day 08)
S3-A1	How I would apply it	Schema versioning + unknown-component fallback as design — not “I shipped a named VersionGate framework as a resume bullet”
S12	Verified	Aces server-driven splash ; cold-start flexibility/freshness; audio streaming context — no invented ms
Learning-lab	Illustrative	Gate / registry / fallback code in this chapter

Forbidden overclaims

- “We eval CMS JavaScript.”
- Invented cold-start latency numbers.
- “S3-A1 was our production VersioningService I named in resume.”
- “SDUI alone produced 99.95% crash-free.”
- Claiming Ads HeroWidget was fully SDUI (S1 is native lifecycle).

2. Verified S3 — STAR (2–3 min)

Opener (~10s)

“I’ll walk through making BookMyShow’s main header backend-driven with a protocolised client so content could move faster without waiting on releases.”

Situation / Task

Main header needed CMS/backend-driven flexibility; content and layout iteration was gated by app releases more than necessary.

Action

1. Migrated header toward a **generalised, protocol-driven** main-screen implementation.
2. Contracted APIs with backend so many layout/content changes didn’t need App Store.
3. Kept client native and structured — not a WebView rewrite of chrome.
4. Paired with search MVVM improvements (debounce/states) on the same surface family.

Result

Faster content iteration on header; clearer client contracts.

Lesson

SDUI needs schema/versioning and client fallbacks — not just “render JSON.” That lesson is why S3-A1 design matters in interviews.

Provenance: Verified · S3 · BookMyShow · backend-driven header

3. S3-A1 — Versioning & unknown fallback as design

When asked “what if backend sends a new component type / breaking schema?”, answer as **design judgment**:

Design element	Intent
schemaVersion gate	Reject incompatible majors to fallback
Unknown component skip	Never crash; metric + continue siblings

Empty-root hard fallback	Blank header/splash is unacceptable
Dual-publish	Breaking changes need runway
Canary payloads	% expose before 100%
Kill switch	Feature-flag back to native default

Script fragment:

“On BMS we shipped a backend-driven, protocolised header. Separately, as design, I’d require schema version gates and unknown-component skip with hard fallbacks — CMS velocity without those controls becomes crash velocity. I’m not claiming a named production VersionGate service as a resume bullet; I’m claiming the header work and the compatibility design I’d insist on.”

Provenance: How I would apply it · S3-A1 · schema versioning / unknown-component fallback as design

4. Verified S12 — Aces splash STAR beat

Opener (~10s)

“On Las Vegas Aces I revamped splash to be server-driven so cold-start content could stay fresh and flexible — alongside live audio streaming work.”

Action highlights

- 1. Server-driven splash content path.
- 2. Treat startup as product surface: prefer cached/flexible content with safe defaults over blocking forever.
- 3. Audio streaming integrated as separate live capability — don’t serialize everything on launch.

Result

Richer live audio; more flexible cold-start content. **Do not invent ms.**

Lesson

Measure time-to-interactive; cache + timeout-to-default.

Provenance: Verified · S12 · Raw/Aces · server-driven splash

5. Interview lines (≤20s)

S3:

“I made the BMS main header backend-driven with a protocolised client so many layout changes didn’t need a release — and I’d pair that with version gates and fail-soft unknowns as design.”

S12:

“On Aces I shipped a server-driven splash for colder-start flexibility and freshness, with safe defaults so startup never depended on a perfect network.”

6. Topic mapping

Topic	Verified?	How to speak
Backend-driven header	Yes · S3	Shipped direction
Protocol-driven main screen	Yes · S3	Shipped
Schema VersionGate type	S3-A1 design	Design I’d apply

Unknown skip policy	S3-A1 design	Design; aligns with crash-free culture
Aces server splash	Yes · S12	Shipped
Exact cold-start ms	No	Do not invent
JS eval SDUI	No	Never
Ads fully SDUI	No	S1 native media

7. Interviewer pushes

Push	Strong reply
"Isn't that a WebView?"	Native registry + schema; WebView is a tool, not the architecture.
"Backend broke schema for everyone."	Gate + fallback + canary + kill switch; client must survive (S8 culture).
"Skip makes empty UI."	Skip children; empty root → hard fallback; alert on skip spikes.
"Why not SDUI the ads video?"	Config maybe; media lifecycle native — S1.
"What ms did splash save?"	Speak flexibility/freshness; no invented ms.

8. Practice versions

60s S3: problem (release-gated header) → protocolised CMS header → faster iteration → lesson (versioning/fallback design).

60s S12: inflexible splash → server-driven splash + safe defaults → flexible cold start → lesson (TTI mindset).

3 min: add unknown skip, dual-publish, cache privacy, Ads contrast, honest S3-A1 label.

9. Links

- Code: [code/](#)
- Questions: [04-questions.md](#)
- Revision: [../../revision/weeks/week-02/day-10.md](#)
- Stories: [S3#s3--backend-driven-header--search-bookmyshow](#) · [S12#s12--audio-streaming--server-driven-splash-raw--las-vegas-aces](#)

04 — Questions (two-layer answers)

*Study flow: cover the full spoken answer → speak from **Answer points** only → uncover and compare timing.*
*Totals: **12 normal + 8 tricky = 20.***

Normal questions

Q1. What is server-driven UI? (30–45s)

Answer points (frame first):

- Backend sends structured layout/content schema
- Client maps types to native components
- Actions allowlisted
- Not WebView-only / not JS eval

Agenda opener: "I'd define schema-to-native, not web cosplay..."

Full spoken answer:

"Server-driven UI means the backend sends a structured schema for layout and content, and the client maps each type to a native SwiftUI or UIKit component through a registry. Actions are allowlisted. It's not evaluating JavaScript from CMS, and it's not 'the whole app is a WebView.' On BookMyShow we used that approach for a backend-driven main header so content could move faster without waiting on every release."

Common wrong answer: "SDUI means render HTML in a WebView."

Follow-up ladder:

Level	Ask	Point + short answer
L1	vs HTML WebView?	Native perf/a11y/brand + schema constraints.
L2	New types?	Still need app release to register.
L3	Search relation?	Header SDUI; search MVVM Day 08.

Provenance: Verified · S3 · BMS header

Q2. Core client components of an SDUI stack? (45–60s)

Answer points (frame first):

- Parser
- Version gate
- ComponentRegistry
- Layout/Action handlers
- FallbackEngine/cache
- Analytics hooks

Agenda opener: "I'd list the pipeline pieces..."

Full spoken answer:

"I'd describe a pipeline: parse the payload, run a schema version gate, resolve nodes through a ComponentRegistry, handle allowlisted actions, and keep a FallbackEngine for last-known-good or baked defaults. Analytics hooks validate events. The ViewModel owns fetch and cache state; the registry stays a pure mapping layer."

Common wrong answer: "Just JSONDecoder into View."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Where does VM sit?	Fetch/cache/error UI; registry maps.
L2	Who owns deeplink?	Action → app router.
L3	Metrics?	unknown_component / schema_reject.

Provenance: Learning-lab · pipeline; Verified S3 direction

Q3. How do you version the schema? (60s)

Answer points (frame first):

- Semantic / major-minor
- Client max-supported
- Dual-write on breaks
- Ignore unknown fields
- Reject incompatible major → fallback

Agenda opener: "I'd treat compatibility as a product feature..."

Full spoken answer:

"I version the schema and have the client declare a max-supported version. Additive fields are ignored by older clients. Breaking changes get a major bump with a dual-publish period. If the payload is too new, I reject to a safe fallback rather than crashing. Capability flags help the server avoid sending unsupported types. On BMS header work I'd insist on that discipline as design even when speaking carefully about what was named in production."

Common wrong answer: "Force upgrade the app for every banner change."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Who owns bumps?	Mobile + backend contract/changelog.
L2	Missing version?	Document legacy default policy.
L3	S3-A1?	Label as How I would apply it.

Provenance: How I would apply it · S3-A1 · versioning design

Q4. Unknown component arrives in prod — what happens? (45s)

Answer points (frame first):

- Skip node
- Log/metric
- Continue siblings
- Never crash
- Empty root → hard fallback

Agenda opener: "I'd fail soft, then measure..."

Full spoken answer:

"The registry skips the unknown node, emits a metric with the type name, and continues rendering siblings. We never crash on CMS strings. If skipping leaves a meaningless root — like a blank splash — we engage a hard fallback layout. Skipping without metrics is how broken contracts hide until users complain."

Common wrong answer: "Force unwrap and crash so QA notices."

Follow-up ladder:

Level	Ask	Point + short answer
L1	QA detection?	Contract tests + canary payloads.
L2	Alerting?	Spike on unknown_component rate.
L3	Crash-free link?	S8 culture — fail soft.

Provenance: How I would apply it · S3-A1; culture S8

Q5. Offline / fetch failure strategy? (45–60s)

Answer points (frame first):

- Disk last-known-good + TTL
- Baked default if none
- Stale affordance if needed
- Retry with backoff
- Splash: cache critical

Agenda opener: "I'd layer cache then default..."

Full spoken answer:

"On network or parse failure I show last-known-good from disk if present and within TTL; otherwise a baked native default. Optionally show a stale affordance if product needs honesty. Retry with backoff in the background. For splash specifically, cached content is critical so cold start isn't held hostage by the network — that's the Aces mindset."

Common wrong answer: "Blank screen until network returns."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Personalized cache?	Key by user; clear on logout.
L2	Timeout?	Short timeout → default on splash.
L3	Version-stamped cache?	Don't apply incompatible cached major.

Provenance: Verified · S12 · splash; Learning-lab fallback

Q6. How did BMS header SDUI help the business? (45s)

Answer points (frame first):

- Faster content/layout iteration
- Many changes without release
- Protocolised client
- Still need release for new types

Agenda opener: "I'd connect CMS velocity to release cost..."

Full spoken answer:

"It let content and layout on the main header iterate faster without waiting on App Store for many changes, behind a generalised protocol-driven client. That's real product velocity. The limit is honest: new component types still require an app release to register native renderers."

Common wrong answer: "Unlimited UI without ever shipping an app again."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Search?	MVVM sibling — Day 08.
L2	Android?	Shared schema + capability negotiation.
L3	Risk?	Without fallbacks → incidents.

Provenance: Verified · S3 · header iteration

Q7. SDUI actions security? (45–60s)

Answer points (frame first):

- Allowlist action types
- Validate URLs/domains
- No script execution
- Auth-sensitive re-check client-side

Agenda opener: "I'd treat actions as a privileged API..."

Full spoken answer:

"Actions are allowlisted — open deeplink, open URL, refresh module — not arbitrary code. URLs go through domain policy; deeplinks go through the app router. Auth-sensitive actions re-check client-side. Never execute CMS scripts. It's the same threat mindset as host whitelisting on networking day."

Common wrong answer: "Trust CMS completely; it's internal."

Follow-up ladder:

Level	Ask	Point + short answer
L1	External URL?	Controlled browser; ATS.
L2	Unknown action?	No-op + metric.
L3	Injection?	Don't string-concat into JS/WebView blindly.

Provenance: Learning-lab · allowlist; Day 09 mindset

Q8. When is SDUI a bad idea? (45s)

Answer points (frame first):

- Highly interactive unique UI
- Heavy custom a11y/animation
- Rare change surfaces
- No schema QA
- Prefer hybrid slots

Agenda opener: "I'd scope SDUI to change velocity vs complexity..."

Full spoken answer:

"SDUI is a poor fit for highly interactive one-off UI, heavy custom animation, or surfaces that rarely change — and it's dangerous if the team won't invest in schema QA. Prefer hybrid: CMS slots inside native chrome. Revenue ads video often stays native for lifecycle guarantees — configure placement maybe, don't SDUI the player."

Common wrong answer: "SDUI everything including checkout micro-interactions."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Ads HeroWidget?	Native pause/play — S1.
L2	Hybrid example?	Native tab bar + CMS header slots.
L3	Legal WebView?	Island OK; not primary nav.

Provenance: Judgment · S1 contrast

Q9. How do analytics work in SDUI? (45s)

Answer points (frame first):

- Server may attach event names
- Client validates
- Inject common context
- Don't trust PII unchecked

Agenda opener: "I'd validate before logging..."

Full spoken answer:

"The payload can attach analytics event names and params, but the client validates them and injects common context like screen and app version. I don't treat CMS as an unchecked PII pipe. Bad event names get dropped or mapped safely."

Common wrong answer: "Log whatever JSON says."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Mixpanel on Grizzlies?	Host analytics vs SDK — S13 adjacent.
L2	Allowlist events?	Yes for high-risk surfaces.
L3	Debug?	Sample unknown event metrics.

Provenance: Learning-lab · hygiene; S3 surface

Q10. Cold start + server splash — how do you think about metrics? (45–60s)

Answer points (frame first):

- TTI over vanity first-frame only
- Cache splash JSON/images
- Timeout → default
- Don't block forever
- No invented ms

Agenda opener: "I'd optimize for safe interactivity..."

Full spoken answer:

"I care about time-to-interactive more than a vanity first-frame number. Splash should read cache quickly, attempt a short network refresh, and fall back to a default if slow. Blocking launch on a perfect CMS response is how you create slow launches. On Aces we made splash server-driven for flexibility and freshness — I won't invent millisecond claims."

Common wrong answer: "Always wait for network so splash is fresh."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Audio init?	Don't serialize everything on main launch path.
L2	Minimum tree?	Logo/brand required.
L3	Metric names?	Launch stages + splash source cache/network/default.

Provenance: Verified · S12 · splash cold start

Q11. How do you test SDUI? (45–60s)

Answer points (frame first):

- Fixtures per version
- Registry unit tests
- Chaos unknown types
- Snapshots for chrome
- Few golden UITests
- Contract tests with backend

Agenda opener: "I'd test contracts more than every CMS combo..."

Full spoken answer:

"I use fixture schemas per version, registry unit tests, chaos payloads with unknown types, and snapshots for critical chrome. Contract tests with backend catch drift. UITests cover a few golden paths — not every CMS combination, because that matrix explodes."

Common wrong answer: "Only manual QA on production CMS."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Empty root fixture?	Assert hard fallback.
L2	Action tests?	Unknown action no-op.
L3	Canary?	Payload % rollout in prod.

Provenance: Learning-lab · S9 test mindset applied

Q12. SDUI vs feature flags vs A/B? (45s)

Answer points (frame first):

- Flags toggle code paths
- A/B assigns variants
- SDUI changes layout/content via payload
- Often combined

Agenda opener: "I'd separate mechanism from experiment..."

Full spoken answer:

"Feature flags toggle code paths. A/B assigns users to variants. SDUI changes layout and content through payloads. They're often combined: a flag enables an SDUI surface, and CMS or experiment config supplies the variant layout. Don't conflate the three in an interview."

Common wrong answer: "They're all the same thing."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Kill switch?	Flag back to native default.
L2	Personalization?	Server targeting + privacy-safe cache keys.
L3	Docs?	Optional feature-flag / ab-testing system design.

Provenance: Learning-lab · experiment vocabulary

Tricky questions

T1. JSON drives UI — isn't that just a WebView? (90–120s)

Answer points (frame first):

- Trap: conflate SDUI with web
- Native components + typed schema
- a11y/perf/offline
- WebView is a tool
- BMS header protocol-native

Agenda opener: "I'd separate architecture from embedding HTML..."

Full spoken answer:

"Driving UI from JSON doesn't mean WebView. SDUI maps a typed schema onto native components with accessibility, performance budgets, and offline cache. A WebView can be an island for documents, but it isn't the architecture. BookMyShow's header direction was protocol-native — structured contracts into native rendering — not 'put the home chrome in WKWebView.'"

Common wrong answer: SDUI = WebView.

Follow-up ladder:

Level	Ask	Point + short answer
L1	When WebView wins?	Complex legal/docs; not primary nav.
L2	Hybrid HTML island?	Possible; still allowlist actions.
L3	SEO?	Irrelevant for native app chrome.

Provenance: Verified · S3 · native header direction

T2. Backend sent a breaking schema to 30L users. (120s)

Answer points (frame first):

- Version gate + fallback
- Kill switch / feature guard
- Crashlytics watch
- Canary %
- IMOC coordination mindset
- Client must survive

Agenda opener: "I'd talk survival first, then process..."

Full spoken answer:

"First, the client should already survive: version gate rejects to fallback, unknown nodes skip, empty root hard-falls back. Operationally I'd want canary payload exposure, a kill switch to native default, Crashlytics and schema-reject metrics, and coordination between mobile and backend owners — IMOC mindset at scale. The mistake is assuming CMS is safe because it's 'our server.'"

Common wrong answer: Only "we crashed" / only blame backend.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Who pages whom?	Mobile + backend; blast radius.
L2	Dual-publish?	Should have prevented hard break.
L3	S3-A1?	This is why versioning design matters.

Provenance: S3-A1 design + S8 culture

T3. How do you keep Android and iOS parity? (90s)

Answer points (frame first):

- Shared schema spec
- Capability negotiation

- Contract tests per registry
- Accept temporary gaps
- Server targeting

Agenda opener: "I'd negotiate parity, not hope..."

Full spoken answer:

"Parity comes from a shared schema spec, capability negotiation, and contract tests in CI against each platform's registry. Temporary capability gaps are OK if the server targets correctly. A new component ships in native apps first, then CMS starts using it. Copy-pasting JSON and hoping is how iOS renders a banner Android skips into a lopsided experiment."

Common wrong answer: "Same JSON always looks identical."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Gap communication?	CMS tooling shows support matrix.
L2	Version skew?	Dual-publish + min versions.
L3	Design system?	Shared primitives help.

Provenance: Learning-lab · parity practice

T4. Skip unknown components — empty screen risk? (90s)

Answer points (frame first):

- Skip is crash-safe default
- Metric + alert
- Minimum viable tree
- Empty root → hard fallback

Agenda opener: "I'd separate child skip from root emptiness..."

Full spoken answer:

"Skipping unknowns is the crash-safe default, but you must emit metrics and alert on spikes. Define a minimum viable tree — for splash, at least brand/logo. If the resolved root is meaningless, hard fallback. Skipping without measurement trades crashes for silent blank UI, which users experience as ‘app is broken’ anyway."

Common wrong answer: Skip forever without metrics.

Follow-up ladder:

Level	Ask	Point + short answer
L1	S12 policy?	Minimum splash content.
L2	Placeholder views?	Only if design allows.
L3	Partial promo missing?	OK; header chrome must remain.

Provenance: S3-A1 design + S12 splash rules

T5. Personalised SDUI + cache — privacy leak? (90–120s)

Answer points (frame first):

- Cache keys include user/session
- Clear on logout

- Limit sensitive retention
- Guest vs logged-in variants

Agenda opener: "I'd key and invalidate deliberately..."

Full spoken answer:

"Personalized payloads must not live in a single global disk slot. Cache keys include user or session, and logout clears them. Be intentional about what splash or header may contain and how long it persists. Guest versus logged-in variants are different keys. A shared cache is how you leak another user's promo — or worse — across accounts on a family device."

Common wrong answer: One disk cache for all users.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Multi-user devices?	Clear aggressively on account switch.
L2	Encryption at rest?	Consider for sensitive CMS.
L3	Analytics in cache?	Don't persist PII needlessly.

Provenance: Learning-lab · privacy

T6. Where should component registry live — app or SDK? (90s)

Answer points (frame first):

- App owns app-specific components
- SDK may own primitives
- Clear public API if shared
- Header registry likely app module

Agenda opener: "I'd put specificity with ownership..."

Full spoken answer:

"App-specific header components live in the app module with protocol hooks. A shared SDK can own common primitives — text, stack, image — if versioned cleanly. The Stories SDK lesson applies: public API and independence matter when multiple apps share code. Don't shove every BMS-only promo widget into a portfolio SDK."

Common wrong answer: Absolute one place forever.

Follow-up ladder:

Level	Ask	Point + short answer
L1	S10 crossover?	Modularity + versioning.
L2	Binary size?	Primitives shared; features app-local.
L3	Registration API?	App registers custom types at launch.

Provenance: S10 adjacent modularity

T7. SDUI for revenue Ads? (90–120s)

Answer points (frame first):

- Ads need strict native lifecycle
- SDUI for placement/config OK
- Media renderers native POP+generics

- Don't SDUI pause/play

Agenda opener: "I'd split config from media runtime..."

Full spoken answer:

"I'd be cautious. Revenue ads often need strict native lifecycle — pause and play with visibility, as with HeroWidget. SDUI can drive placement or configuration, but media renderers should stay native and type-safe. Schema-driving a video player's lifecycle is how you get wasted playback and UI glitches on a money path."

Common wrong answer: SDUI everything including video ads.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Mock fork?	Ads architecture vs SDUI engine.
L2	S1 proof?	POP+generics native pipeline.
L3	Experiment ads layout?	Config/slots yes; player no.

Provenance: Verified · S1 contrast with SDUI

T8. Parser on main thread drops frames — response? (90s)

Answer points (frame first):

- Parse off main
- Budget interactions
- Prewarm splash cache
- Incremental render
- Profile launch

Agenda opener: "I'd treat it as an NFR bug..."

Full spoken answer:

"Parsing large SDUI payloads on the main thread is a performance defect. Move parse off main, prewarm splash cache, render incrementally if needed, and keep interaction budgets tight. Then profile launch with Time Profiler. JSON isn't inherently slow — main-thread work without a budget is."

Common wrong answer: Ignore NFRs / blame JSON.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Where decode?	Repository/background before UI apply.
L2	Images?	Bounded pipeline Week 3.
L3	Metric?	Parse duration histogram.

Provenance: Learning-lab · NFR

05 — Exercises

Prefer speaking before peeking.

A. Conceptual

A1. Pipeline recall

Draw: CMS → ... → native view, including unknown + version reject branches.

Solution:

```
Fetch → Version gate (reject → Fallback)
      → Parse → Registry (unknown → skip+metric)
      → Native views + Action allowlist
      → Fallback: disk LKG / baked default / empty-root hard fallback
```

A2. S3 vs S3-A1 (two sentences)

Solution: S3 verified = backend-driven protocolised header. S3-A1 = schema versioning + unknown fallback spoken as design, not a named shipped runbook bullet.

A3. T/F

1. Unknown components should crash to alert QA.
2. Splash should wait indefinitely for fresh CMS.
3. Actions must be allowlisted.
4. Invented ms are OK if "about right."
5. Empty root after skips needs hard fallback.

Solution: 1F 2F 3T 4F 5T

A4. Provenance rewrite

"I built VersionGate and cut splash by 400ms with SDUI JS."

Solution: "I worked on BMS backend-driven header and Aces server-driven splash. Schema versioning and unknown-component fallback I'd insist on as design (S3-A1). Splash improved flexibility/freshness — I don't claim a millisecond figure or JS-eval SDUI."

B. Coding

B1. SchemaVersionGate

Open `code/SchemaVersionGate.swift` . What happens at `maxSupported+1`?

Solution: `rejectTooNew` → caller should fallback.

B2. Registry chaos

Using `ComponentRegistry` , resolve a tree with one unknown child; assert sibling remains and metric fires.

B3. FallbackEngine

Explain why `cacheKey` includes `userId` for personalized headers.

Solution: Prevent cross-account cache leak; clear on logout.

C. Speaking

1. S3 opener 20s + S3-A1 design sentence.
2. S12 opener 20s without ms.
3. T2 breaking schema 120s.
4. T7 Ads vs SDUI 90s.

D. Whiteboard (5 min)

SDUI HLD + BMS header types you'd register + fallback rules.

E. Timed drill

Q3, Q4, Q10 + T2, T7. Score with timing guide. Log versioning + unknown policy misses.

F. Flashcards

Front	Back
SDUI	Schema → native registry · Trap: JS eval · Prod: S3
Unknown	Skip + metric · Trap: crash · Prod: S3-A1
Fallback	LKG + baked · Trap: blank · Prod: S12
Action	Allowlist · Trap: any URL · Prod: security
S3-A1	Design versioning · Trap: overclaim shipped ops · Prod: honesty

G. Deeper drills

G1. Dual-publish narrative (60s)

Speak how you'd roll a breaking header schema without force-upgrading 30L users.

Solution sketch: Ship client supporting vN+1 behind capability; backend dual-publishes; canary % on new major; watch schema_reject/unknown metrics; then ramp. Old clients keep vN.

G2. Minimum splash tree

List 3 nodes that must survive skip storms on Aces-style splash.

Solution: Brand/logo; optional legal/age gate if required; primary CTA or enter-app affordance. Promo sparkles are skippable.

G3. Action allowlist table

Write 5 action types you'd allow and 2 you'd reject.

Solution allow: open_deeplink, open_url (https+allowlist host), refresh_module, dismiss, track_event (validated).

Reject: eval_js, open_url with arbitrary scheme, send_sms to CMS-provided number without confirm, raw intent broadcasts.

G4. Map S3 search vs S3 header

One paragraph: which day owns which beat.

Solution: Day 08 owns search MVVM debounce/cancel/states. Day 10 owns header SDUI schema/registry/fallback. Same story ID S3; different interview angles.

G5. Incident timeline (whiteboard)

Backend ships unknown `type=heroVideoAutoplay` to 100% by mistake. Write T0–T60 minute client+ops response.

Solution sketch:

T0 metrics spike unknown_component / maybe battery complaints if somehow rendered.

T5 confirm payload; enable kill switch to baked header if available.

T15 canary rollback server-side; client already skipping if unregistered.

T30 verify fallback/header chrome healthy; Crashlytics quiet.

T60 postmortem: dual-publish + canary requirement; contract test gap.

G6. Privacy cache scenario

User A logs out; User B logs in on same device. What must be true?

Solution: Header/splash cache keys differ by user; logout clears A's keys; B never sees A's promos or PII-bearing payload.

G7. Speak Ads contrast (45s)

Without naming invent metrics.

Solution: "For revenue media I'd keep pause/play native — S1 HeroWidget — and let SDUI configure placement. Schema shouldn't own the video lifecycle."

Day 11 — UIKit Lifecycle, Cells, Prefetch · Hybrid UIKit ↔ SwiftUI

Week 2 · Full study (self-contained) · ~4–5 hrs
Revision twin: [revision/weeks/week-02/day-11.md](#)

Outcomes

By end of day, without notes, you can:

- Walk **UIViewController lifecycle** landmarks and where to start/stop work (incl. video/ads pause-play)
- Explain **cell reuse**, `prepareForReuse`, Diffable identity, and **prefetch** budgets
- Host **SwiftUI in UIKit** and **UIKit in SwiftUI** without identity/lifecycle bugs
- Deliver **S13** Grizzlies hybrid + deeplinks; **S6** LE Bottom Sheet (**30%+** fewer full-screen navs)
- Tie **S1** HeroWidget pause/play to visibility/VC lifecycle

How to study

1. `01-foundations.md`
2. `02-deep-dive.md`
3. `03-production-bridge.md`
4. `code/` — `CellReuseGuard`, `PrefetchBudget`, `HybridHosting`
5. `04-questions.md` — Answer points → full spoken
6. `05-exercises.md`
7. Revision twin

Module map

Module	File
Foundations	01-foundations.md
Deep dive	02-deep-dive.md
Production bridge	03-production-bridge.md
Questions	04-questions.md
Exercises	05-exercises.md
Code	code/CellReuseGuard.swift , code/PrefetchBudget.swift , code/HybridHostingNotes.swift

Provenance reminder

Label	Meaning
Verified · S13	Grizzlies SwiftUI↔UIKit interop; deeplinks; Mixpanel; Airship

Verified · S6	LE Bottom Sheet; 30%+ of user flows fewer full-screen navigations
Verified · S1	HeroWidget pause/play with visibility / VC lifecycle
Learning-lab	Cell/prefetch/hosting snippets

Do **not** invent additional nav % beyond resume’s **30%+**; don’t claim dual `NavigationPath+UINavigationController` sync as a virtue.

Agenda opener

“Lifecycle ownership first, then cells and prefetch budgets, then hybrid identity, then product sheet metric and deeplink routing.”

Time budget

Block	Minutes
Foundations	45–60
Deep dive + code	90–120
Production bridge	30–40
Questions	45–60
Exercises	30–40
Revision skim	15

01 — Foundations: Lifecycle, Cells, Hybrid — Mental Model

Intern → mid. Read before deep dive.

1. Plain-English mental model

UIKit screens are **objects with a lifetime**. If you start a video in `viewDidLoad` and never pause, it plays in the graveyard behind another tab. If you bind an image in a cell and ignore reuse, yesterday’s poster smiles on today’s movie.

Hybrid UI means SwiftUI and UIKit share one product. Each side has rules; bolting `UIHostingController` without ownership is how deeplinks double-present and state resets.

Restaurant metaphor:

Metaphor	UIKit idea
Open restaurant	<code>viewDidLoad</code> — one-time setup
Seat guests	<code>viewWillAppear</code> / <code>viewDidAppear</code>
Clear table between guests	<code>prepareForReuse</code>
Call ahead for ingredients	Prefetch
Two kitchens, one ticket system	Hybrid + single router

2. Glossary

Term	Meaning
------	---------

UIViewController lifecycle	Callbacks from load → appear → layout → disappear → deinit
viewWillAppear	iOS 17+ appear-phase hook
Cell reuse	Table/collection recycles views for different indexPaths
prepareForReuse	Reset UI + cancel async before new bind
Diffable Data Source	Identity-based list updates / animated diffs
Prefetching	Warm data/images for near-viewport paths
UIHostingController	Embed SwiftUI in UIKit
UIViewControllerRepresentable	Embed UIKit VC in SwiftUI
UIViewRepresentable	Embed UIKit view in SwiftUI
Coordinator	Glue for delegates inside representables
Structural vs explicit identity	How SwiftUI decides “same view” (Day 12 deepens)
Deeplink intent	Typed navigation request from URL
Bottom sheet	Modal lightweight surface; preserves context
Detents	Sheet heights (medium/large/custom)
Visibility threshold	How much of an ad must show to play

3. Lifecycle order (say aloud)

init → loadView → viewDidLoad → viewWillAppear → viewIsAppearing (iOS 17+) → viewDidAppear → layout callbacks → viewWillDisappear → viewDidDisappear → deinit

Hook	Put here	Avoid
viewDidLoad	One-time setup, bindings	Assuming final bounds; repeat network forever
viewWillAppear	Refresh on-screen state	Heavy sync I/O
viewDidAppear	Analytics “seen”; start players	Assuming visible forever
viewWillDisappear	Pause video; cancel tasks	Forgetting ads pause
deinit	Prove no retain cycle	Relying on it for critical cleanup

Ads link (S1): HeroWidget pause/play tied to visibility / VC lifecycle — lifecycle is part of the revenue contract.

4. Cells — intern path

1. cellForItem dequeues recycled cell.
2. Bind model for indexPath; start image task with ID token.
3. User scrolls; cell goes to reuse pool.
4. prepareForReuse cancels task, clears image, clears handlers.
5. Rebound for new model — no stale poster.

Wrong image bug: Async completion after reuse assigns old image unless you check ID / cancel.

5. Prefetch — intern path

System hints upcoming indexPaths → start warm fetches → cancel when `cancelPrefetchingForItemsAt` fires. Bound concurrency. Not unlimited download.

6. Hybrid — intern path

Direction	API
SwiftUI in UIKit	UIHostingController as child VC
UIKit in SwiftUI	Representable + Coordinator

One navigation owner. Deeplinks write to that owner only (S13).

7. LE Bottom Sheet — product picture (S6)

Lightweight event overview sheet reduced **full-screen navigations for 30%+ of user flows**. Prefer sheet when overview depth is shallow; push when hierarchy is deep. Cross-functional API/content contracts with PM/Design/Backend.

8. Production anchors

- **S13 Grizzlies:** Hybrid architecture; deeplinks; Mixpanel; Airship — interop designed, not bolted.
- **S6 BMS:** LE Bottom Sheet; **30%+** nav reduction metric from resume.
- **S1:** Ads video lifecycle.

9. Self-check

1. Appear vs load distinction
2. prepareForReuse duties
3. Prefetch cancel
4. Why dual nav stacks fail
5. Sheet vs push product rule + 30%+

10. Flash preview

Front	Back
Pause video	willDisappear / offscreen
Reuse	prepareForReuse reset
Prefetch	Warm + cancel + budget
SwiftUI⊂UIKit	UIHostingController
UIKit⊂SwiftUI	Representable
S6	30%+ fewer full-screen navs
S13	Hybrid + deeplink owner

11. Appearance vs load — tab example

User opens Tab A → load + didLoad + appear

User switches B → A disappear; B load (first time) + appear

User back to A → A appear ONLY (didLoad does not rerun)

If Tab A fetched only in `viewDidLoad` , data is stale forever. Throttle refresh on appear (e.g. if older than N seconds).

12. Cell bind sequence (memorize)

```
dequeue → prepareForReuse (previous owner cleanup)
    → configure(model)
    → start async with token
    → on complete: if token matches → apply
```

Never configure without a reuse reset path.

13. Hybrid containment checklist (UIHostingController)

- 1. addChild(hosting)
- 2. Add hosting.view constraints
- 3. hosting.didMove(toParent: self)
- 4. Forward appearance if needed for nested players
- 5. On remove: willMove , remove view, removeFromParent

Skipping containment breaks rotation, safe area, and appearance forwarding.

14. Sheet product heuristics

Signal	Lean sheet	Lean push
Depth	1–2 actions	Multi-step wizard
Context	Keep list underneath	Replace context
Frequency	High overview taps	Rare deep tools
Metric story	S6 30%+	—

15. 90-second teaching script

“UIKit lifecycle separates one-time load from every-show appear work — pause and cancel on disappear. Cells must reset and cancel in prepareForReuse; prefetch must cancel and bound concurrency. Hybrid UI needs one navigation owner for deeplinks and push taps. On Grizzlies I designed SwiftUI↔UIKit interop that way; on BookMyShow the LE Bottom Sheet cut full-screen navigations for 30%+ of flows.”

02 — Deep Dive: Lifecycle, Reuse, Prefetch, Hybrid, Sheets

Senior depth. Assumes [01-foundations.md](#). Self-contained.

1. Lifecycle ownership matrix

Work	Best hook	Why
Add subviews / bind VM once	viewDidLoad	Runs once per load
Refresh scores / header	viewWillAppear	Every show (tabs!)
Screen analytics “viewed”	viewDidAppear	Actually visible
Start muted preview video	viewDidAppear + visibility	Don't play offscreen
Pause ad / cancel search	viewWillDisappear	Leaving screen

Layout-dependent snap	viewDidLayoutSubviews	Bounds ready — carefully
Break retain proof	deinit logging in DEBUG	Not production cleanup

Tab bar trap

Tab view controllers often **load once** and **appear many times**. Network-only-in- `viewDidLoad` means stale data forever. Refresh policy belongs in appear (with caching/throttle).

`viewDidLoad` isn't called again — bug or feature?

Feature when the VC instance is retained in a nav stack. Put repeatable work in appear hooks.

2. Ads / video visibility (S1)

Revenue contract:

```
visible enough → play
below threshold / disappear / background → pause
```

Implementation patterns:

- VC disappear hooks for full-screen players
- UICollectionView visibility % for in-feed ads
- App background notifications as a third pause signal

Interview line: “Lifecycle isn't plumbing — for HeroWidget it was part of the product contract.”

3. Cells & Diffable

prepareForReuse checklist

See [code/CellReuseGuard.swift](#).

1. Cancel image / network tasks
2. Clear image, text, highlighted state
3. Nil out closures / targets that capture VC
4. Reset swipe/gesture transient UI
5. Invalidate display tokens / generation IDs

Wrong-image anatomy

```
cell binds movie A → start download A
scroll → reuse for movie B → bind B → start download B
download A completes → sets image without ID check → wrong poster
```

Fix: store `expectedID` on cell; completion checks ID; cancel on reuse.

Diffable benefits

- Identity-based updates; fewer `reloadData` footguns
- Animated diffs when IDs stable
- Still need **stable Hashable IDs** — unstable hashes → flicker/reorder chaos

Self-sizing jank causes

- Ambiguous Autolayout
- Estimated size far from real
- Image height changes after bind

- Heavy main-thread work in bind

Mitigations: better estimates, prefetch images, stable heights when product allows, avoid multi-pass thrash.

4. Prefetch budgets

See [code/PrefetchBudget.swift](#).

Rule	Reason
Cancel on cancelPrefetching...	Don't download off-screen forever
Bound concurrency	Protect network + battery
Respect Low Data Mode	Platform courtesy / App Store expectations
Tune distance	Aggressive ≠ better
Wi-Fi vs cellular policy	Product decision

Trap: Prefetch = unlimited download → data usage complaints.

Couple with Day 09 cancellation and Week 3 image pipeline.

5. Hybrid UIKit ↔ SwiftUI

See [code/HybridHostingNotes.swift](#).

SwiftUI in UIKit (`UIHostingController`)

Concern	Practice
Containment	Add as child VC correctly (<code>addChild</code> , <code>didMove</code>)
Sizing	Constraints / intrinsic / <code>sizingOptions</code> awareness
Safe area	Hosting <code>safeArea</code> quirks — test on notched devices
Lifecycle	Parent appear/disappear should inform pause policies
State	Pass observable models; don't rebuild hosting VC every bind

UIKit in SwiftUI (Representable)

Concern	Practice
make VS update	Create once; update props
Coordinator	Delegates / target-action
Identity stability	Avoid parent <code>.id</code> churn → remake storms
Navigation	Don't let representable own a second stack blindly

Dual navigation anti-pattern

```

NavigationPath ↔ UINavigationController
  ↑           ↑
deeplink A    deeplink B

```

Both mutate → double present, lost back stack, analytics double-count. **Pick one root owner**; bridge at edges. Grizzlies lesson: design interop + deeplink ownership deliberately (S13).

6. Deeplinks in hybrid apps

URL → Parse at app edge → typed Intent → single Router

- ensure root ready (cold start queue)
- push/present UIKit host OR SwiftUI host
- Airship/push taps use SAME router

Rule	Why
One router	No parallel nav
Queue until ready	Cold start races
Same path for push taps	Mixpanel/Airship don't invent second nav (S13)
Auth gates in router	Not deep in random VCs

7. LE Bottom Sheet (S6) — engineering + product

Decision

Prefer sheet	Prefer push
Glanceable overview	Deep multi-step hierarchy
Keep underlying context	Need strong back-stack history
Reduce nav fatigue	Full-screen tools / checkout

Delivery notes

- Reusable component integrated in high-traffic flows
- API/content contracts with PM/Design/Backend
- Detents + VoiceOver focus matter
- Metric you may claim: **30%+ of user flows** fewer full-screen navigations

Why not a new tab? Tabs change IA; sheet fixes local friction in existing flows.

8. Analytics + push with lifecycle (S13)

- Screen events on appear/disappear — avoid double-count with hybrid hosts
- Push notification taps → same deeplink router
- Gate SDK init on privacy consent
- Don't fire "viewed" from `viewDidLoad` for tab VCs that aren't visible

9. Retain cycles — cells & closures

VC → collectionView → Cell → closure → VC

Use `[weak self]` ; clear handlers in `prepareForReuse` . Instruments Leaks (Day 03 recall). Crash-free hygiene (S8) includes leak discipline under scroll pressure.

10. Representable `update` storms

Symptoms: `updateUIViewController` spam; jank; players restart.

Causes: parent state churn; non-Equatable inputs; identity resets; heavy work inside `update` .

Fixes: reduce observed state; stable identity; move heavy work out; Equatable inputs where measured useful (Day 12).

11. Trade-offs

Choice	When	Cost
Pure UIKit	Mature fine control	Slower feature UI
Pure SwiftUI	New features	Legacy gaps
Hybrid	Real NBA apps	Lifecycle/identity complexity
Bottom sheet	Shallow overview	Not for deep flows
Full-screen push	Deep hierarchy	Nav fatigue (S6 problem)
Aggressive prefetch	Image feeds	Data/battery burn

12. Whiteboard scripts

- A. Lifecycle + ads pause
- B. Deeplink → single router → UIKit vs SwiftUI host
- C. Sheet vs push with 30%+ metric

Agenda: README opener.

13. Failure modes

Failure	Mitigation
Video plays offscreen	Disappear + visibility policy
Wrong cell image	Token + cancel on reuse
Prefetch bill shock	Budget + Low Data Mode
Double present deeplink	Single router + ready queue
Hosting height fights	Intrinsic/sizing ownership
Sheet for checkout wizard	Wrong tool — push/flow

14. Decision rule card

- 1. One-time vs every-show → `didLoad` vs `appear`
 - 2. Leave screen → `pause/cancel`
 - 3. Cells: `reset` + `cancel` in `prepareForReuse`
 - 4. Prefetch: `warm` + `cancel` + `bound`
 - 5. Hybrid: one nav owner; stable representable identity
 - 6. Sheet for shallow overview; claim 30%+ only from S6

15. Optional citations

Apple `UIViewController` / `UIViewControllerRepresentable` docs; WWDC demystify identity; in-repo deeplink system design doc — appendix only.

16. Bridge

→ [03-production-bridge.md](#)

17. Appearance forwarding in custom containers

If you build a custom container VC that swaps children, you must forward appearance transitions (`beginAppearanceTransition` / `endAppearanceTransition`) or child `viewWillAppear` won't run — analytics and pause/play break silently.

UINavigationController and UITabBarController do this for you. Your clever container might not.

18. Prefetch + image pipeline interaction

```
prefetch(indexPaths)
  → repository.warm(ids)    // decode budgets, lower priority
cell configure
  → repository.image(id)    // coalesce with in-flight prefetch
cancelPrefetch
  → cancel warms not yet needed
```

Without coalescing, prefetch and cell bind double-fetch the same URL. Repository single-flight per URL helps (Day 09 cousin idea).

19. Bottom sheet engineering details

Topic	Practice
Detents	Medium for overview; large for expanded details
Grabber	Accessibility + discoverability
Keyboard	Avoid covering inputs if sheet has search
VoiceOver	Focus to sheet title on present; restore on dismiss
Analytics	open / cta / dismiss — don't count underlying screen as finished
Reuse	Shared component API across listing surfaces (S6)

20. Mixpanel screen-name discipline

Hybrid risk: UIKit parent and SwiftUI child both fire `screen_view` . Pick one owner per visible surface. Representable updates must not re-fire viewed events.

21. Extended decision card

```
Lifecycle: load once vs appear many
Media: visibility + disappear + background
Cells: token + cancel + clear handlers
Prefetch: warm + cancel + limit + Low Data Mode
Hybrid: containment + sizing + one router
Product UI: sheet vs push by depth; S6 30%+ only
```

03 — Production Bridge: Grizzlies Hybrid + LE Bottom Sheet

1. Provenance map

ID	Label	Exact claim
S13	Verified	Memphis Grizzlies: SwiftUI + UIKit interop architecture; deeplinks + navigation/lifecycle; Mixpanel + Airship
S6	Verified	BMS LE Bottom Sheet ; reduced full-screen navigations for 30%+ of user flows
S1	Verified	Ads HeroWidget pause/play with visibility / VC lifecycle
Learning-lab	Illustrative	Cell/prefetch/hosting code samples

Forbidden

- Invent nav % other than resume **30%+**
- “Dual stacks in sync continuously” as best practice
- Claim Airship/Mixpanel without lifecycle discipline story when asked nav
- Invent Grizzlies crash-free %

2. S13 STAR (2–3 min)

Opener

“On Grizzlies I designed SwiftUI↔UIKit interop with deliberate lifecycle and deeplink ownership — not ad-hoc hosting.”

Situation

Production NBA app needed modern SwiftUI features beside existing UIKit, plus growth tooling (analytics/push).

Action

1. Architected key UI with **SwiftUI + UIKit interoperability** — hosting/representables with sizing and lifecycle considered.
2. Deep linking with **navigation/lifecycle handling**; cold-start readiness queue mindset.
3. Integrated **Mixpanel + Airship** so taps route through the same navigation story.

Result

Shipped hybrid UI with production navigation and engagement stack.

Lesson

Interop costs (identity, lifecycle, hosting) must be **designed**, not bolted.

Provenance: Verified · S13 · Raw/Grizzlies · hybrid UI + deeplinks + Mixpanel/Airship

3. S6 STAR beat (2–3 min or impact answer)

Opener

“I’ll cover the LE Bottom Sheet — a lightweight event overview that cut full-screen navigations for 30%+ of user flows.”

Action

1. Led end-to-end lightweight event-overview bottom sheet.
2. Aligned PM, Design, Backend on API contracts and content.
3. Shipped reusable component into high-traffic flows.

Result

30%+ fewer full-screen navigations for user flows (resume metric).

Lesson

Small UI surfaces with clear contracts beat large rewrites for navigation pain.

Provenance: Verified · S6 · BookMyShow · LE Bottom Sheet · 30%+ nav

4. S1 lifecycle hook (30–45s insert)

When asked where ads video pauses: disappear / offscreen / background — HeroWidget protocolised behavior.

Provenance: Verified · S1 · HeroWidget lifecycle

5. Interview lines ≤20s

S13: “On Grizzlies I designed SwiftUI↔UIKit interop with deliberate lifecycle and deeplink ownership — not ad-hoc hosting.”

S6: “I shipped a reusable LE Bottom Sheet that reduced full-screen navigations for 30%+ of user flows by keeping event overview in context.”

6. Topic mapping

Topic	Verified?	Speak
Hybrid architecture	S13	Yes
Deeplink router ownership	S13	Yes — designed handling
Mixpanel/Airship	S13	Yes
Exact hosting sizingOptions API	Learning-lab	Teaching detail
LE sheet + 30%+	S6	Yes — resume metric
HeroWidget pause	S1	Yes

7. Interviewer pushes

Push	Strong reply
“Why not pure SwiftUI?”	Legacy UIKit + ship velocity; hybrid with one nav owner.
“Why not a new tab for LE?”	Tabs change IA; sheet fixes local friction; 30%+ metric.
“Hosting height broken?”	Intrinsic/sizing ownership; avoid nested scroll fights.
“Prefetch caused data bills.”	Bound concurrency; cancel; Low Data Mode.
“Push opened wrong screen.”	Single router; same path as deeplinks.

8. Practice

60s S13: hybrid need → interop + deeplink owner → Mixpanel/Airship same path → lesson design costs.

60s S6: nav fatigue → sheet + contracts → 30%+ → small surface lesson.

3 min: add cell reuse / ads pause if asked.

9. Links

- Code: [code/](#)
- Questions: [04-questions.md](#)

- Revision: [../../revision/weeks/week-02/day-11.md](#)
- Stories: [S13#s13--swiftuiuit--deeplinks--analyticspush-raw--memphis-grizzlies](#) · [S6#s6--le-bottom-sheet-bookmyshow](#)

04 — Questions (two-layer answers)

*Study flow: cover the full spoken answer → speak from **Answer points** only → uncover and compare timing.*
Totals: 12 normal + 8 tricky = 20.

Normal questions

Q1. Walk UIViewController appearance lifecycle. (30–45s)

Answer points (frame first):

- load → will/did appear → layout → will/did disappear
- Appear for refresh
- Disappear for pause/cancel
- didLoad one-time

Agenda opener: "I'd separate one-time setup from every-show work..."

Full spoken answer:

"After load and viewDidLoad for one-time setup, viewWillAppear and viewDidAppear run each time the screen shows — that's where I refresh on-screen state and start players. On the way out, viewWillDisappear and viewDidDisappear are where I pause video and cancel tasks. Layout callbacks handle bounds. Relying on deinit for critical pause is too late."

Common wrong answer: Put all networking only in viewDidLoad forever.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Tabs?	Load once, appear many — refresh on appear.
L2	viewsWillAppear?	iOS 17+ appear-phase hook.
L3	Ads?	Pause on disappear — S1.

Provenance: Verified · S1 lifecycle sibling

Q2. Why does cell reuse show wrong images? (45s)

Answer points (frame first):

- Async completes after reuse
- Fix: tokens/IDs
- Cancel in prepareForReuse
- Check identity before assign

Agenda opener: "I'd blame stale async, not UIImage..."

Full spoken answer:

"Cells are recycled. If an image download for movie A finishes after the cell was reused for movie B, you'll assign the wrong poster unless you cancel on prepareForReuse and check a generation token or model ID before applying. Clearing handlers matters too so closures don't retain the wrong context."

Common wrong answer: "UIKit is buggy; call reloadData always."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Prefetch?	Cancel prefetch when leaving paths.
L2	Diffable?	Stable IDs help updates; still cancel loads.
L3	Library?	Still need reuse hygiene.

Provenance: Learning-lab · CellReuseGuard

Q3. Diffable data source benefit? (30–45s)

Answer points (frame first):

- Identity-based updates
- Animated diffs
- Fewer reloadData footguns
- Need stable IDs

Agenda opener: "I'd center identity..."

Full spoken answer:

"Diffable data sources update lists by identity instead of blanket reloadData, which reduces flicker and makes animated diffs practical. You still need stable Hashable identifiers — unstable hashes cause flicker and weird moves."

Common wrong answer: "Diffable fixes all performance problems."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Hashable pitfall?	Unstable hashes → chaos.
L2	Equatable noise?	Avoid changing IDs on every refresh.
L3	SwiftUI ForEach?	Same stable-ID lesson Day 12.

Provenance: Learning-lab · lists

Q4. How does prefetching work? (45–60s)

Answer points (frame first):

- System hints upcoming indexPaths
- Warm fetches
- Cancel when not needed
- Bound concurrency

Agenda opener: "I'd pair warm with cancel and budget..."

Full spoken answer:

"UICollectionView prefetching tells you indexPaths likely to appear soon so you can warm images or data. You must implement cancelPrefetching for paths that scroll away, and bound concurrency so prefetch doesn't become an unbounded downloader — especially on cellular."

Common wrong answer: Prefetch everything at once.

Follow-up ladder:

Level	Ask	Point + short answer
-------	-----	----------------------

L1	Data or images?	Often both via repository.
L2	Low Data Mode?	Reduce aggressiveness.
L3	Day 09?	Same cancellation mindset.

Provenance: Learning-lab · PrefetchBudget

Q5. Embed SwiftUI in a UIKit app? (45–60s)

Answer points (frame first):

- UIHostingController
- Correct child containment
- Pass observable state
- Size with constraints/intrinsic

Agenda opener: “I’d treat hosting as a real child VC...”

Full spoken answer:

“I embed SwiftUI with UIHostingController, add it as a proper child view controller, pass state through an observable model, and constrain sizing carefully — including safe-area quirks. I avoid recreating the hosting controller on every tiny state change. That’s the hosting half of the Grizzlies hybrid approach.”

Common wrong answer: frame = bounds hacks only.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Safe area?	Test notched devices; hosting insets quirks.
L2	Pause media?	Parent disappear → pause.
L3	S13?	Designed interop, not bolt-on.

Provenance: Verified · S13 · hosting

Q6. Embed UIKit in SwiftUI? (45–60s)

Answer points (frame first):

- Representable
- Coordinator for delegates
- update for props
- Stable identity

Agenda opener: “I’d emphasize make-once, update-props...”

Full spoken answer:

“I wrap UIKit in UIViewControllerRepresentable or UIViewRepresentable, use a Coordinator for delegates, apply prop changes in update, and keep identity stable so SwiftUI doesn’t remake the controller every render. Recreate only when you truly need a fresh instance.”

Common wrong answer: New UUID id every body call.

Follow-up ladder:

Level	Ask	Point + short answer
L1	update spam?	Reduce state churn — Day 12.

L2	Navigation?	Don't spawn second stack.
L3	S13?	Interop costs designed.

Provenance: Verified · S13 · representable

Q7. Deeplink into hybrid nav stack — approach? (60–90s)

Answer points (frame first):

- Parse at app edge
- Typed Intent
- Single router
- Defer until root ready
- Avoid double-present

Agenda opener: "I'd insist on one navigation owner..."

Full spoken answer:

"I parse the URL at the app edge into a typed Intent, then a single router builds or pushes the correct UIKit or SwiftUI host. On cold start I queue until the root is ready. Push notification taps use the same router so Airship doesn't invent a parallel navigation path. Dual stacks fighting deeplinks is the classic hybrid failure."

Common wrong answer: Each SDK opens its own screen.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Auth gate?	Router decides.
L2	Analytics?	Don't double-count presents.
L3	S13?	Deeplink + lifecycle owned.

Provenance: Verified · S13 · deeplinks

Q8. Bottom sheet vs push — decision? (45s)

Answer points (frame first):

- Sheet for glanceable overview
- Push for deep hierarchy
- LE cut 30%+ full-screen navs
- Keep context

Agenda opener: "I'd decide by depth and context..."

Full spoken answer:

"Use a sheet when the user needs a glanceable overview and should keep underlying context; push when the flow is a deep hierarchy that needs history. The LE Bottom Sheet on BookMyShow reduced full-screen navigations for 30%+ of user flows by making event overview lightweight."

Common wrong answer: Always push for consistency.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Detents/a11y?	Design detents; VoiceOver focus.

L2	Why not tab?	Tabs change IA; sheet fixes local friction.
L3	Metric?	30%+ from resume — don't invent more.

Provenance: Verified · S6 · 30%+ nav

Q9. Where do you pause ad video? (45s)

Answer points (frame first):

- Disappear
- Out of viewport
- App background
- Resume on visible
- Protocolised HeroWidget

Agenda opener: "I'd make pause part of the contract..."

Full spoken answer:

"Pause when the screen disappears, when the ad scrolls below a visibility threshold, and on app background. Resume only when visibly eligible. On BookMyShow Ads we protocolised HeroWidget pause/play with lifecycle — wasted playback is a product bug on a revenue module."

Common wrong answer: Pause only in deinit.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Half visible?	Define threshold policy.
L2	Audio session?	Coordinate with other players.
L3	Cells?	Reuse must stop playback.

Provenance: Verified · S1 · HeroWidget

Q10. Retain cycle via closure in cell? (45s)

Answer points (frame first):

- Cell → closure → VC → collection → cell
- [weak self]
- Clear actions in prepareForReuse

Agenda opener: "I'd break the ring at capture lists..."

Full spoken answer:

"A cell holding a strong closure to its view controller which owns the collection view creates a cycle. Capture [weak self], and clear handlers in prepareForReuse so recycled cells don't keep old VC references. That's leak hygiene under scroll pressure."

Common wrong answer: Ignore — ARC always saves you.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Instruments?	Leaks / graph.
L2	Combine?	Store cancellables carefully.

L3	S8?	Crash-free includes leak discipline.
----	-----	--------------------------------------

Provenance: Learning-lab · S8 hygiene

Q11. Mixpanel + Airship with UI lifecycle? (45–60s)

Answer points (frame first):

- Screen events on appear/disappear
- Push taps → same deeplink router
- Don't double-count
- Consent gate SDK init

Agenda opener: "I'd unify engagement with navigation..."

Full spoken answer:

"Fire screen analytics from appear/disappear with hybrid hosts in mind so you don't double-count. Route Airship push taps through the same deeplink router as universal links. Gate SDK initialization on privacy consent. Lifecycle and growth tooling share one navigation story — that was part of the Grizzlies work."

Common wrong answer: Let each SDK present UI itself.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Background fetch?	Separate from screen viewed.
L2	Deep link collision?	Inbox queue until ready.
L3	S13?	Mixpanel + Airship verified.

Provenance: Verified · S13 · analytics/push

Q12. Self-sizing collection jank causes? (45–60s)

Answer points (frame first):

- Ambiguous constraints
- Bad estimated sizes
- Image height post-bind
- Heavy main work
- Fix estimates/prefetch/stable heights

Agenda opener: "I'd profile layout, not guess..."

Full spoken answer:

"Self-sizing jank usually comes from ambiguous constraints, estimated sizes far from reality, images changing height after bind, or heavy work on the main thread during bind. Fix estimates, prefetch images, stabilize heights when product allows, and keep cell bind cheap."

Common wrong answer: Only 'switch to SwiftUI List.'

Follow-up ladder:

Level	Ask	Point + short answer
L1	Instruments?	Time Profiler + Core Animation.
L2	SwiftUI list?	Trade-offs Day 12.

L3	Fixed height?	OK when design allows.
----	---------------	------------------------

Provenance: Learning-lab · performance

Tricky questions

T1. viewDidLoad isn’t called again — bug or feature? (90s)

Answer points (frame first):

- Feature when VC retained
- Refresh in appear
- Tab VCs load once
- Memory warning strategies separate

Agenda opener: “I’d call it a feature and relocate refresh...”

Full spoken answer:

“It’s usually a feature — the view controller instance is retained in the stack or tab. One-time setup belongs in viewDidLoad; repeatable refresh belongs in appear hooks. Tab controllers especially load once and appear many times. If data looks stuck, your refresh policy is wrong, not UIKit.”

Common wrong answer: Force recreate VCs every time.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Memory warning?	Drop caches; recreate views if system dumped them.
L2	Children?	Containment lifecycle forwarding.
L3	SwiftUI?	onAppear analogous, identity caveats Day 12.

Provenance: Learning-lab · lifecycle

T2. HostingController messes Auto Layout height. (90–120s)

Answer points (frame first):

- Intrinsic bridging
- sizingOptions awareness
- Constrain edges carefully
- Nested scroll fights
- Sometimes UIKit layout authority wins

Agenda opener: “I’d own sizing deliberately...”

Full spoken answer:

“Hosting height bugs usually mean intrinsic content size isn’t bridged cleanly. Constrain edges carefully, understand sizing options, and avoid nested scroll-view fights where both UIKit and SwiftUI think they own measurement. Sometimes giving UIKit layout authority for that island is the senior call. On Grizzlies-style hybrid, sizing is part of designing interop — not a weekend frame hack.”

Common wrong answer: Only set frame = bounds forever.

Follow-up ladder:

Level	Ask	Point + short answer
-------	-----	----------------------

L1	Scroll inside sheet?	One scroll owner.
L2	Intrinsic zero?	Content not ready — placeholder height.
L3	Recreate hosting?	Last resort; prefer updates.

Provenance: Verified · S13 · hybrid sizing

T3. Two sources of truth — `NavigationPath` and `UINavigationController`. (120s)

Answer points (frame first):

- Pick one root owner
- Bridge at edges
- Deeplinks write to owner only
- Hybrid fails when both mutate

Agenda opener: “I’d forbid dual writers...”

Full spoken answer:

“Two sources of truth will diverge. Pick one root navigation owner — either UIKit nav or SwiftUI path — and bridge at the edges. Deeplinks and push taps write only to that owner. Hybrid apps fail when both stacks mutate independently, causing double presents and lost history. Grizzlies taught deliberate single-router ownership.”

Common wrong answer: Sync both every frame.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Migration period?	Strangler with one writer still.
L2	Modals?	Presentation owned by same router policy.
L3	State restoration?	Serialize owner’s stack only.

Provenance: Verified · S13 · single router

T4. Prefetch caused data usage complaints. (90s)

Answer points (frame first):

- Bound concurrency
- Low Data Mode
- Cancel aggressively
- Tune distance
- Wi-Fi vs cellular policy

Agenda opener: “I’d tune policy, not delete prefetch...”

Full spoken answer:

“Prefetch isn’t bad — unbounded prefetch is. I’d bound concurrency, respect Low Data Mode, cancel aggressively, tune prefetch distance, and consider Wi-Fi versus cellular policy. Then revisit image quality tiers. Prefetch should feel invisible, not like a background downloader.”

Common wrong answer: Disable all prefetch forever.

Follow-up ladder:

Level	Ask	Point + short answer
-------	-----	----------------------

L1	Metric?	Bytes/session; cancel rates.
L2	User setting?	Data saver mode.
L3	Week 3?	Image pipeline quality tiers.

Provenance: Learning-lab · budget

T5. Why not make LE overview a new tab? (90s)

Answer points (frame first):

- Product: reduce steps in existing flows
- Sheet preserves context
- 30%+ metric
- Tabs change IA

Agenda opener: "I'd answer as product + engineering..."

Full spoken answer:

"A new tab changes information architecture for everyone. The LE problem was local friction — users took full-screen navigations for a shallow overview too often. A sheet preserves context and cuts steps in existing flows. We measured impact as 30%+ fewer full-screen navigations. That's a product-minded UI change with cross-functional contracts, not a nav redesign for its own sake."

Common wrong answer: Engineering-only 'sheets are cool.'

Follow-up ladder:

Level	Ask	Point + short answer
L1	Contracts?	PM/Design/Backend API alignment — S6.
L2	When tab wins?	Primary destination users return to constantly.
L3	Deep flow?	Don't force wizards into sheets.

Provenance: Verified · S6 · product UI

T6. Representable update called too often — perf? (90–120s)

Answer points (frame first):

- Reduce state churn
- Equatable inputs
- Avoid recreating VC
- Heavy work out of update
- Identity stability

Agenda opener: "I'd connect SwiftUI invalidation to UIKit cost..."

Full spoken answer:

"Representable update spam means the SwiftUI parent is invalidating too broadly. Reduce observed state, prefer Equatable inputs where measured useful, keep identity stable so you update instead of remake, and move heavy work out of update. This is the same identity/performance story Day 12 deepens — hybrid makes it visible because UIKit recreate is expensive."

Common wrong answer: Ignore — SwiftUI always calls update.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Players restart?	Identity churn — stabilize.
L2	Logging?	Count make vs update in DEBUG.
L3	S13?	Interop perf is design.

Provenance: S13 + Day 12 bridge

T7. Cell starts URLSession that finishes after scroll away. (90s)

Answer points (frame first):

- Cancel on reuse/disappear
- Correlate request ID with model ID
- Optional repository coalescing
- Same as search cancel

Agenda opener: "I'd cancel at the cell and validate on apply..."

Full spoken answer:

"Cancel the task in prepareForReuse and when the screen disappears. Correlate the request with a model ID or generation token before assigning results. Repository-level coalescing can help but doesn't replace reuse hygiene. It's the same race pattern as cancelling in-flight search — stale work must not win."

Common wrong answer: Only blame the image library.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Day 09?	Cancellation participates in Task.
L2	Prefetch?	Cancel paths too.
L3	Error UI?	Don't flash errors for cancelled.

Provenance: Learning-lab · cancel + S3 sibling pattern

T8. How do push + universal links share one router without races? (90s)

Answer points (frame first):

- Typed intents
- Inbox until ready
- Deduplicate identical intents
- Single owner presents

Agenda opener: "I'd queue and dedupe..."

Full spoken answer:

"Both Airship taps and universal links become typed intents into one inbox. Until the root is ready, intents queue. I'd dedupe identical intents in a short window to avoid double present if both systems fire. Only the router presents. That keeps Mixpanel screen events aligned with what users actually saw."

Common wrong answer: Present immediately from each callback.

Follow-up ladder:

Level	Ask	Point + short answer
-------	-----	----------------------

L1	Foreground vs cold?	Same intent; different ready timing.
L2	Auth?	Router holds pending post-login.
L3	S13?	Push + deeplink one story.

Provenance: Verified · S13 · engagement nav

05 — Exercises

A. Conceptual

A1. Lifecycle placement table

For each: start video, refresh list, screen analytics, cancel search — pick hooks.

Solution: start video → `didAppear+visibility`; refresh → `willAppear` (throttled); analytics → `didAppear/disappear`; cancel search → `willDisappear`.

A2. T/F

1. `viewDidLoad` runs every tab switch.
2. `prepareForReuse` should cancel image tasks.
3. Dual `NavigationPath` + `UINavigationController` is fine if you sync hard.
4. S6 metric is 30%+ fewer full-screen navs.
5. Pause ads only in `deinit`.

Solution: 1F 2T 3F 4T 5F

A3. Provenance rewrite

"I rebuilt Grizzlies in pure SwiftUI and cut navigation 50%."

Solution: "I architected SwiftUI↔UIKit interop on Grizzlies with deeplink ownership plus Mixpanel/Airship. Separately, on BookMyShow the LE Bottom Sheet reduced full-screen navigations for 30%+ of user flows — I don't claim a 50% Grizzlies nav metric or a pure-SwiftUI rewrite."

B. Coding

B1. `CellReuseGuard`

Explain why `generation` token is updated in `prepareForReuse`.

Solution: Invalidates in-flight completions tied to prior bind.

B2. `PrefetchBudget`

What happens if cancel isn't called?

Solution: Tasks keep running → data/battery burn.

B3. `DeeplinkInbox`

Write a sequence: `enqueue` before `ready`, `markReady`, `assert` order.

C. Speaking

S13 20s; S6 20s; S6 full 2–3m; T3 dual nav 120s; T5 tab vs sheet 90s.

D. Whiteboard

Deeplink → router → UIKit host vs SwiftUI host; annotate cold-start queue.

E. Timed drill

Q7, Q8, Q5 + T3, T5. Timing guide. Log dual-nav + hosting sizing misses.

F. Flashcards

Front	Back
Pause video	willDisappear/offscreen · S1
Reuse	prepareForReuse · wrong image
Prefetch	cancel + budget
Hosting	UIHostingController · S13
Representable	make/update · stable id
Deeplink	single router · S13
Sheet win	30%+ · S6
Dual stack	forbidden

G. Deeper

1. Visibility threshold policy paragraph for in-feed ads.
2. List three Mixpanel double-count risks in hybrid hosting.
3. Sketch LE sheet API: inputs, dismiss, analytics events.

H. Scenario drills

H1. In-feed ad visibility (90s speak)

Write a threshold policy: play above 50% visible for 250ms; pause below 50% or on VC disappear or background.

H2. Cold-start deeplink + push simultaneously

Both fire for the same game screen. How does DeeplinkInbox + dedupe behave?

Solution sketch: Queue until ready; identical route intents within short window collapse to one present; analytics once.

H3. LE sheet API sketch

```
LEBottomSheet(eventId, summaryDT0, onOpenTickets, onShare, onDismiss)
detents: [.medium, .large]
analytics: le_sheet_open / cta / dismiss
```

H4. Hosting height war story (60s)

Speak: intrinsic bridging, one scroll owner, avoid recreating hosting VC.

H5. Cell closure cycle diagram

Draw VC → CV → Cell → closure → VC; mark weak + prepareForReuse clear.

Day 12 — SwiftUI State, Identity, @Observable, Lists, Performance

Week 2 · Full study (self-contained) · ~4–5 hrs
Revision twin: [revision/weeks/week-02/day-12.md](#)

Outcomes

By end of day, without notes, you can:

- Apply **state ownership** rules: `@State` , `@Binding` , `@Observable` / `@Bindable` , `Environment` — what belongs where
- Explain **identity** (structural vs explicit `.id` , `@State` lifetime) and why UUID-in-body bugs happen
- Build **performant lists** (lazy stacks, stable IDs, avoid invalidation storms)
- Note **@Observable is iOS 17+**; know `ObservableObject` for legacy interviews
- Deliver **S10** Stories SDK: public API, host isolation, stable page identity, pause on disappear

How to study (in Cursor only)

1. `01-foundations.md` — mental model + glossary
2. `02-deep-dive.md` — identity, Observation, lists, SDK state machine
3. `03-production-bridge.md` — Verified S10 + honest API claims
4. `code/` — `StoriesPlayerModel.swift` , `IdentityTraps.swift` , `LazyListNotes.swift`
5. `04-questions.md` — Answer points → full spoken
6. `05-exercises.md`
7. Revision twin

Module map

Module	File
Foundations	01-foundations.md
Deep dive	02-deep-dive.md
Production bridge	03-production-bridge.md
Questions	04-questions.md
Exercises	05-exercises.md
Code	code/StoriesPlayerModel.swift , code/IdentityTraps.swift , code/LazyListNotes.swift

Provenance reminder

Label	Meaning for today
Verified · S10	Stories SDK — standalone reusable; clear public API; isolation from app-specific networking where possible; portfolio reuse (Miami Heat / Raw)
Verified · S13	Hybrid hosting identity sibling (Day 11)
Learning-lab	Player model / identity / list snippets

Do **not** claim a specific third-party image library as required; do not invent portfolio install counts.

Agenda opener

“State ownership, then identity, then list performance, then Stories SDK API isolation with S10.”

Time budget

Block	Minutes
Foundations	45–60
Deep dive + code	90–120
Production bridge + S10 STAR	30–40
Questions	45–60
Exercises	30–40
Revision skim	15

Source / version note

`@Observable` / **Observation framework: iOS 17+**. For older deployment targets, `ObservableObject` + `@Published` remains interview-relevant. This chapter is self-contained; optional citations at end of deep dive.

01 — Foundations: SwiftUI State & Identity Mental Model

Intern → mid. Read before deep dive.

1. Plain-English mental model

SwiftUI is a **state → UI** engine. You describe views as a function of state; the framework diffs and updates. Most “SwiftUI is buggy” reports are **state in the wrong place** or **identity that resets**.

Think of a theater:

Theater	SwiftUI
Script (what’s true)	Your model / <code>@State</code> / <code>@Observable</code>
Stage directions	<code>body</code> — cheap description
Actor’s continuity	Identity — same character across scenes
New casting each night	<code>.id(UUID())</code> — state amnesia

If you recast (new identity) every frame, the actor forgets their lines (`@State` resets), animations restart, and representables remake (Day 11 pain).

2. Glossary

Term	Meaning
<code>@State</code>	View-owned private storage; lifetime tied to view identity
<code>@Binding</code>	Two-way reference to someone else’s state
<code>@Observable</code>	iOS 17+ Observation macro for models; tracks property access
<code>@Bindable</code>	Bridge for bindings into observable fields

ObservableObject / @Published	Combine-era observation (pre-Observation / legacy)
Environment	Values propagated down the tree (theme, locale, DI-lite)
Structural identity	Same position/type in view tree ⇒ same identity
Explicit identity	<code>.id(...)</code> / <code>ForEach(id:)</code>
Invalidation	SwiftUI re-asking body because depended state changed
Invalidation storm	Giant observed object → huge subtree redraws
Lazy container	<code>List</code> / <code>LazyVStack</code> — build views on demand
Eager VStack	Builds all children — fine for tiny; death for thousands
Stable ID	Model key that doesn't change across updates
Stories page	One story unit in a group; identity must survive progress ticks

3. State ownership table (memorize)

Tool	Owner	Use
@State	View (private)	Ephemeral UI — toggle chrome, local draft text
@Binding	Child writes parent	Controls
@Observable model	Often VM / feature model	Async + shared screen state (iOS 17+)
@Bindable	Bridge	Forms into observable fields
Environment	Tree-wide	Theme, layout direction — not every NetworkClient
ObservableObject	Legacy path	Know for interviews / older OS

Rule: View-local for pure UI; hoist async/domain to observable VM (Day 08). Don't duplicate every toggle into VM.

4. Identity — the senior differentiator

SwiftUI recognizes "same view" by identity:

- **Structural:** type + place in hierarchy
- **Explicit:** `.id("profile")` / `ForEach(stories)` with stable `Identifiable`

@State lifetime follows identity. Change `id` → state resets.

Classic bug: `.id(UUID())` inside `body` → new identity every render → text fields clear, timers restart, Stories page resets mid-swipe.

Stories SDK implication (S10): page identity must be stable across progress updates; don't regenerate UUIDs each render.

5. Intern path: happy Stories player

1. Host app supplies story groups via data source protocol.
2. `StoriesPlayerModel` (`@Observable`) owns timeline: idle → loading → playing → paused → finished.
3. Views render progress from model — not ad-hoc view timers alone.
4. `onDisappear` / scene phase → pause AV + timers.
5. Adjacent media prefetch carefully (budget — Day 11 lesson).
6. Host gets callbacks: open/close/CTA — injectable analytics.

6. Lists — intern path

Do	Don't
List / LazyVStack for large data	VStack of 10k rows
Stable Identifiable	id: \.self on mutating values
Cheap body	Format JSON in every row body
Async images with size budgets	Decode huge images on main in body

7. @Observable version note (say this)

"For new code on iOS 17+ I prefer @Observable . On older deployment targets I'd use ObservableObject and @Published . In interviews I can explain both."

8. Environment DI — light touch

OK	Risky
Theme, color scheme	Sole hidden NetworkClient
Layout direction	Service-locator EnvironmentKey soup
Feature-scoped store passed deliberately	SDK forcing host AppModel

S10: SDK prefers **explicit injectable** networking/image loading protocols.

9. Production anchor — S10

Standalone reusable Stories SDK; clear public API; isolation from app-specific networking where possible; adopted across portfolio apps. SDK quality = API surface + versioning + independence from host shortcuts.

10. Self-check

1. @State vs @Observable placement
2. Why UUID in .id breaks state
3. Lazy vs eager lists
4. iOS 17+ note for Observation
5. Stories pause + stable page IDs

11. Flash preview

Front	Back
@State	Local ephemeral UI
@Observable	Feature/async model · iOS 17+
Identity	Stable IDs preserve state
Lazy	Large collections
S10	SDK API + isolation
Trap	.id(UUID()) in body
'''	

12. Observation access tracking (intuition)

With `@Observable` , SwiftUI tracks which properties `body` read. Changing an unread property shouldn't invalidate that view. Stuffing unused fields onto a model the root reads widely still causes storms — split models.

Legacy `ObservableObject` often broadcasts more coarsely via `objectWillChange` .

13. Stories UI chrome vs domain

State	Placement
Progress 0...1	Player model
Phase playing/paused	Player model
"Hold educational tooltip seen"	@State or lightly persisted host pref
CTA destination	Host callback — SDK doesn't push tickets VC
Image bytes	Injectable loader

14. 90-second teaching script

"SwiftUI state belongs either view-local or in an observable feature model — `@Observable` on iOS 17+, `ObservableObject` when you must support older OS. Identity preserves `@State` and representables; `UUID-in-body` resets everything. Lists need lazy containers and stable IDs. The Stories SDK I built exposes a clear API with injectable loading and host isolation so portfolio apps share one player — pause on disappear, stable page ids, progress owned by the model."

02 — Deep Dive: Observation, Identity, Lists, Stories SDK

Senior depth. Assumes [01-foundations.md](#). Self-contained.
Version: `@Observable` requires **iOS 17+**.

1. State ownership decision tree

Is it pure ephemeral UI for one view?
yes → `@State`
no ↓
Do child controls need to write it?
yes → `@Binding` into parent/`@Bindable` model
no ↓
Is it screen/feature async or shared across children?
yes → `@Observable` model (iOS 17+) / `ObservableObject` (legacy)
no ↓
Is it tree-wide theme/locale?
yes → `Environment`
no → You're over-abstracting — keep it simple

Anti-patterns

Anti-pattern	Fix
Every toggle in VM	Keep chrome <code>@State</code>
Fetch inside body	Model/Task on appear; never side-effect in body
One <code>AppModel</code> for entire app	Split by feature — invalidation storms

Environment NetworkClient only	Explicit init for testable/SDK seams
--------------------------------	--------------------------------------

2. Observation vs ObservableObject

	@Observable (iOS 17+)	ObservableObject
Tracking	Property access fine-grained	Often objectWillChange whole object
Boilerplate	Macro @Observable	: ObservableObject + @Published
Bindings	@Bindable	\$ projected values
Deployment	iOS 17+	Older OS

Interview sentence: Prefer Observation on modern targets; still explain @Published when asked about legacy codebases.

Granularity and storms

A monolithic observable with 50 fields used by a root view can invalidate huge trees when any field changes. Split models (player vs chrome vs catalog). Pass slices into children.

3. Identity mechanics

See [code/IdentityTraps.swift](#).

Structural identity

```
VStack {
  Header() // structural position 0
  Content() // structural position 1
}
```

Swap order conditionally without IDs → SwiftUI may treat views as different → state jumps.

Explicit identity

```
ForEach(pages) { page in // Identifiable stable id
  StoryPageView(page: page)
}
```

Forced new identity (intentional)

Logout → .id(userID) so forms reset. **Intentional** reset ≠ accidental UUID churn.

Representable link (Day 11)

Parent identity churn → makeUIViewController storms → players restart. Stabilize IDs; update props.

4. Lists & performance

See [code/LazyListNotes.swift](#).

Container choice

Container	Use
List	Platform list behaviors, edit modes

LazyVStack in ScrollView	Custom scrolls, large stacks
Eager VStack	Tiny static content only

Row rules

1. Stable `Identifiable` — never `UUID()` per `body`
2. Avoid `id: \.self` when value equality changes often
3. Don't observe entire catalog inside each row — pass row models
4. Precompute formatted strings in model
5. Images: `async` + `decode/size` budgets (Week 3)
6. Scope animations — don't `.animation` the universe

When Lazy still janks

Profile: image decode, main-thread JSON, overlapping observations, expensive shadows, unbounded prefetch. Fix **data path**, not only container choice.

5. `body` purity

`body` is called often — that's normal. Problem is **heavy work** or **side effects** inside.

OK in <code>body</code>	Not OK
Compose views from state	Start network
Cheap formatting already cached	Sort 10k rows every call
Branch on enum state	Write to disk

6. Animation & transitions

- Prefer identity-preserving updates for smooth animation
- Destroy/recreate (identity change) feels like jump cuts
- Drive Stories progress from **model timeline**, not scattered `onAppear` timers
- Use explicit `withAnimation` / transactions for intentional motion

7. Stories SDK design (\$10 shape)

See [code/StoriesPlayerModel.swift](#).

Public API sketch

```

StoriesPlayer
├─ DataSource protocol (host supplies groups/pages)
├─ ImageLoading / VideoLoading protocols (injectable)
├─ Event callbacks (onOpen, onClose, onCTA, onPage)
├─ Theming hooks
└─ Versioned module boundary

```

State machine

```

idle → loading → playing ⇄ paused → finished
                ↓
                failed (retry)

```

Pause on: `onDisappear`, scene background, user hold — same lifecycle discipline as Ads video (S1 cousin).

Why not hardcode networking?

Portfolio apps differ (Heat / other Raw apps). Injectable clients → testability + host variance. SDK quality = independence from host shortcuts.

UIKit hosts

SwiftUI-first OK; offer `UIHostingController` façade for legacy. Public API shouldn't force one nav paradigm (S13 hosts).

8. SDUI leaf identity (Day 10 crossover)

Registry views should use **server-stable node ids** for `ForEach` / `.id`. Array indices break when CMS inserts a banner above.

9. Testing SwiftUI logic

Layer	How
Observable model / UseCase	XCTest — primary
Snapshots	Critical chrome sparingly
UITests	Open/close Stories golden path
ViewInspector etc.	Optional; don't rely exclusively

Theater UITests with `sleep` are AI smell (Day 08/S9 culture).

10. Equatable View — micro-opt

Useful for expensive rare-changing subtrees. Don't sprinkle early. Prefer smaller observed state first. Measure.

11. Trade-offs

Choice	When	Cost
<code>@Observable</code> VM	Feature screens iOS 17+	Over-observation if monolithic
All <code>@State</code>	Tiny UI	Untestable async spaghetti
<code>AnyView</code> erasure	Rare need	Kills optimization / clarity
Eager <code>VStack</code>	Tiny lists	Jank at scale
SDK owns networking	Convenience	Couples hosts
<code>.id(UUID())</code> per body	Never	Resets state every frame
SwiftUI-only SDK	Modern hosts	Legacy need façade

12. Whiteboard scripts

- A. State ownership table + one wrong placement
- B. Identity: UUID trap vs stable page ids
- C. Stories SDK boundary + state machine + pause

Agenda: README opener.

13. Failure modes

Failure	Cause	Fix
---------	-------	-----

Text clears while typing	Identity churn	Stable id
Progress bar desync	Timers in views / id reset	Model timeline
List jump	Unstable ForEach ids	Stable Identifiable
Whole screen redraws	God observable	Split models
Representable remake	Parent id churn	Stabilize; update props
Background audio	No scene-phase pause	Pause policy

14. Decision rule card

1. Pure UI → @State; async/domain → @Observable (iOS 17+)
 2. IDs stable unless intentional reset
 3. Large lists → lazy + cheap body + stable ids
 4. Split models to avoid invalidation storms
 5. SDK: protocols in, events out, pause on disappear
 6. Say the iOS 17+ caveat aloud

15. Optional citations

Apple State and Data Flow; Observation framework; WWDC Demystify SwiftUI / SwiftUI performance; `app-modularization.md` — appendix only.

16. Bridge

→ [03-production-bridge.md](#)

03 — Production Bridge: Stories SDK (S10)

Interview stories without overclaiming.

1. Provenance map

ID	Label	Exact claim
S10	Verified	Designed standalone reusable Stories SDK ; clear public API ; isolation from app-specific networking where possible ; adopted across portfolio apps (Raw / Miami Heat)
S13	Verified	Hybrid hosts may embed SDK via hosting — identity/lifecycle sibling
S1	Verified	Pause/play lifecycle cousin for media
Learning-lab	Illustrative	Player model / identity samples

Forbidden

- Invented install counts / “N apps” exact beyond “portfolio”
- “SDK hardcodes Kingfisher/Alamofire only” as verified
- Claiming SwiftUI Observation macros as a resume bullet for S10
- Conflating S10 tooling with S9 District AI story

2. S10 STAR (2–3 min)

Opener (~10s)

"I'll walk through building a reusable Stories SDK — stable public API and host isolation — so multiple NBA/WNBA apps shared one Instagram-style stories implementation."

Situation / Task

Need Instagram-style fan Stories across a client portfolio — not one-off UI per app.

Action

1. Designed a **standalone reusable Stories SDK**.
2. Clear **public API; isolation from app-specific networking** where possible via injectable boundaries.
3. Drove adoption across portfolio apps with shared implementation.

Result

One implementation leveraged by multiple apps → faster feature parity.

Lesson

SDK quality = API surface + versioning + independence from host app shortcuts. Stable identity/state for pages/progress is part of that quality.

Provenance: *Verified · S10 · Raw/Miami Heat · Stories SDK portfolio reuse*

3. Technical beats you may elaborate (honest)

Beat	How to speak
Public API	Entry player, data source, events — learning-lab shape OK if labeled illustrative
Injectable loaders	Design judgment aligned with “isolation from app networking”
Pause on background	Lifecycle discipline — cousin to S1; don’t invent Aces metrics
Stable page IDs	Senior SwiftUI correctness — teaching + SDK necessity
@Observable	Teaching for modern hosts; not a verified S10 resume API name

4. Interview line ≤20s

"I built a reusable Stories SDK — clear public API and host isolation — so multiple NBA/WNBA portfolio apps shared one stories implementation."

5. Cross-app reuse challenges (speak as design)

Theming, analytics hooks, media formats, nav/CTA exits, dependency versions — solve with protocols + defaults. Modularization Week 3/Day 15 deepens SPM, but S10 is the product proof.

6. Interviewer pushes

Push	Strong reply
"Why not copy-paste UI?"	Parity + bugfix cost; one SDK.
"Why not SDK own network?"	Hosts differ; testability; isolation claim.
"SwiftUI-only?"	SwiftUI-first + UIKit hosting façade for legacy.
"Progress desync?"	Model timeline + stable identity — not more view timers.

"UUID ids?"	Never in body; stable model keys.
-------------	-----------------------------------

7. Practice

60s: portfolio need → SDK API + isolation → reuse → lesson API quality.

3 min: add state machine, pause policy, identity, injectable image/video, hybrid host note.

8. Links

- Code: [code/](#)
- Questions: [04-questions.md](#)
- Revision: [../../revision/weeks/week-02/day-12.md](#)
- Story: [S10#s10--stories-sdk-raw--miami-heat](#)

04 — Questions (two-layer answers)

*Study flow: cover the full spoken answer → speak from **Answer points** only → uncover and compare timing.*

*Totals: **12 normal + 10 tricky = 22.***

Normal questions

Q1. @State vs @Observable — when each? (30–45s)

Answer points (frame first):

- @State local ephemeral UI
- @Observable feature/async shared state
- @Bindable for forms
- @Observable is iOS 17+
- Know ObservableObject legacy

Agenda opener: "I'd place state by ownership, and note the OS version..."

Full spoken answer:

"I use @State for view-local ephemeral UI like chrome toggles. For feature-level async and state shared across children I use an @Observable model on iOS 17+, with @Bindable when controls write into that model. On older deployment targets I'd use ObservableObject and @Published. Stories player timeline belongs in the model; local overlay chrome can stay @State."

Common wrong answer: Put everything in @State or everything in a VM.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Legacy codebases?	Explain @Published + objectWillChange.
L2	EnvironmentObject?	Avoid god bags.
L3	Day 08?	MVVM still valid with Observation.

Provenance: Learning-lab · iOS 17+ note; S10 player

Q2. What is view identity? (45–60s)

Answer points (frame first):

- How SwiftUI recognizes same view across updates

- Structural vs explicit .id
- @State tied to identity
- Stable IDs preserve state

Agenda opener: "I'd define identity before blaming SwiftUI bugs..."

Full spoken answer:

"View identity is how SwiftUI decides two renders are the same view. Structural identity comes from type and position; explicit identity uses .id or ForEach identifiers. @State storage follows identity — change the id and state resets. That's why stable Stories page IDs matter and why UUID-in-body is catastrophic."

Common wrong answer: Identity means Equatable views only.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Force new identity?	Intentional reset on logout.
L2	Representable?	Parent id churn remakes VC — S13.
L3	S10?	Page identity must survive progress ticks.

Provenance: Learning-lab · identity; S10 pages

Q3. Why did my Representable reset? (45s)

Answer points (frame first):

- Parent identity changed
- id churn → remake
- Stabilize IDs
- Update props instead of recreate

Agenda opener: "I'd suspect identity before UIKit..."

Full spoken answer:

"Usually the parent's identity changed or an .id churned, so SwiftUI remade the representable instead of calling update. Stabilize identifiers and push prop changes through updateUIViewController. Recreate only when you truly need a fresh controller — hybrid apps feel this cost immediately."

Common wrong answer: SwiftUI randomly remakes controllers.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Day 11 link?	Hybrid hosting.
L2	Players restart?	Classic remake symptom.
L3	Debug?	Log make vs update counts.

Provenance: Verified · S13 adjacent

Q4. ForEach best practices? (45s)

Answer points (frame first):

- Stable Identifiable
- Avoid indices when reordering

- Don't create new IDs in body
- Duplicate IDs are undefined weirdness

Agenda opener: "I'd treat IDs as product keys..."

Full spoken answer:

"ForEach needs stable Identifiable keys from your models. Don't use indices when rows reorder, and never mint UUID ids inside body. Duplicate IDs cause undefined weirdness. Stories pages should carry host/server ids that survive progress updates."

Common wrong answer: id: .self always.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Value type changes equality?	Prefer explicit id property.
L2	CMS lists?	Server-stable ids — Day 10.
L3	Diffable cousin?	Same identity lesson UIKit.

Provenance: Learning-lab · S10 pages

Q5. Make a long list smooth. (60–90s)

Answer points (frame first):

- Lazy containers
- Async images
- Cheap body
- Stable IDs
- Pagination
- Avoid observing whole catalog per row

Agenda opener: "I'd start with lazy + identity + cheap rows..."

Full spoken answer:

"Use List or LazyVStack, keep Identifiable IDs stable, make row body cheap with precomputed formatting, load images asynchronously with size budgets, paginate when needed, and don't observe a giant catalog from every row. If it's still janky, profile decode and main-thread work — Lazy alone isn't a silver bullet."

Common wrong answer: Only wrap in Lazy and ship.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Instruments?	SwiftUI + Time Profiler.
L2	SDK consumers?	Portfolio scale — S10 mindset.
L3	Eager VStack?	Tiny lists only.

Provenance: Learning-lab · list perf

Q6. How do you design Stories SDK API? (60–90s)

Answer points (frame first):

- Player entry
- DataSource protocol

- Event callbacks
- Injectable image/video loader
- Versioned module
- Host supplies DTOs

Agenda opener: "I'd design protocols in, events out..."

Full spoken answer:

"I'd expose a `StoriesPlayer` entry point, a data source protocol so hosts supply story groups, event callbacks for open/close/CTA, and injectable image/video loaders so networking stays host-owned where needed. The module is versioned and theming-hooked. That's how we kept a reusable Stories SDK isolatable across portfolio apps instead of hardcoding one app's networking."

Common wrong answer: Hardcode host URLs inside the SDK.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Why not hardcode net?	Portfolio variance + tests — S10.
L2	UIKit host?	Hosting façade OK.
L3	Analytics?	Callbacks; host decides Mixpanel.

Provenance: Verified · S10 · public API / isolation

Q7. Pause stories on background/disappear? (45s)

Answer points (frame first):

- Scene phase / onDisappear
- Pause timers & AVPlayer
- Explicit resume policy
- Same discipline as Ads video

Agenda opener: "I'd put pause in the model lifecycle..."

Full spoken answer:

"On disappear and background scene phase I pause the player model — timers and AVPlayer — with an explicit resume policy when returning. Views shouldn't keep rogue timers alive. It's the same lifecycle discipline as Ads HeroWidget pause/play: media without visibility policy wastes resources and surprises users."

Common wrong answer: Let stories run forever in background.

Follow-up ladder:

Level	Ask	Point + short answer
L1	S1 parallel?	Yes — media lifecycle contract.
L2	Hold to pause?	User intent → <code>model.pause()</code> .
L3	S10?	SDK must expose/handle pause.

Provenance: Verified · S10 + S1 cousin

Q8. Environment for DI — good idea? (45s)

Answer points (frame first):

- Good for theme/shallow deps
- Bad as hidden NetworkClient locator
- SDK prefers explicit init
- Day 08 DI link

Agenda opener: "I'd allow Environment narrowly..."

Full spoken answer:

"Environment is great for theme and layout direction. It's a poor service locator for NetworkClient everywhere — hidden dependencies hurt tests and SDK hosts. Stories SDK should take injectable loaders through init so hosts see the graph. That matches Day 08's constructor DI default."

Common wrong answer: Put all services in Environment.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Feature store in Environment?	OK if deliberate and scoped.
L2	Testing?	Init fakes beat Environment surgery.
L3	S10?	Injectable deps verified direction.

Provenance: Verified · S10 · isolation; Day 08 DI

Q9. Animation causes list jump — causes? (45–60s)

Answer points (frame first):

- Identity changes
- Height changes without transaction
- Diff mismatch
- Scroll position loss
- Stable IDs + scoped animation

Agenda opener: "I'd debug identity and height first..."

Full spoken answer:

"List jumps usually come from identity changes, row height changes without a careful transaction, or scroll position loss when IDs reshuffle. Fix stable Identifiable keys, animate data changes carefully, and avoid applying animation modifiers to entire giant trees."

Common wrong answer: SwiftUI scrolling is just broken.

Follow-up ladder:

Level	Ask	Point + short answer
L1	.animation on root?	Scope it.
L2	Images loading?	Reserve height/placeholders.
L3	Stories?	Progress shouldn't change page id.

Provenance: Learning-lab · animation/identity

Q10. body called often — is that a problem? (45s)

Answer points (frame first):

- Frequent calls expected
- Problem is heavy work inside
- Keep body cheap
- No side effects
- Move work to model

Agenda opener: "I'd normalize frequent body, attack cost..."

Full spoken answer:

"body being called often is normal — SwiftUI diffs descriptions. It becomes a problem when body does heavy formatting, sorting, or side effects like networking. Keep body cheap and pure-ish; precompute in the model. Fetching in body is an interview classic fail."

Common wrong answer: Cache body manually with ugly globals.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Logging body?	Debug counts OK; don't ship noise.
L2	Observation?	Depend on fewer properties.
L3	AnyView?	Can obscure optimization.

Provenance: Learning-lab · performance classic

Q11. Cross-app reuse challenges for Stories? (45–60s)

Answer points (frame first):

- Theming
- Analytics hooks
- Media formats
- Nav/CTA exits
- Dependency versions
- Protocols + defaults

Agenda opener: "I'd list variance, then seams..."

Full spoken answer:

"Portfolio reuse hits theming, analytics, media formats, how CTAs exit into host navigation, and dependency versions. You solve it with protocols, sensible defaults, and a versioned module — not by forking the SDK per team. That's the modularity lesson behind the Stories SDK adoption across apps."

Common wrong answer: Copy-paste and rename HeatStories.

Follow-up ladder:

Level	Ask	Point + short answer
L1	SPM preview?	Day 15 modularization.
L2	Breaking API?	Semantic version + migration notes.
L3	S10?	Portfolio reuse verified.

Provenance: Verified · S10 · portfolio reuse

Q12. SwiftUI + SDUI registry? (45–60s)

Answer points (frame first):

- Registry returns SwiftUI views by type
- Leaves dumb
- VM owns payload
- Identity from server id

Agenda opener: "I'd keep leaves dumb and ids stable..."

Full spoken answer:

"An SDUI registry can return SwiftUI views keyed by type while the ViewModel owns payload lifecycle. Keep leaf views dumb. Give each node a server-stable id so ForEach identity doesn't break when CMS inserts content — same identity rules as Stories pages."

Common wrong answer: Use array indices for CMS nodes.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Day 10?	Unknown skip still applies.
L2	Observation?	Observe document model, not each leaf service.
L3	S3?	Header components as leaves.

Provenance: Day 10 crossover · S3

Tricky questions

T1. I put UUID in .id inside body — why is state broken? (90s)

Answer points (frame first):

- New identity every render
- Resets @State
- Remakes children
- IDs must be stable model keys
- UUID ok only if stored once on model

Agenda opener: "I'd explain identity lifetime..."

Full spoken answer:

"UUID() inside body creates a brand-new identity on every evaluation. SwiftUI treats it as a different view, so @State resets, animations restart, and representables remake. IDs must be stable model keys. A UUID is fine only if the model created it once and stored it — not if you mint it while rendering."

Common wrong answer: Blame SwiftUI bugs.

Follow-up ladder:

Level	Ask	Point + short answer
L1	When UUID ok?	Created once, stored on model.
L2	Stories?	Server/host page ids.
L3	Intentional reset?	.id(sessionID) on logout.

Provenance: Learning-lab · identity trap

T2. One @Observable AppModel for everything. (90–120s)

Answer points (frame first):

- Broad invalidation
- Tight coupling
- Split by feature
- Pass slices
- SDK mustn't depend on host AppModel

Agenda opener: "I'd reject the god object..."

Full spoken answer:

"A single AppModel is convenient until every property change invalidates half the UI and every feature couples together. Split by feature, pass slices into children, and never make a Stories SDK depend on the host's AppModel. Observation's granularity helps only if you don't stuff the universe into one type."

Common wrong answer: Convenient architecture forever.

Follow-up ladder:

Level	Ask	Point + short answer
L1	S10 isolation?	SDK protocols, not AppModel.
L2	Migration?	Extract feature models incrementally.
L3	EnvironmentObject same sin?	Yes — god bag.

Provenance: Learning-lab · S10 isolation

T3. LazyVStack still janky. (90–120s)

Answer points (frame first):

- Profile image decode
- Main-thread JSON
- Overlapping observations
- Complex effects
- Unbounded prefetch
- Fix data path

Agenda opener: "I'd profile beyond the container..."

Full spoken answer:

"If LazyVStack still janks, the container isn't the only culprit. I'd profile image decode, main-thread parsing, overlapping observations, expensive materials/shadows, and unbounded prefetch. Fix the data path and observation scope. Switching to List without fixing decode often changes nothing meaningful."

Common wrong answer: Only 'use List instead.'

Follow-up ladder:

Level	Ask	Point + short answer
L1	Week 3?	Image pipeline.
L2	Instruments?	Time Profiler + SwiftUI.
L3	Prefetch?	Day 11 budgets.

T4. Should Stories SDK be SwiftUI-only? (90s)

Answer points (frame first):

- SwiftUI-first OK
- UIKit hosting façade for legacy
- Public API shouldn't force nav paradigm
- Hosts vary

Agenda opener: "I'd be SwiftUI-first, not SwiftUI-only dogma..."

Full spoken answer:

"SwiftUI-first is fine if hosts can host it, but a portfolio SDK should offer a UIKit façade via UIHostingController for legacy screens. The public API shouldn't force one navigation paradigm — Grizzlies-style hybrid hosts need flexibility. Being SwiftUI-only without an escape hatch slows adoption."

Common wrong answer: Binary SwiftUI-only forever.

Follow-up ladder:

Level	Ask	Point + short answer
L1	S13?	Hybrid hosts embed SDK.
L2	API surface?	Player + data source unchanged.
L3	Tests?	Model tests don't need UI.

Provenance: Verified · S10 + S13 host reality

T5. Equatable View conformance — worth it? (90s)

Answer points (frame first):

- Useful for expensive rare-changing subtrees
- Don't sprinkle early
- Prefer smaller observed state
- Measure first
- Know Observation interaction

Agenda opener: "I'd treat it as measured micro-opt..."

Full spoken answer:

"Equatable View can help expensive subtrees that rarely change, but sprinkling it everywhere early is noise. Prefer narrowing observation first. Measure before and after. Understanding how Observation already tracks accesses matters more than ritual Equatable."

Common wrong answer: Micro-optimize all views on day one.

Follow-up ladder:

Level	Ask	Point + short answer
L1	When yes?	Heavy chart leaf, rare updates.
L2	When no?	Simple Text rows.
L3	AnyView?	Usually worse for clarity.

T6. How do you test SwiftUI state logic? (90s)

Answer points (frame first):

- Test observable models/UseCases in XCTest
- Snapshots optional
- UITests for critical open/close
- Avoid sleepy flaky tests

Agenda opener: "I'd test the model first..."

Full spoken answer:

"I unit-test observable models and UseCases in XCTest — that's where Stories phase transitions and pause rules live. Snapshots are optional for chrome. UITests cover critical open/close paths only. Flaky sleep-based UITests are a smell — same testing culture as District's review gates."

Common wrong answer: Only UITests.

Follow-up ladder:

Level	Ask	Point + short answer
L1	S9 link?	Tests as gate, not theater.
L2	ViewInspector?	Optional aid.
L3	Async?	Swift Testing / XCTest async.

Provenance: Learning-lab · S9 culture applied

T7. Progress bar desyncs when paging stories. (90–120s)

Answer points (frame first):

- Single timeline in model
- View renders progress
- Page identity mustn't reset timer state
- Pause/resume explicit
- State machine > scattered onAppear timers

Agenda opener: "I'd centralize time in the model..."

Full spoken answer:

"Desync usually means multiple timers or identity resets mid-page. Keep a single timeline in the player model; views only render progress. Page identity must stay stable across ticks. Pause and resume are explicit events. Scattered onAppear timers are how progress bars lie."

Common wrong answer: Add more timers in views.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Advance on complete?	Model.advance().
L2	Hold gesture?	pause()/resume().
L3	S10?	SDK owns this correctness.

T8. Host app injects different image pipelines. (90s)

Answer points (frame first):

- ImageLoading protocol in SDK
- Hosts adapt
- Default implementation optional
- Same DI lesson as networking

Agenda opener: "I'd inject loaders at the boundary..."

Full spoken answer:

"The SDK defines an ImageLoading protocol; hosts adapt their pipeline — whether custom URLSession or a library — and a default can ship for demos. That preserves isolation from app-specific networking and keeps tests fakeable. It's the same DI lesson as Day 08/09: protocols at boundaries."

Common wrong answer: SDK bundles one library only, forever.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Video?	Separate VideoLoading protocol.
L2	S10?	Isolation where possible — verified.
L3	Versioning?	Protocol evolution carefully.

Provenance: Verified · S10 · injectable isolation

T9. @Observable is iOS 17+ — how do you talk deployment? (60–90s)

Answer points (frame first):

- State the caveat
- Observation for modern targets
- ObservableObject for older
- SDK may abstract observation
- Don't pretend macros run on iOS 15

Agenda opener: "I'd lead with the version constraint..."

Full spoken answer:

"I'd say explicitly: @Observable and the Observation framework need iOS 17+. For an app still supporting earlier OS versions, I'd use ObservableObject or isolate Observation behind availability. A portfolio SDK might keep models observation-agnostic with callbacks, or offer dual bindings. Pretending macros work on iOS 15 is how interviews catch you."

Common wrong answer: Ignore deployment targets.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Availability?	if #available / separate types.
L2	Interview?	Know both stacks.
L3	S10 hosts?	Portfolio OS matrices differ.

T10. How does Stories SDK avoid becoming a mini-app? (90s)

Answer points (frame first):

- Narrow public API
- No host AppModel dependency
- Callbacks for nav exits
- Theming hooks not full design system fork
- Version boundaries

Agenda opener: "I'd keep the SDK a player, not a platform..."

Full spoken answer:

"A Stories SDK goes wrong when it absorbs navigation, networking, analytics, and design systems. Keep a narrow public API, injectable loaders, callbacks for CTA/nav exits, and theming hooks without forking the host design system. Version the module. That's how reuse stays leverage instead of a second app inside the app."

Common wrong answer: SDK owns entire fan engagement stack.

Follow-up ladder:

Level	Ask	Point + short answer
L1	S10 lesson?	API + independence.
L2	Feature creep?	Host feature flags outside SDK.
L3	Live scoreboard?	Different module — S11.

Provenance: Verified · S10 · modularity judgment

05 — Exercises

A. Conceptual

A1. Ownership placement

Assign: search query debounce state; button pressed highlight; Stories progress; app theme color.

Solution: debounce → observable VM; highlight → @State; progress → player model; theme → Environment.

A2. T/F

1. @Observable works on iOS 15.
2. UUID in .id inside body is fine.
3. Eager VStack of 5k rows is OK.
4. Stories page ids should be stable.
5. Environment is ideal for all DI.

Solution: 1F (iOS 17+) 2F 3F 4T 5F

A3. Provenance rewrite

"I shipped @Observable Stories across 12 apps and cut jank 70%."

Solution: "I designed a reusable Stories SDK with a clear public API and isolation from app-specific networking where possible, adopted across portfolio apps. I'm not claiming an @Observable resume bullet, an exact app count of 12, or a 70% jank metric."

B. Coding

B1. StoriesPlayerModel

Trace pause on disappear; why progress lives in model.

Solution: handleDisappear → pause; single ticker; views render progress.

B2. IdentityTraps

Speak UUID trap in 30s from `explainUUIDTrap()` .

B3. LazyListNotes

Write a 60s list-perf answer from `interviewParagraph` .

C. Speaking

S10 20s; S10 2–3m; T1 UUID 90s; T2 AppModel 120s; T9 iOS 17 caveat 60s.

D. Whiteboard

SDK boundary: DataSource, loaders, events, state machine; mark pause signals.

E. Timed drill

Q2, Q6, Q5 + T1, T2. Timing guide. Log identity + observation scope misses.

F. Flashcards

Front	Back
@State	Local UI · Trap: async everywhere
@Observable	Feature model · iOS 17+
Identity	Stable ids · Trap: UUID body
Lazy	Large lists · Trap: eager thousands
body	Cheap · Trap: fetch in body
S10 API	DataSource + events + inject
Pause	Scene/disappear · S10/S1
God model	Split · Trap: AppModel

G. Deeper

1. Write intentional `.id(sessionID)` logout rationale.
2. Map Day 10 SDUI node id → ForEach.
3. Contrast S9 AI tooling vs S10 SDK modularity in one paragraph (different stories!).

H. Scenario drills

H1. Logout form reset

Explain intentional `.id(userSessionID)` vs accidental UUID.

H2. Portfolio theming

List 4 theming hooks a Stories SDK might expose without owning the host design system.

Solution ideas: primary tint, font provider, corner radius token, progress track colors.

H3. Observation availability matrix

Host min iOS	Approach
17+	@Observable player
15–16	ObservableObject or callback model
Mixed	Availability wrappers / dual modules

H4. Progress desync repro

Steps: mint new page id each tick → observe reset. Fix: stable id; progress field separate.

H5. 45s contrast

S9 = AI tooling envelope at District. S10 = SDK modularity at Raw. Do not conflate.

Day 13 — DSA: Stack / Queue / Linked List

Week 2 · Full study (self-contained) · ~5–6 hrs

Revision twin: <revision/weeks/week-02/day-13.md>

Outcomes

By end of day, without notes, you can:

- Pick **Stack**, **Queue/Deque**, or **Linked List** (and know when Swift `Array` beats a linked list)
- State complexities for push/pop/enqueue/dequeue and classic patterns (monotonic stack, BFS queue, fast/slow)
- Deliver a crisp **2–3 min agenda-first coding approach** before typing Swift
- Implement worked solutions in `code/` (two-stack queue, min stack, reverse/cycle/merge list)
- Close one Week 2 theory gap without dropping Mock #2 prep tomorrow

How to study (in Cursor only)

1. `01-foundations.md` — mental model + glossary (intern → senior)
2. `02-deep-dive.md` — patterns, complexities, Swift pitfalls, approach scripts (**embedded** — no LeetCode required to learn)
3. `03-production-bridge.md` — honest production bridges (refresh waiters, nav stacks)
4. `code/` — read and narrate the Swift before modifying
5. `04-questions.md` — cover full answers; speak from **Answer points**; compare
6. `05-exercises.md` — timed coding + speaking drills
7. Revision twin for flashcards / timed recall

Module map

Module	File
Foundations	01-foundations.md
Deep dive	02-deep-dive.md
Production bridge	03-production-bridge.md

Questions	04-questions.md
Exercises	05-exercises.md
Code	code/

Provenance reminder

Label	Meaning for today
Learning-lab	DSA patterns + Swift in code/ — interview skill, not a shipped BMS feature
Verified · S4 mindset	Soft bridge: single-flight refresh waiters behave like a queue of continuations (Day 09)
Verified · S6 / S13	Soft bridge: nav / sheet back-stacks as LIFO mental models
Verified · S12 adjacency	Ring/circular buffer language adjacent to audio buffering — do not claim you shipped a ring-buffer audio engine

Do **not** invent “I used linked lists in production UI” claims.

Agenda opener (say this first in a coding round)

“I’ll restate the problem, pick the structure and complexity, walk edges, then code iterative Swift — Array stack by default, and I’ll call out if Array-as-queue would be $O(n)$.”

Time budget

Block	Minutes
Foundations	40–50
Deep dive + embedded patterns	90–120
Code narration	45–60
Production bridge	15–20
Questions (record 6–8)	45–60
Exercises (1 Easy + 1 Medium timed)	45–60
Week 2 catch-up (one weak day)	45–60
Revision twin skim	15

Source / version note

Swift 5+ / modern concurrency vocabulary OK when bridging to actors for hit-counter concurrency. Linked-list interview nodes are `class ListNode` — not Swift `Array` . Optional citations only at the end of `02-deep-dive.md` — this chapter is self-contained.

01 — Foundations: Stack, Queue, Linked List

Read this before the deep dive. Goal: an intern-clear mental model, then senior vocabulary.

1. Plain-English mental model

Three ways to arrange work:

Structure	Rule	Kitchen metaphor
Stack	Last in, first out (LIFO)	Plates — take from the top
Queue	First in, first out (FIFO)	Ticket line — serve the front
Deque	Push/pop both ends	Double door — either side
Linked list	Nodes + pointers	Chain of sticky notes — cheap local splice, slow random access

Swift production honesty: You almost always reach for `Array` (and sometimes Swift Collections `Deque`). Linked lists show up in **interview pointer problems** and specialized designs (LRU node list), not in everyday UIKit lists.

2. Glossary (learn these cold)

Term	Meaning
LIFO	Last in, first out — stack
FIFO	First in, first out — queue
push / pop / peek	Stack insert / remove / look at top
enqueue / dequeue	Queue insert rear / remove front
Amortized O(1)	Cheap on average across many ops (e.g. Array growth, two-stack queue)
Monotonic stack	Stack kept strictly increasing or decreasing to answer “next greater/smaller” in O(n)
BFS	Breadth-first search — explore level by level using a queue
DFS	Depth-first — stack or recursion
Dummy head	Sentinel node before the real head to simplify merge/insert edge cases
Floyd cycle	Slow/fast pointers; meeting $\Rightarrow$ cycle
ListNode	Interview class with <code>val</code> + <code>next</code> (reference semantics)
Ring / circular buffer	Fixed array with head/tail indices mod capacity
Agenda-first coding	2–3 min spoken plan before typing

3. Complexity cheat-sheet (say aloud)

Structure	Op	Typical
Array as stack	<code>append</code> / <code>removeLast</code>	Amortized O(1)
Array as queue	<code>removeFirst</code>	O(n) — shifts
Two-stack queue	<code>enqueue</code> / <code>dequeue</code>	Amortized O(1)
Deque / ring	<code>ends</code>	O(1)
Linked list	<code>insert/delete</code> given node	O(1) local
Linked list	<code>access by index</code>	O(n)
Reverse list iterative	—	O(n) time, O(1) space
Floyd	—	O(n) time, O(1) space

4. Pattern → structure map

Pattern	Structure	Canonical problem
Matching / nesting / undo	Stack	Valid parentheses
Next greater / smaller	Monotonic stack	Daily temperatures
Level-order / shortest unweighted	Queue	Binary tree level order
Sliding window max	Monotonic deque	Max in window
Cycle / middle / reverse / merge	Linked list pointers	Floyd, reverse, merge two lists
LRU (preview)	Hash + doubly linked list	Cache design

5. Why interviewers care

A senior coding answer proves you can:

- **Name the structure** and **complexity before code**
- Avoid Swift footguns (`removeFirst` as queue)
- Narrate pointer updates without losing `next`
- Distinguish amortized vs worst-case for two-stack queue / Array growth
- Bridge lightly to production (nav stack, refresh waiters) without fake claims

6. Coding approach budget (2–3 min) — memorize

Say this every time:

1. **Restate** + 1–2 clarifying questions (constraints, duplicates, empty input)
2. **Brute** + complexity
3. **Optimize** + why this structure
4. **Edges** + API shape (`ListNode?` , mutating vs new list)
5. "I'll code the optimized version now."

Full scripts live in [02-deep-dive.md](#) § Approach scripts and [code/ApproachScripts.md](#).

7. Swift defaults (interview)

```
// Stack – default
var stack: [Int] = []
stack.append(1)           // push
let top = stack.removeLast() // pop

// Queue – do NOT hot-path removeFirst without calling it out
// Prefer two-stack queue, Deque, or head-index ring – see code/

// Linked list – interview node
final class ListNode {
    var val: Int
    var next: ListNode?
    init(_ val: Int, _ next: ListNode? = nil) {
        self.val = val
        self.next = next
    }
}
```

8. Week 2 catch-up (don't skip Mock #2 prep)

If weak on...	Catch-up (45–60m)
MVVM / Clean / DI	Day 08 Qs + S9 opener
Networking / refresh / pin	Day 09 T1/T3 + S4 opener
SDUI versioning / fallback	Day 10 + S3 opener
Hybrid lifecycle	Day 11 + S13 opener
SwiftUI identity	Day 12 + S10 opener

Pick **one** weak day. Still finish today's timed DSA drill.

9. Checkpoint

Before deep dive, say without notes:

1. Stack vs queue in one sentence each
2. Why `Array.removeFirst` is a bad default queue
3. The 2–3 min coding agenda sequence

Then open [02-deep-dive.md](#).

02 — Deep Dive: Patterns, Complexities, Approach Scripts

Senior depth. Self-contained — patterns are taught here; you do not need to leave Cursor to learn them. Optional LeetCode reps are practice, not prerequisites.

1. Stack deep dive

1.1 Array as stack

```
struct Stack<Element> {
    private var storage: [Element] = []
    mutating func push(_ value: Element) { storage.append(value) }
    @discardableResult
    mutating func pop() -> Element? { storage.popLast() }
    var peek: Element? { storage.last }
    var isEmpty: Bool { storage.isEmpty }
}
```

Say: `append` / `removeLast` are amortized $O(1)$. Capacity doubles geometrically — occasional $O(n)$ copy amortized away.

1.2 Valid parentheses (pattern)

Idea: Push opens; on close, pop and match; end empty $\Rightarrow$ valid.

Edges: empty string; only closers; odd length; mixed types `()[]{}.`

Complexity: $O(n)$ time, $O(n)$ space.

Approach script (speak ~90s before code):

"I'll clarify which bracket pairs count. Brute would scan with counters but fails on ordering — `()[]`. Optimized: stack of opening chars; map closers to expected openers. On close, if stack empty or mismatch, false. At end stack must be empty.

$O(n)/O(n)$. Edges: empty $\rightarrow$ true; single closer $\rightarrow$ false. I'll code that."

1.3 Monotonic stack — next greater

Problem shape: For each index, find next greater element to the right (or days until warmer).

Idea: Maintain decreasing stack of **indices**. When current value is greater than stack top's value, pop and record answer.

Complexity: Each index pushed/popped $\leq$ once $\rightarrow O(n)$.

Approach script:

"Brute is $O(n^2)$ nested scan. I'll use a monotonic decreasing stack of indices. Walking left to right, while current is greater than $T[\text{stack.top}]$, pop and set $\text{days} = i - \text{idx}$. Push i . Leftovers get 0. $O(n)/O(n)$."

Worked Swift: [code/MonotonicStack.swift](#).

1.4 Min stack $O(1)$

Idea: Parallel stack of current mins (or encode diffs). Push/pop/getMin all $O(1)$.

Trap: Scanning for min each time $\rightarrow O(n)$.

Duplicates: Push min again when equal so pop stays correct.

Worked Swift: [code/MinStack.swift](#).

2. Queue deep dive

2.1 Why Array.removeFirst is wrong by default

`removeFirst()` shifts every element $\rightarrow O(n)$ per dequeue. Fine for tiny n / rare calls. Wrong for BFS hot paths and interview silence.

Alternatives:

Approach	Dequeue	Notes
Two-stack queue	Amortized $O(1)$	Classic interview
Head index + Array	Amortized $O(1)$ with compact	Simple
Ring buffer	$O(1)$	Fixed capacity; empty/full via count or waste slot
Swift Collections Deque	$O(1)$	Production-friendly if dependency OK

2.2 Two-stack queue

- **inStack:** enqueue via push
- **outStack:** dequeue via pop
- When out empty, **flush** all in $\rightarrow$ out

Each element moves $\leq$ twice $\rightarrow$ amortized $O(1)$. Worst-case dequeue $O(n)$ when flushing — **say amortized**, don't claim strict $O(1)$ always.

Worked Swift: [code/TwoStackQueue.swift](#).

Approach script:

"I'll implement FIFO with two stacks. Enqueue pushes to in. Dequeue: if out empty, pour $\text{in} \rightarrow \text{out}$, then pop out. Amortized $O(1)$. I'll state worst-case flush cost so we're precise."

2.3 BFS / level order

Queue holds frontier. For trees: while queue not empty, drain level size, push children.

Say: "BFS uses a queue; DFS uses a stack or recursion."

2.4 Hit counter / recent requests

Queue of timestamps; while front older than window, dequeue; count = size.

Concurrency (if asked): wrap in an `actor` — Learning-lab design, not a Verified shipped hit counter.

2.5 Circular queue (ring)

Fixed array; `head`, `tail`, `count` (or waste one slot). Indices mod capacity.

Empty vs full: with `count` field, both are unambiguous. Without count, waste one slot so `head == tail` means empty.

S12 adjacency only: ring buffers appear in audio/streaming mental models — do **not** claim you shipped Aces as a custom ring-buffer engine.

3. Linked list deep dive

3.1 Why rare in Swift apps

Arrays win on cache locality, value semantics, and stdlib algorithms. LL shines for:

- Interview pointer skill
- $O(1)$ splice when you **already hold the node**
- LRU: dict + doubly linked list of nodes

Honest senior line: "I practice LL for interviews; in iOS product code I default to Array unless I'm designing something like an LRU."

3.2 Reverse iterative (must-have)

```
prev = null, curr = head
while curr:
    next = curr.next
    curr.next = prev
    prev = curr
    curr = next
return prev
```

$O(n)/O(1)$. Recursive reverse is $O(n)$ stack — mention depth risk.

Worked Swift: [code/LinkedListAlgs.swift](#).

Approach script:

"I'll reverse iteratively with three pointers so space is $O(1)$. Careful to save `next` before rewiring. Empty and single-node are no-ops. Recursive is elegant but $O(n)$ call stack — I'll stick to iterative unless you prefer recursion."

3.3 Floyd cycle detection

Slow +1, fast +2. Meet $\Rightarrow$ cycle. Optional phase 2: reset one pointer to head, both +1 $\rightarrow$ entrance.

Hash-set alternative: $O(n)$ space, simpler narration.

3.4 Middle of list

Same slow/fast; when fast ends, slow is mid. **State even-length policy** (lower vs upper mid).

3.5 Merge two sorted lists

Dummy head; append smaller; attach remainder. $O(n+m)/O(1)$ extra.

3.6 Reverse k-group (tricky)

1. Count k nodes available
2. If fewer than k, leave as-is
3. Reverse segment
4. Reconnect prev tail → new head; iterate

Narrate links; off-by-ones kill this problem.

4. Approach scripts bank (embed — speak these)

Universal opener (15s)

"Let me restate, confirm constraints, then I'll give brute, optimize, complexity, and edges before coding."

Stack — calculator / expression (if asked)

"I'll clarify operators and precedence. Options: shunting-yard, or two stacks for values and ops. I'll assume integer ops and left-to-right within same precedence unless you specify. Watch overflow."

Queue — design recent counter

"I'll keep a queue of event timestamps, drop anything older than the window, and return the queue's count. Unbounded growth is the footgun — eviction keeps memory to the window. If concurrent, I'd isolate behind an actor."

Linked list — merge k (preview)

"Merge two repeatedly is $O(kn)$. Better: min-heap of heads — $O(n \log k)$. That's a Week 4 heap preview; today I'd mention it as the escalation."

Full printable versions: code/ApproachScripts.md.

5. Trade-offs table

Choice	When	Cost
Array stack	Default	Fine
Array queue via removeFirst	Tiny n	$O(n)$ dequeue
Two-stack queue	Interview FIFO	Extra code; amortized story
Ring buffer	Fixed capacity / audio-ish	Empty/full bookkeeping
Linked list algos	Interview	Poor locality in real Swift UI
Recursive reverse	Short lists / clarity	Stack overflow on long lists
Hash cycle detect	Simpler talk	$O(n)$ space
Monotonic stack	Next greater family	Must memorize invariant

6. Failure modes / traps

Trap	Fix
Code immediately	Agenda 2–3 min first

Claim two-stack dequeue always O(1)	Say amortized
Lose next while reversing	Save next before rewrite
Even-length mid ambiguity	State policy aloud
k-group leftover reversed	Leave remainder intact
Use stack for BFS	Queue for BFS
Force LL into UITableView	Array + honesty

7. Catch-up ↔ DSA composure (T8 energy)

If interviewer pivots coding → SDUI mid-problem:

1. Park state: “I have prev/curr wired through node 3...”
2. Answer SDUI with versioning + unknown skip (Day 10 / S3)
3. Offer to resume coding

Shows composure (S8 IMOC energy without overclaiming).

8. Optional citations (not required)

- Apple Swift Collections Deque docs
- CLRS stack/queue chapters (concepts only)
- In-repo: [coding/dsa-track.md](#) for practice volume after this chapter

03 — Production Bridge: Honest DSA ↔ iOS Hooks

Convert patterns into interview bridges **without** inventing “I shipped linked lists in Ads.”

1. Provenance map for today

ID	Label	Exact claim you may make
Learning-lab	DSA chapter	Patterns + Swift in code/ for interviews
S4 mindset	Soft bridge	Token refresh waiters are a queue of continuations awaiting one refresh (Day 09 / Verified S4 stack ownership mindset)
S6	Soft bridge	Navigation / sheet flows — LIFO back stack mental model; Verified metric 30%+ fewer full-screen navs for LE Bottom Sheet
S13	Soft bridge	Hybrid nav ownership — one router; stack discipline matters
S12 adjacency	Vocabulary only	Ring buffers appear in media buffering discussions — not “I built Aces audio as a ring buffer”
S8	Composure	Pivot mid-coding with calm ownership energy (IMOC culture) — don’t invent a DSA incident

Forbidden overclaims

- “We used linked lists for the feed.”
- “I implemented LRU in production at BMS.” (unless you later add real evidence)
- “Aces audio = my circular queue.”
- Fake complexity wins as production metrics.

2. Structure → production bridge table

Structure	Honest bridge	Interview line (≤20s)
Queue	Single-flight refresh waiters; serialise work	"I treat refresh waiters like a queue sharing one in-flight refresh — same FIFO mental model as BFS waiters."
Stack	Nav back stack; undo; nested CMS parent stack	"UIKit nav is LIFO — same structure I'd use for parentheses matching."
Deque / ring	Bounded buffers; sliding window max	"For bounded recent events I'd reach for a deque or ring — and I'd call out empty/full carefully."
Linked list	Interview skill; LRU design interviews	"In Swift apps I prefer Array; I still practice pointer problems because machine rounds ask them."

3. Scripts

Soft bridge — refresh waiters (S4 mindset)

"On networking, concurrent 401s shouldn't each refresh. Waiters line up — conceptually a queue — behind one refresh Task, then fan-out. That's the same FIFO discipline as a BFS queue, applied to auth. The Verified work is the Ads URLSession security ownership; the waiter queue is the concurrency pattern we reason about in that layer."

Provenance: Learning-lab pattern · soft bridge to Verified · S4 / Day 09 single-flight

Soft bridge — nav LIFO (S6 / S13)

"Product navigation is a stack: push screens, pop on back. On LE Bottom Sheet we reduced full-screen pushes — Verified 30%+ fewer full-screen navigations — which is a product win on top of the same LIFO mental model."

Provenance: Verified · S6 · LE Bottom Sheet 30%+ · soft stack metaphor

Honest LL answer

"Linked lists rarely appear in my iOS UI code because Array has better locality and ergonomics. I use them in interviews for reverse/cycle/merge and when designing an LRU with a dict plus node list. I won't pretend BMS Ads was a linked-list codebase."

Provenance: Learning-lab honesty

4. Interview line (≤20s) — day opener

"For coding I'll state structure and complexity first — for example monotonic stack $O(n)$ — then write clean iterative Swift; and I'll call out Array-as-queue costs when relevant."

5. Tomorrow link — Mock #2

DSA today feeds composure, not the 5-min Ads/SDUI architecture talk. Still: choose **Ads** or **SDUI** track tonight so Day 14 isn't cold.

04 — Questions (two-layer answers)

*Study flow: cover the full spoken answer → speak from **Answer points** only → uncover and compare timing.
Totals: **12 normal** + **8 tricky** = **20**.*

Normal questions

Q1. Stack vs queue in one sentence each. (30s)

Answer points:

- Stack = LIFO
- Queue = FIFO
- Optional: deque = both ends

Agenda opener: "One sentence each..."

Full spoken answer:

"A stack is last-in first-out — push and pop the same end. A queue is first-in first-out — enqueue at the rear, dequeue at the front. A deque allows efficient ops at both ends."

Common wrong answer: Confusing FIFO/LIFO labels.

Follow-up ladder:

Level	Ask	Point + short answer
L1	BFS uses which?	Queue.
L2	Undo uses which?	Stack.
L3	Swift default?	Array as stack; careful as queue.

Provenance: Learning-lab

Q2. Implement stack with Array — complexities? (30–45s)

Answer points:

- append / removeLast amortized $O(1)$
- peek $O(1)$
- middle insert $O(n)$
- geometric capacity growth

Full spoken answer:

"I'd wrap an Array: push is append, pop is removeLast, peek is last — all amortized $O(1)$. Random insert or remove in the middle is $O(n)$. Capacity grows geometrically so occasional copies amortize. That's the default Swift stack."

Common wrong answer: "Everything on Array is $O(1)$."

Follow-up ladder:

Level	Ask	Point + short answer
L1	pop empty?	Return optional / precondition.
L2	Thread safety?	Array isn't; isolate if shared.
L3	vs ContiguousArray?	Same idea; Array is fine in interviews.

Provenance: Learning-lab · [code/StackAndQueue.swift](#)

Q3. Why is `Array.removeFirst` bad for queues? (30–45s)

Answer points:

- Shifts all elements $\rightarrow O(n)$
- Prefer deque / two-stack / head index / ring

- OK only for tiny n

Full spoken answer:

"removeFirst shifts every remaining element, so each dequeue is $O(n)$. That's fine for tiny or rare queues, but wrong for BFS or hot paths. I'd use a deque, a two-stack queue, a head index with periodic compact, or a ring buffer — and I'd say that cost aloud if I ever use Array naively."

Common wrong answer: Ignoring the shift cost.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Two-stack complexity?	Amortized $O(1)$.
L2	Production Swift?	Swift Collections Deque if available.
L3	Bridge?	Refresh waiters $\neq$ Array.removeFirst.

Provenance: Learning-lab

Q4. Valid parentheses approach? (45–60s)

Answer points:

- Stack of opens
- Map closers $\rightarrow$ expected openers
- Empty at end
- $O(n)/O(n)$
- Edges: only closers, odd length

Full spoken answer:

"I'll use a stack of opening brackets. For each closer, pop and check it matches via a map; if stack is empty or mismatch, invalid. At the end the stack must be empty. That's $O(n)$ time and $O(n)$ space. Edges: empty string is valid; a single closer is invalid; interleaved types like $([)]$ fail correctly because order is checked."

Common wrong answer: Counters only (misses ordering).

Follow-up ladder:

Level	Ask	Point + short answer
L1	Unicode brackets?	Extend the map.
L2	Early exit?	Odd length can early-fail.
L3	Min remove to valid?	Different problem — stack + counts.

Provenance: Learning-lab · approach script in deep dive

Q5. Monotonic stack — when? (45–60s)

Answer points:

- Next greater/smaller in $O(n)$
- Maintain mono increasing/decreasing indices
- Each element push/pop $\leq$ once

Full spoken answer:

"I reach for a monotonic stack when I need the next greater or next smaller element for every index in linear time. I keep indices in decreasing or increasing order of values; when the invariant breaks, I pop and record answers. Daily temperatures is the

classic — $O(n)$ because each index enters and leaves once.”

Common wrong answer: $O(n^2)$ nested loops without naming the pattern.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Example?	Daily temperatures.
L2	Window max?	Monotonic deque.
L3	Store values or indices?	Indices usually — need positions.

Provenance: Learning-lab · [code/MonotonicStack.swift](#)

Q6. BFS uses which structure? (30s)

Answer points:

- Queue
- Level-order trees
- Shortest path unweighted graphs
- DFS → stack/recursion

Full spoken answer:

“BFS uses a queue so you expand nodes in the order discovered — level order in trees, shortest path in unweighted graphs. DFS uses a stack or the call stack.”

Common wrong answer: “BFS uses a stack.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	Level-order code shape?	Drain count per level.
L2	Weighted shortest?	Dijkstra / heap — not plain BFS.
L3	Bidirectional BFS?	Two queues when useful.

Provenance: Learning-lab

Q7. Reverse linked list — iterative steps? (60s)

Answer points:

- prev nil, curr head
- save next, rewire, advance
- return prev
- $O(n)/O(1)$
- recursive = $O(n)$ stack

Full spoken answer:

“Iteratively: prev starts nil, curr at head. While curr is non-nil, save next, point curr.next to prev, then advance prev and curr. Return prev as the new head. That’s $O(n)$ time and $O(1)$ space. Recursive reverse is fine for clarity but uses $O(n)$ stack space — I prefer iterative in interviews unless asked otherwise.”

Common wrong answer: Losing next and orphaning the rest of the list.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Empty list?	Return nil.
L2	Reverse m...n?	Find window, reverse segment, reconnect.
L3	k-group?	Count, reverse, reconnect.

Provenance: Learning-lab · [code/LinkedListAlgos.swift](#)

Q8. Detect cycle — Floyd. (45–60s)

Answer points:

- Slow +1, fast +2
- Meet $\Rightarrow$ cycle
- Optional entrance phase
- $O(n)/O(1)$
- Hash set alternative $O(n)$ space

Full spoken answer:

"Floyd: slow moves one step, fast two. If they meet, there's a cycle. Optionally reset one pointer to head and walk both one step to find the entrance. Time $O(n)$, space $O(1)$. A hash set of visited nodes is simpler but $O(n)$ space."

Common wrong answer: Only knowing the hash-set approach.

Follow-up ladder:

Level	Ask	Point + short answer
L1	No cycle exit?	Fast hits nil.
L2	Why meeting implies cycle?	Fast gains in a loop.
L3	Find entrance proof?	Sketch tortoise-hare math briefly.

Provenance: Learning-lab

Q9. Merge two sorted lists. (45–60s)

Answer points:

- Dummy head
- Append smaller
- Attach remainder
- $O(n+m)/O(1)$ extra

Full spoken answer:

"I'll use a dummy head and a tail pointer. While both lists remain, append the smaller node and advance that list. Then attach whichever list remains. Return dummy.next. $O(n+m)$ time, $O(1)$ extra space if we reuse nodes."

Common wrong answer: Forgetting null remainders or mutating without dummy (messy head cases).

Follow-up ladder:

Level	Ask	Point + short answer
L1	New nodes vs reuse?	Reuse in-place for $O(1)$ extra.
L2	Merge k?	Heap of heads — Week 4 preview.

L3	Stable merge?	Equal keys — pick policy.
----	---------------	---------------------------

Provenance: Learning-lab

Q10. Middle of linked list. (30–45s)

Answer points:

- Slow/fast
- Fast ends → slow mid
- State even-length policy

Full spoken answer:

“Slow and fast from head; fast moves twice as fast. When fast can’t move, slow is the middle. For even length I’ll state whether I want the lower or upper middle — interviewers care that you noticed.”

Common wrong answer: Counting length in two passes without mentioning two-pointer option.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Delete middle?	Find mid, relink.
L2	Palindrome list?	Mid + reverse second half.
L3	—	—

Provenance: Learning-lab

Q11. Agenda you say before coding? (45s)

Answer points:

- Restate → clarify → brute → optimize → complexity → edges → code
- Budget 2–3 min
- Timing guide

Full spoken answer:

“I budget two to three minutes: restate and clarify, sketch brute force with complexity, present the optimized structure and why, list edge cases and API shape, then say I’m coding the optimized version. Jumping straight into code loses communication points.”

Common wrong answer: Silent coding from the first second.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Ran long on approach?	Cut examples; keep structure+complexity.
L2	Stuck mid-code?	Revert to agenda; restate invariant.
L3	Peer mock tip?	Record approach separately.

Provenance: Learning-lab · timing guide coding budget

Q12. Array vs linked list in Swift production? (45s)

Answer points:

- Array locality + ergonomics
- LL for interview pointers / rare LRU nodes
- Don't force LL into UI lists

Full spoken answer:

"In Swift production I default to Array for locality and stdlib ergonomics. Linked lists are mainly an interview skill and occasionally appear in specialized designs like an LRU's node list. I wouldn't force a linked list into a UITableView data source just to sound clever."

Common wrong answer: "Never learn linked lists" or "always use linked lists for queues."

Follow-up ladder:

Level	Ask	Point + short answer
L1	LRU shape?	Dict + DLL.
L2	Value vs ref nodes?	class ListNode for mutability.
L3	COW Array queue?	Still O(n) removeFirst.

Provenance: Learning-lab honesty

Tricky questions

T1. Implement a queue with two stacks. (90–120s)

Trap: Claiming strict O(1) every dequeue.

Answer points:

- in for enqueue, out for dequeue
- flush in→out when out empty
- amortized O(1)
- state worst-case flush

Full spoken answer:

"Enqueue pushes onto an in-stack. Dequeue pops from an out-stack; if out is empty, I pour all of in into out, then pop. Each element moves at most twice, so amortized O(1). A single dequeue can be O(n) during a flush — I say amortized so I'm precise. Empty dequeue should return nil or throw consistently."

Follow-up ladder:

Level	Ask	Point + short answer
L1	peek?	Same as dequeue without remove; flush if needed.
L2	Thread safety?	Not inherent — wrap if shared.
L3	vs ring?	Ring needs fixed capacity.

Provenance: Learning-lab · [code/TwoStackQueue.swift](#)

T2. Min stack in O(1). (90–120s)

Trap: Scanning stack each getMin.

Answer points:

- Aux min stack (or encode diffs)
- Push/pop/getMin O(1)

- Duplicate mins pushed again

Full spoken answer:

"I'll keep the value stack plus an auxiliary stack of minima. On push, push onto mins if value $\leq$ current min. On pop, if popped equals min top, pop mins too. getMin is mins.peek — all $O(1)$. Duplicates matter: when the new value equals min, push it again so one pop doesn't lose the min."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Space?	$O(n)$ worst case.
L2	Encode diffs?	Possible; more error-prone — mention.
L3	Max stack?	Symmetric.

Provenance: Learning-lab · [code/MinStack.swift](#)

T3. Design hit counter / recent requests queue. (120s)

Trap: Unbounded memory; ignoring concurrency.

Answer points:

- Queue of timestamps
- Evict older than window
- count = size
- actor if concurrent

Full spoken answer:

"I'd store timestamps in a queue. On hit, enqueue now; while the front is older than the window, dequeue. The count is the queue's size — memory stays proportional to hits in the window. If multiple threads call in, I'd isolate behind an actor. In production analytics I'd also budget cardinality and sampling — unbounded raw event queues are dangerous."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Window 5 minutes?	Compare $\text{now} - t \geq \text{window}$.
L2	Persist?	Usually in-memory for this problem.
L3	Prod bridge?	Soft — not a Verified BMS metric.

Provenance: Learning-lab

T4. Reverse nodes in k-group. (120s)

Trap: Off-by-one; reversing leftover $< k$.

Answer points:

- Count k available
- Reverse segment
- Reconnect
- Leave remainder
- $O(n)/O(1)$

Full spoken answer:

"I'll walk counting k nodes. If fewer than k remain, leave them. Otherwise reverse that segment, reconnect the previous group's tail to the new head, and continue. Narrating the pointers matters more than cleverness — leftover nodes stay in original order. $O(n)$ time, $O(1)$ extra space iterative."

Follow-up ladder:

Level	Ask	Point + short answer
L1	$k = 1?$	No-op.
L2	$k > \text{length}?$	Leave whole list.
L3	Recursive?	OK if stack discussed.

Provenance: Learning-lab

T5. Why linked lists rarely appear in your iOS code? (90s)

Trap: Sounding anti-DSA.

Answer points:

- Array locality + Swift ergonomics
- LL for interviews + LRU-like designs
- Still practice pointers

Full spoken answer:

"Because Swift Array wins on cache locality and API surface for UI and model lists. Linked lists excel in interview pointer problems and in designs like LRU where you need $O(1)$ move-to-front given a node. I practice them seriously for machine rounds — I just don't force them into UITableView data sources."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Ever use LL?	LRU design interviews; teaching.
L2	Queue in prod?	Deque / actor mailbox, not ListNode.
L3	—	—

Provenance: Learning-lab honesty

T6. Circular queue with fixed array. (90–120s)

Trap: Empty/full ambiguity.

Answer points:

- head/tail mod capacity
- count field OR waste one slot
- ring buffer language
- S12 adjacency only — don't overclaim audio

Full spoken answer:

"I'd allocate a fixed buffer with head, tail, and a count — enqueue writes at tail, dequeue reads at head, both indices mod capacity. Count distinguishes empty from full cleanly. Alternatively waste one slot so $\text{head} = \text{tail}$ means empty. This is the ring-buffer pattern. I'll use the vocabulary carefully around media — I'm not claiming Aces audio was my custom ring implementation."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Threaded producer/consumer?	Need sync or actor.
L2	Overwrite policy?	Block vs drop oldest — product choice.
L3	—	—

Provenance: Learning-lab · S12 adjacency vocabulary only

T7. Stack for calculator / expression. (120s)

Trap: Coding without grammar/precedence.

Answer points:

- Clarify operators/precedence
- Two stacks or shunting-yard
- State assumptions
- Overflow

Full spoken answer:

“First I’d clarify the grammar — which operators, precedence, associativity, integers only or not. Then either shunting-yard to RPN, or a values stack plus an operators stack. I’ll state assumptions before coding and mention integer overflow. Diving into code without the grammar is how calculator problems implode.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	Parentheses?	Ops stack handles with higher precedence rules.
L2	Unary minus?	Call it out as a tokenisation issue.
L3	—	—

Provenance: Learning-lab

T8. Interviewer pivots coding → SDUI mid-way. (90s)

Trap: Freeze or abandon the code silently.

Answer points:

- Park pointer state verbally
- Answer SDUI: versioning + unknown skip
- Offer to resume
- Composure (S8 energy)

Full spoken answer:

“I’d park the code state out loud — for example, I’ve rewired through node three and next is saved — then answer the SDUI question with schema versioning and fail-soft unknown-component skip from our backend-driven header work. Then I’d ask whether to resume the list reverse. That composure matters as much as the algorithm.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	SDUI one-liner?	Fetch → version → registry → fallback.
L2	Story?	S3 header; don’t invent metrics.

L3	Mock #2?	Same composure for 5-min talks.
----	----------	---------------------------------

Provenance: Learning-lab composure · Verified · S3 for SDUI content · S8 energy

Flashcard strip (quick)

Front	Back
removeFirst queue	O(n) shift
Two-stack dequeue	Amortized O(1)
Monotonic stack	Next greater O(n)
Floyd	Slow/fast O(1) space
Reverse list	Three pointers
Coding agenda	Restate→brute→optimize→edges→code

Revision twin for skeleton recall: [.././../revision/weeks/week-02/day-13.md](https://github.com/robertkowalski/revision/weeks/week-02/day-13.md)

05 — Exercises

*Solutions are in-repo under `code/` and inline below. Speak approach **before** peeking.*

A. Conceptual

A1. Structure picker

For each prompt, name structure + one-line why.

1. Undo typing
2. Print tree level by level
3. Next warmer day
4. Detect loop in singly linked list
5. Bounded last-100 events

Solution: 1 Stack. 2 Queue. 3 Monotonic stack. 4 Floyd / hash. 5 Queue or deque/ring with eviction.

A2. Complexity true/false

1. `Array.removeFirst` is O(1).
2. Two-stack queue dequeue is always O(1) worst-case.
3. Iterative reverse is O(1) space.
4. Monotonic stack next-greater is O(n).

Solution: 1 F (O(n)). 2 F (amortized; flush O(n)). 3 T. 4 T.

A3. Write the universal coding agenda from memory (5 bullets).

Solution: Restate/clarify → brute+complexity → optimize+why → edges/API → code now.

B. Coding (worked files)

B1. Narrate TwoStackQueue

Open [code/TwoStackQueue.swift](#). Speak enqueue/dequeue/flush without looking at comments. Then implement `peek`.

Solution sketch: peek flushes if out empty, returns `outStack.last` without popping.

B2. Reverse + cycle

Open [code/LinkedListAlgos.swift](#). Trace reverse on `1→2→3`. Trace Floyd on a 4-node cycle at node 2.

B3. Daily temperatures

Open [code/MonotonicStack.swift](#). Hand-simulate `T = [73,74,75,71,69,72,76,73]`.

B4. MinStack duplicates

Push 3, push 3, pop — getMin must still be 3. Verify against [code/MinStack.swift](#).

B5. Timed pair (record)

1. Easy: valid parentheses — 15–20 min including approach
2. Medium: reverse list **or** daily temperatures — 25–35 min

Score approach minutes against the timing guide coding budget.

C. Approach scripts (speak aloud)

Use [code/ApproachScripts.md](#). Record three:

1. Valid parentheses
 2. Two-stack queue
 3. Reverse linked list
-

D. Speaking / catch-up

1. Deliver Q12 (Array vs LL honesty) in ≤45s.
 2. Deliver T8 (pivot to SDUI) in ≤90s.
 3. Pick one weak Week 2 day; re-record 2 answers.
-

E. Timed drill checklist

1. ☐ 1 Easy + 1 Medium timed
2. ☐ Approach spoken before each
3. ☐ Gotchas logged (`removeFirst` , off-by-ones)
4. ☐ Ads **or** SDUI track chosen for Day 14
5. ☐ Revision twin skim

Next: revision twin — [../../revision/weeks/week-02/day-13.md](#).

Day 14 — Week 2 Revision + Mock #2 (Ads or SDUI)

Week 2 · Full study (self-contained) · ~5–6 hrs

Revision twin: [revision/weeks/week-02/day-14.md](#)

Outcomes

By end of day, without notes, you can:

- Deliver a **timed 5:00 architecture talk** on **Ads (POP + HeroWidget + URLSession pinning) or SDUI (schema → registry → fallback)** — pick one
- Connect Week 2: MVVM/DI → networking → SDUI → hybrid UI → SwiftUI identity → DSA composure

- Answer mixed Week 2 warm-ups with two-layer discipline (points → full spoken)
- Run **Mock #2** end-to-end from the embedded script and retro into gotchas
- Deliver **S1** and **S3** STAR talk tracks cleanly (supporting S4 / S12 / S9 as needed)

How to study (in Cursor only)

1. 01-foundations.md — Week 2 synthesis + mock format
2. 02-deep-dive.md — full Ads + SDUI 5-min spines (embedded; no required external doc read)
3. 03-production-bridge.md — Verified story map + honest provenance
4. code/MockTalkTracks.md + code/Week2Warmups.md — speak aloud
5. 04-questions.md — warm-up bank (two-layer)
6. 05-exercises.md — run Mock #2 with timer
7. Revision twin for flashcard recall

Module map

Module	File
Foundations	01-foundations.md
Deep dive	02-deep-dive.md
Production bridge	03-production-bridge.md
Questions	04-questions.md
Exercises	05-exercises.md
Talk tracks	code/MockTalkTracks.md

Provenance reminder

Label	Use
Verified · S1	Ads POP+Generics; HeroWidget pause/play
Verified · S4	Alamofire → URLSession; HTTPS; pinning; whitelist
How I would apply it · S4-A1	Pin rotation / backup / break-glass as design
Verified · S3	Backend-driven header; search debounce/MVVM
How I would apply it · S3-A1	Schema versioning + unknown fallback as design emphasis
Verified · S12	Aces audio + server-driven splash
Verified · S9	District Clean/MVVM + AI tooling judgment
Verified · S6	LE Bottom Sheet 30%+ fewer full-screen navs

Do **not** invent fill-rate %, splash ms, or “I shipped the pin runbook.”

Agenda opener — pick one track tonight

Ads:

“I’ll cover revenue Ads scope, POP+generics pipeline, HeroWidget lifecycle, URLSession pinning/whitelist, and trade-offs vs SDUI for media.”

SDUI:

"I'll cover SDUI scope, schema/versioning, registry+actions, fail-soft fallback, BMS header / Aces splash, and limits vs native Ads."

Time budget

Block	Minutes
Foundations + synthesis	40–50
Deep dive chosen track (+ skim other)	60–75
Production bridge + STAR	40–50
Warm-up questions (record 5)	40–50
Mock #2 full run	60–90
Retro + gotchas	15–20
Revision twin	15

Source note

Self-contained: Track A/B spines are fully embedded in `02-deep-dive.md` . Optional later skim of `ios-system-design` docs is enrichment only.

01 — Foundations: Week 2 Synthesis + Mock Format

Build the connective tissue, then run Mock #2 with discipline.

1. Week 2 map (say as one narrative)

Day 08	Layers / DI / search VM
↓	
Day 09	URLSession / refresh / pin / cancel
↓	
Day 10	SDUI schema / registry / fallback
↓	
Day 11	Lifecycle / cells / hybrid / bottom sheet
↓	
Day 12	SwiftUI state / identity / Stories SDK
↓	
Day 13	Stack · Queue · LL composure

One-sentence senior narrative:

"I structure features with clear layers and DI, own networking with cancellation and security, use SDUI where content velocity matters with fail-soft schema, and treat UIKit/SwiftUI lifecycle and identity as production contracts — proven on BMS, District, and Raw apps."

2. Glossary for mock day

Term	Meaning today
Mock #2	Timed self/peer run: warm-ups → tricky → 5-min architecture → STAR → retro

Track A	Ads architecture (S1 + S4)
Track B	SDUI architecture (S3 + S12)
Agenda opener	First 15–20s that scopes the 5-min talk
Fail-soft	Unknown SDUI nodes skip; never crash; hard fallback if root empty
HeroWidget contract	Pause/play tied to visibility + VC lifecycle
Single-flight	One token refresh shared by waiters
Provenance honesty	Verified vs Applied vs Learning-lab

3. Mock #2 format (memorize)

Block	Time	What
Warm-up Qs	15–20 min	4–5 Normal from Week 2 mixed
Deep dive	15–20 min	2 Tricky (refresh or SDUI outage or hybrid nav)
Architecture talk	5 min timed	Ads or SDUI (one track)
STAR	10 min	S1 or S3/S12 + spice S4/S9
Retro	10–15 min	Score timing; update gotchas

Full facilitator script: [02-deep-dive.md](#) § Mock script · [code/MockTalkTracks.md](#).

4. Choose ONE primary track

If your stronger stories are...	Pick
POP, HeroWidget, pinning	Track A — Ads
Schema, fallback, header/splash	Track B — SDUI

Doing both cold → neither talk is crisp. Skim the other track 20 min **after** your recording.

5. Timing muscle (architecture)

Minute	Content
0:00–0:20	Agenda + scope
0:20–1:00	Problem / context
1:00–3:30	Architecture beats (3–4)
3:30–4:30	Failure modes / trade-offs
4:30–5:00	Stop; invite questions

Hard rule: If you hit 5:00 mid-sentence, stop and invite questions. Re-record.

6. Checkpoint

1. Can you say the Week 2 one-liner?
2. Have you picked Ads **or** SDUI?
3. Do you know the mock block times?

02 — Deep Dive: Full Mock Script + Ads/SDUI Spines

Everything you need to run Mock #2 without leaving Cursor.

1. Facilitator script — Mock #2 (self or peer)

Setup (2 min)

1. Timer visible; recording on.
2. Candidate declares **Track A (Ads)** or **Track B (SDUI)**.
3. Facilitator (or you) opens this section and [04-questions.md](#).
4. Score sheet: timing guide rubric 1–5.

Block 1 — Warm-ups (15–20 min)

Ask **4–5** Normals. Prefer mix:

- Layers/DI or search cancel (Day 08 / S3)
- Single-flight refresh (Day 09)
- Unknown SDUI node (Day 10)
- HeroWidget pause **or** SwiftUI identity (S1 / S10)
- One DSA composure (Array queue **or** agenda-first)

Facilitator line: “45–60 seconds. Agenda first if it’s conceptual.”

Block 2 — Tricky (15–20 min)

Ask **2** of:

- Refresh stampede + pin outage leadership
- Why not SDUI the Ads video player?
- Hybrid deeplink dual-stack failure
- Debounce VM vs repository (pick a side)
- Ran 8 minutes on architecture — how fix?

Block 3 — Architecture talk (5:00 hard stop)

Facilitator: “Five minutes. I’ll hard-stop. Agenda in the first twenty seconds.”

- Track A → §2 below
- Track B → §3 below

Record this block as its own take.

Block 4 — STAR (10 min)

- Track A: **S1** (2–3 min) + spice **S4** opener (20s)
- Track B: **S3** (2–3 min) + spice **S12** opener (20–45s)
- Optional either: **S9** AI tooling judgment (45–60s)

Talk tracks: [code/MockTalkTracks.md](#).

Block 5 — Retro (10–15 min)

1. Score each answer 1–5.
2. Note: agenda missing? overclaim? over time?
3. Log gotchas tagged `week-02`.
4. Checklist:
 - ☐ 5-min talk < 5:30 (ideal ≤ 5:00)

- ☐ S1/S3/S4/S9 openers ≤ 20–45s each
- ☐ Refresh + unknown component clean
- ☐ DSA basics not blocking

2. Track A — Ads architecture (5 min spine)

Agenda (≤20s)

"I'll cover problem scope, type-safe component pipeline with POP and generics, HeroWidget lifecycle, networking and pinning on URLSession, and trade-offs versus SDUI for media."

Beats (~1 min each)

1. Context

Highest-revenue Ads module needed safer reusable rendering; video inside HeroWidget needed correct pause/play with lifecycle.

2. Component model

Protocol-oriented ad contracts + generics pipeline — not inheritance trees — so new creatives plug in without forking.

3. HeroWidget

Visibility / VC lifecycle / background → pause/play; `prepareForReuse` stops player; lifecycle is part of the product contract.

4. Networking security

Alamofire → URLSession; HTTPS; SSL pinning; domain whitelist. Pin rotation / backup / break-glass as **design** (S4-A1), not a fake shipped runbook claim.

5. Trade-offs / close

Keep revenue media native; CMS may configure placement; SDUI for config ≠ SDUI for the player. Invite questions.

Failure modes to mention if time

Failure	Mitigation
Off-screen playback	Visibility + disappear pause
Pin mismatch outage	Backup pins + staged rotation design
Cell reuse wrong creative	Cancel + clear + generation token

Full spoken 5-min talk

See [code/MockTalkTracks.md](#) § Track A full script.

3. Track B — SDUI architecture (5 min spine)

Agenda (≤20s)

"I'll define SDUI scope, schema and versioning, registry and allowlisted actions, fail-soft fallback and cache, then BMS header and Aces splash — and limits versus native Ads."

Beats

1. Problem

Content/layout velocity without releases; personalisation; crash-free rendering.

2. Pipeline

Fetch → version gate → parse → component registry → layout → allowlisted actions only.

3. Resilience

Unknown type → skip + metric; last-known-good cache; empty root → hard fallback header/splash; never crash on bad CMS.

4. Production

BMS backend-driven header + search (S3); Aces server-driven splash / cold-start freshness (S12). Measure time-to-interactive, not vanity TTFF alone.

5. Trade-offs / close

New component types still need app release; revenue video Ads often stay native (S1). Invite questions.

Failure modes

Failure	Mitigation
Major schema mismatch	Version gate + hard fallback
Unknown node	Skip, don't throw
Action injection	Allowlist only
Slow splash fetch	Cached last-good splash

Full spoken 5-min talk

See [code/MockTalkTracks.md](#) § Track B full script.

4. Why not both architectures as one religion

Choice	When	Cost
Native Ads media (S1)	Lifecycle, billing viewability, typed players	Less CMS flexibility on the player itself
SDUI header/splash (S3/S12)	Content velocity, experimentation	New widgets need release; must fail-soft
SDUI configures Ads placement	Best of both	Clear boundary — config vs renderer
SDUI the video player	Rarely	Weak lifecycle/typing for revenue media

Bridge answer (T4 energy):

"I'd SDUI the placement and campaign config; I'd keep the video renderer native with HeroWidget's pause/play contract."

5. Week 2 connective micro-answers (embedded)

Single-flight refresh (30s spine)

N×401 → one refresh Task; waiters await; retry once; failure → logout fan-out.

Unknown SDUI (20s spine)

Skip + metric; never crash; empty root → hard fallback.

Search cancel (20s spine)

Debounce in VM; cancel Task; cancellation ≠ user-facing error; ignore stale.

SwiftUI identity (20s spine)

Stable IDs; UUID in `body` resets state; Stories pages need stable identity (S10).

Array as queue (15s spine)

`removeFirst` O(n); deque / two-stack.

6. Trade-offs (meta mock)

Choice	When	Cost
Ads 5-min	Strong S1/S4	Less CMS vocab
SDUI 5-min	Strong S3/S12	Must admit native limits
Both cold	Ego	Neither crisp
Skip STAR	Fatigue	Lose behavioral signal
Skip retro	Hubris	No compounding

7. Optional citations (not required)

- ios-system-design/docs/networking-layer.md
- ios-system-design/docs/sdui-engine.md
- Timing: roadmap/timing/answer-timing-guide.md

03 — Production Bridge: Mock Story Map

Wire Verified stories into Mock #2 without overclaiming.

1. Provenance map

ID	Label	Exact claim
S1	Verified	Ads refactor; POP+Generics; HeroWidget pause/play on revenue-critical module
S4	Verified	Alamofire → URLSession; HTTPS; SSL pinning; domain whitelist
S4-A1	Applied design	Pin rotation, backup pins, break-glass — not “I shipped the ops runbook”
S3	Verified	Backend-driven header; search debounce/state/MVVM
S3-A1	Applied design	Schema versioning + unknown-component fallback emphasis
S12	Verified	Aces audio streaming; server-driven splash; cold-start improvement (no invented ms)
S9	Verified	District Free Parking; Clean/MVVM migration; Context Engineering; tests assist
S6	Verified	LE Bottom Sheet; 30%+ fewer full-screen navigations
S10	Verified	Stories SDK reusable across portfolio
S13	Verified	Grizzlies SwiftUI/UIKit; deeplinks
S8	Verified	30L+ DAU; 99.95%+ CFS; Crashlytics; IMOC — use for leadership composure

Forbidden

- Invented fill-rate % or splash latency ms
- “Shipped pin rotation runbook / shadow traffic” as fact
- Claiming both tracks as one giant project you did in a week

2. Track → story pairing

Track	Core STAR	Supporting
-------	-----------	------------

Ads	S1 (2–3 min)	S4 opener; S5 perf culture optional; S8 scale
SDUI	S3 (2–3 min)	S12 splash/audio; S8 fail-soft culture

Spice either track: S9 (migration + AI judgment), S6 (product UX metric), S10/S13 (modular/hybrid).

3. S1 talk track (2–3 min STAR)

Opener (~10s)

"I'll walk through our highest-revenue Ads refactor — protocols, generics, and video lifecycle."

S/T (~20s)

Highest-revenue Ads module needed a safer reusable rendering path. Video ads inside HeroWidget needed correct pause/play with lifecycle.

Action (~90s)

1. Refactored around **protocol-oriented** ad component contracts + **generics** for a type-safe pipeline.
2. Built reusable **HeroWidget** with explicit pause/play tied to visibility / VC lifecycle.
3. Kept revenue paths behind clear protocols so new ad types didn't fork the pipeline.
4. Coordinated with stakeholders on behavior without breaking the revenue path.

Result (~20–30s)

Maintainable type-safe ads pipeline; lifecycle-correct video reduced wasted playback and UI glitches on a module that mattered for revenue. (**No invented fill-rate %.**)

Lesson (~15–20s)

For revenue-critical UI, prefer POP + generics over inheritance trees; lifecycle is part of the product contract.

Provenance: *Verified · S1 · BookMyShow · Ads / HeroWidget*

4. S3 talk track (2–3 min STAR)

Opener (~10s)

"I'll cover our backend-driven header and the search UX we aligned to MVVM."

S/T (~20s)

Main header needed to be CMS/backend-driven; search needed modern UX — debounce and explicit state — aligned to MVVM.

Action (~90s)

1. Migrated header to a **generalised, protocol-driven** main-screen implementation.
2. Search: **debouncing**, loading/empty/error state, MVVM binding; cancel in-flight appropriately.
3. Contracted with backend so layout/content could change without app release for many cases.
4. Fail-soft mindset: unknown/bad payloads must not crash the shell (emphasize as S3-A1 design if asked about versioning).

Result (~20–30s)

Faster content iteration on header; snappier, race-safer search UX.

Lesson (~15–20s)

SDUI needs schema/versioning + client fallbacks — not just "render JSON."

Provenance: *Verified · S3 · BookMyShow · backend-driven header / search*

5. Supporting openers (≤20–45s)

S4:

"We moved Ads networking off Alamofire onto URLSession so we owned HTTPS, SSL pinning, and a domain whitelist on a high-traffic revenue module."

S12:

"On Aces I integrated live audio streaming and a server-driven splash so cold-start content could stay fresh — measuring time-to-interactive, not just first frame."

S9:

"At District we migrated Free Parking toward Clean/MVVM with an AI-assisted workflow inside a human-owned architecture envelope — tests and review still gate merges."

S6:

"LE Bottom Sheet cut full-screen navigations by 30%+ on targeted flows by giving a lightweight overview instead of a push."

6. Interview lines (≤20s)

Ads mock:

"I'll walk a revenue Ads architecture — POP/generics, HeroWidget lifecycle, and URLSession pinning."

SDUI mock:

"I'll walk a fail-soft SDUI pipeline — schema versioning, registry, and BMS/Aces production usage."

7. Leadership hybrid (T3)

When refresh stampede + pin outage same week:

"I'd lead IMOC-style: blast radius, feature guards, rollback/break-glass design, owners, comms — then technical fixes: single-flight refresh, backup pins. Watch TLS failure rate and crash-free. Verified culture: 30L+ DAU and 99.95%+ CFS bar from S8 — without claiming pinning alone owns CFS."

Provenance: Verified · S8 culture · S4 controls · S4-A1 design

04 — Questions (two-layer warm-ups for Mock #2)

*Study flow: cover full spoken → speak from **Answer points** → compare.*

*Totals: **12 normal + 8 tricky = 20**. These are Week 2 revision warm-ups + mock traps.*

Normal questions

Q1. MVVM vs Clean — when Clean? (45s)

Answer points:

- Shared domain rules / multi-surface → UseCases
- Simple feature → MVVM enough
- District incremental migration (S9)

Full spoken answer:

"I default to MVVM for a single screen with clear UI state. I reach for Clean-style UseCases when domain rules are shared across surfaces or we're migrating a messy feature boundary — like Free Parking at District — so the domain isn't trapped in a ViewModel. Incremental migration beats a big-bang rewrite."

Common wrong answer: "Clean is always more senior."

Follow-up ladder:

Level	Ask	Point + short answer
L1	AI role?	Context inside protocols; human owns architecture (S9).
L2	DI?	Inject UseCases at composition root.
L3	—	—

Provenance: Verified · S9

Q2. Search debounce + cancel ownership? (45s)

Answer points:

- Debounce in VM (presentation policy)
- Cancel Task on new keystroke / disappear
- Cancel ≠ error toast
- Ignore stale results

Full spoken answer:

"Debounce lives in the ViewModel as presentation policy. Each new query cancels the in-flight Task; cancellation is not a user-facing error. Completions check a generation token or task identity so out-of-order responses can't win. That's how we kept BMS search race-safer under MVVM."

Common wrong answer: Showing "error" on cancel.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Repo cancel too?	May cancel network; policy still VM.
L2	Story?	S3.
L3	—	—

Provenance: Verified · S3

Q3. Single-flight token refresh? (60s)

Answer points:

- One refresh for N 401s
- Waiters share result
- Retry once
- Failure fan-out → logout
- Actor / critical section

Full spoken answer:

"When many calls 401 together, only one refresh runs — waiters await that Task. On success, retry originals once; on failure, fan out logout. I isolate that behind an actor or serial critical section so we don't mutate refresh state from unstructured tasks. Blind refresh storms revoke tokens and amplify outages."

Common wrong answer: Each 401 refreshes independently.

Follow-up ladder:

Level	Ask	Point + short answer
L1	POST retry?	Don't blind-retry non-idempotent charges.
L2	Code?	Day 09 SingleFlightRefresh pattern.
L3	—	—

Provenance: Learning-lab · Day 09 · soft S4 stack

Q4. SSL pinning failure mode? (45–60s)

Answer points:

- MITM resistance ↑
- Bad rotation → self-outage
- Backup pins + staged design (S4-A1)
- ATS ≠ pinning

Full spoken answer:

"Pinning reduces MITM risk by requiring expected server identity, but a bad rotation bricks clients. On Ads we moved to URLSession with HTTPS, pinning, and a host whitelist. Separately, as design, I'd require backup pins and a monitored break-glass plan — I'm not claiming I shipped that full ops runbook."

Common wrong answer: "Pinning never causes outages" or "ATS is pinning."

Follow-up ladder:

Level	Ask	Point + short answer
L1	SPKI?	Hash SPKI DER, not raw key export bytes.
L2	Story?	S4.
L3	—	—

Provenance: Verified · S4 · Applied · S4-A1

Q5. Unknown SDUI component policy? (30–45s)

Answer points:

- Skip + metric
- Never crash
- Empty root → hard fallback

Full spoken answer:

"Unknown component types skip with a metric — never crash the shell. If the root is empty after filtering, show a hard fallback header or splash. Fail-soft is how SDUI survives CMS mistakes at scale."

Common wrong answer: Force-unwrap / fatalError on unknown.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Version mismatch?	Gate major versions; fallback.
L2	Story?	S3; S3-A1 for versioning emphasis.
L3	—	—

Q6. SDUI vs WebView? (45s)

Answer points:

- Native registry → a11y/perf
- Allowlisted actions
- WebView for truly open content

Full spoken answer:

"SDUI with a native registry keeps accessibility, performance, and action allowlisting under app control. A WebView is fine for open-ended content you don't want to native-render, but it's the wrong default for a primary home header you need to trust."

Common wrong answer: "WebView is always simpler/better."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Hybrid?	Native shell + limited web islands.
L2	—	—
L3	—	—

Provenance: Learning-lab · Day 10

Q7. Where pause ad / story video? (30–45s)

Answer points:

- Offscreen / disappear / background
- prepareForReuse
- Protocolised (S1 / S10)

Full spoken answer:

"Pause when the widget leaves visibility, on viewWillDisappear, on background, and in prepareForReuse. On Ads, HeroWidget made that an explicit contract so video didn't keep playing off-screen on a revenue surface."

Common wrong answer: Fire-and-forget AVPlayer.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Stories?	Same lifecycle discipline in SDK.
L2	Story?	S1.
L3	—	—

Provenance: Verified · S1

Q8. Hybrid deeplink ownership? (60s)

Answer points:

- Parse at app edge → intent
- Single router
- No dual stacks forever-sync

Full spoken answer:

"Parse deeplinks at the app edge into an intent, then one router owns navigation. Dual UIKit/SwiftUI stacks that try to stay forever in sync are a bug factory — Grizzlies-style hybrid work needs one owner for exits."

Common wrong answer: "Each stack handles its own URLs independently forever."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Cold start?	Queue intent until root ready.
L2	Story?	S13.
L3	—	—

Provenance: Verified · S13

Q9. SwiftUI identity footgun? (45s)

Answer points:

- Unstable `.id` resets state
- UUID in body is bad
- Stories need stable page IDs

Full spoken answer:

"If identity changes every render — like UUID in body — SwiftUI tears down and rebuilds state. Stories pages need stable IDs so progress and media don't reset while paging."

Common wrong answer: "SwiftUI bug — not my identity."

Follow-up ladder:

Level	Ask	Point + short answer
L1	List?	Stable id: on ForEach.
L2	Story?	S10.
L3	—	—

Provenance: Verified · S10 soft bridge · Day 12

Q10. LE Bottom Sheet impact? (30–45s)

Answer points:

- Lightweight overview
- **30%+** fewer full-screen navs
- Cross-functional contracts

Full spoken answer:

"LE Bottom Sheet gave a lightweight overview instead of pushing full screens for many flows — Verified 30%+ fewer full-screen navigations on those flows. The win was product contract plus implementation, not a visual gimmick."

Common wrong answer: Vague "better UX" without the metric you're allowed to use.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Sheet vs push?	Overview vs deep task.
L2	Story?	S6.
L3	—	—

Provenance: Verified · S6 · 30%+

Q11. Array as queue issue? (30s)

Answer points:

- removeFirst $O(n)$
- Deque / two-stack / ring

Full spoken answer:

"Array.removeFirst shifts elements — $O(n)$ per dequeue. I'd use a deque, two-stack queue, or ring, and I'd say that cost if I ever demo Array naively."

Common wrong answer: Ignoring cost.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Day?	13.
L2	—	—
L3	—	—

Provenance: Learning-lab · Day 13

Q12. District AI tooling — senior framing? (45–60s)

Answer points:

- Context Engineering inside protocols
- Human-owned architecture
- Review + tests gate

Full spoken answer:

"At District, AI assisted inside a Context Engineering envelope — protocols, patterns, and tests I still owned. Senior signal isn't 'AI wrote the app'; it's architecture judgment, review, and XCTest/XCUITest discipline around the migration."

Common wrong answer: "AI generated our production architecture unsupervised."

Follow-up ladder:

Level	Ask	Point + short answer
L1	What AI got wrong?	Have one concrete review catch ready.
L2	Story?	S9.
L3	—	—

Provenance: Verified · S9

Tricky questions

T1. Five minutes — design our Ads rendering system. (5 min)

Trap: Dive into cell code; skip lifecycle/security.

Answer points: Use Track A spine — POP, HeroWidget, URLSession pin, trade-offs.

Full spoken answer: Deliver [code/MockTalkTracks.md](#) Track A full script under 5:00.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Pin rotation?	S4-A1 design.
L2	POP vs inheritance?	Extension without fork.
L3	Metrics?	No invented fill-rate %.

Provenance: Verified · S1 · S4

T2. Five minutes — design SDUI for our home header. (5 min)

Trap: Ignore versioning/fallback.

Answer points: Track B spine — schema, registry, fail-soft, BMS/Aces.

Full spoken answer: Deliver Track B full script under 5:00.

Follow-up ladder:

Level	Ask	Point + short answer
L1	Breaking schema at scale?	Version gate + fallback; S8 bar.
L2	Actions?	Allowlist.
L3	—	—

Provenance: Verified · S3 · S12

T3. Refresh stampede + pinning outage same week — how do you lead? (120s)

Trap: Only technical rabbit hole.

Answer points:

- IMOC-style blast radius / owners / comms
- Then single-flight + backup pins design
- Metrics: TLS failures, crash-free
- Honest S8 culture without pinning=owns CFS

Full spoken answer:

"I'd lead with blast radius and owners first — feature guards, rollback, break-glass design — while engineering fixes single-flight refresh and pin backups. I'd watch TLS failure rate and crash-free. At BMS scale we held a 99.95%+ CFS bar at 30L+ DAU — I'm not claiming pinning alone created that number, but that reliability culture shapes how I'd run the week."

Follow-up ladder:

Level	Ask	Point + short answer
-------	-----	----------------------

L1	Comms?	Status cadence; don't silent-fix.
L2	Stories?	S8 + S4 + S4-A1.
L3	—	—

Provenance: Verified · S8 · S4 · Applied · S4-A1

T4. Why not SDUI the Ads video player? (90–120s)

Trap: SDUI religion.

Answer points:

- Revenue media needs typed native lifecycle
- SDUI for placement/config
- Native HeroWidget renderer

Full spoken answer:

"I'd keep the video renderer native. Revenue media needs typed lifecycle — pause/play, reuse, viewability — which is exactly what HeroWidget encoded. SDUI is excellent for placement and campaign configuration. Forcing the player through a generic CMS widget usually weakens the contract that protects UX and revenue."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Bridge tracks A/B?	Config SDUI + native player.
L2	—	—
L3	—	—

Provenance: Verified · S1 · contrast S3

T5. SwiftUI Stories SDK in a UIKit-heavy host. (90–120s)

Trap: Ignore hosting/identity/router.

Answer points:

- UIHostingController façade
- Inject deps (S10)
- Deeplink exits via host router (S13)
- Stable identity

Full spoken answer:

"I'd expose Stories behind a small UIKit façade — hosting controller, injectable content/analytics/image loader — so the host doesn't import SDK internals. Deeplink exits go through the host router, not a second nav religion. Stable SwiftUI identity keeps pages from resetting. That's how a reusable Stories SDK survives portfolio hosts."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Stories?	S10.
L2	Hybrid?	S13.
L3	—	—

T6. Debounce in repository vs VM — argue both, then pick. (90s)

Trap: No decision.

Answer points:

- VM = presentation policy
- Repo may cancel network
- Pick VM for BMS search

Full spoken answer:

“Repository debounce couples policy to data and surprises other callers. ViewModel debounce matches UX timing and is easy to test with fake clocks. The repository should still cancel work when asked. For BMS search I’d keep debounce in the VM and cancel through the networking layer.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	Shared search used by two UIs?	Shared UseCase with explicit policy param.
L2	—	—
L3	—	—

Provenance: Verified · S3 judgment · Day 08

T7. Mock feedback: you ran 8 minutes on architecture. (60s)

Trap: Defensiveness.

Answer points:

- Cut examples
- Agenda + timeboxes
- Re-record 5:00 hard stop

Full spoken answer:

“Fair — I over-indexed on examples. Next pass: agenda in twenty seconds, one minute per beat, hard stop at five, invite questions. I’ll re-record with a visible timer tonight.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	Which beat to cut?	Duplicate prod anecdote.
L2	—	—
L3	—	—

Provenance: Learning-lab mock discipline

T8. Payment + search skim — what do you refuse to botch? (90s)

Trap: Ramble both systems.

Answer points:

- Payments: no blind POST retry; idempotency mindset (S7)
- Search: debounce/cancel/stale (S3)
- Pick depth if timeboxed

Full spoken answer:

"If I only get two minutes: payments must not blindly retry charge POSTs — status poll / idempotency mindset from checkout UX work. Search must debounce, cancel, and ignore stale. If the interviewer wants depth, I'll pick one and agenda it rather than half-finish both."

Follow-up ladder:

Level	Ask	Point + short answer
L1	S7 claim?	Intent to reduce drop-off/support — no invented %.
L2	—	—
L3	—	—

Provenance: Verified · S7 intent · S3

Revision twin: [../../revision/weeks/week-02/day-14.md](https://revision/weeks/week-02/day-14.md)

05 — Exercises (Mock #2 drills)

Prefer speaking before peeking at solutions.

A. Pre-flight (30–40 min)

1. Pick **Track A or B**. Write your agenda opener on paper from memory.
2. Speak S1 **or** S3 STAR once (match track).
3. Speak S4 **or** S12 opener (spice).
4. Skim [code/MockTalkTracks.md](https://code/mocktalktracks.md) for your track only.

Solution: Opener matches README; STAR matches `03-production-bridge.md` §§3–5.

A2. Week 2 one-liner

Recite the connective narrative without notes.

Solution: Layers/DI → networking cancel/security → SDUI fail-soft → UIKit/SwiftUI lifecycle & identity — BMS, District, Raw.

A3. Provenance hygiene

Rewrite: "I shipped SDUI Ads video with pin rotation runbook and cut splash by 400ms."

Solution:

"I kept Ads video on a native HeroWidget lifecycle (S1) and owned Ads URLSession pinning/whitelist (S4). Pin rotation I'd treat as required design (S4-A1), not a shipped runbook claim. On Aces I shipped server-driven splash for fresher cold start (S12) without quoting an invented millisecond win."

B. Architecture recording (required)

1. Timer 5:00. Record Track A **or** B from [code/MockTalkTracks.md](https://code/mocktalktracks.md).
2. If > 5:30, cut one example and re-record.
3. Optional: 20 min skim of the *other* track bullets (familiarity only).
4. Print agenda via [code/MockAgenda.swift](https://code/mockagenda.swift) once for muscle memory.

Self-score: agenda ≤20s? hard stop? invite questions? no overclaims?

C. Full Mock #2 (60–90 min)

Run the facilitator script in [02-deep-dive.md](#) §1 using [code/Week2Warmups.md](#).

Checklist:

- ☐ 4–5 warm-ups scored (timing guide 1–5)
- ☐ 2 tricky scored
- ☐ 5-min architecture recorded separately
- ☐ STAR delivered (S1 or S3 + spice)
- ☐ Gotchas updated (`week-02`)

C2. Forced tricky draw

If peer unavailable, self-ask T3 (leadership) + T4 (why not SDUI player) or T7 (overtime feedback).

D. Warm-up speed round (15 min)

Answer aloud Q2, Q3, Q5, Q7, Q11 from [04-questions.md](#) — 45–60s each, points-only first pass.

E. Light DSA (optional)

1 stack + 1 linked-list Easy/Medium from Day 13 patterns if energy remains.

F. Week 3 readiness checklist

- ☐ 5-min talk ≤ 5:30
- ☐ S1/S3/S4/S9 openers cold
- ☐ Refresh + unknown SDUI clean
- ☐ Array-as-queue cost remembered
- ☐ Sleep before modularization / images week

Revision twin: [../../revision/weeks/week-02/day-14.md](#)

Week 03

Day 15 — App Modularization, SPM & DI Graphs

Week 3 · Full study (self-contained) · ~4–5 hrs

Revision twin: [revision/weeks/week-03/day-15.md](#)

Outcomes

By end of day, without notes, you can:

- Explain why features depend on **interfaces**, never other feature **implementations**
- Map **SPM** targets to Interface / Impl / Core and what that buys for build parallelism
- Wire a **composition root** (App target) without feature-level singletons
- Deliver **Verified S10**: Stories SDK as a reusable module across the portfolio
- Defend trade-offs: CocoaPods vs SPM, static vs dynamic, constructor/tree DI vs service locator

How to study (in Cursor only)

1. `01-foundations.md` — mental model + glossary

- 2. 02-deep-dive.md — graphs, SPM, DI, SDK extraction (embedded)
- 3. 03-production-bridge.md — Verified S10 STAR + Applied hooks
- 4. code/ — Package sketch + DI + Stories public API
- 5. 04-questions.md — two-layer Q&A
- 6. 05-exercises.md — whiteboard + speaking
- 7. Revision twin for timed recall

Module map

Module	File
Foundations	01-foundations.md
Deep dive	02-deep-dive.md
Production bridge	03-production-bridge.md
Questions	04-questions.md
Exercises	05-exercises.md
Code	code/

Provenance reminder

Label	Meaning
Verified · S10	Standalone reusable Stories SDK; portfolio adoption; public API + isolation
Verified · S1	Soft: Ads as module with POP+Generics boundary
Verified · S9	Soft: Clean/MVVM boundaries + test discipline
Verified · S4	Soft: packaging ≠ architecture (Ads pod path ownership)
Learning-lab	Package.swift / DI sketches in code/

Do **not** invent "80 modules cut build by X%" numbers you don't have.

Agenda opener

"I'll clarify module boundaries, show Interface/Impl plus a DI composition root, then deep-dive Stories-as-SDK and build/link trade-offs."

Time budget

Block	Minutes
Foundations	40–50
Deep dive + code	90–110
Production bridge + S10 STAR	30–40
Questions	45–60
Exercises	30–40
Revision twin	15

01 — Foundations: Why Modules Exist

Intern-clear model first; senior vocabulary second.

1. Plain-English mental model

A growing iOS app becomes a traffic jam if everything imports everything:

- Slow **incremental builds**
- Merge conflicts in the project file
- Hidden **singletons** (`NetworkManager.shared`)
- Impossible **reuse** across apps (copy-paste Stories UI per brand)

Modularization = split into packages/targets with an **allowed import graph**.
DI = the App target **wires** implementations into features that only know **protocols**.

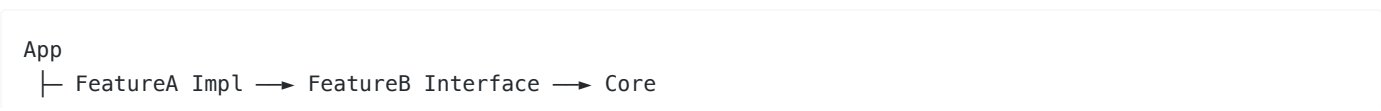
Kitchen metaphor:

Kitchen role	Module role
Recipe card (interface)	Feature Interface — what you can order
Cook (implementation)	Feature Impl — how it’s cooked
Pantry (core)	Network, analytics, design system abstractions
Head chef (app)	Composition root — assigns who cooks what

2. Glossary

Term	Meaning
Feature Interface	Protocols, lightweight DTOs, builder types — no heavy UI/network impl
Feature Impl	Views, VMs, use-cases for one feature
Core	Shared infrastructure abstractions (and carefully shared models)
Composition root	App target that constructs the DI graph
SPM	Swift Package Manager — native packaging
Static linking	Code linked into the app binary at build time
Dynamic framework	Loaded at runtime (dyld cost; useful for app↔extension share)
Constructor injection	Dependencies passed into <code>init</code>
Service locator	Runtime “ask the container” (Swinject-style) — failures late
Needle-style / tree DI	Typed dependency protocols + component tree
Circular dependency	A Impl ↔ B Impl import loop
SDK public API	Stable entry points; hide internals

3. Golden dependency rule



```
└─ FeatureB Impl → FeatureA Interface → Core
└─ wires concrete builders at composition root
```

Forbidden: `FeatureA Impl` imports `FeatureB Impl` .

4. Why interviewers care

- Scale (teams, build time) without hand-waving
- Compile-time safety vs runtime locator surprises
- SDK reuse proof (**S10 Stories**)
- Linking/launch trade-offs

5. 60-second HLD to draw

```
+----- App (DI root, deeplink routing) -----+
| CheckoutImpl    SearchImpl    Stories (SDK)    |
|      |          |            |                |
| CheckoutIface   SearchIface   Stories public API |
|      \          |            /                |
|              CoreNetwork / CoreAnalytics      |
+-----+
```

6. Checkpoint

1. Interface vs Impl in one sentence each
2. Where does DI live?
3. Why folders ≠ modules?

→ [02-deep-dive.md](#)

02 — Deep Dive: SPM Graphs, DI, Stories SDK

Self-contained. Optional Apple/Uber reading is enrichment only.

1. Layer responsibilities

Layer	Owns	Must not own
App	Lifecycle, DI root, module registration, deeplink → feature	Feature business rules
Feature Impl	UI, VM, feature use-cases	Imports of other Feature Impls; .shared network
Feature Interface	Protocols, peer DTOs, Buildable factories	Heavy UIKit VCs, URLSession impl
Core	Network/storage/design/analytics abstractions	Feature-specific product rules

Cross-feature navigation: Feature A depends on `FeatureBBuildable` from B's Interface; App registered the concrete builder.

2. SPM packaging model

```
Package.swift targets (illustrative):
  CheckoutInterface // leaf – rare changes, fast
```

```

Checkout          // Impl – depends on CheckoutInterface + CartInterface + CoreNetwork
CartInterface
Cart
Stories           // reusable SDK product
CoreNetwork / CoreUI / CoreAnalytics

```

Why Interface/Impl split:

- Impls compile in parallel when Interfaces are stable
- Changing Checkout UI doesn't rebuild Cart
- Cycles fail at package resolve time

Worked sketch: [code/Package.swift](#) · [code/StoriesPublicAPI.swift](#)

SPM vs CocoaPods (interview-ready)

	SPM	CocoaPods
Default for new	Yes — native Xcode	Legacy gravity
Ruby / pods repo	No	Yes
Parallelism	Strong	Varies
Binary / weird resources	Sometimes awkward	Often already solved in legacy
Migration	Leaf packages first	Don't big-bang

Senior line: Packaging ≠ architecture. You can have a clean CocoaPod module or a messy SPM soup.

3. DI without hidden globals

Prefer **constructor injection** or a **typed component tree** over service locators:

```

protocol CheckoutDependency {
    var network: NetworkProviding { get }
    var cart: CartProviding { get } // from CartInterface
}

final class CheckoutComponent {
    private let dependency: CheckoutDependency
    init(dependency: CheckoutDependency) { self.dependency = dependency }
    func makeCheckoutVC() -> UIViewController { /* wire VM */ fatalError("sketch") }
}

// App = composition root
final class AppComponent: CheckoutDependency {
    let network: NetworkProviding
    let cart: CartProviding
    // fulfills + registers feature builders
}

```

Worked: [code/CompositionRoot.swift](#)

Anti-patterns

- `NetworkManager.shared` inside feature modules
- `AppDelegate` with 40 singletons
- Interface target secretly importing Impl
- Locator resolves optional deps — crash in prod on first use

Locator pragmatism

Admit legacy Swinject bridges exist; migrate feature-by-feature toward constructor/tree DI. Don't pretend runtime containers are "more senior."

4. Static vs dynamic linking

Choice	When	Cost
Static (default internal)	App modules	Link time; usually better launch than many dylibs
Dynamic	App ↔ Extension shared code	dyld overhead; keep count low
Too many modules	Ego split	Graph/cache thrash; measure Build Timing Summary

5. Stories SDK as reusable module (deep)

Treat Stories like a **product with a public API**, not a folder copied per NBA/WNBA app:

1. **Standalone package** — host supplies theme, analytics, media loader protocols
2. **Stable public API** — entry, callbacks, errors; hide internal VCs/SwiftUI
3. **Host-agnostic content** — inject `StoriesContentProviding`
4. **Versioning** — additive minor; breaking = major + notes
5. **Adoption** — one implementation → portfolio parity

Extraction sequence (interview):

Identify public surface → inject host deps → demo app host →
second production host → semantic version → deprecate carefully

This is **Verified S10** territory — see [03-production-bridge.md](#).

6. Shared models without Core junk drawer

Need	Where
True shared entity (User id)	Small DomainModels / kernel
Checkout-only DTO	Checkout Interface/Impl
"Maybe useful someday"	Not Core

7. Build times got worse after 80 modules

Causes: chatty Interface changes, over-fine splits, too many dynamic frameworks, poor CI caching.
Fix: measure Build Timing Summary; merge leaf packages; stabilize Interfaces; prefer static internals.

Tuist/Bazel: mention when monorepo scale demands — not as cargo cult.

8. Trade-offs summary

Choice	When	Cost
Interface/Impl	Multi-team / large features	More targets
Tree/constructor DI	New modules	Boilerplate at App
Service locator	Legacy glue	Runtime missing deps

SPM	Greenfield / Apple-first	Some vendor binaries awkward
Folders only	Tiny spike	No enforcement
SDK reuse	Portfolio (S10)	Versioning discipline

9. Failure modes

Failure	Fix
Impl⇌Impl import	Depend on Interface; App wires
Interface imports UIKit heavily	Keep lean; factory returns opaque VC
SDK hardcodes Heat branding	Inject theme
SDK owns Kingfisher forever	Inject ImageLoading
Juniors blocked	Templates + forbidden-import lint + golden path

10. Optional citations

- Apple: Swift packages / modularizing apps
- Uber Needle conceptual model
- In-repo: `ios-system-design/docs/app-modularization.md` (optional skim)

03 — Production Bridge: Stories SDK (S10)

1. Provenance map

ID	Label	Exact claim
S10	Verified	Designed standalone reusable Stories SDK; clear public API; isolation from host shortcuts; adopted across portfolio (NBA/WNBA apps); live scoreboard is adjacent Raw ownership but separate story (S11)
S1	Soft	Ads module POP+Generics — module boundary mindset
S9	Soft	Clean/MVVM + tests — module testability culture
S4	Soft	Packaging vs architecture — Ads networking ownership lived behind a module boundary
S13	Soft	Hosting SDK UI in UIKit hosts
Learning-lab	Sketches	code/ Package + DI

Forbidden

- Invented “N apps × M% faster” without evidence
- “I open-sourced Stories internally as Needle” unless true
- Collapsing S10 and S11 into one claim carelessly

2. Verified S10 — STAR (2–3 min)

Opener (~10s)

“I’ll walk through designing Stories as a standalone SDK so multiple NBA/WNBA apps could share one implementation.”

S/T (~20s)

Need Instagram-style fan Stories across the client portfolio — not one-off UI per app.

Action (~90s)

1. Designed a **standalone reusable Stories SDK** with a deliberate public API surface.
2. Kept **isolation** from app-specific networking shortcuts — hosts inject content/analytics/loaders.
3. Hid internals; exposed entry points, callbacks, errors.
4. Drove **adoption across portfolio apps** so feature parity didn't mean copy-paste.
5. Treated versioning/API stability as part of quality (lesson).

Result (~20–30s)

One implementation leveraged by multiple apps → faster feature parity across the portfolio.

Lesson (~15–20s)

SDK quality = API surface + versioning + independence from host app shortcuts.

Provenance: *Verified · S10 · Raw / Miami Heat · Stories SDK portfolio reuse*

3. Interview line (≤20s)

"I shipped Stories as a standalone SDK with a clear public API and injected host dependencies — one module, multiple apps, no copy-paste forks."

4. Applied extensions (speak carefully)

Topic	Line
Image loader	Host injects ImageLoading so portfolio shares one cache policy (Day 16 bridge)
UIKit host	UIHostingController façade; deeplink exits via host router (S13)
Theming	Protocolised tokens — don't hardcode Heat colors in SDK

Provenance: *How I would apply it · extend S10 boundaries (design detail)*

5. Adjacent hooks

S1: Ads as revenue module with POP API — modularization of behavior even if packaging was pod-era.

S9: Independently testable modules; AI stays inside architecture envelope.

S4: Don't confuse CocoaPods vs SPM with "is the Ads boundary clean?"

04 — Questions (two-layer answers)

*Cover full spoken → speak from **Answer points** → compare.*

12 normal + 8 tricky = 20.

Normal questions

Q1. Why modularize an iOS app? (30–45s)

Answer points:

- Build time, ownership, testability, reuse
- Interface/Impl enforcement

- Stories SDK as proof (S10)

Full spoken answer:

"Modularization buys incremental build speed, clearer team ownership, independent tests, and reuse. The key is an enforced import graph — Interface versus Impl — not folders with good intentions. My production proof is the Stories SDK: one module adopted across portfolio apps instead of copy-paste UI."

Common wrong answer: "Modules are just folders."

Follow-up ladder:

Level	Ask	Point + short answer
L1	First module?	Leaf with clear API — design system, network, or Stories.
L2	Story?	S10.
L3	—	—

Provenance: Verified · S10

Q2. CocoaPods vs SPM — how choose? (30–45s)

Answer points:

- SPM default for new
- Pods for binary/legacy gravity
- Incremental migration
- Packaging ≠ architecture

Full spoken answer:

"For new modules I default to SPM — native Xcode, no Ruby. CocoaPods stays when binary pods or legacy resource bundles dominate. Migration is incremental: leaf packages first. And packaging isn't architecture — a CocoaPod can still have a clean API boundary."

Common wrong answer: "Pods are obsolete; always rewrite tomorrow."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Resources in SPM?	Process resources carefully per target.
L2	Ads context?	S4 ownership lived behind a module boundary.
L3	—	—

Provenance: Learning-lab · soft S4

Q3. What belongs in a Feature Interface? (30–45s)

Answer points:

- Protocols + light DTOs + builders
- No heavy VCs / URLSession impl
- May depend on Core carefully

Full spoken answer:

"Interfaces hold protocols, lightweight models peers need, and builder/factory types. They should not own heavy UIKit view controllers or concrete networking. Depending on Core abstractions is OK when it stays lean — Interfaces must stay cheap to compile."

Common wrong answer: "Put the whole feature in Interface for convenience."

Follow-up ladder:

Level	Ask	Point + short answer
L1	UIKit in Interface?	Smell — prefer opaque factories.
L2	—	—
L3	—	—

Provenance: Learning-lab

Q4. Explain a DI composition root. (30–45s)

Answer points:

- App wires graph
- Features receive protocols
- No feature-level `.shared`

Full spoken answer:

"The App target is the composition root: it constructs concrete services and injects them into feature builders. Features depend on protocols, not globals. That compile-time graph beats discovering a missing Swinject registration in production. For Stories, the host injects providers at the root."

Common wrong answer: "Each feature resolves `NetworkManager.shared`."

Follow-up ladder:

Level	Ask	Point + short answer
L1	DeepLink factories?	Registered at App root.
L2	Story?	S10 host injects.
L3	—	—

Provenance: Verified · S10 · Learning-lab DI

Q5. Navigate without importing other Impls? (30–45s)

Answer points:

- Depend on `FeatureBBuildable`
- App registers impl
- Coordinator uses protocol

Full spoken answer:

"Feature A imports FeatureB's Interface and asks DI for a `FeatureBBuildable`. The App registered the concrete builder. That gives compile-time missing-dep failures instead of runtime surprises and prevents `Impl↔Impl` cycles."

Common wrong answer: "Just import FeatureB — it's fine once."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Home routes everywhere?	Home depends on many Interfaces; App wires.

L2	—	—
L3	—	—

Provenance: Learning-lab

Q6. Dynamic vs static — launch impact? (30–45s)

Answer points:

- Many dylibs → dyld cost
- Prefer static internal
- Dynamic for app↔extension share

Full spoken answer:

"Lots of dynamic frameworks cost dyld time at launch. I prefer static linking for internal modules and reserve dynamic for true app-and-extension sharing. Modularization shouldn't mean forty dylibs by accident."

Common wrong answer: "Dynamic is always cleaner."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Binary size?	Watch duplicates; CI budgets.
L2	—	—
L3	—	—

Provenance: Learning-lab

Q7. Independently testable modules? (30–45s)

Answer points:

- Inject protocols
- Mock via Interface
- CI test Checkout without full app

Full spoken answer:

"If dependencies are protocols from Interface packages, unit tests inject fakes and CI can test Checkout without booting the full app. Snapshot tests usually live with Impl UI. That matches the test discipline we used around District architecture work."

Common wrong answer: "Only UITests on the monolith."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Story?	S9 soft.
L2	—	—
L3	—	—

Provenance: Verified · S9 soft · Learning-lab

Q8. Circular dependency — prevent? (30–45s)

Answer points:

- Impl↔Impl is the bug
- Cross-deps only on Interface
- SPM resolve fails early

Full spoken answer:

"Cycles usually mean two Impls import each other. Break them by depending only on Interfaces and letting App register builders. SPM/Xcode will fail resolution instead of giving you mysterious runtime loops."

Common wrong answer: "Import Impl just this once."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Debug SPM?	swift package / dependency graph tools.
L2	—	—
L3	—	—

Provenance: Learning-lab

Q9. Extract Stories into an SDK today? (45–60s)

Answer points:

- Public API → inject content/analytics/loader
- Versioning → demo app → second host
- Theming via protocols

Full spoken answer:

"I'd freeze a public API, inject content, analytics, and image loading, add semantic versioning, prove it in a demo host, then adopt a second production host before wide rollout. Theming stays protocolised so Heat and Aces don't fork the SDK. That's the same shape we used for portfolio Stories reuse."

Common wrong answer: "Copy the SwiftUI files into each app."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Breaking change?	Major bump + migration notes.
L2	Story?	S10.
L3	—	—

Provenance: Verified · S10

Q10. Singletons — ever OK? (30–45s)

Answer points:

- Rare process-wide sinks
- Never hidden in features
- Prefer inject even then

Full spoken answer:

"A process-wide analytics sink might be constructed once at App launch, but features should still receive it via injection. Feature-level `NetworkManager.shared` hides the graph and kills testability. Actors change how we isolate state — they don't excuse hidden globals."

Common wrong answer: "Singletons are fine everywhere for convenience."

Follow-up ladder:

Level	Ask	Point + short answer
L1	S2 bridge?	Shared maps need explicit isolation APIs.
L2	—	—
L3	—	—

Provenance: Learning-lab · soft S2

Q11. Modularization vs CI? (30–45s)

Answer points:

- Parallel package builds
- Selective tests
- Cache SPM artifacts

Full spoken answer:

"Modules let CI build and test only what changed, cache SPM artifacts on runners, and parallelise package builds. That's a Day 20-shaped conversation — today the point is the graph enables selective work."

Common wrong answer: "CI only cares about the app target."

Follow-up ladder:

Level	Ask	Point + short answer
L1	GH Actions?	Preview Day 20.
L2	—	—
L3	—	—

Provenance: Learning-lab

Q12. Binary size risk? (30–45s)

Answer points:

- Dead stripping helps
- Watch duplicate symbols / unused dylibs
- CI budget gates

Full spoken answer:

"Modules don't magically shrink binaries. Dead-code stripping helps, but duplicate symbols and unused dynamic frameworks hurt. I'd put a size budget in CI and watch diffs per release."

Common wrong answer: "More modules always smaller apps."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Metric?	Compressed download / install size.
L2	—	—

L3	—	—
----	---	---

Provenance: Learning-lab

Tricky questions

T1. “Folders as modules — why packages?” (90–120s)

Trap: Equating namespaces with dependency boundaries.

Answer points:

- Folders don’t enforce imports
- SPM/Xcode targets do
- Stories reuse needed a real boundary

Full spoken answer:

“Folders are convention; packages and targets are enforcement. At team scale, convention leaks — someone imports the ‘wrong’ folder and the graph collapses. Stories reuse across portfolio apps required a real module boundary and public API, not a folder named Stories. Folders are enough for a spike; they’re not enough for SDK adoption.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	Tiny app?	Folders OK until pain.
L2	Story?	S10.
L3	—	—

Provenance: Verified · S10

T2. Service locator vs constructor — pick and defend. (90–120s)

Trap: “Whatever the team uses.”

Answer points:

- Prefer constructor/tree for compile-time completeness
- Locator = runtime missing deps
- Pragmatic legacy bridge + migration

Full spoken answer:

“I prefer constructor or typed tree DI because missing dependencies fail at compile time and the graph is readable at the App root. Service locators push failures to runtime and hide who needs what. I’ll bridge legacy Swinject during migration, but I won’t call the locator more senior — it’s a pragmatism tax.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	Migrate singleton feature?	Introduce protocol, inject at edge, delete .shared.
L2	—	—
L3	—	—

Provenance: Learning-lab

T3. Interface started importing UIKit — what’s wrong? (90–120s)

Trap: “UIKit is fine everywhere.”

Answer points:

- Interfaces must stay lean
- Non-UI consumers / fast rebuilds
- Opaque builders / keep UI in Impl

Full spoken answer:

“Heavy UIKit in Interface couples every peer to UI and slows rebuilds. Prefer factory protocols that return an opaque `UIViewController` from Impl, or keep UI types out of Interface entirely. SwiftUI `AnyView` as a public SDK API is often a smell — it erases structure and invites host hacks.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	Extensions?	Non-UI targets can’t take UIKit Interfaces easily.
L2	—	—
L3	—	—

Provenance: Learning-lab

T4. Two features need same User model — where? (90–120s)

Trap: Duplicate everything or dump into Core.

Answer points:

- Shared kernel for true shared entities
- Feature DTOs stay local
- Avoid Core junk drawer

Full spoken answer:

“If it’s truly shared — stable identity fields — put it in a small domain kernel. Feature-specific DTOs stay in that feature’s Interface/Impl. Core becomes a junk drawer when every ‘maybe shared’ struct lands there, and then nothing rebuilds quickly.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	SDK hosts?	Version shared models carefully across apps.
L2	—	—
L3	—	—

Provenance: Learning-lab

T5. Modularize without slowing juniors? (90–120s)

Trap: Process theater.

Answer points:

- Templates Interface+Impl
- DI registration checklist

- Forbidden-import lint
- Golden path package

Full spoken answer:

"Velocity comes from rails: a template for Interface+Impl, a checklist for App registration, one golden-path package to copy, and lint for forbidden Impl→Impl imports. At District we also kept AI inside architecture envelopes — tools don't replace clear module boundaries."

Follow-up ladder:

Level	Ask	Point + short answer
L1	S9?	Context Engineering inside protocols.
L2	—	—
L3	—	—

Provenance: Verified · S9 soft

T6. Ads was a pod — relate to SPM modularization? (90–120s)

Trap: Mixing package manager with architecture.

Answer points:

- Pod vs SPM = packaging
- POP+API = architecture
- S4 ownership inside module boundary

Full spoken answer:

"CocoaPods versus SPM is how you package. Modular architecture is whether Ads exposes a clean API and owns its networking. At BMS, Ads could live as a pod while still having POP contracts and URLSession pinning behind that boundary. You can have clean pods and messy SPM — don't conflate the tools with the graph."

Follow-up ladder:

Level	Ask	Point + short answer
L1	S4?	Alamofire → URLSession inside the module.
L2	—	—
L3	—	—

Provenance: Verified · S4 soft · S1 soft

T7. Build times worse after 80 modules — why? (90–120s)

Trap: "More modules always faster."

Answer points:

- Over-fine splits
- Chatty Interfaces
- Too many dylibs
- Measure; merge; stabilize

Full spoken answer:

"Over-splitting creates chatty Interface changes that rebuild the world, plus dynamic framework overhead and weak caching. I'd measure with Build Timing Summary, merge leaf packages that always change together, stabilize Interface APIs, and prefer

static internals. Tuist or Bazel enter the chat at extreme monorepo scale — not as the first move.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	First metric?	Incremental build of a leaf change.
L2	—	—
L3	—	—

Provenance: Learning-lab

T8. SDK versioning across Heat / Aces / Sky? (90–120s)

Trap: “Always latest everywhere.”

Answer points:

- Semver
- Staggered adoption
- Changelog / API diffs
- Host feature flags if needed

Full spoken answer:

“Semantic versioning, staggered host adoption, and honest changelogs. Portfolio reuse only works with release discipline — a breaking Stories callback needs a major bump and migration notes. Hosts can feature-flag temporarily, but the SDK shouldn’t silently fork per brand.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	Deprecate callback?	Soft deprecate → remove in next major.
L2	Story?	S10.
L3	—	—

Provenance: Verified · S10

Revision twin: [../../revision/weeks/week-03/day-15.md](https://github.com/uber/uber-engineering/blob/master/revision/weeks/week-03/day-15.md)

05 — Exercises

Solutions in-repo. Speak before peeking.

A. Conceptual

A1. Draw the graph

Ticketing app: App, Checkout, Search, Ads, MediaStories, CoreNetwork. Mark Interface vs Impl and one protocol-only nav arrow.

Solution: Impls hang under App; cross arrows only to `*Interface` ; App wires builders. MediaStories/Stories is a reusable product module (S10 shape).

A2. Forbidden import?

`CheckoutImpl` imports `CartImpl` to push a VC. Fix?

Solution: Depend on `CartBuildable` in `CartInterface`; App provides `CartImpl` builder.

A3. Interface smell

`CheckoutInterface` imports `UIKit` and exposes `CheckoutViewController` subclass. What's wrong + fix?

Solution: Interfaces should stay lean. Prefer `CheckoutBuildable` returning opaque `UIViewController` from `Impl`, or keep UI types in `Impl` only.

A4. Provenance rewrite

Bad: "I built Needle at Raw and cut build times 70% with 80 SPM modules."

Solution: "I designed a standalone Stories SDK with a public API and host-injected deps, adopted across portfolio apps (S10). I don't have a verified 70% build-time metric to quote."

A5. Packaging vs architecture

One sentence each: CocoaPods vs SPM; modular architecture.

Solution: Packaging = how you distribute targets. Architecture = Interface/Impl graph + DI + public API. Clean pods and messy SPM both exist.

B. Coding

1. Read [code/Package.swift](#) — explain target deps aloud; name the forbidden edge.
2. Read [code/StoriesPublicAPI.swift](#) — list public protocols hosts must supply.
3. Read [code/CompositionRoot.swift](#) — where is the composition root? Why isn't network a feature singleton?
4. Optional: add `CartBuildable` protocol + register a fake in `AppComponent`.
5. Optional: sketch how `StoriesConfiguration` would be built inside `AppComponent` for Heat vs Aces themes.

C. Speaking drills

1. S10 STAR $\leq$ 3 min (record).
2. Q4 composition root $\leq$ 45s.
3. Q9 extract Stories $\leq$ 60s.
4. T1 folders vs packages $\leq$ 120s.
5. T2 locator vs constructor $\leq$ 120s.
6. T6 pod vs architecture $\leq$ 120s.
7. T8 portfolio versioning $\leq$ 120s.

D. Whiteboard (10–12 min)

Agenda first, then draw:

1. App composition root
2. Two features with Interface-only cross nav
3. Stories SDK box with injected `ImageLoading` / analytics
4. Static vs dynamic callout in one sentence

E. Timed drill

Record **Q4**, **Q9**, **Q6** + **T1**, **T6**. Score via timing guide.

Revision twin: [../..../revision/weeks/week-03/day-15.md](#)

Day 16 — Image/Video Pipelines, Caching & Memory Pressure

Week 3 · Full study (self-contained) · ~4–5 hrs
Revision twin: [revision/weeks/week-03/day-16.md](#)

Outcomes

By end of day, without notes, you can:

- Draw the **image pipeline**: L1 memory → L2 disk → network, with **downsample-before-decode-to-display-size**
- Quote **decoded bitmap math** (width × height × 4) — not JPEG file size
- Explain **dedupe**, **cancellation**, and **cell reuse** for scroll performance
- Deliver **Verified S1**: HeroWidget pause/play on revenue Ads video
- Deliver **Verified S12**: Aces audio + server-driven splash (not an image cache)
- Handle **memory pressure**: NSCache costs, purge, never decode full-res on main

How to study (in Cursor only)

1. 01-foundations.md — mental model + glossary + **cache tiers embedded**
2. 02-deep-dive.md — full pipeline, ImageIO, video/audio distinctions
3. 03-production-bridge.md — S1 + S12 STAR tracks
4. code/ — cache actor sketch + HeroWidget lifecycle
5. 04-questions.md — two-layer Q&A
6. 05-exercises.md — HLD + speaking
7. Revision twin for timed recall

Module map

Module	File
Foundations	01-foundations.md
Deep dive	02-deep-dive.md
Production bridge	03-production-bridge.md
Questions	04-questions.md
Exercises	05-exercises.md
Code	code/

Provenance reminder

Label	Meaning
Verified · S1	HeroWidget pause/play; POP+Generics ads pipeline; revenue-critical (no invented fill-rate %)
Verified · S12	Aces audio streaming; server-driven splash; cold-start improvement (no invented ms)
Verified · S10	Soft: inject image loader into Stories SDK — don't hardcode Kingfisher inside SDK
Learning-lab	Image cache / downsample sketches in code/

Agenda opener

"I'll define cache tiers and downsample math, then cancellation/dedupe, then HeroWidget video lifecycle and how Aces audio differs from image caching."

Time budget

Block	Minutes
Foundations (tiers)	45–60
Deep dive + code	90–120
Production bridge S1/S12	35–45
Questions	45–60
Exercises	30–40
Revision twin	15

Critical correctness (memorize)

Claim	Truth
JPEG 200KB on disk	Decoded bitmap may be multi-MB
URLCache	Stores bytes , not policy for decoded UIImages
Cache key	url + targetSize (not URL alone)
HeroWidget	Pause offscreen / disappear / reuse
Aces audio	Session + interruptions — not LRU bitmaps

01 — Foundations: Images, Caches, Media

Cache tiers are taught **here** — you do not need to leave Cursor.

1. Plain-English mental model

Showing a remote image is a pipeline, not `Data(contentsOf:)` :

1. Ask for a URL at a **target pixel size**
2. Check **memory** for a decoded image already sized
3. Else check **disk** for bytes → decode/downsample off main
4. Else **download**, downsample, store, display
5. If the cell scrolled away — **cancel** and ignore late results

Video ads add a second contract: **don't play when invisible**.

Live audio adds a third: **session interruptions**, not bitmap TTL.

2. Glossary

Term	Meaning
L1	Memory cache of decoded images (NSCache / LRU actor)
L2	Disk cache of encoded bytes (Caches/)
L3	Network / CDN (+ optional HTTP URLCache for bytes)
Downsample	Decode at display size via ImageIO thumbnails

Decoded size	$\approx \text{width} \times \text{height} \times 4 \text{ bytes (ARGB8888)}$
Dedup	One in-flight fetch per cache key; N observers
Generation / token	Ignore stale completions after reuse
Prefetch	Warm upcoming cells with a concurrency budget
Memory warning	Trim L1; pause non-visible media/decodes
HeroWidget	Ads video surface with explicit pause/play (S1)
AVAudioSession	Audio category, interruptions, route changes (S12)
TTI	Time-to-interactive — splash success metric

3. Cache tiers (embedded teaching)

```
imageView.load(url, targetSize: bounds × scale)
  → L1 NSCache (key: url + targetSize)      HIT → UIImage
  → L2 Disk (hash(url) ± size variant)      HIT → decode bg → L1 → UI
  → In-flight dedupe?                       YES → attach observer
  → URLSession fetch
  → ImageIO downsample (max pixel size)
  → Store L1 (+ L2 bytes) → notify observers
```

Tier	Tech	Eviction	Notes
L1	NSCache / custom LRU actor	Memory warning + <code>totalCostLimit</code>	Cost $\approx$ decoded bytes
L2	FileManager / Caches + metadata	LRU + TTL (~7d)	OS may purge — OK for images
L3	CDN / URLSession	HTTP cache / ETag	Not a substitute for decoded L1

Write policy: decode → L1 immediately for UI; write L2 **async** (don't block main on disk).

Math to quote: 1000×1000 ARGB8888 $\approx$ **4MB decoded**. Thirty full-res feed images → catastrophe. Always key by **url + targetSize**.

4. Why URLCache alone is insufficient

`URLCache` may store response **bytes**. UI still needs:

- Downsampled decode
- Memory pressure policy
- Dedupe across image views
- Cancellation on reuse

Senior line: “HTTP cache $\neq$ UIImage cache.”

5. Scroll-safe loading (60s)

1. Cancel when off-screen
2. Dedup same key
3. Priority: visible > prefetch
4. Cap prefetch on fast fling
5. Never decode JPEG on main (<16ms frame budget)

6. Two production media stories (preview)

Strategy	Pros	Cons
Store original bytes once; derive sizes	Saves disk	CPU on each size miss
Store per targetSize	Fast L2 hit	More disk

Either is fine if you **explain**; always include size in **L1 key**.

3. Dedup & cancellation

```
inflight[key] = Task
  observers...
on complete → fan-out → clear inflight
on cancel last observer → cancel Task (policy choice)
```

Cell reuse:

1. `prepareForReuse` — clear image, cancel prior
2. Capture expected URL/token
3. Ignore completion if token mismatch

SwiftUI: `.task(id: url)` cancellation helps — still reason about identity.

4. Prefetch footguns

Benefits: warmer L1/L2 for next cells.

Footguns: decode storms on fling; bandwidth waste.

Fix: bound concurrency; cancel prefetch when direction changes; prioritize visible indexPaths; lower QoS for prefetch vs visible.

5. Memory pressure playbook

On warning / jetsam risk:

1. Trim L1 (`NSCache` helps; custom LRU must listen)
2. Pause non-visible video / decode work
3. Keep L2 (disk) — recreatable
4. Don't wipe user Documents
5. Avoid main-thread purge storms

NSCache vs Dictionary: NSCache can evict under pressure and is friendlier for this use; Dictionary needs manual purge + isolation (actor/lock). NSCache is **not** a strict deterministic LRU — say that.

6. HeroWidget video (S1 deep)

Revenue Ads video must pause when:

- Partially / fully off-screen (pager / scroll)
- `viewWillDisappear`
- App background
- Cell `prepareForReuse`

POP + Generics: ad component protocols so creative types don't fork the loader. Lifecycle is a **product contract**.

Retain cycles: AVPlayer + observers + closures capturing `self`. Pause alone doesn't release — nil player, remove observers, break cycles. Instruments Allocations if players leak.

Worked: [code/HeroWidgetLifecycle.swift](#)

Ads vs editorial autoplay: Ads may need viewability for billing; editorial may mute-autoplay. Encode policy in protocol — don't assume one policy.

7. Aces audio + splash (S12 deep)

Audio ≠ image cache:

Concern	Images	Live audio
Storage	L1/L2 bitmaps/bytes	Continuous buffer / player item
Failure	Retry thumbnail	Interruptions, routes, buffering
Lifecycle	Cancel on reuse	AVAudioSession interruption + resume policy
Background	Usually stop	May continue with background modes

Splash SDUI: server-driven content for freshness/flexibility; measure **time-to-interactive**; cache last-good splash if network slow. (No invented ms.)

8. Should Stories SDK own the image pipeline? (S10 bridge)

No **hardcode Kingfisher inside SDK forever**. Inject `ImageLoading` from host so portfolio apps share one loader/policy (Day 15). SDK stays reusable.

9. Trade-offs

Choice	When	Cost
NSCache L1	Default decoded	Non-deterministic eviction
Custom LRU actor	Strict cost + metrics	More code
Downsample always	UI display	CPU; still beats OOM
Full-res disk only	Zoom/edit	Second decode
Aggressive prefetch	Predictable lists	Fling storms
Third-party loader	Speed	Know internals in interviews
Pause offscreen video	Ads/feeds	Visibility tracking complexity

10. Failure modes

Symptom	Likely cause
Wrong image in cell	Missing cancel/token
RAM spike, small disk	Full-res decoded L1 / image views
Hitch on scroll	Main-thread decode
Off-screen ad audio/video	Missing HeroWidget contract
Audio dies on phone call	Missing interruption handler
Same URL avatar+hero blurry/huge	Keyed by URL only

11. Optional citations

- WWDC Image and Graphics Best Practices / Downsampling
- [ios-system-design/docs/image-loading-library.md](#) (optional skim)

03 — Production Bridge: HeroWidget (S1) + Aces (S12)

1. Provenance map

ID	Label	Exact claim
S1	Verified	Ads refactor; POP+Generics; HeroWidget pause/play on revenue-critical module
S12	Verified	Live audio streaming; server-driven splash; cold-start improvement (no invented ms)
S10	Soft	Inject image loader into Stories SDK
S3	Soft	Fail-soft / SDUI mindset for splash payloads
Learning-lab	Sketches	code/ image cache + lifecycle

Forbidden

- Invented fill-rate % or splash latency ms
- “I wrote Kingfisher” / “I invented NSCache”
- Treating Aces audio as “we used image LRU for audio”

2. S1 STAR — HeroWidget (2–3 min)

Opener

“I’ll walk through our highest-revenue Ads refactor — protocols, generics, and video lifecycle.”

S/T

Highest-revenue Ads module needed safer reusable rendering; video inside HeroWidget needed correct pause/play with lifecycle.

Action

1. Protocol-oriented ad contracts + generics pipeline.
2. HeroWidget with explicit pause/play tied to visibility / VC lifecycle / reuse.
3. Prevented creative-type forks.
4. Stakeholder coordination on revenue behavior.

Result

Maintainable typed pipeline; lifecycle-correct video reduced wasted playback and glitches. (**No fill-rate %.**)

Lesson

Lifecycle is part of the product contract on revenue UI.

Provenance: *Verified · S1 · BookMyShow · Ads / HeroWidget*

3. S12 STAR — Audio + splash (2–3 min)

Opener

“On Aces I’ll cover live audio streaming and a server-driven splash for cold-start freshness.”

S/T

Need live broadcast audio and a splash that wasn’t slow/inflexible.

Action

1. Integrated **audio streaming** for live broadcasts (session, buffering, interruption mindset).
2. Revamped splash with **server-driven content** for freshness / flexibility.
3. Oriented success around **time-to-interactive**, not vanity first-frame alone.

Result

Richer live audio; faster/more flexible cold-start content. **(No invented ms.)**

Lesson

Startup path is a product surface; audio failure modes $\neq$ image cache failure modes.

Provenance: *Verified · S12 · Raw / Las Vegas Aces*

4. Combined ≤ 20 s line

"On our highest-revenue Ads surface I built HeroWidget with explicit pause/play tied to visibility — and on Aces I shipped live audio plus a server-driven splash to tighten cold start."

5. Interview bridges

Question	Hook
Design image loader	L1/L2/L3 + downsample math (Learning-lab)
Video off-screen	S1 HeroWidget
Phone call kills audio	S12 AVAudioSession interruptions
SDK image dependency	S10 inject loader
Memory graph UIImage spike	Decoded L1 / no downsample

04 — Questions (two-layer answers)

*Cover full spoken $\rightarrow$ speak from **Answer points** $\rightarrow$ compare.*
12 normal + 8 tricky = 20.

Normal questions

Q1. Design an image loader for a feed. (45–60s)

Answer points:

- L1/L2/network
- Downsample to targetSize
- Dedupe + cancel on reuse
- Decoded cost math

Full spoken answer:

"I'd build a coordinator with L1 memory keyed by url plus target size, L2 disk for bytes, then network. Decode with ImageIO downsampling off the main thread, dedupe in-flight requests, and cancel when cells reuse. Decoded bitmaps dominate RAM — a 1000×1000 image is about four megabytes — so we never cache full-res for thumbnails."

Common wrong answer: "Just URLSession + UIImage(data:) on main."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Module?	Core image loader; inject into features/SDK.
L2	—	—
L3	—	—

Provenance: Learning-lab

Q2. Why not cache full-resolution UIImage? (30–45s)

Answer points:

- Decoded » file size
- 4 bytes/pixel
- OOM under scroll

Full spoken answer:

"JPEG size on disk is irrelevant compared to decoded ARGB — width times height times about four bytes. Caching full-res UIImages in a feed will OOM. Keep display-sized bitmaps in L1; keep full-res on disk only if zoom/edit needs a second decode."

Common wrong answer: "200KB JPEG is fine in memory."

Follow-up ladder:

Level	Ask	Point + short answer
L1	When full-res?	Crop/zoom/edit flows.
L2	—	—
L3	—	—

Provenance: Learning-lab

Q3. NSCache vs Dictionary for images? (30–45s)

Answer points:

- NSCache evicts under pressure
- Cost limits
- Dict needs manual purge + isolation
- Not strict LRU

Full spoken answer:

"NSCache is a better default for decoded images because it can evict under memory pressure and supports cost limits. A Dictionary needs an actor or locks plus your own purge on warnings. I'd also admit NSCache isn't a guaranteed ordered LRU if the interviewer wants strict eviction metrics."

Common wrong answer: "NSCache is a perfect LRU."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Cost?	Approx decoded byte size.
L2	—	—
L3	—	—

Q4. Cell reuse wrong image — fix? (30–45s)

Answer points:

- Cancel prior
- Clear image
- Token / URL match
- Ignore stale

Full spoken answer:

"On reuse I cancel the prior request, clear the image view, capture the expected URL or generation token, and ignore completions that don't match. Without that, fast scroll shows the classic wrong-poster bug."

Common wrong answer: "UIKit bug — nothing we can do."

Follow-up ladder:

Level	Ask	Point + short answer
L1	SwiftUI?	task cancellation + stable id.
L2	—	—
L3	—	—

Q5. Request deduplication? (30–45s)

Answer points:

- Map key → in-flight task + observers
- One network, N callbacks
- Different targetSizes = different keys

Full spoken answer:

"I keep a map from cache key to an in-flight task and observer list. Ten cells asking for the same poster share one download and fan-out on completion. Avatar 40pt and hero 400pt are different keys even if the URL matches."

Common wrong answer: "One cache entry per URL only, always."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Cancel policy?	Cancel when last observer leaves — product choice.
L2	—	—
L3	—	—

Q6. Video ad playing off-screen — what's wrong? (30–45s)

Answer points:

- Missing visibility/lifecycle pause
- Wasted QoS/UX

- HeroWidget contract (S1)

Full spoken answer:

"The player isn't tied to visibility. On our revenue Ads surface, HeroWidget made pause/play an explicit contract with visibility, disappear, background, and reuse — so video doesn't keep playing off-screen."

Common wrong answer: "Users will scroll back — leave it playing."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Pager detect?	Percent visible / willDisplay.
L2	Story?	S1.
L3	—	—

Provenance: Verified · S1

Q7. Memory warning mid-scroll — what happens? (30–45s)

Answer points:

- Trim L1
- Pause non-visible video/decodes
- Keep L2
- Avoid main-thread storm

Full spoken answer:

"Trim the memory image cache, pause non-visible video and decode work, keep disk cache, and don't schedule a huge purge on the main thread. Documents data stays; Caches are recreatable."

Common wrong answer: "Delete the user's saved tickets."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Metrics?	Bridge Day 17 / MetricKit.
L2	—	—
L3	—	—

Provenance: Learning-lab

Q8. Prefetching — benefits and footguns? (30–45s)

Answer points:

- Warm next cells
- Cancel on direction change
- Cap concurrency
- Decode storms on fling

Full spoken answer:

"Prefetch warms L1/L2 for upcoming cells, but unbounded prefetch on a fast fling creates decode and bandwidth storms. I cap concurrency, cancel when scroll direction changes, and prioritize visible index paths."

Common wrong answer: "Prefetch everything always."

Follow-up ladder:

Level	Ask	Point + short answer
L1	API?	UICollectionViewDataSourcePrefetching.
L2	—	—
L3	—	—

Provenance: Learning-lab

Q9. Downsampling API you'd name? (30–45s)

Answer points:

- CGImageSourceCreateThumbnailAtIndex
- Max pixel size
- Background queue

Full spoken answer:

"ImageIO — create an image source, then CGImageSourceCreateThumbnailAtIndex with kCGImageSourceThumbnailMaxPixelSize set to the display pixel budget, on a background queue. That's how you avoid allocating a 12-megapixel bitmap for a 120-point thumbnail."

Common wrong answer: "UIImageJPEGRepresentation then scale."

Follow-up ladder:

Level	Ask	Point + short answer
L1	From file?	CGImageSourceCreateWithURL.
L2	—	—
L3	—	—

Provenance: Learning-lab

Q10. Live audio vs image caching? (30–45s)

Answer points:

- Continuous buffer / session
- Interruptions & routes
- Not LRU bitmaps
- S12

Full spoken answer:

"Live audio is a continuous buffer with AVAudioSession categories, route changes, and interruption handling — not an LRU of bitmaps with a seven-day TTL. On Aces we integrated live audio streaming; I'd never describe that as 'the image cache with a longer TTL.'"

Common wrong answer: "Same pipeline, different MIME type."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Background?	Modes / Now Playing if product needs.

L2	Story?	S12.
L3	—	—

Provenance: Verified · S12

Q11. Server-driven splash — what measure? (30–45s)

Answer points:

- Time-to-interactive
- Last-good cache fallback
- Not vanity TTFF alone

Full spoken answer:

"I'd measure time-to-interactive / first meaningful content, with a cached last-good splash if the network is slow. First-frame vanity alone can lie. On Aces the splash was server-driven for freshness and flexibility."

Common wrong answer: "Only optimize splash animation fps."

Follow-up ladder:

Level	Ask	Point + short answer
L1	SDUI versioning?	Same fail-soft as header (S3 mindset).
L2	No ms claim?	Correct — don't invent.
L3	—	—

Provenance: Verified · S12

Q12. GIFs / animated images? (30–45s)

Answer points:

- Separate decoder path
- Higher memory
- Often out of scope — call it out

Full spoken answer:

"Animated formats need a separate decoder and higher memory budget. In system-design interviews I call them out of scope unless asked, rather than pretending the JPEG pipeline handles GIF frames the same way."

Common wrong answer: "Same as still images."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Video vs animated WebP?	Product/ perf trade-off.
L2	—	—
L3	—	—

Provenance: Learning-lab

Tricky questions

T1. "URLCache is enough for images." (90–120s)

Trap: HTTP cache = UIImage cache.

Answer points:

- URLCache stores bytes
- Still need decode/downsample/memory policy
- App-level pipeline required

Full spoken answer:

"URLCache can help with HTTP bytes, but the UI still needs downsampled decoding, memory cost limits, dedupe, and cancellation. Without an app-level image pipeline you'll hitch and OOM even with a warm URLCache. I lean on URLCache as L3 byte help, not as the product cache."

Follow-up ladder:

Level	Ask	Point + short answer
L1	When lean on it?	Revalidation / bandwidth save for originals.
L2	—	—
L3	—	—

Provenance: Learning-lab

T2. Same URL, 40pt vs 400pt — one cache entry? (90–120s)

Trap: Key by URL only.

Answer points:

- Key includes target size
- Don't store huge and scale on main
- Disk may store original once

Full spoken answer:

"L1 keys include target size — avatar and hero are different entries. Storing one huge bitmap and scaling on the main thread causes hitches. Disk might keep original bytes once and derive sizes, but memory must not treat them as one UIImage."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Scale factor?	Bounds × screen scale.
L2	—	—
L3	—	—

Provenance: Learning-lab

T3. Player retain cycle in HeroWidget. (90–120s)

Trap: Only talk UI pause.

Answer points:

- AVPlayer / observers / self captures
- Pause ≠ release
- prepareForReuse + nil player

- Allocations / Memory Graph

Full spoken answer:

"Pause stops playback; it doesn't break retain cycles. AVPlayer observers and escaping closures that capture self can keep the widget alive. On reuse and disappear I remove observers, nil the player, and cancel loads. If memory climbs, Allocations or Memory Graph usually show the player subgraph. POP helps because I can inject a fake player in tests."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Story?	S1.
L2	—	—
L3	—	—

Provenance: Verified · S1 · Applied Instruments triage

T4. Memory graph UIImage spike, small disk. (90–120s)

Trap: Blame network.

Answer points:

- Decoded L1 / image views holding full-res
- Check downsample path
- NSCache cost calc

Full spoken answer:

"Disk holding encoded bytes can look small while decoded bitmaps explode RAM. I'd verify downsample is actually used, that L1 costs reflect decoded size, and that image views aren't retaining full-res copies. Network isn't the first suspect when disk is tiny and UIImage is huge."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Cost formula?	w×h×4 roughly.
L2	—	—
L3	—	—

Provenance: Learning-lab

T5. Autoplay policy ads vs editorial. (90–120s)

Trap: One policy.

Answer points:

- Ads may need viewability
- Editorial mute-autoplay OK
- Protocolise policy
- HeroWidget enforces lifecycle

Full spoken answer:

"Ads and editorial don't share one autoplay religion. Ads may require viewability for billing; editorial may mute-autoplay for UX. I'd encode policy in a protocol and let HeroWidget enforce lifecycle either way — and coordinate with stakeholders so revenue behavior stays intentional."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Fill rates?	Don't invent %.
L2	Story?	S1.
L3	—	—

Provenance: Verified · S1**T6. Fast fling decode backlog — fix. (90–120s)****Trap:** “Buy bigger devices.”**Answer points:**

- Bound decode concurrency
- Cancel stale
- Prioritize visible
- Placeholder longer OK

Full spoken answer:

“I'd bound the decode queue's concurrency and QoS, cancel work for cells that scrolled away, prioritize currently visible index paths, and accept placeholders a bit longer under fling. Task groups or an OperationQueue with max count both work — the product fix is prioritisation, not infinite parallelism.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	Prefetch?	Downgrade/cancel on fling.
L2	—	—
L3	—	—

Provenance: Learning-lab**T7. Aces audio drops on phone call. (90–120s)****Trap:** Ignore AVAudioSession.**Answer points:**

- Interruption notifications
- Resume policy
- Route changes
- ≠ image retry

Full spoken answer:

“That's an AVAudioSession interruption. I'd observe interruptions and route changes, define resume policy for live versus VOD, and update UX accordingly. Retrying like a failed image download misses the point — the session was stolen by the call.”

Follow-up ladder:

Level	Ask	Point + short answer
L1	Story?	S12.

L2	Live vs VOD?	Live may catch up; VOD resume position.
L3	—	—

Provenance: Verified · S12

T8. Should Stories SDK own the image pipeline? (90–120s)

Trap: Hardcode Kingfisher in SDK.

Answer points:

- Inject ImageLoading from host
- Portfolio shares policy
- SDK stays reusable (S10)

Full spoken answer:

"I'd inject an ImageLoading protocol from the host. Hardcoding a third-party loader inside the SDK forks cache policy per app and couples releases. Portfolio reuse worked because Stories stayed independent of host shortcuts — the loader is one of those injected seams."

Follow-up ladder:

Level	Ask	Point + short answer
L1	Day 15?	Composition root wires loader.
L2	Story?	S10.
L3	—	—

Provenance: Verified · S10 soft · Day 15

Revision twin: [.././../revision/weeks/week-03/day-16.md](https://github.com/uber/revision/weeks/week-03/day-16.md)

05 — Exercises

Solutions in-repo. Speak before peeking.

A. Conceptual

A1. Draw pipeline from memory

Include L1/L2/network, downsample, dedupe, cancel.

Solution: See [01-foundations.md](#) §3 and [code/CacheTiersCheatsheet.md](#).

A2. Math

Decoded size of 2000×1000 ARGB8888?

Solution: 2000×1000×4 = 8,000,000 bytes ≈ 7.6 MiB.

A3. True/false

1. URLCache stores decoded UIImages.
2. Cache key should include targetSize.
3. Pause always releases AVPlayer.
4. Aces audio uses image TTL eviction.

Solution: 1 F. 2 T. 3 F. 4 F.

A4. Tier table from memory

Fill L1/L2/L3 tech + eviction without looking.

Solution: L1 NSCache/LRU + memory warning/cost; L2 `/Caches` + LRU/TTL; L3 network/URLCache bytes.

A5. Provenance rewrite

Bad: "HeroWidget cut fill rate 12% and splash by 400ms using NSCache for audio."

Solution: "HeroWidget tied Ads video pause/play to visibility on a revenue module (S1) — no invented fill-rate. Aces shipped live audio and server-driven splash for cold-start freshness (S12) — no invented ms; audio ≠ image NSCache."

B. Coding

1. Narrate [code/ImageCacheTiers.swift](#) — where is cost computed? Where is dedupe?
 2. Trace [code/HeroWidgetLifecycle.swift](#) — list all pause triggers + `tearDown` path.
 3. Hand-simulate: cell shows URL A, reused for URL B, late A completion arrives — what must be true to ignore it?
 4. Optional: add cancel that removes an observer and cancels Task when observers empty.
 5. Optional: write `estimatedDecodedBytes` asserts for 40pt vs 400pt @3x mentally.
-

C. HLD speaking (10 min)

Whiteboard:

1. Image loader boxes
2. Add **video ad** box: visibility → pause/play → analytics beacon
3. Side note: Aces audio interruption path ≠ image retry

Agenda opener from README first.

D. Speaking drills

1. Q1 image loader ≤ 60s
 2. Q6 off-screen video ≤ 45s
 3. Q10 audio vs images ≤ 45s
 4. S1 STAR ≤ 3 min
 5. S12 opener ≤ 45s
 6. T2 cache key ≤ 120s
 7. T3 player cycle ≤ 120s
 8. T8 SDK image injection ≤ 120s
-

E. Timed drill

Record **Q1, Q6, Q10 + T2, T3**. Score with timing guide.

Revision twin: [../../revision/weeks/week-03/day-16.md](#)

Day 17 — Performance: Instruments, MetricKit, Startup & Scrolling (p50/p90)

Week 3 · Full study (self-contained) · ~4–5 hrs

Revision twin: [revision/weeks/week-03/day-17.md](#)

Outcomes

By end of day, without notes, you can:

- Run the senior performance loop: **symptom** → **hypothesis** → **lab/field tool** → **fix** → **verify p50/p90**
- Separate **lab** (Instruments) from **field** (MetricKit, Firebase Performance) truth
- Explain cold-start phases (pre-main → init → first frame → TTI) and scroll hitch budgets (~16ms @ 60fps)
- Name Instruments with purpose — and correctly say **retain cycles ≠ Leaks** (use **Memory Graph / Allocations**)
- Deliver **Verified · S5**: Firebase Performance journey traces (listing / checkout / search) with **p50/p90** at BMS scale

How to study (in Cursor only)

1. [01-foundations.md](#) — mental model + glossary
2. [02-deep-dive.md](#) — Instruments matrix, MetricKit, startup, hitches, APM shape
3. [03-production-bridge.md](#) — Verified S5 + adjacent hooks
4. [code/](#) — journey trace sketch + Instruments tool cheat
5. [04-questions.md](#) — speak from **Answer points**; compare to **Full spoken**
6. [05-exercises.md](#) — drills + timed practice
7. Revision twin for flashcards / day-of recall

Module map

Module	File	Role
Foundations	01-foundations.md	Perf loop, percentiles, lab vs field
Deep dive	02-deep-dive.md	Instruments (correct tools), MetricKit, startup, scroll
Production bridge	03-production-bridge.md	S5 STAR + honesty bounds
Questions	04-questions.md	Two-layer Q&A (12N + 8T)
Exercises	05-exercises.md	Whiteboard + speaking drills
Code	code/JourneyTrace.swift , code/InstrumentsToolMap.swift	Teaching sketches

Critical correctness (memorize)

Claim	Truth
Retain cycle / abandoned VC	Still reachable → Memory Graph / Allocations growth
Instruments Leaks	Finds unreachable memory (no live pointers)
Cycles in Leaks?	No — common interview trap
Why p90	Tail latency = real pain at scale; averages hide it
Verified S5	Firebase Performance traces; p50/p90 on listing/checkout/search

Provenance reminder

Label	Use for today
Verified · S5	Firebase Performance; journey p50/p90
How I would apply it · S5-A1	Journey-level traces vs per-request interceptor spans

Verified · S12	Soft bridge: server-driven splash / cold-start product surface
Verified · S6	Soft bridge: fewer navigations as UX performance (30%+ flows)
Verified · S3	Soft bridge: search debounce/cancel when “search feels laggy”
Learning-lab	Code sketches in this chapter

Do **not** invent cold-start ms SLAs, fill-rate %, or “Leaks found our retain cycle.”

Agenda opener (pin this)

“I’ll separate lab vs field, define journey SLIs with p50/p90, attribute with the right Instruments tool — Graph/Allocations for cycles, not Leaks — then close on BMS Firebase Performance.”

Time budget

Block	Minutes
Foundations	45–60
Deep dive + code	90–120
Production bridge + S5 STAR	30–40
Questions (record 5+)	45–60
Exercises + revision twin	30–45

01 — Foundations: Performance Investigation

Read this first. Goal: a crisp mental model you can teach an intern, then the senior edges for interviews.

0. One-sentence north star

Define a user journey SLI, measure with percentiles in the field, attribute with the right lab tool, fix the smallest high-leverage cause, then re-verify p50/p90.

Everything below unpacks that sentence.

1. The performance loop (senior habit)

1. Define user journey + SLI (e.g. checkout success latency)
2. Pull field percentiles OR reproduce in lab
3. Attribute: CPU / wait / network / layout / decode / lock / memory
4. Fix smallest high-leverage cause
5. Re-measure p50/p90 + watch next release for regression

Say aloud: “I never optimize from a single desk iPhone anecdote at 30L+ DAU scale — field distributions lie differently than my device.”

1.1 Lab vs field

Mode	Tools	Strength	Weakness
------	-------	----------	----------

Lab	Instruments, local repro	Precise attribution, signposts	Not representative of all devices/networks
Field	MetricKit, Firebase Performance, Crashlytics hang proxies	Fleet truth, percentiles	Coarser / delayed / sampled

Staff signal: Start where the signal lives. Rare field-only regressions → field first to locate the segment, then lab to attribute.

2. Why p50 / p90 (not averages)

Metric	What it answers
Average	Central tendency — hides tails
p50	“Typical” user
p90	Worst 10% — weak devices, peak traffic, slow networks
p99	Extreme tail — useful for rare Sev paths; noisy on mobile

Interview line:

*“At BookMyShow we instrumented Firebase Performance for listing, checkout, and search and tracked **p50/p90** so prioritisation followed real tails, not averages.”*

Provenance: Verified · S5 · BookMyShow · Firebase Performance p50/p90

Trap: Shipping because “average FPS improved.” Average vanity; check hitch rate and field percentiles.

3. Glossary (intern → mid)

Term	Meaning
SLI	Service level indicator — measurable user journey signal
Cold start	Process not in memory; includes dyld / pre-main
Warm start	Process alive; UI may still rebuild
TTFF	Time to first frame
TTI	Time to interactive (data-ready useful UI)
Hitch	Frame missed vsync / display deadline
Hang	Main thread blocked long enough to feel frozen (tool-defined threshold, often ~250ms+ in APM designs)
Watchdog kill	OS kills unresponsive app — related but not identical to hang metrics
Signpost	Custom interval marker (<code>os_signpost</code>) for Points of Interest
Journey trace	Product span (screen appear → success) you define in APM
Abandoned memory	Still referenced but unused (retain cycles)
True leak	Allocated with no live references

4. Instruments — name with purpose (first pass)

Tool	Use when	You'll find
Time Profiler	CPU hot paths	Heavy JSON, regex, main-thread work
os_signpost / Points of Interest	Custom intervals	Journey spans you define
Allocations	Growth over time	Persistent objects, image spikes
Leaks	Unreachable allocations	True leaks (no pointers)
Memory Graph	Ownership edges	Retain cycles, abandoned VCs
Animations / Hitches	Scroll jank	Main-thread >~16ms frames
Network	Payload / latency	Chatty APIs, large JSON
App Launch	Cold start	Pre-main vs post-main cost
Core Animation	Overdraw / blending	Offscreen rendering, shadows

Critical correctness — retain cycles vs Leaks

Retain cycle / abandoned VC

- objects still REACHABLE (they point at each other)
- Leaks instrument usually CLEAN
- Use Memory Graph (edges) + Allocations (persistent growth)

Claim	Truth
"Leaks finds retain cycles"	False
Correct tools for cycles	Memory Graph, Allocations
What Leaks finds	Unreachable allocated memory

Say aloud once: "Retain cycles are abandoned but reachable — that's Graph and Allocations, not Leaks."

(Day 03 deepens ARC; today you only need the Instruments correctness trap when discussing performance tooling.)

5. MetricKit (field OS aggregates)

Subscribe via `MXMetricManager` / `MXMetricManagerSubscriber` :

- Hang diagnostics / hitch metrics
- CPU / memory histograms
- Disk write exceptions
- Exit reasons (pairs with Day 18 crash/OOM narratives)

Cadence: Aggregated payloads — think **daily**, not realtime dashboards. Privacy-preserving OS metrics.

Staff signal: MetricKit ≠ product journey SLIs. Combine with **custom traces** (Firebase Performance) for listing/checkout/search.

6. Startup anatomy (names you must say)

Pre-main (dyld, static init, +load-class work)

- main / App / Scene setup
- First frame (TTFF)
- Time-to-interactive (data-ready UI)

Optimizations interviewers expect:

- Fewer dynamic frameworks on launch path (modularization — Week 3 Day 15)
- Defer non-critical SDK init
- Avoid sync disk/network on main during launch
- Cache splash/config when product allows (Aces server-driven splash — S12 soft bridge)

Do not invent a fake “we hit 847ms cold start” number. Speak categories and measurement method.

7. Scrolling & hitches (first pass)

- **Budget:** ~16.7ms per frame @ 60Hz; ~8.3ms @ 120Hz
 - **Hitch:** work missed the deadline → jank
 - Common causes: main-thread image decode, Auto Layout thrash, sync file I/O, lock contention, huge cell `configure`
 - Fixes preview: downsample/prefetch (Day 16), reuse hygiene, flatten hierarchies, parse off main, debounce search (S3)
-

8. Firebase Performance at BMS (production proof — preview)

Instrumented **p50/p90** traces across **listing, checkout, search** — an observability layer for data-driven optimisation.

Interview structure:

1. Why percentiles (tail = pain)
2. Which journeys (business-critical)
3. How traces informed PM/backend prioritisation
4. Link to release regression watch (Day 20 CI gates)

Full STAR in [03-production-bridge.md](#) .

9. Mini decision tree — which tool first?

```
“App feels slow”
├─ Can you reproduce on desk? → Instruments (Time Profiler / Hitches / Launch)
├─ Only some users / devices? → Field first (Firebase p90 segment, MetricKit)
├─ Memory climbs on navigate? → Allocations + Memory Graph (not Leaks-first for cycles)
├─ Scroll jank? → Hitches / Time Profiler on main
└─ Checkout/search latency? → Journey traces (S5) + Network instrument if lab
```

10. Teach-back checklist

Before deep dive, say without notes:

1. Perf loop in five steps
 2. Why p90 over average
 3. Lab vs field
 4. Cold-start four phases
 5. Hitch vs hang (one sentence each)
 6. **Retain cycles → Graph/Allocations, not Leaks**
 7. S5 one-liner with p50/p90
-

11. What's next

[02-deep-dive.md](#) expands Instruments failure modes, MetricKit subscriber shape, hang detector concept, APM upload NFRs, and scroll attribution playbooks.

02 — Deep Dive: Instruments, MetricKit, Startup, Scrolling

Mechanics, traps, trade-offs. Self-contained — you do not need the APM doc open, but it mirrors that design vocabulary.

1. Attribution before optimisation

Senior failure mode: micro-optimising JSON encoders when p90 is **network wait**.

Symptom class	Often looks like	First attribution
High CPU on main	Scroll jank, launch stutter	Time Profiler
Waiting / idle CPU	"Slow" but clean profiler	Network / backend / lock wait
Memory climb across pushes	Eventually jetsam	Allocations + Memory Graph
Field-only p90 spike	Desk is fine	Firebase / MetricKit segment
Launch regression	New SDK	App Launch template (pre vs post main)

Rule: Name the **bottleneck class** before proposing a fix.

2. Instruments matrix (interview-ready)

2.1 Time Profiler

- Sampling profiler — heavy **self-time** and **heaviest stacks**
- Look for unexpected main-thread work: JSON decode, regex, image work, sorting
- After fix: confirm the stack shrinks (don't trust vibes)

Trap: Using Time Profiler to "find" network latency. Waiting ≠ computing.

2.2 Points of Interest / `os_signpost`

- Custom intervals around journeys you care about
- Bridge lab to product language ("checkout_tap → confirmation")
- Discipline: too many signposts = noise

2.3 Animations / Hitches

- Hitch = frame late vs display link
- Correlate hitch regions with Time Profiler samples on main
- 120Hz devices tighten the budget — same bugs, less margin

2.4 App Launch

- Separates **pre-main** (dyld, static) from **post-main**
- Classic regressions: dynamic framework load, sync Keychain, eager analytics SDKs
- Pair with "defer non-critical init" plans (keep crash SDK early — Day 18)

2.5 Network instrument

- Payload sizes, chatter, DNS, TLS, request fan-out
- When Firebase network/journey p90 is high and CPU is clean → stay here / backend

2.6 Core Animation

- Offscreen rendering, blending, masks, shadows
- Useful when GPU/commit cost dominates — less common than main-thread decode

2.7 Memory tools — correctness section

Tool	Finds	Does not find well
Leaks	Unreachable allocations	Retain cycles (still reachable)
Allocations	Persistent growth, spikes	Ownership <i>why</i> without Graph
Memory Graph	Retain edges, cycles, abandoned VCs	Every historical transient

Interview script (memorize):

"A retain cycle keeps objects alive through mutual strong references — they're abandoned but still reachable, so the Leaks instrument often stays clean. I use Memory Graph to see the ownership edges and Allocations to confirm those objects persist across navigation generations."

Wrong answer: "We opened Leaks and saw the cycle."

3. MetricKit deep dive

3.1 What you get

- Aggregated CPU, memory, hang/hitch-related diagnostics
- Exit / diagnostic payloads useful with crash narratives
- Delivered on a **delay** (think ~24h class aggregates) — not a live pager by itself

3.2 How you use it in speech

1. Fleet-level OS truth across device classes
2. Segment older devices when scroll jank is class-specific
3. Combine with custom journey traces for product SLIs
4. Privacy: OS aggregation — still scrub anything you add in custom layers

3.3 MetricKit vs Firebase Performance

	MetricKit	Firebase Performance (S5)
Owner	Apple OS aggregates	Your custom traces / network
Journey SLIs	Weak alone	Strong (you define spans)
Latency of signal	Daily-ish	Nearer to release monitoring
Use together?	Yes	Yes

4. Cold start — measurement & optimisation

4.1 Measure correctly

- Process start ≠ `didFinishLaunching` — pre-main happens earlier
- APM designs often use `sysctl` / process start + first-frame markers
- Break into: **preMain + appInit + firstFrame** (+ TTI if you define data-ready)

4.2 Optimisation playbook (say 4–5)

1. Reduce work before first frame (defer SDKs, avoid sync I/O)
2. Watch dynamic framework / dyld cost
3. Don't block main on network for splash content when cacheable
4. Parallelize safe init carefully (ordering: crash SDK early)
5. Product: server-driven splash freshness without blocking forever (S12 soft bridge)

4.3 Soft production bridges

- **S12:** server-driven splash as cold-start *product* surface — measure TTI mindset, don't invent ms.

- **S6:** LE Bottom Sheet cut full-screen navigations for **30%+** flows — UX performance, not only CPU.

5. Hitches vs hangs

	Hitch	Hang
User feel	Scroll / animation jank	App “frozen”
Typical bar	Missed ~16ms frame	Main blocked hundreds of ms+
Lab tools	Hitches, Time Profiler	Hang detection, Time Profiler
Field	MetricKit hitch/hang diagnostics	Hang rate, watchdog adjacency

5.1 Hang detector concept (APM vocabulary)

```
Background ping thread → signal main → wait with timeout (~250ms class)
→ if main doesn't respond, capture stacks (carefully)
→ persist / upload later (not heavy work on critical path)
```

NFR mindset: Overhead must be budgeted; remote kill-switch for APM features; batch uploads; never sync network on the happy path.

Difference vs watchdog kill: Your detector observes; the OS may still kill if unresponsive long enough.

6. Scrolling attribution playbook

```
Scroll jank reported
→ Reproduce with Hitches instrument
→ Time Profiler: main-thread decode? layout? lock?
→ Check cell configure cost / Auto Layout
→ Check image pipeline (downsample, decode off main – Day 16)
→ Check search path debounce/cancel (S3) if search list
→ Verify with hitch rate + field device-class segment
```

Trap: “FPS average is 58 — ship it.” Check hitch rate / p90 frame time / older devices.

7. Journey traces (Firebase Performance pattern)

7.1 What to trace

Journey	Example span
Listing	Appear → first meaningful content
Search	Query committed → results rendered
Checkout	Tap pay → success/fail terminal state

7.2 Placement judgment (S5-A1)

Placement	When	Cost
Journey-level traces	Product SLIs, PM conversations	Need clear start/stop events
Per-request interceptor spans	Debugging API chatter	Cardinality / noise if unbounded

7.3 Cardinality hygiene

- Prefer URL **templates** (`/user/:id`) over raw URLs in metrics
- Avoid PII in trace names/attributes
- Sample if needed; watch SDK overhead

8. APM SDK shape (whiteboard in 10 minutes)

Useful when interview pivots to “design an APM”:

```
App
├─ ColdStartTracker (process start → first frame)
├─ HangDetector (main ping)
├─ CustomTraces (journey API)
├─ MetricKitSubscriber
└─ LocalStore (SQLite WAL) → batch GZIP upload (bg / threshold)
```

Ops: retry/backoff, kill-switch, alert on cold-start p90 / hang rate — cite categories, don’t invent BMS-internal alert thresholds as “Verified.”

9. Trade-offs table

Choice	When	Cost
Instruments deep dive	Reproducible lab issue	Not fleet-representative alone
MetricKit first	Field-only / device-class	Delayed, coarse
Firebase Performance	Product journey SLIs	Vendor overhead; sampling
Signposts everywhere	Hot investigation	Noise if undisciplined
Optimize average	Never for UX SLOs	Hides p90 fires
Defer SDK init	Launch budget	Late feature readiness — order carefully
Micro-optimize JSON	After profiler proof	Premature otherwise
Leaks-only memory pass	Never for cycle hunts	False confidence

10. Failure modes & gotchas

Gotcha	Fix in speech
Average worship	Demand p90 / hitch rate
Profiler on wait-bound issue	Switch to network/backend
Leaks clean ⇒ no cycles	Graph + Allocations
Launch blame on UI only	Check pre-main / dyld
One device “proof”	Segment field data
Trace every URL raw	Templates + scrub
Ignore older devices	MetricKit / device-class p90

11. Cross-day links (light)

- Day 16 — image decode on main as hitch cause
 - Day 18 — hang/OOM vs crash; crash SDK init order vs launch budget
 - Day 20 — perf/CFS gates on release trains
 - Day 03 — ARC/cycles deep; today only Instruments correctness
-

12. Optional citations (end only)

- Apple MetricKit / Understanding hitches / App launch documentation
- Repo mirror: `ios-system-design/docs/app-performance-monitoring.md`
- Story: [story-bank.md](#) S5, S12, S6, S3

This chapter remains self-contained without those opens.

03 — Production Bridge: Firebase Performance (S5)

Convert theory into interview stories without overclaiming.

1. Provenance map for today

ID	Label	Exact claim you may make
S5	Verified	Firebase Performance traces for listing / checkout / search ; tracked p50/p90
S5-A1	How I would apply it	Journey-level traces vs per-request interceptor spans
S12	Verified	Server-driven splash / cold-start product surface (no invented ms)
S6	Verified	LE Bottom Sheet — 30%+ flows fewer full-screen navigations
S3	Verified	Search debounce / cancel (laggy search checklist)
Learning-lab	Illustrative	JourneyTrace.swift, Instruments tool map

Forbidden overclaims

- Invented cold-start milliseconds or hitch SLA numbers as “what we hit”
- “Leaks instrument showed our retain cycle”
- “I built an in-house APM SDK at BMS” (S5 is Firebase Performance traces)
- Claiming MetricKit subscriber code as shipped personal work without evidence

2. Verified S5 — STAR (2–3 min)

Timed opener (~10s)

“I’ll walk through building journey-level observability with Firebase Performance at BookMyShow — listing, checkout, and search with p50/p90.”

Situation / Task (~20s)

We needed data-driven latency visibility across business-critical journeys. Anecdotes from desk devices weren’t enough at consumer scale.

Action (~90s)

1. Instrumented **Firebase Performance** traces around key journeys: **listing, checkout, search**.
2. Reported **p50 and p90**, not averages — so tails on weak devices and peak traffic stayed visible.

3. Used traces to prioritise optimisation work with PM and backend — conversations shifted from “feels slow” to percentile movement.
4. Treated traces as an **observability layer across releases** — watch regressions, not one-off screenshots.

Result (~20–30s)

An observability layer for journey latency; release and optimisation discussions grounded in percentiles.

Lesson (~15–20s)

p90 matters more than average for user-perceived pain under load. Lab Instruments attribute; field percentiles decide priority.

Provenance tag

Provenance: Verified · S5 · BookMyShow · Firebase Performance p50/p90 on listing/checkout/search

3. Interview line (≤20s)

“I instrumented Firebase Performance traces for listing, checkout, and search — we tracked p50/p90 so release decisions followed real tails, not averages.”

4. Adjacent hooks (keep honest)

Cold start product — S12

“Splash is a product surface — server-driven content can improve freshness and cold-start experience; I talk TTI mindset, not a fake millisecond trophy.”

Provenance: Verified · S12 · Aces server-driven splash

UX performance — S6

“Performance isn’t only CPU — reducing full-screen navigations for 30%+ flows with the LE Bottom Sheet cut stack cost for high-traffic journeys.”

Provenance: Verified · S6 · LE Bottom Sheet 30%+ flows

Search lag checklist — S3

Debounce, cancel in-flight, don’t apply stale results; pair with search journey traces.

Provenance: Verified · S3 · search debounce/cancel

Journey vs interceptor — S5-A1

When asked where to put spans:

“I’d keep product SLIs as journey traces, and use finer interceptor spans for debugging chatter — carefully, with cardinality hygiene.”

Provenance: How I would apply it · S5-A1

5. Instruments correctness (say in perf interviews)

If memory comes up during a performance discussion:

“For abandoned view controllers / retain cycles I use Memory Graph and Allocations. Leaks is for unreachable memory — cycles usually won’t show there.”

Do **not** claim you ran that triage as Verified BMS personal workflow unless you add evidence later — it’s **technical correctness**, labeled Applied/Learning when personal process is implied.

6. Whiteboard — where you'd place 3 traces

Listing: `viewDidAppear` / `onAppear` → first content bind
Search: `debounced query fire` → `results render` (ignore cancel)
Checkout: CTA tap → terminal success/fail (not every polling tick as separate journey)

7. Anti-patterns to refuse

Temptation	Refuse with
"Average improved 12%"	Show p90 / hitch rate
"FPS is fine"	Field hitch + device class
"Optimize everything in Time Profiler"	Wait-bound ≠ CPU-bound
"One number performance score"	Small SLI set: cold start p90, journey p90, hitch, CFS

8. Bridge to Days 18–20

- Day 18: hangs/OOM may not move CFS — don't trust crash-free alone for "freezes"
- Day 19: security work must not silently add launch/main cost without measurement
- Day 20: release trains pause on perf/CFS gates — S5 culture feeds ops

04 — Questions (two-layer Q&A)

How to practice: Cover **Full spoken answer**. Speak from **Answer points** only. Uncover and compare.

Timing: Normal ≈ 30–45s (some 45–60s) · Tricky ≈ 90–120s.

Totals: **12 normal + 8 tricky = 20.**

Normal questions

Q1. How do you approach an "app feels slow" bug? (45–60s)

Answer points:

- Clarify journey + reproduce lab vs field-only
- Pull percentiles if field; Instruments if lab
- Attribute bottleneck class before fix
- Re-verify p50/p90
- Hook S5 culture

Agenda opener: "Clarify journey → lab vs field → attribute → fix → verify percentiles..."

Full spoken answer:

"First I clarify which journey feels slow and whether it's reproducible on a desk device or only in the field. If it's field-only, I look at Firebase Performance or MetricKit segments before guessing. If I can reproduce, I pick the right Instruments tool — Time Profiler for CPU, Hitches for scroll, App Launch for startup. I attribute the bottleneck class — compute versus wait versus layout versus memory — fix the smallest high-leverage cause, then re-check p50/p90. At BookMyShow we built that percentile culture with Firebase traces on listing, checkout, and search."

Common wrong answer: "Open Time Profiler and optimise the first hot function I see."

Follow-up ladder:

Level	Ask	Point
L1	Only 5% of users?	Segment device/OS/network; severity × rarity.
L2	Clean CPU profile?	Likely wait-bound — network/backend/locks.
L3	What do you show PM?	Journey p50/p90 trend, not FPS vanity.

Provenance: Verified · S5 · Firebase Performance culture

Q2. Why p90 over average? (30–45s)

Answer points:

- Averages hide tails
- Weak devices / peak traffic live in p90+
- BMS journeys used percentiles
- p50 still useful for “typical”

Agenda opener: “Tail latency is where real pain lives...”

Full spoken answer:

“Averages hide the tail. The users on older devices, congested networks, or peak sale traffic live in p90 and beyond. We instrumented Firebase Performance for listing, checkout, and search and tracked p50/p90 so optimisation followed real pain, not a flattering mean. p50 still tells me the typical path; p90 tells me who is suffering.”

Common wrong answer: “p90 is only for backend SLOs; mobile just uses FPS.”

Follow-up ladder:

Level	Ask	Point
L1	When p99?	Rare critical paths; noisier on mobile.
L2	Average FPS ship gate?	No — hitch rate / field percentiles.
L3	How communicate to PM?	“10% of sessions slower than X.”

Provenance: Verified · S5

Q3. Time Profiler — what do you look for? (30–45s)

Answer points:

- Heaviest self-time / stacks
- Unexpected main-thread work
- Confirm after fix
- Sampling bias awareness

Agenda opener: “I’m hunting main-thread self-time that surprises me...”

Full spoken answer:

“I look for the heaviest stacks and unexpected self-time on the main thread — JSON decode, image work, regex, heavy cell configuration. I treat it as a sampling profiler, so I confirm patterns across a longer capture. After a fix I re-run to see the stack shrink. If the profile is clean but the user still waits, I stop chasing CPU and look at network or lock wait.”

Common wrong answer: “Time Profiler shows network latency hotspots.”

Follow-up ladder:

Level	Ask	Point
-------	-----	-------

L1	Sampling bias?	Short captures miss rare spikes — lengthen / repeat.
L2	Signposts help how?	Align samples to journey intervals.
L3	Release vs Debug?	Optimisations change profiles — test Release-like.

Provenance: Learning-lab · Instruments

Q4. What is a hitch? (30–45s)

Answer points:

- Missed frame vs vsync / deadline
- ~16ms @ 60Hz budget
- Decode/layout on main common
- ≠ hang (longer freeze)

Agenda opener: “A hitch is a missed frame deadline...”

Full spoken answer:

“A hitch means a frame missed its display deadline — at 60Hz you have roughly sixteen milliseconds of main-thread budget. Scroll jank is the user-facing symptom. Classic causes are main-thread image decode, Auto Layout thrash, or synchronous I/O in cell configure. That’s different from a hang, which is a longer main-thread stall that feels like a freeze.”

Common wrong answer: “Low CPU usage means no hitches.”

Follow-up ladder:

Level	Ask	Point
L1	120Hz budget?	~8.3ms — same bugs, less margin.
L2	First instrument?	Hitches / Animations, then Time Profiler.
L3	Field signal?	MetricKit hitch diagnostics + device class.

Provenance: Learning-lab

Q5. MetricKit vs Firebase Performance? (30–45s)

Answer points:

- MetricKit = OS aggregates / hangs / histograms
- Firebase = custom journey/network traces you define
- Use both
- MetricKit delayed; Firebase for product SLIs

Agenda opener: “OS aggregates versus product journey SLIs...”

Full spoken answer:

“MetricKit gives privacy-preserving OS aggregates — CPU, memory, hang and hitch related diagnostics — typically on a delayed cadence. Firebase Performance is where I define custom journey traces and network signals for product SLIs. At BookMyShow we used Firebase traces with p50/p90 on listing, checkout, and search. I use MetricKit for fleet OS truth and Firebase for journey conversations — they’re complementary, not substitutes.”

Common wrong answer: “MetricKit replaces the need for custom traces.”

Follow-up ladder:

Level	Ask	Point
-------	-----	-------

L1	Privacy?	Prefer aggregates; scrub custom attributes.
L2	Realtime pager?	MetricKit isn't your only realtime alert path.
L3	S5-A1?	Journey traces vs noisy per-request spans.

Provenance: Verified · S5 · + MetricKit teaching

Q6. Cold start — name the phases. (30–45s)

Answer points:

- Pre-main → app/scene init → first frame → TTI
- Optimise dyld + defer work
- Don't invent fake ms
- Soft S12 splash product

Agenda opener: "Pre-main, init, first frame, then interactive..."

Full spoken answer:

"I break cold start into pre-main — dyld and static work — then app and scene initialisation, then first frame, then time-to-interactive when the UI is actually data-ready. Optimisations target fewer launch-path frameworks, deferred non-critical SDK init, and no sync disk or network on main. I measure from process start, not only didFinishLaunching. On Aces we treated splash as a server-driven product surface for freshness and cold-start experience — I talk categories and TTI mindset, not a trophy millisecond."

Common wrong answer: "Cold start is just time until didFinishLaunching returns."

Follow-up ladder:

Level	Ask	Point
L1	Dynamic frameworks?	dyld cost on pre-main / launch.
L2	Crash SDK order?	Early but fast — Day 18.
L3	TTFF vs TTI?	First pixels ≠ usable.

Provenance: Verified · S12 · soft bridge; Learning-lab phases

Q7. How do you measure checkout latency in production? (30–45s)

Answer points:

- Custom journey span appear/CTA → terminal state
- Segment network/device
- p50/p90 dashboards
- Don't average-only

Agenda opener: "I'd define a journey trace with clear start and stop..."

Full spoken answer:

"I'd define a Firebase Performance journey from a clear start — screen ready or CTA tap — to a terminal success or failure. I'd avoid turning every poll tick into its own journey. Then I watch p50/p90, segmented by network or device class when needed. That's the pattern we used across listing, checkout, and search at BookMyShow — percentiles driving prioritisation."

Common wrong answer: "Log print timestamps in debug builds only."

Follow-up ladder:

Level	Ask	Point
L1	Payment UX during delay?	S7 popup — communicate state.
L2	Cardinality?	Template names; scrub PII.
L3	Interceptor spans?	S5-A1 — secondary for chatter.

Provenance: Verified · S5 · (+ S7 if UX asked)

Q8. Main-thread hang detection — concept? (30–45s)

Answer points:

- Ping main from another thread
- Threshold capture stacks
- Overhead + kill-switch
- ≠ watchdog kill itself

Agenda opener: “Watchdog-style ping of the main thread...”

Full spoken answer:

“A hang detector typically pings the main thread from a background helper and waits with a timeout — on the order of a few hundred milliseconds in many APM designs. If main doesn’t respond, you capture diagnostic stacks carefully and persist for later upload. You must budget overhead and ideally support a remote kill-switch. That’s observing unresponsiveness; the OS watchdog kill is a separate, harsher outcome.”

Common wrong answer: “Hang detection is just Crashlytics.”

Follow-up ladder:

Level	Ask	Point
L1	Upload when?	Later / batched — not heavy in-signal style work.
L2	CFS relationship?	Hangs may not count as crashes — Day 18.
L3	Threshold?	Tool-defined; know your vendor’s definition.

Provenance: Learning-lab · APM vocabulary

Q9. Binary size vs performance? (30–45s)

Answer points:

- Download time + dyld + page-ins
- CI budget gates
- Thinning / modularization
- Not only “disk MB vanity”

Agenda opener: “Size hits download, dyld, and page-in cost...”

Full spoken answer:

“Binary size isn’t just a store listing number — it affects download time, dyld work, and page-ins that show up in launch and memory behaviour. I’d put a size budget in CI, use app thinning realities, and keep modularization from dragging unnecessary code onto the launch path. It’s a performance input, not a separate vanity metric.”

Common wrong answer: “Size only matters for users on cellular.”

Follow-up ladder:

Level	Ask	Point
L1	Day 15 link?	Dynamic frameworks / modular boundaries.
L2	Asset catalogs?	On-demand / thinning strategies.
L3	Gate in CI?	Fail PR over budget.

Provenance: Learning-lab

Q10. Search feels laggy — checklist? (30–45s)

Answer points:

- Debounce + cancel in-flight
- Main-thread parse?
- Layout thrash / images
- Search journey traces

Agenda opener: “Cancel and debounce first, then profile...”

Full spoken answer:

“I’d check debounce and cancellation so keystrokes don’t stampede the network or apply stale results — that’s the search modernisation pattern we used. Then I’d ask whether JSON parse or cell configure is on main, whether images decode on main, and whether layout is thrashing. Finally I’d look at the search journey p50/p90 so fixes are verified in the field, not just by feel.”

Common wrong answer: “Add a bigger loading spinner.”

Follow-up ladder:

Level	Ask	Point
L1	Callback tasks?	Explicit cancel + generation guard.
L2	Instruments?	Hitches + Time Profiler.
L3	Story?	S3 + S5.

Provenance: Verified · S3 · S5

Q11. When is Instruments the wrong first tool? (30–45s)

Answer points:

- Rare field-only regressions
- Need segment from MetricKit/Firebase first
- Then lab to attribute
- Avoid premature micro-opts

Agenda opener: “When the desk can’t see the pain...”

Full spoken answer:

“If only a slice of the fleet feels it — a device class, an OS version, a region — Instruments on my phone may waste an hour. I start with field percentiles and MetricKit-style aggregates to locate the segment, then reproduce and attribute in lab. Instruments is superb at attribution, not always at discovery.”

Common wrong answer: “Always Instruments first — field tools are marketing.”

Follow-up ladder:

Level	Ask	Point
-------	-----	-------

L1	Premature optimisation?	No SLI movement → don't micro-opt.
L2	Both needed?	Discovery field, attribution lab.
L3	Scale context?	30L+ DAU → distributions matter.

Provenance: Verified · S5 · field culture

Q12. LE Bottom Sheet as performance work? (30–45s)

Answer points:

- Fewer full-screen pushes
- 30%+ flows metric
- Product perf ≠ only CPU
- Still measure journeys

Agenda opener: “Not all performance is Instruments...”

Full spoken answer:

“Performance is also product architecture. The LE Bottom Sheet reduced full-screen navigations for thirty percent plus of flows — less navigation stack cost and faster access to event overview. That’s UX performance at scale, complementary to CPU profiling. I’d still keep journey traces so we don’t confuse a UX win with unrelated latency regressions.”

Common wrong answer: “If it isn’t Time Profiler, it isn’t performance.”

Follow-up ladder:

Level	Ask	Point
L1	How measure?	Navigation reduction metric + journey p90.
L2	Cross-functional?	PM/Design/Backend contracts — S6.
L3	Tie S5?	Same percentile culture validates.

Provenance: Verified · S6 · (+ S5 culture)

Tricky questions

T1. “We improved average FPS — ship it?” (90–120s)

Answer points:

- Average vanity
- Hitch rate / p90 frame / MetricKit
- Device-class tails
- Refuse ship on average alone

Trap: Average worship.

Full spoken answer:

“I wouldn’t ship on average FPS alone. Averages can look healthy while hitch rate and the slowest devices tank. I’d ask for hitch metrics, p90 frame time if we have it, and field signals segmented by device class. At scale, the tail is the product experience for a lot of people. Show me percentile movement on the journeys that matter — listing, checkout, search style SLIs — then we talk ship.”

Follow-up: What dashboard do you show PM? → Small SLI set, not one score.

Provenance: Verified · S5 · judgment

T2. Trace shows network p90 high but Time Profiler clean. (90–120s)

Answer points:

- Waiting ≠ computing
- Payloads, fan-out, cache, backend
- Firebase network/journey traces help
- Don't micro-opt UI

Trap: Optimise client CPU anyway.

Full spoken answer:

"That's a classic wait-bound case. A clean Time Profiler means I'm probably not CPU-bound on device — I'm waiting on network, backend queues, or lock/contention elsewhere. I'd inspect payload size, request fan-out, caching, and backend latency with the same journey traces. Client micro-optimisations won't move that p90. Firebase Performance helps separate network time inside the journey so we don't blame the wrong layer."

Follow-up: How coordinate with backend? → Shared trace IDs / percentile ownership.

Provenance: Verified · S5

T3. Launch regression after adding analytics SDKs. (90–120s)

Answer points:

- App Launch pre vs post main
- Defer non-critical init
- Keep crash SDK early + fast
- Feature-flag costly SDKs

Trap: Delete all analytics immediately as only answer.

Full spoken answer:

"I'd measure with the App Launch template to see whether cost landed pre-main or post-main. Then I'd defer non-critical SDK initialisation off the first-frame path, keep crash reporting early because launch crashes matter, and make sure anything heavy is flag-gated. The answer isn't 'never analytics' — it's ordered, measured init with a launch budget. If needed we stage rollout and watch cold-start field percentiles."

Follow-up: Ordering vs Day 18? → Crash SDK early; heavy SDKs later.

Provenance: Learning-lab · launch discipline

T4. Only Android-like "ANR" thinking on iOS. (90–120s)

Answer points:

- Use hang / watchdog / main-block language
- MetricKit hang diagnostics
- Same idea: don't block main
- Know tool thresholds

Trap: Copy Android terms blindly / dismiss the concept.

Full spoken answer:

"I'd translate to iOS vocabulary — hangs, watchdog kills, main-thread stalls — rather than forcing ANR as a term. The engineering idea is the same: don't block main. MetricKit hang diagnostics and an APM hang detector are how we observe it. I also separate hitches — frame misses — from longer hangs. Thresholds are tool-defined; I learn the vendor's definition instead of pretending there's one universal number."

Follow-up: 250ms vs multi-second? → Detector vs OS kill vs UX freeze.

T5. p50 improved, p90 worsened after caching. (90–120s)

Answer points:

- Don't celebrate p50 alone
- Stampede / locks / miss penalty
- Main-thread disk on miss
- Inspect miss path distribution

Trap: Ship because median looks great.

Full spoken answer:

"That's a red flag that the cache helps the happy path and punishes the miss path or creates contention. I'd look for stampede on expiry, lock contention around the cache, or main-thread disk hits on miss — especially if decode landed on main. p50 can improve while p90 burns. I'd profile the miss path and watch field p90 until both move the right way — same discipline we used for journey percentiles."

Follow-up: Day 16 decode on miss? → Off-main decode + downsample.

Provenance: Verified · S5 · judgment

T6. How much overhead may an APM SDK add? (90–120s)

Answer points:

- Budget it; literature ~1% class targets
- Batch uploads; sample
- Kill-switch
- Never sync I/O on crash-free path casually

Trap: "Negligible always."

Full spoken answer:

"I treat APM overhead as a first-class NFR, not vibes. You budget CPU and wakeups, batch and compress uploads, sample where needed, and keep a remote kill-switch. You don't do synchronous network on the hot path, and you especially don't make the crash-free experience worse with a chatty telemetry layer. If overhead shows in launch or hitch metrics, the observability tool became the problem."

Follow-up: Design batch uploader? → SQLite WAL buffer → GZIP → bg upload.

Provenance: Learning-lab · APM NFR

T7. Scroll jank only on older devices — ship to 100%? (90–120s)

Answer points:

- Segment MetricKit / field by device class
- Protect tails
- Gated rollout
- More aggressive downsample on memory class

Trap: "Most users are fine."

Full spoken answer:

"I wouldn't shrug off older-device tails at consumer scale. I'd segment field hitch and journey metrics by device class, protect those tails with gated rollout, and tune the image/layout path more aggressively on lower memory classes. Shipping to 100% because flagship devices are smooth is how you donate one-star reviews. Percentile culture means the slow cohort still counts."

Follow-up: Phased release + gates? → Day 20 / Day 18 pause criteria.

Provenance: Verified · S5 · scale judgment

T8. Stakeholder wants a single “performance score.” (90–120s)

Answer points:

- Offer small SLI set
- Cold start p90, journey p90, hitch, CFS
- Composite hides actionability
- BMS percentile conversations

Trap: One number to rule them all.

Full spoken answer:

“I’d push back gently and offer a small SLI set instead — cold-start p90, critical journey p90 like checkout, hitch rate, and crash-free as a reliability sibling. A single composite score hides which lever to pull and invites gaming. At BookMyShow we moved conversations to journey percentiles for listing, checkout, and search — that’s actionable for PM and engineering. Scores are fine as executive summaries only if the underlying SLIs stay visible.”

Follow-up: How you ran BMS conversations? → S5 STAR.

Provenance: Verified · S5

Timed set recommendation

Record: **Q1, Q2, Q6 + T2, T5**. Then deliver **S5** in ≤3 min with explicit p50/p90 language.

05 — Exercises

Do these after foundations + deep dive + production bridge. Speak out loud even for whiteboard prompts.

Exercise 1 — Perf loop teach-back (10 min)

Prompt: Without notes, recite:

1. Five-step performance loop
2. Lab vs field (one sentence each)
3. Why p90 over average
4. Retain cycles → which Instruments? (explicitly **not** Leaks)

Done when: You correct yourself if you say “Leaks for cycles.”

Exercise 2 — Instruments routing (15 min)

For each symptom, name the **first** tool and why:

Symptom	Your tool	Why
Scroll jank on listing		
Cold start +200ms after SDK		
Memory climbs every push/pop		
Checkout slow; CPU idle		

Only iPhone 11 users complain		
-------------------------------	--	--

Answers (check after): Hitches/Time Profiler · App Launch · Allocations+Memory Graph · Network/Firebase journey · Field MetricKit/Firebase first.

Exercise 3 — Place three traces (15–20 min)

Using [code/JourneyTrace.swift](#) as a sketch:

- 1. Define start/stop events for listing, search, checkout
- 2. Speak cardinality hygiene (templates, no PII)
- 3. Mark one span as **S5-A1** journey-level vs a noisy per-request alternative

Speak (≤90s): Verified S5 → what you shipped → how you’d place spans.

Exercise 4 — APM whiteboard (15 min)

Outline boxes only:

ColdStartTracker HangDetector CustomTraces MetricKitSubscriber SQLite batch upload
--

Say aloud: overhead budget, batch upload, kill-switch, why not upload in a signal handler (preview Day 18).

Exercise 5 — S5 STAR record (10–12 min)

Record the full STAR from [03-production-bridge.md](#) .
Score: opener / actions with p50/p90 / lesson / no invented metrics.

Exercise 6 — Correct the interviewer (5 min)

Interviewer: “If Leaks is clean, you don’t have retain cycles.”

Your correction (≤45s): reachable abandoned memory → Graph/Allocations; Leaks = unreachable.

Timed drill

- 1. Pick 3 Normal + 2 Tricky from [04-questions.md](#) (recommend Q1, Q2, Q6 + T2, T5).
 - 2. Score against [answer-timing-guide.md](#) .
 - 3. Log misses: average worship, Instruments-for-network-waits, Leaks-for-cycles.
 - 4. Skim revision twin flashcards.
-

Flashcards (also on revision twin)

Front	Back
Perf loop	Symptom → attribute → fix → p50/p90 · Trap: vibe opt · Prod: S5
Why p90	Tail = pain · Trap: averages · Prod: BMS journeys
Cycles tool	Graph/Allocations · Trap: Leaks · Prod: correctness
MetricKit	OS aggregates · Trap: replaces traces · Prod: fleet
Firebase Perf	Custom journeys · Trap: only FPS · Prod: S5
Hitch	Missed frame · Trap: low CPU=smooth · Prod: decode on main

Cold start	Pre-main→init→frame→TTI · Trap: only TTFF · Prod: S12
Lab vs field	Instruments vs MetricKit/Firebase · Trap: one device=fleet · Prod: 30L DAU

Day 18 — Crash Reporting, Observability & Incident Response (IMOC)

Week 3 · Full study (self-contained) · ~4–5 hrs

Revision twin: [revision/weeks/week-03/day-18.md](#)

Outcomes

By end of day, without notes, you can:

- Explain crash SDK capture → **async-signal-safe** persist → next-launch upload → **dSYM** symbolication
- Distinguish **crash vs OOM vs hang**, and why CFS can look fine while users report freezes
- Deliver **Verified · S8: 30L+ DAU, 99.95%+ CFS**, Crashlytics triage, **IMOC** on P0/P1
- Use **Verified · S2** as a **path-scoped** concurrency fix that *contributed* to reliability — **never** “S2 alone caused 99.95% CFS”
- Run an incident script: owner, blast radius, mitigate first, communicate, postmortem

How to study

1. [01-foundations.md](#)
2. [02-deep-dive.md](#)
3. [03-production-bridge.md](#) — S8 STAR + S2 honesty bound
4. [code/](#) — breadcrumb ring + signal-safety notes
5. [04-questions.md](#) — Answer points → Full spoken
6. [05-exercises.md](#)
7. Revision twin

Module map

Module	File
Foundations	01-foundations.md
Deep dive	02-deep-dive.md
Production bridge	03-production-bridge.md
Questions	04-questions.md
Exercises	05-exercises.md
Code	code/BreadcrumbRing.swift , code/CrashReportNotes.swift

Critical correctness

Claim	Truth
S8	Sustained 99.95%+ CFS at 30L+ DAU via Crashlytics workflows + IMOC — multi-factor ops excellence
S2	Synchronised dictionaries eliminated path-scoped race crashes — contributes , does not solely own CFS
Signal handler	Async-signal-safe only — no malloc / ObjC / locks
Upload timing	Persist on crash; upload next launch

Hang vs CFS	Freezes / hangs may not move crash-free
-------------	--

Provenance reminder

Label	Use
Verified · S8	30L+ DAU, 99.95%+ CFS, Crashlytics, IMOC
Verified · S2	GCD serial-queue / RW sync dictionaries — race crashes on that path
How I would apply it · S2-A1	Greenfield shared maps as Swift actor
Learning-lab	Code sketches

Agenda opener

"I'll cover capture → persist → upload → symbolicate, then OOM/hang gaps, then incident command using our 99.95% CFS / IMOC practice — with concurrency fixes as one contributor, not the whole CFS story."

Time budget

Block	Minutes
Foundations + deep dive	90–120
Production bridge + code	40–50
Questions	45–60
Exercises + revision	30–45

01 — Foundations: Crash Reporting & Incidents

*Mental model first. Hard rule burned in early: **S2 ≠ sole cause of CFS**.*

0. North star

Catch fatals safely, make stacks readable, triage with process, lead incidents with mitigate-first ownership — and never attribute org-wide CFS to a single dictionary fix.

1. Crash SDK pipeline

```
Init (<~10ms class) → register handlers → breadcrumb ring
Crash (signal / uncaught / fatalError path)
  → async-signal-safe write to preallocated mmap/disk
  → terminate
Next launch → discover report → upload → server symbolicates with dSYM
```

Keyword	Why interviewers listen
sigaction / signal handlers	POSIX fatals (SEGV, ABRT, ...)
Uncaught Obj-C / Swift fatal paths	Not covered by try

Async-signal-safe	No alloc, ObjC, Swift runtime, locks in handler
Lock-free breadcrumb buffer	Safe-ish context for last N events
dSYM UUID match	Readable stacks in Crashlytics

2. What crash-free sessions (CFS) means

- **CFS:** sessions without a **fatal crash** (vendor definitions vary slightly — say Crashlytics sessions)
- **Verified · S8:** sustained **99.95%+** at **30+ lakh DAU** — operational excellence under peak load
- Non-fatals ≠ crashes — still triage if user-impacting
- **OOM / jetsam:** often no clean client stack; heuristics + MetricKit exit reasons (Day 17)
- **Hangs:** may not count against CFS — users still churn

Honesty bound (memorize)

S8 = CFS + Crashlytics triage + IMOC (system of reliability)
S2 = synchronised dictionaries fixed races on a shared-state PATH
S2 ⊂ reliability work, but S2 ≠ "I alone made 99.95% CFS"

Say aloud: "Synchronised dictionaries removed intermittent race crashes on that shared state path. Crash-free at 99.95% was sustained by triage workflows, incident command, and many fixes — not one API."

Provenance: *Verified · S8 · CFS/IMOC; Verified · S2 · path-scoped races — do not merge into one causal claim*

3. Triage workflow

1. **Detect** — spike / CFS drop alert
2. **Classify** — new vs regressed; top stacks; version %; journey tags
3. **Reproduce** — symbolicated stack + breadcrumbs + device/OS matrix
4. **Mitigate** — flag off, pause phased rollout, hotfix path
5. **Fix** — root cause + regression test
6. **Write-up** — blameless postmortem for P0/P1

4. IMOC (leadership proof)

As **IMOC**, coordinate **iOS + backend + QA** on **P0/P1** during high-traffic events.

Pillar	Meaning
Single owner	One incident commander / clear channel
Blast radius	Feature, %, geo, payment path?
Mitigate first	User harm down before perfect RCA
Cadenced comms	Ops/customer updates on a clock
Handoff + postmortem	Actions: alerts, tests, runbooks

5. Glossary

Term	Meaning
Async-signal-safe	APIs safe inside signal handlers

Breadcrumb	Last-N event trail for context
dSYM	Debug symbols for symbolication
Symbolication	Map addresses → functions/lines
Non-fatal	Caught/logged error, not session crash
Jetsam / OOM	Memory pressure kill
IMOC	Incident Management On-Call / commander role
Blast radius	Scope of user impact

6. Crash vs OOM vs hang

	Crash	OOM	Hang
Typical signal	Fatal signal/exception	SIGKILL / jetsam	Main blocked
Clean stack?	Often yes (if handlers work)	Often heuristic	Hang detector / MetricKit
Moves CFS?	Yes (fatal)	Vendor-dependent / often separate	Often no

Trap: “CFS is fine so the app isn’t freezing.”

7. Launch ordering preview

- Prefer **crash SDK early** so launch crashes report
- Keep init **fast** (budget milliseconds-class)
- Defer heavy analytics (Day 17 launch budget)

8. Teach-back checklist

1. Pipeline: capture → persist → upload → symbolicate
2. Async-signal-safe one-liner
3. CFS definition + 99.95% / 30L context (S8)
4. **S2 contributes; does not solely cause CFS**
5. IMOC pillars
6. Hang gap vs CFS

9. One-minute CFS honesty drill

Speak continuously:

“Crash-free at ninety-nine point nine five percent on thirty-plus lakh DAU was sustained through Crashlytics triage workflows and IMOC coordination on P0/P1s during peak traffic. Separately, synchronised dictionaries removed intermittent race crashes on a shared async state path — that contributed to reliability. I don’t collapse those into one causal story.”

Stop. If you said “because of dictionaries,” restart.

Next: [02-deep-dive.md](#) .

02 — Deep Dive: Crash SDK, OOM, IMOC

1. Signal handling (what “safe” means)

1.1 Handler responsibilities (conceptual)

On fatal signal:

1. Suspend other threads (implementation-dependent; be honest it's delicate)
2. Capture backtrace / registers into PREALLOCATED buffer
3. write() to mmap / file descriptor – no malloc
4. Reset handler / abort to terminate

1.2 Forbidden in-handler

Forbidden	Why
malloc / new objects	Allocator locks / reentrancy
Obj-C / many Swift runtime calls	May allocate or lock
Normal logging (OSLog, print frameworks)	May deadlock
Taking locks	Deadlock if crash held the lock
Starting network upload	Not signal-safe; wrong time

1.3 Allowed mindset

- Preallocate crash region at init
- Async-signal-safe syscalls only (write , etc.)
- Breadcrumbs already in a lock-free ring filled on the happy path

Interview line:

“The crash path is about surviving long enough to persist a report — not about doing a full-featured dump with normal APIs.”

2. Breadcrumbs

Design	Detail
Structure	Ring buffer, last N events (~50–100 class)
Content	Navigations, key actions, HTTP status codes
Scrub	Tokens, auth headers, PII — at source
Concurrency	Lock-free / careful — crash may interrupt

Privacy incident: breadcrumbs with auth headers → treat as Sev; rotate; lint forbidden keys.

3. Symbolication & dSYM

Build UUID in report

- server finds matching dSYM
- load address + frame offset → function/file/line

Failure	Symptom	Fix
Missing dSYM upload in CI	Unreadable stacks	Fix pipeline; re-upload
Mismatched build	Wrong/partial symbols	UUID discipline

Bitcode legacy confusion	Old advice	Modern: always upload dSYMs you ship
--------------------------	------------	--------------------------------------

CI link (Day 20): every TestFlight/App Store build uploads symbols.

4. OOM detection (honest limits)

- SIGKILL from jetsam is **not** a normal catchable crash path
- Next-launch heuristics: high last footprint + no clean exit + no crash file
- MetricKit OOM / exit diagnostics arrive on delay
- Memory Graph / Allocations (Day 17) for lab reproduction — **cycles ≠ Leaks**

5. Upload reliability

Do	Don't
Persist on crash; upload next cold start	Network inside signal handler
Backoff + quota awareness	Delete last report naively before ack
Cap pending reports	Unlimited disk fill

6. Non-fatals & alert hygiene

- Group, sample, fix top offenders
- Don't equate non-fatal volume with CFS
- Promote to P1 when user-impacting critical path (payments)

7. IMOC deep dive — scripts you can speak

7.1 First 10 minutes

1. Confirm spike is real (not symbolication outage / bad deploy tag)
2. Declare IMOC / channel
3. Blast radius: version %, feature flag, geo, payment?
4. Mitigate: pause phased release / kill switch / disable feature
5. Comms cadence: next update in N minutes

7.2 Mitigate vs hotfix

Lever	When
Remote config / flag	Fast; preferred when path optional
Pause phased rollout	Binary already bad for %
Hotfix	Native crash on mandatory path; review latency trade-off

7.3 Cross-team blame spiral

Force shared timeline + correlation IDs + % failing by layer. Mitigate user harm first (fallback, disable). RCA second.

8. S2 concurrency — correct framing

What S2 was: shared async maps hit from multiple queues → races → intermittent crashes; fixed with **GCD serial queues** / RW locks and a safe API boundary.

What S2 was not: the single explanation for org-wide **99.95% CFS**.

OK to say	Not OK to say
"Removed race crashes on that shared state path"	"S2 is why we have 99.95% CFS"
"One reliability engineering input among triage + IMOC + others"	"Dictionaries alone held CFS"

Provenance: *Verified · S2 · path-scoped; Verified · S8 · CFS/IMOC system*

Today's Applied: greenfield → Swift actor (S2-A1) with same API surface mindset.

9. Trade-offs

Choice	When	Cost
Vendor Crashlytics	Fast symbolication pipeline	Vendor + privacy review
In-house crash SDK	Staff SD / special needs	Signal-safety correctness risk
Heavy breadcrumbs	Debuggability	CPU + PII risk
Crash SDK first	Reliable launch reports	Must stay fast
Non-fatal spam	Noise	Alert fatigue
Always hotfix	Rare true need	Process cost; prefer flags
Blame culture	Never	Kills reporting

10. Failure modes

Mode	Senior response
try/catch will catch SEGV	No — signals/fatalError need handlers + discipline
CFS fine, users freeze	Hang/OOM/MetricKit — Day 17
Unsymbolicated spike	dSYM CI break
iOS vs backend war room	IMOC timeline, mitigate first
Breadcrumbs logged tokens	Scrub + rotate + Sev

11. Optional citations

- `ios-system-design/docs/crash-reporting-sdk.md`
- Apple Understanding crashes / MetricKit diagnostics
- Stories S8, S2

Chapter is self-contained without opening them.

03 — Production Bridge: S8 IMOC/CFS + S2 Honesty

1. Provenance map

ID	Label	Exact claim
S8	Verified	30+ lakh DAU; 99.95%+ CFS; Crashlytics triage; IMOC for P0/P1

S2	Verified	Synchronised dictionaries (GCD serial / RW) — eliminated concurrent-access crashes on that path
S2-A1	Applied	Greenfield shared maps as Swift actor
Learning-lab	Illustrative	Breadcrumb ring / crash notes sketches

Forbidden overclaims

- “S2 alone caused / delivered 99.95% CFS”
- “I wrote our in-house signal handler crash SDK” (unless you add evidence — default is Crashlytics workflows)
- Invented downtime minutes or crash counts
- “CFS proves no hangs”

2. Verified S8 — STAR (2–3 min)

Opener (~10s)

“I’ll cover sustaining 99.95%+ crash-free sessions at 30L+ DAU — Crashlytics triage plus IMOC coordination on P0/P1s.”

Situation / Task (~20s)

Consumer ticketing app at very large DAU. Reliability is a product feature during peak traffic. Need structured crash response and multi-team incident command.

Action (~90s)

1. Crashlytics triage with structured workflows — detect, classify, reproduce, mitigate, fix, write-up.
2. As **IMOC**, coordinated **iOS, backend, and QA** on P0/P1 during high-traffic events.
3. Drove mitigations first: feature guards, rollout pause, hotfix path, cadenced comms.
4. Treated CFS as a sustained operational bar, not a slide metric.

Result (~20–30s)

Sustained high crash-free sessions; reduced downtime impact in peak events through clear ownership and blast-radius thinking.

Lesson (~15–20s)

Incident leadership is owner + blast radius + rollback — not lone-hero stack debugging.

Provenance: *Verified · S8 · BookMyShow · 30L+ DAU · 99.95%+ CFS · IMOC*

3. Interview line (≤20s)

“At 30L+ DAU we held 99.95%+ crash-free sessions — Crashlytics triage plus IMOC coordination on P0/P1s, not just fixing stacks alone.”

4. S2 as technical add-on (≤90s) — correct coupling

OK script

“Separately, on shared async dictionaries we had intermittent race crashes. We gated access with GCD serial queues and read-write locks behind a safe API. That removed concurrent-access crashes on that path and fed reliability. I’m not claiming that one fix is the whole 99.95% CFS story — CFS was sustained by triage, incident process, and many engineering inputs including concurrency hygiene.”

Forbidden script

“We hit 99.95% CFS because I synchronised the dictionaries.”

Provenance: *Verified · S2 · path-scoped races · must not sole-own S8 metrics*

5. S2-A1 bridge

“For greenfield shared maps I’d evaluate a Swift actor with the same boundary mindset — serialize mutation at the API edge.”

Provenance: *How I would apply it · S2-A1*

6. Behavioral variants (same facts)

Prompt type	Emphasize
Conflict	IMOC forces shared timeline vs blame
Pressure / peak sale	Mitigate first, cadence comms
Leadership	Owner clarity, handoff, postmortem actions
Technical depth	Add S2 path + signal-safety vocabulary

7. Whiteboard crash SDK (10 min)

Handlers → mmap writer → breadcrumb ring → next-launch uploader → dSYM symbolication. Call out signal safety and “no upload in handler.”

04 — Questions (two-layer Q&A)

Cover **Full spoken**. Speak from **Answer points**.

12 normal + 8 tricky. Hard rule: never let S2 sole-own CFS.

Normal questions

Q1. How does Crashlytics get a readable stack? (30–45s)

Answer points:

- Client uploads report + build UUID
- Server applies dSYM
- CI must upload symbols

Agenda opener: “UUID-matched dSYMs on the server...”

Full spoken answer:

“The client persists a crash report with addresses and the build UUID, then uploads on a later launch. The backend matches that UUID to a dSYM and symbolicates frames into functions and lines. If CI forgets to upload symbols, triage sees unreadable stacks — so dSYM upload is part of the release pipeline, not a local-only step.”

Common wrong answer: “Swift always ships readable stacks without symbols.”

Follow-up ladder:

L	Ask	Point
L1	Bitcode legacy?	Modern: always upload dSYMs you ship.
L2	App Store Connect vs Firebase?	Both can matter — know your pipeline.
L3	Re-upload?	Possible for missing symbols on a build.

Q2. What is async-signal-safe and why care? (30–45s)

Answer points:

- Handler can't malloc/lock/ObjC
- Preallocate mmap
- Secondary crash / deadlock risk

Agenda opener: "Crash handlers can't use normal APIs..."

Full spoken answer:

"Async-signal-safe means only a tiny set of operations are legal inside a signal handler. You can't malloc, take locks, or call into Objective-C or much of the Swift runtime — those can deadlock or crash again. So we preallocate a crash buffer at init and write with safe primitives. The goal is persist-the-report, not a full-featured logger in-handler."

Common wrong answer: "Just OSLog from the signal handler."

Follow-up ladder:

L	Ask	Point
L1	What can you call?	Limited async-signal-safe syscalls.
L2	Breadcrumbs?	Filled on happy path into lock-free ring.
L3	Upload in handler?	No — next launch.

Provenance: Learning-lab · crash SDK

Q3. Breadcrumbs — what do you log? (30–45s)

Answer points:

- Last N navigations / actions / status codes
- Scrub PII/tokens
- Ring buffer

Full spoken answer:

"I keep a short ring of recent navigations, key user actions, and coarse network status codes — enough to reconstruct the path to a crash. Tokens and PII get scrubbed at the source. The buffer is designed so a crash mid-write doesn't depend on taking locks casually. Breadcrumbs are context, not a second analytics product."

Common wrong answer: "Log Authorization headers for debugging."

Follow-up ladder:

L	Ask	Point
L1	Why ring?	Bound memory; last-N relevance.
L2	Privacy Sev?	Rotate credentials; treat as incident.
L3	SQLite in handler?	No — happy-path only.

Provenance: Learning-lab

Q4. Crash vs OOM vs hang? (30–45s)

Answer points:

- Fatal signal/exception vs jetsam vs main freeze
- Different detectors
- CFS may ignore hangs

Full spoken answer:

"A crash is a fatal signal or uncaught fatal path we can often report with a stack. OOM is frequently jetsam — SIGKILL territory — so we lean on heuristics and MetricKit exit reasons. A hang is the main thread stuck; users feel a freeze even when crash-free looks fine. I refuse to treat those as one bucket."

Common wrong answer: "They're all crashes in Crashlytics."

Follow-up ladder:

L	Ask	Point
L1	Day 17 hang detector?	Ping main; threshold capture.
L2	Cycles tool?	Graph/Allocations, not Leaks.
L3	CFS gap?	Freezes can hide under healthy CFS.

Provenance: Learning-lab · + S8 reliability culture

Q5. CFS drop during a sale — response? (45–60s)

Answer points:

- IMOC confirm spike
- Blast radius
- Pause rollout / flag
- Comms + postmortem

Full spoken answer:

"I'd run IMOC playbook: confirm the spike is real, declare ownership and a channel, map blast radius by version and feature, then mitigate — pause phased release or kill-switch the path — before perfect root cause. Cadence communications to stakeholders, ship a fix or hotfix as needed, and write a blameless postmortem with alert and test actions. That's how we operated P0/P1s at BookMyShow scale."

Common wrong answer: "Keep shipping while we investigate privately."

Follow-up ladder:

L	Ask	Point
L1	Who do you page?	iOS + backend + QA as needed.
L2	Flag vs hotfix?	Flag when possible; hotfix if mandatory native path.
L3	Story?	S8.

Provenance: Verified · S8 · IMOC

Q6. How do you talk about 99.95% CFS? (30–45s)

Answer points:

- Sustained under peak via process
- Not vanity slide
- Multi-factor: triage + IMOC + engineering fixes
- Don't credit S2 alone

Full spoken answer:

"I talk about it as a sustained operational bar at thirty-plus lakh DAU — Crashlytics triage workflows plus IMOC during peak events, backed by engineering fixes across the app. It's not a vanity slide from one PR. When I mention synchronised dictionaries, I frame them as removing race crashes on a specific shared-state path — a contributor to reliability, not the sole cause of ninety-nine point nine five."

Common wrong answer: "We reached 99.95% because of synchronised dictionaries."

Follow-up ladder:

L	Ask	Point
L1	What threatens CFS next?	New SDK, races, missing dSYM, bad rollout.
L2	Hangs?	Separate metrics.
L3	S2 role?	Path-scoped only.

Provenance: Verified · S8 · (S2 bounded)

Q7. Crash only on iOS beta, 2% users — ship? (30–45s)

Answer points:

- Severity × rarity
- Payment path → mitigate
- Don't ignore rare×critical

Full spoken answer:

"I multiply severity by rarity. Two percent on a beta might be trackable if it's a non-critical screen. If it's checkout or payments, I mitigate immediately — flag, pause, or block the path — even at low percentage. Rare times critical still makes P0."

Common wrong answer: "Under 5% we always ignore."

Provenance: Learning-lab · S8 judgment

Q8. dSYM missing — symptoms? (30–45s)

Answer points:

- Unsymbolicated frames
- Broken triage speed
- Fix CI upload

Full spoken answer:

"You see addresses instead of symbols, triage slows to a crawl, and duplicate issues fragment. The fix is pipeline discipline — upload dSYMs for every shipped build and re-upload if a build slipped through. I treat missing symbols as an incident for the reliability process, not a 'Swift bug.'"

Provenance: Verified · S8 · workflow dependency

Q9. How did synchronised dictionaries help CFS? (30–45s)

Answer points:

- Removed data races on shared maps
- Serial queue / RW boundary
- Path-scoped contribution
- Explicitly NOT sole CFS cause

Full spoken answer:

"Shared dictionaries were hit from multiple queues, which caused intermittent race crashes. We serialized access with GCD serial queues and read-write locks behind a safe API so call sites couldn't touch raw storage. That eliminated concurrent-access crashes on that path and improved reliability. I don't claim that fix alone produced our ninety-nine point nine five crash-free rate — that bar was sustained by triage, IMOC, and many fixes."

Common wrong answer: "That one change is why CFS is 99.95%."

Follow-up ladder:

L	Ask	Point
L1	Actor today?	S2-A1 for greenfield.
L2	How validate?	Concurrency stress + Crashlytics watch.
L3	Sprinkle locks?	No — boundary API.

Provenance: Verified · S2 · bounded; Verified · S8 · system

Q10. Non-fatals flooding the dashboard. (30–45s)

Answer points:

- Sample/group
- Fix top offenders
- ≠ CFS
- Promote if critical UX

Full spoken answer:

"I'd sample and group, fix the top noisy offenders, and keep non-fatals conceptually separate from crash-free sessions. If a non-fatal represents broken checkout UX, I escalate on user impact even without a fatal. Volume alone isn't severity."

Provenance: Learning-lab

Q11. What belongs in a crash postmortem? (30–45s)

Answer points:

- Timeline, impact, RCA, mitigations, actions
- Blameless
- Alert gaps

Full spoken answer:

"Timeline, user impact, root cause, what mitigation bought us time, and concrete action items — tests, alerts, runbooks — written blamelessly. The goal is preventing repeat, not finding a villain. That's part of how IMOC closes the loop."

Provenance: Verified · S8

Q12. Crash SDK before analytics? (30–45s)

Answer points:

- Prefer early crash init
- Keep it fast
- Defer heavy SDKs
- Launch budget (Day 17)

Full spoken answer:

"I prefer crash reporting early so launch failures still report, but initialisation must stay cheap. Heavy analytics and non-critical SDKs defer off the first-frame path. Reliability and launch performance are a joint budget, not a fight."

Provenance: Learning-lab · + Day 17

Tricky questions

T1. "Just wrap everything in try/catch." (90–120s)

Trap: Swift try covers SIGSEGV.

Answer points:

- try covers thrown errors only
- Signals / fatalError / asserts bypass catch
- Still need handlers + discipline + triage
- Necessary locally; insufficient as strategy

Full spoken answer:

"Try/catch helps for thrown errors, but many fatals are POSIX signals, assertion failures, or fatalError-style paths that never become a catchable Swift error. You still need crash handlers, coding discipline around force unwraps and concurrency, and a triage pipeline. try/catch is necessary in places and wildly insufficient as a crash strategy."

Provenance: Learning-lab

T2. Logging with OSLog inside the signal handler. (90–120s)

Trap: Normal logging in signal context.

Answer points:

- OSLog / frameworks ≠ async-signal-safe
- Secondary crash / deadlock risk
- Preallocated mmap + safe writes only
- Breadcrumbs filled on happy path

Full spoken answer:

"That's how you get a secondary crash or deadlock. Logging frameworks aren't async-signal-safe. The crash writer must use a precomputed path — mmap and safe writes — filled without allocating. Breadcrumbs are recorded earlier on the happy path so the handler only persists what's already there."

Provenance: Learning-lab

T3. CFS stable but users report freezes. (90–120s)

Trap: Trust CFS alone.

Answer points:

- CFS ≠ hang-free / session quality
- Pull hang, MetricKit, APM hang rate
- Reproduce with Time Profiler
- Treat freezes as reliability at scale

Full spoken answer:

"Crash-free can look healthy while hangs and memory pressure destroy UX. I'd pull hang diagnostics, MetricKit signals, and any APM hang rate, reproduce with Time Profiler, and treat freezes as reliability even when CFS doesn't move. At BMS scale we cared about session quality, not a single green metric."

Provenance: Verified · S8 · culture; Day 17 tools

T4. iOS blames backend, backend blames iOS. (90–120s)

Trap: Argue in war room.

Answer points:

- Shared timeline + correlation IDs
- Failure % by layer
- Mitigate user harm first
- Data before finger-pointing (IMOC)

Full spoken answer:

"As IMOC I force a shared timeline, correlation IDs, and failure percentages by layer. We mitigate user harm first — fallback, disable feature, pause rollout — then argue root cause with data. Finger-pointing extends downtime; ownership shortens it. That's the leadership pattern behind our P0/P1 coordination."

Provenance: Verified · S8 · IMOC

T5. Hotfix vs config kill switch. (90–120s)

Trap: Always hotfix.

Answer points:

- Flag first if it cuts blast radius
- Hotfix for mandatory native path
- Accept review latency trade-off
- Payments bias fast mitigate; hotfix ≠ default ego

Full spoken answer:

"If a flag can remove blast radius in minutes, I take it. Hotfix when the crash is in a mandatory native path and flags can't save users — accepting review latency. Payment paths bias toward fast mitigate. Process cost of hotfixes means they shouldn't be the default ego move."

Provenance: Verified · S8 · (+ S7 if payments)

T6. Breadcrumbs captured auth headers. (90–120s)

Trap: "Debug later."

Answer points:

- Treat as privacy/security incident
- Scrub at source; rotate tokens
- Assess exposure
- Lint / review forbid secret keys in logs

Full spoken answer:

"That's a privacy/security incident. Scrub at source, rotate tokens, assess exposure, and add lint or code review checks for forbidden keys in logging APIs. Debuggability never outranks credential leakage. Crosses into Day 19 security thinking."

Provenance: Learning-lab · Sev mindset

T7. Crash SDK upload reliability design. (90–120s)

Trap: Upload during crash.

Answer points:

- Persist durably; upload next launch

- Backoff + cap pending reports
- Ack before delete last crash
- No networking in signal context

Full spoken answer:

"Almost never upload in the crashing process context. Persist durably, upload next launch with backoff, cap pending reports, and don't delete the last crash before acknowledgement. Battery and quota matter. Signal-time networking is both unsafe and unreliable."

Provenance: Learning-lab

T8. Intermittent unreproducible crash. (90–120s)

Trap: Close as unrepro.

Answer points:

- Better breadcrumbs + careful non-fatals
- Concurrency stress / device matrix / flag bisect
- Often races (S2 shared-map pattern)
- TSan in CI; don't close without instrumentation

Full spoken answer:

"Improve breadcrumbs, add careful non-fatal probes, concurrency stress, device matrix, and feature-flag bisect. Intermittent crashes are often races — the same reasoning as our synchronised dictionary work on a shared path. Thread Sanitizer in CI helps for some classes. Closing as unrepro without instrumentation is how intermittent becomes permanent."

Provenance: Verified · S2 · reasoning pattern; S8 triage

Timed set

Recommend **Q5, Q6, Q9 + T3, T4**. Deliver S8 STAR ≤3 min; S2 add-on ≤90s with explicit non-sole-cause line.

05 — Exercises

Exercise 1 — Honesty drill (5 min)

Speak both lines:

1. S8 CFS/IMOC one-liner
2. S2 path-scoped contribution

Then deliberately say the **wrong** line ("S2 caused 99.95%") and correct yourself out loud.

Exercise 2 — Signal-safety true/false (10 min)

1. You may `malloc` in a signal handler.
2. Upload should happen inside the handler.
3. Breadcrumbs should be filled on the happy path.
4. OSLog is always async-signal-safe.
5. dSYM UUID mismatch breaks symbolication.

Answers: F, F, T, F, T

Exercise 3 — IMOC tabletop (20 min)

Scenario: CFS drops during a sale; iOS and backend disagree.

Write a 8-line script: confirm → owner → blast radius → mitigate → comms cadence → hotfix/flag decision → watch → postmortem.

Record yourself ≤2 min.

Exercise 4 — Code read (15 min)

Read [code/BreadcrumbRing.swift](#) and [code/CrashReportNotes.swift](#) .

Speak: why ring + scrub + no network in notes.

Exercise 5 — Crash SDK whiteboard (15 min)

Boxes: handlers, mmap, ring, uploader, symbolication.

Annotate: async-signal-safe; next-launch upload; CI dSYM.

Timed drill

Q5, Q6, Q9 + T3, T4. Log misses: try/catch myth, CFS≠hangs, S2 sole-cause slip.

Day 19 — Security: ATS, SSL Pinning, Keychain & Persistence

Week 3 · Full study (self-contained) · ~4–5 hrs

Revision twin: [revision/weeks/week-03/day-19.md](#)

Outcomes

By end of day, without notes, you can:

- Explain **ATS** vs **pinning** (baseline policy ≠ identity pin)
- Implement mental model of **SPKI pinning**: SHA-256 of **Subject Public Key Info DER** — **not** raw `SecKeyCopyExternalRepresentation` bytes
- Deliver **Verified · S4** only for what shipped: Ads **Alamofire** → **URLSession**, **HTTPS**, **SSL pinning**, **domain whitelist**
- Speak **S4-A1** pin rotation / backup pins / break-glass as **design**, not “I shipped the ops runbook”
- Recite the full **persistence decision tree** (embedded in this chapter)

How to study

1. [01-foundations.md](#) — ATS, Keychain, **full persistence tree**
2. [02-deep-dive.md](#) — SPKI mechanics, rotation design, auth client lens
3. [03-production-bridge.md](#) — S4 STAR + S4-A1 honesty
4. [code/](#) — SPKI notes + persistence router sketch
5. [04-questions.md](#)
6. [05-exercises.md](#)
7. Revision twin

Critical correctness

Claim	Truth
ATS	System HTTPS/TLS baseline — not pinning
SPKI	Hash of SPKI DER
<code>SecKeyCopyExternalRepresentation</code>	Raw key bytes ≠ SPKI DER — common sample-code mistake
S4 Verified	HTTPS + pinning + domain whitelist on Ads <code>URLSession</code> migration

S4-A1	Rotation / backup / break-glass = design judgment
Tokens	Keychain only — never UserDefaults

Provenance

Label	Use
Verified · S4	Ads Alamofire → URLSession; HTTPS; SSL pinning; domain whitelisting
How I would apply it · S4-A1	Pin rotation, backup pins, staged break-glass as design
Learning-lab	Code / persistence tree teaching

Agenda opener

“I’ll cover ATS baseline, SPKI pinning with rotation as design, Keychain vs UD, then the full persistence decision tree — closing with the Ads URLSession migration.”

01 — Foundations: Transport Security & Persistence

0. North star

ATS is the floor; pinning is an extra identity check on sensitive hosts; secrets live in Keychain; pick storage from a decision tree — not one tool for everything.

1. Transport security stack

ATS (HTTPS / TLS defaults / no cleartext by default)

→ System certificate chain trust

→ Optional Pinning (SPKI hash match in URLSession delegate)

→ Domain allowlist (only call / pin known hosts)

Layer	What it does	What it is not
ATS	Blocks accidental cleartext; modern TLS expectations	MITM-proofing / pinning
System trust	CA chain validation	App-defined identity pin
Pinning	Embedded pin must match	A substitute for HTTPS
Allowlist	Limits hosts you talk to	Global “pin the internet”

ATS exceptions in Info.plist are **debt** — justify (legacy partner) + expiry plan. Never ship “Allow Arbitrary Loads” casually in prod.

2. Pinning modes (first pass)

Mode	Pins	Pros	Cons
Certificate (leaf)	Exact cert	Simple	Breaks on routine renew
SPKI / public key	Key hash	Survives cert reissue if key reused	Still need rotation when keys change

SPKI correctness (memorize)

```
SPKI pin = SHA-256( SubjectPublicKeyInfo DER bytes )
```

Correct	Incorrect
Hash SPKI DER from the certificate	Treat SecKeyCopyExternalRepresentation raw bytes as SPKI
Prefer SPKI over leaf for renew resilience	"Pinning = ATS"

Say aloud: "SPKI is the hash of the Subject Public Key Info DER — not the raw bytes from SecKeyCopyExternalRepresentation. That's a common sample-code mistake."

3. Full persistence decision tree (EMBEDDED — recite this)

Quote this table in interviews. It lives **in-chapter** so you are not dependent on opening the cheatsheet.

Use case	Store	Key config / notes
Structured data, complex queries	SQLite (WAL)	PRAGMA journal_mode=WAL; concurrent readers + one writer; off main thread
Object graph / Apple ecosystem bias	Core Data	NSPersistentContainer; background context for writes
Secrets, tokens, keys	Keychain	kSecAttrAccessibleAfterFirstUnlock (or stricter); prefer synchronizable = false for auth tokens
Simple preferences / flags	UserDefaults	Non-sensitive only — never tokens
Large binary / media	FileManager /Caches or /Application Support	Caches purgeable ; user data → Application Support
In-session, auto-evict	NSCache / LRU	Memory-pressure aware; cost bounds (e.g. image bytes)
Concurrent in-session state	Actor / serial queue	Race-free shared mutation boundary

Hard rules

- 1. **Tokens ≠ UserDefaults** (plist/backups/jailbreak exposure)
- 2. PII at rest → encrypt when threat model requires (SQLCipher / file protection; keys in Keychain/SEP)
- 3. Don't invent Core Data for three booleans
- 4. Caches can vanish — never sole source of truth for irreplaceable user data
- 5. Main thread: no heavy SQLite/Core Data writes

Recite opener (≤20s)

"It depends on sensitivity and access pattern — secrets in Keychain, prefs in UserDefaults, media in FileManager, queries in SQLite WAL or Core Data for graphs, NSCache for session images, actors for concurrent in-session state."

4. Keychain vs UserDefaults (zoom)

	Keychain	UserDefaults
For	Tokens, secrets, keys	Theme, flags, non-sensitive prefs

Protection	System keychain item ACLs	Essentially plaintext plist
Speed	Slightly slower; still ms-class reads	Fast
Sync	Can sync if enabled — disable for auth tokens usually	Backups may include

Biometrics: `LocalAuthentication` + `SecAccessControl` unlocks a Keychain item — you do **not** store Face templates in your app.

5. BMS production proof (preview)

Migrated Ads **Alamofire** → **URLSession**; enforced **HTTPS**, **SSL pinning**, **domain whitelisting**.

Pin **rotation / backup / break-glass** → speak as **S4-A1 design**, not Verified shipped runbook.

Full STAR in [03-production-bridge.md](#) .

6. Glossary

Term	Meaning
ATS	App Transport Security
SPKI	Subject Public Key Info
MITM	Man-in-the-middle
Break-glass	Emergency pin/disable path (must be designed carefully)
Allowlist / whitelist	Approved hosts only
WAL	Write-Ahead Logging (SQLite)
SEP	Secure Enclave Processor

7. Teach-back

1. ATS ≠ pinning
2. SPKI DER hash ≠ SecKey raw export
3. Full persistence table (7 rows)
4. S4 shipped vs S4-A1 design

Next: [02-deep-dive.md](#) .

02 — Deep Dive: Pinning, Rotation Design, Auth Persistence

1. Where pinning lives on iOS

```

URLSessionDelegate
→ urlSession(_:didReceive:completionHandler:)
→ server trust challenge
→ evaluate trust + extract leaf/chain cert
→ compute SHA-256(SPKI DER)
→ compare to embedded allowlisted pins for that host
→ succeed or cancel (fail closed for sensitive APIs)
```

Domain allowlist: only pin / call hosts you own or tightly control. Don't pin arbitrary CDNs you don't operate — operational surprise.

2. SPKI extraction — correctness deep dive

2.1 Right mental model

- 1. Obtain certificate (SecCertificate)
- 2. Get **Subject Public Key Info** encoding (**DER**)
- 3. SHA-256 that DER blob
- 4. Compare to pinned hashes (often base64 or hex in config)

2.2 Common mistake

Mistake	Why wrong
Hash SecKeyCopyExternalRepresentation bytes and call it SPKI	External representation is not the SPKI structure; pins won't match real SPKI tooling / browsers
Pin leaf certificate DER only	Breaks on routine cert renew even when key unchanged
Copy random GitHub gist without verifying DER step	Silent wrong pins → false sense of security or instant outage

Interview correction script:

"If a snippet hashes SecKeyCopyExternalRepresentation and labels it SPKI, I'd reject it. SPKI pinning hashes the Subject Public Key Info DER from the certificate."

3. S4-A1 — Pin ops as design (not shipped runbook claim)

When interviewers ask "what happens when cert/key rotates?":

Design element	Intent
Backup pins	Ship ≥2 acceptable SPKI hashes for planned key transition
Ship client before server key rotate	Never rotate server first against a single old pin
Staged exposure	Phased release; watch TLS/pin failure metrics
Break-glass	Monitored, time-boxed emergency path — not "silent forever off"
Fail closed default	Sensitive Ads/auth/payments: mismatch cancels challenge

Script fragment (honesty):

"On Ads we shipped pinning and whitelist on URLSession. Separately, as design, I'd require backup pins and a monitored break-glass plan — pinning without rotation thinking is an outage generator. I'm not claiming I shipped that full ops runbook; I'm claiming the control set we owned and the design I'd insist on."

Provenance: How I would apply it · S4-A1

3.1 Hard-fail vs soft-fail

Mode	When	Risk
Hard-fail (cancel)	Ads, auth, payments	Outage if rotation weak — fix ops, don't silent downgrade
Soft-fail / report only	Low sensitivity experiments	Silent security regression

4. Why leave Alamofire for Ads (S4)

- Own URLSession trust challenges end-to-end
- Smaller dependency surface on revenue-adjacent module
- HTTPS + pinning + host allowlist as explicit policy
- Interceptors/refresh still designable on first-party client (Day 09)

5. Auth & biometric (client lens)

Topic	Senior line
Access token	Short-lived; memory + Keychain as needed
Refresh token	Keychain only ; synchronizable false typically
Refresh races	Single-flight actor (Day 09) — not security day primary but linked
Biometric	Step-up / unlock Keychain item — not primary identity
App Attest / DeviceCheck	Integrity for high-risk actions — mention, don't overclaim shipped
Logout	Clear Keychain locally + server revoke

Obfuscation trap: Base64 in UserDefaults is **not** security.

6. Persistence tree — worked examples

Need	Choice	Why
Refresh token + theme + image cache	Keychain + UserDefaults + FileManager Caches (+ NSCache L1)	Sensitivity split
Offline feed with queries	SQLite WAL	Explicit SQL + concurrency
Complex object graph + CloudKit bias	Core Data	Tooling; still bg writes
Session decoded images	NSCache	Memory warnings
Shared mutable map	Actor / serial queue	Races (S2 / S2-A1)
Encrypted messages DB	SQLCipher + Keychain-wrapped key	Threat model

Re-embed full table (same as foundations — muscle memory):

Use case	Store	Key config
Structured queries	SQLite (WAL)	WAL + off-main writes
Object graph	Core Data	Background context writes
Secrets / tokens	Keychain	AfterFirstUnlock or stricter; no sync for auth
Simple prefs	UserDefaults	Non-sensitive only
Large media	FileManager Caches / App Support	Purgeable vs durable
In-session auto-evict	NSCache / LRU	Cost bounds
Concurrent in-session	Actor / serial queue	Isolation boundary

7. Jailbreak / attestation (light)

- Client detection is bypassable — defense in depth only
 - Never sole control for payments
 - Prefer server attestation for high-risk when in scope
 - False positives destroy UX — be careful
-

8. Trade-offs

Choice	When	Cost
System trust only	Low-sensitivity public content	MITM via hostile trust scenarios still a risk
SPKI pinning	Payments, ads, auth APIs	Rotation ops burden
Backup pins	Production pinning	Must ship before primary rotates
Remote pin disable	Break-glass	Attack surface — protect config channel
Keychain	Secrets	Entitlement/migration pain
UserDefaults	UX prefs	Trivial exposure if misused for secrets
Pin all hosts	Never casually	Vendor CDN rotation nightmare
SQLCipher	Sensitive DB	Perf/complexity

9. Failure modes

Failure	Senior response
Cert renew bricked app	Expected without backup pins — S4-A1 design lesson
Wrong SPKI bytes	Audit pin generation pipeline
UD “encrypted” token	Move to Keychain; rotate
Soft-fail forever	Silent insecure fallback — refuse for Ads
Core Data for booleans	Decision tree discipline

10. Pin failure incident script (60–90s)

“Pin failure spike on Ads hosts — confirm with TLS error metrics and domain allowlist hits. Mitigate with staged break-glass only if designed and monitored — never silent forever-off. Ship a build with corrected/backup SPKI hashes generated from DER, not SecKey raw export. Communicate as Sev because revenue path is hard-fail by design. Postmortem: rotation checklist gaps.”

11. Persistence tree stress drills (embed again)

Recite without notes:

1. Refresh token → **Keychain**
2. Dark mode flag → **UserDefaults**
3. Offline orders query → **SQLite WAL**
4. Ticket PDF user saved → **Application Support** (not Caches)
5. Poster images session → **NSCache** + **Caches** files

- 6. Shared in-flight map → **Actor** / serial queue
- 7. Complex graph + tooling → **Core Data** bg writes

Wrong pairings to refuse: token in UD · media in UD · Core Data for two booleans · secrets in NSCache.

12. Optional citations

- mobile-security-privacy-engine.md , authentication-oauth-biometric.md , cheatsheet storage tree
- Story S4

Self-contained without opens.

03 — Production Bridge: S4 Pinning + S4-A1 Design

1. Provenance map

ID	Label	Exact claim
S4	Verified	Ads Alamofire → URLSession ; HTTPS ; SSL pinning ; domain whitelisting
S4-A1	How I would apply it	Pin rotation , backup pins , staged break-glass as design — not “I shipped the ops runbook”
Learning-lab	Illustrative	SPKI notes, persistence router

Forbidden

- “I shipped the pin-rotation runbook / break-glass flag to production” as Verified
- “We hash SecKeyCopyExternalRepresentation as SPKI”
- “ATS is our pinning”
- “Pinning alone fixed app-wide CFS”
- Shadow-traffic rollout theater

2. Verified S4 — STAR (2–3 min)

Opener

“I’ll walk through migrating BookMyShow Ads networking from Alamofire to URLSession so we owned transport security on a revenue-critical module.”

Situation / Task

Ads on Alamofire; need stronger first-party control — HTTPS, SSL pinning, domain allowlisting — and less dependency surface.

Action

- First-party **URLSession** path for Ads networking.
- Enforced **HTTPS**.
- SSL pinning** via session trust evaluation — fail closed on mismatch for pinned hosts.
- Domain whitelist** — only approved hosts.
- Treated pinning incomplete without ops mindset → see S4-A1 as design.

Result

Reduced MITM exposure on high-traffic Ads module; clearer ownership of trust evaluation.

Lesson

Pinning without rotation/backup/break-glass **design** creates self-inflicted outages.

Provenance: Verified · S4 · Ads URLSession + HTTPS + pinning + whitelist

3. ≤20s line

"I moved Ads off Alamofire onto URLSession with HTTPS, SSL pinning, and domain allowlisting — and I'd pair pinning with backup pins and a break-glass design so rotation doesn't brick the app."

4. S4-A1 script (separate breath)

"Backup pins, ship-before-rotate, staged exposure, monitored break-glass — that's the design I'd insist on. I'm not claiming I shipped that full runbook; I'm claiming the controls we owned plus the operational design required to keep pinning safe."

Provenance: How I would apply it · S4-A1

5. Persistence recite (45s drill)

Recite the 7-row tree from foundations without notes. End with: "Tokens never in UserDefaults."

04 — Questions (two-layer Q&A)

12 normal + 8 tricky. SPKI DER · S4 vs S4-A1 · full persistence tree.

Normal questions

Q1. What does ATS protect you from? (30–45s)

Answer points: Accidental cleartext; weak TLS configs; baseline not full MITM defense.

Full spoken answer:

"App Transport Security is the system baseline that prefers HTTPS and blocks cleartext by default, pushing modern TLS configurations. It reduces accidental insecure transport. It is not pinning and it does not make you immune to every MITM scenario — compromised trust stores or hostile enterprise inspection are separate conversations. Exceptions in Info.plist are debt with an expiry plan."

Common wrong answer: "ATS means we're pinned."

Provenance: Learning-lab

Q2. Certificate pinning vs SPKI pinning? (30–45s)

Answer points: Leaf breaks on renew; SPKI pins key material; still need rotation when keys change; SPKI = DER hash.

Full spoken answer:

"Leaf certificate pinning breaks when the cert is reissued even if the key is unchanged. SPKI pinning hashes the Subject Public Key Info DER — so it can survive cert renewal when the same key is reused. Keys still change eventually, so you need a rotation design with backup pins. And SPKI means the DER structure hash — not raw SecKeyCopyExternalRepresentation bytes."

Provenance: Verified · S4 · + SPKI teaching

Q3. Where do you implement pinning on iOS? (30–45s)

Answer points: URLSessionDelegate server trust challenge; evaluate + compare SPKI; domain allowlist.

Full spoken answer:

"In the URLSession delegate authentication challenge for server trust. I evaluate trust, extract the certificate's SPKI DER, SHA-256 it, and compare against embedded pins for allowlisted hosts. Mismatch cancels the challenge for sensitive Ads or auth"

traffic. That's the ownership we wanted when Ads moved onto URLSession."

Provenance: Verified · S4

Q4. Pin rotation strategy? (45–60s)

Answer points: Ship new pin before server rotate; old+new; break-glass design; monitor fail rates; label S4-A1.

Full spoken answer:

"I'd ship a client build that already contains the new SPKI hash alongside the old one, then rotate the server key, watch pin-failure metrics, and retire the old pin later. I'd design a monitored break-glass path for emergencies. On Ads we shipped pinning and whitelist; this rotation playbook I'm describing as design judgment — S4-A1 — not as a claim I shipped the full ops runbook."

Common wrong answer: "Rotate the server first; clients will catch up."

Provenance: How I would apply it · S4-A1 · (Verified S4 shipped pins)

Q5. Keychain vs UserDefaults? (30–45s)

Answer points: Secrets→Keychain; prefs→UD; never tokens in UD.

Full spoken answer:

"UserDefaults is for non-sensitive preferences. Tokens, refresh credentials, and keys go in Keychain with appropriate accessibility — often AccessibleAfterFirstUnlock if background refresh matters — and I usually disable iCloud keychain sync for auth tokens. Putting a token in UserDefaults is a security bug, not a shortcut."

Provenance: Learning-lab · auth persistence

Q6. Persist refresh token + theme + image cache — where? (30–45s)

Answer points: Keychain / UserDefaults / FileManager Caches (+ NSCache); recite tree.

Full spoken answer:

"Refresh token in Keychain, theme in UserDefaults, image files in Caches via FileManager with an NSCache memory layer for the session. That's the persistence decision tree in miniature — sensitivity and lifetime drive the store, not convenience."

Provenance: Learning-lab · embedded decision tree

Q7. Why domain whitelisting with pinning? (30–45s)

Answer points: Limit pinned hosts; avoid CDNs you don't control; reduce surprise failures.

Full spoken answer:

"Allowlisting limits which hosts the client will talk to and which identities we pin. Pinning random third-party hosts you don't operate is how you inherit their cert rotation pain. On Ads we combined pinning with a domain whitelist so the revenue module only spoke to approved hosts."

Provenance: Verified · S4

Q8. Biometric auth — what is stored? (30–45s)

Answer points: Not face template; LA unlocks Keychain / confirms presence; policy for passcode fallback.

Full spoken answer:

"The app doesn't store the user's face or fingerprint templates. LocalAuthentication confirms presence and can gate Keychain items via access control. Biometrics are a device-owner step-up, not the primary account identity. Fallback to passcode or fail-closed is an explicit product policy for high-risk actions."

Q9. SQLite vs Core Data for offline feed? (30–45s)

Answer points: SQLite/WAL for explicit control; Core Data for graph+tooling; both off-main writes.

Full spoken answer:

"For an offline feed with explicit queries I'd lean SQLite in WAL mode — concurrent readers, one writer, predictable SQL. Core Data wins when I want an object graph and Apple tooling. Either way, writes stay off the main thread. I wouldn't drag Core Data in for a handful of preference booleans."

Provenance: Learning-lab · decision tree

Q10. Jailbreak detection — worth it? (30–45s)

Answer points: Defense-in-depth; bypassable; never sole control; server attestation for high-risk.

Full spoken answer:

"Client jailbreak signals are bypassable, so I treat them as defense-in-depth for high-risk apps, never as the only control. For payments-grade trust I'd rather combine server-side attestation and least privilege than invent a false sense of security from path checks alone — and I'd watch false positives that punish legitimate users."

Provenance: Learning-lab

Q11. Why leave Alamofire for Ads? (30–45s)

Answer points: Ownership of trust/pinning; smaller dependency; URLSession enough.

Full spoken answer:

"Alamofire is fine for rapid CRUD, but Ads needed first-party ownership of the trust evaluation path — HTTPS, pinning, host allowlist — with a smaller dependency surface. URLSession gave us that control on a high-traffic revenue module. It's threat-model driven, not ideology against libraries."

Provenance: Verified · S4

Q12. Encrypt local DB — when? (30–45s)

Answer points: Sensitive PII/finance at rest; SQLCipher or file protect; keys in Keychain/SEP.

Full spoken answer:

"When the threat model says PII or financial data at rest matters — SQLCipher or strong file protection, with keys in Keychain or Secure Enclave-backed protection. I don't encrypt everything by cargo cult; I match the store to sensitivity using the persistence decision tree."

Provenance: Learning-lab

Tricky questions

T1. Pinning broke production after cert renew. (90–120s)

Trap: Blame DevOps only / deny design failure.

Answer points: Predictable without backup pins + ship-before-rotate; break-glass + ship new SPKI; incident/comms; S4 shipped pins, rotation = S4-A1 design.

Full spoken answer:

"That's the predictable failure mode of pinning without backup pins and a ship-before-rotate sequence. I'd mitigate with break-glass if designed, ship a build containing the new SPKI, and treat it as an incident with comms. The lesson I cite from Ads work: we shipped pinning for MITM resistance, and rotation thinking has to be part of the design — which I frame as S4-A1 design judgment, not theater that I personally ran every ops runbook."

Provenance: Verified · S4 · + S4-A1 design

T2. UserDefaults for "obfuscated" token. (90–120s)

Trap: Base64 as security.

Answer points: Obfuscation ≠ protection; UD in plists/backups; move to Keychain; rotate + lint against UD secrets; Base64 = encoding.

Full spoken answer:

"Obfuscation isn't protection. UserDefaults lands in plists and backups; jailbroken or forensic access reads it trivially. Move tokens to Keychain, shorten access-token lifetime, rotate what leaked, and add review lint against UD secret keys. Base64 is encoding, not encryption."

Provenance: Learning-lab

T3. Pin all hosts including analytics. (90–120s)

Trap: Pin everything.

Answer points: Vendor rotation pain; pin owned critical surfaces + allowlist; commodity analytics on system trust unless threat model says otherwise; Ads whitelist = scoped.

Full spoken answer:

"That's an operational nightmare. Vendor endpoints rotate unpredictably. I pin critical surfaces we own — auth, payments, Ads APIs — with an allowlist, and leave commodity analytics on normal system trust unless there's a specific threat model. BMS Ads whitelist was about scoped control, not pinning the universe."

Provenance: Verified · S4 · judgment

T4. MITM on public Wi-Fi — ATS alone enough? (90–120s)

Trap: Yes.

Answer points: ATS = config hygiene, not full MITM story; hostile/atypical trust still matter; pinning checks SPKI beyond "some CA"; avoid-Wi-Fi ≠ eng control.

Full spoken answer:

"ATS helps configuration hygiene but isn't a complete MITM story. Hostile networks plus compromised or atypical trust scenarios still matter for sensitive APIs. Pinning raises the bar for Ads and auth by checking SPKI identity beyond 'some trusted CA signed this.' Telling users to avoid public Wi-Fi is not an engineering control."

Provenance: Verified · S4 · threat model

T5. OAuth tokens in a modular app? (90–120s)

Trap: Feature modules stash tokens in UD.

Answer points: Auth module owns Keychain; TokenProviding protocol; single-flight refresh; modules never UD for secrets.

Full spoken answer:

"A core Auth module owns Keychain. Features depend on a TokenProviding protocol. Refresh is single-flight so concurrent 401s don't stampede. Modules never reach into UserDefaults for secrets. That's Day 09 networking plus Day 15 modular boundaries meeting Day 19 persistence rules."

T6. Face ID fails → unlock secrets anyway? (90–120s)

Trap: Silent insecure fallback.

Answer points: Explicit policy; high-risk → fail-closed or passcode; never silent unlock on biometric error; log without PII.

Full spoken answer:

"Policy must be explicit. For high-risk actions I may fail closed or require passcode fallback — never silently unlock secrets because biometrics erred. Log carefully without PII. Biometrics gate Keychain items; they don't justify weakening storage."

Provenance: Learning-lab

T7. Pinning hard-fail vs crash-free pressure. (90–120s)

Trap: Always soft-fail to protect CFS vanity.

Answer points: Soft-fail → insecure transport risk; hard-fail on Ads/payments mismatch; invest in rotation so rare; mass fails = incident, not quiet CFS protect.

Full spoken answer:

"Soft-fail can silently degrade to insecure transport. For Ads and payments I prefer hard-fail on pin mismatch, and I invest in rotation design so hard-fail is rare. Mass pin failures become an IMOC-style incident — mitigate with break-glass design and a fixed pin build — not a quiet security regression to keep a dashboard green."

Provenance: S4 · S4-A1 · S8 crossover judgment

T8. "Just use Core Data for everything." (90–120s)

Trap: One store to rule them all.

Answer points: Walk persistence tree; prefs/secrets/media/queries differ; CD for 3 booleans = overkill; UD tokens = bug; sensitivity + access pattern.

Full spoken answer:

"I'd walk the persistence decision tree. Prefs, secrets, media, and query workloads differ. Core Data for three booleans is complexity without benefit; UserDefaults for tokens is a security bug; SQLite WAL shines for explicit offline queries. Wrong store creates security or complexity failures. Sensitivity plus access pattern — not ideology."

Provenance: Learning-lab · embedded tree

Extra spoken — Recite the persistence tree (45–60s)

Answer points: Walk all 7 rows; end with tokens≠UD.

Full spoken answer:

"Storage follows sensitivity and access pattern. Secrets and tokens go in Keychain with appropriate accessibility. Simple non-sensitive preferences go in UserDefaults — never tokens. Structured offline queries use SQLite in WAL mode off the main thread. Object graphs with Apple tooling lean Core Data with background writes. Large media uses FileManager — Caches if purgeable, Application Support if durable user data. In-session images use NSCache or an LRU. Concurrent in-session mutable state uses an actor or serial queue. I refuse one-store-for-everything answers."

Provenance: Learning-lab · in-chapter decision tree

Extra spoken — SPKI vs SecKey one-liner (20s)

Answer points: SPKI = Subject Public Key Info DER hash; ≠ SecKeyCopyExternalRepresentation bytes.

Full spoken answer:

"SPKI pinning hashes the Subject Public Key Info DER from the certificate. Hashing SecKeyCopyExternalRepresentation bytes and calling it SPKI is a common mistake — those aren't the same bytes."

Timed set

Q2, Q4, Q6 + T1, T8. Add persistence-tree recite. S4 STAR ≤3 min.

Log: UD tokens, cert-only pins, SecKey-as-SPKI, rotation as Verified runbook slip.

05 — Exercises

Exercise 1 — Persistence recite (10 min)

Recite all 7 rows of the decision tree twice without notes. Record once.

Exercise 2 — SPKI true/false (10 min)

1. SPKI pinning hashes Subject Public Key Info DER.
2. SecKeyCopyExternalRepresentation bytes equal SPKI DER.
3. SPKI pins always survive key rotation.
4. ATS equals pinning.
5. S4-A1 rotation is Verified shipped runbook.

Answers: T, F, F (only if key unchanged / still need design), F, F (design)

Exercise 3 — Pin challenge whiteboard (15 min)

Draw URLSession trust challenge → SPKI hash → allowlist → fail closed. Annotate S4 shipped vs S4-A1 design.

Exercise 4 — Code (15 min)

Read [code/SPKIPinningNotes.swift](#) and [code/PersistenceDecision.swift](#). Speak the router choices for token/theme/image/feed.

Exercise 5 — S4 STAR (12 min)

Record STAR; end with one breath of S4-A1 honesty.

Timed drill

Q2, Q4, Q6 + T1, T8.

Day 20 — Deep Links, Push Notifications & CI/CD Release Trains

Week 3 · Full study (self-contained) · ~4–5 hrs

Revision twin: [revision/weeks/week-03/day-20.md](#)

Outcomes

By end of day, without notes, you can:

- Contrast **Universal Links vs custom schemes**, explain **AASA**, cold-start **queue-until-ready**, deferred links
- Walk push: permission → APNs token → provider (**Airship**) → payload → **same DeepLinkRouter**
- Outline **GitHub Actions** → **build/lint** → **TestFlight** and release-train **pause** criteria (CFS/perf)

- Deliver **Verified · S13**: Grizzlies **SwiftUI↔UIKit, deeplinks, Mixpanel, Airship**
- Deliver BMS **CI/CD** automation honestly (Actions → TestFlight; AI PR review as assist — S9 judgment)

How to study

1. [01-foundations.md](#)
2. [02-deep-dive.md](#)
3. [03-production-bridge.md](#) — S13 + BMS CI
4. [code/](#) — router queue sketch
5. [04-questions.md](#)
6. [05-exercises.md](#)
7. Revision twin

Provenance

Label	Use
Verified · S13	Grizzlies: SwiftUI+UIKit interop; deeplinks; Mixpanel; Airship
BMS CI	GitHub Actions build/lint → TestFlight (resume automation)
Verified · S9	AI-assisted review judgment — assist, don’t own architecture
Verified · S8	Soft bridge: pause rollout on CFS

Module map

Module	File
Foundations	01-foundations.md
Deep dive	02-deep-dive.md
Production bridge	03-production-bridge.md
Questions	04-questions.md
Exercises	05-exercises.md
Code	code/DeepLinkQueue.swift

Agenda opener

“I’ll unify Universal Links and push as endpoints into one router, cover cold-start queuing, then walk GitHub Actions → TestFlight with rollout gates.”

Time budget

Block	Minutes
Foundations + deep dive	90–110
Production bridge + code	35–45
Questions	45–60
Exercises + revision	30–40

01 — Foundations: Deeplinks, Push, Release Trains

0. North star

One router for every endpoint; queue links until navigation is ready; automate the train; keep a stop button for CFS/perf.

1. Deep linking pipeline

```
https://domain/path?params (Universal Link if AASA ok)
OR myapp://... (custom scheme – hijackable)
→ Scene / onOpenURL
→ DeepLinkRouter (parse, validate, match)
→ AppCoordinator (nav ready?)
→ Target screen OR queue until warm
```

	Universal Links	Custom scheme
Security	Domain association via AASA	Hijackable by other apps
UX	https works in browser too	myapp:// only
Setup	AASA + entitlements	URL types in Info

AASA: JSON at `/.well-known/apple-app-site-association` over HTTPS; lists appIDs + paths; size limits; CDN caching can delay fixes (~OS cache).

Cold start race: link arrives before DI/nav ready → **queue** intent; flush when root UI ready; invalid → safe home + metric.

Security: validate params; auth-gate checkout; never trust URL for price/user authority.

2. Push pipeline

```
Permission (contextual) → register for remote notifications
→ APNs device token → backend / Airship
→ Campaign / transactional push
→ Tap → deep link / category action → SAME router as UL
```

Topic	Senior line
Airship	Engagement layer on APNs — segments/journeys; still know APNs
Payload	Small; route id/URL; defensive decode (CFS!)
Token lifecycle	Register; upload; refresh on reinstall/OS; invalidate old
Foreground vs tap	Different handlers; unify routing

Verified · S13: Memphis Grizzlies — deeplinks + **Airship** + **Mixpanel** atop hybrid SwiftUI/UIKit navigation.

3. CI/CD & release trains

```
PR → GitHub Actions (lint, build, unit tests)
  → merge → archive + sign
  → TestFlight (internal/external)
  → phased App Store
  → monitor CFS / perf → PAUSE if needed
```

BMS proof: automated **GitHub Actions** for build, lint, **TestFlight** — cut manual release overhead.

Gates: tests green, size budget, **dSYM upload** (Day 18), CFS/perf thresholds on phased rollout, feature flags.

Signing: secrets in CI — never commit `.p12` in clear.

AI PR review: accelerate regression spotting; humans own architecture/security (**S9** judgment).

4. Glossary

Term	Meaning
AASA	apple-app-site-association
Deferred deep link	Post-install route attribution
APNs	Apple Push Notification service
Collapse id	Collapse related notifications
Release train	Cadenced ship + phased %
match / signing	Cert/profile management in CI

5. Teach-back

1. UL vs scheme
2. One router for UL + push
3. Cold-start queue
4. Actions → TestFlight + pause criteria
5. S13 one-liner

Next: [02-deep-dive.md](#) .

02 — Deep Dive: Routing, Airship, CI Gates

1. Universal Links debugging checklist

When links open Safari instead of the app:

1. AASA reachable over HTTPS; correct content-type
2. `appID` = `TEAMID.bundle`
3. Paths match (not excluded)
4. Entitlements associated domains
5. Apple CDN / device cache delay after AASA changes
6. User long-press “Open in Safari” preference
7. Universal Links vs opening from notes/apps quirks

Trap: “iOS bug” as first answer.

2. Router architecture

```
Entrypoints: UL | custom scheme | push tap | Spotlight (optional)
  ↓
DeepLinkParser → AppRoute (typed enum)
  ↓
AuthGate / Validate params
  ↓
Coordinator.navigate(route) OR PendingDeepLinkStore.enqueue
```

Why one table: dual routers (push vs UL) drift — checkout works from one entry and 404s from another.

Hybrid UI (S13): SwiftUI hosted in UIKit (or reverse) — coordinator owns stack identity; don't ad-hoc `UIHostingController` without lifecycle plan.

3. Deferred deep links

- Install attribution window; first launch fetches pending route
 - Expiry (industry often ~days, not forever)
 - Privacy / probabilistic matching limits — don't overclaim certainty
 - Prefer first-party login then route when identity matters
-

4. Push deep dive

Concern	Practice
Permission timing	Contextual, not instant first launch
Token every launch	Tokens change; upsert server-side
410 Unregistered	Invalidate token in DB
Malformed payload	Defensive decode → home; protect CFS
Rich images	Extension memory budgets
Silent push	Limited wake budget; don't rely as cron

Airship vs Mixpanel (S13): Airship = push/engagement; Mixpanel = analytics product behavior — complementary, not synonyms.

5. CI/CD deep dive

5.1 PR checks

Lint + build + unit tests on macOS runners; selective tests when modularized; quarantine flakes.

5.2 Signing

Encrypted certs/profiles (match-style); CI secrets / OIDC; `CODE_SIGNING` deterministic; never log secrets.

5.3 TestFlight + phased release

- Internal smoke before external
- Phased % with **stop criteria**: CFS drop, pin-fail spike, journey p90 cliff
- Feature flags decouple binary ship from exposure

5.4 dSYM

Upload every build that can reach users — Day 18 triage depends on it.

5.5 AI on PRs (S9)

"AI accelerates review for obvious regressions; humans own architecture, security, and product trade-offs. I never say 'AI approved so it's fine.'"

6. Secure checkout link

- Auth gate
- Server-authoritative cart/price
- Ignore spoofed query prices
- Confirm destructive actions

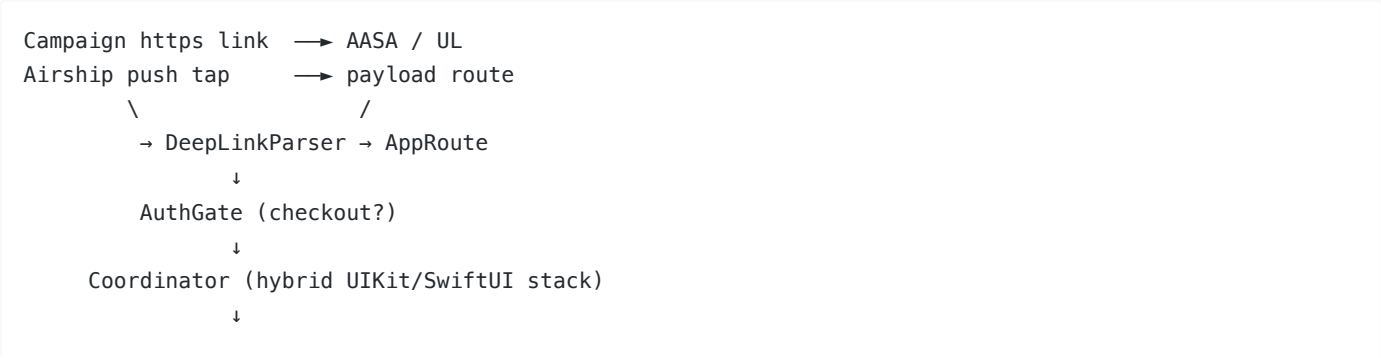
7. Trade-offs

Choice	When	Cost
Universal Links	Consumer production	AASA/CDN debug pain
Custom schemes	Legacy/internal	Hijack risk
Central router	Multi-feature	Must stay module-friendly
Airship	Speed + marketing tools	Vendor; still know APNs
Manual TestFlight	Tiny team	Human error
Fully auto prod	Only with strong gates	Risk if gates weak
AI PR review	Diff noise reduction	Must not replace ownership

8. Failure modes

Mode	Response
Dual routers drifted	Unify table
Push JSON crash	Defensive parse
TF crash-loop, CI green	Release flags / entitlements / race — smoke TF
CFS drop at 10% phased	Pause — IMOC (S8)
Secrets in git	Rotate; move to CI secrets

9. End-to-end story you can draw in 3 minutes



Mixpanel screen event | Airship engagement attribution

Meanwhile release train:

PR Actions → TestFlight → phased % → CFS/p90 monitors → pause/IMOC

Say: "Entrypoints differ; routing and release gates are shared disciplines."

10. BMS CI elevator (45s) + Grizzlies (90s) back-to-back

CI:

"We automated GitHub Actions for lint, build, and TestFlight upload so the release path stopped depending on who remembered the manual checklist. Signing secrets stay in CI — not the repo. Phased rollout still needs human stop criteria on crash-free and perf."

Grizzlies:

"On Grizzlies I designed hybrid SwiftUI/UIKit surfaces with real navigation ownership, deep links into that stack, Mixpanel for product analytics, and Airship for push engagement — one router mindset so campaigns and https links don't drift."

11. Optional citations

- deep-linking / push / mobile-ci-cd docs
- Stories S13, S8, S9

Self-contained without opens.

03 — Production Bridge: S13 Grizzlies + BMS CI

1. Provenance map

ID / claim	Label	Exact claim
S13	Verified	Grizzlies: SwiftUI↔UIKit ; deeplinks ; Mixpanel ; Airship
BMS CI	Verified (resume)	GitHub Actions automation for build/lint/ TestFlight
S9	Verified	AI tooling / Context Engineering judgment — assist ≠ own
S8	Verified	Soft bridge — pause on CFS during rollout

Forbidden

- Invented CTR %, opt-in %, or "zero release incidents"
- Claiming you built Airship itself
- "AI approved the release"

2. Verified S13 — STAR (2–3 min)

Opener

"I'll cover hybrid SwiftUI/UIKit navigation on the Grizzlies app with production deeplinks and the Airship/Mixpanel engagement stack."

Situation / Task

NBA consumer app needing modern UI atop existing UIKit, plus growth tooling — deeplinks, analytics, push.

Action

1. Architected key surfaces with **SwiftUI + UIKit interoperability** — hosting/lifecycle designed, not bolted.
2. Deep linking + navigation/lifecycle handling into a coherent routing approach.
3. Integrated **Mixpanel** (analytics) and **Airship** (push/engagement).

Result

Shipped hybrid UI with production navigation and engagement stack.

Lesson

Interop costs — identity, lifecycle, hosting — must be designed. Push and Universal Links should share one routing table.

Provenance: Verified · S13 · Raw / Memphis Grizzlies

3. Interview lines

S13 ≤20s:

“On Grizzlies I owned deeplinks and Airship push atop hybrid SwiftUI/UIKit navigation, with Mixpanel for analytics.”

BMS CI ≤20s:

“At BookMyShow I automated GitHub Actions for build, lint, and TestFlight so releases weren’t a manual ceremony.”

Combined ≤25s:

“On Grizzlies I shipped deeplinks and Airship on hybrid UI; at BMS I automated GitHub Actions into TestFlight — entrypoints and release trains as one reliability story.”

4. AI review breath (S9)

“We used AI-assisted PR review to catch regressions faster; humans still owned architecture and security calls.”

Provenance: Verified · S9 · judgment

04 — Questions (two-layer Q&A)

12 normal + 8 tricky. S13 Airship/deeplinks · BMS Actions→TestFlight.

Normal questions

Q1. Universal Links vs custom URL schemes? (30–45s)

Answer points: AASA domain proof vs hijackable schemes; prefer UL for consumer.

Full spoken answer:

“Universal Links use https with an AASA file that associates your domain to your app IDs and paths — stronger against hijacking and better web interop. Custom schemes are easy but other apps can register the same scheme. For consumer apps I prefer Universal Links publicly and keep schemes as legacy fallback.”

Provenance: Verified · S13 · deeplinks

Q2. What is an AASA file? (30–45s)

Answer points: JSON at well-known path; appIDs + paths; HTTPS; cache pitfalls.

Full spoken answer:

"apple-app-site-association is a JSON document served over HTTPS at the well-known path listing application identifiers and path patterns. The OS uses it to decide whether an https link opens your app. CDN caching and Apple's cache mean AASA mistakes can take a while to clear — so debugging is systematic, not vibes."

Provenance: Learning-lab

Q3. Deep link on cold start — race? (30–45s)

Answer points: Queue until coordinator/DI ready; then route; fallback home.

Full spoken answer:

"On cold start the link can arrive before the navigation graph and DI are ready. I enqueue the pending route and flush once the root coordinator is live. If the path is invalid I fall back to home and emit a metric. Routing into a half-built stack is how you get crashes and blank screens."

Provenance: Verified · S13 · navigation/lifecycle

Q4. How do pushes open a screen? (30–45s)

Answer points: Payload carries route; tap → same DeepLinkRouter as UL.

Full spoken answer:

"The notification payload carries a route identifier or URL. On tap I hand that to the same DeepLinkRouter Universal Links use — one table, two entrypoints. That way Airship campaigns and https links can't drift. Defensive decoding protects crash-free if marketing ships malformed JSON."

Provenance: Verified · S13 · Airship + deeplinks

Q5. APNs token lifecycle? (30–45s)

Answer points: Register; upload; refresh on reinstall/OS; invalidate old.

Full spoken answer:

"I register for remote notifications, receive the APNs device token, and upsert it to the provider — Airship or our backend. Tokens change across reinstall and some OS events, so I refresh regularly. When APNs reports unregistered, I invalidate server-side so we don't fan out to dead tokens forever."

Provenance: Learning-lab · (+ S13 Airship context)

Q6. Why Airship instead of raw APNs only? (30–45s)

Answer points: Segmentation, journeys, creative tooling; still APNs underneath.

Full spoken answer:

"Airship sits on top of APNs and gives segmentation, automation, and campaign tooling marketing needs. I still understand raw APNs — tokens, priorities, payloads — because vendors don't remove failure modes. On Grizzlies we integrated Airship for push/engagement alongside Mixpanel for analytics."

Provenance: Verified · S13

Q7. Outline a mobile CI pipeline. (45–60s)

Answer points: PR build/test/lint → archive/sign → TestFlight → phased + monitors; dSYM.

Full spoken answer:

"PRs run GitHub Actions for lint, build, and unit tests on macOS runners. Main cuts an archived, signed build uploaded to TestFlight, then phased App Store release. Gates include tests, binary size, dSYM upload, and pause criteria on crash-free or perf regressions. At BookMyShow we automated Actions into TestFlight to remove manual release ceremony."

Q8. What did automation fix vs manual releases? (30–45s)

Answer points: Human signing/upload variance; standardizes build/lint/TF.

Full spoken answer:

"Manual trains vary by who remembers signing steps and upload checklists. Actions standardizes build, lint, and TestFlight upload so the path is repeatable. Humans still own rollout judgment and stop criteria — automation removes toil, not responsibility."

Provenance: BMS CI

Q9. Feature flags vs release train? (30–45s)

Answer points: Train ships binary; flags decouple exposure; together enable safe rollout.

Full spoken answer:

"The release train ships a binary on a cadence. Feature flags decide who sees risky surfaces inside that binary. Together you can pause exposure without waiting solely on App Review. CFS drops at ten percent phased should pause the train — flags and IMOC complete the story."

Provenance: Learning-lab · + S8 crossover

Q10. Hybrid SwiftUI/UIKit + deeplink footgun? (30–45s)

Answer points: Hosting VC identity; nav stack ownership; double lifecycle; design coordinator.

Full spoken answer:

"Hosting controllers change identity and lifecycle in ways that break naive push/pop assumptions. Deeplinks need a coordinator that owns the real stack across UIKit and SwiftUI islands. On Grizzlies we designed interop deliberately — bolting UIHostingController ad hoc is how deeplinks land on the wrong screen."

Provenance: Verified · S13

Q11. Secure a /checkout deep link. (30–45s)

Answer points: Auth gate; server cart truth; validate params; ignore spoofed prices.

Full spoken answer:

"Checkout links must pass an auth gate. The server remains authoritative for cart and price — query parameters never set what you pay. I validate and sanitize params, and I don't auto-confirm destructive actions from a link alone."

Provenance: Learning-lab · payments-adjacent

Q12. AI on PRs — how describe? (30–45s)

Answer points: Accelerates regression review; humans own architecture/security; S9.

Full spoken answer:

"AI-assisted review speeds catching obvious regressions and diff noise. Humans still own architecture, security, and product trade-offs. I frame it as Context Engineering and assistive tooling — never 'the model approved so we shipped.'"

Provenance: Verified · S9

Tricky questions

T1. Universal Links open Safari instead of app. (90–120s)

Trap: "iOS bug."

Answer points: Checklist AASA/HTTPS/content-type; team+bundle+paths+entitlements; CDN/Apple cache; user Safari preference; verify after fix.

Full spoken answer:

"I run a checklist: AASA HTTPS reachability and content-type, team ID and bundle match, path patterns, associated domain entitlements, CDN/Apple cache delay, and user preference to open in Safari. Systematic debug beats shrugging. After an AASA fix I expect cache lag — I verify with controlled opens and server logs."

Provenance: Learning-lab

T2. Push deep link crashes on malformed JSON. (90–120s)

Trap: Trust marketing payload.

Answer points: Payloads untrusted; defensive decode + schema version; no force-unwrap; home on failure; same router as UL; protect CFS.

Full spoken answer:

"Payloads are untrusted input. I defensively decode, version the schema, never force-unwrap, and route to home on failure. A marketing typo must not move crash-free. Same router as Universal Links, same validation mindset. That protects the CFS bar we care about operationally."

Provenance: S13 · + S8 CFS culture

T3. Two routers drifted. (90–120s)

Trap: Duplicate switch statements forever.

Answer points: Push/UL = entypoints only; one DeepLinkRouter + AppRoute table; duplication causes drift; modules register via interfaces.

Full spoken answer:

"Push and Universal Links are entypoints only. One DeepLinkRouter and one AppRoute table own behavior. Duplication is how checkout works from a banner and 404s from an https campaign. Module feature builders register routes through interfaces — Day 15 modular thinking."

Provenance: Learning-lab · S13 lesson

T4. CI green but TestFlight crash-loops. (90–120s)

Trap: "Tests enough."

Answer points: Release ≠ Debug; entitlements/symbols/races; TF smoke + dSYMs; unit tests ≠ signing proof; S2 race classes still matter.

Full spoken answer:

"Release compiler settings, missing entitlements, stripped symbols, and concurrency races show up outside Debug. I smoke on TestFlight, keep dSYMs, and compare Release vs Debug flags. Green unit tests never proved entitlements or real signing. Race classes like shared maps — S2 thinking — still matter in Release."

Provenance: Learning-lab · + S2 reasoning optional

T5. Phased release CFS drops at 10%. (90–120s)

Trap: Continue to 100%.

Answer points: Pause immediately; IMOC + bisect version/flags; disable path / hotfix; stop criteria required; CFS = ops bar at BMS scale.

Full spoken answer:

"Pause immediately. Enter IMOC, bisect version and feature flags, disable the bad path, hotfix if needed. CI/CD without stop criteria is a conveyor belt into an incident. At BMS scale we treated crash-free as an operational bar — continuing to one hundred percent would be malpractice."

Provenance: Verified · S8 · + BMS CI gates

T6. Deferred deep link attributes wrong user. (90–120s)

Trap: Fingerprinting forever / overclaim.

Answer points: Probabilistic + privacy-constrained; expiry windows; don't overclaim; prefer post-login when identity matters; no infinite fingerprint retention.

Full spoken answer:

"Deferred attribution is probabilistic and privacy-constrained. I use expiry windows, avoid overclaiming certainty, and prefer routing after first-party login when identity matters. Infinite retention of install fingerprints is both a privacy and correctness smell."

Provenance: Learning-lab

T7. Secrets in GitHub Actions. (90–120s)

Trap: Commit profiles/p12.

Answer points: Encrypted secrets / match-style store; least-privilege ASC keys; rotate on exposure; never log; p12 in repo = incident.

Full spoken answer:

"Signing material lives in encrypted secrets or a match-style private store, least-privilege App Store Connect keys, rotation on exposure, and never logged. Committing a p12 is an incident. Automation should reduce secret sprawl, not paste certs into the repo."

Provenance: Learning-lab · CI security

T8. Marketing wants myapp:// everywhere. (90–120s)

Trap: Refuse without options / accept hijack risk silently.

Answer points: Prefer UL for public campaigns; explain scheme hijack risk; schemes as legacy fallback; migrate gradually; one route table either way (S13).

Full spoken answer:

"I prefer Universal Links for public campaigns and explain hijack risk of custom schemes. I offer schemes as transitional fallback for legacy and migrate campaigns gradually. On Grizzlies we lived in a practical mix while designing routing so either endpoint still hit one table."

Provenance: Verified · S13 · judgment

Timed set

Q3, Q7, Q10 + T1, T5. S13 STAR + 45s CI elevator.

05 — Exercises

Exercise 1 — One-router whiteboard (15 min)

Draw UL + push tap → Parser → AppRoute → AuthGate → Coordinator / PendingQueue.

Exercise 2 — AASA debug teach-back (10 min)

List 6 reasons links open Safari. No “iOS bug” as #1.

Exercise 3 — Code (15 min)

Read [code/DeepLinkQueue.swift](#) . Speak cold-start flush rules.

Exercise 4 — CI stages (10 min)

Recite: PR → Actions → TF → phased → pause on CFS/perf. Include dSYM.

Exercise 5 — Stories (15 min)

Record S13 (2–3 min) + BMS CI (45s) + optional S9 AI breath (20s).

Timed drill

Q3, Q7, Q10 + T1, T5.

Day 21 — System-Design Mock #3 (45 min)

Week 3 · Full study (self-contained) · ~5–6 hrs (weekend-style)
Revision twin: [revision/weeks/week-03/day-21.md](#)

Outcomes

By end of day you can:

- Run a **45-min** mobile SD: **clarify → HLD → API → deep dive → ops**
- Deliver a **fully written script** for **either** SDUI **or** Networking+pinning (both scripts are in-chapter — pick one live)
- Ground claims in resume-true proof only (S3/S12/S6 or S4/S5/S2) — **no fabricated metrics**
- Self-score with the **two-layer** rubric (structure + content) and log gaps for Week 4

How to study

1. [01-foundations.md](#) — 45-min spine + agenda openers
2. [02-deep-dive.md](#) — **full spoken scripts** for Prompt A & B
3. [03-production-bridge.md](#) — which stories for which prompt
4. [04-questions.md](#) — warm-up Qs **and** scoring-layer questions
5. [05-exercises.md](#) — live mock procedure
6. Optional [code/](#) — tiny schema / client interfaces for whiteboard crutches
7. Revision twin

Provenance (resume-only metrics)

Prompt	Verified hooks
A · SDUI	S3 header/search; S12 splash; optional S6 30%+ flows as UX beat

B • Networking+pin	S4 URLSession/HTTPS/pin/whitelist; S5 p50/p90; S2 path races if token cache; S4-A1 rotation as design
Ops both	S8 99.95% CFS / IMOC pause; 30L+ DAU scale context

Do **not**: invent QPS; claim S2 alone caused CFS; claim S4-A1 runbook shipped; claim SecKey bytes are SPKI.

Agenda opener (memorize)

"I'll spend ~5 minutes on scope and scale, then a full architecture pass, then deep-dive X and Y — does that match what you want?"

01 — Foundations: 45-Minute Mobile System Design

0. North star

Timebox ruthlessly: clarify, draw a readable HLD, define API/data, deep-dive 2–3 hard subsystems, close on ops — grounded in real production proof.

1. Universal spine (memorize)

0–5 min	→ CLARIFY:	Scope (in/out), scale (DAU), offline?, platform?
5–15 min	→ HLD:	4-layer diagram + data flow
15–25 min	→ DATA/API:	Entities, endpoints, pagination/versioning
25–40 min	→ DEEP DIVE:	2–3 hardest subsystems (you choose)
40–45 min	→ OPS:	Failures, metrics, rollout, flags, pause criteria

Staff/EM opener:

"I'll spend ~5 minutes on scope and scale, then a full architecture pass, then deep-dive X and Y — does that match what you want?"

2. Clarify checklist (resume-true numbers only)

Ask	Your honest anchor
Platform	iOS (SwiftUI/UIKit as relevant)
Scale	30+ lakh DAU when BMS-like
Reliability	99.95%+ CFS as ops constraint when rollout discussed
Offline?	Ask — don't assume
In/out	Cut web admin, Android, ML ranking unless asked

Trap: Invent precise QPS. If forced to estimate, label assumptions transparently.

3. Pick one primary prompt (coin flip live)

Prompt A — SDUI engine

HLD layers: CMS/config → API → schema parser → component registry → renderer → actions/analytics → fallbacks/cache

Strong deep dives (pick 2–3):

- 1. Schema versioning + unknown component fallback
- 2. Action routing / navigation (Day 20)
- 3. Caching & freshness of layout JSON
- 4. Performance — parse budget, images, p50/p90 (Day 17)
- 5. Security — allowlisted actions (no arbitrary script)

Agenda script (SDUI):

“Scope an iOS SDUI engine for CMS-driven home surfaces at ~30L DAU. I’ll do HLD, schema/API, then deep-dive versioning/fallbacks and action routing, and close on rollout metrics.”

Prompt B — Networking + SSL pinning

HLD layers: Features → API client → interceptors → URLSession → pin/trust → cache → offline/reachability

Strong deep dives (pick 2–3):

- 1. Auth refresh single-flight + 401
- 2. SPKI pinning + rotation/break-glass **design (S4-A1)**
- 3. Caching / idempotency for POSTs
- 4. Observability — journey p50/p90 (S5)
- 5. Modular Ads client ownership (S4)

Agenda script (Networking):

“Scope a networking layer with auth and SSL pinning for a high-traffic iOS app. I’ll do HLD, API/error model, then deep-dive token refresh and pin rotation, and close on observability and failure modes.”

Full spoken walkthroughs: [02-deep-dive.md](#) .

4. Ops closer (both prompts — last 5 min)

Always hit:

- Failure modes (schema fail / pin fail / CDN down)
- Metrics: CFS, journey p90, pin fail rate, SDUI fallback rate
- Rollout: flags, phased release, IMOC pause
- A/B or Remote Config gates

60s ops closer template:

“Failures I’d watch: X and Y. Metrics: crash-free, journey p90, Z rate. Rollout behind flags with phased release and pause if CFS or p90 cliffs — IMOC owns the stop.”

5. Communication habits

Habit	Why
Checkpoint each phase	Silence ≠ agreement
Cut dive short to save ops	Ops is scored
Negotiate if asked both prompts	Primary + 5-min secondary
Interfaces over code dumps	Senior signal

6. Teach-back

- 1. Spine timings

2. Two agenda scripts
3. Resume-only metrics discipline
4. Ops never optional

Next: full scripts in [02-deep-dive.md](#) .

02 — Deep Dive: Full 45-Min Scripts (SDUI OR Networking+Pinning)

*Both scripts are fully written. **Live mock: pick ONE.** Skim the other after.
Speak naturally — these are target narratives, not teleprompter robots.*

PROMPT A — SDUI Engine (full script)

A0. Clarify (0–5)

Say:

"I'll design an iOS server-driven UI engine for CMS-driven home surfaces — header, splash-style screens, modular sections — not a full web CMS admin and not Android unless you want parity notes. Scale context from my work: consumer apps around thirty-plus lakh DAU. I'll assume online-first with last-good cache unless you want offline-first. Out of scope unless you pull me in: ML ranking, pixel-perfect design-tool export, and arbitrary script execution on device. Deep dives I'll prioritise: schema versioning with unknown-component fallback, and action routing. Ops at the end with flags and crash-free pause criteria. Does that match?"

Listen / adjust offline, in/out, which deep dives.

A1. HLD (5–15)

Draw & narrate:

```
CMS / Config service
  ↓
Layout API (versioned JSON)
  ↓
iOS: Network client (pinned if config is sensitive)
  ↓
Schema parser → ComponentRegistry → Renderer (UIKit/SwiftUI)
  ↓           ↓
Fallback/cache ActionHandler → DeepLinkRouter / API / analytics
```

Say:

"CMS authors layouts as a component tree — type, props, optional action, analytics map. The client owns the design system pixels. Parser validates version. Registry maps type strings to native components; unknown types become empty or placeholder with a non-fatal metric — never crash. Actions go through a central handler with an allowlist — deeplink, api_call, open_web — no eval. FallbackEngine serves last-good layout from disk when network fails. I've shipped protocol-driven backend headers and search modernisation at BookMyShow, and server-driven splash on Aces — same family of problems: versioning, fallbacks, and not trusting the payload blindly."

Provenance hooks: Verified · S3 · S12 (soft). Optional S6 as "lightweight surface" beat only if time.

A2. Data / API (15–25)

Entities:

```
Layout { id, version, screenId, components: [Component], etag? }  
Component { type, id, props, children?, action?, analytics? }  
Action { type: deeplink|api_call|web, payload }
```

Endpoints (example):

```
GET /v1/layouts/{screenId}?version=clientMax  
Headers: X-Client-Schema: 3  
Response: Layout JSON (<~50KB gzip target mindset)
```

Say:

"Client sends its max supported schema version. Server may strip unknown newer components for older clients. Pagination belongs inside list components via cursor fetch_more — not usually the whole screen. I'll keep payloads lean; parse belongs off the critical path if heavy, with a frame-budget mindset so we don't hitch."

A3. Deep dive 1 — Versioning + unknown component (25–33)

Say:

"Two version axes: layout content version and schema major. If major is unsupported, I serve disk cache or prompt force-update depending on product. Unknown component types must not crash — EmptyView or placeholder, log non-fatal, metric sdui_unknown_type . That's how you keep crash-free while CMS ships experiments. Backend can also strip components the client can't render. The failure mode I refuse is 'JSON decode fails entire home to white screen.'"

A4. Deep dive 2 — Action routing (33–40)

Say:

"Actions are data, not code. Allowlisted types only. Deeplink actions enter the same DeepLinkRouter as Universal Links and push — one table — so CMS can't invent a second navigation world. Sensitive routes still auth-gate. Analytics maps on components let PM change event names without an app release. Kill switch via remote config disables SDUI surfaces and falls back to native scaffolding."

A5. Ops (40–45)

Say:

"Failure modes: schema incompatibility, CDN/API down, poison layout, unknown-type spikes. Metrics: layout fetch p50/p90, fallback hit rate, unknown-type rate, crash-free sessions — we sustain ninety-nine point nine five plus at large DAU as an ops bar — and journey traces on home. Rollout: flag the new renderer, phased release, pause if CFS or p90 cliffs. IMOC owns stop criteria during peak traffic."

Checkpoint: "Happy to go deeper on caching or image pipeline if you want."

PROMPT B — Networking + SSL Pinning (full script)

B0. Clarify (0–5)

Say:

"I'll design a first-party iOS networking layer with auth and SSL pinning for a high-traffic consumer app — think ads or checkout-adjacent APIs. Scale: thirty-plus lakh DAU context. Out of scope unless asked: full backend service mesh, Android parity. Deep dives: single-flight token refresh, and SPKI pinning with rotation as design. I'll close on observability with p50/p90 and failure modes. Match?"

B1. HLD (5–15)

Draw & narrate:

```
Feature modules
  ↓ (protocols)
APIClient (endpoint → build → intercept → execute → decode → map errors)
  ↓
Interceptors: Auth, Retry, Tracing
  ↓
URLSession + Delegate (server trust / SPKI pin) + Domain allowlist
  ↓
URLCache / App cache | Token store (Keychain) | Reachability / offline queue
```

Say:

"I prefer URLSession over Alamofire when we need ownership of trust challenges — that's exactly what we did on BookMyShow Ads: migrated to URLSession with HTTPS, SSL pinning, and domain whitelisting on a revenue-critical module. Features never see Alamofire types. Errors become a typed NetworkError. Tokens live in Keychain, not UserDefaults."

Provenance: Verified · S4

B2. Data / API (15–25)

Say:

"APIEndpoint protocol with path, method, headers, body, requiresAuth, associated Response: Decodable. Client exposes request(_:) async throws. HTTP 401 triggers refresh coordinator. Retry only transient 408/429/5xx on idempotent GETs with backoff and jitter — never blind retry charge POSTs without idempotency keys. Tracing uses URL templates to protect cardinality."

B3. Deep dive 1 — Single-flight refresh (25–33)

Say:

"N parallel 401s must not start N refreshes. An actor owns refresh: first caller starts; others await the same result; success updates Keychain and retries once; failure fans out to logout. I don't mutate actor state inside an unstructured Task body. If we discuss shared token maps historically, we serialised dictionary access with GCD on that path — today I'd evaluate an actor for greenfield — and I won't claim that alone owned crash-free."

Provenance: Learning-lab refresh · Verified · S2 bounded · not sole CFS

B4. Deep dive 2 — SPKI pinning + rotation design (33–40)

Say:

"Pinning happens in the URLSession server-trust challenge. I pin SHA-256 of the certificate's Subject Public Key Info DER — true SPKI — not SecKeyCopyExternalRepresentation raw bytes, which is a common mistake. Domain allowlist limits hosts. For Ads we shipped pinning; for rotation I insist on backup pins, ship client before server key rotate, staged exposure, and a monitored break-glass — that's design judgment, not me claiming I shipped the full ops runbook. Sensitive hosts fail closed on mismatch."

Provenance: Verified · S4 · How I would apply it · S4-A1 · SPKI correctness

B5. Ops (40–45)

Say:

“Failures: pin mismatch outage, refresh stampede, poison cache, TLS middleboxes. Metrics: journey p50/p90 from Firebase Performance style traces — listing, checkout, search is what we instrumented — pin failure rate, 401/refresh rate, crash-free. Rollout: flag networking changes, phased release, pause on CFS or p90 regressions. Pin emergencies use break-glass design carefully — never silent forever-off.”

Provenance: Verified · S5 · S8 ops culture

Shared trade-offs (use live)

Choice	When	Cost
SDUI for header/splash	Fast content iteration	Schema + fallbacks
Native-only UI	Unique interactive UX	App release per change
Hard-fail pinning	Sensitive APIs	Outage if rotation weak
Soft-fail pinning	Low sensitivity	Silent security regression
Cursor pagination	Feeds	Slightly harder clients
Journey traces	Product SLIs	Need clear start/stop

Anti-patterns (both)

Don't	Do
Draw buttons for 20 minutes	Park UI; return to schema/reliability
Invent QPS	DAU + labeled estimates
Skip ops	Force last 5 minutes
Claim S4-A1 runbook shipped	Label design
Claim S2 = 99.95% CFS	Path contribution only
Hash SecKey as SPKI	SPKI DER
End on class diagram	Failures + SLIs + pause

03 — Production Bridge: Which Story for Which Prompt

1. Prompt A — SDUI

Story	Use
S3	Backend-driven header; search debounce/MVVM — primary SDUI proof
S12	Server-driven splash / cold-start product surface
S6	Optional: lightweight LE sheet for 30%+ flows — product UX, not whole engine
S3-A1	Applied: schema versioning + unknown-component fallback design

≤20s grounding line:

"I'll ground this in patterns I've shipped — protocol-driven backend header/search and server-driven splash — then generalise the engine cleanly."

2. Prompt B — Networking + pinning

Story	Use
S4	Alamofire → URLSession; HTTPS; pinning; domain whitelist
S4-A1	Rotation/backup/break-glass as design
S5	Firebase Performance p50/p90 journeys
S2	Only if token/shared-map races — path-scoped , not sole CFS

≤20s grounding line:

"I'll ground this in Ads URLSession ownership with HTTPS, pinning, and allowlisting — plus percentile latency culture — then generalise the client."

3. Ops vocabulary (both)

- **30L+ DAU** scale context
- **99.95%+ CFS** + IMOC pause (S8)
- Journey **p50/p90** (S5)
- Never fabricate QPS or drop-off %

4. Forbidden mash-ups

Slip	Fix
S2 caused CFS	Contributor only
S4-A1 "we shipped runbook"	Design label
SecKey = SPKI	DER hash
Averages only in ops	Percentiles

04 — Questions (two-layer) + Two-Layer Scoring

Part I = warm-up interview Qs (Answer points + Full spoken).

*Part II = **scoring-layer** questions — how you grade a mock (also two-layer).*

*Totals: **12 normal** + **8 tricky** warm-up · **plus** scoring Q set.*

Part I — Warm-up (Normal)

Q1. How do you start any mobile SD interview? (30–45s)

Answer points: Agenda → clarify scope/scale/offline → confirm.

Full spoken answer:

"I propose a timed agenda — clarify, HLD, API, two deep dives, ops — confirm it matches what they want, then ask scale, offline, and explicit out-of-scope. Drawing silently without a plan is how seniors look junior."

Provenance: Learning-lab · cheatsheet spine

Q2. What scale number do you use? (30–45s)

Answer points: 30L+ DAU from resume when BMS-like; don't invent QPS.

Full spoken answer:

"For BMS-like consumer systems I use thirty-plus lakh DAU from my resume. If they demand QPS I derive a labeled estimate from DAU and session assumptions — I never invent a precise fake QPS as fact."

Provenance: Verified · S8 scale context

Q3. Name four HLD layers for SDUI. (30–45s)

Answer points: CMS → delivery API → parser/registry → renderer/actions.

Full spoken answer:

"CMS or config, a versioned layout API, a client parser plus component registry, and a native renderer with a central action handler and fallbacks. Analytics hooks live on components; unknown types never crash."

Provenance: Verified · S3 · family

Q4. Unknown component type — behavior? (30–45s)

Answer points: Skip/placeholder; metric; never crash; version strategy.

Full spoken answer:

"Render empty or placeholder, emit a non-fatal metric, continue the tree. Schema versioning and server-side stripping protect older clients. Crashing home on an unknown CMS experiment is unacceptable under a high crash-free bar."

Provenance: How I would apply it · S3-A1

Q5. Where does pinning sit in a networking HLD? (30–45s)

Answer points: Trust evaluation in session delegate; allowlisted domains; SPKI DER.

Full spoken answer:

"Beneath the API client, inside URLSession's server-trust challenge. I compare SHA-256 of SPKI DER to embedded pins for allowlisted hosts — not raw SecKey export bytes. On Ads we owned that path after moving to URLSession."

Provenance: Verified · S4 · SPKI correctness

Q6. Which deep dives for networking mock? (30–45s)

Answer points: Refresh single-flight + pinning/rotation design + cache/observability.

Full spoken answer:

"I'd pick single-flight refresh and SPKI pinning with rotation as design, and if time allows caching or journey p50/p90 observability. Equal time on every box looks shallow — seniors choose hard subsystems."

Provenance: Learning-lab · S4/S5

Q7. Ops metrics at the end? (30–45s)

Answer points: CFS 99.95%+, journey p50/p90, pin fail / fallback rate, rollout %.

Full spoken answer:

"Crash-free as a reliability bar — ninety-nine point nine five plus at large DAU — journey p50/p90, pin-failure or SDUI fallback rates, and phased rollout percentage with pause ownership. Averages alone don't close a senior mock."

Q8. How do modules appear in the diagram? (30–45s)

Answer points: Features talk to Network/SDUI core via protocols; App wires DI.

Full spoken answer:

"Feature modules depend on protocols — LayoutProviding, APIClient — and the app composition root wires implementations. That keeps Ads pinning ownership and SDUI core replaceable without a monolith import graph."

Provenance: Learning-lab · Day 15 mindset

Q9. When to deep-dive images in SDUI? (30–45s)

Answer points: Media-heavy layouts; else mention pipeline and move on.

Full spoken answer:

"If layouts are media-heavy I deep-dive downsample and off-main decode. Otherwise I name the image pipeline once and spend dive time on versioning and actions — timeboxed judgment."

Provenance: Learning-lab · Day 16

Q10. Offline for SDUI? (30–45s)

Answer points: Ask in clarify; cache last-good; stale policy; don't brick first launch.

Full spoken answer:

"I ask in clarify. Default I'd cache last-good layout with TTL and stale-while-revalidate, and ensure first launch has a native scaffold if cache is empty. Splash can be server-driven for freshness without blocking forever."

Provenance: Verified · S12 · soft

Q11. Idempotency — when? (30–45s)

Answer points: POST payments/bookings; Idempotency-Key; don't blind retry.

Full spoken answer:

"Any charge or booking POST needs idempotency keys and status polling discipline — the networking layer must not blindly retry. UX can still show a processing popup while status is uncertain."

Provenance: Verified · S7 · intent (no fake %)

Q12. Avoid turning SD into coding? (30–45s)

Answer points: Interfaces + data flow first; snippets only for critical APIs.

Full spoken answer:

"I stay on interfaces, data flow, and failure modes. I only sketch code for critical seams — pin challenge, schema enum, refresh actor — and I don't implement table view cells in a design interview."

Provenance: Learning-lab

Part I — Warm-up (Tricky)

T1. Interviewer pulls you into pixel UI. (90–120s)

Answer points: Park pixels; one component example; return to schema/reliability/rollout; timebox 5 min if insisted.

Full spoken answer:

"I'll park pixel layout, give one component example, and return to schema, reliability, and rollout — those are the senior signals. If they insist on UI I'll timebox five minutes then resume the agenda."

T2. "What's your QPS?" you never measured. (90–120s)

Answer points: Don't invent precise QPS; offer DAU; labeled estimate from sessions if needed; refocus on client arch.

Full spoken answer:

"I don't invent precise QPS. I offer DAU context and, if needed, a transparent estimate from sessions and think time, labeled as an estimate. Then I refocus on client architecture unless they truly want capacity planning."

T3. They ask both SDUI and pinning in one 45. (90–120s)

Answer points: Primary + 5-min secondary; or SDUI with security dive (allowlisted actions + pinned config); don't boil both oceans.

Full spoken answer:

"I'll propose a primary system and a five-minute secondary, or SDUI with a security deep-dive on allowlisted actions plus a pinned config API. Boiling both oceans fails ops and depth."

T4. Pinning deep dive becomes cert lecture. (90–120s)

Answer points: SPKI vs leaf; rotation as design; failure metrics; Ads ownership; stop — no TLS history dump.

Full spoken answer:

"I stay on SPKI versus leaf, rotation timeline as design, failure metrics, and Ads ownership — then stop. TLS history dumps burn the clock."

T5. SDUI that runs arbitrary native code paths. (90–120s)

Answer points: Allowlisted components/actions only; versioned schema; kill switch; no script eval.

Full spoken answer:

"Allowlisted components and actions only, versioned schema, kill switch, no script eval. A Turing-complete CMS client is a security incident waiting to happen."

T6. You forget ops in last 5 minutes. (90–120s)

Answer points: 60s closer: failures, CFS/p90, flags, phased, IMOC pause; cut dive rather than skip ops.

Full spoken answer:

"I practice a sixty-second closer: failures, CFS and p90, flags, phased rollout, IMOC pause — and I cut the dive short rather than skip ops. Ops is scored."

T7. "Why not just WebView for SDUI?" (90–120s)

Answer points: WebView = fast iterate; trade-offs perf/offline/UX/security; hybrid OK some surfaces; native for core chrome; state trade-off.

Full spoken answer:

"WebView iterates fast but trades off performance, offline, platform UX, and security surface. Hybrid can work for some surfaces; native renderer fits core ticketing chrome. I state the trade-off, not a religion."

T8. Mock partner is silent. (90–120s)

Answer points: Checkpoint every phase; silence ≠ agreement; ask rather than monologue past misunderstanding.

Full spoken answer:

"I checkpoint every phase — 'OK to deep-dive refresh next?' — silence isn't agreement. I'd rather ask than monologue past a misunderstanding."

Part II — Two-layer scoring questions

*Use these to **grade yourself** after the live mock. Same shape: Answer points + Full spoken (as if explaining the rubric to a peer coach).*

Scoring Layer A — Structure (how you used the 45 minutes)

SA1. What does "excellent agenda & clarify" look like? (45s)

Answer points: States plan; asks scale/offline/in-out; confirms with interviewer; ≤5 min.

Full spoken answer:

"Excellent means you said the plan aloud, asked scale and offline and cut scope explicitly, and got a yes before drawing. Weak is silent boxing or clarifying for fifteen minutes."

Rubric pts: 15 / 100

SA2. What fails HLD clarity? (45s)

Answer points: Mystery boxes; no data-flow arrows; unreadable layers.

Full spoken answer:

"If a peer can't narrate request flow from CMS or client to UI, the HLD failed — regardless of how many buzzwords you listed. Arrows and layers beat ornament."

Rubric pts: 20 / 100

SA3. Why is ops weighted even if small minutes? (45s)

Answer points: Senior judgment; pause criteria; metrics; last 5 min used.

Full spoken answer:

"Ops shows you ship. Ten points isn't large, but a zero on ops — no metrics, no pause — fails the pass bar even with pretty boxes. I force the last five minutes."

Rubric pts: 10 / 100 · pass needs ops ≥6/10

Scoring Layer B — Content honesty (what you claimed)

SB1. How do you score production grounding? (45s)

Answer points: Resume-true hooks; no invented metrics; correct S4 vs S4-A1 / S2 vs S8 labels.

Full spoken answer:

"Full points if SDUI cites S3/S12-style proof or networking cites S4/S5 without fake QPS, and if rotation is labeled design when it is design. Invented metrics or 'S2 caused CFS' caps this dimension low."

Rubric pts: 10 / 100

SB2. Deep dive quality — what earns 25? (45s)

Answer points: 2–3 hard subsystems; trade-offs; failure modes inside dives.

Full spoken answer:

“Excellent dives pick hard seams — unknown components, refresh single-flight, SPKI rotation design — and include failure modes inside the dive, not only happy path. Listing six shallow topics scores poorly.”

Rubric pts: 25 / 100

SB3. Two-layer scoring — how do you combine? (60s)

Answer points: Structure layer + content layer; pass ≥ 70 with ops ≥ 6 and no fabricated metrics.

Full spoken answer:

“I score structure — agenda, HLD, API, dives, ops, communication — and content honesty — provenance and technical correctness like SPKI DER — as two layers. Pass Mock #3 at seventy plus with ops at least six out of ten and zero fabricated metrics. A beautiful diagram with fake QPS and no ops still fails.”

Full rubric (100 pts)

Dimension	Points	Excellent
Agenda & clarify	15	Plan + scale/offline/in-out + confirm
HLD clarity	20	Layered diagram + data flow
API / data model	15	Entities + key endpoints + version/pagination
Deep dive quality	25	2–3 hard subsystems + trade-offs + failures
Production grounding	10	Resume-true; correct labels
Ops / rollout	10	Metrics, flags, phased, pause — last 5 used
Communication	5	Checkpoints, timeboxes, scope cuts

Pass bar: ≥ 70 with ops $\geq 6/10$ and no fabricated metrics.

Timed warm-up set

Q1, Q4, Q5 + T3, T6 — then run the full 45-min mock once.

05 — Exercises · Live Mock #3

Exercise 1 — Spine recite (5 min)

Clarify 5 / HLD 10 / API 10 / Dive 15 / Ops 5 — with both agenda openers (SDUI + Networking).

Exercise 2 — Script rehearsal (30–40 min)

Read aloud **one** full script from [02-deep-dive.md](#) while drawing on paper.

Do **not** look at notes in the live timed mock later.

Exercise 3 — Interfaces skim (10 min)

Read [code/SDUISketch.swift](#) and [code/NetworkPinSketch.swift](#) — whiteboard crutches only.

Exercise 4 — Live mock (REQUIRED) (45 min)

1. Coin flip Prompt A or B.
2. Timer 45:00. Blank paper only.
3. Record audio + photo of diagram.
4. Force ops in final 5:00.
5. Score with Part II rubric in [04-questions.md](#) (be harsh).

Exercise 5 — Lightning alternate (20 min)

Other prompt: clarify + HLD + one dive + ops only.

After-action

- Top 3 misses → flashcards / Week 4 Day 27
- Self-review: p50/p90 said? rotation labeled design? unknown component? fabricated metrics?
- Rest voice

Pass checklist

- ☐ ≥70 total
- ☐ Ops ≥6/10
- ☐ No fabricated metrics
- ☐ S2 not sole-CFS; S4-A1 not “shipped runbook”; SPKI ≠ SecKey bytes

Week 04

Day 22 — DSA Trees: BFS / DFS Patterns

Week 4 · Full study (self-contained) · ~4–5 hrs

Revision twin: [revision/weeks/week-04/day-22.md](#)

Outcomes

By end of day, without notes, you can:

- Choose **BFS vs DFS** from the prompt trigger (levels / unweighted shortest vs path / subtree / LCA) and state **O(n)** time with **O(h)** or **O(w)** space in one breath
- Code the **level-size BFS** idiom and the four DFS shapes (pre / in / post / “recurse children”) in Swift
- Open every tree problem with a **2–3 min “say this first”** script before typing
- Solve **8–12** medium tree problems from the track list while narrating complexity *before* implementation
- Map traversal patterns to production mental models (SDUI node tree, view hierarchy, deeplink routes) **without inventing LeetCode metrics**

How to study (in Cursor only)

1. [01-foundations.md](#) — mental model + glossary + first walk-throughs
2. [02-deep-dive.md](#) — pattern catalog, skeletons, traps, complexity scripts
3. [03-production-bridge.md](#) — Verified hooks (S3 / S10 / S13) + interview lines
4. [code/](#) — worked Swift solutions (BFS, DFS, BST, diameter, LCA, serialize)
5. [04-questions.md](#) — cover **Full spoken answer**; speak from **Answer points**; compare
6. [05-exercises.md](#) — problem set + timed drills
7. Revision twin for flashcards / timed recall day-of

Module map

Module	File	Role
Foundations	01-foundations.md	Intern → mid mental model
Deep dive	02-deep-dive.md	Pattern triggers, skeletons, trade-offs
Production bridge	03-production-bridge.md	SDUI / nav / SDK trees — honest mapping
Questions	04-questions.md	Two-layer Q&A (normal + tricky)
Exercises	05-exercises.md	Solve log + speaking drills
Code	code/	Runnable interview-style Swift

Time budget (suggested)

Block	Minutes
Foundations	45–60
Deep dive + skim code	75–90
Production bridge	20–30
Questions (speak 6–8 aloud)	60–75
Exercises (solve + agenda memos)	90–120

Provenance reminder

Only use Verified IDs from [../..../provenance/README.md](#) .

Label	Use for today
Verified · S3	Backend-driven header / search — nested CMS / SDUI <i>shape</i>
Verified · S10	Stories SDK — component tree boundaries
Verified · S13	Deeplink + hybrid navigation hierarchy
Learning-lab	All LeetCode Swift in code/ — interview practice, not shipped product

Do **not** invent fill-rate %, latency ms, or “I solved N trees in prod.” Resume metrics stay for behavioral days: **30L+ DAU, 99.95% crash-free, 30%+ nav reduction**.

Agenda opener (pin this)

“Clarify binary vs BST, path vs bool, mutate OK? Brute $O(n^2)$ if needed. Pattern is BFS level-size / DFS postorder / bounds. Time $O(n)$, space $O(h)$ or $O(w)$. Edges: null, skewed, duplicates. Coding now.”

01 — Foundations: Tree BFS / DFS

Read this first. Goal: an intern can teach back when to BFS vs DFS, then you layer the senior edges (complexity honesty, BST traps, production analogies).

0. One-sentence north star

BFS walks by distance (levels / width); DFS walks by structure (paths / subtrees / combine-after-children). Pick from the prompt trigger, then state O(n) time and O(h) or O(w) space before you type.

Everything below unpacks that sentence.

1. Mental model: what a tree problem is asking

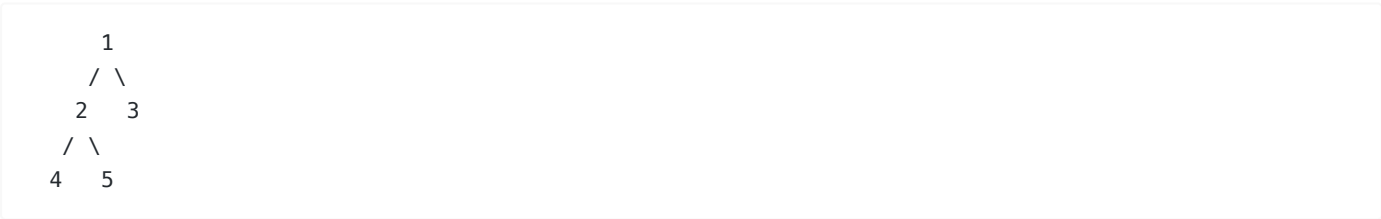
1.1 Two boxes in your head

Box	Meaning	Interview trigger words
Level / distance box	Process nodes in waves from the root	"level by level", "leftmost at each depth", "zigzag", "right side view", "shortest path" (unweighted)
Structure / path box	Recurse into children; combine results on the way up or accumulate on the way down	"path sum", "subtree", "same tree", "symmetric", "LCA", "diameter", "validate BST", "serialize"

BFS:	root → [level 0] → [level 1] → [level 2] ...	(queue / FIFO)
DFS:	root → deep left ... backtrack → siblings ...	(stack / recursion)

1.2 The intern demo (say aloud once)

Draw this tree:



- **BFS order (visit):** 1, then 2–3, then 4–5 → levels `[[1], [2,3], [4,5]]`
- **DFS preorder:** visit, left, right → `1,2,4,5,3`
- **DFS inorder:** left, visit, right → `4,2,5,1,3` (BST sorted if it were a BST)
- **DFS postorder:** left, right, visit → `4,5,2,3,1`

Say aloud: "BFS answers width questions. DFS answers structure and path questions. Inorder is special for BSTs."

1.3 Binary tree vs BST (do not blur)

Type	Extra invariant	Extra algorithms unlocked
Binary tree	Each node ≤ 2 children	BFS/DFS structural only
BST	Left < node < right (clarify duplicates)	Bounds validate, inorder sorted, BST LCA walk O(h), k-th smallest via inorder

Trap: Using "BST walk" LCA on a plain binary tree — wrong.

2. Glossary (interview vocabulary)

Term	Crisp meaning
Height / depth	Confirm with interviewer: edges vs nodes; "max depth" usually nodes along longest root→leaf

Level-size loop	Snapshot <code>queue.count</code> each wave so you don't mix levels
Preorder	Process node before children — path building, serialize prefixes
Inorder	Left → node → right — BST sorted order
Postorder	Children first — height, diameter, LCA combine
h	Height (skewed → n ; balanced → $\sim \log n$)
w	Max width of a level (can be $\mathcal{O}(n)$ in a fat tree)
Morris	Threaded $\mathcal{O}(1)$ aux inorder — mutate links temporarily; mention only if solid

3. Complexity script (memorize verbatim)

" n = number of nodes. Visit each once → $\mathcal{O}(n)$ time. Recursion depth or queue holds up to $\mathcal{O}(h)$ or $\mathcal{O}(w)$ space; worst case skewed tree $\mathcal{O}(n)$, balanced $\mathcal{O}(\log n)$ height."

Never say " $\mathcal{O}(1)$ space" for recursive DFS unless you explicitly discuss the call stack. Prefer: "auxiliary $\mathcal{O}(1)$ besides the call stack $\mathcal{O}(h)$."

4. BFS foundation — level-size idiom

Always separate "nodes currently in the queue" from "this level":

```
queue = [root]
while queue not empty:
    levelCount = queue.count
    for _ in 0..<levelCount:
        node = dequeue
        // process / collect
        enqueue children if non-nil
```

Why interviewers care: Without `levelCount`, zigzag / right-side-view / level averages become messy index math.

Edge cases to name: empty root → `[]`; single node; skewed left chain (width 1, height n); very wide bushy level.

5. DFS foundation — three return styles

5.1 Void / side-effect DFS

Build a path array, push before recurse, pop after (backtracking). Classic Path Sum II.

5.2 Return a value DFS

Height, boolean "found", optional node pointer. Classic max depth, same tree, LCA.

5.3 Return + global update

Diameter: return height; update `maxDiameter` with `leftH + rightH` at every node.

Say aloud: "I decide up front: am I accumulating down, combining up, or both?"

6. First worked micro-examples (no full solution yet)

6.1 Maximum depth — DFS

```
depth(null) = 0
depth(node) = 1 + max(depth(left), depth(right))
```

BFS alternative: count waves. Same $O(n)$; space trade-off h vs w .

6.2 Level order — BFS

Collect one array per wave. This is the template for zigzag (reverse odd levels) and right side view (last node of each wave).

6.3 Validate BST — bounds (preview)

Parent-only check fails on:

```
    5
   / \
  1   6
   / \
  4   7    ← 4 is wrong: still in right subtree of 5
```

Need low/high bounds (or inorder strictly increasing).

7. "Say this first" — 2–3 min coding approach

Aligns with [answer-timing-guide.md](#).

1. **Clarify (20–30s)**: Binary vs BST? Path or bool? Mutate OK? n up to?
2. **Brute (20–30s)**: Enumerate all paths / compare every subtree — often $O(n^2)$.
3. **Optimize (45–60s)**: Name pattern + why correct.
4. **Complexity (15s)**: $O(n)$ time; $O(h)$ or $O(w)$ space; worst case.
5. **Edges (20s)**: null, one child, skewed, duplicates.
6. **Commit (5s)**: "I'll code the optimized version now."

Example — Level Order:

"I'll BFS with a queue and process by level size so each result array is one depth. Empty root returns []. Time $O(n)$, space $O(w)$. Edges: single node, skewed chain. Coding now."

Example — LCA (binary tree):

"Postorder DFS: recurse left/right; if both sides find targets, current is LCA; if one side finds, bubble that up. Confirm both nodes exist. $O(n)/O(h)$."

Example — Validate BST:

"Not enough to compare only to parent. I'll DFS with low-high bounds, or inorder strictly increasing. $O(n)/O(h)$."

8. Swift node shape (interview default)

```
final class TreeNode {
    var val: Int
    var left: TreeNode?
    var right: TreeNode?
    init(_ val: Int, _ left: TreeNode? = nil, _ right: TreeNode? = nil) {
        self.val = val; self.left = left; self.right = right
    }
}
```

Interview aside (10s max): Production SDUI trees are often value/ `Codable` enums; LeetCode uses reference nodes. Same *traversal* ideas; different ownership model. Don't derail into ARC unless asked (Day 03 territory).

9. Checklist before you open Deep Dive

- ☐ I can draw BFS vs DFS on a 5-node tree
- ☐ I can recite the complexity script without "O(1) recursive"
- ☐ I know BST ≠ binary tree for LCA / validate
- ☐ I can speak the 6-step "say this first" agenda
- ☐ I know `level-size = for _ in 0..queue.count`

→ Continue: [02-deep-dive.md](#) ''')

02 — Deep Dive: Tree Pattern Catalog & Skeletons

Mechanics, traps, trade-offs. Self-contained — you do not need external blogs to interview on trees today.

1. Pattern catalog (trigger → algorithm)

Trigger phrase in prompt	Default pattern	Why
"Level by level", "leftmost/rightmost at each depth", "zigzag level order"	BFS + queue	Natural FIFO; O(w) space for width
"Shortest path" in unweighted tree/graph	BFS	First hit = shortest
"Path sum", "root-to-leaf", "all paths"	DFS preorder + accumulator / backtrack	Path state rides the stack
"Subtree of another", "same tree", "symmetric"	DFS compare two pointers / mirrored	Structural recursion
"LCA of two nodes" (binary tree)	DFS postorder return markers	Bottom-up combine
"LCA" on BST	Walk comparing values	O(h); don't use full tree scan unless asked
"Serialize / deserialize", "clone N-ary"	DFS or BFS with null markers	Encoding must be reversible
"Max depth / diameter / balanced?"	DFS return height + side effects	One pass, multiple facts
"Validate BST"	DFS low/high bounds or inorder	Parent-only check is wrong
"k-th smallest in BST"	Inorder (stack or Morris)	Sorted order
"Connect next pointers / cousins"	BFS level-size	Sibling awareness
"Right / left side view"	BFS last/first in level or DFS prioritize side	Same answer, different space profile

2. BFS skeleton (production-quality interview form)

```
func levelOrder(_ root: TreeNode?) -> [[Int]] {
    guard let root else { return [] }
    var result: [[Int]] = []
    var queue: [TreeNode] = [root]
```

```

while !queue.isEmpty {
    let levelCount = queue.count
    var level: [Int] = []
    level.reserveCapacity(levelCount)
    for _ in 0..

```

Senior note: `removeFirst()` on `Array` is $O(n)$. In interviews, clarity usually wins; if interviewer cares, say “I’d use a true deque / index head.” See `code/TreeBFS.swift`.

Zigzag variant

Same BFS; on odd levels `level.reverse()` (or insert at front / use deque). Prefer reverse of a level array — fewer bugs than index gymnastics.

Right side view

Per level, append `level.last` (or track last dequeued). DFS alternative: recurse right before left, record first visit per depth.

3. DFS skeletons

3.1 Recursive template

```

func dfs(node):
    if node == nil: return base
    // pre: act before children
    L = dfs(node.left)
    // in: act between (BST / inorder)
    R = dfs(node.right)
    // post: combine L,R with node
    return combined

```

3.2 Iterative preorder

Push root; while stack: pop, visit, **push right then left** (so left processed first).

3.3 Iterative inorder

Walk left pushing; pop/visit; go right — BST k-th / validate helper.

When iterative wins: “I need explicit control / avoid deep recursion on skewed input” or follow-up asks for stack form. Morris only if you can finish link/restore cleanly.

4. Worked pattern deep-dives

4.1 Diameter — why root-only fails

Diameter = max over **all** nodes of `leftHeight + rightHeight` (edges) or that sum + 1 (nodes) — **confirm definition**.

```

At each node:
    leftH = height(left); rightH = height(right)
    best = max(best, leftH + rightH)
    return 1 + max(leftH, rightH)    // height in nodes

```

A long path entirely in the left subtree never touches the root — hence root-only `depth(L)+depth(R)` is insufficient if you only compute it once at root *without* scanning all nodes... Actually computing at root alone misses diameters that don't pass through root. You must update at every node.

4.2 LCA binary tree — postorder markers

```

if node == nil or node == p or node == q: return node
L = lca(left); R = lca(right)
if L != nil and R != nil: return node    // split here
return L ?? R

```

Assumptions to clarify: Both nodes exist in tree? Nodes are unique? Parent pointers available?

BST LCA: If both vals < node, go left; both > node, go right; else node is LCA. $O(h)$.

4.3 Validate BST — bounds

```

func ok(node, low, high): // low/high exclusive bounds as Optionals
    if node == nil: return true
    if node.val <= low or node.val >= high: return false // adjust for duplicate policy
    return ok(left, low, node.val) and ok(right, node.val, high)

```

Or inorder: previous value must be strictly less than current.

4.4 Path Sum II — backtracking

Push `node.val` ; if leaf and remaining == val, copy path to answer; recurse left/right with `remaining - val` ; pop.

4.5 Serialize — null markers required

Preorder values alone are ambiguous. Encode nulls (`"#"` / `"null"`) or use BFS with null children like LeetCode Codec. Deserialize with queue or index pointer. $O(n)$.

Production nod (10s): SDUI JSON needs `schemaVersion` + unknown-type fallback — same *reversibility* / *versioning* instinct, different format (S3 / Day 25 Brief B).

4.6 Symmetric tree

Mirror DFS: `same(a.left, b.right) && same(a.right, b.left)` , or BFS pairing queue.

4.7 Construct from preorder + inorder

Preorder[0] is root; find root in inorder to split left/right sizes; recurse. Use hashmap value→index for $O(n)$ if values unique.

5. Trade-offs

Choice	When	Cost
BFS	Levels, width-aware, unweighted shortest	Queue $O(w)$; path reconstruction needs parent map
DFS recursive	Structure, path, subtree, LCA	Call stack $O(h)$; skewed → stack overflow risk in prod

DFS iterative	Same as DFS, explicit stack	More boilerplate; easier to pause/resume
Inorder on BST	Sorted queries, validate, k-th	Wrong on non-BST
Parent pointer / map	Path to root, cousin distance	Extra $O(n)$ space
Two-pass vs one-pass	Diameter	Prefer one DFS returning height + global max
Morris inorder	$O(1)$ aux space	Mutates links; easy to get wrong under timer

6. Failure modes seniors mention

1. Claiming recursive DFS is $O(1)$ space.
2. BST validate via parent-only comparison.
3. Diameter computed only at root once without per-node updates.
4. Serialize without nulls → ambiguous trees.
5. Using BST LCA walk on a general binary tree.
6. Mixing levels because you forgot `levelCount`.
7. `removeFirst` performance rabbit hole mid-interview — acknowledge, don't rewrite half the solution.
8. Silent coding — no agenda → looks junior even if code is fine.

7. Production mental models (shape only)

Domain	BFS-like	DFS-like
UIKit / SwiftUI hierarchy	Layout passes “by depth”	Hit-test / preference key fold
SDUI document	Render section rows by depth	Resolve nested actions / find component by id path
Deeplink graph	Breadth of sibling destinations	Walk nested route path
CMS nesting	Cap max depth (fail-soft)	Recurse children with budget

Honesty: These are *isomorphisms for teaching*, not “I ran LeetCode BFS in BookMyShow.” Provenance in [03-production-bridge.md](#).

8. Complexity quick reference

Problem family	Time	Space
Traversal / level order / views	$O(n)$	$O(w)$ or $O(h)$
Diameter / balanced / max depth	$O(n)$	$O(h)$
LCA binary tree	$O(n)$	$O(h)$
LCA BST	$O(h)$	$O(1)$ iterative / $O(h)$ recursion
Validate BST	$O(n)$	$O(h)$
Serialize	$O(n)$	$O(n)$ output + $O(h)/O(w)$
Construct pre+in	$O(n)$ with index map	$O(n)$
Path Sum II	$O(n^2)$ worst copies / $O(n)$ if count only	$O(h)$ path

9. Optional citations (not required to study)

- LeetCode Explore: Binary Tree cards (visual BFS/DFS)

- Apple docs: value types vs classes (aside only)
- Repo timing: [answer-timing-guide.md](#) § Coding

→ Code: `code/` · Bridge: `03-production-bridge.md` '''

03 — Production Bridge: Trees Without Fake LeetCode Stories

Map traversal shapes to shipped work. Labels from `../../..provenance/README.md`.

1. What you may say (Verified)

Verified · S3 — Backend-driven header / search (BookMyShow)

Resume anchor: Generalised protocol-driven main header; search debounce + MVVM state.

Tree-shaped truth: CMS/header documents are nested component trees. Reasoning about parent/child layout and “walk the document” is the same *shape* as DFS; “process each section depth” can feel BFS-like. Schema/versioning + fallbacks matter more than which traversal you name.

Interview line (≤20s):

“Tree BFS/DFS show up in product as view and SDUI trees — I pick BFS for level-aware layout passes and DFS when I need path or subtree decisions.”

Do not claim: “We used LeetCode level-order in the header renderer.”

Verified · S10 — Stories SDK (Raw / Miami Heat)

Resume anchor: Standalone reusable Stories SDK across portfolio apps.

Tree-shaped truth: SDK public API + internal story/page/frame hierarchy; isolation boundaries are a tree of ownership. Traversal thinking helps when debugging nested presenters — still not a DSA flex.

Interview line (≤20s):

“In the Stories SDK, nested page/frame structure forced clear ownership boundaries — same discipline as not leaking recursion past a max depth.”

Verified · S13 — SwiftUI↔UIKit + deeplinks (Raw / Grizzlies)

Resume anchor: Hybrid UI; deeplinks + navigation/lifecycle.

Tree-shaped truth: Deeplink destinations and nested coordinators form a route tree. Resolving a path is DFS-like; listing siblings at a nav level is BFS-like.

Interview line (≤20s):

“Deeplink routing is a tree walk — I clarify path vs level concerns the same way I do in interview tree problems.”

2. How I would apply it (design, not shipped DSA)

Applied idea	Honest framing
Max-depth guard on SDUI JSON	“I’d fail-soft with a placeholder after N nests — crash-free culture, not hero recursion.” Tie soft to 99.95% mindset (Verified · S8 culture), not a claim you shipped this exact guard.
Parent map for “find path to component”	Design interview only — O(n) preprocess.

Iterative walk for huge CMS trees	Prefer iterative in production if depth unbounded.
-----------------------------------	--

3. Metrics — what is and isn't allowed today

Allowed if topic drifts behavioral	Not allowed on DSA day
30L+ DAU, 99.95% crash-free, 30%+ nav (LE sheet) as <i>company context</i>	Invented "reduced tree walk latency by X%"
Qualitative: reusable SDK, hybrid nav shipped	Fake LeetCode volume as resume impact

4. STAR pointers (architecture days, not coding narration)

Need	Story
SDUI / header	story-bank.md #S3
SDK modularity	#S10
Deeplink / hybrid	#S13
Fail-soft / reliability culture (depth guards analogy)	#S8 (light touch)

Full spoken STAR lives on Day 26. Today: ≤20s hooks only while coding.

5. Bridge script (45s) if interviewer asks “when do you use this at work?”

“I don’t force interview patterns into product code for sport. Where trees appear — SDUI documents, view hierarchies, deeplink graphs — I use the same decision rule: level-aware work leans BFS-shaped; path and subtree decisions lean DFS-shaped. In production I also budget depth and prefer iterative walks when input is untrusted CMS depth, because crash-free sessions beat clever recursion.”

Provenance: Verified · S3 · S10 · S13 · learning-lab DSA in `code/`

6. Anti-patterns in production answers

1. Pretending BookMyShow search is “Two Sum.”
2. Claiming Morris traversal in a UIKit codebase.
3. Confusing District **AI tooling** (S9) with tree algorithms.
4. Skipping complexity when the “production” analogy starts — still say $O(n)/O(h)$ for the interview problem.

→ Questions: [04-questions.md](#) '''

04 — Questions (two-layer Q&A)

How to practice: Cover **Full spoken answer**. Speak from **Answer points** only. Uncover and compare.
Timing: Normal ≈ 30–45s (some 45–60s) · Tricky ≈ 90–120s.
Every question uses the same shape.

Normal questions

Q1. When do you choose BFS over DFS on a tree? (30–45s)

Answer points (frame first):

- Levels / width / unweighted shortest → BFS
- Path / subtree / LCA / structure combine → DFS
- Space: queue width vs recursion height
- One production shape hook only if natural (SDUI depth vs path resolve)

Agenda opener:

"Trigger first — levels versus structure — then space."

Full spoken answer:

"I choose BFS when the prompt is level-oriented — level order, zigzag, side view — or when I need shortest path in an unweighted tree. I choose DFS when I care about paths, subtrees, or combining children results like height, diameter, or LCA. Space-wise BFS holds a level of width w , DFS holds height h on the stack. In product terms, a depth-by-depth layout pass feels BFS-shaped; resolving a nested component path feels DFS-shaped."

Common wrong answer:

"BFS is always better" or "DFS is faster." (Same $O(n)$ visits; different access order and space profile.)

Follow-up ladder:

- **L1:** Extremely wide tree — what changes?
- **L2:** Unweighted vs weighted shortest path?
- **L3:** When would you DFS with an explicit depth parameter instead of BFS?

Provenance: Learning-lab · tree patterns; soft Verified · S3 shape

Q2. What's the time and space of tree traversal? (30–45s)

Answer points:

- Visit each node once → $O(n)$ time
- Space $O(h)$ recursion or $O(w)$ queue
- Worst $O(n)$ skewed or full-wide
- Balanced BST height $\sim O(\log n)$
- Don't call recursive DFS $O(1)$

Agenda opener:

" $O(n)$ visits — space is height or width."

Full spoken answer:

"Any full traversal is $O(n)$ time because each node is processed a constant number of times. Recursive DFS uses $O(h)$ call-stack space; BFS uses $O(w)$ for the widest level. Worst case both can be $O(n)$ — skewed chain for height, bushy level for width. On a balanced BST I'd say $O(\log n)$ height. I never call recursive DFS $O(1)$ space unless I'm explicitly excluding the stack, and interviewers usually want the stack counted."

Common wrong answer:

"BFS is $O(1)$ space" or "recursion is free."

Follow-up ladder:

- **L1:** What is h for a linked-list-shaped tree?
- **L2:** Output space for level-order arrays — count it?
- **L3:** Morris traversal trade-off?

Provenance: Learning-lab

Q3. How do you validate a BST correctly? (30–45s)

Answer points:

- Bounds (low, high) on each node **or** inorder strictly increasing
- Parent-only check is wrong
- Clarify duplicate policy
- $O(n)/O(h)$

Agenda opener:

"Bounds or sorted inorder — not parent-only."

Full spoken answer:

"Comparing each node only to its parent is insufficient — a value can satisfy the parent and still violate an ancestor. I DFS with exclusive low and high bounds updated as I go left and right, or I do an inorder walk and ensure values are strictly increasing. I clarify whether duplicates are allowed. Time $O(n)$, space $O(h)$."

Common wrong answer:

"Left less than node and right greater is enough locally." (Misses ancestor constraints.)

Follow-up ladder:

- **L1:** Show the classic counterexample (4 under 6 under 5).
- **L2:** Duplicate keys — $\text{left} \leq$ or $<$?
- **L3:** Recover a BST from a swapped pair?

Provenance: Learning-lab

Q4. Explain level-order traversal implementation. (30–45s)

Answer points:

- Queue + `levelCount = queue.count` inner loop
- Collect or process per level
- Empty root $\rightarrow []$
- Zigzag = reverse alternate levels

Agenda opener:

"Queue waves — snapshot count each level."

Full spoken answer:

"I BFS with a queue starting at the root. While the queue isn't empty, I snapshot the current count as the level size, dequeue that many nodes, enqueue their children, and append that level's values to the result. That snapshot is what keeps levels from mixing. Empty root returns an empty list. For zigzag I reverse every other level after collecting it."

Common wrong answer:

"Just BFS into one flat list" when the API wants `[[Int]]` .

Follow-up ladder:

- **L1:** Right side view from the same skeleton?
- **L2:** `removeFirst` cost on Array?
- **L3:** N-ary level order?

Provenance: Learning-lab · see `code/TreeBFS.swift`

Q5. How does recursive LCA work on a binary tree? (45s)

Answer points:

- Null or match p/q $\rightarrow$ return node
- Recurse L/R
- Both non-null $\rightarrow$ current is LCA

- Else bubble non-null side
- Contrast BST walk $O(h)$

Agenda opener:

"Postorder markers — split means current."

Full spoken answer:

"For a general binary tree I recurse: if the node is null or equals p or q, I return it. I recurse left and right. If both sides return non-null, the current node is the LCA because the targets split here. If only one side is non-null, I bubble that up. I confirm both nodes exist in the tree. Complexity $O(n)$ time and $O(h)$ space. If it's a BST, I'd instead walk comparing values in $O(h)$."

Common wrong answer:

Using BST comparison walk on a non-BST.

Follow-up ladder:

- **L1:** Parent pointers available — alternate approach?
- **L2:** LCA of more than two nodes?
- **L3:** First common ancestor in a deeplink route tree (shape analogy)?

Provenance: Learning-lab; soft Verified · S13 route-tree analogy

Q6. Preorder vs inorder vs postorder — when each? (30–45s)

Answer points:

- Pre: copy/serialize prefix, path push before children
- In: BST sorted ops
- Post: height/diameter/LCA combine after children
- Don't force inorder on non-BST for "sorted"

Agenda opener:

"When I process relative to children decides the order."

Full spoken answer:

"Preorder processes the node before children — useful for serialization prefixes and building paths downward. Inorder is left-node-right — on a BST that yields sorted order for validate and k-th. Postorder processes children first — natural for height, diameter, and LCA where you combine left and right results. On a general binary tree, inorder is not a sorted sequence."

Common wrong answer:

"Inorder always sorts any binary tree."

Follow-up ladder:

- **L1:** Iterative inorder skeleton?
- **L2:** Why serialize often uses preorder + nulls?

Provenance: Learning-lab

Q7. How do you approach a cold tree Medium in the first two minutes? (45–60s)

Answer points:

- Clarify constraints
- Brute one-liner
- Name BFS vs DFS pattern
- Complexity + edges
- Then code

Agenda opener:

"Clarify, brute, pattern, complexity, edges, code."

Full spoken answer:

"I restate the problem and clarify binary versus BST, whether I need a path or a boolean, and if mutation is allowed. I give a one-line brute force — often enumerating paths or comparing all subtrees in $O(n^2)$. Then I name the optimized pattern: BFS with level size, DFS with bounds, postorder LCA, and so on. I state $O(n)$ time and $O(h)$ or $O(w)$ space, list edges like null root and skewed trees, and only then start coding."

Common wrong answer:

Silent coding for three minutes.

Follow-up ladder:

- **L1:** What if you misclassified BFS vs DFS?
- **L2:** Tie to Day 23 unknown-pattern drill.

Provenance: Learning-lab · timing guide

Tricky questions

T1. "Can you do this in $O(1)$ extra space?" (90–120s)

Answer points:

- Distinguish auxiliary heap vs call stack
- Morris exists for inorder but mutates links
- BFS inherently $O(w)$ unless output-only tricks
- Prefer honest $O(h)$ recursive unless asked Morris and you're solid

Agenda opener:

"Clarify what counts as extra — stack versus heap."

Full spoken answer:

"I'd clarify whether the call stack counts. Recursive DFS is $O(h)$ stack space even if I allocate no containers. For inorder, Morris threading can get $O(1)$ auxiliary space by temporarily mutating child links and restoring them — I'd only code that if I'm fluent, because it's easy to corrupt the tree under time pressure. For BFS, $O(w)$ queue space is inherent to processing levels unless we're playing output-accounting games. My default senior answer is honest $O(h)$ recursive or explicit $O(h)$ iterative stack, and I'll outline Morris only as an option with mutation trade-offs."

Common wrong answer:

"BFS is $O(1)$ " or claiming Morris without knowing restore.

Follow-up ladder:

- **L1:** Walk Morris link/restore at a high level.
- **L2:** Production: would you mutate a shared tree?

Provenance: Learning-lab

T2. "Diameter of binary tree — why isn't it just max depth left + right at root?" (90–120s)

Answer points:

- Diameter may lie entirely in a subtree
- At every node: leftH + rightH candidate
- DFS returns height; updates global best
- Confirm edges vs nodes definition

- One $O(n)$ pass

Agenda opener:

"Not only through the root — update at every node."

Full spoken answer:

"If I only add left and right depths at the root, I miss diameters that never pass through the root — a long path entirely in the left subtree, for example. The correct approach is a postorder DFS that returns height from each node and, at every node, updates a global maximum with leftHeight plus rightHeight. That one pass is $O(n)$ time and $O(h)$ space. I also confirm whether diameter is counted in edges or nodes so I don't ship an off-by-one."

Common wrong answer:

Root-only sum; or two full depth passes without explaining per-node updates.

Follow-up ladder:

- **L1:** Balanced tree vs line tree examples.
- **L2:** Return a pair (height, diameter) instead of global?

Provenance: Learning-lab · `code/TreeDFS.swift`

T3. "Serialize binary tree — how do you handle nulls and ambiguity?" (90–120s)

Answer points:

- Preorder without nulls is ambiguous
- Explicit null tokens or BFS-with-nulls
- Deserialize with index or queue
- $O(n)$
- Light SDUI versioning nod

Agenda opener:

"Encoding must be reversible — nulls or counts."

Full spoken answer:

"If I serialize only values in preorder, different shapes can produce the same sequence. I include explicit null markers, or I BFS and record null children the way the classic codec does. Deserialization consumes the stream with an index pointer or a queue to rebuild pointers. Complexity is $O(n)$. In product SDUI we face a related problem — schemas need versioning and unknown-type fallbacks so documents stay interpretable across app versions — but that's versioning judgment, not the same as LeetCode null tokens."

Common wrong answer:

Values-only preorder "because structure is obvious."

Follow-up ladder:

- **L1:** N-ary encode?
- **L2:** Compress runs of nulls?

Provenance: Learning-lab; soft Verified · S3 schema instinct

T4. "DFS recursion blows the stack on skewed trees — what do you do in production?" (90–120s)

Answer points:

- Prefer iterative
- Validate / cap input depth
- Chunk processing
- UI/CMS trees: max depth + fail-soft UI

- Tie to crash-free culture — defensive limits

Agenda opener:

"Iterative plus depth budget — don't hero-recurse."

Full spoken answer:

"In production I'd prefer an iterative traversal with an explicit stack when depth may be unbounded. I'd also validate or cap depth on untrusted CMS or SDUI input and fail soft with a placeholder rather than crash. UI trees are often shallow, but nested documents can surprise you. That matches a crash-free mindset at consumer scale — defensive limits beat relying on a larger stack. Same judgment I'll use later for on-device AI fail-soft: degrade usefully instead of hard-failing."

Common wrong answer:

"Never happens on iOS" or only "increase stack size."

Follow-up ladder:

- **L1:** What max depth would you pick for SDUI?
- **L2:** Metrics if depth cap hits often?

Provenance: Soft Verified · S8 crash-free culture; S3 SDUI shape; preview Day 24 fail-soft

T5. "Symmetric tree — is checking `root.left.val == root.right.val` enough?" (90–120s)

Answer points:

- Need mirror structure recursively
- Compare outer/inner pairs
- BFS pair queue alternative
- Null symmetry matters

Agenda opener:

"Mirror recursively — not one root check."

Full spoken answer:

"A single comparison of the root's children is nowhere near enough. I need a mirror predicate: two nodes match if values are equal and the left of one matches the right of the other recursively, including nulls. Alternatively I BFS with a queue of pairs. Time $O(n)$, space $O(h)$ or $O(w)$."

Common wrong answer:

Only compare immediate children or only values inorder.

Follow-up ladder:

- **L1:** Asymmetric nulls on one side?
- **L2:** Perfect binary tree vs symmetric?

Provenance: Learning-lab

T6. "Kth smallest in a BST — why not sort all values?" (90s)

Answer points:

- Inorder yields sorted order
- Stop at k — $O(h + k)$
- Full sort $O(n \log n)$ unnecessary
- Clarify 1-indexed k

Agenda opener:

"Inorder is already sorted — stop early."

Full spoken answer:

"In a BST, inorder traversal visits values in sorted order, so I can decrement k as I visit and return when k hits zero — typically $O(h + k)$ with a stack, better than collecting everything and sorting. Sorting all node values would be unnecessary work. I clarify whether k is 1-based and that the tree is a valid BST."

Common wrong answer:

Dump to array and `sort()` without using BST property.

Follow-up ladder:

- **L1:** Follow-up: many kth queries — augment tree with sizes?
- **L2:** Morris for $O(1)$ aux?

Provenance: Learning-lab

Practice set for today

Record **3 Normal + 2 Tricky**. Target ≥ 4 Normal, ≥ 3 Tricky on [answer-timing-guide.md](#).

Suggested set: Q1, Q3, Q5 + T2, T3. '''

05 — Exercises: Trees BFS / DFS

Goal: pattern ID + spoken agenda > clever tricks. Log every solve.

1. Code lab (read before inventing)

File	What to extract
code/TreeNode.swift	Shared node type
code/TreeBFS.swift	Level order, zigzag, right side view
code/TreeDFS.swift	Max depth, diameter, path sum, symmetric, LCA
code/BSTProblems.swift	Validate BST, kth smallest
code/SerializeTree.swift	Codec with null markers

Drill: For each function, cover the solution, speak the 60–90s agenda from memory, then uncover and compare.

2. Solve set (target 8–12 Medium)

From [dsa-track.md](#) Week 4 Trees:

#	Problem	Pattern tag	Agenda memo? (Y/N)	Time to first correct	Miss note
1	Binary Tree Level Order Traversal				
2	Binary Tree Zigzag Level Order				
3	Maximum Depth of Binary Tree				
4	Diameter of Binary Tree				
5	Path Sum II (or Path Sum)				

6	Lowest Common Ancestor of a Binary Tree				
7	Validate Binary Search Tree				
8	Kth Smallest Element in a BST				
9	Symmetric Tree				
10	Binary Tree Right Side View				
11	Serialize and Deserialize Binary Tree (<i>stretch</i>)				
12	Construct Binary Tree from Preorder and Inorder (<i>stretch</i>)				

Rule: No memo of the master script → solve does not count as done.

3. Whiteboard-only (no IDE) — 25 min

Pick **Right Side View** cold:

- 1. Speak agenda (≤90s).
- 2. Write level-size BFS outline.
- 3. Trace the 5-node example from Foundations.
- 4. State complexity.
- 5. Only then peek `code/TreeBFS.swift` .

4. Speaking drills

Drill	Time	Pass bar
Complexity script ×5	5 min	No “O(1) recursive” slip
BST validate trap explanation	60s	Counterexample included
Diameter root-only failure	60s	Subtree diameter named
Production bridge 45s	45s	No fake LeetCode metrics

5. Timed Q drill

- 1. Pick **3 Normal + 2 Tricky** from [04-questions.md](#) .
- 2. Record; score vs [answer-timing-guide.md](#).
- 3. Log misses in gotchas (especially: BST validate, diameter root-only, O(1) space).

6. Bonus (10 min)

Cold-open **only** the agenda + code outline for Construct from Preorder and Inorder — no full polish. Tag pattern: “hash index + recurse sizes.”

7. Exit criteria

- ☐ ≥8 problems logged with pattern tags
- ☐ Complexity said aloud before each
- ☐ Two-layer Q set recorded once
- ☐ Skimmed revision twin: [.././revision/weeks/week-04/day-22.md](#)

Tomorrow: Day 23 HashMap / Heap + mixed unknown-pattern — reuse today’s tree tags in the mixed set. '''

Day 23 — DSA HashMap / Heap + Mixed Unknown-Pattern

Week 4 · Full study (self-contained) · ~4–5 hrs
Revision twin: [revision/weeks/week-04/day-23.md](#)

Outcomes

By end of day, without notes, you can:

- Deploy **HashMap** patterns: frequency / index / prefix-state / sliding-window counts / two-sum family — and when hashing beats sorting
- Deploy **Heap** patterns: Top-K, merge K, Kth in stream, K closest — with min vs max heap intuition in Swift
- Run a **mixed unknown-pattern** drill: 60–90s classify *before* coding when the topic isn’t labeled
- Speak complexity: expected vs worst hash; heap $O(\log k)$; space trade-offs
- Stop pattern-worship: stuck → brute → constrain → named structure

How to study

1. [01-foundations.md](#) — mental models for hash + heap
2. [02-deep-dive.md](#) — catalogs, mixed classification, traps
3. [03-production-bridge.md](#) — S2 / S3 / S16 honest hooks
4. [code/](#) — worked Swift (Two Sum, anagrams, prefix+K, top-K, merge-K sketch, window)
5. [04-questions.md](#) — two-layer approach scripts
6. [05-exercises.md](#) — hash/heap blocks + mixed simulation
7. Revision twin for timed recall

Module map

Module	File
Foundations	01-foundations.md
Deep dive	02-deep-dive.md
Production bridge	03-production-bridge.md
Questions	04-questions.md
Exercises	05-exercises.md
Code	code/

Time budget

Block	Minutes
Foundations	40–50
Deep dive + code	70–90
Production bridge	15–25
Questions	50–60
Hash/Heap solves	60–75

Mixed 90s tags + ≥ 3 solves	75–90
----------------------------------	-------

Provenance

Label	Today
Verified · S2	Synchronised dictionaries — map access culture, not hash math
Verified · S3	Search debounce / coalesce keys
Verified · S16	GymFlow top-K similar via embeddings — heap/select mental model
Learning-lab	All code/ Swift

Do **not** invent LeetCode metrics. Resume trio only if behavioral drift: **30L+ DAU**, **99.95%**, **30%+ nav**.

Agenda opener

“Clarify uniqueness, order, online vs offline. Brute nested loops. Optimized: hash complements/frequencies or size-K heap. Expected $O(n)$ or $O(n \log k)$. Edges: empty, dups, negatives. Coding now.”

01 — Foundations: HashMaps & Heaps

Intern → mid mental model. Mixed unknown-pattern discipline starts here.

0. North star

HashMaps turn repeated scans into expected $O(1)$ lookups when you can design a key. Heaps turn “keep the best K” into $O(n \log k)$ instead of full sorts. When the problem is unlabeled, spend 90 seconds classifying before you romanticize either.

1. HashMap mental model

1.1 What a Dictionary buys you

Need	Pattern
“Have I seen X?”	Set / map presence
“Where was X?”	value → index
“How many X?”	frequency map
“Group by signature”	canonical key → list
“Subarray math with negatives”	prefixSum → frequency
“Window invariant”	counts in current window; shrink when broken

Brute: for i, for j > i ... → $O(n^2)$
Hash: one pass, look up complement → expected $O(n)$, space $O(n)$

1.2 Intern demo — Two Sum

Need indices of two numbers that add to target.

- Sort + two pointers loses original indices unless you store pairs.

- Hash: for each `nums[i]`, if `target - nums[i]` in map $\rightarrow$ answer; else store `nums[i]  $\rightarrow$  i`.

Say aloud: "I pick hash when I need indices or unsorted input; I mention sort alternative when memory is tight and indices don't matter."

1.3 Average vs worst case (interview script)

"Dictionary lookup is **expected $O(1)$** . Pathological collisions can degrade toward $O(n)$ — theoretical in interviews; I design good keys and state expected $O(n)$ total. Space $O(n)$."

Swift note: keys must be `Hashable`. Don't hash mutable identity carelessly.

2. Heap mental model

2.1 What a heap buys you

Need	Structure
Top K largest	Min-heap size K (root = smallest of the elite)
Top K smallest	Max-heap size K
Kth largest in stream	Min-heap size K; root is answer
Merge K sorted lists	Min-heap of current heads
K closest points	Max-heap size K by distance (or min of n)

Complexity script:

"offer/poll $O(\log k)$. Top-K over n elements $\rightarrow O(n \log k)$. Better than full sort $O(n \log n)$ when $k \ll n$. Heapify build is $O(n)$."

2.2 Min vs max — the interview trap

For **Kth largest**, a **min-heap of size K** stores the K largest seen; the root is the smallest among them = Kth largest. Using a max-heap of all n is wasteful.

2.3 Swift interview pragmatism

- iOS 18+ / packages may expose `Heap`; in interviews, a clear array + sift or "I'll use a binary heap API" narration beats broken boilerplate.
- If n is tiny, say "I'd sort — $O(n \log n)$ — acceptable here" and move on.

3. Mixed unknown-pattern — the 90-second protocol

Interviewers paste unlabeled Mediums. First 90 seconds:

1. **Restate + constraints** (sorted? online? duplicates? memory?).
2. **Brute** one sentence.
3. **Classify**: array scan / two pointer / window / hash / heap / tree / graph / binary search / DP-lite.
4. **Pick one** + reject one alternative aloud.

Ask yourself	If yes $\rightarrow$
Need counts / complements / seen set?	Hash
Always care about "best K so far"?	Heap
Contiguous constraint?	Window / prefix
Sorted property exploitable?	Two pointer / binary search

Hierarchy / nested?	Tree DFS/BFS (Day 22)
Overlapping subproblems stated?	DP (don't force today)

Anti-pattern: Jumping to DP because it “feels hard.”

4. Complexity cheat card

Structure	Time (typical)	Space
Hash one-pass	Expected $O(n)$	$O(n)$
Sort + two pointer	$O(n \log n)$	$O(1)$ aux (ignore sort storage debate)
Heap top-K	$O(n \log k)$	$O(k)$ (+ freq map $O(n)$)
Bucket by freq	$O(n)$ when freq $\leq n$	$O(n)$
Sliding window + map	$O(n)$	$O(\Sigma)$ alphabet or $O(n)$

5. First scripts to memorize

Hash:

“Clarify uniqueness and whether order matters. Brute nested loops $O(n^2)$. Optimized: hash complements / frequencies for expected $O(n)$. Edges: empties, duplicates, negatives, unicode keys if strings.”

Heap:

“I need the best K under a score. Keep a size- K heap — each insert $O(\log k)$, total $O(n \log k)$. Clarify sort order of the output. Edges: $k > n$, ties, empty.”

Unknown:

“I’ll classify in 90 seconds: constraints → brute → hash/heap/window/tree → commit.”

→ [02-deep-dive.md](#)

6. Swift-specific gotchas (say if relevant)

Topic	Interview line
String indexing	Not $O(1)$ random access — convert to [Character] when needed
Array.removeFirst	$O(n)$ — fine to acknowledge in BFS/queue talk
Dictionary keys	Must be Hashable; unstable hashes → pain
Heap boilerplate	Narrate operations if API missing; clarity > clever sift bugs

7. Checklist before Deep Dive

- ☐ Two Sum hash script clean
- ☐ Min-heap size K for K th largest explained
- ☐ Negatives → prefix+hash (not window) internalized
- ☐ 90s unknown-pattern protocol rehearsed once

02 — Deep Dive: Hash / Heap Catalogs & Mixed Simulation

1. HashMap pattern catalog

Trigger	Pattern	Complexity script
Two sum / complement	value → index	$O(n)/O(n)$; sort+two-pointer if indices unused
Group anagrams / isomorphic	canonical key → list	sorted string $O(k \log k)$ or count tuple $O(k)$
Subarray sum equals K	prefixSum → frequency	$O(n)$; careful with 0 / negatives
Longest substring without repeat	window + last-seen/count	$O(n)$
First unique / LRU building block	ordered dict mental model	Dict + list in design interviews
Clone graph / random pointer	old → new map	$O(n)$; DFS/BFS + map
Time-based / snapshot	key → sorted timestamps	Binary search values

Prefix sum + hash (negatives break two pointers)

```
count = 0; map = {0: 1}; prefix = 0
for x in nums:
    prefix += x
    count += map[prefix - K]    # how many earlier prefixes make subarray sum K
    map[prefix] += 1
```

Trap: Using sliding window two pointers when negatives exist.

Window + counts hygiene

When a key's count hits 0, **remove** it (or treat carefully) so `map.count` reflects distinct keys. Forgetting this breaks “exactly K distinct” variants.

2. Heap pattern catalog

Trigger	Pattern	Notes
Top K frequent / largest	size-K heap or bucket by freq	Bucket $O(n)$ when $\text{freq} \leq n$
Kth largest in stream	min-heap size K	Root = kth
Merge K sorted lists	min-heap of heads	$O(N \log K)$
K closest points	max-heap size K by distance	Or quickselect
Meeting rooms / intervals	sort + min-heap of ends	Scheduling
Reorganize / task scheduler	max-heap by freq + cooldown	Greedy

Merge K — why not concat + sort?

Concat + sort is $O(N \log N)$. Heap-of-heads is $O(N \log K)$. When $K \ll N$, heap wins. Divide-and-conquer pairwise merge is also strong.

Top K frequent — bucket alternative

After frequency map, place numbers into buckets indexed by frequency `0...n` . Walk buckets from high to low. $O(n)$ time. Say this as the optimization over heap when applicable.

3. Mixed unknown-pattern simulation (today's differentiator)

Drill protocol (90 min block):

- Blindfold topic: pick **6** mixed problems (Array/Hash/Heap/Tree).
- For each: **timer 90s classification only** (no code) → write pattern name → then solve or skip.
- Goal: **5/6 correct pattern tags** before coding.

Misclassification recovery:

"I started window but negatives break monotonicity — switching to prefix+hash."

Seniors narrate the pivot; juniors silently thrash.

4. Trade-offs

Choice	When	Cost
HashMap	Lookups, counts, grouping	$O(n)$ space; key-design bugs
Sort + two pointer	Memory tight, sorted OK	Loses indices unless pairs kept
Heap size K	Top-K online/streaming	$O(n \log k)$; comparator bugs
Full sort	Need full order anyway	Overkill for top-K
Bucket / counting	Freq in bounded range	Extra buckets; clarity win
TreeMap / sorted dict	Range queries on keys	Heavier; rare on iOS LC

5. Failure modes

- Hashing **mutable** objects as keys.
- Forgetting decrement/remove at zero in window maps.
- Max-heap when you needed min-heap for Kth largest stream.
- Returning heap contents unsorted when problem wants sorted top-K — clarify.
- Mixed drill: jumping to DP because it "feels hard."
- Silent coding with no classification.

6. Approach scripts (two-layer ready)

Use these as **Answer points** skeletons in [04-questions.md](#) :

Family	60s script spine
Two Sum	indices? → hash complement; else sort+two pointer
Anagrams	key = sorted or count[26]; bucket lists
Subarray sum K	negatives? → prefix+hash; non-neg → window OK
Top K freq	count → min-heap K or bucket
Merge K	heap heads $O(N \log K)$ vs sort $N \log N$
Unknown	clarify → brute → classify → pick → edges

7. Worked complexity examples

Problem	Say
Group Anagrams	$O(n \cdot k \log k)$ sorted keys or $O(n \cdot k)$ counts; space $O(n \cdot k)$
Top K Frequent	$O(n \log k)$ heap or $O(n)$ bucket
Merge K Lists	$O(N \log K)$
Longest substr no repeat	$O(n)$ time, $O(\min(n, \Sigma))$ space

→ [03-production-bridge.md](#) · [code/](#)

03 — Production Bridge: Maps & Top-K Without Cosplay

Verified hooks

Verified · S2 — Synchronised dictionaries (BookMyShow)

Truth: Shared maps needed **serialized access** (GCD serial queues / RW locks) to stop races — concurrency ownership, not “I optimized Dictionary hash.”

≤20s:

“In product code I treat maps as shared mutable state that needs a concurrency boundary — same respect for keyed aggregation I show in hash problems, plus synchronization.”

Verified · S3 — Search debounce / state

Truth: Coalesce in-flight search keys; cancel stale responses — map/set of request identity.

≤20s:

“Search coalescing is keyed state — counts and last-seen identities show up the same way window maps do in interviews.”

Verified · S16 — GymFlow top-K similar

Truth: Cosine top-K over embeddings; TF-IDF fail-soft. Heap/select is the *mental model* for “best K neighbors,” not a claim you shipped `CFBinaryHeap` .

≤20s:

“Recommender top-K is the product cousin of heap patterns — keep best K under a score, with a lexical fallback if the model path fails.”

Honesty table

OK	Not OK
“Maps for coalescing and counts”	“Our search is Two Sum”
“Top-K similar exercises”	Fake latency % from heap choice
Soft S8: aggregate Crashlytics by key	Invented triage metrics

Bridge script (45s)

"Interview hash and heap problems train keyed aggregation and top-K selection. In production I've used maps for coalescing and safe shared state, and top-K style selection in an on-device recommender. I don't force LeetCode names onto resume bullets — I reuse the complexity instincts."

Provenance: Verified · S2 · S3 · S16 · learning-lab `code/`

→ [04-questions.md](#)

04 — Questions (two-layer approach scripts)

Cover **Full spoken answer**. Speak from **Answer points** only. Uncover and compare.

Normal $\approx$ 30–45s · Tricky $\approx$ 90–120s · Unknown-pattern meta up to 90s classification + answer.

Normal questions

Q1. Two Sum — hash vs sort? (30–45s)

Answer points:

- Hash complement $O(n)/O(n)$ keeps indices
- Sort + two pointer $O(n \log n)/O(1)$ aux loses indices unless pairs stored
- Prefer hash when indices required
- Clarify duplicates / multiple answers

Agenda opener:

"Indices decide hash versus sort."

Full spoken answer:

"If I need original indices, I scan once and store each value to index in a dictionary, looking up the complement target minus current. That's expected $O(n)$ time and $O(n)$ space. Sorting plus two pointers is $O(n \log n)$ and better on auxiliary memory, but sorting loses indices unless I store value-index pairs first. I clarify duplicates and whether any pair is enough."

Common wrong answer:

Always sort first "because two pointers are elegant," then scramble to recover indices.

Follow-up ladder:

- **L1:** Multiset duplicates?
- **L2:** Three Sum bridge?
- **L3:** Streaming one-pass constraint?

Provenance: Learning-lab · `code/HashPatterns.swift`

Q2. Top K frequent elements approach? (30–45s)

Answer points:

- Frequency map first
- Min-heap size $K \rightarrow O(n \log k)$ **or** bucket by frequency $O(n)$
- Stream version $\rightarrow$ maintain heap online
- Soft GymFlow top-K analogy

Agenda opener:

"Count, then size-K heap or buckets."

Full spoken answer:

"I count frequencies in a hash map, then either keep a min-heap of size K keyed by frequency for $O(n \log k)$, or bucket numbers by frequency for $O(n)$ when frequencies are at most n . I clarify whether the output must be sorted. In a recommender, keeping top- K similar items is the same instinct — bound the elite set under a score."

Common wrong answer:

Full sort of all uniques every time without discussing $k \ll n$.

Follow-up ladder:

- **L1:** Stream of numbers — K th largest?
- **L2:** Ties in frequency?
- **L3:** GymFlow: heap vs full sort of catalog?

Provenance: Learning-lab; soft Verified · S16

Q3. Average vs worst-case for Dictionary? (30–45s)

Answer points:

- Expected $O(1)$ lookup
- Pathological collisions $O(n)$ theoretical
- Good hash + load factor
- Swift Hashable pitfalls (don't use unstable keys)

Agenda opener:

"Expected $O(1)$ — I state expected totals."

Full spoken answer:

"A hash map gives expected $O(1)$ lookup and insert. In the pathological collision case operations can degrade toward $O(n)$, which I mention as theoretical in interviews while stating expected $O(n)$ for a linear pass. Keys need a solid Hashable implementation — mutable or poorly distributed keys cause real pain."

Common wrong answer:

"Dictionary is always worst-case $O(1)$."

Follow-up ladder:

- **L1:** Why not hash a class instance identity carelessly?
- **L2:** S2 — sync access vs hash complexity?

Provenance: Learning-lab; soft Verified · S2

Q4. When is a heap the wrong tool? (30–45s)

Answer points:

- Need full sort anyway
- Need random access by key
- $K \approx n \rightarrow$ just sort
- Arbitrary delete-by-id without handle

Agenda opener:

"Heap when I only need best K — not always."

Full spoken answer:

"A heap is wrong when I need the fully sorted order anyway, when I need keyed random access, when K is basically n so $O(n \log k) \approx O(n \log n)$, or when I must delete arbitrary elements by id without an index handle. In those cases I sort, use a dictionary, or an indexed priority queue design."

Common wrong answer:

"Always use a heap for anything with K in the name."

Follow-up ladder:

- **L1:** Indexed priority queue sketch?
- **L2:** Quickselect alternative?

Provenance: Learning-lab

Q5. Sliding window + hash — longest substring without repeating? (45s)

Answer points:

- Expand right; shrink left on duplicate
- Last-seen index or counts
- $O(n)$
- Exactly K distinct is a variant

Agenda opener:

"Expand, shrink when invariant breaks."

Full spoken answer:

"I slide a window with a map of characters to last seen index or counts. I expand the right pointer; when I see a duplicate inside the window I advance the left until the invariant holds, and I track the max length. Time $O(n)$. The same map hygiene shows up in 'exactly K distinct' problems — I shrink when distinct count exceeds K."

Common wrong answer:

Restart the whole scan on every duplicate ($O(n^2)$ disguised).

Follow-up ladder:

- **L1:** Exactly K distinct characters?
- **L2:** Unicode / String indexing in Swift?

Provenance: Learning-lab · `code/HashPatterns.swift`

Q6. Unknown-pattern: first 90 seconds — what do you say? (45–60s)

Answer points:

- Restate constraints
- Brute one-liner
- Classify hash/heap/window/tree/...
- Pick + reject alternative
- Complexity + edges

Agenda opener:

"Classify before I commit."

Full spoken answer:

"I restate the problem and constraints — sorted, online, duplicates, memory. I give a one-line brute force. Then I classify: do I need counts or complements, best K, a contiguous window, a tree shape, or sorted order I can exploit. I pick one structure and briefly reject an alternative, state complexity and edges, and only then code. If I'm wrong, I narrate the pivot instead of silent thrashing."

Common wrong answer:

Silence, or "this must be DP."

Follow-up ladder:

- **L1:** Demo on an unlabeled Medium aloud.
- **L2:** Tie to machine-round Day 25 discipline.

Provenance: Learning-lab

Tricky questions

T1. "Subarray sum equals K with negatives — why can't I use two pointers?" (90–120s)

Answer points:

- Window shrink assumes monotonicity
- Negatives break that
- Prefix sums + hash of prefix frequencies
- $O(n)/O(n)$
- Count vs boolean variants

Agenda opener:

"Negatives kill window monotonicity — prefix plus hash."

Full spoken answer:

"Two-pointer sliding windows for subarray sums rely on monotonicity — adding elements moves the sum in a predictable direction so shrinking from the left is safe. Negatives break that. Instead I keep a running prefix sum and a hash map from prefix value to how many times I've seen it. For each index, prefix minus K tells me how many earlier prefixes form a subarray summing to K. That's $O(n)$ time and $O(n)$ space. I clarify whether we need the count of subarrays or just existence."

Common wrong answer:

Forcing two pointers and hand-waving negatives.

Follow-up ladder:

- **L1:** All non-negative — would you switch back to window?
- **L2:** Longest vs count?

Provenance: Learning-lab

T2. "Merge K sorted lists — why not concatenate and sort?" (90–120s)

Answer points:

- Concat+sort works $O(N \log N)$
- Heap-of-heads $O(N \log K)$
- Better when $K \ll N$
- Divide-and-conquer merge alternative
- Lazy streams / memory

Agenda opener:

"Both work — heap wins when K is small."

Full spoken answer:

"Concatenating all nodes and sorting is correct in $O(N \log N)$. A min-heap of the current head of each list extracts the next global minimum in $O(\log K)$ per node, totaling $O(N \log K)$, which is better when K is much smaller than N. I'd also mention pairwise divide-and-conquer merging. If lists are lazy streams, the heap approach streams naturally without materializing everything first."

Common wrong answer:

Accepting $O(N \log N)$ without discussing K .

Follow-up ladder:

- **L1:** $K = N$ singleton lists?
- **L2:** Swift heap boilerplate under time pressure?

Provenance: Learning-lab · `code/HeapPatterns.swift`

T3. "Design $O(1)$ insert, delete, getRandom" (90–120s)

Answer points:

- Dict value→index + array
- Delete = swap with last
- Hash alone can't getRandom uniformly in $O(1)$
- Array alone delete $O(n)$
- Duplicates → multiset variant

Agenda opener:

"Array for random, map for index — swap-delete."

Full spoken answer:

"A hash map alone doesn't give uniform random in $O(1)$, and an array alone makes delete $O(n)$. I keep an array of values plus a dictionary from value to its index. Insert appends and records the index. Delete swaps the target with the last element, updates the map, and pops — $O(1)$. getRandom picks a random index into the array. If duplicates are allowed, I store sets of indices per value."

Common wrong answer:

Only HashMap or only array.

Follow-up ladder:

- **L1:** Random among weighted items?
- **L2:** Thread safety (S2 soft)?

Provenance: Learning-lab

T4. Unknown-pattern meta: "I don't recognize this — what do you do?" (90–120s)

Answer points:

- Clarify → brute on example → name constraints
- Pick hash/heap/window/tree
- State complexity
- Code brute if needed then optimize
- Seniors communicate uncertainty with a plan

Agenda opener:

"Uncertainty with a plan beats silent panic."

Full spoken answer:

"I say what I'm unsure about, then work a small example brute force so I see the structure. I ask whether the data is online, contiguous, or needs top- K . That usually collapses to hash, heap, window, or tree. I state the complexity I believe, code the clearest correct approach first if time is tight, and narrate the optimization. The same discipline shows up in a three-hour machine round — classify, vertical slice, don't freeze."

Common wrong answer:

Silence or random DP.

Follow-up ladder:

- **L1:** Live classify a problem from the mixed set.
- **L2:** Day 25 / Day 27 transfer.

Provenance: Learning-lab

T5. "Top K frequent — heap vs bucket — defend one." (90s)

Answer points:

- Heap general when K small
- Bucket $O(n)$ when $\text{freq} \leq n$
- Clarity under timer
- Output order requirements

Agenda opener:

"Bucket when frequencies are bounded by n ."

Full spoken answer:

"After the frequency map, if I only need top K and K is tiny, a size-K heap is simple and $O(n \log k)$. If frequencies are integers between 0 and n , I prefer buckets indexed by frequency and walk from high to low for $O(n)$. I pick based on constraints and what I can code correctly in time. I clarify whether ties or sorted output matter."

Common wrong answer:

Always heap; never mention bucket.

Follow-up ladder:

- **L1:** Follow-up: return any order vs sorted by freq.

Provenance: Learning-lab

Suggested record set

Q1, Q2, Q6 + T1, T4.

05 — Exercises: Hash / Heap / Mixed

1. Code lab

File	Extract
code/HashPatterns.swift	Two Sum, Group Anagrams, Subarray Sum K, Longest Substring
code/HeapPatterns.swift	Top K Frequent, Kth Largest stream sketch, Merge K lists

Cover → speak agenda → uncover.

2. Hash block (4–5)

#	Problem	Tag	Agenda Y/N	Time	Miss
1	Two Sum				
2	Group Anagrams				

3	Subarray Sum Equals K				
4	Longest Substring Without Repeating				
5	Top K Frequent (hash+heap bridge)				

3. Heap block (3–4)

#	Problem	Tag	Agenda Y/N	Time	Miss
1	Kth Largest Element in an Array				
2	Merge K Sorted Lists				
3	K Closest Points to Origin				
4	Task Scheduler (<i>optional</i>)				

4. Mixed simulation (mandatory)

- 1. Pick **6** unlabeled Mediums (include ≥ 1 tree from Day 22 leftovers; ≥ 1 “feels like DP but is hash/window”).
 - 2. Each: **90s timer — tag only** on paper.
 - 3. Goal **5/6** correct tags.
 - 4. Then fully solve ≥ 3 of them with spoken agendas.
- Anti-perfectionism:** 25+ min stuck → write brute, state optimize idea, move on; log and revisit.

5. Timed Q drill

3 Normal + 2 Tricky from [04-questions.md](#) . Extra: one cold mixed Medium — **3-minute approach only** (no code). Grade classification.

6. Exit criteria

- ☐ Hash block ≥ 4 , Heap block ≥ 3
- ☐ Mixed tags 5/6
- ☐ Two-layer Qs recorded
- ☐ Revision twin skim: [../revision/weeks/week-04/day-23.md](#)

Tomorrow: Day 24 On-device AI — different muscle; DSA stays warm via flashcards only if weak.

Day 24 — On-Device AI (FinTrack / GymFlow)

Week 4 · Full study (self-contained) · ~4–5 hrs
Revision twin: [revision/weeks/week-04/day-24.md](#)

Outcomes

By end of day, without notes, you can:

- Draw end-to-end **on-device AI assistant** flow: eligibility → retrieve → prompt → local infer / fallback → stream → metrics
- Defend **FinTrack (S15)**: BM25 offline RAG + Apple Intelligence / Foundation Models bridge + **deterministic rule** fail-soft; **no cloud sync** of financial data
- Defend **GymFlow (S16)**: INT8 MiniLM **TFLite** + cosine top-K + **TF-IDF** fail-soft
- Explain why seniors design **degradation paths** before the happy path

- Answer normal + tricky AI questions with agenda, trade-off, and honest provenance (product AI ≠ District tooling AI)

How to study

1. [01-foundations.md](#) — primer (tokens, quant, embeddings, RAG, fail-soft)
2. [02-deep-dive.md](#) — FinTrack + GymFlow full teaching + hybrid router + privacy
3. [03-production-bridge.md](#) — Verified S15/S16 + Applied design + 30L DAU caution
4. [code/](#) — Swift teaching sketches (BM25-ish ranker, router, TF-IDF fallback, eligibility)
5. [04-questions.md](#) — two-layer Q&A
6. [05-exercises.md](#) — HLD whiteboard + STAR timing
7. Revision twin for flashcards

Optional deepen (after chapter is solid): [on-device-llm-ai-engine.md](#) — this chapter is self-contained first.

Provenance

Label	Today
Verified · S15	FinTrack: Hive local-first, BM25 RAG, Apple FM / native AI bridge, rule fallback, no cloud financial sync
Verified · S16	GymFlow: INT8 MiniLM TFLite, cosine top-K, TF-IDF fail-soft
Verified · S9	Contrast only — AI <i>tooling</i> / context engineering ≠ product on-device AI
Verified · S8	Soft — kill switch / crash-free judgment at consumer scale
Learning-lab	code/ sketches — teaching, not production Flutter/Dart paste

Do **not** invent model latency ms, accuracy %, or “we fine-tuned on user ledgers.”

Agenda opener (SD)

“I’ll cover privacy constraints, retrieval, on-device inference, hybrid fallback, and ops metrics — starting with FinTrack and GymFlow as concrete instances.”

01 — Foundations: On-Device AI Primer

Teach-back target: an intern can explain RAG + quantization + fail-soft before you deep-dive FinTrack/GymFlow.

0. North star

On-device AI is retrieval + guarded inference + an explicit degrade path — not “call an LLM.” Private data stays local; models are optional accelerators behind capability checks.

1. Staff-level pipeline (draw every time)

```
User query
→ Device eligibility (OS, Neural Engine, memory, thermal, Low Power)
→ Retrieve local context (BM25 / vector / rules)
→ Assemble prompt (budget tokens; minimize PII for any cloud path)
→ Local generate OR cloud fallback OR deterministic template
→ Stream tokens to UI (AsyncSequence / partial updates)
→ Log quality + failure reason (not raw private prompts)
```

2. Primer glossary (45–60s each aloud)

Concept	One-liner + mobile sting
Token	Model I/O unit $\approx \frac{3}{4}$ word; context window is a hard budget
Quantization	FP16 3B $\approx$ huge RAM; INT4/INT8 makes on-device feasible; quality trade-off
Embedding	Fixed-dim vector of meaning; cosine $\approx$ semantic closeness
RAG	Retrieve private/local chunks $\rightarrow$ stuff into prompt; model not “trained” on your Hive DB
KV-cache	Speeds decode; eats RAM; drop under memory pressure
Hybrid router	Local first; cloud/rules when thermal, complexity, or capability fails
BM25	Classical lexical retrieval — strong offline, no embedder required
TF-IDF	Lexical weighting — GymFlow’s fail-soft cousin
Fail-soft	Feature degrades to a useful subset; never hard-crashes the app
ANE / NPU	Hardware path for inference; not always available / not always free thermally

Quantization intuition (say cleanly)

“Unquantized multi-billion-parameter weights blow mobile RAM budgets. INT8/INT4 shrink weights so mmap + Neural Engine can run. You trade some quality for fit — and you still need a path when the model file is missing.”

RAG intuition (say cleanly)

“The model doesn’t know the user’s ledger. I retrieve relevant local rows or docs, put them in the prompt, and ground answers in that context. Numbers for money come from the database, not from free-form generation.”

3. Two products, one principle

	FinTrack (S15)	GymFlow (S16)
Domain	Personal finance coach	Workout recommender
Retrieve	BM25 over local spend index	MiniLM embeddings + cosine
Generate / rank	Apple Intelligence / FM when present	Embedding similarity top-K
Fail-soft	Deterministic rules	TF-IDF
Privacy invariant	No cloud sync of financial records	Prefer on-device; no cloud LLM required

Say aloud: “Same architecture shape — different retrieval and fallback knobs.”

4. Fail-soft before happy path (senior habit)

Failure	Fallback
No Foundation Model / no TFLite	Rules / TF-IDF; hide generative chrome
Model missing / checksum fail	Same
Thermal / Low Power	Pause infer; lexical or cached tips
Memory warning	Unload weights; clear KV

Empty retrieval	Honest empty + generic tips — don't hallucinate
-----------------	--

5. Product AI vs tooling AI (do not conflate)

	Product (S15/S16)	Tooling (S9 District)
Who uses it	End user in the app	You as engineer
Risk	Privacy, hallucination, thermal	Bad tests, false greens
Story	Architecture + fail-soft	Context engineering + review

6. Complexity / ops scripts

Privacy 90s (memorize spine): local-first → retrieve local → generative optional → analytics coarse → cloud only consent + minimize.

SD agenda 10s: privacy → retrieve → infer → fallback → metrics.

→ [02-deep-dive.md](#)

02 — Deep Dive: FinTrack, GymFlow, Router, Privacy

Full teaching in-chapter. Repo HLD doc is optional deepen after this.

1. FinTrack — privacy-first Savings Coach (Verified · S15)

1.1 Problem

Expense coach that must **not** ship ledger data to a cloud LLM by default.

1.2 Architecture you can defend

1. **Local store:** Flutter + Hive; biometric lock; **no cloud sync** of financial records.
2. **Retrieval:** **BM25** over local spending index (lexical, offline, explainable).
3. **Generation path:** `flutter_native_ai` → **Apple Foundation Models** / platform AI where available; else on-device Nano-class path when present.
4. **Fail-soft: Deterministic rule engine** (budget thresholds, category heuristics) when model unavailable — still a coach, not a blank screen.
5. **Privacy invariant:** Prefer on-device; if a design interview adds cloud, **PII sanitize + user consent + minimize fields**.

1.3 BM25 teaching (enough for interviews)

BM25 scores a document against a query using term frequency, document length normalization, and inverse document frequency. You do **not** need to derive the formula live.

Say:

“BM25 ranks local spend notes and category text by lexical relevance. It’s offline, explainable, and needs no embedder binary — weaker on paraphrase (‘UBER’ vs ‘rideshare’), which is an accepted trade-off for FinTrack’s text.”

1.4 Grounding money (non-negotiable)

Generative text is **never** source of truth for balances or amounts. Prompt: only use provided context. UI can show retrieved rows / citations. Rules path reads DB deterministically.

1.5 Interview line (≤20s)

"FinTrack is local-first — BM25 RAG over Hive, Apple Intelligence when present, rules when not; financial data doesn't leave the device."

→ STAR: [story-bank.md](#)#S15

2. GymFlow — edge embeddings recommender (Verified · S16)

2.1 Problem

Workout recommendations without cloud LLM cost/latency/privacy drag.

2.2 Architecture

- 1. **INT8 MiniLM TFLite** + WordPiece tokenizer (Dart in the shipped app).
- 2. Embed query / user context; **cosine similarity** over exercise catalog; trainer routine as RAG context.
- 3. **Fail-soft: TF-IDF** lexical fallback if TFLite model missing/slow/thermal.
- 4. Principle: **retrieval + ranking + degradation**, not "call an LLM."

2.3 Cosine teaching

score(a,b) = dot(a,b) / (|a| |b|)

Top-K by score. Same "best K" instinct as Day 23 heaps (mental model only).

2.4 Interview line (≤20s)

"GymFlow ranks exercises with on-device MiniLM embeddings and falls back to TF-IDF when the model path can't run."

→ STAR: #S16

3. Map portfolio → generic on-device HLD

Doc subsystem	Your proof
Local RAG / vector or lexical index	FinTrack BM25; GymFlow MiniLM + cosine
Quantized runtime	GymFlow INT8 TFLite; Apple FM path on FinTrack
Hybrid router / thermal	Speak design + eligibility checks
Token streaming UI	Partial updates; cancel on navigate away
Memory / Jetsam	Unload model, clear caches on warning
Privacy guard	FinTrack no-sync; cloud only with sanitize

4. Trade-offs

Choice	When	Cost
BM25 lexical RAG	Offline, explainable, no embedder	Weaker paraphrase
MiniLM / vector RAG	Semantic paraphrase	Model size, tokenizer bugs, jank if on main
On-device LLM generate	Privacy, latency, offline	Quality, thermal, device coverage
Cloud LLM fallback	Hard reasoning, huge context	Privacy, cost, latency, consent
Deterministic rules / TF-IDF	Fail-soft always-on	Less "magical"; set expectations

Apple Intelligence / FM	Native path, privacy story	OS/device gating; Android abstraction
Always-cloud “AI feature”	Fast demo	Fails senior privacy + offline bars

5. Fail-soft matrix (memorize)

Failure	Detection	Fallback
OS / device lacks FM / TFLite	Capability flag at launch	Rules / TF-IDF; hide generative chrome
Model file missing / checksum fail	Load error	Same; remote config disable
Thermal .serious / Low Power	ProcessInfo	Pause inference; lexical or cached tips
Memory warning	Notification	Unload weights, clear KV; degrade
Retrieval empty	Top-K = 0	Honest empty + generic tips — don’t hallucinate
Cloud path needed (design)	Router decision	Sanitize PII; TLS; timeout → local message

6. Privacy-first script (90s) — speak verbatim until automatic

“Financial and health-adjacent data stays on device. Retrieval runs locally. Generative models are optional accelerators behind capability checks. Analytics get coarse events — coach_shown, fallback_rule — not raw transactions. If a hybrid cloud path is required, it’s consent-gated, field-minimized, and never the only path.”

7. Hybrid router sketch (teaching)

```
func route(query):
  if !eligible || thermalSerious || lowPower: return rulesOrTfIdf(query)
  ctx = retrieve(query)           // BM25 or embeddings
  if ctx.isEmpty: return emptyState()
  if canLocalGenerate: return streamLocal(ctx, query)
  if cloudAllowed && consented: return streamCloud(sanitize(ctx, query))
  return rulesOrTfIdf(query)
```

See `code/HybridAIRouter.swift` .

8. Scale caution (BookMyShow judgment)

At **30L+ DAU**, unconstrained on-device LLM on checkout is a reliability/battery risk. You’d want feature flags, sampling, kill switches — same culture as **99.95% crash-free** (Verified · S8). FinTrack/GymFlow are personal-scale proofs of architecture; BMS teaches when *not* to be reckless.

9. Optional citations (after mastery)

- [on-device-llm-ai-engine.md](#)
- Apple Foundation Models / WWDC on-device sessions
- [cheatsheet.md](#) for Day 27 SD timing

→ [03-production-bridge.md](#)

03 — Production Bridge: S15 / S16 Honest Proof

Verified · S15 — FinTrack

Claim	Status
Flutter + Hive; biometric; no cloud sync of financial data	Verified
BM25 offline RAG + rule fallback	Verified
Bridge to Apple Foundation Models / platform AI	Verified
Play Store ops awareness (Crashlytics, Remote Config, AdMob)	Verified qualitative
Invented accuracy % / latency SLOs	Forbidden

2–3 min STAR spine: Situation local-first finance → Action Hive+BM25+FM+rules → Result privacy coach with degrade path → Lesson retrieval+fallback not model worship.

Verified · S16 — GymFlow

Claim	Status
INT8 MiniLM TFLite + WordPiece	Verified
Cosine top-K + trainer context	Verified
TF-IDF fail-soft	Verified
Cloud LLM dependency	Not claimed

Contrast · S9 — District tooling

Use only to **separate** concerns when asked “how do you use AI?”

“District was context engineering for migrations/tests. FinTrack/GymFlow are product on-device systems with privacy and fail-soft. I don’t conflate Cursor with RAG.”

Soft · S8

Kill switches, capability flags, not crashing on model load — crash-free culture applied to AI features.

Bridge script (45s)

“I treat on-device AI as retrieval plus guarded inference with an explicit degrade path — FinTrack BM25 and GymFlow MiniLM both ship that way. Money and recommendations stay grounded; models are optional.”

Provenance: Verified · S15 · S16; contrast S9; soft S8

→ [04-questions.md](#)

04 — Questions (two-layer Q&A) — On-Device AI

*Cover **Full spoken answer**. Speak from **Answer points**. Normal 30–45s · Tricky 90–120s · Architecture once at 3–5 min if asked.*

Normal questions

Q1. What is on-device RAG? (30–45s)

Answer points:

- Index/retrieve local docs or rows
- Inject into prompt / ranking context
- Model stays general; private data stays local
- FinTrack BM25 example

Agenda opener:

"Retrieve local context, then generate or rank."

Full spoken answer:

"On-device RAG means I keep the user's private data local, retrieve the most relevant chunks for a query, and feed that context into generation or ranking. The model isn't fine-tuned on the ledger — it reasons over retrieved rows. On FinTrack I use BM25 over a local spending index so retrieval works offline without an embedder."

Common wrong answer:

"We fine-tune the LLM on user transactions."

Follow-up ladder:

- **L1:** BM25 vs embeddings?
- **L2:** Empty retrieval behavior?

Provenance: Verified · S15

Q2. Why quantize models on mobile? (30–45s)**Answer points:**

- RAM/disk budgets
- INT8/INT4 trade accuracy for fit
- mmap + Neural Engine
- Unquantized multi-B blows Jetsam
- GymFlow INT8 MiniLM

Agenda opener:

"Fit the weights — quality trade for shippability."

Full spoken answer:

"Mobile RAM budgets can't host large full-precision weights. Quantization shrinks parameters to INT8 or INT4 so we can mmap and run on the Neural Engine. You trade some quality for fit. GymFlow ships an INT8 MiniLM TFLite model for that reason — and still has TF-IDF if the model path fails."

Common wrong answer:

Quantize with no quality/ops story.

Follow-up ladder:

- **L1:** How monitor quality regressions?
- **L2:** FP16 vs INT8 when?

Provenance: Verified · S16

Q3. How do you stream tokens to UI without jank? (30–45s)**Answer points:**

- Inference off main
- AsyncSequence / callbacks

- MainActor append
- Cancel on dismiss
- Batch UI updates

Agenda opener:

"Background infer, main append, cancelable."

Full spoken answer:

"I run inference off the main thread and stream tokens through an AsyncSequence or callback into MainActor UI updates. I cancel the task when the user navigates away so work doesn't outlive the screen. I batch rapid tokens to avoid layout thrash. Partial answers stay on screen if we pause for thermal."

Common wrong answer:

Synchronous inference on main.

Follow-up ladder:

- **L1:** Partial markdown rendering?
- **L2:** Backpressure if UI slower than decode?

Provenance: Learning-lab design; soft S15 UX

Q4. Privacy benefits of on-device AI? (30–45s)

Answer points:

- Data minimization / offline
- Lower exfil risk
- Aligns platform privacy narrative
- Still need local disk security (biometric, encryption)
- FinTrack no cloud sync

Agenda opener:

"Local-first — HTTPS alone isn't privacy."

Full spoken answer:

"On-device AI keeps sensitive context on the phone, works offline, and reduces exfiltration risk. That matches Apple's privacy narrative, but it's not automatic — local vaults still need biometric locks and encryption at rest. FinTrack's invariant is no cloud sync of financial records; analytics stay coarse events, not raw transactions."

Common wrong answer:

"We use HTTPS so it's private."

Follow-up ladder:

- **L1:** Analytics design?
- **L2:** When is cloud ever OK?

Provenance: Verified · S15

Q5. What is fail-soft in an AI feature? (30–45s)

Answer points:

- Capability detect
- Degrade to rules/TF-IDF/cached
- User-visible honesty
- Never crash; remote config kill

- S15/S16

Agenda opener:

"Degrade usefully — don't blank or crash."

Full spoken answer:

"Fail-soft means I detect capability and runtime pressure, then degrade to a still-useful path — deterministic rules on FinTrack, TF-IDF on GymFlow — instead of crashing or showing a dead feature. I hide generative chrome when the model isn't there, and I keep a remote-config kill switch. Users get honesty, not hallucinated magic."

Common wrong answer:

Only an error alert with no useful fallback.

Follow-up ladder:

- **L1:** Metrics for fallback rate?
- **L2:** Thermal mid-generation?

Provenance: Verified · S15 · S16

Q6. BM25 vs vector search — when each? (45s)

Answer points:

- BM25: keyword, offline light, explainable
- Vector: paraphrase/semantic
- Hybrid often best
- FinTrack BM25 vs GymFlow MiniLM split

Agenda opener:

"Lexical versus semantic — pick for the text."

Full spoken answer:

"BM25 is strong for keyword-ish personal text, fully offline, and explainable without shipping an embedder. Vector search with MiniLM handles paraphrase and semantic closeness better at the cost of model size and tokenizer complexity. FinTrack chose BM25 for spend text simplicity; GymFlow chose embeddings for exercise similarity — with TF-IDF as lexical fail-soft."

Common wrong answer:

One-size-fits-all vectors for everything.

Follow-up ladder:

- **L1:** Hybrid reciprocal rank fusion?
- **L2:** Why not BM25 alone for GymFlow?

Provenance: Verified · S15 · S16

Tricky questions

T1. "On-device model hallucinates a transaction amount — who's at fault?" (90–120s)

Answer points:

- Generative text ≠ source of truth for money
- Ground numbers in deterministic DB reads
- Prompt: only use provided context
- UI shows retrieved rows
- Rules path for balances

- Eval set; low retrieval score → fallback

Agenda opener:

"Database owns money — model doesn't."

Full spoken answer:

"That's a product-control failure if free-form generation can invent amounts. I never let the model be source of truth for money. RAG must ground numbers from deterministic database reads, the prompt must forbid inventing values, and the UI should show the retrieved rows. The rules path reads balances directly. I keep an eval set of prompts and fall back when retrieval scores are low. Hallucination becomes an architecture bug we design against, not an excuse."

Common wrong answer:

"Models just do that."

Follow-up ladder:

- **L1:** Citation chips UX?
- **L2:** Logging without leaking PII?

Provenance: Verified · S15

T2. "Why not always call GPT-4o from the phone?" (90–120s)

Answer points:

- Privacy, offline, latency, cost
- App Store / finance optics
- Cloud as optional escalate with consent
- On-device for personal context
- BMS at 30L DAU → sampling, budgets, server-side

Agenda opener:

"Privacy and offline first — cloud is escalate."

Full spoken answer:

"Always-cloud fails privacy, offline, latency, and cost bars — especially for finance. On-device retrieval and generation keep personal context local. Cloud can be an optional escalate for hard reasoning with consent and minimized fields, never the only path. At BookMyShow scale I'd be even stricter: feature flags, sampling, and budgets before any heavy AI on hot paths."

Common wrong answer:

Cost-only answer.

Follow-up ladder:

- **L1:** How would ads AI differ at 30L DAU?
- **L2:** Consent UX?

Provenance: Verified · S15; soft S8 scale judgment

T3. "Thermal throttling mid-generation — design the behavior." (90–120s)

Answer points:

- Observe thermal / Low Power
- Pause or switch to rules/TF-IDF
- Keep last partial answer
- Don't spin until Jetsam
- GymFlow: skip embed refresh

- Metrics: throttle_events, completion_rate

Agenda opener:

"Pause, degrade, keep partial — don't melt."

Full spoken answer:

"I observe ProcessInfo thermal and Low Power state. On serious pressure I pause local generation, keep the last partial answer on screen, and switch to rules or TF-IDF rather than spinning until Jetsam. GymFlow can skip embedding refresh and serve lexical recommendations. I emit coarse metrics for throttle events and completion rate — not raw prompts."

Common wrong answer:

Ignore thermal; block UI with a spinner forever.

Follow-up ladder:

- **L1:** Resume policy when thermal clears?
- **L2:** Remote config to disable generate entirely?

Provenance: Verified · S16 fail-soft; learning-lab router

T4. "How is this different from a server RAG blog demo?" (90–120s)

Answer points:

- RAM ≤ hundreds of MB, battery, offline
- Fail-soft matrix
- Privacy invariants
- Capability matrix across devices
- Cancellation / lifecycle
- Shipped FinTrack/GymFlow proof

Agenda opener:

"Mobile constraints plus shipped degrade paths."

Full spoken answer:

"A notebook cosine demo ignores Jetsam, thermal, device coverage, and cancelation. Mobile on-device AI designs for RAM budgets, battery, offline, a fail-soft matrix, privacy invariants, and lifecycle cancellation. My proof is FinTrack BM25 with rule fallback and GymFlow MiniLM with TF-IDF — not a blog embedding loop."

Common wrong answer:

Buzzword soup without constraints.

Follow-up ladder:

- **L1:** Walk FinTrack vs HLD mapping.
- **L2:** What would you add for 30L DAU?

Provenance: Verified · S15 · S16

T5. "PII in prompts when hybrid cloud is on — concrete steps." (90–120s)

Answer points:

- HTTPS necessary not sufficient
- Redact account numbers, names
- Send aggregates not raw SMS
- User toggle; retention limits
- Prefer local; Keychain/biometric vault

Agenda opener:

“Minimize, redact, consent — TLS isn’t enough.”

Full spoken answer:

“TLS is necessary and nowhere near sufficient. Before any cloud path I redact account numbers and names, prefer category aggregates over raw messages, require an explicit user toggle, and bound retention. Local vault stays biometric-locked. The default remains local-only — cloud is a narrow escalate.”

Common wrong answer:

“We encrypt HTTPS.”

Follow-up ladder:

- L1: What fields never leave device?
- L2: Audit logging design?

Provenance: Verified · S15 invariant + Applied hybrid design

T6. “District used AI for tests — isn’t that the same as on-device AI?” (90–120s)

Answer points:

- Separate concerns
- S9 = engineering accelerator + review judgment
- S15/S16 = product AI privacy/fallback
- Seniors don’t confuse Cursor with on-device RAG

Agenda opener:

“Tooling AI versus product AI — different risk.”

Full spoken answer:

“They’re different systems. At District I used AI as an engineering accelerator for migrations and tests inside a review bar — context engineering, not autopilot. FinTrack and GymFlow are product features with privacy invariants and fail-soft architecture. Seniors don’t confuse Cursor or Copilot with on-device RAG.”

Common wrong answer:

Tool-worship conflation.

Follow-up ladder:

- L1: Day 26 leadership follow-up.
- L2: When forbid AI in workflow?

Provenance: Verified · S9 · S15 · S16

Suggested record set

Q1, Q4, Q5 + T1, T6. Also deliver S15 and S16 once each at 2:30.

05 — Exercises: On-Device AI

1. Code lab (teaching sketches)

File	Role
------	------

code/BM25Ranker.swift	Lexical retrieve teaching model
code/HybridAIRouter.swift	Eligibility + fail-soft routing
code/TFIDFFallback.swift	GymFlow-style lexical fallback
code/DeviceAIEligibility.swift	Thermal / low power / capability flags

These are **learning-lab Swift** sketches of concepts verified in Flutter/Dart products — not a claim the production apps are Swift.

2. Whiteboard HLD (25–30 min)

Using SD timing micro: Clarify 5 + HLD 10 + fail-soft 5 + metrics 5.

Prompt: “Design an on-device savings coach for iOS.”

Must include: privacy invariant, BM25 retrieve, FM path, rules fallback, ops events.

3. Verbal deep-dive (15 min)

Pick **one**: FinTrack BM25 path **or** GymFlow TFLite path. Speak fail-soft matrix end-to-end.

4. STAR timing

- S15 once at 2:30; re-cut to 2:00
 - S16 once at 2:30; re-cut to 2:00
-

5. Timed Q drill

3 Normal + 2 Tricky (include T1 or T4). Score vs timing guide. Architecture answers use **3–5 min** once.

6. Exit criteria

- ☐ Pipeline drawn from memory
- ☐ Fail-soft matrix recited
- ☐ Privacy 90s script clean
- ☐ S15 + S16 timed
- ☐ Revision twin: [.././revision/weeks/week-04/day-24.md](#)

Tomorrow: Day 25 machine round — different muscle; keep AI flashcards weak-only if needed.

Day 25 — Machine Round 3hr (Briefs A & B)

Week 4 · Full study + timed build (self-contained) · ~3 hrs machine + ~1 hr debrief

Revision twin: [revision/weeks/week-04/day-25.md](#)

Outcomes

- Timebox a 3-hour machine round: clarify → skeleton → vertical slice → tests → polish — **no perfectionism**
- Deliver a demoable slice for **Brief A** (paginated list + cache + tests) **or Brief B** (SDUI renderer)
- Self-grade against rubrics; narrate trade-offs like a proctor is watching
- Answer two-layer debrief questions with **sample architecture answers**

How to study

- 1. [01-foundations.md](#) — operating system + anti-perfectionism
- 2. [02-deep-dive.md](#) — Brief A & B full specs, architectures, cut lines
- 3. [03-production-bridge.md](#) — S3 / S9 / S12 hooks (honest)
- 4. [04-questions.md](#) — two-layer Q&A + **sample architecture answers**
- 5. [05-exercises.md](#) — run the 3hr session + debrief (adapted “exercises”)
- 6. No code/ folder — your Xcode project is the artifact today

Provenance

Label	Today
Verified · S3	SDUI header / search list UX instincts
Verified · S9	Clean/MVVM + AI-assisted tests judgment
Verified · S12	Server-driven splash awareness
Learning-lab	Whatever you build in the 3hr window

Proctor opener (0:15)

“I’ll clarify briefly, then build a vertical slice: UI → ViewModel → protocol repository — fake data first. I’ll leave time for tests and document cut lines.”

01 — Foundations: Machine-Round Operating System

0. North star

Ship a tested happy path with documented cut lines. A polished unfinished cathedral loses to a demoable core.

1. Clock (memorize)

Phase	Clock	Do	Don’t
0. Clarify	0:00–0:15	Restate, API shape, offline?, UIKit/SwiftUI, tests	Code immediately
1. Agenda	0:15–0:20	Layers + milestones aloud	Silent architecture
2. Skeleton	0:20–0:45	Types, protocols, empty UI, fake repo	Pixel-perfect UI
3. Vertical slice	0:45–2:00	One happy path E2E	All edges first
4. Depth	2:00–2:30	Cache or 2nd component	Aesthetic refactor
5. Tests	2:30–2:50	3–6 meaningful unit tests	100% coverage chase
6. Buffer	2:50–3:00	Trade-offs + known gaps README	New features

2. Vertical slice definition

You can demo in **60 seconds**: load → show data → one secondary behavior (next page **or** unknown component fallback) → mention test.

If you cannot demo, you do not have a slice yet.

3. Anti-perfectionism checklist

- ☐ Styling fonts while pagination broken? → stop
- ☐ Demo in 60s now? → if no, slice
- ☐ ≥1 test after 2:30? → write two now
- ☐ Narrated cache/SDUI policy? → say in buffer
- ☐ Renaming for beauty? → stop

4. Pick A or B

Choose A if	Choose B if
Weaker on networking/state/cache	Weaker on SDUI / decoding / fallbacks
Want list UX proof	High ROI before Day 27 SDUI SD

Outline the other in 20 min notes only if surplus time — do not build both.

→ [02-deep-dive.md](#)

02 — Deep Dive: Brief A & Brief B

Brief A — Paginated list + cache + tests

Prompt (use as-is)

*Build a mobile screen that loads a **paginated** remote list (cursor or page number). Show loading / empty / error. Support **pull-to-refresh** and next-page on scroll. Add a **cache** so revisiting shows last-good data quickly, then refresh. Include **unit tests** for pagination and cache policy.*

Suggested architecture

```
ListView (SwiftUI or UIKit)
  → ListViewModel (items, page/cursor, isLoading, error)
    → ListRepository protocol
      → RemoteDataSource (URLSession or stub)
      → CacheDataSource (memory + optional disk)
```

Must-have acceptance

1. First page renders from network (or stub).
2. Next page appends without wiping.
3. Failure on page 2 keeps page 1 visible.
4. Cache: cold open can show stale then refresh (define policy aloud).
5. Tests: pagination reducer / repository with mock — **not** only UI snapshots.

Cache policy options (pick one, say why)

Policy	When	Cost
Memory LRU only	Fast interview slice	Lost on kill
Memory + disk (Codable)	Stronger “senior”	Serialization time
Stale-while-revalidate	Best UX story	Need version/TTL

Cut lines (OK)

Fancy skeletons, Diffable animations, image pipeline, auth refresh, perfect offline merge.

Brief B — SDUI component renderer

Prompt (use as-is)

Build a **server-driven UI** renderer: JSON document of components (`type` , `props` , optional `children`) → native views. Support ≥3 types (e.g. `text` , `image` , `button` / `vstack`). Unknown `type` → safe fallback. Include `schemaVersion` check. Unit-test decoding + unknown-type fallback.

Suggested architecture

JSON Data

→ SDUIDocument (Codable, schemaVersion)

→ ComponentNode enum / protocol + factory

→ SDUIRenderer → AnyView / UIView

→ Feature flag / default fallback leaf

Must-have acceptance

1. Decode sample JSON.
2. Render ≥3 types.
3. Unknown type does **not** crash — placeholder + analytics stub.
4. Nested children for one container.
5. Tests: decoder + factory fallback.

Cut lines

Full CMS tooling, live reload, expression language, rich actions, Figma pixel parity.

Rubrics (1–5)

Dimension	5	3	1
Correctness	Happy + key failure solid	Happy only	Doesn’t run
Architecture	Clear layers + protocols	Mixed but navigable	God VC/VM
Cache or SDUI depth	Policy explained + coded	Partial	Missing
Tests	3+ meaningful, fast	1 weak	None
Communication	Agenda + trade-offs	Occasional	Silent
Time honesty	Cut lines documented	Overran polish	Unfinished core

Pass bar: average ≥3.5, correctness ≥4, tests ≥3.

Trade-offs

Choice	When	Cost
SwiftUI list	Faster slice	UIKit shops may want Diffable — say assumption
Protocol + fake repo first	Tests & progress	Swap to live later

Real network in round	Impressive if stable	Flaky Wi-Fi — prefer stub + one live
Perfect Clean Architecture	Rarely fits 3 hrs	Pragmatic MVVM + protocols

→ [03-production-bridge.md](#)

Worked vertical-slice scripts (speak at 0:15)

Brief A — 90s plan

"SwiftUI List + ListViewModel. Repository protocol with stub remote and memory SWR cache. Page-based pagination with in-flight guard and request generation id. Acceptance: page1, append page2, failed page2 keeps page1, stale-then-refresh, three unit tests. Cut: images, disk, auth."

Brief B — 90s plan

"Codable SDUIDocument with schemaVersion. Factory for text, image, vstack. Recursive renderer. Unknown → PlaceholderView + log stub. Tests: decode fixture, three types, unknown doesn't throw. Cut: actions DSL, live reload."

Brief A — state machine (teach)

```
idle → loadingFirst
loadingFirst → loaded | empty | error
loaded → loadingMore | refreshing
loadingMore → loaded (append) | loaded (keep + nonblocking error)
refreshing → loaded (replace) | loaded (keep stale + error)
```

Stale response rule: each fetch captures `generation` ; commit only if `generation == viewModel.generation` .

Brief B — decode + factory sketch (interview whiteboard)

```
struct SDUIDocument: Codable {
    let schemaVersion: Int
    let root: ComponentDTO
}
struct ComponentDTO: Codable {
    let type: String
    let props: [String: String]?
    let children: [ComponentDTO]?
}
enum ComponentNode {
    case text(String)
    case image(URL?)
    case vstack([ComponentNode])
    case unknown(type: String)
}
```

Factory: `switch type` → known cases; `default: .unknown(type)` . Renderer never force-unwraps.

Communication rubric cues (what graders hear)

Minute	They should hear
--------	------------------

0:10	Restated brief + 2 clarifying Qs
0:18	Layer plan + cut lines
1:00	"Happy path compiling"
2:10	Cache policy or unknown-type policy named
2:40	Test names aloud while writing
2:55	Known gaps README

03 — Production Bridge: Machine Round ↔ Shipped Instincts

Verified · S3

Search lists + backend-driven header → pagination/debounce instincts and SDUI fallback thinking (unknown components shouldn't crash).

Verified · S9

AI may scaffold tests; **you** own architecture and review. Same rule in the machine round.

Verified · S12

Server-driven splash — schema flexibility + client resilience.

≤20s

"In machine rounds I optimize for a tested vertical slice — same bias shipping SDUI and list UX: contracts, fallbacks, and observable state."

Do not claim the 3hr project is BookMyShow production code.

→ [04-questions.md](#)

04 — Questions + Sample Architecture Answers (two-layer)

Debrief / proctor style. Cover full answer; speak from points.

Normal questions

Q1. How do you approach a 3-hour machine round? (30–45s)

Answer points: Clarify → agenda → vertical slice → second requirement → tests → known gaps · demoability

Agenda opener:

"Clarify, slice, test, document cuts."

Full spoken answer:

"I clarify requirements and assumptions in the first fifteen minutes, say the layer plan aloud, then chase one end-to-end happy path before depth work like cache or a second component. I reserve the last thirty minutes for meaningful unit tests and I document cut lines instead of chasing polish. Graders should see a demoable core."

Common wrong answer:

Pixel-perfect UI first.

Follow-up: What do you cut first?

Provenance: Learning-lab method

Q2. How do you paginate without duplicates or lost pages? (45s)

Answer points: Cursor vs page · append-only · in-flight guard · ignore stale responses (generation token)

Full spoken answer:

"I pick cursor or page explicitly. New pages append; refresh replaces. An in-flight flag or Task prevents double fetch. Each request carries a generation token so late responses from an old refresh can't append onto new state. Failures on page two keep page one visible."

Provenance: Soft Verified · S3 list UX

Q3. What cache policy did you pick and why? (30–45s)

Answer points: Name SWR / TTL / memory-only · stale shown · refresh error keeps stale

Full spoken answer:

"I chose stale-while-revalidate: show last-good memory cache immediately, then refresh. On refresh failure I keep stale and surface a non-blocking error. Price-critical fields would use a short TTL in production — I'll call that out as a hardening step."

Provenance: Learning-lab · Applied design

Q4. SDUI unknown component — what should happen? (30–45s)

Answer points: Fallback view · don't crash · log type · schema version gate · optional skip children

Full spoken answer:

"The factory switches on type; default returns a PlaceholderView and emits an analytics stub with the unknown type string. Unsupported schemaVersion shows a full-screen upgrade/fallback state instead of partial corruption. Nested unknown children can be skipped while siblings still render."

Provenance: Soft Verified · S3; Applied · S3-A1 shape

Q5. How many tests are enough in 3 hours? (30–45s)

Answer points: Happy pagination or decode · one failure · one unknown/cache hit · quality over count · AI tests still need review (S9)

Full spoken answer:

"Three meaningful tests beat forty flaky ones: happy-path pagination or decode, a failure that preserves prior state, and either a cache hit or unknown-type fallback. If AI scaffolds test code, I still review assertions — District lesson."

Provenance: Verified · S9 judgment

Tricky + sample architecture answers

T1. "Your cache served stale event prices — defend." (90–120s)

Answer points: TTL + SWR · price-critical bypass · pull-to-refresh · last-updated · no cross-user cache · 30L hardening note

Full spoken answer:

"For the machine round I optimized perceived performance with SWR on a generic list. In production at consumer scale I'd classify fields: price and seat availability get short TTL or bypass cache, pull-to-refresh is obvious, UI shows last-updated, and caches are keyed per user/session. Stale generic metadata is acceptable; stale money is not. I'd document that split in the README as the next hardening step."

Provenance: Learning-lab + soft S8/S3 judgment · no fake incident

T2. "Why not generate the whole app with AI?" (90–120s)

Answer points: Scaffold OK · own architecture/edges/tests · explain every line · District accelerator not author of record

Full spoken answer:

"AI can scaffold boilerplate, but I own architecture, edge cases, and tests, and I must explain every line to the proctor. Same District rule — accelerator inside a review bar, not author of record."

Provenance: Verified · S9

T3. "You didn't finish image loading — fail?" (90–120s)

Answer points: Pre-declared cut line · core works · placeholders · honesty > spiraling

Full spoken answer:

"Image pipeline was an explicit cut line at minute twenty. Core pagination and cache work; cells use placeholders. In production I'd plug a shared image loader — I've owned lifecycle-sensitive media in ads HeroWidget work — but finishing images here would have traded away tests."

Provenance: Soft Verified · S1 media lifecycle nod · learning-lab cuts

T4. "Show concurrency bugs in your pager." (90–120s)

Answer points: Request generation id · cancel on refresh · MainActor state · no out-of-order append · S2 high-level

Full spoken answer:

"ViewModel holds a monotonic requestID. Each fetch captures the id; only matching ids commit. Refresh cancels the in-flight Task. State mutations hop to MainActor. I won't append page N+1 from a stale task after a reset. That's the same class of discipline as serializing shared mutable maps — don't let concurrent writers corrupt state."

Provenance: Soft Verified · S2

Sample architecture card — Brief A (2–3 min spoken)

Answer points: layers · fake repo first · SWR · pagination guards · 3 tests · cut lines

Full spoken answer:

"I'll use SwiftUI List bound to a ListViewModel with explicit loading, empty, and error. The ViewModel talks only to a ListRepository protocol. Today the live object composes a stub remote and an in-memory SWR cache so I can demo offline-ish revisit. Pagination uses page index with an in-flight guard and a generation token. Tests cover append page two, failed page two keeping page one, and cache hit returning stale before refresh. Cut lines: images, Diffable animations, disk codec, auth."

Sample architecture card — Brief B (2–3 min spoken)

Answer points: Codable document · schemaVersion · factory · 3 types · fallback · nested stack · tests

Full spoken answer:

"JSON decodes into SwiftUIDocument with schemaVersion. A factory maps type strings to an enum of text, image, and vstack. Renderer walks nodes recursively for children. Unknown types become PlaceholderView plus a log stub. Old schema versions fail closed to a safe screen. Tests decode a fixture, assert three component types, and assert unknown type doesn't throw. Cut lines: actions DSL, live reload, expression binding."

Suggested record set

Q1, Q2, Q4 + T1, T4 + one sample architecture card for your chosen brief.

Additional two-layer prompts

Q6. Cursor vs page number pagination — pick one (45s)

Answer points: Cursor stable under inserts · Page simpler stub · Say assumption

Full spoken answer:

"For production lists that churn, I prefer opaque cursors so inserts don't shift pages. In a three-hour stub, page numbers are fine if I say the assumption and still guard in-flight and stale responses. I'll note cursor migration as a hardening step."

Provenance: Learning-lab

Q7. Where does caching live — ViewModel or Repository? (45s)

Answer points: Repository owns cache policy · VM owns UI state · testable seam

Full spoken answer:

"I keep cache policy in the repository so the ViewModel stays about UI state and pagination intent. That makes repository tests cover SWR without spinning up views, and matches Clean/MVVM seams I used in migrations."

Provenance: Soft Verified · S9

T5. "Proctor asks you to add auth mid-round." (90s)

Answer points: Acknowledge · park behind protocol · don't derail slice · document

Full spoken answer:

"I'd introduce an AuthProviding dependency on the remote data source, stub a static token now, and document real refresh as out of scope. I won't rebuild the pager to chase auth unless the brief made it acceptance-critical."

Provenance: Learning-lab honesty

05 — Exercises (Machine Day): Run the Round

Adapted: the exercise is the 3-hour build + debrief. No LeetCode volume.

1. Setup (before timer)

- ☐ Xcode project or playground ready
 - ☐ Sample JSON / API stub pasted
 - ☐ Timer visible
 - ☐ Brief A or B chosen
 - ☐ Record first 3 min clarifying + plan (graded)
-

2. Execute 3:00:00

Follow Foundations clock. Narrate. Document cut lines in a short README in the project.

3. Debrief (60 min — mandatory)

Step	Action
1	Score rubrics 1–5 each dimension
2	Write 5 bullets keep / fix
3	Answer 3 Normal + 1 Tricky from 04-questions.md about <i>your</i> code
4	Speak sample architecture card once for your brief
5	Log gotchas — no all-night rewrite

4. Pass / fail

Pass if average ≥ 3.5 , correctness ≥ 4 , tests ≥ 3 . Else note ≤ 3 fix-forwards for after Mock #4 — still no rewrite tonight.

5. Exit

- ☐ Demoable 60s path
- ☐ Rubric scored
- ☐ Revision twin skim: [.././revision/weeks/week-04/day-25.md](#)

Tomorrow: Behavioral leadership STAR scripts — rest the IDE.

Day 26 — Behavioral Leadership STAR

Week 4 · Full study (self-contained) · ~4–5 hrs
Revision twin: [revision/weeks/week-04/day-26.md](#)

Outcomes

- Deliver **2–3 min** STARs with clock discipline (20s S/T → 90s Action → 30s Result → 20s Lesson)
- Ship **full spoken scripts** for **S8 / S6 / S9 / S1** (priority set)
- Handle conflict, incident, mentorship, and **AI tooling judgment without tool-worship**
- Map prompts → story IDs in ≤ 10 s; metrics only: **30L+ DAU, 99.95% crash-free, 30%+ nav**

How to study

- [01-foundations.md](#) — STAR rhythm + prompt router
- [02-deep-dive.md](#) — conflict / incident / mentorship / AI judgment
- [03-production-bridge.md](#) — provenance walls
- [04-questions.md](#) — two-layer prompts + **full scripts**
- [05-exercises.md](#) — record reps (no coding day)
- No `code/` — voice memos are the artifact

Provenance

Only resume-backed metrics. S9 = tooling judgment $\neq$ S15/S16 product AI.

01 — Foundations: STAR Rhythm & Prompt Router

0. North star

Situation/Task 20s → Action 90s (3–5 concrete steps + one trade-off) → Result 30s (real metric or crisp outcome) → Lesson 20s. Leadership = ownership clarity, not title theater.

1. Timing table

Segment	Time	Do	Don't
Situation + Task	~20s	Scale + your ownership	Company history lecture
Action	~90s	3–5 steps, one trade-off	"We" without your role
Result	~30s	Metric or crisp outcome	Fake numbers
Lesson	~20s	Reusable principle	Humble-brag essay

Opener:

"I'll take ~2 minutes on [X] — context, what I owned, outcome."

2. Prompt → story router (≤10s)

Interviewer asks	Reach for	Backup
Hard technical project	S1 Ads/HeroWidget	S10
Conflict / disagreement	S6 LE sheet or S8 IMOC	S14
Incident / on-call	S8	S2
Mentorship / leadership sans title	S6, S10, S14, S1	—
AI / productivity tools	S9 District	contrast S15
Privacy / on-device AI	S15	S16
Why senior / impact	S6 30%+, S8 99.95%, 30L+ DAU	—
Failure	Real miss + process change	No fake hero failure

3. Metric trio (say cleanly)

"I've owned features on BookMyShow at 30L+ DAU, helped drive 99.95%+ crash-free discipline on my paths, and shipped UX like the LE bottom sheet that cut full-screen navigations by 30%+ on targeted flows."

4. AI tooling vs product AI (one breath)

"District S9 is how I use AI as an engineer — context and review. FinTrack/GymFlow are product on-device systems. I don't conflate them."

→ [02-deep-dive.md](#)

02 — Deep Dive: Conflict, Incident, Mentorship, AI Judgment

1. Conflict script structure (S6 primary)

- 1. Disagreement named (API shape, scope, UX).
- 2. What you did: data, options, user impact, contracts.
- 3. Disagree *with* people, not *at* them.
- 4. Resolution + relationship intact.
- 5. Lesson: contracts and metrics over opinions.

LE Bottom Sheet: Align PM/Design/Backend; lighter overview; **30%+** flows fewer full-screen navigations.

2. Incident script structure (S8)

- 1. Severity + user impact at **30L+ DAU**.
- 2. IMOC role: stabilize → communicate → RCA → prevent.
- 3. Concrete technical/process action.
- 4. Result tied to **99.95%+ crash-free** culture.
- 5. Lesson: ownership + blast radius + rollback > hero debug.

3. Mentorship / leadership without title (S1 and friends)

"Leadership was setting technical direction, unblocking others, and owning rollout checklists — not waiting for a manager title."

S1: POP + generics standards on revenue module. S10/S14: SDK boundaries / zero-regression checklist.

4. AI tooling judgment (S9) — no tool-worship

Wrong: "AI writes my code; I'm 10x."

Senior:

- 1. Problem: migration / tests needed leverage without surrendering design.
- 2. Action: shaped **context** — modules, conventions, acceptance, review bar.
- 3. Judgment: rejected bad generations; human oracle on critical tests.
- 4. Separate product AI (S15/S16).
- 5. Lesson: tools amplify clear thinking; ownership stays yours at scale stakes.

Timed opener:

"At District I used AI as an accelerator for migration/tests — the skill was context engineering and review judgment, not prompting theater."

5. Trade-offs

Choice	When	Cost
Data-backed disagreement	Cross-functional conflict	Prep time
Escalate early	Safety/security/revenue	Political capital
AI-generated tests	Boilerplate	False green
Hand-written critical tests	Money, identity, crashes	Slower
STAR >4 min	Never	Looks junior

→ [03-production-bridge.md](#)

6. Failure STAR — choose a real miss (do not invent)

Pick **one** you can defend:

Candidate miss	Process change you can name
Pin rotation / break-glass under-specified	Document S4-A1 design; staged rollout thinking
SDUI unknown-type gap	Fallback + schemaVersion (S3-A1)
Race that escaped before sync maps	Concurrency boundary + tests (S2)

Script spine: miss → impact → what you changed → how you detect recurrence.

7. "Tell me about yourself" collage (90–120s)

BookMyShow scale + reliability + delivery metrics → District architecture + AI judgment → Raw SDK/hybrid → FinTrack/GymFlow → target senior ownership.

Do **not** turn this into four full STARs — one breath per chapter of your career.

03 — Production Bridge: Provenance Walls

Story	Allowed	Forbidden
S1	POP, generics, HeroWidget lifecycle, revenue-critical	Invented fill-rate %
S6	30%+ fewer full-screen navs on targeted flows	"All of BMS nav –30%" overclaim
S8	30L+ DAU, 99.95%+ CFS , IMOC P0/P1	Sole ownership of all CFS
S9	Context engineering, AI-assisted tests/reviews, Clean/MVVM	"AI wrote the app"
S15/S16	Only if asked product AI	Mixing into tooling answer

Bank: [story-bank.md](#)

→ [04-questions.md](#)

04 — Questions + Full Spoken STAR Scripts (two-layer)

For STAR prompts: **Answer points** = timing spine; **Full spoken answer** = 2–3 min script. Practice covering the script.

Priority full scripts

S6 — LE Bottom Sheet (conflict / impact) (2–3 min)

Answer points:

- S/T: full-screen overview too heavy
- A: E2E bottom sheet; align PM/Design/Backend contracts; reusable component
- R: **30%+** flows fewer full-screen navigations
- L: small surface + contracts > rewrite

Agenda opener:

"I'll take ~2 minutes on the LE bottom sheet — cross-functional delivery and the navigation impact."

Full spoken answer:

*"On BookMyShow, users were taking full-screen navigations for event overview more often than they needed to — friction on high-traffic flows I shared ownership of. I led an end-to-end lightweight overview as a bottom sheet. That meant aligning Product, Design, and Backend on the API contract and content rules so we weren't debating pixels without data. I shipped it as a reusable component and integrated it into the targeted flows. The resume outcome is that we reduced full-screen navigations for **30%+ of user flows** in that scope. The lesson I reuse is that a small UI surface with clear contracts often beats a large navigation rewrite — and conflict gets resolved by making the user impact and the interface contract explicit, not by winning a taste argument."*

Common wrong answer:

Vague "we collaborated well" with no metric or contract.

Follow-up ladder:

- **L1:** What did Backend push back on?
- **L2:** What would you do differently?
- **L3:** How did you measure the 30%+?

Provenance: Verified · S6 · BookMyShow · LE Bottom Sheet

S8 — IMOC + crash-free (incident / leadership) (2–3 min)

Answer points:

- S/T: **30L+ DAU**; **99.95%+** crash-free bar; P0/P1 during peaks
- A: Crashlytics triage; IMOC coords iOS/backend/QA; mitigations + comms
- R: Sustained crash-free discipline; minimized downtime in peaks
- L: owner, blast radius, rollback > hero debug

Agenda opener:

"I'll cover incident ownership at BookMyShow scale — IMOC and crash-free discipline."

Full spoken answer:

*"BookMyShow consumer iOS sits at **30+ lakh DAU**, so reliability isn't abstract — we held a **99.95%+ crash-free** bar and needed fast P0/P1 response during high-traffic events. I worked Crashlytics triage with structured crash workflows, and as **IMOC** I coordinated iOS, backend, and QA when incidents hit: stabilize first, communicate blast radius, then root cause and prevent. That looked like feature guards, hotfix paths, and clear ownership instead of everyone debugging in a pile. The result was sustaining high crash-free sessions and minimizing downtime when traffic spiked. My lesson is that incident leadership is clarity of owner, blast radius, and rollback — hero debugging alone doesn't scale."*

Common wrong answer:

Claiming sole credit for company-wide CFS; drama without process.

Follow-up ladder:

- **L1:** Example mitigation you drove?
- **L2:** How do you prevent recurrence?
- **L3:** Conflict during an incident?

Provenance: Verified · S8 · BookMyShow · IMOC / CFS

S9 — District AI tooling (judgment, no worship) (2–3 min)

Answer points:

- S/T: Free Parking + MVVM/Clean migration; need velocity without surrendering design
- A: Context engineering; AI for reviews/tests inside envelope; structured logging
- R: Feature shipped; migration progressed with review discipline
- L: AI amplifies clear boundaries; you own architecture/tests
- Contrast S15/S16 if asked

Agenda opener:

"At District I used AI as an accelerator — the skill was context engineering and review judgment, not prompting theater."

Full spoken answer:

*"At District I was shipping Free Parking billing adjustments and helping migrate patterns across MVVM and Clean Architecture without burning the regression budget. I used Cursor, Claude, and Copilot as accelerators — but only after I shaped context: module boundaries, conventions, acceptance criteria, and a review bar. AI helped with boilerplate and XCTest/XCUI Test drafts; I rejected weak generations and kept human ownership on architecture and critical-path assertions. We also used structured logging to debug faster. The feature shipped and the migration moved with velocity **and** discipline. The lesson interviewers should hear: tools amplify clear thinking; they don't replace ownership — especially when you've also worked consumer scale where bad releases hit **30L+ DAU**. And this is different from FinTrack/GymFlow, which are product on-device AI systems, not engineering-tooling stories."*

Common wrong answer:

"AI wrote the app" / tool-worship / conflating with S15.

Follow-up ladder:

- **L1:** Example of rejected AI output?
- **L2:** When do you forbid AI?
- **L3:** Team shipping junk with Copilot — what do you do?

Provenance: Verified · S9 · District · Context Engineering

S1 — Ads refactor + HeroWidget (hard technical) (2–3 min)

Answer points:

- S/T: Highest-revenue Ads module; video in HeroWidget lifecycle
- A: POP + generics pipeline; HeroWidget pause/play; stakeholder alignment
- R: Maintainable type-safe pipeline; correct lifecycle
- L: POP + generics over inheritance; lifecycle is product contract
- No invented fill-rate %

Agenda opener:

"I'll walk through our highest-revenue Ads refactor — protocols, generics, and video lifecycle."

Full spoken answer:

"I owned work on BookMyShow's highest-revenue Ads module when we needed a safer, reusable rendering path — including video ads inside a HeroWidget that had to pause and play correctly with visibility and view-controller lifecycle. I refactored rendering around protocol-oriented ad component contracts and generics so the pipeline stayed type-safe as new ad types arrived, instead of growing an inheritance thicket. I built the reusable HeroWidget with explicit pause/play tied to lifecycle so we weren't wasting playback or glitching UI. I coordinated behavior with stakeholders without breaking the revenue path. We shipped a maintainable pipeline with lifecycle-correct video. The lesson: for revenue-critical UI, prefer protocol-oriented design and generics over deep inheritance, and treat lifecycle as part of the product contract — not an afterthought."

Common wrong answer:

Invented fill-rate or revenue percentage.

Follow-up ladder:

- **L1:** Why not inheritance?
- **L2:** How test lifecycle?
- **L3:** Conflict on API shape?

Provenance: Verified · S1 · BookMyShow · Ads / HeroWidget

Normal prompt pack (two-layer short)

Q1. Tell me about yourself (90–120s)

Answer points: BMS scale highlights → District architecture → Raw SDK → FinTrack/GymFlow → senior iOS ownership goal

Full spoken answer:

*"I'm an iOS engineer who's owned consumer features at BookMyShow scale — **30L+ DAU** — across ads, SDUI-ish surfaces, networking security, and reliability culture toward **99.95%+ crash-free**. At District I shipped Free Parking and drove Clean/MVVM migration with disciplined AI-assisted engineering. At Raw I built reusable SDK and hybrid UIKit/SwiftUI work across NBA/WNBA apps. On the side I shipped privacy-first on-device AI in FinTrack and GymFlow. I'm targeting senior roles where I own architecture, reliability, and delivery end-to-end."*

Provenance: Collage of Verified stories

Q2. Conflict with PM/Design (→ S6 script above)

Use **S6 Full spoken answer**. Points: contracts, options, **30%+**.

Q3. Incident you owned (→ S8 script above)

Use **S8 Full spoken answer**.

Q4. Mentored / led without authority (2–3 min)

Answer points: Pick S1 standards or S10/S14 checklist · direction + unblock + rollout ownership

Full spoken answer (S1-angled):

"I lead without waiting for a title by setting the technical standard and unblocking others. On the Ads refactor I defined protocol contracts so teammates could add ad types without forking the pipeline, and I made lifecycle rules explicit in HeroWidget so reviewers had a clear bar. That's mentorship through architecture — people move faster when the envelope is obvious. Same pattern later with SDK boundaries and migration checklists on other teams."

Provenance: Verified · S1; soft S10/S14

Q5. How do you use AI in your workflow? (45–90s)

Answer points: S9 spine compressed · not autopilot · contrast product AI

Full spoken answer:

"I use AI inside a strict envelope — clear module boundaries, acceptance criteria, and review. It accelerates boilerplate and test drafts; I still own architecture and reject bad output. That's District context engineering. Separately, FinTrack and GymFlow are product on-device AI with privacy and fail-soft — different story."

Provenance: Verified · S9; contrast S15/S16

Q6. Biggest impact metric? (30–45s)

Answer points: Pick one and scope it honestly

Full spoken answer:

*"The cleanest product metric is the LE bottom sheet — **30%+** fewer full-screen navigations on targeted flows. For reliability context I cite **99.95%+ crash-free** and **30L+ DAU** ownership environment — with honest scope wording."*

Provenance: Verified · S6 · S8

Tricky prompts

T1. Tell me about a failure (90–120s)

Answer points: Real miss · impact · process change · prevention · no humblebrag

Full spoken answer (template — fill with your real miss):

"I'll use a real miss, not a humblebrag. [Name the miss — e.g. under-specified pin rotation readiness / SDUI unknown-type gap / race that escaped]. Impact was [user or eng cost]. I owned the fix and changed process by [checklist / test / guard / review bar]. I know it won't recur because [detection + ownership]."

Provenance: Honest personal; do not invent

T2. Team leans on Copilot and ships junk (90–120s)

Answer points: Review bar · critical paths human-designed tests · pair on architecture · measure escaped defects · teach context engineering

Full spoken answer:

"I don't ban tools and I don't shrug. I set a review bar, require human-designed tests on money/identity/crash paths, pair on architecture so context is shared, and watch escaped defects. I teach the same context-engineering habit I used at District — tools stay, accountability stays."

Provenance: Verified · S9 judgment

T3. Why senior vs mid? (90–120s)

Answer points: Scope 30L+ · revenue module · reliability · SDK thinking · cross-functional metrics · incidents · migration judgment · privacy-aware AI

Full spoken answer:

*"Senior for me means scope and judgment: shipping on **30L+ DAU** surfaces, owning revenue-critical UI architecture, participating in **99.95%+** crash-free culture and incident coordination, delivering measurable UX wins like **30%+** nav reduction, designing reusable SDK boundaries, migrating architecture with checklists, and separating product AI privacy work from engineering-tooling AI. Mid executes tickets; senior owns trade-offs and blast radius."*

Provenance: Verified collage

T4. Did AI do the District migration? (90–120s)

Use compressed **S9** script; emphasize rejected output + human architecture ownership.

T5. Push back on a bad deadline (90–120s)

Answer points: Make risk visible · MVP cut · protect crash-free class outcomes · escalate with data

Full spoken answer:

"I make quality and reliability risk visible with options: cut scope to an MVP, shift date, or accept explicit risk — I don't silent-hero overtime as the only plan. At consumer scale I'd protect crash-free and incident load the way we treated P0/P1 seriousness at BookMyShow. I escalate with data, not vibes."

Provenance: Soft Verified · S8 / S6 negotiation style

Suggested record set

S6, S8, S9, S1 full scripts + T2.

05 — Exercises (Behavioral Day)

No coding. Voice memos are the deliverable.

1. Full pass (priority)

Record each once with timer 2:30:

1. S6
2. S8
3. S9
4. S1

Re-record any >3:15 — cut Action to 3 bullets.

2. Second pass

S10, S14, S15, S2 — once each if energy remains; else park for Day 28.

3. Cold prompts

Partner throws Q1–Q6 / T1–T5; you may only answer with bank stories.

4. Timed drill

5 STARs including conflict, incident, AI tooling. Score ≥ 4 on timing guide behavioral rubric.

5. Exit

- ☐ Four priority scripts clean
- ☐ Metric trio once
- ☐ No tool-worship slip on S9
- ☐ Revision twin: [../../revision/weeks/week-04/day-26.md](https://github.com/isaacmeade/revision/weeks/week-04/day-26.md)

Tomorrow: Expert Mock #4 — sleep.

Day 27 — Expert Mock #4 (Full Loop)

Week 4 · Full loop dress rehearsal (self-contained) · ~4.5–5.5 hrs

Revision twin: [revision/weeks/week-04/day-27.md](https://github.com/isaacmeade/revision/weeks/week-04/day-27.md)

Outcomes

- Execute **Coding 45 + iOS deep dive 45 + System design 45** with calendar discipline
- Open each segment with the correct agenda from the timing guide
- Score against **full scorecards**; leave **≤ 3 fix-forwards** for Day 28 (flashcards only)
- Keep proof loaded: **30L+ DAU**, **99.95%**, **30%+ nav**; District S9; FinTrack/GymFlow if AI; Raw as needed

How to study / run

1. [01-foundations.md](https://github.com/isaacmeade/01-foundations.md) — schedule + openers
2. [02-deep-dive.md](https://github.com/isaacmeade/02-deep-dive.md) — segment playbooks + scorecards
3. [03-production-bridge.md](https://github.com/isaacmeade/03-production-bridge.md) — proof trio + pivot rules

- 4. [04-questions.md](#) — warm-up two-layer Q pools
- 5. [05-exercises.md](#) — run the mock + debrief (adapted)
- 6. No new topics; no code/ kit — fresh DSA problem day-of

Expert briefing (paste)

Muhammed is targeting senior iOS. Coding 45 (Medium, unlabeled pattern), iOS deep dive 45 (concurrency, architecture, networking/SDUI, perf/crashes — push trade-offs), System design 45 (SDUI **or** networking+pinning **or** on-device AI). Score communication and timing. Metrics allowed: 30L+ DAU, 99.95% crash-free, 30%+ nav reduction.

01 — Foundations: Mock #4 Operating System

0. North star

Dress rehearsal. Timer visible. No notes in-frame. Agenda first every segment. Debrief ends with exactly three fix-forwards — then stop.

1. Day schedule

Block	Duration	Mode
Warm-up	15 min	Weak flashcards; one STAR opener (S6 or S8)
Coding	45 min	Expert or self-proctor
Break	10–15 min	Walk; no doomscroll
iOS deep dive	45 min	Conceptual + production
Break	10–15 min	
System design	45 min	Cheatsheet timing
Debrief	45–60 min	Scorecards + ≤3 fix-forwards; stop

2. Openers (rehearse 30s each before segment)

Coding: Clarify → brute → optimize → complexity → edges → code.

Deep dive: Definition → mechanism → trade-off → production.

System design:

"I'll spend 5 minutes on scope, then architecture, then deep-dive and ops. Does that work?"

Behavioral if pulled:

"I'll take ~2 minutes — context, what I owned, outcome."

3. Blank recovery

Say assumption → start from requirements → draw → offer brute → recover. No apology spiral.

→ [02-deep-dive.md](#)

02 — Deep Dive: Segment Playbooks & Scorecards

Segment A — Coding (45)

Min	Action
0–3	Clarify + say-this-first
3–35	Code + narrate
35–42	Edges / tests aloud; fix
42–45	Complexity recap + alternative

Sources: [dsa-track.md](#) Medium — prefer **mixed unknown** (Day 23). Trees/hash/heap/window fair.

Scorecard — Coding

Criterion	5	3	1
Clarify + agenda	Explicit, timed	Partial	Jumped to code
Correctness	Passes cases	Minor bug	Wrong approach
Complexity	Accurate	Vague	Missing/wrong
Communication	Continuous	Occasional	Silent
Edges / tests	Covered	One mention	None
Time use	Buffer	Barely done	Incomplete core

Target: average ≥ 3.5 ; agenda ≥ 4 .

Segment B — iOS deep dive (45)

Format: 4–6 topics. Answers **30–45s** or **90–120s**; one architecture **3–5 min** if asked “design X.”

Topic pools: ARC/retain · GCD vs actors · MVVM–Clean–DI · URLSession/pinning · SDUI fallbacks · SwiftUI identity/hybrid · Instruments p50/p90 · Crash/IMOC 99.95% · SPM/Stories SDK · On-device AI fail-soft if pivoted (tight)

Scorecard — iOS deep dive

Criterion	5	3	1
Timing	Hits budgets	$\pm 30\%$	Ramble/blank
Mechanism	Correct	Fuzzy	Wrong
Trade-off	Present	Weak	None
Production proof	BMS/Raw/District	Generic	None
Agenda on long answers	Yes	Sometimes	Never
Honesty	Clear assumptions	Hedge	Bluff

Target: ≥ 4 Normals; ≥ 3 Trickies; ≥ 2 production hooks.

Segment C — System design (45)

0–5 Clarify: scope, DAU, offline?, iOS-only?, latency SLO

5–15 HLD: 4-layer client + data flow

15–25 Data/API: entities, pagination, payloads, idempotency

25–40 Deep dive: 2–3 hardest subsystems

40–45 Ops: failure modes, metrics, rollout, kill switch

Prompt options: SDUI engine · Networking+pinning · On-device AI · Image/feed

Scorecard — System design

Criterion	5	3	1
Clarify	Constraints + SLOs	Few Qs	Assumed all
HLD	4 layers	Blob	None
Deep dive	2 crisp	One shallow	Skipped
Failure modes	Kill switch, degrade	Mention	Happy only
Metrics / rollout	Concrete	Vague	None
Timing	On clock	Drifted 10+	Chaos
Personal proof	Shipped work	Light	Generic

Target: ≥3.5; clarify + failure modes ≥4.

Protect ops: If deep dive runs long, park third subsystem — **never** skip last 5 for failures/metrics.

Meta trade-offs

Choice	When	Cost
Expert human	Best signal	Scheduling
Self + recorded	Always	Blind spots
Fresh DSA	Realism	Stress — intended
Reuse Day 25 app	Comfort	Weaker coding signal

→ [03-production-bridge.md](#)

03 — Production Bridge: Proofs Under Pressure

Proof trio (always loaded)

- 1. **30L+ DAU** context (S8)
- 2. **99.95%+ crash-free** (S8)
- 3. **30%+ nav** LE sheet (S6)

Pivot kit

If they ask...	Reach
----------------	-------

SDUI	S3 / S12 + schema fallback
Networking security	S4 pinning
Concurrency	S2
Architecture migration / AI tools	S9
On-device AI	S15 / S16
SDK	S10
Hybrid UI	S13

≤20s before mock

"I'll run this like a real loop — agenda first, trade-offs, and metrics I actually shipped."

→ [04-questions.md](#)

04 — Warm-up Two-Layer Q Pools

Use in the **15 min warm-up** or between segments — **not** instead of the mock.
Cover full answer; speak from points.

Pool A — Coding openers

W1. How will you open the coding round? (30–45s)

Answer points: Restate · clarify · brute · optimize · complexity · edges · code

Full spoken answer:

"I restate the problem, clarify constraints, give a one-line brute force, name the optimized pattern — hash, heap, window, or tree — state complexity and edges, then code. I narrate throughout and leave a few minutes for edges."

Provenance: Learning-lab · Days 22–23

W2. You misclassified the pattern at minute five — what now? (45s)

Answer points: Narrate pivot · don't silent rewrite entire file · salvage correct pieces

Full spoken answer:

"I say the new classification aloud, keep any reusable helpers, and rewrite the core loop. Communication of the pivot scores; silent thrashing doesn't."

Pool B — iOS deep dive warm-ups

W3. GCD serial queue vs actor — 45s

Answer points: Both serialize · actor language-native isolation · S2 bridge · reentrancy caveat high-level

Full spoken answer:

"A serial queue serializes work on shared state; we used that pattern around synchronised dictionaries at BookMyShow. A Swift actor is a reference type with isolated state and await at the boundary — I'd prefer it for greenfield shared maps. Actors can reenter across await — I design for that."

Provenance: Verified · S2; Applied · S2-A1

W4. SDUI unknown type — 45s

Answer points: Fallback · don't crash · log · schema version

Full spoken answer:

"Unknown components render a placeholder, emit a coarse analytics event, and never force-unwrap. Schema version gates fail closed. Same instinct as backend-driven header work — resilience over assuming perfect CMS."

Provenance: Soft Verified · S3

W5. Crash-free one-liner + ownership (30–45s)

Answer points: 99.95% · triage · IMOC · prevent

Full spoken answer:

*"We treat **99.95%+ crash-free** as a product bar at **30L+ DAU**. I triage via Crashlytics workflows and as IMOC I drive stabilize-communicate-prevent on P0/P1 — not blame theater."*

Provenance: Verified · S8

Pool C — System design warm-ups

W6. How will you open system design? (30–45s)

Answer points: Staff agenda · 5 min clarify · DAU/offline/SLO

Full spoken answer:

"I'll spend five minutes clarifying scope, DAU, offline needs, and latency expectations, then draw a four-layer client HLD, deep-dive two hard subsystems, and reserve the last five for failure modes, metrics, and kill switches."

W7. Name three production proofs (30–45s)

Answer points: Trio + one specialty

Full spoken answer:

*"**30L+ DAU** context, **99.95%+ crash-free**, **30%+ LE** navigation reduction — plus I'll keep one specialty loaded: SDUI, pinning, SDK, or on-device AI depending on the prompt."*

Pool D — Recovery / meta

W8. What if you blank? (30–45s)

Answer points: Assumption · requirements · draw · brute · no apology spiral

Full spoken answer:

"I state an assumption, return to requirements, draw the shape I know, offer a brute approach, and recover. I don't burn time apologizing."

W9. SD deep dive running long — what do you cut? (90s)

Answer points: Park third subsystem · protect ops 5 min

Full spoken answer:

"I summarize and park the third deep dive. Seniors protect the last five minutes for failure modes, metrics, and rollout. Happy-path forever is a mid signal."

W10. Coding still $O(n^2)$ near time (90s)

Answer points: State optimal · sketch key function · partial credit via communication

Full spoken answer:

"I stop polishing the suboptimal path, state the $O(n)$ approach and complexity, sketch the key function, and note what I'd change. Communication recovers points; silent rewrite from zero usually doesn't finish."

W11. Interviewer challenges a BMS metric (90s)

Answer points: Clarify measurement scope · don't inflate · offer related proof

Full spoken answer:

"I clarify what the number measures — crash-free sessions, or nav reduction on targeted flows — and I don't generalize beyond the resume. I can offer a related proof if they want another angle."

W12. They ask on-device AI but you prepared SDUI (90s)

Answer points: Pivot · privacy · RAG · fail-soft · FinTrack/GymFlow · reuse 4-layer client

Full spoken answer:

"I pivot cleanly: privacy constraints, local retrieval, on-device inference, fail-soft matrix, FinTrack BM25 and GymFlow MiniLM as proof. I still use a four-layer client sketch; a slightly shorter HLD is fine if clarify was strong."

Provenance: Verified · S15 · S16

Suggested warm-up set (15 min)

W1, W6, W7, W3 — speak once each. Save W9–W12 for mental rehearsal only.

Pool E — Rapid iOS definitions (warm-up only)

W13. Retain cycle 30s

Answer points: Strong ref loop · closures capture self · weak/unowned · Instruments

Full spoken answer:

"A retain cycle is a strong reference loop keeping objects alive. Closures capturing self are the usual UIKit suspect — I break them with weak self and verify with Allocations/Leaks when needed."

W14. SSL pinning one-liner + S4 (45s)

Answer points: ATS ≠ pinning · SPKI hash · Ads URLSession migration

Full spoken answer:

"ATS is system TLS policy; pinning is an app-level identity check on the certificate or SPKI. On Ads we moved Alamofire to URLSession with HTTPS, pinning, and host whitelist."

Provenance: Verified · S4

W15. p50/p90 why (30–45s)

Answer points: Tail latency · Firebase Performance · listing/checkout/search

Full spoken answer:

“Averages hide pain. p50 and p90 on listing, checkout, and search traces show typical and tail experience — we instrumented those with Firebase Performance.”

Provenance: Verified · S5

W16. Fail-soft AI 30s

Answer points: Capability · rules/TF-IDF · no crash · S15/S16

Full spoken answer:

“If the model path can’t run, FinTrack falls to deterministic rules and GymFlow to TF-IDF — useful degrade, not a blank crash.”

Provenance: Verified · S15 · S16

05 — Exercises (Mock Day): Run the Full Loop

The mock **is** the exercise. No extra LeetCode volume after.

1. Materials

- ☐ Scorecards printed / notes doc ready (for proctor, not you mid-answer)
- ☐ Timer · water · breaks planned
- ☐ SD prompt chosen or expert chooses
- ☐ DSA problem fresh / unlabeled

2. Run

Warm-up 15 → Coding 45 → break → iOS 45 → break → SD 45 → debrief 45–60.

Before each segment: 30s quiet opener rehearsal only.

3. Debrief template

Segment	Score avg	Agenda score	One note
Coding			
iOS			
SD			

Exactly three fix-forwards for Day 28 (flashcard fronts only):

1. ...
2. ...
3. ...

4. Hard stop

No new frameworks. No Day 25 rewrite. One STAR only if an iOS answer needed a story and failed timing — else rest.

5. Exit

- ☐ Three scorecards filled
- ☐ ≤3 fix-forwards written
- ☐ Revision twin: [../revision/weeks/week-04/day-27.md](https://github.com/robertblackburn/revision/weeks/week-04/day-27.md)

Tomorrow: Game day taper — trust the reps.

Day 28 — Game Day Taper

Week 4 · Taper / interview readiness (self-contained) · ~2–3 hrs max then rest

Revision twin: [revision/weeks/week-04/day-28.md](https://github.com/robertblackburn/revision/weeks/week-04/day-28.md)

Outcomes

- Weak-area flashcards **only** (Mock #4's ≤3 fix-forwards + chronic gotchas)
- Priority story bank **once each** with run-through scripts — then stop
- Calm **pre-interview checklist** complete
- **ZERO new topics** — no new LC patterns, WWDC, SD docs, or machine rewrites

North star

Protect sleep and confidence. Fresh recall beats last-minute cram.

How to use

1. [01-foundations.md](https://github.com/robertblackburn/01-foundations.md) — taper rules
2. [02-deep-dive.md](https://github.com/robertblackburn/02-deep-dive.md) — checklist + opener cards (retrieval only)
3. [03-production-bridge.md](https://github.com/robertblackburn/03-production-bridge.md) — metric trio + honesty
4. [04-questions.md](https://github.com/robertblackburn/04-questions.md) — light two-layer only (max ~5)
5. [05-exercises.md](https://github.com/robertblackburn/05-exercises.md) — timed taper protocol + **story run-through scripts**
6. No `code/`

Forbidden

Entire Anki binge · rewriting STARs from scratch · “just one more Medium” · new SD HLD · caffeine experiments · all-nighter

01 — Foundations: Taper Rules

Allowed vs forbidden

Allowed	Forbidden
Weak flashcards (≤30–40 cards)	Entire deck binge
Story bank once each (priority set)	Rewriting STARs from scratch
Skim timing budgets (5 min)	New SD HLD from scratch
Checklist + environment setup	Day 25 rewrite
Light walk / meal prep	“Just one more Medium”
Early sleep	Late caffeine / all-nighter

If anxiety says “study more,” do **one** 45s definition aloud, then close the laptop.

Weak-card protocol (45–60 min cap)

1. Open gotchas + Mock #4 three fix-forwards.
2. Pull **only** matching cards.
3. Round 1: front → say back → check.
4. Round 2: misses only.
5. Box closed.

Priority stories (once each)

Order	ID	Why
1	S6	30%+ nav
2	S8	99.95% ownership
3	S9	AI judgment without worship
4	S1	Hard technical + POP
5	S3 or S4	SDUI or pinning (target-company pick)
6	S10 or S15	SDK or on-device AI

Optional micro 30–45s: “Tell me about yourself” spine.

→ [02-deep-dive.md](#)

02 — Deep Dive: Openers + Checklist (Retrieval Only)

Agenda openers (read once)

- Coding:** Clarify → brute → optimize → complexity → edges → code.
- Deep dive:** Definition → mechanism → trade-off → production.
- System design:** “5 min scope, then HLD, deep dives, ops — sound good?”
- Behavioral:** “I’ll take ~2 minutes — context, what I owned, outcome.”

Metric trio

*“I’ve owned features on BookMyShow at **30L+ DAU**, helped drive **99.95%+ crash-free** discipline on my paths, and shipped UX like the LE bottom sheet that cut full-screen navigations by **30%+** on targeted flows.”*

Pre-interview checklist

Environment

- ☐ Quiet room / headphones tested
- ☐ Camera, lighting, neutral background
- ☐ Charger; notifications DND
- ☐ Water; notepad + pen if allowed
- ☐ Backup network known

Materials

- ☐ Calendar link / login works

- ☐ IDE ready only if machine round — else closed
- ☐ Resume PDF synced with story metrics
- ☐ Mental budgets: 45s / 2m / 5m / 45m SD

Content (retrieval only)

- ☐ Metric trio once
- ☐ S6, S8, S9 once (minimum)
- ☐ Coding + SD openers once
- ☐ Weak flashcards capped

Body

- ☐ Real meal; light movement
- ☐ Sleep 7–8h; alarm set
- ☐ No new coffee experiment

Trade-offs

Choice	When	Cost
Taper	Day before / morning of	FOMO
Cram new topic	Never Day 28	Confusion + poor sleep
Full mock today	Only if Mock #4 missed — prefer rest	Fatigue

→ [03-production-bridge.md](#)

03 — Production Bridge: Retrieve, Don’t Invent

You already mapped four weeks to interview language. Today only **retrieve**.

≤20s:

“I’m ready to talk ownership, trade-offs, and systems I’ve shipped — I’ll keep answers timed and concrete.”

Bank: [story-bank.md](#)

No new claims. No new metrics.

→ [04-questions.md](#)

04 — Light Two-Layer Prompts (Cap ~5 Aloud)

Max five total today. Skip tricky if tired.

Q1. Metric trio? (30–45s)

Answer points: 30L+ DAU · 99.95% · 30%+ nav · honest scope

Full spoken answer:

“30L+ DAU context at BookMyShow, 99.95%+ crash-free discipline on my paths, and 30%+ fewer full-screen navigations from the LE bottom sheet on targeted flows.”

Provenance: Verified · S8 · S6

Q2. Coding opener? (30s)

Answer points: say-this-first spine

Full spoken answer:

“Clarify constraints, brute force, optimized pattern, complexity, edges, then code.”

Q3. SD opener? (30s)

Answer points: staff agenda + clarify

Full spoken answer:

“Five minutes on scope, then architecture, deep dives, and ops — starting with clarifying questions.”

Q4. AI tools? (45s)

Answer points: S9 accelerator + review · not author of record · product AI separate

Full spoken answer:

“I use AI as an accelerator inside context and review — District lesson. I’m still author of record on architecture and critical tests. Product on-device AI is a different story.”

Provenance: Verified · S9

Q5. What will you not do tonight? (20s)

Answer points: no new topics · sleep

Full spoken answer:

“No new topics. Sleep.”

T1. Anxiety: I didn’t cover X (60–90s to yourself)

Answer points: if on fix list → flashcard; else park · interview: assume + nearest shipped system

Full spoken answer:

“If X was a Mock #4 fix-forward, one flashcard. Otherwise park it. In interview I’ll clarify, state an assumption, and relate to the nearest system I’ve shipped.”

Then close the laptop.

05 — Exercises (Game Day): Taper Protocol + Story Run-Throughs

Adapted exercises: **no new learning**. Scripts below are run-throughs — speak once, stop (≤1 re-take if derailed).

Timed protocol

Block	Minutes
Weak flashcards only	45–60
Story bank priority once each	40–50
Checklist + environment dry run	15

Skim timing budgets table	5
Stop	—

Log: "Day 28 complete — tapered."

Story run-through scripts (speak once)

Use Day 26 full scripts where noted; below are **game-day compressions** if energy is low — still 2–3 min target.

1) S6 — LE Bottom Sheet

Run-through:

*"Full-screen overview friction → I led a lightweight bottom sheet end-to-end → aligned PM/Design/Backend on contracts → shipped reusable component → **30%+** flows fewer full-screen navigations → small surfaces with clear contracts beat big rewrites."*

(Prefer Day 26 full S6 script if fresh.)

2) S8 — IMOC / crash-free

Run-through:

*"**30L+ DAU** and **99.95%+** bar → Crashlytics triage + IMOC stabilize/communicate/prevent → mitigations and clear ownership in peaks → sustained crash-free discipline → incident leadership is blast radius and rollback, not hero debug."*

3) S9 — AI tooling judgment

Run-through:

"District Free Parking + Clean/MVVM migration → context engineering for AI tools → accelerate tests/reviews inside a review bar → reject bad generations → ship with discipline → tools amplify thinking; I still own architecture — not the same as FinTrack product AI."

4) S1 — Ads / HeroWidget

Run-through:

"Revenue-critical Ads → POP + generics pipeline → HeroWidget pause/play lifecycle → stakeholder alignment without inventing fill-rate % → maintainable type-safe path → lifecycle is part of the product contract."

5) S3 or S4 (pick one)

S3 run-through:

"Backend-driven header + search debounce/MVVM → protocol-driven main screen → faster content iteration + safer search UX → SDUI needs schema/fallbacks, not just render JSON."

S4 run-through:

"Ads networking Alamofire → URLSession → HTTPS, SSL pinning, domain whitelist → security ownership on revenue path → ATS ≠ pinning; rotation is design follow-up not a fake runbook claim."

6) S10 or S15 (pick one)

S10 run-through:

"Stories needed across portfolio → standalone SDK with clear API → adopted across apps → reuse beats one-off UI → SDK quality is boundary + versioning."

S15 run-through:

"Local-first finance → Hive + BM25 RAG + Apple Intelligence when present → deterministic rules fail-soft → no cloud sync of financial data → on-device AI is retrieval + fallback, not model worship."

Optional micro — Tell me about yourself (45s)

*"BookMyShow at **30L+ DAU** across ads, reliability, and delivery metrics like **30%+** nav reduction and **99.95%+** crash-free culture → District architecture + AI judgment → Raw SDK/hybrid apps → FinTrack/GymFlow on-device AI → I want senior ownership of systems and trade-offs."*

Full spoken run-throughs (preferred if energy allows)

These mirror Day 26; speak once with timer 2:30.

S6 full

Use Day 26 [../day-26/04-questions.md](#) S6 Full spoken answer.

S8 full

Use Day 26 S8 Full spoken answer.

S9 full

Use Day 26 S9 Full spoken answer.

S1 full

Use Day 26 S1 Full spoken answer.

Rule: Do not rewrite scripts today. Retrieve and speak.

Exit criteria

- ☐ Weak cards capped
- ☐ Priority stories once each
- ☐ Checklist done
- ☐ Laptop closed for cram
- ☐ Revision twin: [../../revision/weeks/week-04/day-28.md](#)

You've finished the 4-week system. Trust: **agenda · trade-off · proof · time.**
